

BLOCKCHAIN + AI PAYMENT / WALLET SYSTEM

Offline / Local Network Full Implementation Handbook

Hyperledger Fabric + TypeScript + Node.js + PostgreSQL/pgvector + Python/ONNX Runtime +
llama.cpp

*200+ page production engineering guide for an Internet-independent permissioned wallet and payment
platform with bounded local AI*

Architecture baseline: Examples use Hyperledger Fabric 2.5.x LTS with a three-node Raft ordering service, local Fabric CAs, TypeScript chaincode, Fabric Gateway, PostgreSQL/pgvector, ONNX Runtime risk inference, and a local llama.cpp LLM/RAG service. Normal runtime has no public Internet dependency.

Financial boundary: The safest implementation baseline is a closed-loop stored-value or internal-credit wallet. If the system stores, processes, or transmits regulated fiat/card/bank value, legal, licensing, AML/KYC, consumer-protection, tax, PCI DSS, and other jurisdiction-specific obligations must be handled separately by qualified professionals.

AI authority boundary: AI can classify, score, detect anomalies, retrieve policy, summarize evidence, and recommend. AI does not own keys or unilateral authority to issue, transfer, capture, refund, settle, freeze, unfreeze, or redeem value.

Prepared: August 2026

Architecture, accounting, blockchain, AI, security, testing, operations, recovery, and implementation labs

How to Use This Handbook

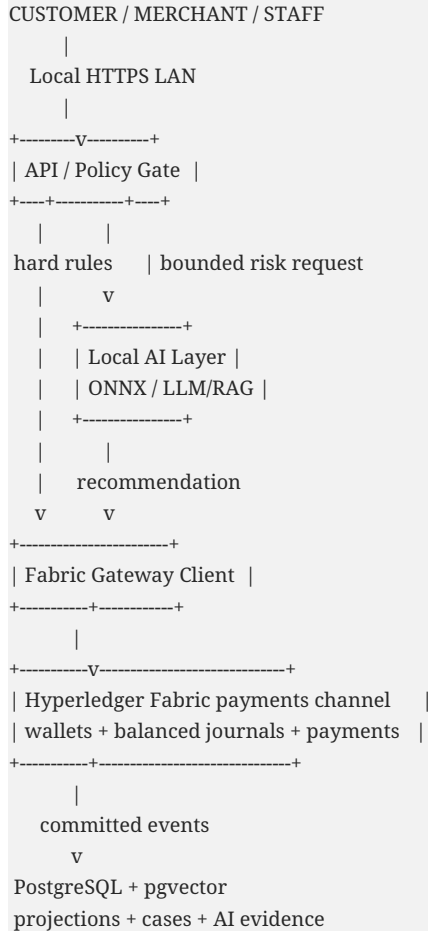
This handbook is designed as a build-and-operate reference. Parts I-II define the financial and accounting model before technology. Parts III-V establish offline infrastructure, Fabric trust, and identity. Parts VI-VIII implement deterministic financial chaincode, local APIs, and wallet/POS applications. Parts IX-XI add bounded local AI for fraud and knowledge assistance. Parts XII-XV cover security, testing, production operations, scaling, and advanced designs. Part XVI provides fifteen reproducible implementation labs. The appendices collect code, schemas, release checklists, and official references.

Core engineering rule: Financial correctness must remain understandable when every AI component is turned off. The double-entry ledger, authorization, hard limits, idempotency, human approval, reconciliation, and recovery rules are deterministic system responsibilities.

Target System Outcome

- Customers can hold and transfer approved closed-loop value on a private LAN without Internet access.
- Merchants can accept local wallet payments, holds/capture, refunds, and settlement with immutable journal evidence.
- Fraud/anomaly models run locally and return versioned, explainable recommendations with deterministic fallback.
- A local LLM/RAG assistant can explain transactions, summarize disputes, answer approved policy questions, and help operators without financial signing authority.
- The platform has documented backup, restore, reconciliation, certificate/key rotation, model rollback, incident response, and offline release procedures.

Executive Reference Architecture



Item	Implementation Guidance
Trust domains	WalletOrg, MerchantOrg, AuditOrg, and OrdererOrg with separately controlled identities.
Ledger	Hyperledger Fabric payments channel; wallet state, balanced journals, payments, holds, refunds, approvals/evidence references.
Application	Node.js/TypeScript local API and policy gate through Fabric Gateway.
Operational data	PostgreSQL projections, users/devices/cases/approvals/AI decisions; pgvector for local RAG.
AI	Python/ONNX Runtime fraud service plus local llama.cpp LLM/embeddings/RAG; advisory and bounded.
Clients	Customer wallet, merchant POS, treasury/admin, fraud review, security operations, audit explorer.
Network	Private VLANs, local DNS/NTP/TLS, internal monitoring, no runtime cloud dependency.

Detailed Contents

Part I - Architecture, Financial Boundary, and Trust Model

1. Why Combine Blockchain and AI in a Local Payment System
2. Closed-Loop Wallet vs Real-Money Payment Boundary
3. Trust Domains and Organization Separation
4. Reference Offline LAN Architecture
5. Source-of-Truth Matrix
6. On-Chain vs Off-Chain Data Placement
7. Non-Functional Requirements
8. AI Authority and Human Oversight Boundary
9. Threat and Failure Domain Overview
10. Architecture Decision Records

Part II - Wallet, Accounting, and Payment Domain Model

11. Double-Entry Accounting as the Financial Core
12. Wallet Types and Ledger Accounts
13. Money Representation and Asset Codes
14. Wallet Lifecycle
15. Issuance of Stored Value
16. Redemption and Cash-Out
17. Peer-to-Peer Transfer
18. Merchant Payment
19. Authorization Holds
20. Capture of a Hold
21. Release and Expiry of a Hold
22. Refunds
23. Reversals and Compensating Entries
24. Fees and Commissions
25. Limits and Velocity Controls
26. Merchant Settlement
27. Reconciliation
28. Disputes and Cases

29. Daily Close and Financial Reporting

Part III - Offline LAN Infrastructure and Platform Foundation

30. Physical Host and VM Topology

31. Network Segmentation and VLAN Design

32. Local DNS

33. Local Time Synchronization

34. Offline Artifact and Dependency Bundle

35. Container Runtime and Compose Baseline

36. Internal TLS and PKI

37. Storage and Filesystem Layout

38. Backup Vault and Media Handling

39. Local Monitoring Stack

40. Local Logging and Correlation

41. Capacity and Resource Budget

42. Offline Update Staging Zone

43. High Availability Baseline

44. Disaster-Recovery Site Option

Part IV - Hyperledger Fabric Network Implementation

45. Fabric Concepts Mapped to Payments

46. Fabric 2.5 LTS Baseline

47. WalletOrg MSP and CA

48. MerchantOrg MSP and CA

49. AuditOrg MSP and CA

50. OrdererOrg MSP and CA

51. Three-Node Raft Ordering Service

52. Payments Channel Design

53. Peer Deployment and CouchDB

54. Anchor Peers and Gossip

55. Channel Policies and Endorsement

56. State-Based Endorsement

57. Private Data Collections

- 58. Chaincode Lifecycle
- 59. Fabric Gateway Client Model
- 60. Fabric HSM and PKCS11 Option
- 61. Certificate Renewal and Revocation
- 62. Fabric Network Health Verification

Part V - Identity, Wallet Authorization, and Key Management

- 63. Separate Application Identity from Fabric Identity
- 64. Local User Directory and Authentication
- 65. Role-Based Access Control
- 66. MFA and TOTP for Privileged Operations
- 67. Device Registration
- 68. Signed Authorization Envelope
- 69. Nonce and Replay Protection
- 70. Customer Wallet Key Models
- 71. Merchant/POS Key Model
- 72. Fabric Client Signing Keys
- 73. HSM/PKCS11 Operations
- 74. Key Rotation
- 75. Certificate Revocation
- 76. Break-Glass Administration
- 77. Treasury Dual Control
- 78. Audit Attribution
- 79. Identity Recovery and Account Takeover Handling

Part VI - TypeScript Chaincode and Deterministic Financial Rules

- 80. Chaincode Project Structure
- 81. Wallet State Model
- 82. Journal Model
- 83. Accounting Helper Functions
- 84. Authorization Helpers
- 85. CreateWallet Transaction
- 86. Activate Suspend Freeze and Unfreeze Wallet

- 87. IssueValue Transaction
- 88. RedeemValue Transaction
- 89. Transfer Transaction
- 90. MerchantPayment Transaction
- 91. CreateHold Transaction
- 92. CaptureHold Transaction
- 93. ReleaseHold Transaction
- 94. Refund Transaction
- 95. Reversal and Adjustment Transactions
- 96. Fee Calculation
- 97. Hard Limit Enforcement
- 98. Daily and Rolling Counters
- 99. Ledger-Level Idempotency
- 100. Signed Authorization Digest Verification Reference
- 101. Composite Keys and Indexes
- 102. Chaincode Events
- 103. MVCC and Concurrent Spend
- 104. Stable Error Model
- 105. Chaincode Upgrade and Schema Migration

Part VII - Node.js/TypeScript Backend and Integration APIs

- 106. Fabric Gateway Connection Helper
- 107. Command vs Query Architecture
- 108. REST API Standards
- 109. Authentication Middleware
- 110. Authorization Service
- 111. Transaction Authorization Service
- 112. AI Risk Client
- 113. Deterministic Policy Gate
- 114. Human Approval Service
- 115. Wallet REST API
- 116. Transfer REST API

- 117. Merchant Payment REST API
- 118. Hold/Capture/Release REST API
- 119. Refund REST API
- 120. Issue and Redeem REST API
- 121. Transaction Status and Recovery API
- 122. PostgreSQL Projection Database
- 123. Fabric Event Listener
- 124. Outbox and Local Job Queue
- 125. Merchant and Wallet Notifications
- 126. Error Mapping
- 127. Audit API
- 128. Health and Readiness API
- 129. Configuration and Secret Loading

Part VIII - Wallet, Merchant POS, and Operator User Interfaces

- 130. LAN-Only Frontend Architecture
- 131. Customer Wallet Dashboard
- 132. P2P Transfer UX
- 133. QR Payment Format
- 134. Merchant POS Payment Flow
- 135. Hold and Capture UI
- 136. Refund and Dispute UI
- 137. Treasury Console
- 138. Merchant Settlement Dashboard
- 139. Fraud Review Queue
- 140. Security Operations Dashboard
- 141. Audit Explorer
- 142. Transaction Receipt
- 143. Pending/Unknown Transaction Recovery UX
- 144. Accessibility and Operator Error Prevention
- 145. Degraded Mode UX

Part IX - AI Architecture, Governance, and Decision Contracts

- 146. Why AI Must Stay Outside Chaincode
- 147. Offline AI Topology
- 148. AI Decision Contract
- 149. Feature Store Boundary
- 150. Model Registry and SHA-256 Anchoring
- 151. Prompt and RAG Version Registry
- 152. Shadow Mode
- 153. Advisory Mode
- 154. Controlled Enforcement
- 155. Human Review Queue
- 156. AI Fallback Matrix
- 157. AI Data Retention
- 158. AI Change Management
- 159. AI Governance Using NIST AI RMF Concepts
- 160. AI Evidence Linked to Ledger

Part X - Fraud Detection, Anomaly Models, and Risk Engineering

- 161. Deterministic Fraud Rules Baseline
- 162. Transaction Feature Engineering
- 163. Velocity Features
- 164. Device and Session Risk Features
- 165. Recipient and Graph Features
- 166. Merchant Risk Features
- 167. Supervised Classification
- 168. Anomaly Detection
- 169. Baseline Logistic/Tree Models
- 170. Threshold Calibration
- 171. Precision Recall and PR Curves
- 172. Calibration and Expected Loss
- 173. Reason Codes and Explainability
- 174. False Positive Management
- 175. Model Drift Monitoring

- 176. Offline Training Pipeline
- 177. ONNX Model Export
- 178. ONNX Runtime Risk Service
- 179. Model Service High Availability
- 180. Rules Plus Model Ensemble
- 181. Manual Review Feedback Loop
- 182. Fraud Decision Audit Trail
- 183. Adversarial Fraud Testing

Part XI - Local LLM, RAG, Support, and Operations AI

- 184. Local llama.cpp Server
- 185. OpenAI-Compatible Local API Pattern
- 186. Schema-Constrained LLM Outputs
- 187. Local Embeddings Service
- 188. PostgreSQL plus pgvector
- 189. Knowledge Corpus Governance
- 190. Document Parsing and Chunking
- 191. Policy RAG Assistant
- 192. Dispute Summarizer
- 193. Transaction Explanation Assistant
- 194. Operations Copilot
- 195. AI Support Classification
- 196. LLM Tool Policy
- 197. Prompt Injection Defense
- 198. RAG Access Control
- 199. RAG Evaluation
- 200. LLM Safety Evaluation
- 201. Prompt and Model Rollback
- 202. LLM Data Retention and Privacy
- 203. Air-Gapped Model Import

Part XII - Security, Privacy, and Financial Abuse Resistance

- 204. Security Threat Model

- 205. Network Least Privilege
- 206. Host Hardening
- 207. Secret Management
- 208. Fabric Key Management
- 209. NIST Key Management Alignment
- 210. Application Security Verification
- 211. OWASP LLM Security Verification
- 212. NIST AI Risk Management Profile
- 213. PII Minimization
- 214. Payment Card Data Boundary
- 215. Replay and Request Forgery Protection
- 216. Transaction Tampering Controls
- 217. Database Tamper Detection
- 218. Model Tamper Detection
- 219. Training Data Poisoning Controls
- 220. Prompt Injection and Data Exfiltration
- 221. Insider Fraud and Separation of Duties
- 222. Physical Security
- 223. Audit Logging
- 224. Incident Kill Switches
- 225. Security Incident Response
- 226. Offline Software Update Security
- 227. Backup Security

Part XIII - Testing, Evaluation, and Quality Engineering

- 228. Accounting Invariant Unit Tests
- 229. Chaincode Unit Tests
- 230. Fabric Integration Tests
- 231. Concurrent Double-Spend Tests
- 232. Idempotency Tests
- 233. Authorization Tests
- 234. Raft Quorum and Orderer Fault Tests

- 235. Peer and CouchDB Fault Tests
- 236. PostgreSQL Projection Failure Tests
- 237. AI Service Failure Tests
- 238. Fraud Model Offline Evaluation
- 239. Shadow Mode Comparison
- 240. LLM and RAG Evaluation
- 241. Adversarial AI Tests
- 242. REST API Contract Tests
- 243. End-to-End Payment Tests
- 244. Reconciliation Tests
- 245. Backup and Restore Drill
- 246. Certificate Rotation Drill
- 247. Model Rollback Drill
- 248. Performance and Soak Tests
- 249. User Acceptance Tests
- 250. Production Rehearsal

Part XIV - Production Operations, Backup, Recovery, and Change Management

- 251. Startup Procedure
- 252. Shutdown Procedure
- 253. Daily Health Checks
- 254. Financial Reconciliation Monitoring
- 255. Backup Architecture
- 256. Fabric Recovery Procedure
- 257. PostgreSQL Restore and Projection Rebuild
- 258. Certificate Rotation Operations
- 259. Chaincode Upgrade Operations
- 260. API and UI Release Operations
- 261. Fraud Model Promotion
- 262. LLM/Prompt/RAG Promotion
- 263. Incident Operations
- 264. Offline Patch Management

265. Capacity Monitoring

266. Operator Access Review

267. Key and Secret Inventory Review

268. Daily Close Runbook

269. Disaster Recovery Exercise

270. Decommissioning

Part XV - Performance, High Availability, Multi-Site, and Advanced Design

271. Performance Measurement Methodology

272. Fabric Throughput Tuning

273. API Scaling

274. PostgreSQL High Availability

275. AI Inference Capacity

276. Local LLM Capacity Planning

277. Multi-Branch LAN/WAN Design

278. Disconnected Branch Voucher Mode

279. Multi-Asset Wallet

280. Advanced Settlement Networks

281. Fabric 3.x and SmartBFT Upgrade Path

282. Hardware Security Module Expansion

283. Ledger Evidence Anchoring for AI Releases

284. Local Analytics Warehouse

285. Graph Analytics for Fraud

286. Privacy-Preserving Analytics Options

287. Business Continuity Without AI

288. Migration from Legacy Payment Database

289. External Integration Gateway Option

Part XVI - Reproducible Full Implementation Labs

290. Lab 1 - Build the Offline Artifact Bundle

291. Lab 2 - Build the Four-Domain Fabric Network

292. Lab 3 - Deploy Wallet Chaincode Skeleton

293. Lab 4 - Implement Double-Entry Transfer

294. Lab 5 - Implement Holds Capture Release and Refund

295. Lab 6 - Build the Fabric Gateway REST API

296. Lab 7 - Build PostgreSQL Projection and Reconciliation

297. Lab 8 - Build Customer Wallet and Merchant POS

298. Lab 9 - Build Fraud Rules and Feature Pipeline

299. Lab 10 - Train and Serve an ONNX Risk Model

300. Lab 11 - Build Local LLM and RAG

301. Lab 12 - Integrate AI Risk with Deterministic Payment Gate

302. Lab 13 - Build Human Review and Dual Approval

303. Lab 14 - Run Security and Failure Drills

304. Lab 15 - Backup Restore Reconcile and Go-Live Rehearsal

Appendix A - Recommended Repository Structure

Appendix B - Core TypeScript Wallet Chaincode

Appendix C - Node.js Fabric Gateway Backend

Appendix D - Python ONNX Risk Service

Appendix E - llama.cpp and Local RAG

Appendix F - Docker Compose and LAN Configuration

Appendix G - REST API Specification

Appendix H - PostgreSQL and pgvector Schemas

Appendix I - Fraud Feature and Evaluation Template

Appendix J - Security and Production Readiness Checklist

Appendix K - Comprehensive Test and Failure Matrix

Appendix L - Glossary and Official References

Master Financial Invariants

Item	Implementation Guidance
Balanced journal	For every asset and journal: $\text{sum}(\text{debitMinor}) == \text{sum}(\text{creditMinor})$.
Conservation	Ordinary transfer/payment/hold/refund does not create value. Only explicitly authorized issuance/redemption changes closed-loop supply boundary.
Integer money	All financial amounts are integer minor units with an explicit asset code and documented scale.
Immutability	Committed financial history is not edited. Corrections are new compensating journals.
Idempotency	One business requestId can create at most one logical financial result.
Authorization	An authenticated identity may debit only wallets/actions it is authorized to control.
AI boundary	AI recommendation cannot bypass hard rules, ownership, approval, or signing controls.
Reconciliation	Ledger journals, wallet balances, settlement reports, and projections must reconcile or create a visible exception.

Example: customer pays merchant 25.00 LOCAL

Journal JRN-1001, asset=LOCAL

Customer wallet CUST-1 DEBIT 2500 minor units

Merchant wallet MERCH-7 CREDIT 2500 minor units

Invariant:

$\text{sum}(\text{debits}) == \text{sum}(\text{credits}) == 2500$

No value is created or destroyed by Transfer/Payment.

Only authorized Issuance/Redemption transactions change the closed-loop supply boundary.

Part I - Architecture, Financial Boundary, and Trust Model

A payment system is a financial state machine, not merely a set of CRUD screens. The blockchain layer should exist only where immutable multi-party evidence, endorsement, and deterministic settlement add value. AI is a separate advisory capability and must never become the accounting source of truth.

Part focus: This part contains 10 implementation chapters. Each chapter includes design rules, implementation procedure, a practical artifact, verification, failure analysis, and expert review questions.

Chapter 1 - Why Combine Blockchain and AI in a Local Payment System

Define the distinct business problems blockchain and AI solve without confusing immutability with intelligence. A payment system is a financial state machine, not merely a set of CRUD screens. The blockchain layer should exist only where immutable multi-party evidence, endorsement, and deterministic settlement add value. AI is a separate advisory capability and must never become the accounting source of truth.

Design rule: Blockchain supplies shared immutable financial evidence

Implementation Objective

The required output is: Technology decision memo. It must be version-controlled, reviewable, testable with the WAN disabled, and linked to the architecture/source-of-truth model. For a payment platform, every implementation decision should make it possible to explain who authorized an action, which facts were trusted, which controls ran, what financial journal committed, and how the result is recovered after failure.

Key Engineering Decisions

- Blockchain supplies shared immutable financial evidence
- AI supplies bounded risk and language assistance
- Neither replaces identity, accounting controls, or human governance

Deep Implementation Guidance

Write the decision as an architecture decision record with context, alternatives, selected option, security impact, operational impact, and reversal criteria. A private blockchain is valuable only if its independent identities and endorsement policies correspond to real governance boundaries.

Data and Control Questions

- What is the authoritative source for every input used by this component?
- Which user, device, service, MSP, or administrator identity is allowed to initiate or approve it?
- Does the operation move value, reserve value, create a side effect, or only provide advisory information?
- What idempotency/replay key proves that a retry is the same business intent?
- What evidence is retained off-chain and what stable reference/hash belongs on the ledger?

Chapter 1 Implementation - Why Combine Blockchain and AI in a Local Payment System

Implementation Procedure

1. Write the business outcome and financial boundary before selecting technology.
2. Identify actors, trust domains, systems of record, failure domains, and regulated-data boundaries.
3. Create architecture decision records for blockchain, AI, database, and identity choices.
4. Define measurable latency, availability, RPO/RTO, audit, and offline-operation requirements.
5. Review whether a simpler signed database would meet the same trust model.

Implementation Example

```
# Why Combine Blockchain and AI in a Local Payment System
INPUT  -> authenticate, validate schema, bind requestId/nonce
FACTS  -> read authoritative wallet/merchant/state data
AI     -> optional bounded recommendation with decisionId/version
POLICY -> enforce ownership, hard limits, approvals, risk fallback
LEDGER -> submit one idempotent deterministic transaction when value changes
EVENT  -> update projection/case/monitoring asynchronously
VERIFY -> discover committed result and retain correlationId + txId evidence
```

Verification Checklist

- The technology decision memo exists and is version-controlled.
- The feature works with all public Internet/WAN connectivity disabled.
- Authorization, malformed input, duplicate retry, and failure behavior have explicit tests.
- If financial value changes, journal debits and credits balance exactly and historical entries remain immutable.
- If AI is involved, model/prompt/feature version and fallback behavior are recorded and deterministic policy remains final.
- A timeout/restart cannot create a duplicate financial result; request status can be rediscovered.
- Logs/evidence contain correlation references but not secrets or prohibited sensitive payment data.

Common Failure Modes

- Blockchain chosen without a multi-party integrity requirement.
- AI given authority that deterministic policy should own.
- Regulatory/payment boundary left undefined.
- No measurable offline availability or recovery targets.

Expert Review Questions

- What happens if this component succeeds but the next component fails?
- How is the authorizing identity/device/key revoked or recovered?
- Can the operation be safely retried after the client loses the response?
- What happens if AI is unavailable, slow, or wrong?
- Can an auditor reconstruct debit, credit, actor, risk decision, approval, and Fabric txId from retained evidence?
- Which compensating transaction resolves a downstream operational error without rewriting history?

Chapter 2 - Closed-Loop Wallet vs Real-Money Payment Boundary

Choose a legally and operationally clear value model before implementing wallet code. A payment system is a financial state machine, not merely a set of CRUD screens. The blockchain layer should exist only where immutable multi-party evidence, endorsement, and deterministic settlement add value. AI is a separate advisory capability and must never become the accounting source of truth.

Design rule: Closed-loop stored value/internal credit is the safest engineering baseline

Implementation Objective

The required output is: Financial boundary statement. It must be version-controlled, reviewable, testable with the WAN disabled, and linked to the architecture/source-of-truth model. For a payment platform, every implementation decision should make it possible to explain who authorized an action, which facts were trusted, which controls ran, what financial journal committed, and how the result is recovered after failure.

Key Engineering Decisions

- Closed-loop stored value/internal credit is the safest engineering baseline
- Fiat/card/bank integration adds jurisdiction-specific obligations
- Cardholder data should remain outside the ledger and minimized

Deep Implementation Guidance

Write the decision as an architecture decision record with context, alternatives, selected option, security impact, operational impact, and reversal criteria. A private blockchain is valuable only if its independent identities and endorsement policies correspond to real governance boundaries.

Data and Control Questions

- What is the authoritative source for every input used by this component?
- Which user, device, service, MSP, or administrator identity is allowed to initiate or approve it?
- Does the operation move value, reserve value, create a side effect, or only provide advisory information?
- What idempotency/replay key proves that a retry is the same business intent?
- What evidence is retained off-chain and what stable reference/hash belongs on the ledger?

Chapter 2 Implementation - Closed-Loop Wallet vs Real-Money Payment Boundary

Implementation Procedure

6. Write the business outcome and financial boundary before selecting technology.
7. Identify actors, trust domains, systems of record, failure domains, and regulated-data boundaries.
8. Create architecture decision records for blockchain, AI, database, and identity choices.
9. Define measurable latency, availability, RPO/RTO, audit, and offline-operation requirements.
10. Review whether a simpler signed database would meet the same trust model.

Implementation Example

```
# Closed-Loop Wallet vs Real-Money Payment Boundary
INPUT  -> authenticate, validate schema, bind requestId/nonce
FACTS  -> read authoritative wallet/merchant/state data
AI     -> optional bounded recommendation with decisionId/version
POLICY -> enforce ownership, hard limits, approvals, risk fallback
LEDGER -> submit one idempotent deterministic transaction when value changes
EVENT  -> update projection/case/monitoring asynchronously
VERIFY -> discover committed result and retain correlationId + txId evidence
```

Verification Checklist

- The financial boundary statement exists and is version-controlled.
- The feature works with all public Internet/WAN connectivity disabled.
- Authorization, malformed input, duplicate retry, and failure behavior have explicit tests.
- If financial value changes, journal debits and credits balance exactly and historical entries remain immutable.
- If AI is involved, model/prompt/feature version and fallback behavior are recorded and deterministic policy remains final.
- A timeout/restart cannot create a duplicate financial result; request status can be rediscovered.
- Logs/evidence contain correlation references but not secrets or prohibited sensitive payment data.

Common Failure Modes

- Blockchain chosen without a multi-party integrity requirement.
- AI given authority that deterministic policy should own.
- Regulatory/payment boundary left undefined.
- No measurable offline availability or recovery targets.

Expert Review Questions

- What happens if this component succeeds but the next component fails?
- How is the authorizing identity/device/key revoked or recovered?
- Can the operation be safely retried after the client loses the response?
- What happens if AI is unavailable, slow, or wrong?
- Can an auditor reconstruct debit, credit, actor, risk decision, approval, and Fabric txId from retained evidence?
- Which compensating transaction resolves a downstream operational error without rewriting history?

Chapter 3 - Trust Domains and Organization Separation

Separate administrative control so the ledger has meaningful independent verification. A payment system is a financial state machine, not merely a set of CRUD screens. The blockchain layer should exist only where immutable multi-party evidence, endorsement, and deterministic settlement add value. AI is a separate advisory capability and must never become the accounting source of truth.

Design rule: WalletOrg operates customer wallets and treasury

Implementation Objective

The required output is: Trust-domain diagram. It must be version-controlled, reviewable, testable with the WAN disabled, and linked to the architecture/source-of-truth model. For a payment platform, every implementation decision should make it possible to explain who authorized an action, which facts were trusted, which controls ran, what financial journal committed, and how the result is recovered after failure.

Key Engineering Decisions

- WalletOrg operates customer wallets and treasury
- MerchantOrg operates merchant acceptance and settlement
- AuditOrg independently verifies ledger history
- OrdererOrg controls ordering service administration

Deep Implementation Guidance

Write the decision as an architecture decision record with context, alternatives, selected option, security impact, operational impact, and reversal criteria. A private blockchain is valuable only if its independent identities and endorsement policies correspond to real governance boundaries.

Data and Control Questions

- What is the authoritative source for every input used by this component?
- Which user, device, service, MSP, or administrator identity is allowed to initiate or approve it?
- Does the operation move value, reserve value, create a side effect, or only provide advisory information?
- What idempotency/replay key proves that a retry is the same business intent?
- What evidence is retained off-chain and what stable reference/hash belongs on the ledger?

Chapter 3 Implementation - Trust Domains and Organization Separation

Implementation Procedure

11. Write the business outcome and financial boundary before selecting technology.
12. Identify actors, trust domains, systems of record, failure domains, and regulated-data boundaries.
13. Create architecture decision records for blockchain, AI, database, and identity choices.
14. Define measurable latency, availability, RPO/RTO, audit, and offline-operation requirements.
15. Review whether a simpler signed database would meet the same trust model.

Implementation Example

```
# Trust Domains and Organization Separation
INPUT  -> authenticate, validate schema, bind requestId/nonce
FACTS  -> read authoritative wallet/merchant/state data
AI     -> optional bounded recommendation with decisionId/version
POLICY -> enforce ownership, hard limits, approvals, risk fallback
LEDGER -> submit one idempotent deterministic transaction when value changes
EVENT  -> update projection/case/monitoring asynchronously
VERIFY -> discover committed result and retain correlationId + txId evidence
```

Verification Checklist

- The trust-domain diagram exists and is version-controlled.
- The feature works with all public Internet/WAN connectivity disabled.
- Authorization, malformed input, duplicate retry, and failure behavior have explicit tests.
- If financial value changes, journal debits and credits balance exactly and historical entries remain immutable.
- If AI is involved, model/prompt/feature version and fallback behavior are recorded and deterministic policy remains final.
- A timeout/restart cannot create a duplicate financial result; request status can be rediscovered.
- Logs/evidence contain correlation references but not secrets or prohibited sensitive payment data.

Common Failure Modes

- Blockchain chosen without a multi-party integrity requirement.
- AI given authority that deterministic policy should own.
- Regulatory/payment boundary left undefined.
- No measurable offline availability or recovery targets.

Expert Review Questions

- What happens if this component succeeds but the next component fails?
- How is the authorizing identity/device/key revoked or recovered?
- Can the operation be safely retried after the client loses the response?
- What happens if AI is unavailable, slow, or wrong?
- Can an auditor reconstruct debit, credit, actor, risk decision, approval, and Fabric txId from retained evidence?
- Which compensating transaction resolves a downstream operational error without rewriting history?

Chapter 4 - Reference Offline LAN Architecture

Define the complete local architecture and data paths before writing services. A payment system is a financial state machine, not merely a set of CRUD screens. The blockchain layer should exist only where immutable multi-party evidence, endorsement, and deterministic settlement add value. AI is a separate advisory capability and must never become the accounting source of truth.

Design rule: Runtime traffic stays on private LAN

Implementation Objective

The required output is: Level-1 architecture diagram. It must be version-controlled, reviewable, testable with the WAN disabled, and linked to the architecture/source-of-truth model. For a payment platform, every implementation decision should make it possible to explain who authorized an action, which facts were trusted, which controls ran, what financial journal committed, and how the result is recovered after failure.

Key Engineering Decisions

- Runtime traffic stays on private LAN
- Fabric is the financial evidence layer
- PostgreSQL/pgvector are operational read and AI stores
- Local AI services never require cloud access

Deep Implementation Guidance

Write the decision as an architecture decision record with context, alternatives, selected option, security impact, operational impact, and reversal criteria. A private blockchain is valuable only if its independent identities and endorsement policies correspond to real governance boundaries.

Data and Control Questions

- What is the authoritative source for every input used by this component?
- Which user, device, service, MSP, or administrator identity is allowed to initiate or approve it?
- Does the operation move value, reserve value, create a side effect, or only provide advisory information?
- What idempotency/replay key proves that a retry is the same business intent?
- What evidence is retained off-chain and what stable reference/hash belongs on the ledger?

Chapter 4 Implementation - Reference Offline LAN Architecture

Implementation Procedure

16. Write the business outcome and financial boundary before selecting technology.
17. Identify actors, trust domains, systems of record, failure domains, and regulated-data boundaries.
18. Create architecture decision records for blockchain, AI, database, and identity choices.
19. Define measurable latency, availability, RPO/RTO, audit, and offline-operation requirements.
20. Review whether a simpler signed database would meet the same trust model.

Implementation Example

```
# Reference Offline LAN Architecture
INPUT  -> authenticate, validate schema, bind requestId/nonce
FACTS  -> read authoritative wallet/merchant/state data
AI     -> optional bounded recommendation with decisionId/version
POLICY -> enforce ownership, hard limits, approvals, risk fallback
LEDGER -> submit one idempotent deterministic transaction when value changes
EVENT  -> update projection/case/monitoring asynchronously
VERIFY -> discover committed result and retain correlationId + txId evidence
```

Verification Checklist

- The level-1 architecture diagram exists and is version-controlled.
- The feature works with all public Internet/WAN connectivity disabled.
- Authorization, malformed input, duplicate retry, and failure behavior have explicit tests.
- If financial value changes, journal debits and credits balance exactly and historical entries remain immutable.
- If AI is involved, model/prompt/feature version and fallback behavior are recorded and deterministic policy remains final.
- A timeout/restart cannot create a duplicate financial result; request status can be rediscovered.
- Logs/evidence contain correlation references but not secrets or prohibited sensitive payment data.

Common Failure Modes

- Blockchain chosen without a multi-party integrity requirement.
- AI given authority that deterministic policy should own.
- Regulatory/payment boundary left undefined.
- No measurable offline availability or recovery targets.

Expert Review Questions

- What happens if this component succeeds but the next component fails?
- How is the authorizing identity/device/key revoked or recovered?
- Can the operation be safely retried after the client loses the response?
- What happens if AI is unavailable, slow, or wrong?
- Can an auditor reconstruct debit, credit, actor, risk decision, approval, and Fabric txId from retained evidence?
- Which compensating transaction resolves a downstream operational error without rewriting history?

Chapter 5 - Source-of-Truth Matrix

Assign every critical fact to exactly one authoritative system. A payment system is a financial state machine, not merely a set of CRUD screens. The blockchain layer should exist only where immutable multi-party evidence, endorsement, and deterministic settlement add value. AI is a separate advisory capability and must never become the accounting source of truth.

Design rule: Ledger owns wallet/journal/payment state

Implementation Objective

The required output is: Source-of-truth matrix. It must be version-controlled, reviewable, testable with the WAN disabled, and linked to the architecture/source-of-truth model. For a payment platform, every implementation decision should make it possible to explain who authorized an action, which facts were trusted, which controls ran, what financial journal committed, and how the result is recovered after failure.

Key Engineering Decisions

- Ledger owns wallet/journal/payment state
- PostgreSQL owns projections/search/session/PII
- AI owns recommendations only
- Operators own approvals according to policy

Deep Implementation Guidance

Write the decision as an architecture decision record with context, alternatives, selected option, security impact, operational impact, and reversal criteria. A private blockchain is valuable only if its independent identities and endorsement policies correspond to real governance boundaries.

Data and Control Questions

- What is the authoritative source for every input used by this component?
- Which user, device, service, MSP, or administrator identity is allowed to initiate or approve it?
- Does the operation move value, reserve value, create a side effect, or only provide advisory information?
- What idempotency/replay key proves that a retry is the same business intent?
- What evidence is retained off-chain and what stable reference/hash belongs on the ledger?

Chapter 5 Implementation - Source-of-Truth Matrix

Implementation Procedure

21. Write the business outcome and financial boundary before selecting technology.
22. Identify actors, trust domains, systems of record, failure domains, and regulated-data boundaries.
23. Create architecture decision records for blockchain, AI, database, and identity choices.
24. Define measurable latency, availability, RPO/RTO, audit, and offline-operation requirements.
25. Review whether a simpler signed database would meet the same trust model.

Implementation Example

```
# Source-of-Truth Matrix
INPUT  -> authenticate, validate schema, bind requestId/nonce
FACTS  -> read authoritative wallet/merchant/state data
AI     -> optional bounded recommendation with decisionId/version
POLICY -> enforce ownership, hard limits, approvals, risk fallback
LEDGER -> submit one idempotent deterministic transaction when value changes
EVENT  -> update projection/case/monitoring asynchronously
VERIFY -> discover committed result and retain correlationId + txId evidence
```

Verification Checklist

- The source-of-truth matrix exists and is version-controlled.
- The feature works with all public Internet/WAN connectivity disabled.
- Authorization, malformed input, duplicate retry, and failure behavior have explicit tests.
- If financial value changes, journal debits and credits balance exactly and historical entries remain immutable.
- If AI is involved, model/prompt/feature version and fallback behavior are recorded and deterministic policy remains final.
- A timeout/restart cannot create a duplicate financial result; request status can be rediscovered.
- Logs/evidence contain correlation references but not secrets or prohibited sensitive payment data.

Common Failure Modes

- Blockchain chosen without a multi-party integrity requirement.
- AI given authority that deterministic policy should own.
- Regulatory/payment boundary left undefined.
- No measurable offline availability or recovery targets.

Expert Review Questions

- What happens if this component succeeds but the next component fails?
- How is the authorizing identity/device/key revoked or recovered?
- Can the operation be safely retried after the client loses the response?
- What happens if AI is unavailable, slow, or wrong?
- Can an auditor reconstruct debit, credit, actor, risk decision, approval, and Fabric txId from retained evidence?
- Which compensating transaction resolves a downstream operational error without rewriting history?

Chapter 6 - On-Chain vs Off-Chain Data Placement

Keep immutable financial evidence on-chain while retaining privacy and operational flexibility off-chain. A payment system is a financial state machine, not merely a set of CRUD screens. The blockchain layer should exist only where immutable multi-party evidence, endorsement, and deterministic settlement add value. AI is a separate advisory capability and must never become the accounting source of truth.

Design rule: Journal entries and transaction states belong on-chain

Implementation Objective

The required output is: Data placement matrix. It must be version-controlled, reviewable, testable with the WAN disabled, and linked to the architecture/source-of-truth model. For a payment platform, every implementation decision should make it possible to explain who authorized an action, which facts were trusted, which controls ran, what financial journal committed, and how the result is recovered after failure.

Key Engineering Decisions

- Journal entries and transaction states belong on-chain
- PII/secrets/model artifacts/log detail stay off-chain
- Hashes and stable references can anchor external evidence

Deep Implementation Guidance

Write the decision as an architecture decision record with context, alternatives, selected option, security impact, operational impact, and reversal criteria. A private blockchain is valuable only if its independent identities and endorsement policies correspond to real governance boundaries.

Data and Control Questions

- What is the authoritative source for every input used by this component?
- Which user, device, service, MSP, or administrator identity is allowed to initiate or approve it?
- Does the operation move value, reserve value, create a side effect, or only provide advisory information?
- What idempotency/replay key proves that a retry is the same business intent?
- What evidence is retained off-chain and what stable reference/hash belongs on the ledger?

Chapter 6 Implementation - On-Chain vs Off-Chain Data Placement

Implementation Procedure

26. Write the business outcome and financial boundary before selecting technology.
27. Identify actors, trust domains, systems of record, failure domains, and regulated-data boundaries.
28. Create architecture decision records for blockchain, AI, database, and identity choices.
29. Define measurable latency, availability, RPO/RTO, audit, and offline-operation requirements.
30. Review whether a simpler signed database would meet the same trust model.

Implementation Example

```
# On-Chain vs Off-Chain Data Placement
INPUT  -> authenticate, validate schema, bind requestId/nonce
FACTS  -> read authoritative wallet/merchant/state data
AI     -> optional bounded recommendation with decisionId/version
POLICY -> enforce ownership, hard limits, approvals, risk fallback
LEDGER -> submit one idempotent deterministic transaction when value changes
EVENT  -> update projection/case/monitoring asynchronously
VERIFY -> discover committed result and retain correlationId + txId evidence
```

Verification Checklist

- The data placement matrix exists and is version-controlled.
- The feature works with all public Internet/WAN connectivity disabled.
- Authorization, malformed input, duplicate retry, and failure behavior have explicit tests.
- If financial value changes, journal debits and credits balance exactly and historical entries remain immutable.
- If AI is involved, model/prompt/feature version and fallback behavior are recorded and deterministic policy remains final.
- A timeout/restart cannot create a duplicate financial result; request status can be rediscovered.
- Logs/evidence contain correlation references but not secrets or prohibited sensitive payment data.

Common Failure Modes

- Blockchain chosen without a multi-party integrity requirement.
- AI given authority that deterministic policy should own.
- Regulatory/payment boundary left undefined.
- No measurable offline availability or recovery targets.

Expert Review Questions

- What happens if this component succeeds but the next component fails?
- How is the authorizing identity/device/key revoked or recovered?
- Can the operation be safely retried after the client loses the response?
- What happens if AI is unavailable, slow, or wrong?
- Can an auditor reconstruct debit, credit, actor, risk decision, approval, and Fabric txId from retained evidence?
- Which compensating transaction resolves a downstream operational error without rewriting history?

Chapter 7 - Non-Functional Requirements

Define measurable service goals for an Internet-independent financial platform. A payment system is a financial state machine, not merely a set of CRUD screens. The blockchain layer should exist only where immutable multi-party evidence, endorsement, and deterministic settlement add value. AI is a separate advisory capability and must never become the accounting source of truth.

Design rule: Specify p95 authorization latency and transaction commit target

Implementation Objective

The required output is: NFR specification. It must be version-controlled, reviewable, testable with the WAN disabled, and linked to the architecture/source-of-truth model. For a payment platform, every implementation decision should make it possible to explain who authorized an action, which facts were trusted, which controls ran, what financial journal committed, and how the result is recovered after failure.

Key Engineering Decisions

- Specify p95 authorization latency and transaction commit target
- Define RPO/RTO and maximum degraded-mode duration
- State that normal payment/wallet operation must survive WAN loss

Deep Implementation Guidance

Write the decision as an architecture decision record with context, alternatives, selected option, security impact, operational impact, and reversal criteria. A private blockchain is valuable only if its independent identities and endorsement policies correspond to real governance boundaries.

Data and Control Questions

- What is the authoritative source for every input used by this component?
- Which user, device, service, MSP, or administrator identity is allowed to initiate or approve it?
- Does the operation move value, reserve value, create a side effect, or only provide advisory information?
- What idempotency/replay key proves that a retry is the same business intent?
- What evidence is retained off-chain and what stable reference/hash belongs on the ledger?

Chapter 7 Implementation - Non-Functional Requirements

Implementation Procedure

31. Write the business outcome and financial boundary before selecting technology.
32. Identify actors, trust domains, systems of record, failure domains, and regulated-data boundaries.
33. Create architecture decision records for blockchain, AI, database, and identity choices.
34. Define measurable latency, availability, RPO/RTO, audit, and offline-operation requirements.
35. Review whether a simpler signed database would meet the same trust model.

Implementation Example

```
# Non-Functional Requirements
INPUT  -> authenticate, validate schema, bind requestId/nonce
FACTS  -> read authoritative wallet/merchant/state data
AI      -> optional bounded recommendation with decisionId/version
POLICY  -> enforce ownership, hard limits, approvals, risk fallback
LEDGER  -> submit one idempotent deterministic transaction when value changes
EVENT   -> update projection/case/monitoring asynchronously
VERIFY  -> discover committed result and retain correlationId + txId evidence
```

Verification Checklist

- The nfr specification exists and is version-controlled.
- The feature works with all public Internet/WAN connectivity disabled.
- Authorization, malformed input, duplicate retry, and failure behavior have explicit tests.
- If financial value changes, journal debits and credits balance exactly and historical entries remain immutable.
- If AI is involved, model/prompt/feature version and fallback behavior are recorded and deterministic policy remains final.
- A timeout/restart cannot create a duplicate financial result; request status can be rediscovered.
- Logs/evidence contain correlation references but not secrets or prohibited sensitive payment data.

Common Failure Modes

- Blockchain chosen without a multi-party integrity requirement.
- AI given authority that deterministic policy should own.
- Regulatory/payment boundary left undefined.
- No measurable offline availability or recovery targets.

Expert Review Questions

- What happens if this component succeeds but the next component fails?
- How is the authorizing identity/device/key revoked or recovered?
- Can the operation be safely retried after the client loses the response?
- What happens if AI is unavailable, slow, or wrong?
- Can an auditor reconstruct debit, credit, actor, risk decision, approval, and Fabric txId from retained evidence?
- Which compensating transaction resolves a downstream operational error without rewriting history?

Chapter 8 - AI Authority and Human Oversight Boundary

Prevent AI recommendations from silently becoming financial authorization. A payment system is a financial state machine, not merely a set of CRUD screens. The blockchain layer should exist only where immutable multi-party evidence, endorsement, and deterministic settlement add value. AI is a separate advisory capability and must never become the accounting source of truth.

Design rule: AI may score/explain/summarize/recommend

Implementation Objective

The required output is: AI authority policy. It must be version-controlled, reviewable, testable with the WAN disabled, and linked to the architecture/source-of-truth model. For a payment platform, every implementation decision should make it possible to explain who authorized an action, which facts were trusted, which controls ran, what financial journal committed, and how the result is recovered after failure.

Key Engineering Decisions

- AI may score/explain/summarize/recommend
- Deterministic policy owns hard limits and permissions
- Humans approve high-risk exceptions
- AI never owns ledger signing keys

Deep Implementation Guidance

Write the decision as an architecture decision record with context, alternatives, selected option, security impact, operational impact, and reversal criteria. A private blockchain is valuable only if its independent identities and endorsement policies correspond to real governance boundaries.

Data and Control Questions

- What is the authoritative source for every input used by this component?
- Which user, device, service, MSP, or administrator identity is allowed to initiate or approve it?
- Does the operation move value, reserve value, create a side effect, or only provide advisory information?
- What idempotency/replay key proves that a retry is the same business intent?
- What evidence is retained off-chain and what stable reference/hash belongs on the ledger?

Chapter 8 Implementation - AI Authority and Human Oversight Boundary

Implementation Procedure

36. Write the business outcome and financial boundary before selecting technology.
37. Identify actors, trust domains, systems of record, failure domains, and regulated-data boundaries.
38. Create architecture decision records for blockchain, AI, database, and identity choices.
39. Define measurable latency, availability, RPO/RTO, audit, and offline-operation requirements.
40. Review whether a simpler signed database would meet the same trust model.

Implementation Example

```
# AI Authority and Human Oversight Boundary
INPUT  -> authenticate, validate schema, bind requestId/nonce
FACTS  -> read authoritative wallet/merchant/state data
AI     -> optional bounded recommendation with decisionId/version
POLICY -> enforce ownership, hard limits, approvals, risk fallback
LEDGER -> submit one idempotent deterministic transaction when value changes
EVENT  -> update projection/case/monitoring asynchronously
VERIFY -> discover committed result and retain correlationId + txId evidence
```

Verification Checklist

- The ai authority policy exists and is version-controlled.
- The feature works with all public Internet/WAN connectivity disabled.
- Authorization, malformed input, duplicate retry, and failure behavior have explicit tests.
- If financial value changes, journal debits and credits balance exactly and historical entries remain immutable.
- If AI is involved, model/prompt/feature version and fallback behavior are recorded and deterministic policy remains final.
- A timeout/restart cannot create a duplicate financial result; request status can be rediscovered.
- Logs/evidence contain correlation references but not secrets or prohibited sensitive payment data.

Common Failure Modes

- Blockchain chosen without a multi-party integrity requirement.
- AI given authority that deterministic policy should own.
- Regulatory/payment boundary left undefined.
- No measurable offline availability or recovery targets.

Expert Review Questions

- What happens if this component succeeds but the next component fails?
- How is the authorizing identity/device/key revoked or recovered?
- Can the operation be safely retried after the client loses the response?
- What happens if AI is unavailable, slow, or wrong?
- Can an auditor reconstruct debit, credit, actor, risk decision, approval, and Fabric txId from retained evidence?
- Which compensating transaction resolves a downstream operational error without rewriting history?

Chapter 9 - Threat and Failure Domain Overview

Identify which failures can corrupt, stop, or mislead the payment system. A payment system is a financial state machine, not merely a set of CRUD screens. The blockchain layer should exist only where immutable multi-party evidence, endorsement, and deterministic settlement add value. AI is a separate advisory capability and must never become the accounting source of truth.

Design rule: Separate host/network/storage/key/model/database failures

Implementation Objective

The required output is: Failure-domain map. It must be version-controlled, reviewable, testable with the WAN disabled, and linked to the architecture/source-of-truth model. For a payment platform, every implementation decision should make it possible to explain who authorized an action, which facts were trusted, which controls ran, what financial journal committed, and how the result is recovered after failure.

Key Engineering Decisions

- Separate host/network/storage/key/model/database failures
- Define which failures stop writes but allow reads
- Plan evidence-preserving recovery

Deep Implementation Guidance

Write the decision as an architecture decision record with context, alternatives, selected option, security impact, operational impact, and reversal criteria. A private blockchain is valuable only if its independent identities and endorsement policies correspond to real governance boundaries.

Data and Control Questions

- What is the authoritative source for every input used by this component?
- Which user, device, service, MSP, or administrator identity is allowed to initiate or approve it?
- Does the operation move value, reserve value, create a side effect, or only provide advisory information?
- What idempotency/replay key proves that a retry is the same business intent?
- What evidence is retained off-chain and what stable reference/hash belongs on the ledger?

Chapter 9 Implementation - Threat and Failure Domain Overview

Implementation Procedure

41. Write the business outcome and financial boundary before selecting technology.
42. Identify actors, trust domains, systems of record, failure domains, and regulated-data boundaries.
43. Create architecture decision records for blockchain, AI, database, and identity choices.
44. Define measurable latency, availability, RPO/RTO, audit, and offline-operation requirements.
45. Review whether a simpler signed database would meet the same trust model.

Implementation Example

```
# Threat and Failure Domain Overview
INPUT  -> authenticate, validate schema, bind requestId/nonce
FACTS  -> read authoritative wallet/merchant/state data
AI     -> optional bounded recommendation with decisionId/version
POLICY -> enforce ownership, hard limits, approvals, risk fallback
LEDGER -> submit one idempotent deterministic transaction when value changes
EVENT  -> update projection/case/monitoring asynchronously
VERIFY -> discover committed result and retain correlationId + txId evidence
```

Verification Checklist

- The failure-domain map exists and is version-controlled.
- The feature works with all public Internet/WAN connectivity disabled.
- Authorization, malformed input, duplicate retry, and failure behavior have explicit tests.
- If financial value changes, journal debits and credits balance exactly and historical entries remain immutable.
- If AI is involved, model/prompt/feature version and fallback behavior are recorded and deterministic policy remains final.
- A timeout/restart cannot create a duplicate financial result; request status can be rediscovered.
- Logs/evidence contain correlation references but not secrets or prohibited sensitive payment data.

Common Failure Modes

- Blockchain chosen without a multi-party integrity requirement.
- AI given authority that deterministic policy should own.
- Regulatory/payment boundary left undefined.
- No measurable offline availability or recovery targets.

Expert Review Questions

- What happens if this component succeeds but the next component fails?
- How is the authorizing identity/device/key revoked or recovered?
- Can the operation be safely retried after the client loses the response?
- What happens if AI is unavailable, slow, or wrong?
- Can an auditor reconstruct debit, credit, actor, risk decision, approval, and Fabric txId from retained evidence?
- Which compensating transaction resolves a downstream operational error without rewriting history?

Chapter 10 - Architecture Decision Records

Create durable records for the choices that make the system safe and maintainable. A payment system is a financial state machine, not merely a set of CRUD screens. The blockchain layer should exist only where immutable multi-party evidence, endorsement, and deterministic settlement add value. AI is a separate advisory capability and must never become the accounting source of truth.

Design rule: Record alternatives and consequences

Implementation Objective

The required output is: ADR set. It must be version-controlled, reviewable, testable with the WAN disabled, and linked to the architecture/source-of-truth model. For a payment platform, every implementation decision should make it possible to explain who authorized an action, which facts were trusted, which controls ran, what financial journal committed, and how the result is recovered after failure.

Key Engineering Decisions

- Record alternatives and consequences
- Include security/operations/reversal conditions
- Require review for changes to accounting, identity, AI authority, or trust domains

Deep Implementation Guidance

Write the decision as an architecture decision record with context, alternatives, selected option, security impact, operational impact, and reversal criteria. A private blockchain is valuable only if its independent identities and endorsement policies correspond to real governance boundaries.

Data and Control Questions

- What is the authoritative source for every input used by this component?
- Which user, device, service, MSP, or administrator identity is allowed to initiate or approve it?
- Does the operation move value, reserve value, create a side effect, or only provide advisory information?
- What idempotency/replay key proves that a retry is the same business intent?
- What evidence is retained off-chain and what stable reference/hash belongs on the ledger?

Chapter 10 Implementation - Architecture Decision Records

Implementation Procedure

46. Write the business outcome and financial boundary before selecting technology.
47. Identify actors, trust domains, systems of record, failure domains, and regulated-data boundaries.
48. Create architecture decision records for blockchain, AI, database, and identity choices.
49. Define measurable latency, availability, RPO/RTO, audit, and offline-operation requirements.
50. Review whether a simpler signed database would meet the same trust model.

Implementation Example

```
# Architecture Decision Records
INPUT  -> authenticate, validate schema, bind requestId/nonce
FACTS  -> read authoritative wallet/merchant/state data
AI     -> optional bounded recommendation with decisionId/version
POLICY -> enforce ownership, hard limits, approvals, risk fallback
LEDGER -> submit one idempotent deterministic transaction when value changes
EVENT  -> update projection/case/monitoring asynchronously
VERIFY -> discover committed result and retain correlationId + txId evidence
```

Verification Checklist

- The adr set exists and is version-controlled.
- The feature works with all public Internet/WAN connectivity disabled.
- Authorization, malformed input, duplicate retry, and failure behavior have explicit tests.
- If financial value changes, journal debits and credits balance exactly and historical entries remain immutable.
- If AI is involved, model/prompt/feature version and fallback behavior are recorded and deterministic policy remains final.
- A timeout/restart cannot create a duplicate financial result; request status can be rediscovered.
- Logs/evidence contain correlation references but not secrets or prohibited sensitive payment data.

Common Failure Modes

- Blockchain chosen without a multi-party integrity requirement.
- AI given authority that deterministic policy should own.
- Regulatory/payment boundary left undefined.
- No measurable offline availability or recovery targets.

Expert Review Questions

- What happens if this component succeeds but the next component fails?
- How is the authorizing identity/device/key revoked or recovered?
- Can the operation be safely retried after the client loses the response?
- What happens if AI is unavailable, slow, or wrong?
- Can an auditor reconstruct debit, credit, actor, risk decision, approval, and Fabric txId from retained evidence?
- Which compensating transaction resolves a downstream operational error without rewriting history?

Part II - Wallet, Accounting, and Payment Domain Model

The wallet domain is built around double-entry accounting. Every movement of value has balanced debits and credits, integer minor-unit amounts, explicit asset codes, lifecycle states, and immutable journal evidence. Corrections are compensating entries rather than silent edits.

Part focus: This part contains 19 implementation chapters. Each chapter includes design rules, implementation procedure, a practical artifact, verification, failure analysis, and expert review questions.

Chapter 11 - Double-Entry Accounting as the Financial Core

Make every movement of value auditable and mathematically balanced. The wallet domain is built around double-entry accounting. Every movement of value has balanced debits and credits, integer minor-unit amounts, explicit asset codes, lifecycle states, and immutable journal evidence. Corrections are compensating entries rather than silent edits.

Design rule: Every transaction has debit and credit lines

Implementation Objective

The required output is: Accounting invariant specification. It must be version-controlled, reviewable, testable with the WAN disabled, and linked to the architecture/source-of-truth model. For a payment platform, every implementation decision should make it possible to explain who authorized an action, which facts were trusted, which controls ran, what financial journal committed, and how the result is recovered after failure.

Key Engineering Decisions

- Every transaction has debit and credit lines
- Sum debits equals sum credits per asset
- Historical lines are immutable
- Current balance is derived/maintained consistently

Deep Implementation Guidance

For each financial entity, define creation authority, allowed states, immutable references, debit/credit effects, cancellation/compensation behavior, and reconciliation evidence. Never let a UI status string become the accounting model.

Data and Control Questions

- What is the authoritative source for every input used by this component?
- Which user, device, service, MSP, or administrator identity is allowed to initiate or approve it?
- Does the operation move value, reserve value, create a side effect, or only provide advisory information?
- What idempotency/replay key proves that a retry is the same business intent?
- What evidence is retained off-chain and what stable reference/hash belongs on the ledger?

Chapter 11 Implementation - Double-Entry Accounting as the Financial Core

Implementation Procedure

51. Define entity keys, lifecycle states, authorities, and immutable references.
52. Represent money as integer minor units plus explicit asset code.
53. Define debit/credit journal lines and prove every financial transaction balances.
54. Define idempotency, correction, reversal, and reconciliation semantics.
55. Create examples for normal, edge, and failure cases before writing chaincode.

Implementation Example

Example: customer pays merchant 25.00 LOCAL

Journal JRN-1001, asset=LOCAL

Customer wallet CUST-1 DEBIT 2500 minor units

Merchant wallet MERCH-7 CREDIT 2500 minor units

Invariant:

$\text{sum}(\text{debits}) == \text{sum}(\text{credits}) == 2500$

No value is created or destroyed by Transfer/Payment.

Only authorized Issuance/Redemption transactions change the closed-loop supply boundary.

Verification Checklist

- The accounting invariant specification exists and is version-controlled.
- The feature works with all public Internet/WAN connectivity disabled.
- Authorization, malformed input, duplicate retry, and failure behavior have explicit tests.
- If financial value changes, journal debits and credits balance exactly and historical entries remain immutable.
- If AI is involved, model/prompt/feature version and fallback behavior are recorded and deterministic policy remains final.
- A timeout/restart cannot create a duplicate financial result; request status can be rediscovered.
- Logs/evidence contain correlation references but not secrets or prohibited sensitive payment data.

Common Failure Modes

- Floating-point money.
- Unbalanced journal entries.
- Mutable historical financial records.
- Missing idempotency or compensating-entry semantics.

Expert Review Questions

- What happens if this component succeeds but the next component fails?
- How is the authorizing identity/device/key revoked or recovered?
- Can the operation be safely retried after the client loses the response?
- What happens if AI is unavailable, slow, or wrong?
- Can an auditor reconstruct debit, credit, actor, risk decision, approval, and Fabric txId from retained evidence?
- Which compensating transaction resolves a downstream operational error without rewriting history?

Chapter 12 - Wallet Types and Ledger Accounts

Model customer, merchant, treasury, fee, settlement, suspense, and system accounts. The wallet domain is built around double-entry accounting. Every movement of value has balanced debits and credits, integer minor-unit amounts, explicit asset codes, lifecycle states, and immutable journal evidence. Corrections are compensating entries rather than silent edits.

Design rule: Each wallet has ownerRef, asset, status, limits, and account type

Implementation Objective

The required output is: Wallet/account schema. It must be version-controlled, reviewable, testable with the WAN disabled, and linked to the architecture/source-of-truth model. For a payment platform, every implementation decision should make it possible to explain who authorized an action, which facts were trusted, which controls ran, what financial journal committed, and how the result is recovered after failure.

Key Engineering Decisions

- Each wallet has ownerRef, asset, status, limits, and account type
- System accounts have explicit purpose
- No anonymous magical balance bucket

Deep Implementation Guidance

For each financial entity, define creation authority, allowed states, immutable references, debit/credit effects, cancellation/compensation behavior, and reconciliation evidence. Never let a UI status string become the accounting model.

Data and Control Questions

- What is the authoritative source for every input used by this component?
- Which user, device, service, MSP, or administrator identity is allowed to initiate or approve it?
- Does the operation move value, reserve value, create a side effect, or only provide advisory information?
- What idempotency/replay key proves that a retry is the same business intent?
- What evidence is retained off-chain and what stable reference/hash belongs on the ledger?

Chapter 12 Implementation - Wallet Types and Ledger Accounts

Implementation Procedure

56. Define entity keys, lifecycle states, authorities, and immutable references.
57. Represent money as integer minor units plus explicit asset code.
58. Define debit/credit journal lines and prove every financial transaction balances.
59. Define idempotency, correction, reversal, and reconciliation semantics.
60. Create examples for normal, edge, and failure cases before writing chaincode.

Implementation Example

```
# Wallet Types and Ledger Accounts
INPUT  -> authenticate, validate schema, bind requestId/nonce
FACTS  -> read authoritative wallet/merchant/state data
AI     -> optional bounded recommendation with decisionId/version
POLICY -> enforce ownership, hard limits, approvals, risk fallback
LEDGER -> submit one idempotent deterministic transaction when value changes
EVENT  -> update projection/case/monitoring asynchronously
VERIFY -> discover committed result and retain correlationId + txId evidence
```

Verification Checklist

- The wallet/account schema exists and is version-controlled.
- The feature works with all public Internet/WAN connectivity disabled.
- Authorization, malformed input, duplicate retry, and failure behavior have explicit tests.
- If financial value changes, journal debits and credits balance exactly and historical entries remain immutable.
- If AI is involved, model/prompt/feature version and fallback behavior are recorded and deterministic policy remains final.
- A timeout/restart cannot create a duplicate financial result; request status can be rediscovered.
- Logs/evidence contain correlation references but not secrets or prohibited sensitive payment data.

Common Failure Modes

- Floating-point money.
- Unbalanced journal entries.
- Mutable historical financial records.
- Missing idempotency or compensating-entry semantics.

Expert Review Questions

- What happens if this component succeeds but the next component fails?
- How is the authorizing identity/device/key revoked or recovered?
- Can the operation be safely retried after the client loses the response?
- What happens if AI is unavailable, slow, or wrong?
- Can an auditor reconstruct debit, credit, actor, risk decision, approval, and Fabric txId from retained evidence?
- Which compensating transaction resolves a downstream operational error without rewriting history?

Chapter 13 - Money Representation and Asset Codes

Prevent rounding and currency-mixing defects. The wallet domain is built around double-entry accounting. Every movement of value has balanced debits and credits, integer minor-unit amounts, explicit asset codes, lifecycle states, and immutable journal evidence. Corrections are compensating entries rather than silent edits.

Design rule: Use signed/unsigned integer minor units as appropriate

Implementation Objective

The required output is: Money representation standard. It must be version-controlled, reviewable, testable with the WAN disabled, and linked to the architecture/source-of-truth model. For a payment platform, every implementation decision should make it possible to explain who authorized an action, which facts were trusted, which controls ran, what financial journal committed, and how the result is recovered after failure.

Key Engineering Decisions

- Use signed/unsigned integer minor units as appropriate
- Store explicit asset code and scale
- Never use binary floating point for financial amounts

Deep Implementation Guidance

For each financial entity, define creation authority, allowed states, immutable references, debit/credit effects, cancellation/compensation behavior, and reconciliation evidence. Never let a UI status string become the accounting model.

Data and Control Questions

- What is the authoritative source for every input used by this component?
- Which user, device, service, MSP, or administrator identity is allowed to initiate or approve it?
- Does the operation move value, reserve value, create a side effect, or only provide advisory information?
- What idempotency/replay key proves that a retry is the same business intent?
- What evidence is retained off-chain and what stable reference/hash belongs on the ledger?

Chapter 13 Implementation - Money Representation and Asset Codes

Implementation Procedure

61. Define entity keys, lifecycle states, authorities, and immutable references.
62. Represent money as integer minor units plus explicit asset code.
63. Define debit/credit journal lines and prove every financial transaction balances.
64. Define idempotency, correction, reversal, and reconciliation semantics.
65. Create examples for normal, edge, and failure cases before writing chaincode.

Implementation Example

```
# Money Representation and Asset Codes
INPUT  -> authenticate, validate schema, bind requestId/nonce
FACTS  -> read authoritative wallet/merchant/state data
AI     -> optional bounded recommendation with decisionId/version
POLICY -> enforce ownership, hard limits, approvals, risk fallback
LEDGER -> submit one idempotent deterministic transaction when value changes
EVENT  -> update projection/case/monitoring asynchronously
VERIFY -> discover committed result and retain correlationId + txId evidence
```

Verification Checklist

- The money representation standard exists and is version-controlled.
- The feature works with all public Internet/WAN connectivity disabled.
- Authorization, malformed input, duplicate retry, and failure behavior have explicit tests.
- If financial value changes, journal debits and credits balance exactly and historical entries remain immutable.
- If AI is involved, model/prompt/feature version and fallback behavior are recorded and deterministic policy remains final.
- A timeout/restart cannot create a duplicate financial result; request status can be rediscovered.
- Logs/evidence contain correlation references but not secrets or prohibited sensitive payment data.

Common Failure Modes

- Floating-point money.
- Unbalanced journal entries.
- Mutable historical financial records.
- Missing idempotency or compensating-entry semantics.

Expert Review Questions

- What happens if this component succeeds but the next component fails?
- How is the authorizing identity/device/key revoked or recovered?
- Can the operation be safely retried after the client loses the response?
- What happens if AI is unavailable, slow, or wrong?
- Can an auditor reconstruct debit, credit, actor, risk decision, approval, and Fabric txId from retained evidence?
- Which compensating transaction resolves a downstream operational error without rewriting history?

Chapter 14 - Wallet Lifecycle

Define creation, activation, suspension, freeze, closure, and archival behavior. The wallet domain is built around double-entry accounting. Every movement of value has balanced debits and credits, integer minor-unit amounts, explicit asset codes, lifecycle states, and immutable journal evidence. Corrections are compensating entries rather than silent edits.

Design rule: Creation requires unique wallet ID and owner mapping

Implementation Objective

The required output is: Wallet state machine. It must be version-controlled, reviewable, testable with the WAN disabled, and linked to the architecture/source-of-truth model. For a payment platform, every implementation decision should make it possible to explain who authorized an action, which facts were trusted, which controls ran, what financial journal committed, and how the result is recovered after failure.

Key Engineering Decisions

- Creation requires unique wallet ID and owner mapping
- Freeze blocks selected debit operations
- Closure requires zero/settled balance and audit trail

Deep Implementation Guidance

For each financial entity, define creation authority, allowed states, immutable references, debit/credit effects, cancellation/compensation behavior, and reconciliation evidence. Never let a UI status string become the accounting model.

Data and Control Questions

- What is the authoritative source for every input used by this component?
- Which user, device, service, MSP, or administrator identity is allowed to initiate or approve it?
- Does the operation move value, reserve value, create a side effect, or only provide advisory information?
- What idempotency/replay key proves that a retry is the same business intent?
- What evidence is retained off-chain and what stable reference/hash belongs on the ledger?

Chapter 14 Implementation - Wallet Lifecycle

Implementation Procedure

66. Define entity keys, lifecycle states, authorities, and immutable references.
67. Represent money as integer minor units plus explicit asset code.
68. Define debit/credit journal lines and prove every financial transaction balances.
69. Define idempotency, correction, reversal, and reconciliation semantics.
70. Create examples for normal, edge, and failure cases before writing chaincode.

Implementation Example

```
# Wallet Lifecycle
INPUT  -> authenticate, validate schema, bind requestId/nonce
FACTS  -> read authoritative wallet/merchant/state data
AI     -> optional bounded recommendation with decisionId/version
POLICY -> enforce ownership, hard limits, approvals, risk fallback
LEDGER -> submit one idempotent deterministic transaction when value changes
EVENT  -> update projection/case/monitoring asynchronously
VERIFY -> discover committed result and retain correlationId + txId evidence
```

Verification Checklist

- The wallet state machine exists and is version-controlled.
- The feature works with all public Internet/WAN connectivity disabled.
- Authorization, malformed input, duplicate retry, and failure behavior have explicit tests.
- If financial value changes, journal debits and credits balance exactly and historical entries remain immutable.
- If AI is involved, model/prompt/feature version and fallback behavior are recorded and deterministic policy remains final.
- A timeout/restart cannot create a duplicate financial result; request status can be rediscovered.
- Logs/evidence contain correlation references but not secrets or prohibited sensitive payment data.

Common Failure Modes

- Floating-point money.
- Unbalanced journal entries.
- Mutable historical financial records.
- Missing idempotency or compensating-entry semantics.

Expert Review Questions

- What happens if this component succeeds but the next component fails?
- How is the authorizing identity/device/key revoked or recovered?
- Can the operation be safely retried after the client loses the response?
- What happens if AI is unavailable, slow, or wrong?
- Can an auditor reconstruct debit, credit, actor, risk decision, approval, and Fabric txId from retained evidence?
- Which compensating transaction resolves a downstream operational error without rewriting history?

Chapter 15 - Issuance of Stored Value

Create value only through explicitly authorized treasury issuance. The wallet domain is built around double-entry accounting. Every movement of value has balanced debits and credits, integer minor-unit amounts, explicit asset codes, lifecycle states, and immutable journal evidence. Corrections are compensating entries rather than silent edits.

Design rule: Debit issuance/treasury source and credit customer/merchant wallet

Implementation Objective

The required output is: Issuance transaction design. It must be version-controlled, reviewable, testable with the WAN disabled, and linked to the architecture/source-of-truth model. For a payment platform, every implementation decision should make it possible to explain who authorized an action, which facts were trusted, which controls ran, what financial journal committed, and how the result is recovered after failure.

Key Engineering Decisions

- Debit issuance/treasury source and credit customer/merchant wallet
- Require privileged role and requestId
- Record business reference and approval when threshold exceeded

Deep Implementation Guidance

For each financial entity, define creation authority, allowed states, immutable references, debit/credit effects, cancellation/compensation behavior, and reconciliation evidence. Never let a UI status string become the accounting model.

Data and Control Questions

- What is the authoritative source for every input used by this component?
- Which user, device, service, MSP, or administrator identity is allowed to initiate or approve it?
- Does the operation move value, reserve value, create a side effect, or only provide advisory information?
- What idempotency/replay key proves that a retry is the same business intent?
- What evidence is retained off-chain and what stable reference/hash belongs on the ledger?

Chapter 15 Implementation - Issuance of Stored Value

Implementation Procedure

71. Define entity keys, lifecycle states, authorities, and immutable references.
72. Represent money as integer minor units plus explicit asset code.
73. Define debit/credit journal lines and prove every financial transaction balances.
74. Define idempotency, correction, reversal, and reconciliation semantics.
75. Create examples for normal, edge, and failure cases before writing chaincode.

Implementation Example

```
# Issuance of Stored Value
INPUT  -> authenticate, validate schema, bind requestId/nonce
FACTS  -> read authoritative wallet/merchant/state data
AI     -> optional bounded recommendation with decisionId/version
POLICY -> enforce ownership, hard limits, approvals, risk fallback
LEDGER -> submit one idempotent deterministic transaction when value changes
EVENT  -> update projection/case/monitoring asynchronously
VERIFY -> discover committed result and retain correlationId + txId evidence
```

Verification Checklist

- The issuance transaction design exists and is version-controlled.
- The feature works with all public Internet/WAN connectivity disabled.
- Authorization, malformed input, duplicate retry, and failure behavior have explicit tests.
- If financial value changes, journal debits and credits balance exactly and historical entries remain immutable.
- If AI is involved, model/prompt/feature version and fallback behavior are recorded and deterministic policy remains final.
- A timeout/restart cannot create a duplicate financial result; request status can be rediscovered.
- Logs/evidence contain correlation references but not secrets or prohibited sensitive payment data.

Common Failure Modes

- Floating-point money.
- Unbalanced journal entries.
- Mutable historical financial records.
- Missing idempotency or compensating-entry semantics.

Expert Review Questions

- What happens if this component succeeds but the next component fails?
- How is the authorizing identity/device/key revoked or recovered?
- Can the operation be safely retried after the client loses the response?
- What happens if AI is unavailable, slow, or wrong?
- Can an auditor reconstruct debit, credit, actor, risk decision, approval, and Fabric txId from retained evidence?
- Which compensating transaction resolves a downstream operational error without rewriting history?

Chapter 16 - Redemption and Cash-Out

Remove stored value through an auditable redemption transaction. The wallet domain is built around double-entry accounting. Every movement of value has balanced debits and credits, integer minor-unit amounts, explicit asset codes, lifecycle states, and immutable journal evidence. Corrections are compensating entries rather than silent edits.

Design rule: Verify available balance and policy

Implementation Objective

The required output is: Redemption flow. It must be version-controlled, reviewable, testable with the WAN disabled, and linked to the architecture/source-of-truth model. For a payment platform, every implementation decision should make it possible to explain who authorized an action, which facts were trusted, which controls ran, what financial journal committed, and how the result is recovered after failure.

Key Engineering Decisions

- Verify available balance and policy
- Debit wallet and credit redemption/treasury account
- External cash/bank action is a separate controlled process if present

Deep Implementation Guidance

For each financial entity, define creation authority, allowed states, immutable references, debit/credit effects, cancellation/compensation behavior, and reconciliation evidence. Never let a UI status string become the accounting model.

Data and Control Questions

- What is the authoritative source for every input used by this component?
- Which user, device, service, MSP, or administrator identity is allowed to initiate or approve it?
- Does the operation move value, reserve value, create a side effect, or only provide advisory information?
- What idempotency/replay key proves that a retry is the same business intent?
- What evidence is retained off-chain and what stable reference/hash belongs on the ledger?

Chapter 16 Implementation - Redemption and Cash-Out

Implementation Procedure

76. Define entity keys, lifecycle states, authorities, and immutable references.
77. Represent money as integer minor units plus explicit asset code.
78. Define debit/credit journal lines and prove every financial transaction balances.
79. Define idempotency, correction, reversal, and reconciliation semantics.
80. Create examples for normal, edge, and failure cases before writing chaincode.

Implementation Example

```
# Redemption and Cash-Out
INPUT  -> authenticate, validate schema, bind requestId/nonce
FACTS  -> read authoritative wallet/merchant/state data
AI     -> optional bounded recommendation with decisionId/version
POLICY -> enforce ownership, hard limits, approvals, risk fallback
LEDGER -> submit one idempotent deterministic transaction when value changes
EVENT  -> update projection/case/monitoring asynchronously
VERIFY -> discover committed result and retain correlationId + txId evidence
```

Verification Checklist

- The redemption flow exists and is version-controlled.
- The feature works with all public Internet/WAN connectivity disabled.
- Authorization, malformed input, duplicate retry, and failure behavior have explicit tests.
- If financial value changes, journal debits and credits balance exactly and historical entries remain immutable.
- If AI is involved, model/prompt/feature version and fallback behavior are recorded and deterministic policy remains final.
- A timeout/restart cannot create a duplicate financial result; request status can be rediscovered.
- Logs/evidence contain correlation references but not secrets or prohibited sensitive payment data.

Common Failure Modes

- Floating-point money.
- Unbalanced journal entries.
- Mutable historical financial records.
- Missing idempotency or compensating-entry semantics.

Expert Review Questions

- What happens if this component succeeds but the next component fails?
- How is the authorizing identity/device/key revoked or recovered?
- Can the operation be safely retried after the client loses the response?
- What happens if AI is unavailable, slow, or wrong?
- Can an auditor reconstruct debit, credit, actor, risk decision, approval, and Fabric txId from retained evidence?
- Which compensating transaction resolves a downstream operational error without rewriting history?

Chapter 17 - Peer-to-Peer Transfer

Transfer value between customer wallets atomically. The wallet domain is built around double-entry accounting. Every movement of value has balanced debits and credits, integer minor-unit amounts, explicit asset codes, lifecycle states, and immutable journal evidence. Corrections are compensating entries rather than silent edits.

Design rule: Validate sender/recipient/asset/status

Implementation Objective

The required output is: Transfer schema and flow. It must be version-controlled, reviewable, testable with the WAN disabled, and linked to the architecture/source-of-truth model. For a payment platform, every implementation decision should make it possible to explain who authorized an action, which facts were trusted, which controls ran, what financial journal committed, and how the result is recovered after failure.

Key Engineering Decisions

- Validate sender/recipient/asset/status
- Debit sender and credit recipient in one transaction
- Use idempotency key and limits
- Emit transfer event

Deep Implementation Guidance

For each financial entity, define creation authority, allowed states, immutable references, debit/credit effects, cancellation/compensation behavior, and reconciliation evidence. Never let a UI status string become the accounting model.

Data and Control Questions

- What is the authoritative source for every input used by this component?
- Which user, device, service, MSP, or administrator identity is allowed to initiate or approve it?
- Does the operation move value, reserve value, create a side effect, or only provide advisory information?
- What idempotency/replay key proves that a retry is the same business intent?
- What evidence is retained off-chain and what stable reference/hash belongs on the ledger?

Chapter 17 Implementation - Peer-to-Peer Transfer

Implementation Procedure

81. Define entity keys, lifecycle states, authorities, and immutable references.
82. Represent money as integer minor units plus explicit asset code.
83. Define debit/credit journal lines and prove every financial transaction balances.
84. Define idempotency, correction, reversal, and reconciliation semantics.
85. Create examples for normal, edge, and failure cases before writing chaincode.

Implementation Example

```
# Peer-to-Peer Transfer
INPUT  -> authenticate, validate schema, bind requestId/nonce
FACTS  -> read authoritative wallet/merchant/state data
AI     -> optional bounded recommendation with decisionId/version
POLICY -> enforce ownership, hard limits, approvals, risk fallback
LEDGER -> submit one idempotent deterministic transaction when value changes
EVENT  -> update projection/case/monitoring asynchronously
VERIFY -> discover committed result and retain correlationId + txId evidence
```

Verification Checklist

- The transfer schema and flow exists and is version-controlled.
- The feature works with all public Internet/WAN connectivity disabled.
- Authorization, malformed input, duplicate retry, and failure behavior have explicit tests.
- If financial value changes, journal debits and credits balance exactly and historical entries remain immutable.
- If AI is involved, model/prompt/feature version and fallback behavior are recorded and deterministic policy remains final.
- A timeout/restart cannot create a duplicate financial result; request status can be rediscovered.
- Logs/evidence contain correlation references but not secrets or prohibited sensitive payment data.

Common Failure Modes

- Floating-point money.
- Unbalanced journal entries.
- Mutable historical financial records.
- Missing idempotency or compensating-entry semantics.

Expert Review Questions

- What happens if this component succeeds but the next component fails?
- How is the authorizing identity/device/key revoked or recovered?
- Can the operation be safely retried after the client loses the response?
- What happens if AI is unavailable, slow, or wrong?
- Can an auditor reconstruct debit, credit, actor, risk decision, approval, and Fabric txId from retained evidence?
- Which compensating transaction resolves a downstream operational error without rewriting history?

Chapter 18 - Merchant Payment

Move value from customer to merchant with payment-specific references and states. The wallet domain is built around double-entry accounting. Every movement of value has balanced debits and credits, integer minor-unit amounts, explicit asset codes, lifecycle states, and immutable journal evidence. Corrections are compensating entries rather than silent edits.

Design rule: Bind merchant/store/order reference

Implementation Objective

The required output is: Merchant payment design. It must be version-controlled, reviewable, testable with the WAN disabled, and linked to the architecture/source-of-truth model. For a payment platform, every implementation decision should make it possible to explain who authorized an action, which facts were trusted, which controls ran, what financial journal committed, and how the result is recovered after failure.

Key Engineering Decisions

- Bind merchant/store/order reference
- Apply fees deterministically if configured
- Record authorization/risk reference
- Return final ledger transaction ID

Deep Implementation Guidance

For each financial entity, define creation authority, allowed states, immutable references, debit/credit effects, cancellation/compensation behavior, and reconciliation evidence. Never let a UI status string become the accounting model.

Data and Control Questions

- What is the authoritative source for every input used by this component?
- Which user, device, service, MSP, or administrator identity is allowed to initiate or approve it?
- Does the operation move value, reserve value, create a side effect, or only provide advisory information?
- What idempotency/replay key proves that a retry is the same business intent?
- What evidence is retained off-chain and what stable reference/hash belongs on the ledger?

Chapter 18 Implementation - Merchant Payment

Implementation Procedure

86. Define entity keys, lifecycle states, authorities, and immutable references.
87. Represent money as integer minor units plus explicit asset code.
88. Define debit/credit journal lines and prove every financial transaction balances.
89. Define idempotency, correction, reversal, and reconciliation semantics.
90. Create examples for normal, edge, and failure cases before writing chaincode.

Implementation Example

```
# Merchant Payment
INPUT  -> authenticate, validate schema, bind requestId/nonce
FACTS  -> read authoritative wallet/merchant/state data
AI     -> optional bounded recommendation with decisionId/version
POLICY -> enforce ownership, hard limits, approvals, risk fallback
LEDGER -> submit one idempotent deterministic transaction when value changes
EVENT  -> update projection/case/monitoring asynchronously
VERIFY -> discover committed result and retain correlationId + txId evidence
```

Verification Checklist

- The merchant payment design exists and is version-controlled.
- The feature works with all public Internet/WAN connectivity disabled.
- Authorization, malformed input, duplicate retry, and failure behavior have explicit tests.
- If financial value changes, journal debits and credits balance exactly and historical entries remain immutable.
- If AI is involved, model/prompt/feature version and fallback behavior are recorded and deterministic policy remains final.
- A timeout/restart cannot create a duplicate financial result; request status can be rediscovered.
- Logs/evidence contain correlation references but not secrets or prohibited sensitive payment data.

Common Failure Modes

- Floating-point money.
- Unbalanced journal entries.
- Mutable historical financial records.
- Missing idempotency or compensating-entry semantics.

Expert Review Questions

- What happens if this component succeeds but the next component fails?
- How is the authorizing identity/device/key revoked or recovered?
- Can the operation be safely retried after the client loses the response?
- What happens if AI is unavailable, slow, or wrong?
- Can an auditor reconstruct debit, credit, actor, risk decision, approval, and Fabric txId from retained evidence?
- Which compensating transaction resolves a downstream operational error without rewriting history?

Chapter 19 - Authorization Holds

Reserve spendable balance without final settlement. The wallet domain is built around double-entry accounting. Every movement of value has balanced debits and credits, integer minor-unit amounts, explicit asset codes, lifecycle states, and immutable journal evidence. Corrections are compensating entries rather than silent edits.

Design rule: Move available amount into reserved/hold accounting

Implementation Objective

The required output is: Hold state model. It must be version-controlled, reviewable, testable with the WAN disabled, and linked to the architecture/source-of-truth model. For a payment platform, every implementation decision should make it possible to explain who authorized an action, which facts were trusted, which controls ran, what financial journal committed, and how the result is recovered after failure.

Key Engineering Decisions

- Move available amount into reserved/hold accounting
- Use expiry policy
- Hold cannot exceed available funds
- Capture/release references original hold

Deep Implementation Guidance

For each financial entity, define creation authority, allowed states, immutable references, debit/credit effects, cancellation/compensation behavior, and reconciliation evidence. Never let a UI status string become the accounting model.

Data and Control Questions

- What is the authoritative source for every input used by this component?
- Which user, device, service, MSP, or administrator identity is allowed to initiate or approve it?
- Does the operation move value, reserve value, create a side effect, or only provide advisory information?
- What idempotency/replay key proves that a retry is the same business intent?
- What evidence is retained off-chain and what stable reference/hash belongs on the ledger?

Chapter 19 Implementation - Authorization Holds

Implementation Procedure

91. Define entity keys, lifecycle states, authorities, and immutable references.
92. Represent money as integer minor units plus explicit asset code.
93. Define debit/credit journal lines and prove every financial transaction balances.
94. Define idempotency, correction, reversal, and reconciliation semantics.
95. Create examples for normal, edge, and failure cases before writing chaincode.

Implementation Example

```
# Authorization Holds
INPUT  -> authenticate, validate schema, bind requestId/nonce
FACTS  -> read authoritative wallet/merchant/state data
AI     -> optional bounded recommendation with decisionId/version
POLICY -> enforce ownership, hard limits, approvals, risk fallback
LEDGER -> submit one idempotent deterministic transaction when value changes
EVENT  -> update projection/case/monitoring asynchronously
VERIFY -> discover committed result and retain correlationId + txId evidence
```

Verification Checklist

- The hold state model exists and is version-controlled.
- The feature works with all public Internet/WAN connectivity disabled.
- Authorization, malformed input, duplicate retry, and failure behavior have explicit tests.
- If financial value changes, journal debits and credits balance exactly and historical entries remain immutable.
- If AI is involved, model/prompt/feature version and fallback behavior are recorded and deterministic policy remains final.
- A timeout/restart cannot create a duplicate financial result; request status can be rediscovered.
- Logs/evidence contain correlation references but not secrets or prohibited sensitive payment data.

Common Failure Modes

- Floating-point money.
- Unbalanced journal entries.
- Mutable historical financial records.
- Missing idempotency or compensating-entry semantics.

Expert Review Questions

- What happens if this component succeeds but the next component fails?
- How is the authorizing identity/device/key revoked or recovered?
- Can the operation be safely retried after the client loses the response?
- What happens if AI is unavailable, slow, or wrong?
- Can an auditor reconstruct debit, credit, actor, risk decision, approval, and Fabric txId from retained evidence?
- Which compensating transaction resolves a downstream operational error without rewriting history?

Chapter 20 - Capture of a Hold

Convert an authorized hold into a final merchant payment. The wallet domain is built around double-entry accounting. Every movement of value has balanced debits and credits, integer minor-unit amounts, explicit asset codes, lifecycle states, and immutable journal evidence. Corrections are compensating entries rather than silent edits.

Design rule: Verify hold active and amount allowed

Implementation Objective

The required output is: Capture flow. It must be version-controlled, reviewable, testable with the WAN disabled, and linked to the architecture/source-of-truth model. For a payment platform, every implementation decision should make it possible to explain who authorized an action, which facts were trusted, which controls ran, what financial journal committed, and how the result is recovered after failure.

Key Engineering Decisions

- Verify hold active and amount allowed
- Release reserved liability correctly
- Credit merchant and fee accounts
- Mark hold captured atomically

Deep Implementation Guidance

For each financial entity, define creation authority, allowed states, immutable references, debit/credit effects, cancellation/compensation behavior, and reconciliation evidence. Never let a UI status string become the accounting model.

Data and Control Questions

- What is the authoritative source for every input used by this component?
- Which user, device, service, MSP, or administrator identity is allowed to initiate or approve it?
- Does the operation move value, reserve value, create a side effect, or only provide advisory information?
- What idempotency/replay key proves that a retry is the same business intent?
- What evidence is retained off-chain and what stable reference/hash belongs on the ledger?

Chapter 20 Implementation - Capture of a Hold

Implementation Procedure

96. Define entity keys, lifecycle states, authorities, and immutable references.
97. Represent money as integer minor units plus explicit asset code.
98. Define debit/credit journal lines and prove every financial transaction balances.
99. Define idempotency, correction, reversal, and reconciliation semantics.
100. Create examples for normal, edge, and failure cases before writing chaincode.

Implementation Example

```
# Capture of a Hold
INPUT  -> authenticate, validate schema, bind requestId/nonce
FACTS  -> read authoritative wallet/merchant/state data
AI     -> optional bounded recommendation with decisionId/version
POLICY -> enforce ownership, hard limits, approvals, risk fallback
LEDGER -> submit one idempotent deterministic transaction when value changes
EVENT  -> update projection/case/monitoring asynchronously
VERIFY -> discover committed result and retain correlationId + txId evidence
```

Verification Checklist

- The capture flow exists and is version-controlled.
- The feature works with all public Internet/WAN connectivity disabled.
- Authorization, malformed input, duplicate retry, and failure behavior have explicit tests.
- If financial value changes, journal debits and credits balance exactly and historical entries remain immutable.
- If AI is involved, model/prompt/feature version and fallback behavior are recorded and deterministic policy remains final.
- A timeout/restart cannot create a duplicate financial result; request status can be rediscovered.
- Logs/evidence contain correlation references but not secrets or prohibited sensitive payment data.

Common Failure Modes

- Floating-point money.
- Unbalanced journal entries.
- Mutable historical financial records.
- Missing idempotency or compensating-entry semantics.

Expert Review Questions

- What happens if this component succeeds but the next component fails?
- How is the authorizing identity/device/key revoked or recovered?
- Can the operation be safely retried after the client loses the response?
- What happens if AI is unavailable, slow, or wrong?
- Can an auditor reconstruct debit, credit, actor, risk decision, approval, and Fabric txId from retained evidence?
- Which compensating transaction resolves a downstream operational error without rewriting history?

Chapter 21 - Release and Expiry of a Hold

Return reserved funds when payment will not complete. The wallet domain is built around double-entry accounting. Every movement of value has balanced debits and credits, integer minor-unit amounts, explicit asset codes, lifecycle states, and immutable journal evidence. Corrections are compensating entries rather than silent edits.

Design rule: Release only active/unconsumed amount

Implementation Objective

The required output is: Release flow. It must be version-controlled, reviewable, testable with the WAN disabled, and linked to the architecture/source-of-truth model. For a payment platform, every implementation decision should make it possible to explain who authorized an action, which facts were trusted, which controls ran, what financial journal committed, and how the result is recovered after failure.

Key Engineering Decisions

- Release only active/unconsumed amount
- Use explicit reason and actor/system policy
- Expiry worker submits deterministic release transaction

Deep Implementation Guidance

For each financial entity, define creation authority, allowed states, immutable references, debit/credit effects, cancellation/compensation behavior, and reconciliation evidence. Never let a UI status string become the accounting model.

Data and Control Questions

- What is the authoritative source for every input used by this component?
- Which user, device, service, MSP, or administrator identity is allowed to initiate or approve it?
- Does the operation move value, reserve value, create a side effect, or only provide advisory information?
- What idempotency/replay key proves that a retry is the same business intent?
- What evidence is retained off-chain and what stable reference/hash belongs on the ledger?

Chapter 21 Implementation - Release and Expiry of a Hold

Implementation Procedure

101. Define entity keys, lifecycle states, authorities, and immutable references.
102. Represent money as integer minor units plus explicit asset code.
103. Define debit/credit journal lines and prove every financial transaction balances.
104. Define idempotency, correction, reversal, and reconciliation semantics.
105. Create examples for normal, edge, and failure cases before writing chaincode.

Implementation Example

```
# Release and Expiry of a Hold
INPUT  -> authenticate, validate schema, bind requestId/nonce
FACTS  -> read authoritative wallet/merchant/state data
AI     -> optional bounded recommendation with decisionId/version
POLICY -> enforce ownership, hard limits, approvals, risk fallback
LEDGER -> submit one idempotent deterministic transaction when value changes
EVENT  -> update projection/case/monitoring asynchronously
VERIFY -> discover committed result and retain correlationId + txId evidence
```

Verification Checklist

- The release flow exists and is version-controlled.
- The feature works with all public Internet/WAN connectivity disabled.
- Authorization, malformed input, duplicate retry, and failure behavior have explicit tests.
- If financial value changes, journal debits and credits balance exactly and historical entries remain immutable.
- If AI is involved, model/prompt/feature version and fallback behavior are recorded and deterministic policy remains final.
- A timeout/restart cannot create a duplicate financial result; request status can be rediscovered.
- Logs/evidence contain correlation references but not secrets or prohibited sensitive payment data.

Common Failure Modes

- Floating-point money.
- Unbalanced journal entries.
- Mutable historical financial records.
- Missing idempotency or compensating-entry semantics.

Expert Review Questions

- What happens if this component succeeds but the next component fails?
- How is the authorizing identity/device/key revoked or recovered?
- Can the operation be safely retried after the client loses the response?
- What happens if AI is unavailable, slow, or wrong?
- Can an auditor reconstruct debit, credit, actor, risk decision, approval, and Fabric txId from retained evidence?
- Which compensating transaction resolves a downstream operational error without rewriting history?

Chapter 22 - Refunds

Return all or part of a completed payment with immutable linkage. The wallet domain is built around double-entry accounting. Every movement of value has balanced debits and credits, integer minor-unit amounts, explicit asset codes, lifecycle states, and immutable journal evidence. Corrections are compensating entries rather than silent edits.

Design rule: Reference original payment and remaining refundable amount

Implementation Objective

The required output is: Refund schema. It must be version-controlled, reviewable, testable with the WAN disabled, and linked to the architecture/source-of-truth model. For a payment platform, every implementation decision should make it possible to explain who authorized an action, which facts were trusted, which controls ran, what financial journal committed, and how the result is recovered after failure.

Key Engineering Decisions

- Reference original payment and remaining refundable amount
- Debit merchant/refund source and credit customer
- Use approval thresholds and idempotency

Deep Implementation Guidance

For each financial entity, define creation authority, allowed states, immutable references, debit/credit effects, cancellation/compensation behavior, and reconciliation evidence. Never let a UI status string become the accounting model.

Data and Control Questions

- What is the authoritative source for every input used by this component?
- Which user, device, service, MSP, or administrator identity is allowed to initiate or approve it?
- Does the operation move value, reserve value, create a side effect, or only provide advisory information?
- What idempotency/replay key proves that a retry is the same business intent?
- What evidence is retained off-chain and what stable reference/hash belongs on the ledger?

Chapter 22 Implementation - Refunds

Implementation Procedure

106. Define entity keys, lifecycle states, authorities, and immutable references.
107. Represent money as integer minor units plus explicit asset code.
108. Define debit/credit journal lines and prove every financial transaction balances.
109. Define idempotency, correction, reversal, and reconciliation semantics.
110. Create examples for normal, edge, and failure cases before writing chaincode.

Implementation Example

```
# Refunds
INPUT  -> authenticate, validate schema, bind requestId/nonce
FACTS  -> read authoritative wallet/merchant/state data
AI     -> optional bounded recommendation with decisionId/version
POLICY -> enforce ownership, hard limits, approvals, risk fallback
LEDGER -> submit one idempotent deterministic transaction when value changes
EVENT  -> update projection/case/monitoring asynchronously
VERIFY -> discover committed result and retain correlationId + txId evidence
```

Verification Checklist

- The refund schema exists and is version-controlled.
- The feature works with all public Internet/WAN connectivity disabled.
- Authorization, malformed input, duplicate retry, and failure behavior have explicit tests.
- If financial value changes, journal debits and credits balance exactly and historical entries remain immutable.
- If AI is involved, model/prompt/feature version and fallback behavior are recorded and deterministic policy remains final.
- A timeout/restart cannot create a duplicate financial result; request status can be rediscovered.
- Logs/evidence contain correlation references but not secrets or prohibited sensitive payment data.

Common Failure Modes

- Floating-point money.
- Unbalanced journal entries.
- Mutable historical financial records.
- Missing idempotency or compensating-entry semantics.

Expert Review Questions

- What happens if this component succeeds but the next component fails?
- How is the authorizing identity/device/key revoked or recovered?
- Can the operation be safely retried after the client loses the response?
- What happens if AI is unavailable, slow, or wrong?
- Can an auditor reconstruct debit, credit, actor, risk decision, approval, and Fabric txId from retained evidence?
- Which compensating transaction resolves a downstream operational error without rewriting history?

Chapter 23 - Reversals and Compensating Entries

Correct mistakes without mutating financial history. The wallet domain is built around double-entry accounting. Every movement of value has balanced debits and credits, integer minor-unit amounts, explicit asset codes, lifecycle states, and immutable journal evidence. Corrections are compensating entries rather than silent edits.

Design rule: Never edit committed journal lines

Implementation Objective

The required output is: Compensation pattern. It must be version-controlled, reviewable, testable with the WAN disabled, and linked to the architecture/source-of-truth model. For a payment platform, every implementation decision should make it possible to explain who authorized an action, which facts were trusted, which controls ran, what financial journal committed, and how the result is recovered after failure.

Key Engineering Decisions

- Never edit committed journal lines
- Create new reversing/adjusting journal
- Reference original tx and correction reason
- Require stronger authorization for manual adjustments

Deep Implementation Guidance

For each financial entity, define creation authority, allowed states, immutable references, debit/credit effects, cancellation/compensation behavior, and reconciliation evidence. Never let a UI status string become the accounting model.

Data and Control Questions

- What is the authoritative source for every input used by this component?
- Which user, device, service, MSP, or administrator identity is allowed to initiate or approve it?
- Does the operation move value, reserve value, create a side effect, or only provide advisory information?
- What idempotency/replay key proves that a retry is the same business intent?
- What evidence is retained off-chain and what stable reference/hash belongs on the ledger?

Chapter 23 Implementation - Reversals and Compensating Entries

Implementation Procedure

111. Define entity keys, lifecycle states, authorities, and immutable references.
112. Represent money as integer minor units plus explicit asset code.
113. Define debit/credit journal lines and prove every financial transaction balances.
114. Define idempotency, correction, reversal, and reconciliation semantics.
115. Create examples for normal, edge, and failure cases before writing chaincode.

Implementation Example

```
# Reversals and Compensating Entries
INPUT  -> authenticate, validate schema, bind requestId/nonce
FACTS  -> read authoritative wallet/merchant/state data
AI     -> optional bounded recommendation with decisionId/version
POLICY -> enforce ownership, hard limits, approvals, risk fallback
LEDGER -> submit one idempotent deterministic transaction when value changes
EVENT  -> update projection/case/monitoring asynchronously
VERIFY -> discover committed result and retain correlationId + txId evidence
```

Verification Checklist

- The compensation pattern exists and is version-controlled.
- The feature works with all public Internet/WAN connectivity disabled.
- Authorization, malformed input, duplicate retry, and failure behavior have explicit tests.
- If financial value changes, journal debits and credits balance exactly and historical entries remain immutable.
- If AI is involved, model/prompt/feature version and fallback behavior are recorded and deterministic policy remains final.
- A timeout/restart cannot create a duplicate financial result; request status can be rediscovered.
- Logs/evidence contain correlation references but not secrets or prohibited sensitive payment data.

Common Failure Modes

- Floating-point money.
- Unbalanced journal entries.
- Mutable historical financial records.
- Missing idempotency or compensating-entry semantics.

Expert Review Questions

- What happens if this component succeeds but the next component fails?
- How is the authorizing identity/device/key revoked or recovered?
- Can the operation be safely retried after the client loses the response?
- What happens if AI is unavailable, slow, or wrong?
- Can an auditor reconstruct debit, credit, actor, risk decision, approval, and Fabric txId from retained evidence?
- Which compensating transaction resolves a downstream operational error without rewriting history?

Chapter 24 - Fees and Commissions

Model platform/merchant fees as explicit balanced journal lines. The wallet domain is built around double-entry accounting. Every movement of value has balanced debits and credits, integer minor-unit amounts, explicit asset codes, lifecycle states, and immutable journal evidence. Corrections are compensating entries rather than silent edits.

Design rule: Calculate deterministic fee from versioned policy

Implementation Objective

The required output is: Fee model. It must be version-controlled, reviewable, testable with the WAN disabled, and linked to the architecture/source-of-truth model. For a payment platform, every implementation decision should make it possible to explain who authorized an action, which facts were trusted, which controls ran, what financial journal committed, and how the result is recovered after failure.

Key Engineering Decisions

- Calculate deterministic fee from versioned policy
- Credit fee wallet and debit appropriate source
- Persist fee policy/version reference

Deep Implementation Guidance

For each financial entity, define creation authority, allowed states, immutable references, debit/credit effects, cancellation/compensation behavior, and reconciliation evidence. Never let a UI status string become the accounting model.

Data and Control Questions

- What is the authoritative source for every input used by this component?
- Which user, device, service, MSP, or administrator identity is allowed to initiate or approve it?
- Does the operation move value, reserve value, create a side effect, or only provide advisory information?
- What idempotency/replay key proves that a retry is the same business intent?
- What evidence is retained off-chain and what stable reference/hash belongs on the ledger?

Chapter 24 Implementation - Fees and Commissions

Implementation Procedure

116. Define entity keys, lifecycle states, authorities, and immutable references.
117. Represent money as integer minor units plus explicit asset code.
118. Define debit/credit journal lines and prove every financial transaction balances.
119. Define idempotency, correction, reversal, and reconciliation semantics.
120. Create examples for normal, edge, and failure cases before writing chaincode.

Implementation Example

```
# Fees and Commissions
INPUT  -> authenticate, validate schema, bind requestId/nonce
FACTS  -> read authoritative wallet/merchant/state data
AI     -> optional bounded recommendation with decisionId/version
POLICY -> enforce ownership, hard limits, approvals, risk fallback
LEDGER -> submit one idempotent deterministic transaction when value changes
EVENT  -> update projection/case/monitoring asynchronously
VERIFY -> discover committed result and retain correlationId + txId evidence
```

Verification Checklist

- The fee model exists and is version-controlled.
- The feature works with all public Internet/WAN connectivity disabled.
- Authorization, malformed input, duplicate retry, and failure behavior have explicit tests.
- If financial value changes, journal debits and credits balance exactly and historical entries remain immutable.
- If AI is involved, model/prompt/feature version and fallback behavior are recorded and deterministic policy remains final.
- A timeout/restart cannot create a duplicate financial result; request status can be rediscovered.
- Logs/evidence contain correlation references but not secrets or prohibited sensitive payment data.

Common Failure Modes

- Floating-point money.
- Unbalanced journal entries.
- Mutable historical financial records.
- Missing idempotency or compensating-entry semantics.

Expert Review Questions

- What happens if this component succeeds but the next component fails?
- How is the authorizing identity/device/key revoked or recovered?
- Can the operation be safely retried after the client loses the response?
- What happens if AI is unavailable, slow, or wrong?
- Can an auditor reconstruct debit, credit, actor, risk decision, approval, and Fabric txId from retained evidence?
- Which compensating transaction resolves a downstream operational error without rewriting history?

Chapter 25 - Limits and Velocity Controls

Apply hard transaction, daily, wallet, device, and merchant limits independently of AI. The wallet domain is built around double-entry accounting. Every movement of value has balanced debits and credits, integer minor-unit amounts, explicit asset codes, lifecycle states, and immutable journal evidence. Corrections are compensating entries rather than silent edits.

Design rule: Per-transaction limits are deterministic

Implementation Objective

The required output is: Limit matrix. It must be version-controlled, reviewable, testable with the WAN disabled, and linked to the architecture/source-of-truth model. For a payment platform, every implementation decision should make it possible to explain who authorized an action, which facts were trusted, which controls ran, what financial journal committed, and how the result is recovered after failure.

Key Engineering Decisions

- Per-transaction limits are deterministic
- Daily/rolling windows use authoritative counters or ledger facts
- AI can tighten/recommend but cannot silently loosen hard limits

Deep Implementation Guidance

For each financial entity, define creation authority, allowed states, immutable references, debit/credit effects, cancellation/compensation behavior, and reconciliation evidence. Never let a UI status string become the accounting model.

Data and Control Questions

- What is the authoritative source for every input used by this component?
- Which user, device, service, MSP, or administrator identity is allowed to initiate or approve it?
- Does the operation move value, reserve value, create a side effect, or only provide advisory information?
- What idempotency/replay key proves that a retry is the same business intent?
- What evidence is retained off-chain and what stable reference/hash belongs on the ledger?

Chapter 25 Implementation - Limits and Velocity Controls

Implementation Procedure

121. Define entity keys, lifecycle states, authorities, and immutable references.
122. Represent money as integer minor units plus explicit asset code.
123. Define debit/credit journal lines and prove every financial transaction balances.
124. Define idempotency, correction, reversal, and reconciliation semantics.
125. Create examples for normal, edge, and failure cases before writing chaincode.

Implementation Example

```
# Limits and Velocity Controls
INPUT  -> authenticate, validate schema, bind requestId/nonce
FACTS  -> read authoritative wallet/merchant/state data
AI     -> optional bounded recommendation with decisionId/version
POLICY -> enforce ownership, hard limits, approvals, risk fallback
LEDGER -> submit one idempotent deterministic transaction when value changes
EVENT  -> update projection/case/monitoring asynchronously
VERIFY -> discover committed result and retain correlationId + txId evidence
```

Verification Checklist

- The limit matrix exists and is version-controlled.
- The feature works with all public Internet/WAN connectivity disabled.
- Authorization, malformed input, duplicate retry, and failure behavior have explicit tests.
- If financial value changes, journal debits and credits balance exactly and historical entries remain immutable.
- If AI is involved, model/prompt/feature version and fallback behavior are recorded and deterministic policy remains final.
- A timeout/restart cannot create a duplicate financial result; request status can be rediscovered.
- Logs/evidence contain correlation references but not secrets or prohibited sensitive payment data.

Common Failure Modes

- Floating-point money.
- Unbalanced journal entries.
- Mutable historical financial records.
- Missing idempotency or compensating-entry semantics.

Expert Review Questions

- What happens if this component succeeds but the next component fails?
- How is the authorizing identity/device/key revoked or recovered?
- Can the operation be safely retried after the client loses the response?
- What happens if AI is unavailable, slow, or wrong?
- Can an auditor reconstruct debit, credit, actor, risk decision, approval, and Fabric txId from retained evidence?
- Which compensating transaction resolves a downstream operational error without rewriting history?

Chapter 26 - Merchant Settlement

Move accumulated merchant balances into settlement/redemption accounts with reconciliation evidence. The wallet domain is built around double-entry accounting. Every movement of value has balanced debits and credits, integer minor-unit amounts, explicit asset codes, lifecycle states, and immutable journal evidence. Corrections are compensating entries rather than silent edits.

Design rule: Define settlement batch ID and cutoff

Implementation Objective

The required output is: Settlement flow. It must be version-controlled, reviewable, testable with the WAN disabled, and linked to the architecture/source-of-truth model. For a payment platform, every implementation decision should make it possible to explain who authorized an action, which facts were trusted, which controls ran, what financial journal committed, and how the result is recovered after failure.

Key Engineering Decisions

- Define settlement batch ID and cutoff
- Sum included merchant entries
- Create balanced settlement journal
- Anchor report hash

Deep Implementation Guidance

For each financial entity, define creation authority, allowed states, immutable references, debit/credit effects, cancellation/compensation behavior, and reconciliation evidence. Never let a UI status string become the accounting model.

Data and Control Questions

- What is the authoritative source for every input used by this component?
- Which user, device, service, MSP, or administrator identity is allowed to initiate or approve it?
- Does the operation move value, reserve value, create a side effect, or only provide advisory information?
- What idempotency/replay key proves that a retry is the same business intent?
- What evidence is retained off-chain and what stable reference/hash belongs on the ledger?

Chapter 26 Implementation - Merchant Settlement

Implementation Procedure

126. Define entity keys, lifecycle states, authorities, and immutable references.
127. Represent money as integer minor units plus explicit asset code.
128. Define debit/credit journal lines and prove every financial transaction balances.
129. Define idempotency, correction, reversal, and reconciliation semantics.
130. Create examples for normal, edge, and failure cases before writing chaincode.

Implementation Example

```
# Merchant Settlement
INPUT  -> authenticate, validate schema, bind requestId/nonce
FACTS  -> read authoritative wallet/merchant/state data
AI     -> optional bounded recommendation with decisionId/version
POLICY -> enforce ownership, hard limits, approvals, risk fallback
LEDGER -> submit one idempotent deterministic transaction when value changes
EVENT  -> update projection/case/monitoring asynchronously
VERIFY -> discover committed result and retain correlationId + txId evidence
```

Verification Checklist

- The settlement flow exists and is version-controlled.
- The feature works with all public Internet/WAN connectivity disabled.
- Authorization, malformed input, duplicate retry, and failure behavior have explicit tests.
- If financial value changes, journal debits and credits balance exactly and historical entries remain immutable.
- If AI is involved, model/prompt/feature version and fallback behavior are recorded and deterministic policy remains final.
- A timeout/restart cannot create a duplicate financial result; request status can be rediscovered.
- Logs/evidence contain correlation references but not secrets or prohibited sensitive payment data.

Common Failure Modes

- Floating-point money.
- Unbalanced journal entries.
- Mutable historical financial records.
- Missing idempotency or compensating-entry semantics.

Expert Review Questions

- What happens if this component succeeds but the next component fails?
- How is the authorizing identity/device/key revoked or recovered?
- Can the operation be safely retried after the client loses the response?
- What happens if AI is unavailable, slow, or wrong?
- Can an auditor reconstruct debit, credit, actor, risk decision, approval, and Fabric txId from retained evidence?
- Which compensating transaction resolves a downstream operational error without rewriting history?

Chapter 27 - Reconciliation

Prove balances and transaction totals agree across ledger and projections. The wallet domain is built around double-entry accounting. Every movement of value has balanced debits and credits, integer minor-unit amounts, explicit asset codes, lifecycle states, and immutable journal evidence. Corrections are compensating entries rather than silent edits.

Design rule: Recompute journal totals

Implementation Objective

The required output is: Reconciliation design. It must be version-controlled, reviewable, testable with the WAN disabled, and linked to the architecture/source-of-truth model. For a payment platform, every implementation decision should make it possible to explain who authorized an action, which facts were trusted, which controls ran, what financial journal committed, and how the result is recovered after failure.

Key Engineering Decisions

- Recompute journal totals
- Compare projections to ledger
- Compare merchant/customer/system account conservation
- Open exceptions instead of auto-hiding differences

Deep Implementation Guidance

For each financial entity, define creation authority, allowed states, immutable references, debit/credit effects, cancellation/compensation behavior, and reconciliation evidence. Never let a UI status string become the accounting model.

Data and Control Questions

- What is the authoritative source for every input used by this component?
- Which user, device, service, MSP, or administrator identity is allowed to initiate or approve it?
- Does the operation move value, reserve value, create a side effect, or only provide advisory information?
- What idempotency/replay key proves that a retry is the same business intent?
- What evidence is retained off-chain and what stable reference/hash belongs on the ledger?

Chapter 27 Implementation - Reconciliation

Implementation Procedure

131. Define entity keys, lifecycle states, authorities, and immutable references.
132. Represent money as integer minor units plus explicit asset code.
133. Define debit/credit journal lines and prove every financial transaction balances.
134. Define idempotency, correction, reversal, and reconciliation semantics.
135. Create examples for normal, edge, and failure cases before writing chaincode.

Implementation Example

```
# Reconciliation
INPUT  -> authenticate, validate schema, bind requestId/nonce
FACTS  -> read authoritative wallet/merchant/state data
AI     -> optional bounded recommendation with decisionId/version
POLICY -> enforce ownership, hard limits, approvals, risk fallback
LEDGER -> submit one idempotent deterministic transaction when value changes
EVENT  -> update projection/case/monitoring asynchronously
VERIFY -> discover committed result and retain correlationId + txId evidence
```

Verification Checklist

- The reconciliation design exists and is version-controlled.
- The feature works with all public Internet/WAN connectivity disabled.
- Authorization, malformed input, duplicate retry, and failure behavior have explicit tests.
- If financial value changes, journal debits and credits balance exactly and historical entries remain immutable.
- If AI is involved, model/prompt/feature version and fallback behavior are recorded and deterministic policy remains final.
- A timeout/restart cannot create a duplicate financial result; request status can be rediscovered.
- Logs/evidence contain correlation references but not secrets or prohibited sensitive payment data.

Common Failure Modes

- Floating-point money.
- Unbalanced journal entries.
- Mutable historical financial records.
- Missing idempotency or compensating-entry semantics.

Expert Review Questions

- What happens if this component succeeds but the next component fails?
- How is the authorizing identity/device/key revoked or recovered?
- Can the operation be safely retried after the client loses the response?
- What happens if AI is unavailable, slow, or wrong?
- Can an auditor reconstruct debit, credit, actor, risk decision, approval, and Fabric txId from retained evidence?
- Which compensating transaction resolves a downstream operational error without rewriting history?

Chapter 28 - Disputes and Cases

Represent operational investigation without changing original financial facts. The wallet domain is built around double-entry accounting. Every movement of value has balanced debits and credits, integer minor-unit amounts, explicit asset codes, lifecycle states, and immutable journal evidence. Corrections are compensating entries rather than silent edits.

Design rule: Case links transaction IDs and evidence hashes

Implementation Objective

The required output is: Dispute case model. It must be version-controlled, reviewable, testable with the WAN disabled, and linked to the architecture/source-of-truth model. For a payment platform, every implementation decision should make it possible to explain who authorized an action, which facts were trusted, which controls ran, what financial journal committed, and how the result is recovered after failure.

Key Engineering Decisions

- Case links transaction IDs and evidence hashes
- AI may summarize evidence
- Human decision creates explicit approved resolution transaction

Deep Implementation Guidance

For each financial entity, define creation authority, allowed states, immutable references, debit/credit effects, cancellation/compensation behavior, and reconciliation evidence. Never let a UI status string become the accounting model.

Data and Control Questions

- What is the authoritative source for every input used by this component?
- Which user, device, service, MSP, or administrator identity is allowed to initiate or approve it?
- Does the operation move value, reserve value, create a side effect, or only provide advisory information?
- What idempotency/replay key proves that a retry is the same business intent?
- What evidence is retained off-chain and what stable reference/hash belongs on the ledger?

Chapter 28 Implementation - Disputes and Cases

Implementation Procedure

136. Define entity keys, lifecycle states, authorities, and immutable references.
137. Represent money as integer minor units plus explicit asset code.
138. Define debit/credit journal lines and prove every financial transaction balances.
139. Define idempotency, correction, reversal, and reconciliation semantics.
140. Create examples for normal, edge, and failure cases before writing chaincode.

Implementation Example

```
# Disputes and Cases
INPUT  -> authenticate, validate schema, bind requestId/nonce
FACTS  -> read authoritative wallet/merchant/state data
AI     -> optional bounded recommendation with decisionId/version
POLICY -> enforce ownership, hard limits, approvals, risk fallback
LEDGER -> submit one idempotent deterministic transaction when value changes
EVENT  -> update projection/case/monitoring asynchronously
VERIFY -> discover committed result and retain correlationId + txId evidence
```

Verification Checklist

- The dispute case model exists and is version-controlled.
- The feature works with all public Internet/WAN connectivity disabled.
- Authorization, malformed input, duplicate retry, and failure behavior have explicit tests.
- If financial value changes, journal debits and credits balance exactly and historical entries remain immutable.
- If AI is involved, model/prompt/feature version and fallback behavior are recorded and deterministic policy remains final.
- A timeout/restart cannot create a duplicate financial result; request status can be rediscovered.
- Logs/evidence contain correlation references but not secrets or prohibited sensitive payment data.

Common Failure Modes

- Floating-point money.
- Unbalanced journal entries.
- Mutable historical financial records.
- Missing idempotency or compensating-entry semantics.

Expert Review Questions

- What happens if this component succeeds but the next component fails?
- How is the authorizing identity/device/key revoked or recovered?
- Can the operation be safely retried after the client loses the response?
- What happens if AI is unavailable, slow, or wrong?
- Can an auditor reconstruct debit, credit, actor, risk decision, approval, and Fabric txId from retained evidence?
- Which compensating transaction resolves a downstream operational error without rewriting history?

Chapter 29 - Daily Close and Financial Reporting

Create repeatable end-of-day control evidence. The wallet domain is built around double-entry accounting. Every movement of value has balanced debits and credits, integer minor-unit amounts, explicit asset codes, lifecycle states, and immutable journal evidence. Corrections are compensating entries rather than silent edits.

Design rule: Snapshot totals and counts

Implementation Objective

The required output is: Daily close package. It must be version-controlled, reviewable, testable with the WAN disabled, and linked to the architecture/source-of-truth model. For a payment platform, every implementation decision should make it possible to explain who authorized an action, which facts were trusted, which controls ran, what financial journal committed, and how the result is recovered after failure.

Key Engineering Decisions

- Snapshot totals and counts
- Record reconciliation result and exceptions
- Hash/sign report package
- Retain operator approval

Deep Implementation Guidance

For each financial entity, define creation authority, allowed states, immutable references, debit/credit effects, cancellation/compensation behavior, and reconciliation evidence. Never let a UI status string become the accounting model.

Data and Control Questions

- What is the authoritative source for every input used by this component?
- Which user, device, service, MSP, or administrator identity is allowed to initiate or approve it?
- Does the operation move value, reserve value, create a side effect, or only provide advisory information?
- What idempotency/replay key proves that a retry is the same business intent?
- What evidence is retained off-chain and what stable reference/hash belongs on the ledger?

Chapter 29 Implementation - Daily Close and Financial Reporting

Implementation Procedure

141. Define entity keys, lifecycle states, authorities, and immutable references.
142. Represent money as integer minor units plus explicit asset code.
143. Define debit/credit journal lines and prove every financial transaction balances.
144. Define idempotency, correction, reversal, and reconciliation semantics.
145. Create examples for normal, edge, and failure cases before writing chaincode.

Implementation Example

```
# Daily Close and Financial Reporting
INPUT  -> authenticate, validate schema, bind requestId/nonce
FACTS  -> read authoritative wallet/merchant/state data
AI     -> optional bounded recommendation with decisionId/version
POLICY -> enforce ownership, hard limits, approvals, risk fallback
LEDGER -> submit one idempotent deterministic transaction when value changes
EVENT  -> update projection/case/monitoring asynchronously
VERIFY -> discover committed result and retain correlationId + txId evidence
```

Verification Checklist

- The daily close package exists and is version-controlled.
- The feature works with all public Internet/WAN connectivity disabled.
- Authorization, malformed input, duplicate retry, and failure behavior have explicit tests.
- If financial value changes, journal debits and credits balance exactly and historical entries remain immutable.
- If AI is involved, model/prompt/feature version and fallback behavior are recorded and deterministic policy remains final.
- A timeout/restart cannot create a duplicate financial result; request status can be rediscovered.
- Logs/evidence contain correlation references but not secrets or prohibited sensitive payment data.

Common Failure Modes

- Floating-point money.
- Unbalanced journal entries.
- Mutable historical financial records.
- Missing idempotency or compensating-entry semantics.

Expert Review Questions

- What happens if this component succeeds but the next component fails?
- How is the authorizing identity/device/key revoked or recovered?
- Can the operation be safely retried after the client loses the response?
- What happens if AI is unavailable, slow, or wrong?
- Can an auditor reconstruct debit, credit, actor, risk decision, approval, and Fabric txId from retained evidence?
- Which compensating transaction resolves a downstream operational error without rewriting history?

Part III - Offline LAN Infrastructure and Platform Foundation

An Internet-independent platform must own every runtime dependency: addressing, DNS, time, certificates, package artifacts, model files, monitoring, backup media, and operator procedures. If a service secretly requires cloud DNS, public telemetry, or a package registry, the design is not truly local.

Part focus: This part contains 15 implementation chapters. Each chapter includes design rules, implementation procedure, a practical artifact, verification, failure analysis, and expert review questions.

Chapter 30 - Physical Host and VM Topology

Size and place nodes for fault isolation and manageable operations. An Internet-independent platform must own every runtime dependency: addressing, DNS, time, certificates, package artifacts, model files, monitoring, backup media, and operator procedures. If a service secretly requires cloud DNS, public telemetry, or a package registry, the design is not truly local.

Design rule: Separate orderers across failure domains where possible

Implementation Objective

The required output is: Host topology. It must be version-controlled, reviewable, testable with the WAN disabled, and linked to the architecture/source-of-truth model. For a payment platform, every implementation decision should make it possible to explain who authorized an action, which facts were trusted, which controls ran, what financial journal committed, and how the result is recovered after failure.

Key Engineering Decisions

- Separate orderers across failure domains where possible
- Separate database/AI workloads from critical ledger nodes
- Document replacement capacity

Deep Implementation Guidance

Maintain a machine-readable inventory with hostname, IP, VLAN, OS, CPU/RAM/storage, ports, certificates, persistent volume paths, artifact versions, model hashes, backup paths, and responsible owner. Test every critical workflow after disabling WAN access.

Data and Control Questions

- What is the authoritative source for every input used by this component?
- Which user, device, service, MSP, or administrator identity is allowed to initiate or approve it?
- Does the operation move value, reserve value, create a side effect, or only provide advisory information?
- What idempotency/replay key proves that a retry is the same business intent?
- What evidence is retained off-chain and what stable reference/hash belongs on the ledger?

Chapter 30 Implementation - Physical Host and VM Topology

Implementation Procedure

146. Inventory hosts, static IPs, VLANs, ports, storage, clocks, and operators.
147. Build a checksummed offline artifact/model/package manifest.
148. Configure local DNS, time service, TLS trust, firewall policy, and persistent volumes.
149. Test with WAN physically/logically disabled.
150. Document backup, restore, replacement host, and offline update procedures.

Implementation Example

```
# Physical Host and VM Topology
INPUT  -> authenticate, validate schema, bind requestId/nonce
FACTS  -> read authoritative wallet/merchant/state data
AI      -> optional bounded recommendation with decisionId/version
POLICY  -> enforce ownership, hard limits, approvals, risk fallback
LEDGER  -> submit one idempotent deterministic transaction when value changes
EVENT   -> update projection/case/monitoring asynchronously
VERIFY  -> discover committed result and retain correlationId + txId evidence
```

Verification Checklist

- The host topology exists and is version-controlled.
- The feature works with all public Internet/WAN connectivity disabled.
- Authorization, malformed input, duplicate retry, and failure behavior have explicit tests.
- If financial value changes, journal debits and credits balance exactly and historical entries remain immutable.
- If AI is involved, model/prompt/feature version and fallback behavior are recorded and deterministic policy remains final.
- A timeout/restart cannot create a duplicate financial result; request status can be rediscovered.
- Logs/evidence contain correlation references but not secrets or prohibited sensitive payment data.

Common Failure Modes

- Hidden public DNS/CDN/telemetry dependency.
- Clock drift that breaks certificates/audit ordering.
- Unpinned container/model/package versions.
- Backups stored on the same failure domain as production.

Expert Review Questions

- What happens if this component succeeds but the next component fails?
- How is the authorizing identity/device/key revoked or recovered?
- Can the operation be safely retried after the client loses the response?
- What happens if AI is unavailable, slow, or wrong?
- Can an auditor reconstruct debit, credit, actor, risk decision, approval, and Fabric txId from retained evidence?
- Which compensating transaction resolves a downstream operational error without rewriting history?

Chapter 31 - Network Segmentation and VLAN Design

Restrict trust paths among clients, APIs, Fabric, databases, AI, and management. An Internet-independent platform must own every runtime dependency: addressing, DNS, time, certificates, package artifacts, model files, monitoring, backup media, and operator procedures. If a service secretly requires cloud DNS, public telemetry, or a package registry, the design is not truly local.

Design rule: POS/wallet clients reach API only

Implementation Objective

The required output is: VLAN/firewall matrix. It must be version-controlled, reviewable, testable with the WAN disabled, and linked to the architecture/source-of-truth model. For a payment platform, every implementation decision should make it possible to explain who authorized an action, which facts were trusted, which controls ran, what financial journal committed, and how the result is recovered after failure.

Key Engineering Decisions

- POS/wallet clients reach API only
- API reaches Gateway peers, DB, AI
- Management/CA endpoints are isolated
- Default deny between segments

Deep Implementation Guidance

Maintain a machine-readable inventory with hostname, IP, VLAN, OS, CPU/RAM/storage, ports, certificates, persistent volume paths, artifact versions, model hashes, backup paths, and responsible owner. Test every critical workflow after disabling WAN access.

Data and Control Questions

- What is the authoritative source for every input used by this component?
- Which user, device, service, MSP, or administrator identity is allowed to initiate or approve it?
- Does the operation move value, reserve value, create a side effect, or only provide advisory information?
- What idempotency/replay key proves that a retry is the same business intent?
- What evidence is retained off-chain and what stable reference/hash belongs on the ledger?

Chapter 31 Implementation - Network Segmentation and VLAN Design

Implementation Procedure

151. Inventory hosts, static IPs, VLANs, ports, storage, clocks, and operators.
152. Build a checksummed offline artifact/model/package manifest.
153. Configure local DNS, time service, TLS trust, firewall policy, and persistent volumes.
154. Test with WAN physically/logically disabled.
155. Document backup, restore, replacement host, and offline update procedures.

Implementation Example

```
# Network Segmentation and VLAN Design
INPUT  -> authenticate, validate schema, bind requestId/nonce
FACTS  -> read authoritative wallet/merchant/state data
AI      -> optional bounded recommendation with decisionId/version
POLICY  -> enforce ownership, hard limits, approvals, risk fallback
LEDGER  -> submit one idempotent deterministic transaction when value changes
EVENT   -> update projection/case/monitoring asynchronously
VERIFY  -> discover committed result and retain correlationId + txId evidence
```

Verification Checklist

- The vlan/firewall matrix exists and is version-controlled.
- The feature works with all public Internet/WAN connectivity disabled.
- Authorization, malformed input, duplicate retry, and failure behavior have explicit tests.
- If financial value changes, journal debits and credits balance exactly and historical entries remain immutable.
- If AI is involved, model/prompt/feature version and fallback behavior are recorded and deterministic policy remains final.
- A timeout/restart cannot create a duplicate financial result; request status can be rediscovered.
- Logs/evidence contain correlation references but not secrets or prohibited sensitive payment data.

Common Failure Modes

- Hidden public DNS/CDN/telemetry dependency.
- Clock drift that breaks certificates/audit ordering.
- Unpinned container/model/package versions.
- Backups stored on the same failure domain as production.

Expert Review Questions

- What happens if this component succeeds but the next component fails?
- How is the authorizing identity/device/key revoked or recovered?
- Can the operation be safely retried after the client loses the response?
- What happens if AI is unavailable, slow, or wrong?
- Can an auditor reconstruct debit, credit, actor, risk decision, approval, and Fabric txId from retained evidence?
- Which compensating transaction resolves a downstream operational error without rewriting history?

Chapter 32 - Local DNS

Provide stable service names without public DNS. An Internet-independent platform must own every runtime dependency: addressing, DNS, time, certificates, package artifacts, model files, monitoring, backup media, and operator procedures. If a service secretly requires cloud DNS, public telemetry, or a package registry, the design is not truly local.

Design rule: Use internal domain names

Implementation Objective

The required output is: DNS zone file. It must be version-controlled, reviewable, testable with the WAN disabled, and linked to the architecture/source-of-truth model. For a payment platform, every implementation decision should make it possible to explain who authorized an action, which facts were trusted, which controls ran, what financial journal committed, and how the result is recovered after failure.

Key Engineering Decisions

- Use internal domain names
- Document static records and failover
- Do not let production depend on Internet DNS

Deep Implementation Guidance

Maintain a machine-readable inventory with hostname, IP, VLAN, OS, CPU/RAM/storage, ports, certificates, persistent volume paths, artifact versions, model hashes, backup paths, and responsible owner. Test every critical workflow after disabling WAN access.

Data and Control Questions

- What is the authoritative source for every input used by this component?
- Which user, device, service, MSP, or administrator identity is allowed to initiate or approve it?
- Does the operation move value, reserve value, create a side effect, or only provide advisory information?
- What idempotency/replay key proves that a retry is the same business intent?
- What evidence is retained off-chain and what stable reference/hash belongs on the ledger?

Chapter 32 Implementation - Local DNS

Implementation Procedure

156. Inventory hosts, static IPs, VLANs, ports, storage, clocks, and operators.
157. Build a checksummed offline artifact/model/package manifest.
158. Configure local DNS, time service, TLS trust, firewall policy, and persistent volumes.
159. Test with WAN physically/logically disabled.
160. Document backup, restore, replacement host, and offline update procedures.

Implementation Example

```
# Local DNS
INPUT  -> authenticate, validate schema, bind requestId/nonce
FACTS  -> read authoritative wallet/merchant/state data
AI     -> optional bounded recommendation with decisionId/version
POLICY -> enforce ownership, hard limits, approvals, risk fallback
LEDGER -> submit one idempotent deterministic transaction when value changes
EVENT  -> update projection/case/monitoring asynchronously
VERIFY -> discover committed result and retain correlationId + txId evidence
```

Verification Checklist

- The dns zone file exists and is version-controlled.
- The feature works with all public Internet/WAN connectivity disabled.
- Authorization, malformed input, duplicate retry, and failure behavior have explicit tests.
- If financial value changes, journal debits and credits balance exactly and historical entries remain immutable.
- If AI is involved, model/prompt/feature version and fallback behavior are recorded and deterministic policy remains final.
- A timeout/restart cannot create a duplicate financial result; request status can be rediscovered.
- Logs/evidence contain correlation references but not secrets or prohibited sensitive payment data.

Common Failure Modes

- Hidden public DNS/CDN/telemetry dependency.
- Clock drift that breaks certificates/audit ordering.
- Unpinned container/model/package versions.
- Backups stored on the same failure domain as production.

Expert Review Questions

- What happens if this component succeeds but the next component fails?
- How is the authorizing identity/device/key revoked or recovered?
- Can the operation be safely retried after the client loses the response?
- What happens if AI is unavailable, slow, or wrong?
- Can an auditor reconstruct debit, credit, actor, risk decision, approval, and Fabric txId from retained evidence?
- Which compensating transaction resolves a downstream operational error without rewriting history?

Chapter 33 - Local Time Synchronization

Keep certificates, logs, audit timelines, and distributed systems consistent. An Internet-independent platform must own every runtime dependency: addressing, DNS, time, certificates, package artifacts, model files, monitoring, backup media, and operator procedures. If a service secretly requires cloud DNS, public telemetry, or a package registry, the design is not truly local.

Design rule: Run redundant local time sources

Implementation Objective

The required output is: NTP/time design. It must be version-controlled, reviewable, testable with the WAN disabled, and linked to the architecture/source-of-truth model. For a payment platform, every implementation decision should make it possible to explain who authorized an action, which facts were trusted, which controls ran, what financial journal committed, and how the result is recovered after failure.

Key Engineering Decisions

- Run redundant local time sources
- Monitor drift
- Do not use blockchain timestamp as a substitute for trusted operational time

Deep Implementation Guidance

Maintain a machine-readable inventory with hostname, IP, VLAN, OS, CPU/RAM/storage, ports, certificates, persistent volume paths, artifact versions, model hashes, backup paths, and responsible owner. Test every critical workflow after disabling WAN access.

Data and Control Questions

- What is the authoritative source for every input used by this component?
- Which user, device, service, MSP, or administrator identity is allowed to initiate or approve it?
- Does the operation move value, reserve value, create a side effect, or only provide advisory information?
- What idempotency/replay key proves that a retry is the same business intent?
- What evidence is retained off-chain and what stable reference/hash belongs on the ledger?

Chapter 33 Implementation - Local Time Synchronization

Implementation Procedure

161. Inventory hosts, static IPs, VLANs, ports, storage, clocks, and operators.
162. Build a checksummed offline artifact/model/package manifest.
163. Configure local DNS, time service, TLS trust, firewall policy, and persistent volumes.
164. Test with WAN physically/logically disabled.
165. Document backup, restore, replacement host, and offline update procedures.

Implementation Example

```
# Local Time Synchronization
INPUT  -> authenticate, validate schema, bind requestId/nonce
FACTS  -> read authoritative wallet/merchant/state data
AI      -> optional bounded recommendation with decisionId/version
POLICY  -> enforce ownership, hard limits, approvals, risk fallback
LEDGER  -> submit one idempotent deterministic transaction when value changes
EVENT   -> update projection/case/monitoring asynchronously
VERIFY  -> discover committed result and retain correlationId + txId evidence
```

Verification Checklist

- The ntp/time design exists and is version-controlled.
- The feature works with all public Internet/WAN connectivity disabled.
- Authorization, malformed input, duplicate retry, and failure behavior have explicit tests.
- If financial value changes, journal debits and credits balance exactly and historical entries remain immutable.
- If AI is involved, model/prompt/feature version and fallback behavior are recorded and deterministic policy remains final.
- A timeout/restart cannot create a duplicate financial result; request status can be rediscovered.
- Logs/evidence contain correlation references but not secrets or prohibited sensitive payment data.

Common Failure Modes

- Hidden public DNS/CDN/telemetry dependency.
- Clock drift that breaks certificates/audit ordering.
- Unpinned container/model/package versions.
- Backups stored on the same failure domain as production.

Expert Review Questions

- What happens if this component succeeds but the next component fails?
- How is the authorizing identity/device/key revoked or recovered?
- Can the operation be safely retried after the client loses the response?
- What happens if AI is unavailable, slow, or wrong?
- Can an auditor reconstruct debit, credit, actor, risk decision, approval, and Fabric txId from retained evidence?
- Which compensating transaction resolves a downstream operational error without rewriting history?

Chapter 34 - Offline Artifact and Dependency Bundle

Make clean rebuilds possible with no package downloads. An Internet-independent platform must own every runtime dependency: addressing, DNS, time, certificates, package artifacts, model files, monitoring, backup media, and operator procedures. If a service secretly requires cloud DNS, public telemetry, or a package registry, the design is not truly local.

Design rule: Include Fabric binaries/images, Node/Python packages, OS packages, model files, DB images, checksums, licenses

Implementation Objective

The required output is: Artifact manifest. It must be version-controlled, reviewable, testable with the WAN disabled, and linked to the architecture/source-of-truth model. For a payment platform, every implementation decision should make it possible to explain who authorized an action, which facts were trusted, which controls ran, what financial journal committed, and how the result is recovered after failure.

Key Engineering Decisions

- Include Fabric binaries/images, Node/Python packages, OS packages, model files, DB images, checksums, licenses
- Pin exact versions/digests

Deep Implementation Guidance

Maintain a machine-readable inventory with hostname, IP, VLAN, OS, CPU/RAM/storage, ports, certificates, persistent volume paths, artifact versions, model hashes, backup paths, and responsible owner. Test every critical workflow after disabling WAN access.

Data and Control Questions

- What is the authoritative source for every input used by this component?
- Which user, device, service, MSP, or administrator identity is allowed to initiate or approve it?
- Does the operation move value, reserve value, create a side effect, or only provide advisory information?
- What idempotency/replay key proves that a retry is the same business intent?
- What evidence is retained off-chain and what stable reference/hash belongs on the ledger?

Chapter 34 Implementation - Offline Artifact and Dependency Bundle

Implementation Procedure

166. Inventory hosts, static IPs, VLANs, ports, storage, clocks, and operators.
167. Build a checksummed offline artifact/model/package manifest.
168. Configure local DNS, time service, TLS trust, firewall policy, and persistent volumes.
169. Test with WAN physically/logically disabled.
170. Document backup, restore, replacement host, and offline update procedures.

Implementation Example

```
# Offline Artifact and Dependency Bundle
INPUT  -> authenticate, validate schema, bind requestId/nonce
FACTS  -> read authoritative wallet/merchant/state data
AI     -> optional bounded recommendation with decisionId/version
POLICY -> enforce ownership, hard limits, approvals, risk fallback
LEDGER -> submit one idempotent deterministic transaction when value changes
EVENT  -> update projection/case/monitoring asynchronously
VERIFY -> discover committed result and retain correlationId + txId evidence
```

Verification Checklist

- The artifact manifest exists and is version-controlled.
- The feature works with all public Internet/WAN connectivity disabled.
- Authorization, malformed input, duplicate retry, and failure behavior have explicit tests.
- If financial value changes, journal debits and credits balance exactly and historical entries remain immutable.
- If AI is involved, model/prompt/feature version and fallback behavior are recorded and deterministic policy remains final.
- A timeout/restart cannot create a duplicate financial result; request status can be rediscovered.
- Logs/evidence contain correlation references but not secrets or prohibited sensitive payment data.

Common Failure Modes

- Hidden public DNS/CDN/telemetry dependency.
- Clock drift that breaks certificates/audit ordering.
- Unpinned container/model/package versions.
- Backups stored on the same failure domain as production.

Expert Review Questions

- What happens if this component succeeds but the next component fails?
- How is the authorizing identity/device/key revoked or recovered?
- Can the operation be safely retried after the client loses the response?
- What happens if AI is unavailable, slow, or wrong?
- Can an auditor reconstruct debit, credit, actor, risk decision, approval, and Fabric txId from retained evidence?
- Which compensating transaction resolves a downstream operational error without rewriting history?

Chapter 35 - Container Runtime and Compose Baseline

Standardize service deployment while retaining offline reproducibility. An Internet-independent platform must own every runtime dependency: addressing, DNS, time, certificates, package artifacts, model files, monitoring, backup media, and operator procedures. If a service secretly requires cloud DNS, public telemetry, or a package registry, the design is not truly local.

Design rule: Use `pull_policy` never or equivalent

Implementation Objective

The required output is: Compose stack baseline. It must be version-controlled, reviewable, testable with the WAN disabled, and linked to the architecture/source-of-truth model. For a payment platform, every implementation decision should make it possible to explain who authorized an action, which facts were trusted, which controls ran, what financial journal committed, and how the result is recovered after failure.

Key Engineering Decisions

- Use `pull_policy` never or equivalent
- Mount persistent volumes intentionally
- Set resource limits and healthchecks
- Store configuration separately from secrets

Deep Implementation Guidance

Maintain a machine-readable inventory with hostname, IP, VLAN, OS, CPU/RAM/storage, ports, certificates, persistent volume paths, artifact versions, model hashes, backup paths, and responsible owner. Test every critical workflow after disabling WAN access.

Data and Control Questions

- What is the authoritative source for every input used by this component?
- Which user, device, service, MSP, or administrator identity is allowed to initiate or approve it?
- Does the operation move value, reserve value, create a side effect, or only provide advisory information?
- What idempotency/replay key proves that a retry is the same business intent?
- What evidence is retained off-chain and what stable reference/hash belongs on the ledger?

Chapter 35 Implementation - Container Runtime and Compose Baseline

Implementation Procedure

171. Inventory hosts, static IPs, VLANs, ports, storage, clocks, and operators.
172. Build a checksummed offline artifact/model/package manifest.
173. Configure local DNS, time service, TLS trust, firewall policy, and persistent volumes.
174. Test with WAN physically/logically disabled.
175. Document backup, restore, replacement host, and offline update procedures.

Implementation Example

```
# Container Runtime and Compose Baseline
INPUT  -> authenticate, validate schema, bind requestId/nonce
FACTS  -> read authoritative wallet/merchant/state data
AI     -> optional bounded recommendation with decisionId/version
POLICY -> enforce ownership, hard limits, approvals, risk fallback
LEDGER -> submit one idempotent deterministic transaction when value changes
EVENT  -> update projection/case/monitoring asynchronously
VERIFY -> discover committed result and retain correlationId + txId evidence
```

Verification Checklist

- The compose stack baseline exists and is version-controlled.
- The feature works with all public Internet/WAN connectivity disabled.
- Authorization, malformed input, duplicate retry, and failure behavior have explicit tests.
- If financial value changes, journal debits and credits balance exactly and historical entries remain immutable.
- If AI is involved, model/prompt/feature version and fallback behavior are recorded and deterministic policy remains final.
- A timeout/restart cannot create a duplicate financial result; request status can be rediscovered.
- Logs/evidence contain correlation references but not secrets or prohibited sensitive payment data.

Common Failure Modes

- Hidden public DNS/CDN/telemetry dependency.
- Clock drift that breaks certificates/audit ordering.
- Unpinned container/model/package versions.
- Backups stored on the same failure domain as production.

Expert Review Questions

- What happens if this component succeeds but the next component fails?
- How is the authorizing identity/device/key revoked or recovered?
- Can the operation be safely retried after the client loses the response?
- What happens if AI is unavailable, slow, or wrong?
- Can an auditor reconstruct debit, credit, actor, risk decision, approval, and Fabric txId from retained evidence?
- Which compensating transaction resolves a downstream operational error without rewriting history?

Chapter 36 - Internal TLS and PKI

Encrypt LAN service traffic and authenticate infrastructure endpoints. An Internet-independent platform must own every runtime dependency: addressing, DNS, time, certificates, package artifacts, model files, monitoring, backup media, and operator procedures. If a service secretly requires cloud DNS, public telemetry, or a package registry, the design is not truly local.

Design rule: Use internal CA/trust roots

Implementation Objective

The required output is: TLS certificate plan. It must be version-controlled, reviewable, testable with the WAN disabled, and linked to the architecture/source-of-truth model. For a payment platform, every implementation decision should make it possible to explain who authorized an action, which facts were trusted, which controls ran, what financial journal committed, and how the result is recovered after failure.

Key Engineering Decisions

- Use internal CA/trust roots
- Set SANs to internal DNS names
- Monitor expiry and rotate before emergency

Deep Implementation Guidance

Maintain a machine-readable inventory with hostname, IP, VLAN, OS, CPU/RAM/storage, ports, certificates, persistent volume paths, artifact versions, model hashes, backup paths, and responsible owner. Test every critical workflow after disabling WAN access.

Data and Control Questions

- What is the authoritative source for every input used by this component?
- Which user, device, service, MSP, or administrator identity is allowed to initiate or approve it?
- Does the operation move value, reserve value, create a side effect, or only provide advisory information?
- What idempotency/replay key proves that a retry is the same business intent?
- What evidence is retained off-chain and what stable reference/hash belongs on the ledger?

Chapter 36 Implementation - Internal TLS and PKI

Implementation Procedure

176. Inventory hosts, static IPs, VLANs, ports, storage, clocks, and operators.
177. Build a checksummed offline artifact/model/package manifest.
178. Configure local DNS, time service, TLS trust, firewall policy, and persistent volumes.
179. Test with WAN physically/logically disabled.
180. Document backup, restore, replacement host, and offline update procedures.

Implementation Example

```
# Internal TLS and PKI
INPUT  -> authenticate, validate schema, bind requestId/nonce
FACTS  -> read authoritative wallet/merchant/state data
AI      -> optional bounded recommendation with decisionId/version
POLICY  -> enforce ownership, hard limits, approvals, risk fallback
LEDGER  -> submit one idempotent deterministic transaction when value changes
EVENT   -> update projection/case/monitoring asynchronously
VERIFY  -> discover committed result and retain correlationId + txId evidence
```

Verification Checklist

- The tls certificate plan exists and is version-controlled.
- The feature works with all public Internet/WAN connectivity disabled.
- Authorization, malformed input, duplicate retry, and failure behavior have explicit tests.
- If financial value changes, journal debits and credits balance exactly and historical entries remain immutable.
- If AI is involved, model/prompt/feature version and fallback behavior are recorded and deterministic policy remains final.
- A timeout/restart cannot create a duplicate financial result; request status can be rediscovered.
- Logs/evidence contain correlation references but not secrets or prohibited sensitive payment data.

Common Failure Modes

- Hidden public DNS/CDN/telemetry dependency.
- Clock drift that breaks certificates/audit ordering.
- Unpinned container/model/package versions.
- Backups stored on the same failure domain as production.

Expert Review Questions

- What happens if this component succeeds but the next component fails?
- How is the authorizing identity/device/key revoked or recovered?
- Can the operation be safely retried after the client loses the response?
- What happens if AI is unavailable, slow, or wrong?
- Can an auditor reconstruct debit, credit, actor, risk decision, approval, and Fabric txId from retained evidence?
- Which compensating transaction resolves a downstream operational error without rewriting history?

Chapter 37 - Storage and Filesystem Layout

Protect ledger, database, model, log, and backup data with clear ownership. An Internet-independent platform must own every runtime dependency: addressing, DNS, time, certificates, package artifacts, model files, monitoring, backup media, and operator procedures. If a service secretly requires cloud DNS, public telemetry, or a package registry, the design is not truly local.

Design rule: Use durable volumes for Fabric production data and PostgreSQL

Implementation Objective

The required output is: Storage map. It must be version-controlled, reviewable, testable with the WAN disabled, and linked to the architecture/source-of-truth model. For a payment platform, every implementation decision should make it possible to explain who authorized an action, which facts were trusted, which controls ran, what financial journal committed, and how the result is recovered after failure.

Key Engineering Decisions

- Use durable volumes for Fabric production data and PostgreSQL
- Separate logs from critical data
- Monitor capacity/inodes

Deep Implementation Guidance

Maintain a machine-readable inventory with hostname, IP, VLAN, OS, CPU/RAM/storage, ports, certificates, persistent volume paths, artifact versions, model hashes, backup paths, and responsible owner. Test every critical workflow after disabling WAN access.

Data and Control Questions

- What is the authoritative source for every input used by this component?
- Which user, device, service, MSP, or administrator identity is allowed to initiate or approve it?
- Does the operation move value, reserve value, create a side effect, or only provide advisory information?
- What idempotency/replay key proves that a retry is the same business intent?
- What evidence is retained off-chain and what stable reference/hash belongs on the ledger?

Chapter 37 Implementation - Storage and Filesystem Layout

Implementation Procedure

181. Inventory hosts, static IPs, VLANs, ports, storage, clocks, and operators.
182. Build a checksummed offline artifact/model/package manifest.
183. Configure local DNS, time service, TLS trust, firewall policy, and persistent volumes.
184. Test with WAN physically/logically disabled.
185. Document backup, restore, replacement host, and offline update procedures.

Implementation Example

```
# Storage and Filesystem Layout
INPUT  -> authenticate, validate schema, bind requestId/nonce
FACTS  -> read authoritative wallet/merchant/state data
AI      -> optional bounded recommendation with decisionId/version
POLICY  -> enforce ownership, hard limits, approvals, risk fallback
LEDGER  -> submit one idempotent deterministic transaction when value changes
EVENT   -> update projection/case/monitoring asynchronously
VERIFY  -> discover committed result and retain correlationId + txId evidence
```

Verification Checklist

- The storage map exists and is version-controlled.
- The feature works with all public Internet/WAN connectivity disabled.
- Authorization, malformed input, duplicate retry, and failure behavior have explicit tests.
- If financial value changes, journal debits and credits balance exactly and historical entries remain immutable.
- If AI is involved, model/prompt/feature version and fallback behavior are recorded and deterministic policy remains final.
- A timeout/restart cannot create a duplicate financial result; request status can be rediscovered.
- Logs/evidence contain correlation references but not secrets or prohibited sensitive payment data.

Common Failure Modes

- Hidden public DNS/CDN/telemetry dependency.
- Clock drift that breaks certificates/audit ordering.
- Unpinned container/model/package versions.
- Backups stored on the same failure domain as production.

Expert Review Questions

- What happens if this component succeeds but the next component fails?
- How is the authorizing identity/device/key revoked or recovered?
- Can the operation be safely retried after the client loses the response?
- What happens if AI is unavailable, slow, or wrong?
- Can an auditor reconstruct debit, credit, actor, risk decision, approval, and Fabric txId from retained evidence?
- Which compensating transaction resolves a downstream operational error without rewriting history?

Chapter 38 - Backup Vault and Media Handling

Store recovery artifacts outside the main failure domain. An Internet-independent platform must own every runtime dependency: addressing, DNS, time, certificates, package artifacts, model files, monitoring, backup media, and operator procedures. If a service secretly requires cloud DNS, public telemetry, or a package registry, the design is not truly local.

Design rule: Encrypt sensitive backups

Implementation Objective

The required output is: Backup design. It must be version-controlled, reviewable, testable with the WAN disabled, and linked to the architecture/source-of-truth model. For a payment platform, every implementation decision should make it possible to explain who authorized an action, which facts were trusted, which controls ran, what financial journal committed, and how the result is recovered after failure.

Key Engineering Decisions

- Encrypt sensitive backups
- Maintain offline/offsite copies as required
- Record hashes and restore instructions
- Test media readability

Deep Implementation Guidance

Maintain a machine-readable inventory with hostname, IP, VLAN, OS, CPU/RAM/storage, ports, certificates, persistent volume paths, artifact versions, model hashes, backup paths, and responsible owner. Test every critical workflow after disabling WAN access.

Data and Control Questions

- What is the authoritative source for every input used by this component?
- Which user, device, service, MSP, or administrator identity is allowed to initiate or approve it?
- Does the operation move value, reserve value, create a side effect, or only provide advisory information?
- What idempotency/replay key proves that a retry is the same business intent?
- What evidence is retained off-chain and what stable reference/hash belongs on the ledger?

Chapter 38 Implementation - Backup Vault and Media Handling

Implementation Procedure

186. Inventory hosts, static IPs, VLANs, ports, storage, clocks, and operators.
187. Build a checksummed offline artifact/model/package manifest.
188. Configure local DNS, time service, TLS trust, firewall policy, and persistent volumes.
189. Test with WAN physically/logically disabled.
190. Document backup, restore, replacement host, and offline update procedures.

Implementation Example

```
# Backup Vault and Media Handling
INPUT  -> authenticate, validate schema, bind requestId/nonce
FACTS  -> read authoritative wallet/merchant/state data
AI      -> optional bounded recommendation with decisionId/version
POLICY  -> enforce ownership, hard limits, approvals, risk fallback
LEDGER  -> submit one idempotent deterministic transaction when value changes
EVENT   -> update projection/case/monitoring asynchronously
VERIFY  -> discover committed result and retain correlationId + txId evidence
```

Verification Checklist

- The backup design exists and is version-controlled.
- The feature works with all public Internet/WAN connectivity disabled.
- Authorization, malformed input, duplicate retry, and failure behavior have explicit tests.
- If financial value changes, journal debits and credits balance exactly and historical entries remain immutable.
- If AI is involved, model/prompt/feature version and fallback behavior are recorded and deterministic policy remains final.
- A timeout/restart cannot create a duplicate financial result; request status can be rediscovered.
- Logs/evidence contain correlation references but not secrets or prohibited sensitive payment data.

Common Failure Modes

- Hidden public DNS/CDN/telemetry dependency.
- Clock drift that breaks certificates/audit ordering.
- Unpinned container/model/package versions.
- Backups stored on the same failure domain as production.

Expert Review Questions

- What happens if this component succeeds but the next component fails?
- How is the authorizing identity/device/key revoked or recovered?
- Can the operation be safely retried after the client loses the response?
- What happens if AI is unavailable, slow, or wrong?
- Can an auditor reconstruct debit, credit, actor, risk decision, approval, and Fabric txId from retained evidence?
- Which compensating transaction resolves a downstream operational error without rewriting history?

Chapter 39 - Local Monitoring Stack

Observe service health without external SaaS. An Internet-independent platform must own every runtime dependency: addressing, DNS, time, certificates, package artifacts, model files, monitoring, backup media, and operator procedures. If a service secretly requires cloud DNS, public telemetry, or a package registry, the design is not truly local.

Design rule: Collect host/container/Fabric/API/DB/AI metrics locally

Implementation Objective

The required output is: Monitoring architecture. It must be version-controlled, reviewable, testable with the WAN disabled, and linked to the architecture/source-of-truth model. For a payment platform, every implementation decision should make it possible to explain who authorized an action, which facts were trusted, which controls ran, what financial journal committed, and how the result is recovered after failure.

Key Engineering Decisions

- Collect host/container/Fabric/API/DB/AI metrics locally
- Store alert rules locally
- Retain enough logs for incident reconstruction

Deep Implementation Guidance

Maintain a machine-readable inventory with hostname, IP, VLAN, OS, CPU/RAM/storage, ports, certificates, persistent volume paths, artifact versions, model hashes, backup paths, and responsible owner. Test every critical workflow after disabling WAN access.

Data and Control Questions

- What is the authoritative source for every input used by this component?
- Which user, device, service, MSP, or administrator identity is allowed to initiate or approve it?
- Does the operation move value, reserve value, create a side effect, or only provide advisory information?
- What idempotency/replay key proves that a retry is the same business intent?
- What evidence is retained off-chain and what stable reference/hash belongs on the ledger?

Chapter 39 Implementation - Local Monitoring Stack

Implementation Procedure

191. Inventory hosts, static IPs, VLANs, ports, storage, clocks, and operators.
192. Build a checksummed offline artifact/model/package manifest.
193. Configure local DNS, time service, TLS trust, firewall policy, and persistent volumes.
194. Test with WAN physically/logically disabled.
195. Document backup, restore, replacement host, and offline update procedures.

Implementation Example

```
# Local Monitoring Stack
INPUT  -> authenticate, validate schema, bind requestId/nonce
FACTS  -> read authoritative wallet/merchant/state data
AI     -> optional bounded recommendation with decisionId/version
POLICY -> enforce ownership, hard limits, approvals, risk fallback
LEDGER -> submit one idempotent deterministic transaction when value changes
EVENT  -> update projection/case/monitoring asynchronously
VERIFY -> discover committed result and retain correlationId + txId evidence
```

Verification Checklist

- The monitoring architecture exists and is version-controlled.
- The feature works with all public Internet/WAN connectivity disabled.
- Authorization, malformed input, duplicate retry, and failure behavior have explicit tests.
- If financial value changes, journal debits and credits balance exactly and historical entries remain immutable.
- If AI is involved, model/prompt/feature version and fallback behavior are recorded and deterministic policy remains final.
- A timeout/restart cannot create a duplicate financial result; request status can be rediscovered.
- Logs/evidence contain correlation references but not secrets or prohibited sensitive payment data.

Common Failure Modes

- Hidden public DNS/CDN/telemetry dependency.
- Clock drift that breaks certificates/audit ordering.
- Unpinned container/model/package versions.
- Backups stored on the same failure domain as production.

Expert Review Questions

- What happens if this component succeeds but the next component fails?
- How is the authorizing identity/device/key revoked or recovered?
- Can the operation be safely retried after the client loses the response?
- What happens if AI is unavailable, slow, or wrong?
- Can an auditor reconstruct debit, credit, actor, risk decision, approval, and Fabric txId from retained evidence?
- Which compensating transaction resolves a downstream operational error without rewriting history?

Chapter 40 - Local Logging and Correlation

Create one traceable evidence chain from user request to AI decision to ledger commit. An Internet-independent platform must own every runtime dependency: addressing, DNS, time, certificates, package artifacts, model files, monitoring, backup media, and operator procedures. If a service secretly requires cloud DNS, public telemetry, or a package registry, the design is not truly local.

Design rule: Use correlationId/requestId/decisionId/approvalId/txId

Implementation Objective

The required output is: Logging standard. It must be version-controlled, reviewable, testable with the WAN disabled, and linked to the architecture/source-of-truth model. For a payment platform, every implementation decision should make it possible to explain who authorized an action, which facts were trusted, which controls ran, what financial journal committed, and how the result is recovered after failure.

Key Engineering Decisions

- Use correlationId/requestId/decisionId/approvalId/txId
- Time-sync logs
- Redact secrets and sensitive payment fields

Deep Implementation Guidance

Maintain a machine-readable inventory with hostname, IP, VLAN, OS, CPU/RAM/storage, ports, certificates, persistent volume paths, artifact versions, model hashes, backup paths, and responsible owner. Test every critical workflow after disabling WAN access.

Data and Control Questions

- What is the authoritative source for every input used by this component?
- Which user, device, service, MSP, or administrator identity is allowed to initiate or approve it?
- Does the operation move value, reserve value, create a side effect, or only provide advisory information?
- What idempotency/replay key proves that a retry is the same business intent?
- What evidence is retained off-chain and what stable reference/hash belongs on the ledger?

Chapter 40 Implementation - Local Logging and Correlation

Implementation Procedure

196. Inventory hosts, static IPs, VLANs, ports, storage, clocks, and operators.
197. Build a checksummed offline artifact/model/package manifest.
198. Configure local DNS, time service, TLS trust, firewall policy, and persistent volumes.
199. Test with WAN physically/logically disabled.
200. Document backup, restore, replacement host, and offline update procedures.

Implementation Example

```
# Local Logging and Correlation
INPUT  -> authenticate, validate schema, bind requestId/nonce
FACTS  -> read authoritative wallet/merchant/state data
AI      -> optional bounded recommendation with decisionId/version
POLICY  -> enforce ownership, hard limits, approvals, risk fallback
LEDGER  -> submit one idempotent deterministic transaction when value changes
EVENT   -> update projection/case/monitoring asynchronously
VERIFY  -> discover committed result and retain correlationId + txId evidence
```

Verification Checklist

- The logging standard exists and is version-controlled.
- The feature works with all public Internet/WAN connectivity disabled.
- Authorization, malformed input, duplicate retry, and failure behavior have explicit tests.
- If financial value changes, journal debits and credits balance exactly and historical entries remain immutable.
- If AI is involved, model/prompt/feature version and fallback behavior are recorded and deterministic policy remains final.
- A timeout/restart cannot create a duplicate financial result; request status can be rediscovered.
- Logs/evidence contain correlation references but not secrets or prohibited sensitive payment data.

Common Failure Modes

- Hidden public DNS/CDN/telemetry dependency.
- Clock drift that breaks certificates/audit ordering.
- Unpinned container/model/package versions.
- Backups stored on the same failure domain as production.

Expert Review Questions

- What happens if this component succeeds but the next component fails?
- How is the authorizing identity/device/key revoked or recovered?
- Can the operation be safely retried after the client loses the response?
- What happens if AI is unavailable, slow, or wrong?
- Can an auditor reconstruct debit, credit, actor, risk decision, approval, and Fabric txId from retained evidence?
- Which compensating transaction resolves a downstream operational error without rewriting history?

Chapter 41 - Capacity and Resource Budget

Allocate CPU/RAM/storage for transaction and AI workloads. An Internet-independent platform must own every runtime dependency: addressing, DNS, time, certificates, package artifacts, model files, monitoring, backup media, and operator procedures. If a service secretly requires cloud DNS, public telemetry, or a package registry, the design is not truly local.

Design rule: Reserve critical ledger resources

Implementation Objective

The required output is: Capacity worksheet. It must be version-controlled, reviewable, testable with the WAN disabled, and linked to the architecture/source-of-truth model. For a payment platform, every implementation decision should make it possible to explain who authorized an action, which facts were trusted, which controls ran, what financial journal committed, and how the result is recovered after failure.

Key Engineering Decisions

- Reserve critical ledger resources
- Keep LLM workloads from starving Fabric/API
- Define storage growth assumptions

Deep Implementation Guidance

Maintain a machine-readable inventory with hostname, IP, VLAN, OS, CPU/RAM/storage, ports, certificates, persistent volume paths, artifact versions, model hashes, backup paths, and responsible owner. Test every critical workflow after disabling WAN access.

Data and Control Questions

- What is the authoritative source for every input used by this component?
- Which user, device, service, MSP, or administrator identity is allowed to initiate or approve it?
- Does the operation move value, reserve value, create a side effect, or only provide advisory information?
- What idempotency/replay key proves that a retry is the same business intent?
- What evidence is retained off-chain and what stable reference/hash belongs on the ledger?

Chapter 41 Implementation - Capacity and Resource Budget

Implementation Procedure

201. Inventory hosts, static IPs, VLANs, ports, storage, clocks, and operators.
202. Build a checksummed offline artifact/model/package manifest.
203. Configure local DNS, time service, TLS trust, firewall policy, and persistent volumes.
204. Test with WAN physically/logically disabled.
205. Document backup, restore, replacement host, and offline update procedures.

Implementation Example

```
# Capacity and Resource Budget
INPUT  -> authenticate, validate schema, bind requestId/nonce
FACTS  -> read authoritative wallet/merchant/state data
AI     -> optional bounded recommendation with decisionId/version
POLICY -> enforce ownership, hard limits, approvals, risk fallback
LEDGER -> submit one idempotent deterministic transaction when value changes
EVENT  -> update projection/case/monitoring asynchronously
VERIFY -> discover committed result and retain correlationId + txId evidence
```

Verification Checklist

- The capacity worksheet exists and is version-controlled.
- The feature works with all public Internet/WAN connectivity disabled.
- Authorization, malformed input, duplicate retry, and failure behavior have explicit tests.
- If financial value changes, journal debits and credits balance exactly and historical entries remain immutable.
- If AI is involved, model/prompt/feature version and fallback behavior are recorded and deterministic policy remains final.
- A timeout/restart cannot create a duplicate financial result; request status can be rediscovered.
- Logs/evidence contain correlation references but not secrets or prohibited sensitive payment data.

Common Failure Modes

- Hidden public DNS/CDN/telemetry dependency.
- Clock drift that breaks certificates/audit ordering.
- Unpinned container/model/package versions.
- Backups stored on the same failure domain as production.

Expert Review Questions

- What happens if this component succeeds but the next component fails?
- How is the authorizing identity/device/key revoked or recovered?
- Can the operation be safely retried after the client loses the response?
- What happens if AI is unavailable, slow, or wrong?
- Can an auditor reconstruct debit, credit, actor, risk decision, approval, and Fabric txId from retained evidence?
- Which compensating transaction resolves a downstream operational error without rewriting history?

Chapter 42 - Offline Update Staging Zone

Prevent unverified media from going directly to production. An Internet-independent platform must own every runtime dependency: addressing, DNS, time, certificates, package artifacts, model files, monitoring, backup media, and operator procedures. If a service secretly requires cloud DNS, public telemetry, or a package registry, the design is not truly local.

Design rule: Acquire externally on controlled workstation

Implementation Objective

The required output is: Staging procedure. It must be version-controlled, reviewable, testable with the WAN disabled, and linked to the architecture/source-of-truth model. For a payment platform, every implementation decision should make it possible to explain who authorized an action, which facts were trusted, which controls ran, what financial journal committed, and how the result is recovered after failure.

Key Engineering Decisions

- Acquire externally on controlled workstation
- Verify signatures/checksums
- Malware scan
- Test in isolated staging LAN
- Promote via signed internal manifest

Deep Implementation Guidance

Maintain a machine-readable inventory with hostname, IP, VLAN, OS, CPU/RAM/storage, ports, certificates, persistent volume paths, artifact versions, model hashes, backup paths, and responsible owner. Test every critical workflow after disabling WAN access.

Data and Control Questions

- What is the authoritative source for every input used by this component?
- Which user, device, service, MSP, or administrator identity is allowed to initiate or approve it?
- Does the operation move value, reserve value, create a side effect, or only provide advisory information?
- What idempotency/replay key proves that a retry is the same business intent?
- What evidence is retained off-chain and what stable reference/hash belongs on the ledger?

Chapter 42 Implementation - Offline Update Staging Zone

Implementation Procedure

206. Inventory hosts, static IPs, VLANs, ports, storage, clocks, and operators.
207. Build a checksummed offline artifact/model/package manifest.
208. Configure local DNS, time service, TLS trust, firewall policy, and persistent volumes.
209. Test with WAN physically/logically disabled.
210. Document backup, restore, replacement host, and offline update procedures.

Implementation Example

```
# Offline Update Staging Zone
INPUT  -> authenticate, validate schema, bind requestId/nonce
FACTS  -> read authoritative wallet/merchant/state data
AI     -> optional bounded recommendation with decisionId/version
POLICY -> enforce ownership, hard limits, approvals, risk fallback
LEDGER -> submit one idempotent deterministic transaction when value changes
EVENT  -> update projection/case/monitoring asynchronously
VERIFY -> discover committed result and retain correlationId + txId evidence
```

Verification Checklist

- The staging procedure exists and is version-controlled.
- The feature works with all public Internet/WAN connectivity disabled.
- Authorization, malformed input, duplicate retry, and failure behavior have explicit tests.
- If financial value changes, journal debits and credits balance exactly and historical entries remain immutable.
- If AI is involved, model/prompt/feature version and fallback behavior are recorded and deterministic policy remains final.
- A timeout/restart cannot create a duplicate financial result; request status can be rediscovered.
- Logs/evidence contain correlation references but not secrets or prohibited sensitive payment data.

Common Failure Modes

- Hidden public DNS/CDN/telemetry dependency.
- Clock drift that breaks certificates/audit ordering.
- Unpinned container/model/package versions.
- Backups stored on the same failure domain as production.

Expert Review Questions

- What happens if this component succeeds but the next component fails?
- How is the authorizing identity/device/key revoked or recovered?
- Can the operation be safely retried after the client loses the response?
- What happens if AI is unavailable, slow, or wrong?
- Can an auditor reconstruct debit, credit, actor, risk decision, approval, and Fabric txId from retained evidence?
- Which compensating transaction resolves a downstream operational error without rewriting history?

Chapter 43 - High Availability Baseline

Define which local failures must not stop payments. An Internet-independent platform must own every runtime dependency: addressing, DNS, time, certificates, package artifacts, model files, monitoring, backup media, and operator procedures. If a service secretly requires cloud DNS, public telemetry, or a package registry, the design is not truly local.

Design rule: Three Raft orderers for crash-fault tolerance

Implementation Objective

The required output is: HA decision matrix. It must be version-controlled, reviewable, testable with the WAN disabled, and linked to the architecture/source-of-truth model. For a payment platform, every implementation decision should make it possible to explain who authorized an action, which facts were trusted, which controls ran, what financial journal committed, and how the result is recovered after failure.

Key Engineering Decisions

- Three Raft orderers for crash-fault tolerance
- Multiple API instances as needed
- Database HA according to RTO
- AI may degrade to deterministic rules

Deep Implementation Guidance

Maintain a machine-readable inventory with hostname, IP, VLAN, OS, CPU/RAM/storage, ports, certificates, persistent volume paths, artifact versions, model hashes, backup paths, and responsible owner. Test every critical workflow after disabling WAN access.

Data and Control Questions

- What is the authoritative source for every input used by this component?
- Which user, device, service, MSP, or administrator identity is allowed to initiate or approve it?
- Does the operation move value, reserve value, create a side effect, or only provide advisory information?
- What idempotency/replay key proves that a retry is the same business intent?
- What evidence is retained off-chain and what stable reference/hash belongs on the ledger?

Chapter 43 Implementation - High Availability Baseline

Implementation Procedure

211. Inventory hosts, static IPs, VLANs, ports, storage, clocks, and operators.
212. Build a checksummed offline artifact/model/package manifest.
213. Configure local DNS, time service, TLS trust, firewall policy, and persistent volumes.
214. Test with WAN physically/logically disabled.
215. Document backup, restore, replacement host, and offline update procedures.

Implementation Example

```
# High Availability Baseline
INPUT  -> authenticate, validate schema, bind requestId/nonce
FACTS  -> read authoritative wallet/merchant/state data
AI      -> optional bounded recommendation with decisionId/version
POLICY  -> enforce ownership, hard limits, approvals, risk fallback
LEDGER  -> submit one idempotent deterministic transaction when value changes
EVENT   -> update projection/case/monitoring asynchronously
VERIFY  -> discover committed result and retain correlationId + txId evidence
```

Verification Checklist

- The ha decision matrix exists and is version-controlled.
- The feature works with all public Internet/WAN connectivity disabled.
- Authorization, malformed input, duplicate retry, and failure behavior have explicit tests.
- If financial value changes, journal debits and credits balance exactly and historical entries remain immutable.
- If AI is involved, model/prompt/feature version and fallback behavior are recorded and deterministic policy remains final.
- A timeout/restart cannot create a duplicate financial result; request status can be rediscovered.
- Logs/evidence contain correlation references but not secrets or prohibited sensitive payment data.

Common Failure Modes

- Hidden public DNS/CDN/telemetry dependency.
- Clock drift that breaks certificates/audit ordering.
- Unpinned container/model/package versions.
- Backups stored on the same failure domain as production.

Expert Review Questions

- What happens if this component succeeds but the next component fails?
- How is the authorizing identity/device/key revoked or recovered?
- Can the operation be safely retried after the client loses the response?
- What happens if AI is unavailable, slow, or wrong?
- Can an auditor reconstruct debit, credit, actor, risk decision, approval, and Fabric txId from retained evidence?
- Which compensating transaction resolves a downstream operational error without rewriting history?

Chapter 44 - Disaster-Recovery Site Option

Plan recovery to spare hardware or second local site without Internet. An Internet-independent platform must own every runtime dependency: addressing, DNS, time, certificates, package artifacts, model files, monitoring, backup media, and operator procedures. If a service secretly requires cloud DNS, public telemetry, or a package registry, the design is not truly local.

Design rule: Maintain documented cold/warm spare

Implementation Objective

The required output is: DR architecture. It must be version-controlled, reviewable, testable with the WAN disabled, and linked to the architecture/source-of-truth model. For a payment platform, every implementation decision should make it possible to explain who authorized an action, which facts were trusted, which controls ran, what financial journal committed, and how the result is recovered after failure.

Key Engineering Decisions

- Maintain documented cold/warm spare
- Replicate/transport encrypted backups
- Keep independent recovery credentials and network plan

Deep Implementation Guidance

Maintain a machine-readable inventory with hostname, IP, VLAN, OS, CPU/RAM/storage, ports, certificates, persistent volume paths, artifact versions, model hashes, backup paths, and responsible owner. Test every critical workflow after disabling WAN access.

Data and Control Questions

- What is the authoritative source for every input used by this component?
- Which user, device, service, MSP, or administrator identity is allowed to initiate or approve it?
- Does the operation move value, reserve value, create a side effect, or only provide advisory information?
- What idempotency/replay key proves that a retry is the same business intent?
- What evidence is retained off-chain and what stable reference/hash belongs on the ledger?

Chapter 44 Implementation - Disaster-Recovery Site Option

Implementation Procedure

216. Inventory hosts, static IPs, VLANs, ports, storage, clocks, and operators.
217. Build a checksummed offline artifact/model/package manifest.
218. Configure local DNS, time service, TLS trust, firewall policy, and persistent volumes.
219. Test with WAN physically/logically disabled.
220. Document backup, restore, replacement host, and offline update procedures.

Implementation Example

```
# Disaster-Recovery Site Option
INPUT  -> authenticate, validate schema, bind requestId/nonce
FACTS  -> read authoritative wallet/merchant/state data
AI      -> optional bounded recommendation with decisionId/version
POLICY  -> enforce ownership, hard limits, approvals, risk fallback
LEDGER  -> submit one idempotent deterministic transaction when value changes
EVENT   -> update projection/case/monitoring asynchronously
VERIFY  -> discover committed result and retain correlationId + txId evidence
```

Verification Checklist

- The dr architecture exists and is version-controlled.
- The feature works with all public Internet/WAN connectivity disabled.
- Authorization, malformed input, duplicate retry, and failure behavior have explicit tests.
- If financial value changes, journal debits and credits balance exactly and historical entries remain immutable.
- If AI is involved, model/prompt/feature version and fallback behavior are recorded and deterministic policy remains final.
- A timeout/restart cannot create a duplicate financial result; request status can be rediscovered.
- Logs/evidence contain correlation references but not secrets or prohibited sensitive payment data.

Common Failure Modes

- Hidden public DNS/CDN/telemetry dependency.
- Clock drift that breaks certificates/audit ordering.
- Unpinned container/model/package versions.
- Backups stored on the same failure domain as production.

Expert Review Questions

- What happens if this component succeeds but the next component fails?
- How is the authorizing identity/device/key revoked or recovered?
- Can the operation be safely retried after the client loses the response?
- What happens if AI is unavailable, slow, or wrong?
- Can an auditor reconstruct debit, credit, actor, risk decision, approval, and Fabric txId from retained evidence?
- Which compensating transaction resolves a downstream operational error without rewriting history?

Part IV - Hyperledger Fabric Network Implementation

Hyperledger Fabric provides permissioned identities, endorsement policies, deterministic transaction validation, immutable block history, and current world state. Treat Fabric configuration as versioned infrastructure code and separate the administrative domains that are supposed to provide independent assurance.

Part focus: This part contains 18 implementation chapters. Each chapter includes design rules, implementation procedure, a practical artifact, verification, failure analysis, and expert review questions.

Chapter 45 - Fabric Concepts Mapped to Payments

Explain channels, peers, orderers, MSPs, chaincode, world state, and history in financial terms.

Hyperledger Fabric provides permissioned identities, endorsement policies, deterministic transaction validation, immutable block history, and current world state. Treat Fabric configuration as versioned infrastructure code and separate the administrative domains that are supposed to provide independent assurance.

Design rule: Peers validate and store payment ledger

Implementation Objective

The required output is: Fabric concept map. It must be version-controlled, reviewable, testable with the WAN disabled, and linked to the architecture/source-of-truth model. For a payment platform, every implementation decision should make it possible to explain who authorized an action, which facts were trusted, which controls ran, what financial journal committed, and how the result is recovered after failure.

Key Engineering Decisions

- Peers validate and store payment ledger
- Orderers establish final transaction order
- MSPs identify organizations
- Chaincode enforces accounting rules

Deep Implementation Guidance

Store Fabric configuration as code. The release evidence should tie chaincode source commit to package ID, lifecycle sequence, organization approvals, channel policy, container/binary version, and test results.

Data and Control Questions

- What is the authoritative source for every input used by this component?
- Which user, device, service, MSP, or administrator identity is allowed to initiate or approve it?
- Does the operation move value, reserve value, create a side effect, or only provide advisory information?
- What idempotency/replay key proves that a retry is the same business intent?
- What evidence is retained off-chain and what stable reference/hash belongs on the ledger?

Chapter 45 Implementation - Fabric Concepts Mapped to Payments

Implementation Procedure

221. Version all Fabric configuration and scripts.
222. Enroll/issue identities from local CAs and protect private keys.
223. Create Raft ordering, peers, channel, policies, and lifecycle approvals.
224. Deploy/test chaincode through Fabric Gateway.
225. Test quorum, peer loss, revocation, and upgrade procedures.

Implementation Example

```
# Fabric Concepts Mapped to Payments
INPUT  -> authenticate, validate schema, bind requestId/nonce
FACTS  -> read authoritative wallet/merchant/state data
AI     -> optional bounded recommendation with decisionId/version
POLICY -> enforce ownership, hard limits, approvals, risk fallback
LEDGER -> submit one idempotent deterministic transaction when value changes
EVENT  -> update projection/case/monitoring asynchronously
VERIFY -> discover committed result and retain correlationId + txId evidence
```

Verification Checklist

- The fabric concept map exists and is version-controlled.
- The feature works with all public Internet/WAN connectivity disabled.
- Authorization, malformed input, duplicate retry, and failure behavior have explicit tests.
- If financial value changes, journal debits and credits balance exactly and historical entries remain immutable.
- If AI is involved, model/prompt/feature version and fallback behavior are recorded and deterministic policy remains final.
- A timeout/restart cannot create a duplicate financial result; request status can be rediscovered.
- Logs/evidence contain correlation references but not secrets or prohibited sensitive payment data.

Common Failure Modes

- One administrator owns every trust domain/key.
- Incorrect endorsement policies.
- Certificate expiry not monitored.
- Quorum behavior never tested.

Expert Review Questions

- What happens if this component succeeds but the next component fails?
- How is the authorizing identity/device/key revoked or recovered?
- Can the operation be safely retried after the client loses the response?
- What happens if AI is unavailable, slow, or wrong?
- Can an auditor reconstruct debit, credit, actor, risk decision, approval, and Fabric txId from retained evidence?
- Which compensating transaction resolves a downstream operational error without rewriting history?

Chapter 46 - Fabric 2.5 LTS Baseline

Pin a conservative Fabric production baseline and document upgrade criteria. Hyperledger Fabric provides permissioned identities, endorsement policies, deterministic transaction validation, immutable block history, and current world state. Treat Fabric configuration as versioned infrastructure code and separate the administrative domains that are supposed to provide independent assurance.

Design rule: Use approved 2.5.x LTS packages

Implementation Objective

The required output is: Version policy. It must be version-controlled, reviewable, testable with the WAN disabled, and linked to the architecture/source-of-truth model. For a payment platform, every implementation decision should make it possible to explain who authorized an action, which facts were trusted, which controls ran, what financial journal committed, and how the result is recovered after failure.

Key Engineering Decisions

- Use approved 2.5.x LTS packages
- Record Fabric CA version
- Pin container digests
- Evaluate 3.x migration separately

Deep Implementation Guidance

Store Fabric configuration as code. The release evidence should tie chaincode source commit to package ID, lifecycle sequence, organization approvals, channel policy, container/binary version, and test results.

Data and Control Questions

- What is the authoritative source for every input used by this component?
- Which user, device, service, MSP, or administrator identity is allowed to initiate or approve it?
- Does the operation move value, reserve value, create a side effect, or only provide advisory information?
- What idempotency/replay key proves that a retry is the same business intent?
- What evidence is retained off-chain and what stable reference/hash belongs on the ledger?

Chapter 46 Implementation - Fabric 2.5 LTS Baseline

Implementation Procedure

226. Version all Fabric configuration and scripts.
227. Enroll/issue identities from local CAs and protect private keys.
228. Create Raft ordering, peers, channel, policies, and lifecycle approvals.
229. Deploy/test chaincode through Fabric Gateway.
230. Test quorum, peer loss, revocation, and upgrade procedures.

Implementation Example

```
# Fabric 2.5 LTS Baseline
INPUT  -> authenticate, validate schema, bind requestId/nonce
FACTS  -> read authoritative wallet/merchant/state data
AI     -> optional bounded recommendation with decisionId/version
POLICY -> enforce ownership, hard limits, approvals, risk fallback
LEDGER -> submit one idempotent deterministic transaction when value changes
EVENT  -> update projection/case/monitoring asynchronously
VERIFY -> discover committed result and retain correlationId + txId evidence
```

Verification Checklist

- The version policy exists and is version-controlled.
- The feature works with all public Internet/WAN connectivity disabled.
- Authorization, malformed input, duplicate retry, and failure behavior have explicit tests.
- If financial value changes, journal debits and credits balance exactly and historical entries remain immutable.
- If AI is involved, model/prompt/feature version and fallback behavior are recorded and deterministic policy remains final.
- A timeout/restart cannot create a duplicate financial result; request status can be rediscovered.
- Logs/evidence contain correlation references but not secrets or prohibited sensitive payment data.

Common Failure Modes

- One administrator owns every trust domain/key.
- Incorrect endorsement policies.
- Certificate expiry not monitored.
- Quorum behavior never tested.

Expert Review Questions

- What happens if this component succeeds but the next component fails?
- How is the authorizing identity/device/key revoked or recovered?
- Can the operation be safely retried after the client loses the response?
- What happens if AI is unavailable, slow, or wrong?
- Can an auditor reconstruct debit, credit, actor, risk decision, approval, and Fabric txId from retained evidence?
- Which compensating transaction resolves a downstream operational error without rewriting history?

Chapter 47 - WalletOrg MSP and CA

Create identities for wallet backend, wallet peers, admins, and operators. Hyperledger Fabric provides permissioned identities, endorsement policies, deterministic transaction validation, immutable block history, and current world state. Treat Fabric configuration as versioned infrastructure code and separate the administrative domains that are supposed to provide independent assurance.

Design rule: Separate enrollment/admin roles

Implementation Objective

The required output is: WalletOrg identity tree. It must be version-controlled, reviewable, testable with the WAN disabled, and linked to the architecture/source-of-truth model. For a payment platform, every implementation decision should make it possible to explain who authorized an action, which facts were trusted, which controls ran, what financial journal committed, and how the result is recovered after failure.

Key Engineering Decisions

- Separate enrollment/admin roles
- Issue least-privilege client identities
- Protect CA admin credentials offline

Deep Implementation Guidance

Store Fabric configuration as code. The release evidence should tie chaincode source commit to package ID, lifecycle sequence, organization approvals, channel policy, container/binary version, and test results.

Data and Control Questions

- What is the authoritative source for every input used by this component?
- Which user, device, service, MSP, or administrator identity is allowed to initiate or approve it?
- Does the operation move value, reserve value, create a side effect, or only provide advisory information?
- What idempotency/replay key proves that a retry is the same business intent?
- What evidence is retained off-chain and what stable reference/hash belongs on the ledger?

Chapter 47 Implementation - WalletOrg MSP and CA

Implementation Procedure

231. Version all Fabric configuration and scripts.
232. Enroll/issue identities from local CAs and protect private keys.
233. Create Raft ordering, peers, channel, policies, and lifecycle approvals.
234. Deploy/test chaincode through Fabric Gateway.
235. Test quorum, peer loss, revocation, and upgrade procedures.

Implementation Example

```
# WalletOrg MSP and CA
INPUT -> authenticate, validate schema, bind requestId/nonce
FACTS -> read authoritative wallet/merchant/state data
AI -> optional bounded recommendation with decisionId/version
POLICY -> enforce ownership, hard limits, approvals, risk fallback
LEDGER -> submit one idempotent deterministic transaction when value changes
EVENT -> update projection/case/monitoring asynchronously
VERIFY -> discover committed result and retain correlationId + txId evidence
```

Verification Checklist

- The walletorg identity tree exists and is version-controlled.
- The feature works with all public Internet/WAN connectivity disabled.
- Authorization, malformed input, duplicate retry, and failure behavior have explicit tests.
- If financial value changes, journal debits and credits balance exactly and historical entries remain immutable.
- If AI is involved, model/prompt/feature version and fallback behavior are recorded and deterministic policy remains final.
- A timeout/restart cannot create a duplicate financial result; request status can be rediscovered.
- Logs/evidence contain correlation references but not secrets or prohibited sensitive payment data.

Common Failure Modes

- One administrator owns every trust domain/key.
- Incorrect endorsement policies.
- Certificate expiry not monitored.
- Quorum behavior never tested.

Expert Review Questions

- What happens if this component succeeds but the next component fails?
- How is the authorizing identity/device/key revoked or recovered?
- Can the operation be safely retried after the client loses the response?
- What happens if AI is unavailable, slow, or wrong?
- Can an auditor reconstruct debit, credit, actor, risk decision, approval, and Fabric txId from retained evidence?
- Which compensating transaction resolves a downstream operational error without rewriting history?

Chapter 48 - MerchantOrg MSP and CA

Create merchant network identities and peers independently of WalletOrg. Hyperledger Fabric provides permissioned identities, endorsement policies, deterministic transaction validation, immutable block history, and current world state. Treat Fabric configuration as versioned infrastructure code and separate the administrative domains that are supposed to provide independent assurance.

Design rule: MerchantOrg endorses merchant-sensitive flows as policy requires

Implementation Objective

The required output is: MerchantOrg identity tree. It must be version-controlled, reviewable, testable with the WAN disabled, and linked to the architecture/source-of-truth model. For a payment platform, every implementation decision should make it possible to explain who authorized an action, which facts were trusted, which controls ran, what financial journal committed, and how the result is recovered after failure.

Key Engineering Decisions

- MerchantOrg endorses merchant-sensitive flows as policy requires
- Use distinct CA trust root
- Keep operator roles separate

Deep Implementation Guidance

Store Fabric configuration as code. The release evidence should tie chaincode source commit to package ID, lifecycle sequence, organization approvals, channel policy, container/binary version, and test results.

Data and Control Questions

- What is the authoritative source for every input used by this component?
- Which user, device, service, MSP, or administrator identity is allowed to initiate or approve it?
- Does the operation move value, reserve value, create a side effect, or only provide advisory information?
- What idempotency/replay key proves that a retry is the same business intent?
- What evidence is retained off-chain and what stable reference/hash belongs on the ledger?

Chapter 48 Implementation - MerchantOrg MSP and CA

Implementation Procedure

236. Version all Fabric configuration and scripts.
237. Enroll/issue identities from local CAs and protect private keys.
238. Create Raft ordering, peers, channel, policies, and lifecycle approvals.
239. Deploy/test chaincode through Fabric Gateway.
240. Test quorum, peer loss, revocation, and upgrade procedures.

Implementation Example

```
# MerchantOrg MSP and CA
INPUT  -> authenticate, validate schema, bind requestId/nonce
FACTS  -> read authoritative wallet/merchant/state data
AI      -> optional bounded recommendation with decisionId/version
POLICY  -> enforce ownership, hard limits, approvals, risk fallback
LEDGER  -> submit one idempotent deterministic transaction when value changes
EVENT   -> update projection/case/monitoring asynchronously
VERIFY  -> discover committed result and retain correlationId + txId evidence
```

Verification Checklist

- The merchantorg identity tree exists and is version-controlled.
- The feature works with all public Internet/WAN connectivity disabled.
- Authorization, malformed input, duplicate retry, and failure behavior have explicit tests.
- If financial value changes, journal debits and credits balance exactly and historical entries remain immutable.
- If AI is involved, model/prompt/feature version and fallback behavior are recorded and deterministic policy remains final.
- A timeout/restart cannot create a duplicate financial result; request status can be rediscovered.
- Logs/evidence contain correlation references but not secrets or prohibited sensitive payment data.

Common Failure Modes

- One administrator owns every trust domain/key.
- Incorrect endorsement policies.
- Certificate expiry not monitored.
- Quorum behavior never tested.

Expert Review Questions

- What happens if this component succeeds but the next component fails?
- How is the authorizing identity/device/key revoked or recovered?
- Can the operation be safely retried after the client loses the response?
- What happens if AI is unavailable, slow, or wrong?
- Can an auditor reconstruct debit, credit, actor, risk decision, approval, and Fabric txId from retained evidence?
- Which compensating transaction resolves a downstream operational error without rewriting history?

Chapter 49 - AuditOrg MSP and CA

Give auditors an independently administered ledger copy and read-focused identities. Hyperledger Fabric provides permissioned identities, endorsement policies, deterministic transaction validation, immutable block history, and current world state. Treat Fabric configuration as versioned infrastructure code and separate the administrative domains that are supposed to provide independent assurance.

Design rule: Audit peer receives channel blocks

Implementation Objective

The required output is: AuditOrg design. It must be version-controlled, reviewable, testable with the WAN disabled, and linked to the architecture/source-of-truth model. For a payment platform, every implementation decision should make it possible to explain who authorized an action, which facts were trusted, which controls ran, what financial journal committed, and how the result is recovered after failure.

Key Engineering Decisions

- Audit peer receives channel blocks
- Auditor application access is read-only by policy
- Audit administrators do not control wallet issuance

Deep Implementation Guidance

Store Fabric configuration as code. The release evidence should tie chaincode source commit to package ID, lifecycle sequence, organization approvals, channel policy, container/binary version, and test results.

Data and Control Questions

- What is the authoritative source for every input used by this component?
- Which user, device, service, MSP, or administrator identity is allowed to initiate or approve it?
- Does the operation move value, reserve value, create a side effect, or only provide advisory information?
- What idempotency/replay key proves that a retry is the same business intent?
- What evidence is retained off-chain and what stable reference/hash belongs on the ledger?

Chapter 49 Implementation - AuditOrg MSP and CA

Implementation Procedure

241. Version all Fabric configuration and scripts.
242. Enroll/issue identities from local CAs and protect private keys.
243. Create Raft ordering, peers, channel, policies, and lifecycle approvals.
244. Deploy/test chaincode through Fabric Gateway.
245. Test quorum, peer loss, revocation, and upgrade procedures.

Implementation Example

```
# AuditOrg MSP and CA
INPUT  -> authenticate, validate schema, bind requestId/nonce
FACTS  -> read authoritative wallet/merchant/state data
AI     -> optional bounded recommendation with decisionId/version
POLICY -> enforce ownership, hard limits, approvals, risk fallback
LEDGER -> submit one idempotent deterministic transaction when value changes
EVENT  -> update projection/case/monitoring asynchronously
VERIFY -> discover committed result and retain correlationId + txId evidence
```

Verification Checklist

- The auditorg design exists and is version-controlled.
- The feature works with all public Internet/WAN connectivity disabled.
- Authorization, malformed input, duplicate retry, and failure behavior have explicit tests.
- If financial value changes, journal debits and credits balance exactly and historical entries remain immutable.
- If AI is involved, model/prompt/feature version and fallback behavior are recorded and deterministic policy remains final.
- A timeout/restart cannot create a duplicate financial result; request status can be rediscovered.
- Logs/evidence contain correlation references but not secrets or prohibited sensitive payment data.

Common Failure Modes

- One administrator owns every trust domain/key.
- Incorrect endorsement policies.
- Certificate expiry not monitored.
- Quorum behavior never tested.

Expert Review Questions

- What happens if this component succeeds but the next component fails?
- How is the authorizing identity/device/key revoked or recovered?
- Can the operation be safely retried after the client loses the response?
- What happens if AI is unavailable, slow, or wrong?
- Can an auditor reconstruct debit, credit, actor, risk decision, approval, and Fabric txId from retained evidence?
- Which compensating transaction resolves a downstream operational error without rewriting history?

Chapter 50 - OrdererOrg MSP and CA

Separate ordering-service control from application organizations. Hyperledger Fabric provides permissioned identities, endorsement policies, deterministic transaction validation, immutable block history, and current world state. Treat Fabric configuration as versioned infrastructure code and separate the administrative domains that are supposed to provide independent assurance.

Design rule: Orderer identities are distinct

Implementation Objective

The required output is: OrdererOrg design. It must be version-controlled, reviewable, testable with the WAN disabled, and linked to the architecture/source-of-truth model. For a payment platform, every implementation decision should make it possible to explain who authorized an action, which facts were trusted, which controls ran, what financial journal committed, and how the result is recovered after failure.

Key Engineering Decisions

- Orderer identities are distinct
- TLS and enrollment certs have lifecycle plans
- Admin access restricted to management segment

Deep Implementation Guidance

Store Fabric configuration as code. The release evidence should tie chaincode source commit to package ID, lifecycle sequence, organization approvals, channel policy, container/binary version, and test results.

Data and Control Questions

- What is the authoritative source for every input used by this component?
- Which user, device, service, MSP, or administrator identity is allowed to initiate or approve it?
- Does the operation move value, reserve value, create a side effect, or only provide advisory information?
- What idempotency/replay key proves that a retry is the same business intent?
- What evidence is retained off-chain and what stable reference/hash belongs on the ledger?

Chapter 50 Implementation - OrdererOrg MSP and CA

Implementation Procedure

246. Version all Fabric configuration and scripts.
247. Enroll/issue identities from local CAs and protect private keys.
248. Create Raft ordering, peers, channel, policies, and lifecycle approvals.
249. Deploy/test chaincode through Fabric Gateway.
250. Test quorum, peer loss, revocation, and upgrade procedures.

Implementation Example

```
# OrdererOrg MSP and CA
INPUT -> authenticate, validate schema, bind requestId/nonce
FACTS -> read authoritative wallet/merchant/state data
AI -> optional bounded recommendation with decisionId/version
POLICY -> enforce ownership, hard limits, approvals, risk fallback
LEDGER -> submit one idempotent deterministic transaction when value changes
EVENT -> update projection/case/monitoring asynchronously
VERIFY -> discover committed result and retain correlationId + txId evidence
```

Verification Checklist

- The ordererorg design exists and is version-controlled.
- The feature works with all public Internet/WAN connectivity disabled.
- Authorization, malformed input, duplicate retry, and failure behavior have explicit tests.
- If financial value changes, journal debits and credits balance exactly and historical entries remain immutable.
- If AI is involved, model/prompt/feature version and fallback behavior are recorded and deterministic policy remains final.
- A timeout/restart cannot create a duplicate financial result; request status can be rediscovered.
- Logs/evidence contain correlation references but not secrets or prohibited sensitive payment data.

Common Failure Modes

- One administrator owns every trust domain/key.
- Incorrect endorsement policies.
- Certificate expiry not monitored.
- Quorum behavior never tested.

Expert Review Questions

- What happens if this component succeeds but the next component fails?
- How is the authorizing identity/device/key revoked or recovered?
- Can the operation be safely retried after the client loses the response?
- What happens if AI is unavailable, slow, or wrong?
- Can an auditor reconstruct debit, credit, actor, risk decision, approval, and Fabric txId from retained evidence?
- Which compensating transaction resolves a downstream operational error without rewriting history?

Chapter 51 - Three-Node Raft Ordering Service

Deploy a local crash-fault-tolerant ordering cluster. Hyperledger Fabric provides permissioned identities, endorsement policies, deterministic transaction validation, immutable block history, and current world state. Treat Fabric configuration as versioned infrastructure code and separate the administrative domains that are supposed to provide independent assurance.

Design rule: Three consenters tolerate one unavailable node

Implementation Objective

The required output is: Raft topology. It must be version-controlled, reviewable, testable with the WAN disabled, and linked to the architecture/source-of-truth model. For a payment platform, every implementation decision should make it possible to explain who authorized an action, which facts were trusted, which controls ran, what financial journal committed, and how the result is recovered after failure.

Key Engineering Decisions

- Three consenters tolerate one unavailable node
- Use distinct hosts/failure domains when possible
- Test quorum loss and recovery

Deep Implementation Guidance

Store Fabric configuration as code. The release evidence should tie chaincode source commit to package ID, lifecycle sequence, organization approvals, channel policy, container/binary version, and test results.

Data and Control Questions

- What is the authoritative source for every input used by this component?
- Which user, device, service, MSP, or administrator identity is allowed to initiate or approve it?
- Does the operation move value, reserve value, create a side effect, or only provide advisory information?
- What idempotency/replay key proves that a retry is the same business intent?
- What evidence is retained off-chain and what stable reference/hash belongs on the ledger?

Chapter 51 Implementation - Three-Node Raft Ordering Service

Implementation Procedure

251. Version all Fabric configuration and scripts.
252. Enroll/issue identities from local CAs and protect private keys.
253. Create Raft ordering, peers, channel, policies, and lifecycle approvals.
254. Deploy/test chaincode through Fabric Gateway.
255. Test quorum, peer loss, revocation, and upgrade procedures.

Implementation Example

```
# Three-Node Raft Ordering Service
INPUT  -> authenticate, validate schema, bind requestId/nonce
FACTS  -> read authoritative wallet/merchant/state data
AI     -> optional bounded recommendation with decisionId/version
POLICY -> enforce ownership, hard limits, approvals, risk fallback
LEDGER -> submit one idempotent deterministic transaction when value changes
EVENT  -> update projection/case/monitoring asynchronously
VERIFY -> discover committed result and retain correlationId + txId evidence
```

Verification Checklist

- The raft topology exists and is version-controlled.
- The feature works with all public Internet/WAN connectivity disabled.
- Authorization, malformed input, duplicate retry, and failure behavior have explicit tests.
- If financial value changes, journal debits and credits balance exactly and historical entries remain immutable.
- If AI is involved, model/prompt/feature version and fallback behavior are recorded and deterministic policy remains final.
- A timeout/restart cannot create a duplicate financial result; request status can be rediscovered.
- Logs/evidence contain correlation references but not secrets or prohibited sensitive payment data.

Common Failure Modes

- One administrator owns every trust domain/key.
- Incorrect endorsement policies.
- Certificate expiry not monitored.
- Quorum behavior never tested.

Expert Review Questions

- What happens if this component succeeds but the next component fails?
- How is the authorizing identity/device/key revoked or recovered?
- Can the operation be safely retried after the client loses the response?
- What happens if AI is unavailable, slow, or wrong?
- Can an auditor reconstruct debit, credit, actor, risk decision, approval, and Fabric txId from retained evidence?
- Which compensating transaction resolves a downstream operational error without rewriting history?

Chapter 52 - Payments Channel Design

Create a channel containing the organizations that require the shared financial ledger. Hyperledger Fabric provides permissioned identities, endorsement policies, deterministic transaction validation, immutable block history, and current world state. Treat Fabric configuration as versioned infrastructure code and separate the administrative domains that are supposed to provide independent assurance.

Design rule: Define application/orderer consortium policies

Implementation Objective

The required output is: Channel profile. It must be version-controlled, reviewable, testable with the WAN disabled, and linked to the architecture/source-of-truth model. For a payment platform, every implementation decision should make it possible to explain who authorized an action, which facts were trusted, which controls ran, what financial journal committed, and how the result is recovered after failure.

Key Engineering Decisions

- Define application/orderer consortium policies
- Join required peers
- Document whether future branches use same channel or separate channel

Deep Implementation Guidance

Store Fabric configuration as code. The release evidence should tie chaincode source commit to package ID, lifecycle sequence, organization approvals, channel policy, container/binary version, and test results.

Data and Control Questions

- What is the authoritative source for every input used by this component?
- Which user, device, service, MSP, or administrator identity is allowed to initiate or approve it?
- Does the operation move value, reserve value, create a side effect, or only provide advisory information?
- What idempotency/replay key proves that a retry is the same business intent?
- What evidence is retained off-chain and what stable reference/hash belongs on the ledger?

Chapter 52 Implementation - Payments Channel Design

Implementation Procedure

256. Version all Fabric configuration and scripts.
257. Enroll/issue identities from local CAs and protect private keys.
258. Create Raft ordering, peers, channel, policies, and lifecycle approvals.
259. Deploy/test chaincode through Fabric Gateway.
260. Test quorum, peer loss, revocation, and upgrade procedures.

Implementation Example

```
# Payments Channel Design
INPUT  -> authenticate, validate schema, bind requestId/nonce
FACTS  -> read authoritative wallet/merchant/state data
AI     -> optional bounded recommendation with decisionId/version
POLICY -> enforce ownership, hard limits, approvals, risk fallback
LEDGER -> submit one idempotent deterministic transaction when value changes
EVENT  -> update projection/case/monitoring asynchronously
VERIFY -> discover committed result and retain correlationId + txId evidence
```

Verification Checklist

- The channel profile exists and is version-controlled.
- The feature works with all public Internet/WAN connectivity disabled.
- Authorization, malformed input, duplicate retry, and failure behavior have explicit tests.
- If financial value changes, journal debits and credits balance exactly and historical entries remain immutable.
- If AI is involved, model/prompt/feature version and fallback behavior are recorded and deterministic policy remains final.
- A timeout/restart cannot create a duplicate financial result; request status can be rediscovered.
- Logs/evidence contain correlation references but not secrets or prohibited sensitive payment data.

Common Failure Modes

- One administrator owns every trust domain/key.
- Incorrect endorsement policies.
- Certificate expiry not monitored.
- Quorum behavior never tested.

Expert Review Questions

- What happens if this component succeeds but the next component fails?
- How is the authorizing identity/device/key revoked or recovered?
- Can the operation be safely retried after the client loses the response?
- What happens if AI is unavailable, slow, or wrong?
- Can an auditor reconstruct debit, credit, actor, risk decision, approval, and Fabric txId from retained evidence?
- Which compensating transaction resolves a downstream operational error without rewriting history?

Chapter 53 - Peer Deployment and CouchDB

Deploy peers and optional CouchDB state database for rich operational queries. Hyperledger Fabric provides permissioned identities, endorsement policies, deterministic transaction validation, immutable block history, and current world state. Treat Fabric configuration as versioned infrastructure code and separate the administrative domains that are supposed to provide independent assurance.

Design rule: Ledger data persists independently of containers

Implementation Objective

The required output is: Peer deployment. It must be version-controlled, reviewable, testable with the WAN disabled, and linked to the architecture/source-of-truth model. For a payment platform, every implementation decision should make it possible to explain who authorized an action, which facts were trusted, which controls ran, what financial journal committed, and how the result is recovered after failure.

Key Engineering Decisions

- Ledger data persists independently of containers
- CouchDB credentials are protected
- Indexes are packaged with chaincode where used

Deep Implementation Guidance

Store Fabric configuration as code. The release evidence should tie chaincode source commit to package ID, lifecycle sequence, organization approvals, channel policy, container/binary version, and test results.

Data and Control Questions

- What is the authoritative source for every input used by this component?
- Which user, device, service, MSP, or administrator identity is allowed to initiate or approve it?
- Does the operation move value, reserve value, create a side effect, or only provide advisory information?
- What idempotency/replay key proves that a retry is the same business intent?
- What evidence is retained off-chain and what stable reference/hash belongs on the ledger?

Chapter 53 Implementation - Peer Deployment and CouchDB

Implementation Procedure

261. Version all Fabric configuration and scripts.
262. Enroll/issue identities from local CAs and protect private keys.
263. Create Raft ordering, peers, channel, policies, and lifecycle approvals.
264. Deploy/test chaincode through Fabric Gateway.
265. Test quorum, peer loss, revocation, and upgrade procedures.

Implementation Example

```
# Peer Deployment and CouchDB
INPUT  -> authenticate, validate schema, bind requestId/nonce
FACTS  -> read authoritative wallet/merchant/state data
AI     -> optional bounded recommendation with decisionId/version
POLICY -> enforce ownership, hard limits, approvals, risk fallback
LEDGER -> submit one idempotent deterministic transaction when value changes
EVENT  -> update projection/case/monitoring asynchronously
VERIFY -> discover committed result and retain correlationId + txId evidence
```

Verification Checklist

- The peer deployment exists and is version-controlled.
- The feature works with all public Internet/WAN connectivity disabled.
- Authorization, malformed input, duplicate retry, and failure behavior have explicit tests.
- If financial value changes, journal debits and credits balance exactly and historical entries remain immutable.
- If AI is involved, model/prompt/feature version and fallback behavior are recorded and deterministic policy remains final.
- A timeout/restart cannot create a duplicate financial result; request status can be rediscovered.
- Logs/evidence contain correlation references but not secrets or prohibited sensitive payment data.

Common Failure Modes

- One administrator owns every trust domain/key.
- Incorrect endorsement policies.
- Certificate expiry not monitored.
- Quorum behavior never tested.

Expert Review Questions

- What happens if this component succeeds but the next component fails?
- How is the authorizing identity/device/key revoked or recovered?
- Can the operation be safely retried after the client loses the response?
- What happens if AI is unavailable, slow, or wrong?
- Can an auditor reconstruct debit, credit, actor, risk decision, approval, and Fabric txId from retained evidence?
- Which compensating transaction resolves a downstream operational error without rewriting history?

Chapter 54 - Anchor Peers and Gossip

Ensure ledger dissemination and organization discovery operate correctly on the LAN. Hyperledger Fabric provides permissioned identities, endorsement policies, deterministic transaction validation, immutable block history, and current world state. Treat Fabric configuration as versioned infrastructure code and separate the administrative domains that are supposed to provide independent assurance.

Design rule: Use internal DNS/endpoints

Implementation Objective

The required output is: Gossip plan. It must be version-controlled, reviewable, testable with the WAN disabled, and linked to the architecture/source-of-truth model. For a payment platform, every implementation decision should make it possible to explain who authorized an action, which facts were trusted, which controls ran, what financial journal committed, and how the result is recovered after failure.

Key Engineering Decisions

- Use internal DNS/endpoints
- Restrict ports by network policy
- Test block propagation after peer restart

Deep Implementation Guidance

Store Fabric configuration as code. The release evidence should tie chaincode source commit to package ID, lifecycle sequence, organization approvals, channel policy, container/binary version, and test results.

Data and Control Questions

- What is the authoritative source for every input used by this component?
- Which user, device, service, MSP, or administrator identity is allowed to initiate or approve it?
- Does the operation move value, reserve value, create a side effect, or only provide advisory information?
- What idempotency/replay key proves that a retry is the same business intent?
- What evidence is retained off-chain and what stable reference/hash belongs on the ledger?

Chapter 54 Implementation - Anchor Peers and Gossip

Implementation Procedure

266. Version all Fabric configuration and scripts.
267. Enroll/issue identities from local CAs and protect private keys.
268. Create Raft ordering, peers, channel, policies, and lifecycle approvals.
269. Deploy/test chaincode through Fabric Gateway.
270. Test quorum, peer loss, revocation, and upgrade procedures.

Implementation Example

```
# Anchor Peers and Gossip
INPUT  -> authenticate, validate schema, bind requestId/nonce
FACTS  -> read authoritative wallet/merchant/state data
AI     -> optional bounded recommendation with decisionId/version
POLICY -> enforce ownership, hard limits, approvals, risk fallback
LEDGER -> submit one idempotent deterministic transaction when value changes
EVENT  -> update projection/case/monitoring asynchronously
VERIFY -> discover committed result and retain correlationId + txId evidence
```

Verification Checklist

- The gossip plan exists and is version-controlled.
- The feature works with all public Internet/WAN connectivity disabled.
- Authorization, malformed input, duplicate retry, and failure behavior have explicit tests.
- If financial value changes, journal debits and credits balance exactly and historical entries remain immutable.
- If AI is involved, model/prompt/feature version and fallback behavior are recorded and deterministic policy remains final.
- A timeout/restart cannot create a duplicate financial result; request status can be rediscovered.
- Logs/evidence contain correlation references but not secrets or prohibited sensitive payment data.

Common Failure Modes

- One administrator owns every trust domain/key.
- Incorrect endorsement policies.
- Certificate expiry not monitored.
- Quorum behavior never tested.

Expert Review Questions

- What happens if this component succeeds but the next component fails?
- How is the authorizing identity/device/key revoked or recovered?
- Can the operation be safely retried after the client loses the response?
- What happens if AI is unavailable, slow, or wrong?
- Can an auditor reconstruct debit, credit, actor, risk decision, approval, and Fabric txId from retained evidence?
- Which compensating transaction resolves a downstream operational error without rewriting history?

Chapter 55 - Channel Policies and Endorsement

Encode which organizations must approve transaction classes. Hyperledger Fabric provides permissioned identities, endorsement policies, deterministic transaction validation, immutable block history, and current world state. Treat Fabric configuration as versioned infrastructure code and separate the administrative domains that are supposed to provide independent assurance.

Design rule: Issuance may require WalletOrg plus Audit/treasury approval

Implementation Objective

The required output is: Policy matrix. It must be version-controlled, reviewable, testable with the WAN disabled, and linked to the architecture/source-of-truth model. For a payment platform, every implementation decision should make it possible to explain who authorized an action, which facts were trusted, which controls ran, what financial journal committed, and how the result is recovered after failure.

Key Engineering Decisions

- Issuance may require WalletOrg plus Audit/treasury approval
- Merchant settlement can require WalletOrg plus MerchantOrg
- Ordinary transfers may use a simpler policy

Deep Implementation Guidance

Store Fabric configuration as code. The release evidence should tie chaincode source commit to package ID, lifecycle sequence, organization approvals, channel policy, container/binary version, and test results.

Data and Control Questions

- What is the authoritative source for every input used by this component?
- Which user, device, service, MSP, or administrator identity is allowed to initiate or approve it?
- Does the operation move value, reserve value, create a side effect, or only provide advisory information?
- What idempotency/replay key proves that a retry is the same business intent?
- What evidence is retained off-chain and what stable reference/hash belongs on the ledger?

Chapter 55 Implementation - Channel Policies and Endorsement

Implementation Procedure

271. Version all Fabric configuration and scripts.
272. Enroll/issue identities from local CAs and protect private keys.
273. Create Raft ordering, peers, channel, policies, and lifecycle approvals.
274. Deploy/test chaincode through Fabric Gateway.
275. Test quorum, peer loss, revocation, and upgrade procedures.

Implementation Example

```
# Channel Policies and Endorsement
INPUT  -> authenticate, validate schema, bind requestId/nonce
FACTS  -> read authoritative wallet/merchant/state data
AI     -> optional bounded recommendation with decisionId/version
POLICY -> enforce ownership, hard limits, approvals, risk fallback
LEDGER -> submit one idempotent deterministic transaction when value changes
EVENT  -> update projection/case/monitoring asynchronously
VERIFY -> discover committed result and retain correlationId + txId evidence
```

Verification Checklist

- The policy matrix exists and is version-controlled.
- The feature works with all public Internet/WAN connectivity disabled.
- Authorization, malformed input, duplicate retry, and failure behavior have explicit tests.
- If financial value changes, journal debits and credits balance exactly and historical entries remain immutable.
- If AI is involved, model/prompt/feature version and fallback behavior are recorded and deterministic policy remains final.
- A timeout/restart cannot create a duplicate financial result; request status can be rediscovered.
- Logs/evidence contain correlation references but not secrets or prohibited sensitive payment data.

Common Failure Modes

- One administrator owns every trust domain/key.
- Incorrect endorsement policies.
- Certificate expiry not monitored.
- Quorum behavior never tested.

Expert Review Questions

- What happens if this component succeeds but the next component fails?
- How is the authorizing identity/device/key revoked or recovered?
- Can the operation be safely retried after the client loses the response?
- What happens if AI is unavailable, slow, or wrong?
- Can an auditor reconstruct debit, credit, actor, risk decision, approval, and Fabric txId from retained evidence?
- Which compensating transaction resolves a downstream operational error without rewriting history?

Chapter 56 - State-Based Endorsement

Apply stronger endorsement to selected high-value or administrative keys. Hyperledger Fabric provides permissioned identities, endorsement policies, deterministic transaction validation, immutable block history, and current world state. Treat Fabric configuration as versioned infrastructure code and separate the administrative domains that are supposed to provide independent assurance.

Design rule: Use only where per-key policy adds value

Implementation Objective

The required output is: SBE design. It must be version-controlled, reviewable, testable with the WAN disabled, and linked to the architecture/source-of-truth model. For a payment platform, every implementation decision should make it possible to explain who authorized an action, which facts were trusted, which controls ran, what financial journal committed, and how the result is recovered after failure.

Key Engineering Decisions

- Use only where per-key policy adds value
- Document policy update authority
- Test upgrade/rotation impact

Deep Implementation Guidance

Store Fabric configuration as code. The release evidence should tie chaincode source commit to package ID, lifecycle sequence, organization approvals, channel policy, container/binary version, and test results.

Data and Control Questions

- What is the authoritative source for every input used by this component?
- Which user, device, service, MSP, or administrator identity is allowed to initiate or approve it?
- Does the operation move value, reserve value, create a side effect, or only provide advisory information?
- What idempotency/replay key proves that a retry is the same business intent?
- What evidence is retained off-chain and what stable reference/hash belongs on the ledger?

Chapter 56 Implementation - State-Based Endorsement

Implementation Procedure

276. Version all Fabric configuration and scripts.
277. Enroll/issue identities from local CAs and protect private keys.
278. Create Raft ordering, peers, channel, policies, and lifecycle approvals.
279. Deploy/test chaincode through Fabric Gateway.
280. Test quorum, peer loss, revocation, and upgrade procedures.

Implementation Example

```
# State-Based Endorsement
INPUT  -> authenticate, validate schema, bind requestId/nonce
FACTS  -> read authoritative wallet/merchant/state data
AI     -> optional bounded recommendation with decisionId/version
POLICY -> enforce ownership, hard limits, approvals, risk fallback
LEDGER -> submit one idempotent deterministic transaction when value changes
EVENT  -> update projection/case/monitoring asynchronously
VERIFY -> discover committed result and retain correlationId + txId evidence
```

Verification Checklist

- The sbe design exists and is version-controlled.
- The feature works with all public Internet/WAN connectivity disabled.
- Authorization, malformed input, duplicate retry, and failure behavior have explicit tests.
- If financial value changes, journal debits and credits balance exactly and historical entries remain immutable.
- If AI is involved, model/prompt/feature version and fallback behavior are recorded and deterministic policy remains final.
- A timeout/restart cannot create a duplicate financial result; request status can be rediscovered.
- Logs/evidence contain correlation references but not secrets or prohibited sensitive payment data.

Common Failure Modes

- One administrator owns every trust domain/key.
- Incorrect endorsement policies.
- Certificate expiry not monitored.
- Quorum behavior never tested.

Expert Review Questions

- What happens if this component succeeds but the next component fails?
- How is the authorizing identity/device/key revoked or recovered?
- Can the operation be safely retried after the client loses the response?
- What happens if AI is unavailable, slow, or wrong?
- Can an auditor reconstruct debit, credit, actor, risk decision, approval, and Fabric txId from retained evidence?
- Which compensating transaction resolves a downstream operational error without rewriting history?

Chapter 57 - Private Data Collections

Keep selected sensitive shared facts off the full channel while retaining hash evidence. Hyperledger Fabric provides permissioned identities, endorsement policies, deterministic transaction validation, immutable block history, and current world state. Treat Fabric configuration as versioned infrastructure code and separate the administrative domains that are supposed to provide independent assurance.

Design rule: Use PDC only when multiple orgs need private shared state

Implementation Objective

The required output is: PDC policy. It must be version-controlled, reviewable, testable with the WAN disabled, and linked to the architecture/source-of-truth model. For a payment platform, every implementation decision should make it possible to explain who authorized an action, which facts were trusted, which controls ran, what financial journal committed, and how the result is recovered after failure.

Key Engineering Decisions

- Use PDC only when multiple orgs need private shared state
- Keep highly mutable PII in conventional DB instead
- Define collection membership and retention

Deep Implementation Guidance

Store Fabric configuration as code. The release evidence should tie chaincode source commit to package ID, lifecycle sequence, organization approvals, channel policy, container/binary version, and test results.

Data and Control Questions

- What is the authoritative source for every input used by this component?
- Which user, device, service, MSP, or administrator identity is allowed to initiate or approve it?
- Does the operation move value, reserve value, create a side effect, or only provide advisory information?
- What idempotency/replay key proves that a retry is the same business intent?
- What evidence is retained off-chain and what stable reference/hash belongs on the ledger?

Chapter 57 Implementation - Private Data Collections

Implementation Procedure

281. Version all Fabric configuration and scripts.
282. Enroll/issue identities from local CAs and protect private keys.
283. Create Raft ordering, peers, channel, policies, and lifecycle approvals.
284. Deploy/test chaincode through Fabric Gateway.
285. Test quorum, peer loss, revocation, and upgrade procedures.

Implementation Example

```
# Private Data Collections
INPUT  -> authenticate, validate schema, bind requestId/nonce
FACTS  -> read authoritative wallet/merchant/state data
AI     -> optional bounded recommendation with decisionId/version
POLICY -> enforce ownership, hard limits, approvals, risk fallback
LEDGER -> submit one idempotent deterministic transaction when value changes
EVENT  -> update projection/case/monitoring asynchronously
VERIFY -> discover committed result and retain correlationId + txId evidence
```

Verification Checklist

- The pdc policy exists and is version-controlled.
- The feature works with all public Internet/WAN connectivity disabled.
- Authorization, malformed input, duplicate retry, and failure behavior have explicit tests.
- If financial value changes, journal debits and credits balance exactly and historical entries remain immutable.
- If AI is involved, model/prompt/feature version and fallback behavior are recorded and deterministic policy remains final.
- A timeout/restart cannot create a duplicate financial result; request status can be rediscovered.
- Logs/evidence contain correlation references but not secrets or prohibited sensitive payment data.

Common Failure Modes

- One administrator owns every trust domain/key.
- Incorrect endorsement policies.
- Certificate expiry not monitored.
- Quorum behavior never tested.

Expert Review Questions

- What happens if this component succeeds but the next component fails?
- How is the authorizing identity/device/key revoked or recovered?
- Can the operation be safely retried after the client loses the response?
- What happens if AI is unavailable, slow, or wrong?
- Can an auditor reconstruct debit, credit, actor, risk decision, approval, and Fabric txId from retained evidence?
- Which compensating transaction resolves a downstream operational error without rewriting history?

Chapter 58 - Chaincode Lifecycle

Package, install, approve, commit, and upgrade wallet chaincode reproducibly. Hyperledger Fabric provides permissioned identities, endorsement policies, deterministic transaction validation, immutable block history, and current world state. Treat Fabric configuration as versioned infrastructure code and separate the administrative domains that are supposed to provide independent assurance.

Design rule: Record package ID/source hash/version/sequence

Implementation Objective

The required output is: Lifecycle runbook. It must be version-controlled, reviewable, testable with the WAN disabled, and linked to the architecture/source-of-truth model. For a payment platform, every implementation decision should make it possible to explain who authorized an action, which facts were trusted, which controls ran, what financial journal committed, and how the result is recovered after failure.

Key Engineering Decisions

- Record package ID/source hash/version/sequence
- Require org approvals
- Test upgrade in staging and rollback strategy

Deep Implementation Guidance

Store Fabric configuration as code. The release evidence should tie chaincode source commit to package ID, lifecycle sequence, organization approvals, channel policy, container/binary version, and test results.

Data and Control Questions

- What is the authoritative source for every input used by this component?
- Which user, device, service, MSP, or administrator identity is allowed to initiate or approve it?
- Does the operation move value, reserve value, create a side effect, or only provide advisory information?
- What idempotency/replay key proves that a retry is the same business intent?
- What evidence is retained off-chain and what stable reference/hash belongs on the ledger?

Chapter 58 Implementation - Chaincode Lifecycle

Implementation Procedure

286. Version all Fabric configuration and scripts.
287. Enroll/issue identities from local CAs and protect private keys.
288. Create Raft ordering, peers, channel, policies, and lifecycle approvals.
289. Deploy/test chaincode through Fabric Gateway.
290. Test quorum, peer loss, revocation, and upgrade procedures.

Implementation Example

```
# Chaincode Lifecycle
INPUT  -> authenticate, validate schema, bind requestId/nonce
FACTS  -> read authoritative wallet/merchant/state data
AI     -> optional bounded recommendation with decisionId/version
POLICY -> enforce ownership, hard limits, approvals, risk fallback
LEDGER -> submit one idempotent deterministic transaction when value changes
EVENT  -> update projection/case/monitoring asynchronously
VERIFY -> discover committed result and retain correlationId + txId evidence
```

Verification Checklist

- The lifecycle runbook exists and is version-controlled.
- The feature works with all public Internet/WAN connectivity disabled.
- Authorization, malformed input, duplicate retry, and failure behavior have explicit tests.
- If financial value changes, journal debits and credits balance exactly and historical entries remain immutable.
- If AI is involved, model/prompt/feature version and fallback behavior are recorded and deterministic policy remains final.
- A timeout/restart cannot create a duplicate financial result; request status can be rediscovered.
- Logs/evidence contain correlation references but not secrets or prohibited sensitive payment data.

Common Failure Modes

- One administrator owns every trust domain/key.
- Incorrect endorsement policies.
- Certificate expiry not monitored.
- Quorum behavior never tested.

Expert Review Questions

- What happens if this component succeeds but the next component fails?
- How is the authorizing identity/device/key revoked or recovered?
- Can the operation be safely retried after the client loses the response?
- What happens if AI is unavailable, slow, or wrong?
- Can an auditor reconstruct debit, credit, actor, risk decision, approval, and Fabric txId from retained evidence?
- Which compensating transaction resolves a downstream operational error without rewriting history?

Chapter 59 - Fabric Gateway Client Model

Use Gateway APIs for transaction evaluation, endorsement, submission, commit status, and events. Hyperledger Fabric provides permissioned identities, endorsement policies, deterministic transaction validation, immutable block history, and current world state. Treat Fabric configuration as versioned infrastructure code and separate the administrative domains that are supposed to provide independent assurance.

Design rule: Client connects to trusted peer Gateway

Implementation Objective

The required output is: Gateway integration design. It must be version-controlled, reviewable, testable with the WAN disabled, and linked to the architecture/source-of-truth model. For a payment platform, every implementation decision should make it possible to explain who authorized an action, which facts were trusted, which controls ran, what financial journal committed, and how the result is recovered after failure.

Key Engineering Decisions

- Client connects to trusted peer Gateway
- Set deadlines
- Wait for commit status
- Use chaincode events for projections

Deep Implementation Guidance

Store Fabric configuration as code. The release evidence should tie chaincode source commit to package ID, lifecycle sequence, organization approvals, channel policy, container/binary version, and test results.

Data and Control Questions

- What is the authoritative source for every input used by this component?
- Which user, device, service, MSP, or administrator identity is allowed to initiate or approve it?
- Does the operation move value, reserve value, create a side effect, or only provide advisory information?
- What idempotency/replay key proves that a retry is the same business intent?
- What evidence is retained off-chain and what stable reference/hash belongs on the ledger?

Chapter 59 Implementation - Fabric Gateway Client Model

Implementation Procedure

291. Version all Fabric configuration and scripts.
292. Enroll/issue identities from local CAs and protect private keys.
293. Create Raft ordering, peers, channel, policies, and lifecycle approvals.
294. Deploy/test chaincode through Fabric Gateway.
295. Test quorum, peer loss, revocation, and upgrade procedures.

Implementation Example

```
import * as grpc from '@grpc/grpc-js';
import { connect, hash, signers } from '@hyperledger/fabric-gateway';
import { readFile } from 'node:fs/promises';
import { createPrivateKey } from 'node:crypto';

const tlsRootCert = await readFile(process.env.PEER_TLS_CA!);
const client = new grpc.Client(process.env.PEER_ENDPOINT!, grpc.credentials.createSsl(tlsRootCert));
const credentials = await readFile(process.env.CLIENT_CERT!);
const privateKey = createPrivateKey(await readFile(process.env.CLIENT_KEY!));
const gateway = connect({
  client,
  identity: { mspId: process.env.MSP_ID!, credentials },
  signer: signers.newPrivateKeySigner(privateKey),
  hash: hash.sha256,
  evaluateOptions: () => ({ deadline: Date.now() + 5_000 }),
  endorseOptions: () => ({ deadline: Date.now() + 10_000 }),
  submitOptions: () => ({ deadline: Date.now() + 10_000 }),
  commitStatusOptions: () => ({ deadline: Date.now() + 30_000 }),
});
export const contract = gateway.getNetwork('payments').getContract('walletcc');
```

Verification Checklist

- The gateway integration design exists and is version-controlled.
- The feature works with all public Internet/WAN connectivity disabled.
- Authorization, malformed input, duplicate retry, and failure behavior have explicit tests.
- If financial value changes, journal debits and credits balance exactly and historical entries remain immutable.
- If AI is involved, model/prompt/feature version and fallback behavior are recorded and deterministic policy remains final.
- A timeout/restart cannot create a duplicate financial result; request status can be rediscovered.
- Logs/evidence contain correlation references but not secrets or prohibited sensitive payment data.

Common Failure Modes

- One administrator owns every trust domain/key.
- Incorrect endorsement policies.
- Certificate expiry not monitored.
- Quorum behavior never tested.

Expert Review Questions

- What happens if this component succeeds but the next component fails?
- How is the authorizing identity/device/key revoked or recovered?

- Can the operation be safely retried after the client loses the response?
- What happens if AI is unavailable, slow, or wrong?
- Can an auditor reconstruct debit, credit, actor, risk decision, approval, and Fabric txId from retained evidence?
- Which compensating transaction resolves a downstream operational error without rewriting history?

Chapter 60 - Fabric HSM and PKCS11 Option

Protect high-value node/client signing keys using supported HSM interfaces when justified. Hyperledger Fabric provides permissioned identities, endorsement policies, deterministic transaction validation, immutable block history, and current world state. Treat Fabric configuration as versioned infrastructure code and separate the administrative domains that are supposed to provide independent assurance.

Design rule: Fabric supports PKCS11 for MSP signing identities

Implementation Objective

The required output is: HSM design. It must be version-controlled, reviewable, testable with the WAN disabled, and linked to the architecture/source-of-truth model. For a payment platform, every implementation decision should make it possible to explain who authorized an action, which facts were trusted, which controls ran, what financial journal committed, and how the result is recovered after failure.

Key Engineering Decisions

- Fabric supports PKCS11 for MSP signing identities
- Test vendor module offline
- Remember TLS private keys use separate handling in Fabric guidance

Deep Implementation Guidance

Store Fabric configuration as code. The release evidence should tie chaincode source commit to package ID, lifecycle sequence, organization approvals, channel policy, container/binary version, and test results.

Data and Control Questions

- What is the authoritative source for every input used by this component?
- Which user, device, service, MSP, or administrator identity is allowed to initiate or approve it?
- Does the operation move value, reserve value, create a side effect, or only provide advisory information?
- What idempotency/replay key proves that a retry is the same business intent?
- What evidence is retained off-chain and what stable reference/hash belongs on the ledger?

Chapter 60 Implementation - Fabric HSM and PKCS11 Option

Implementation Procedure

296. Version all Fabric configuration and scripts.
297. Enroll/issue identities from local CAs and protect private keys.
298. Create Raft ordering, peers, channel, policies, and lifecycle approvals.
299. Deploy/test chaincode through Fabric Gateway.
300. Test quorum, peer loss, revocation, and upgrade procedures.

Implementation Example

```
# Fabric HSM and PKCS11 Option
INPUT  -> authenticate, validate schema, bind requestId/nonce
FACTS  -> read authoritative wallet/merchant/state data
AI     -> optional bounded recommendation with decisionId/version
POLICY -> enforce ownership, hard limits, approvals, risk fallback
LEDGER -> submit one idempotent deterministic transaction when value changes
EVENT  -> update projection/case/monitoring asynchronously
VERIFY -> discover committed result and retain correlationId + txId evidence
```

Verification Checklist

- The hsm design exists and is version-controlled.
- The feature works with all public Internet/WAN connectivity disabled.
- Authorization, malformed input, duplicate retry, and failure behavior have explicit tests.
- If financial value changes, journal debits and credits balance exactly and historical entries remain immutable.
- If AI is involved, model/prompt/feature version and fallback behavior are recorded and deterministic policy remains final.
- A timeout/restart cannot create a duplicate financial result; request status can be rediscovered.
- Logs/evidence contain correlation references but not secrets or prohibited sensitive payment data.

Common Failure Modes

- One administrator owns every trust domain/key.
- Incorrect endorsement policies.
- Certificate expiry not monitored.
- Quorum behavior never tested.

Expert Review Questions

- What happens if this component succeeds but the next component fails?
- How is the authorizing identity/device/key revoked or recovered?
- Can the operation be safely retried after the client loses the response?
- What happens if AI is unavailable, slow, or wrong?
- Can an auditor reconstruct debit, credit, actor, risk decision, approval, and Fabric txId from retained evidence?
- Which compensating transaction resolves a downstream operational error without rewriting history?

Chapter 61 - Certificate Renewal and Revocation

Operate local identity lifecycle without external certificate services. Hyperledger Fabric provides permissioned identities, endorsement policies, deterministic transaction validation, immutable block history, and current world state. Treat Fabric configuration as versioned infrastructure code and separate the administrative domains that are supposed to provide independent assurance.

Design rule: Inventory expiry

Implementation Objective

The required output is: Cert lifecycle runbook. It must be version-controlled, reviewable, testable with the WAN disabled, and linked to the architecture/source-of-truth model. For a payment platform, every implementation decision should make it possible to explain who authorized an action, which facts were trusted, which controls ran, what financial journal committed, and how the result is recovered after failure.

Key Engineering Decisions

- Inventory expiry
- Renew before deadline
- Revoke compromise
- Distribute CRLs/trust changes
- Test client/node restarts

Deep Implementation Guidance

Store Fabric configuration as code. The release evidence should tie chaincode source commit to package ID, lifecycle sequence, organization approvals, channel policy, container/binary version, and test results.

Data and Control Questions

- What is the authoritative source for every input used by this component?
- Which user, device, service, MSP, or administrator identity is allowed to initiate or approve it?
- Does the operation move value, reserve value, create a side effect, or only provide advisory information?
- What idempotency/replay key proves that a retry is the same business intent?
- What evidence is retained off-chain and what stable reference/hash belongs on the ledger?

Chapter 61 Implementation - Certificate Renewal and Revocation

Implementation Procedure

301. Version all Fabric configuration and scripts.
302. Enroll/issue identities from local CAs and protect private keys.
303. Create Raft ordering, peers, channel, policies, and lifecycle approvals.
304. Deploy/test chaincode through Fabric Gateway.
305. Test quorum, peer loss, revocation, and upgrade procedures.

Implementation Example

```
# Certificate Renewal and Revocation
INPUT  -> authenticate, validate schema, bind requestId/nonce
FACTS  -> read authoritative wallet/merchant/state data
AI     -> optional bounded recommendation with decisionId/version
POLICY -> enforce ownership, hard limits, approvals, risk fallback
LEDGER -> submit one idempotent deterministic transaction when value changes
EVENT  -> update projection/case/monitoring asynchronously
VERIFY -> discover committed result and retain correlationId + txId evidence
```

Verification Checklist

- The cert lifecycle runbook exists and is version-controlled.
- The feature works with all public Internet/WAN connectivity disabled.
- Authorization, malformed input, duplicate retry, and failure behavior have explicit tests.
- If financial value changes, journal debits and credits balance exactly and historical entries remain immutable.
- If AI is involved, model/prompt/feature version and fallback behavior are recorded and deterministic policy remains final.
- A timeout/restart cannot create a duplicate financial result; request status can be rediscovered.
- Logs/evidence contain correlation references but not secrets or prohibited sensitive payment data.

Common Failure Modes

- One administrator owns every trust domain/key.
- Incorrect endorsement policies.
- Certificate expiry not monitored.
- Quorum behavior never tested.

Expert Review Questions

- What happens if this component succeeds but the next component fails?
- How is the authorizing identity/device/key revoked or recovered?
- Can the operation be safely retried after the client loses the response?
- What happens if AI is unavailable, slow, or wrong?
- Can an auditor reconstruct debit, credit, actor, risk decision, approval, and Fabric txId from retained evidence?
- Which compensating transaction resolves a downstream operational error without rewriting history?

Chapter 62 - Fabric Network Health Verification

Create repeatable checks for ledger height, channel membership, orderer quorum, and chaincode definition. Hyperledger Fabric provides permissioned identities, endorsement policies, deterministic transaction validation, immutable block history, and current world state. Treat Fabric configuration as versioned infrastructure code and separate the administrative domains that are supposed to provide independent assurance.

Design rule: Verify peers synchronized

Implementation Objective

The required output is: Health checklist. It must be version-controlled, reviewable, testable with the WAN disabled, and linked to the architecture/source-of-truth model. For a payment platform, every implementation decision should make it possible to explain who authorized an action, which facts were trusted, which controls ran, what financial journal committed, and how the result is recovered after failure.

Key Engineering Decisions

- Verify peers synchronized
- Verify orderer leader/quorum
- Evaluate read
- Submit test transaction
- Confirm event/projection

Deep Implementation Guidance

Store Fabric configuration as code. The release evidence should tie chaincode source commit to package ID, lifecycle sequence, organization approvals, channel policy, container/binary version, and test results.

Data and Control Questions

- What is the authoritative source for every input used by this component?
- Which user, device, service, MSP, or administrator identity is allowed to initiate or approve it?
- Does the operation move value, reserve value, create a side effect, or only provide advisory information?
- What idempotency/replay key proves that a retry is the same business intent?
- What evidence is retained off-chain and what stable reference/hash belongs on the ledger?

Chapter 62 Implementation - Fabric Network Health Verification

Implementation Procedure

306. Version all Fabric configuration and scripts.
307. Enroll/issue identities from local CAs and protect private keys.
308. Create Raft ordering, peers, channel, policies, and lifecycle approvals.
309. Deploy/test chaincode through Fabric Gateway.
310. Test quorum, peer loss, revocation, and upgrade procedures.

Implementation Example

```
# Fabric Network Health Verification
INPUT  -> authenticate, validate schema, bind requestId/nonce
FACTS  -> read authoritative wallet/merchant/state data
AI     -> optional bounded recommendation with decisionId/version
POLICY -> enforce ownership, hard limits, approvals, risk fallback
LEDGER -> submit one idempotent deterministic transaction when value changes
EVENT  -> update projection/case/monitoring asynchronously
VERIFY -> discover committed result and retain correlationId + txId evidence
```

Verification Checklist

- The health checklist exists and is version-controlled.
- The feature works with all public Internet/WAN connectivity disabled.
- Authorization, malformed input, duplicate retry, and failure behavior have explicit tests.
- If financial value changes, journal debits and credits balance exactly and historical entries remain immutable.
- If AI is involved, model/prompt/feature version and fallback behavior are recorded and deterministic policy remains final.
- A timeout/restart cannot create a duplicate financial result; request status can be rediscovered.
- Logs/evidence contain correlation references but not secrets or prohibited sensitive payment data.

Common Failure Modes

- One administrator owns every trust domain/key.
- Incorrect endorsement policies.
- Certificate expiry not monitored.
- Quorum behavior never tested.

Expert Review Questions

- What happens if this component succeeds but the next component fails?
- How is the authorizing identity/device/key revoked or recovered?
- Can the operation be safely retried after the client loses the response?
- What happens if AI is unavailable, slow, or wrong?
- Can an auditor reconstruct debit, credit, actor, risk decision, approval, and Fabric txId from retained evidence?
- Which compensating transaction resolves a downstream operational error without rewriting history?

Part V - Identity, Wallet Authorization, and Key Management

Financial authorization begins with identity. Application users, devices, service identities, Fabric node identities, and signing keys have different trust purposes. The system must bind an authenticated human/device intent to a deterministic financial command without exposing high-value Fabric signing keys to browsers or terminals.

Part focus: This part contains 17 implementation chapters. Each chapter includes design rules, implementation procedure, a practical artifact, verification, failure analysis, and expert review questions.

Chapter 63 - Separate Application Identity from Fabric Identity

Prevent every end user/device from needing direct Fabric credentials. Financial authorization begins with identity. Application users, devices, service identities, Fabric node identities, and signing keys have different trust purposes. The system must bind an authenticated human/device intent to a deterministic financial command without exposing high-value Fabric signing keys to browsers or terminals.

Design rule: Users authenticate to local app

Implementation Objective

The required output is: Identity architecture. It must be version-controlled, reviewable, testable with the WAN disabled, and linked to the architecture/source-of-truth model. For a payment platform, every implementation decision should make it possible to explain who authorized an action, which facts were trusted, which controls ran, what financial journal committed, and how the result is recovered after failure.

Key Engineering Decisions

- Users authenticate to local app
- Backend service submits Fabric commands
- Ledger stores actor/user/device references for attribution

Deep Implementation Guidance

Model identity proof, authentication, authorization, signing, and attribution separately. A Fabric client certificate proves an organizational identity, while the application must still prove which user/device requested the financial action and whether that action was permitted.

Data and Control Questions

- What is the authoritative source for every input used by this component?
- Which user, device, service, MSP, or administrator identity is allowed to initiate or approve it?
- Does the operation move value, reserve value, create a side effect, or only provide advisory information?
- What idempotency/replay key proves that a retry is the same business intent?
- What evidence is retained off-chain and what stable reference/hash belongs on the ledger?

Chapter 63 Implementation - Separate Application Identity from Fabric Identity

Implementation Procedure

311. Define separate user, device, service, Fabric, and administrator identity models.
312. Authenticate locally and bind roles/permissions to stable IDs.
313. Verify signed device/user intent before creating a ledger command when required.
314. Protect keys using OS security or HSM/PKCS11 for high-value identities.
315. Test revocation, rotation, lockout, replay, and break-glass behavior.

Implementation Example

```
# Separate Application Identity from Fabric Identity
INPUT  -> authenticate, validate schema, bind requestId/nonce
FACTS  -> read authoritative wallet/merchant/state data
AI      -> optional bounded recommendation with decisionId/version
POLICY  -> enforce ownership, hard limits, approvals, risk fallback
LEDGER  -> submit one idempotent deterministic transaction when value changes
EVENT   -> update projection/case/monitoring asynchronously
VERIFY  -> discover committed result and retain correlationId + txId evidence
```

Verification Checklist

- The identity architecture exists and is version-controlled.
- The feature works with all public Internet/WAN connectivity disabled.
- Authorization, malformed input, duplicate retry, and failure behavior have explicit tests.
- If financial value changes, journal debits and credits balance exactly and historical entries remain immutable.
- If AI is involved, model/prompt/feature version and fallback behavior are recorded and deterministic policy remains final.
- A timeout/restart cannot create a duplicate financial result; request status can be rediscovered.
- Logs/evidence contain correlation references but not secrets or prohibited sensitive payment data.

Common Failure Modes

- Shared operator accounts.
- Browser/terminal holds high-value Fabric key.
- No revocation/rotation test.
- User-supplied identity trusted without authentication binding.

Expert Review Questions

- What happens if this component succeeds but the next component fails?
- How is the authorizing identity/device/key revoked or recovered?
- Can the operation be safely retried after the client loses the response?
- What happens if AI is unavailable, slow, or wrong?
- Can an auditor reconstruct debit, credit, actor, risk decision, approval, and Fabric txId from retained evidence?
- Which compensating transaction resolves a downstream operational error without rewriting history?

Chapter 64 - Local User Directory and Authentication

Implement Internet-independent staff/customer authentication. Financial authorization begins with identity. Application users, devices, service identities, Fabric node identities, and signing keys have different trust purposes. The system must bind an authenticated human/device intent to a deterministic financial command without exposing high-value Fabric signing keys to browsers or terminals.

Design rule: Use local password hashing/MFA as appropriate

Implementation Objective

The required output is: Auth service design. It must be version-controlled, reviewable, testable with the WAN disabled, and linked to the architecture/source-of-truth model. For a payment platform, every implementation decision should make it possible to explain who authorized an action, which facts were trusted, which controls ran, what financial journal committed, and how the result is recovered after failure.

Key Engineering Decisions

- Use local password hashing/MFA as appropriate
- Store credentials off-chain
- Rate-limit login and lock suspicious attempts

Deep Implementation Guidance

Model identity proof, authentication, authorization, signing, and attribution separately. A Fabric client certificate proves an organizational identity, while the application must still prove which user/device requested the financial action and whether that action was permitted.

Data and Control Questions

- What is the authoritative source for every input used by this component?
- Which user, device, service, MSP, or administrator identity is allowed to initiate or approve it?
- Does the operation move value, reserve value, create a side effect, or only provide advisory information?
- What idempotency/replay key proves that a retry is the same business intent?
- What evidence is retained off-chain and what stable reference/hash belongs on the ledger?

Chapter 64 Implementation - Local User Directory and Authentication

Implementation Procedure

316. Define separate user, device, service, Fabric, and administrator identity models.
317. Authenticate locally and bind roles/permissions to stable IDs.
318. Verify signed device/user intent before creating a ledger command when required.
319. Protect keys using OS security or HSM/PKCS11 for high-value identities.
320. Test revocation, rotation, lockout, replay, and break-glass behavior.

Implementation Example

```
# Local User Directory and Authentication
INPUT -> authenticate, validate schema, bind requestId/nonce
FACTS -> read authoritative wallet/merchant/state data
AI -> optional bounded recommendation with decisionId/version
POLICY -> enforce ownership, hard limits, approvals, risk fallback
LEDGER -> submit one idempotent deterministic transaction when value changes
EVENT -> update projection/case/monitoring asynchronously
VERIFY -> discover committed result and retain correlationId + txId evidence
```

Verification Checklist

- The auth service design exists and is version-controlled.
- The feature works with all public Internet/WAN connectivity disabled.
- Authorization, malformed input, duplicate retry, and failure behavior have explicit tests.
- If financial value changes, journal debits and credits balance exactly and historical entries remain immutable.
- If AI is involved, model/prompt/feature version and fallback behavior are recorded and deterministic policy remains final.
- A timeout/restart cannot create a duplicate financial result; request status can be rediscovered.
- Logs/evidence contain correlation references but not secrets or prohibited sensitive payment data.

Common Failure Modes

- Shared operator accounts.
- Browser/terminal holds high-value Fabric key.
- No revocation/rotation test.
- User-supplied identity trusted without authentication binding.

Expert Review Questions

- What happens if this component succeeds but the next component fails?
- How is the authorizing identity/device/key revoked or recovered?
- Can the operation be safely retried after the client loses the response?
- What happens if AI is unavailable, slow, or wrong?
- Can an auditor reconstruct debit, credit, actor, risk decision, approval, and Fabric txId from retained evidence?
- Which compensating transaction resolves a downstream operational error without rewriting history?

Chapter 65 - Role-Based Access Control

Map customer, merchant operator, cashier, supervisor, treasury, auditor, security, and admin roles. Financial authorization begins with identity. Application users, devices, service identities, Fabric node identities, and signing keys have different trust purposes. The system must bind an authenticated human/device intent to a deterministic financial command without exposing high-value Fabric signing keys to browsers or terminals.

Design rule: Deny by default

Implementation Objective

The required output is: RBAC matrix. It must be version-controlled, reviewable, testable with the WAN disabled, and linked to the architecture/source-of-truth model. For a payment platform, every implementation decision should make it possible to explain who authorized an action, which facts were trusted, which controls ran, what financial journal committed, and how the result is recovered after failure.

Key Engineering Decisions

- Deny by default
- Separate financial and infrastructure administration
- Enforce in API and chaincode for critical commands

Deep Implementation Guidance

Model identity proof, authentication, authorization, signing, and attribution separately. A Fabric client certificate proves an organizational identity, while the application must still prove which user/device requested the financial action and whether that action was permitted.

Data and Control Questions

- What is the authoritative source for every input used by this component?
- Which user, device, service, MSP, or administrator identity is allowed to initiate or approve it?
- Does the operation move value, reserve value, create a side effect, or only provide advisory information?
- What idempotency/replay key proves that a retry is the same business intent?
- What evidence is retained off-chain and what stable reference/hash belongs on the ledger?

Chapter 65 Implementation - Role-Based Access Control

Implementation Procedure

321. Define separate user, device, service, Fabric, and administrator identity models.
322. Authenticate locally and bind roles/permissions to stable IDs.
323. Verify signed device/user intent before creating a ledger command when required.
324. Protect keys using OS security or HSM/PKCS11 for high-value identities.
325. Test revocation, rotation, lockout, replay, and break-glass behavior.

Implementation Example

```
# Role-Based Access Control
INPUT  -> authenticate, validate schema, bind requestId/nonce
FACTS  -> read authoritative wallet/merchant/state data
AI     -> optional bounded recommendation with decisionId/version
POLICY -> enforce ownership, hard limits, approvals, risk fallback
LEDGER -> submit one idempotent deterministic transaction when value changes
EVENT  -> update projection/case/monitoring asynchronously
VERIFY -> discover committed result and retain correlationId + txId evidence
```

Verification Checklist

- The rbac matrix exists and is version-controlled.
- The feature works with all public Internet/WAN connectivity disabled.
- Authorization, malformed input, duplicate retry, and failure behavior have explicit tests.
- If financial value changes, journal debits and credits balance exactly and historical entries remain immutable.
- If AI is involved, model/prompt/feature version and fallback behavior are recorded and deterministic policy remains final.
- A timeout/restart cannot create a duplicate financial result; request status can be rediscovered.
- Logs/evidence contain correlation references but not secrets or prohibited sensitive payment data.

Common Failure Modes

- Shared operator accounts.
- Browser/terminal holds high-value Fabric key.
- No revocation/rotation test.
- User-supplied identity trusted without authentication binding.

Expert Review Questions

- What happens if this component succeeds but the next component fails?
- How is the authorizing identity/device/key revoked or recovered?
- Can the operation be safely retried after the client loses the response?
- What happens if AI is unavailable, slow, or wrong?
- Can an auditor reconstruct debit, credit, actor, risk decision, approval, and Fabric txId from retained evidence?
- Which compensating transaction resolves a downstream operational error without rewriting history?

Chapter 66 - MFA and TOTP for Privileged Operations

Add local second factor without cloud dependency. Financial authorization begins with identity.

Application users, devices, service identities, Fabric node identities, and signing keys have different trust purposes. The system must bind an authenticated human/device intent to a deterministic financial command without exposing high-value Fabric signing keys to browsers or terminals.

Design rule: TOTP can work offline with local time sync

Implementation Objective

The required output is: MFA policy. It must be version-controlled, reviewable, testable with the WAN disabled, and linked to the architecture/source-of-truth model. For a payment platform, every implementation decision should make it possible to explain who authorized an action, which facts were trusted, which controls ran, what financial journal committed, and how the result is recovered after failure.

Key Engineering Decisions

- TOTP can work offline with local time sync
- Require stronger auth for treasury/admin actions
- Provide controlled recovery process

Deep Implementation Guidance

Model identity proof, authentication, authorization, signing, and attribution separately. A Fabric client certificate proves an organizational identity, while the application must still prove which user/device requested the financial action and whether that action was permitted.

Data and Control Questions

- What is the authoritative source for every input used by this component?
- Which user, device, service, MSP, or administrator identity is allowed to initiate or approve it?
- Does the operation move value, reserve value, create a side effect, or only provide advisory information?
- What idempotency/replay key proves that a retry is the same business intent?
- What evidence is retained off-chain and what stable reference/hash belongs on the ledger?

Chapter 66 Implementation - MFA and TOTP for Privileged Operations

Implementation Procedure

326. Define separate user, device, service, Fabric, and administrator identity models.
327. Authenticate locally and bind roles/permissions to stable IDs.
328. Verify signed device/user intent before creating a ledger command when required.
329. Protect keys using OS security or HSM/PKCS11 for high-value identities.
330. Test revocation, rotation, lockout, replay, and break-glass behavior.

Implementation Example

```
# MFA and TOTP for Privileged Operations
INPUT  -> authenticate, validate schema, bind requestId/nonce
FACTS  -> read authoritative wallet/merchant/state data
AI     -> optional bounded recommendation with decisionId/version
POLICY -> enforce ownership, hard limits, approvals, risk fallback
LEDGER -> submit one idempotent deterministic transaction when value changes
EVENT  -> update projection/case/monitoring asynchronously
VERIFY -> discover committed result and retain correlationId + txId evidence
```

Verification Checklist

- The mfa policy exists and is version-controlled.
- The feature works with all public Internet/WAN connectivity disabled.
- Authorization, malformed input, duplicate retry, and failure behavior have explicit tests.
- If financial value changes, journal debits and credits balance exactly and historical entries remain immutable.
- If AI is involved, model/prompt/feature version and fallback behavior are recorded and deterministic policy remains final.
- A timeout/restart cannot create a duplicate financial result; request status can be rediscovered.
- Logs/evidence contain correlation references but not secrets or prohibited sensitive payment data.

Common Failure Modes

- Shared operator accounts.
- Browser/terminal holds high-value Fabric key.
- No revocation/rotation test.
- User-supplied identity trusted without authentication binding.

Expert Review Questions

- What happens if this component succeeds but the next component fails?
- How is the authorizing identity/device/key revoked or recovered?
- Can the operation be safely retried after the client loses the response?
- What happens if AI is unavailable, slow, or wrong?
- Can an auditor reconstruct debit, credit, actor, risk decision, approval, and Fabric txId from retained evidence?
- Which compensating transaction resolves a downstream operational error without rewriting history?

Chapter 67 - Device Registration

Bind wallet/POS actions to registered devices where risk warrants it. Financial authorization begins with identity. Application users, devices, service identities, Fabric node identities, and signing keys have different trust purposes. The system must bind an authenticated human/device intent to a deterministic financial command without exposing high-value Fabric signing keys to browsers or terminals.

Design rule: Assign deviceId, owner/user binding, key fingerprint, status

Implementation Objective

The required output is: Device registry. It must be version-controlled, reviewable, testable with the WAN disabled, and linked to the architecture/source-of-truth model. For a payment platform, every implementation decision should make it possible to explain who authorized an action, which facts were trusted, which controls ran, what financial journal committed, and how the result is recovered after failure.

Key Engineering Decisions

- Assign deviceId, owner/user binding, key fingerprint, status
- Support revoke/replace
- Record device risk attributes off-chain

Deep Implementation Guidance

Model identity proof, authentication, authorization, signing, and attribution separately. A Fabric client certificate proves an organizational identity, while the application must still prove which user/device requested the financial action and whether that action was permitted.

Data and Control Questions

- What is the authoritative source for every input used by this component?
- Which user, device, service, MSP, or administrator identity is allowed to initiate or approve it?
- Does the operation move value, reserve value, create a side effect, or only provide advisory information?
- What idempotency/replay key proves that a retry is the same business intent?
- What evidence is retained off-chain and what stable reference/hash belongs on the ledger?

Chapter 67 Implementation - Device Registration

Implementation Procedure

331. Define separate user, device, service, Fabric, and administrator identity models.
332. Authenticate locally and bind roles/permissions to stable IDs.
333. Verify signed device/user intent before creating a ledger command when required.
334. Protect keys using OS security or HSM/PKCS11 for high-value identities.
335. Test revocation, rotation, lockout, replay, and break-glass behavior.

Implementation Example

```
# Device Registration
INPUT  -> authenticate, validate schema, bind requestId/nonce
FACTS  -> read authoritative wallet/merchant/state data
AI     -> optional bounded recommendation with decisionId/version
POLICY -> enforce ownership, hard limits, approvals, risk fallback
LEDGER -> submit one idempotent deterministic transaction when value changes
EVENT  -> update projection/case/monitoring asynchronously
VERIFY -> discover committed result and retain correlationId + txId evidence
```

Verification Checklist

- The device registry exists and is version-controlled.
- The feature works with all public Internet/WAN connectivity disabled.
- Authorization, malformed input, duplicate retry, and failure behavior have explicit tests.
- If financial value changes, journal debits and credits balance exactly and historical entries remain immutable.
- If AI is involved, model/prompt/feature version and fallback behavior are recorded and deterministic policy remains final.
- A timeout/restart cannot create a duplicate financial result; request status can be rediscovered.
- Logs/evidence contain correlation references but not secrets or prohibited sensitive payment data.

Common Failure Modes

- Shared operator accounts.
- Browser/terminal holds high-value Fabric key.
- No revocation/rotation test.
- User-supplied identity trusted without authentication binding.

Expert Review Questions

- What happens if this component succeeds but the next component fails?
- How is the authorizing identity/device/key revoked or recovered?
- Can the operation be safely retried after the client loses the response?
- What happens if AI is unavailable, slow, or wrong?
- Can an auditor reconstruct debit, credit, actor, risk decision, approval, and Fabric txId from retained evidence?
- Which compensating transaction resolves a downstream operational error without rewriting history?

Chapter 68 - Signed Authorization Envelope

Cryptographically bind a user/device intent to amount, destination, asset, nonce, and request ID. Financial authorization begins with identity. Application users, devices, service identities, Fabric node identities, and signing keys have different trust purposes. The system must bind an authenticated human/device intent to a deterministic financial command without exposing high-value Fabric signing keys to browsers or terminals.

Design rule: Canonicalize signed fields

Implementation Objective

The required output is: Authorization envelope schema. It must be version-controlled, reviewable, testable with the WAN disabled, and linked to the architecture/source-of-truth model. For a payment platform, every implementation decision should make it possible to explain who authorized an action, which facts were trusted, which controls ran, what financial journal committed, and how the result is recovered after failure.

Key Engineering Decisions

- Canonicalize signed fields
- Verify signature and nonce
- Hash envelope
- Anchor digest/reference in ledger transaction

Deep Implementation Guidance

Model identity proof, authentication, authorization, signing, and attribution separately. A Fabric client certificate proves an organizational identity, while the application must still prove which user/device requested the financial action and whether that action was permitted.

Data and Control Questions

- What is the authoritative source for every input used by this component?
- Which user, device, service, MSP, or administrator identity is allowed to initiate or approve it?
- Does the operation move value, reserve value, create a side effect, or only provide advisory information?
- What idempotency/replay key proves that a retry is the same business intent?
- What evidence is retained off-chain and what stable reference/hash belongs on the ledger?

Chapter 68 Implementation - Signed Authorization Envelope

Implementation Procedure

336. Define separate user, device, service, Fabric, and administrator identity models.
337. Authenticate locally and bind roles/permissions to stable IDs.
338. Verify signed device/user intent before creating a ledger command when required.
339. Protect keys using OS security or HSM/PKCS11 for high-value identities.
340. Test revocation, rotation, lockout, replay, and break-glass behavior.

Implementation Example

```
{
  "requestId": "8e2752ef-...",
  "walletId": "WAL-C-1001",
  "intent": "TRANSFER",
  "toWalletId": "WAL-C-2008",
  "asset": "LOCAL",
  "amountMinor": 2500,
  "nonce": "dev-42:00001827",
  "deviceId": "DEV-42",
  "keyVersion": 3,
  "signature": "base64(signature-over-canonical-fields)"
}
```

Server procedure:

1. Load authenticated user/device binding.
2. Verify signature, nonce, amount, destination, asset, and requestId.
3. Compute SHA-256 over canonical authorization bytes.
4. Persist full evidence off-chain.
5. Submit only authDigest + stable refs to the financial ledger.

Verification Checklist

- The authorization envelope schema exists and is version-controlled.
- The feature works with all public Internet/WAN connectivity disabled.
- Authorization, malformed input, duplicate retry, and failure behavior have explicit tests.
- If financial value changes, journal debits and credits balance exactly and historical entries remain immutable.
- If AI is involved, model/prompt/feature version and fallback behavior are recorded and deterministic policy remains final.
- A timeout/restart cannot create a duplicate financial result; request status can be rediscovered.
- Logs/evidence contain correlation references but not secrets or prohibited sensitive payment data.

Common Failure Modes

- Shared operator accounts.
- Browser/terminal holds high-value Fabric key.
- No revocation/rotation test.
- User-supplied identity trusted without authentication binding.

Expert Review Questions

- What happens if this component succeeds but the next component fails?
- How is the authorizing identity/device/key revoked or recovered?
- Can the operation be safely retried after the client loses the response?

- What happens if AI is unavailable, slow, or wrong?
- Can an auditor reconstruct debit, credit, actor, risk decision, approval, and Fabric txId from retained evidence?
- Which compensating transaction resolves a downstream operational error without rewriting history?

Chapter 69 - Nonce and Replay Protection

Prevent reuse of valid signed payment intents. Financial authorization begins with identity. Application users, devices, service identities, Fabric node identities, and signing keys have different trust purposes. The system must bind an authenticated human/device intent to a deterministic financial command without exposing high-value Fabric signing keys to browsers or terminals.

Design rule: Use unique requestId plus nonce/counter

Implementation Objective

The required output is: Replay-control design. It must be version-controlled, reviewable, testable with the WAN disabled, and linked to the architecture/source-of-truth model. For a payment platform, every implementation decision should make it possible to explain who authorized an action, which facts were trusted, which controls ran, what financial journal committed, and how the result is recovered after failure.

Key Engineering Decisions

- Use unique requestId plus nonce/counter
- Reject already consumed authorization digest
- Expire intents according to deterministic/server policy

Deep Implementation Guidance

Model identity proof, authentication, authorization, signing, and attribution separately. A Fabric client certificate proves an organizational identity, while the application must still prove which user/device requested the financial action and whether that action was permitted.

Data and Control Questions

- What is the authoritative source for every input used by this component?
- Which user, device, service, MSP, or administrator identity is allowed to initiate or approve it?
- Does the operation move value, reserve value, create a side effect, or only provide advisory information?
- What idempotency/replay key proves that a retry is the same business intent?
- What evidence is retained off-chain and what stable reference/hash belongs on the ledger?

Chapter 69 Implementation - Nonce and Replay Protection

Implementation Procedure

341. Define separate user, device, service, Fabric, and administrator identity models.
342. Authenticate locally and bind roles/permissions to stable IDs.
343. Verify signed device/user intent before creating a ledger command when required.
344. Protect keys using OS security or HSM/PKCS11 for high-value identities.
345. Test revocation, rotation, lockout, replay, and break-glass behavior.

Implementation Example

```
# Nonce and Replay Protection
INPUT  -> authenticate, validate schema, bind requestId/nonce
FACTS  -> read authoritative wallet/merchant/state data
AI     -> optional bounded recommendation with decisionId/version
POLICY -> enforce ownership, hard limits, approvals, risk fallback
LEDGER -> submit one idempotent deterministic transaction when value changes
EVENT  -> update projection/case/monitoring asynchronously
VERIFY -> discover committed result and retain correlationId + txId evidence
```

Verification Checklist

- The replay-control design exists and is version-controlled.
- The feature works with all public Internet/WAN connectivity disabled.
- Authorization, malformed input, duplicate retry, and failure behavior have explicit tests.
- If financial value changes, journal debits and credits balance exactly and historical entries remain immutable.
- If AI is involved, model/prompt/feature version and fallback behavior are recorded and deterministic policy remains final.
- A timeout/restart cannot create a duplicate financial result; request status can be rediscovered.
- Logs/evidence contain correlation references but not secrets or prohibited sensitive payment data.

Common Failure Modes

- Shared operator accounts.
- Browser/terminal holds high-value Fabric key.
- No revocation/rotation test.
- User-supplied identity trusted without authentication binding.

Expert Review Questions

- What happens if this component succeeds but the next component fails?
- How is the authorizing identity/device/key revoked or recovered?
- Can the operation be safely retried after the client loses the response?
- What happens if AI is unavailable, slow, or wrong?
- Can an auditor reconstruct debit, credit, actor, risk decision, approval, and Fabric txId from retained evidence?
- Which compensating transaction resolves a downstream operational error without rewriting history?

Chapter 70 - Customer Wallet Key Models

Choose between server-custodial, device-assisted, or stronger noncustodial authorization models. Financial authorization begins with identity. Application users, devices, service identities, Fabric node identities, and signing keys have different trust purposes. The system must bind an authenticated human/device intent to a deterministic financial command without exposing high-value Fabric signing keys to browsers or terminals.

Design rule: Server-custodial is simplest for closed-loop enterprise wallet

Implementation Objective

The required output is: Key model ADR. It must be version-controlled, reviewable, testable with the WAN disabled, and linked to the architecture/source-of-truth model. For a payment platform, every implementation decision should make it possible to explain who authorized an action, which facts were trusted, which controls ran, what financial journal committed, and how the result is recovered after failure.

Key Engineering Decisions

- Server-custodial is simplest for closed-loop enterprise wallet
- Device key can provide stronger intent evidence
- Do not imply cryptocurrency-style ownership unless actually implemented

Deep Implementation Guidance

Model identity proof, authentication, authorization, signing, and attribution separately. A Fabric client certificate proves an organizational identity, while the application must still prove which user/device requested the financial action and whether that action was permitted.

Data and Control Questions

- What is the authoritative source for every input used by this component?
- Which user, device, service, MSP, or administrator identity is allowed to initiate or approve it?
- Does the operation move value, reserve value, create a side effect, or only provide advisory information?
- What idempotency/replay key proves that a retry is the same business intent?
- What evidence is retained off-chain and what stable reference/hash belongs on the ledger?

Chapter 70 Implementation - Customer Wallet Key Models

Implementation Procedure

346. Define separate user, device, service, Fabric, and administrator identity models.
347. Authenticate locally and bind roles/permissions to stable IDs.
348. Verify signed device/user intent before creating a ledger command when required.
349. Protect keys using OS security or HSM/PKCS11 for high-value identities.
350. Test revocation, rotation, lockout, replay, and break-glass behavior.

Implementation Example

```
# Customer Wallet Key Models
INPUT  -> authenticate, validate schema, bind requestId/nonce
FACTS  -> read authoritative wallet/merchant/state data
AI     -> optional bounded recommendation with decisionId/version
POLICY -> enforce ownership, hard limits, approvals, risk fallback
LEDGER -> submit one idempotent deterministic transaction when value changes
EVENT  -> update projection/case/monitoring asynchronously
VERIFY -> discover committed result and retain correlationId + txId evidence
```

Verification Checklist

- The key model adr exists and is version-controlled.
- The feature works with all public Internet/WAN connectivity disabled.
- Authorization, malformed input, duplicate retry, and failure behavior have explicit tests.
- If financial value changes, journal debits and credits balance exactly and historical entries remain immutable.
- If AI is involved, model/prompt/feature version and fallback behavior are recorded and deterministic policy remains final.
- A timeout/restart cannot create a duplicate financial result; request status can be rediscovered.
- Logs/evidence contain correlation references but not secrets or prohibited sensitive payment data.

Common Failure Modes

- Shared operator accounts.
- Browser/terminal holds high-value Fabric key.
- No revocation/rotation test.
- User-supplied identity trusted without authentication binding.

Expert Review Questions

- What happens if this component succeeds but the next component fails?
- How is the authorizing identity/device/key revoked or recovered?
- Can the operation be safely retried after the client loses the response?
- What happens if AI is unavailable, slow, or wrong?
- Can an auditor reconstruct debit, credit, actor, risk decision, approval, and Fabric txId from retained evidence?
- Which compensating transaction resolves a downstream operational error without rewriting history?

Chapter 71 - Merchant/POS Key Model

Protect merchant terminal identity and minimize blast radius of a stolen terminal. Financial authorization begins with identity. Application users, devices, service identities, Fabric node identities, and signing keys have different trust purposes. The system must bind an authenticated human/device intent to a deterministic financial command without exposing high-value Fabric signing keys to browsers or terminals.

Design rule: Per-terminal credentials

Implementation Objective

The required output is: POS key plan. It must be version-controlled, reviewable, testable with the WAN disabled, and linked to the architecture/source-of-truth model. For a payment platform, every implementation decision should make it possible to explain who authorized an action, which facts were trusted, which controls ran, what financial journal committed, and how the result is recovered after failure.

Key Engineering Decisions

- Per-terminal credentials
- Device revocation
- No CA admin or treasury key on POS
- Backend enforces merchant/store binding

Deep Implementation Guidance

Model identity proof, authentication, authorization, signing, and attribution separately. A Fabric client certificate proves an organizational identity, while the application must still prove which user/device requested the financial action and whether that action was permitted.

Data and Control Questions

- What is the authoritative source for every input used by this component?
- Which user, device, service, MSP, or administrator identity is allowed to initiate or approve it?
- Does the operation move value, reserve value, create a side effect, or only provide advisory information?
- What idempotency/replay key proves that a retry is the same business intent?
- What evidence is retained off-chain and what stable reference/hash belongs on the ledger?

Chapter 71 Implementation - Merchant/POS Key Model

Implementation Procedure

351. Define separate user, device, service, Fabric, and administrator identity models.
352. Authenticate locally and bind roles/permissions to stable IDs.
353. Verify signed device/user intent before creating a ledger command when required.
354. Protect keys using OS security or HSM/PKCS11 for high-value identities.
355. Test revocation, rotation, lockout, replay, and break-glass behavior.

Implementation Example

```
# Merchant/POS Key Model
INPUT  -> authenticate, validate schema, bind requestId/nonce
FACTS  -> read authoritative wallet/merchant/state data
AI     -> optional bounded recommendation with decisionId/version
POLICY -> enforce ownership, hard limits, approvals, risk fallback
LEDGER -> submit one idempotent deterministic transaction when value changes
EVENT  -> update projection/case/monitoring asynchronously
VERIFY -> discover committed result and retain correlationId + txId evidence
```

Verification Checklist

- The pos key plan exists and is version-controlled.
- The feature works with all public Internet/WAN connectivity disabled.
- Authorization, malformed input, duplicate retry, and failure behavior have explicit tests.
- If financial value changes, journal debits and credits balance exactly and historical entries remain immutable.
- If AI is involved, model/prompt/feature version and fallback behavior are recorded and deterministic policy remains final.
- A timeout/restart cannot create a duplicate financial result; request status can be rediscovered.
- Logs/evidence contain correlation references but not secrets or prohibited sensitive payment data.

Common Failure Modes

- Shared operator accounts.
- Browser/terminal holds high-value Fabric key.
- No revocation/rotation test.
- User-supplied identity trusted without authentication binding.

Expert Review Questions

- What happens if this component succeeds but the next component fails?
- How is the authorizing identity/device/key revoked or recovered?
- Can the operation be safely retried after the client loses the response?
- What happens if AI is unavailable, slow, or wrong?
- Can an auditor reconstruct debit, credit, actor, risk decision, approval, and Fabric txId from retained evidence?
- Which compensating transaction resolves a downstream operational error without rewriting history?

Chapter 72 - Fabric Client Signing Keys

Protect backend Fabric identity used to submit commands. Financial authorization begins with identity. Application users, devices, service identities, Fabric node identities, and signing keys have different trust purposes. The system must bind an authenticated human/device intent to a deterministic financial command without exposing high-value Fabric signing keys to browsers or terminals.

Design rule: Use least-privilege certificate attributes

Implementation Objective

The required output is: Service key SOP. It must be version-controlled, reviewable, testable with the WAN disabled, and linked to the architecture/source-of-truth model. For a payment platform, every implementation decision should make it possible to explain who authorized an action, which facts were trusted, which controls ran, what financial journal committed, and how the result is recovered after failure.

Key Engineering Decisions

- Use least-privilege certificate attributes
- Restrict filesystem/process access
- Consider HSM for high-value deployments
- Rotate and revoke

Deep Implementation Guidance

Model identity proof, authentication, authorization, signing, and attribution separately. A Fabric client certificate proves an organizational identity, while the application must still prove which user/device requested the financial action and whether that action was permitted.

Data and Control Questions

- What is the authoritative source for every input used by this component?
- Which user, device, service, MSP, or administrator identity is allowed to initiate or approve it?
- Does the operation move value, reserve value, create a side effect, or only provide advisory information?
- What idempotency/replay key proves that a retry is the same business intent?
- What evidence is retained off-chain and what stable reference/hash belongs on the ledger?

Chapter 72 Implementation - Fabric Client Signing Keys

Implementation Procedure

356. Define separate user, device, service, Fabric, and administrator identity models.
357. Authenticate locally and bind roles/permissions to stable IDs.
358. Verify signed device/user intent before creating a ledger command when required.
359. Protect keys using OS security or HSM/PKCS11 for high-value identities.
360. Test revocation, rotation, lockout, replay, and break-glass behavior.

Implementation Example

```
# Fabric Client Signing Keys
INPUT  -> authenticate, validate schema, bind requestId/nonce
FACTS  -> read authoritative wallet/merchant/state data
AI     -> optional bounded recommendation with decisionId/version
POLICY -> enforce ownership, hard limits, approvals, risk fallback
LEDGER -> submit one idempotent deterministic transaction when value changes
EVENT  -> update projection/case/monitoring asynchronously
VERIFY -> discover committed result and retain correlationId + txId evidence
```

Verification Checklist

- The service key sop exists and is version-controlled.
- The feature works with all public Internet/WAN connectivity disabled.
- Authorization, malformed input, duplicate retry, and failure behavior have explicit tests.
- If financial value changes, journal debits and credits balance exactly and historical entries remain immutable.
- If AI is involved, model/prompt/feature version and fallback behavior are recorded and deterministic policy remains final.
- A timeout/restart cannot create a duplicate financial result; request status can be rediscovered.
- Logs/evidence contain correlation references but not secrets or prohibited sensitive payment data.

Common Failure Modes

- Shared operator accounts.
- Browser/terminal holds high-value Fabric key.
- No revocation/rotation test.
- User-supplied identity trusted without authentication binding.

Expert Review Questions

- What happens if this component succeeds but the next component fails?
- How is the authorizing identity/device/key revoked or recovered?
- Can the operation be safely retried after the client loses the response?
- What happens if AI is unavailable, slow, or wrong?
- Can an auditor reconstruct debit, credit, actor, risk decision, approval, and Fabric txId from retained evidence?
- Which compensating transaction resolves a downstream operational error without rewriting history?

Chapter 73 - HSM/PKCS11 Operations

Integrate local hardware security modules for protected signing where required. Financial authorization begins with identity. Application users, devices, service identities, Fabric node identities, and signing keys have different trust purposes. The system must bind an authenticated human/device intent to a deterministic financial command without exposing high-value Fabric signing keys to browsers or terminals.

Design rule: Provision keys securely

Implementation Objective

The required output is: HSM runbook. It must be version-controlled, reviewable, testable with the WAN disabled, and linked to the architecture/source-of-truth model. For a payment platform, every implementation decision should make it possible to explain who authorized an action, which facts were trusted, which controls ran, what financial journal committed, and how the result is recovered after failure.

Key Engineering Decisions

- Provision keys securely
- Back up according to vendor policy
- Test failover/replacement
- Document PIN/operator separation

Deep Implementation Guidance

Model identity proof, authentication, authorization, signing, and attribution separately. A Fabric client certificate proves an organizational identity, while the application must still prove which user/device requested the financial action and whether that action was permitted.

Data and Control Questions

- What is the authoritative source for every input used by this component?
- Which user, device, service, MSP, or administrator identity is allowed to initiate or approve it?
- Does the operation move value, reserve value, create a side effect, or only provide advisory information?
- What idempotency/replay key proves that a retry is the same business intent?
- What evidence is retained off-chain and what stable reference/hash belongs on the ledger?

Chapter 73 Implementation - HSM/PKCS11 Operations

Implementation Procedure

361. Define separate user, device, service, Fabric, and administrator identity models.
362. Authenticate locally and bind roles/permissions to stable IDs.
363. Verify signed device/user intent before creating a ledger command when required.
364. Protect keys using OS security or HSM/PKCS11 for high-value identities.
365. Test revocation, rotation, lockout, replay, and break-glass behavior.

Implementation Example

```
# HSM/PKCS11 Operations
INPUT  -> authenticate, validate schema, bind requestId/nonce
FACTS  -> read authoritative wallet/merchant/state data
AI     -> optional bounded recommendation with decisionId/version
POLICY -> enforce ownership, hard limits, approvals, risk fallback
LEDGER -> submit one idempotent deterministic transaction when value changes
EVENT  -> update projection/case/monitoring asynchronously
VERIFY -> discover committed result and retain correlationId + txId evidence
```

Verification Checklist

- The hsm runbook exists and is version-controlled.
- The feature works with all public Internet/WAN connectivity disabled.
- Authorization, malformed input, duplicate retry, and failure behavior have explicit tests.
- If financial value changes, journal debits and credits balance exactly and historical entries remain immutable.
- If AI is involved, model/prompt/feature version and fallback behavior are recorded and deterministic policy remains final.
- A timeout/restart cannot create a duplicate financial result; request status can be rediscovered.
- Logs/evidence contain correlation references but not secrets or prohibited sensitive payment data.

Common Failure Modes

- Shared operator accounts.
- Browser/terminal holds high-value Fabric key.
- No revocation/rotation test.
- User-supplied identity trusted without authentication binding.

Expert Review Questions

- What happens if this component succeeds but the next component fails?
- How is the authorizing identity/device/key revoked or recovered?
- Can the operation be safely retried after the client loses the response?
- What happens if AI is unavailable, slow, or wrong?
- Can an auditor reconstruct debit, credit, actor, risk decision, approval, and Fabric txId from retained evidence?
- Which compensating transaction resolves a downstream operational error without rewriting history?

Chapter 74 - Key Rotation

Rotate user/device/service/Fabric keys without breaking audit continuity. Financial authorization begins with identity. Application users, devices, service identities, Fabric node identities, and signing keys have different trust purposes. The system must bind an authenticated human/device intent to a deterministic financial command without exposing high-value Fabric signing keys to browsers or terminals.

Design rule: New key gets new fingerprint/version

Implementation Objective

The required output is: Rotation workflow. It must be version-controlled, reviewable, testable with the WAN disabled, and linked to the architecture/source-of-truth model. For a payment platform, every implementation decision should make it possible to explain who authorized an action, which facts were trusted, which controls ran, what financial journal committed, and how the result is recovered after failure.

Key Engineering Decisions

- New key gets new fingerprint/version
- Old key revoked/retired
- Historical transactions keep original signer evidence

Deep Implementation Guidance

Model identity proof, authentication, authorization, signing, and attribution separately. A Fabric client certificate proves an organizational identity, while the application must still prove which user/device requested the financial action and whether that action was permitted.

Data and Control Questions

- What is the authoritative source for every input used by this component?
- Which user, device, service, MSP, or administrator identity is allowed to initiate or approve it?
- Does the operation move value, reserve value, create a side effect, or only provide advisory information?
- What idempotency/replay key proves that a retry is the same business intent?
- What evidence is retained off-chain and what stable reference/hash belongs on the ledger?

Chapter 74 Implementation - Key Rotation

Implementation Procedure

366. Define separate user, device, service, Fabric, and administrator identity models.
367. Authenticate locally and bind roles/permissions to stable IDs.
368. Verify signed device/user intent before creating a ledger command when required.
369. Protect keys using OS security or HSM/PKCS11 for high-value identities.
370. Test revocation, rotation, lockout, replay, and break-glass behavior.

Implementation Example

```
# Key Rotation
INPUT  -> authenticate, validate schema, bind requestId/nonce
FACTS  -> read authoritative wallet/merchant/state data
AI     -> optional bounded recommendation with decisionId/version
POLICY -> enforce ownership, hard limits, approvals, risk fallback
LEDGER -> submit one idempotent deterministic transaction when value changes
EVENT  -> update projection/case/monitoring asynchronously
VERIFY -> discover committed result and retain correlationId + txId evidence
```

Verification Checklist

- The rotation workflow exists and is version-controlled.
- The feature works with all public Internet/WAN connectivity disabled.
- Authorization, malformed input, duplicate retry, and failure behavior have explicit tests.
- If financial value changes, journal debits and credits balance exactly and historical entries remain immutable.
- If AI is involved, model/prompt/feature version and fallback behavior are recorded and deterministic policy remains final.
- A timeout/restart cannot create a duplicate financial result; request status can be rediscovered.
- Logs/evidence contain correlation references but not secrets or prohibited sensitive payment data.

Common Failure Modes

- Shared operator accounts.
- Browser/terminal holds high-value Fabric key.
- No revocation/rotation test.
- User-supplied identity trusted without authentication binding.

Expert Review Questions

- What happens if this component succeeds but the next component fails?
- How is the authorizing identity/device/key revoked or recovered?
- Can the operation be safely retried after the client loses the response?
- What happens if AI is unavailable, slow, or wrong?
- Can an auditor reconstruct debit, credit, actor, risk decision, approval, and Fabric txId from retained evidence?
- Which compensating transaction resolves a downstream operational error without rewriting history?

Chapter 75 - Certificate Revocation

Stop compromised Fabric identities from being accepted. Financial authorization begins with identity. Application users, devices, service identities, Fabric node identities, and signing keys have different trust purposes. The system must bind an authenticated human/device intent to a deterministic financial command without exposing high-value Fabric signing keys to browsers or terminals.

Design rule: Revoke in CA

Implementation Objective

The required output is: Revocation procedure. It must be version-controlled, reviewable, testable with the WAN disabled, and linked to the architecture/source-of-truth model. For a payment platform, every implementation decision should make it possible to explain who authorized an action, which facts were trusted, which controls ran, what financial journal committed, and how the result is recovered after failure.

Key Engineering Decisions

- Revoke in CA
- Regenerate/distribute CRL as required
- Restart/reload affected components
- Test denial

Deep Implementation Guidance

Model identity proof, authentication, authorization, signing, and attribution separately. A Fabric client certificate proves an organizational identity, while the application must still prove which user/device requested the financial action and whether that action was permitted.

Data and Control Questions

- What is the authoritative source for every input used by this component?
- Which user, device, service, MSP, or administrator identity is allowed to initiate or approve it?
- Does the operation move value, reserve value, create a side effect, or only provide advisory information?
- What idempotency/replay key proves that a retry is the same business intent?
- What evidence is retained off-chain and what stable reference/hash belongs on the ledger?

Chapter 75 Implementation - Certificate Revocation

Implementation Procedure

371. Define separate user, device, service, Fabric, and administrator identity models.
372. Authenticate locally and bind roles/permissions to stable IDs.
373. Verify signed device/user intent before creating a ledger command when required.
374. Protect keys using OS security or HSM/PKCS11 for high-value identities.
375. Test revocation, rotation, lockout, replay, and break-glass behavior.

Implementation Example

```
# Certificate Revocation
INPUT  -> authenticate, validate schema, bind requestId/nonce
FACTS  -> read authoritative wallet/merchant/state data
AI     -> optional bounded recommendation with decisionId/version
POLICY -> enforce ownership, hard limits, approvals, risk fallback
LEDGER -> submit one idempotent deterministic transaction when value changes
EVENT  -> update projection/case/monitoring asynchronously
VERIFY -> discover committed result and retain correlationId + txId evidence
```

Verification Checklist

- The revocation procedure exists and is version-controlled.
- The feature works with all public Internet/WAN connectivity disabled.
- Authorization, malformed input, duplicate retry, and failure behavior have explicit tests.
- If financial value changes, journal debits and credits balance exactly and historical entries remain immutable.
- If AI is involved, model/prompt/feature version and fallback behavior are recorded and deterministic policy remains final.
- A timeout/restart cannot create a duplicate financial result; request status can be rediscovered.
- Logs/evidence contain correlation references but not secrets or prohibited sensitive payment data.

Common Failure Modes

- Shared operator accounts.
- Browser/terminal holds high-value Fabric key.
- No revocation/rotation test.
- User-supplied identity trusted without authentication binding.

Expert Review Questions

- What happens if this component succeeds but the next component fails?
- How is the authorizing identity/device/key revoked or recovered?
- Can the operation be safely retried after the client loses the response?
- What happens if AI is unavailable, slow, or wrong?
- Can an auditor reconstruct debit, credit, actor, risk decision, approval, and Fabric txId from retained evidence?
- Which compensating transaction resolves a downstream operational error without rewriting history?

Chapter 76 - Break-Glass Administration

Provide emergency access without creating a permanent superuser shortcut. Financial authorization begins with identity. Application users, devices, service identities, Fabric node identities, and signing keys have different trust purposes. The system must bind an authenticated human/device intent to a deterministic financial command without exposing high-value Fabric signing keys to browsers or terminals.

Design rule: Store sealed/restricted credentials

Implementation Objective

The required output is: Break-glass procedure. It must be version-controlled, reviewable, testable with the WAN disabled, and linked to the architecture/source-of-truth model. For a payment platform, every implementation decision should make it possible to explain who authorized an action, which facts were trusted, which controls ran, what financial journal committed, and how the result is recovered after failure.

Key Engineering Decisions

- Store sealed/restricted credentials
- Require dual control and incident ticket
- Log every use
- Rotate after use

Deep Implementation Guidance

Model identity proof, authentication, authorization, signing, and attribution separately. A Fabric client certificate proves an organizational identity, while the application must still prove which user/device requested the financial action and whether that action was permitted.

Data and Control Questions

- What is the authoritative source for every input used by this component?
- Which user, device, service, MSP, or administrator identity is allowed to initiate or approve it?
- Does the operation move value, reserve value, create a side effect, or only provide advisory information?
- What idempotency/replay key proves that a retry is the same business intent?
- What evidence is retained off-chain and what stable reference/hash belongs on the ledger?

Chapter 76 Implementation - Break-Glass Administration

Implementation Procedure

376. Define separate user, device, service, Fabric, and administrator identity models.
377. Authenticate locally and bind roles/permissions to stable IDs.
378. Verify signed device/user intent before creating a ledger command when required.
379. Protect keys using OS security or HSM/PKCS11 for high-value identities.
380. Test revocation, rotation, lockout, replay, and break-glass behavior.

Implementation Example

```
# Break-Glass Administration
INPUT  -> authenticate, validate schema, bind requestId/nonce
FACTS  -> read authoritative wallet/merchant/state data
AI     -> optional bounded recommendation with decisionId/version
POLICY -> enforce ownership, hard limits, approvals, risk fallback
LEDGER -> submit one idempotent deterministic transaction when value changes
EVENT  -> update projection/case/monitoring asynchronously
VERIFY -> discover committed result and retain correlationId + txId evidence
```

Verification Checklist

- The break-glass procedure exists and is version-controlled.
- The feature works with all public Internet/WAN connectivity disabled.
- Authorization, malformed input, duplicate retry, and failure behavior have explicit tests.
- If financial value changes, journal debits and credits balance exactly and historical entries remain immutable.
- If AI is involved, model/prompt/feature version and fallback behavior are recorded and deterministic policy remains final.
- A timeout/restart cannot create a duplicate financial result; request status can be rediscovered.
- Logs/evidence contain correlation references but not secrets or prohibited sensitive payment data.

Common Failure Modes

- Shared operator accounts.
- Browser/terminal holds high-value Fabric key.
- No revocation/rotation test.
- User-supplied identity trusted without authentication binding.

Expert Review Questions

- What happens if this component succeeds but the next component fails?
- How is the authorizing identity/device/key revoked or recovered?
- Can the operation be safely retried after the client loses the response?
- What happens if AI is unavailable, slow, or wrong?
- Can an auditor reconstruct debit, credit, actor, risk decision, approval, and Fabric txId from retained evidence?
- Which compensating transaction resolves a downstream operational error without rewriting history?

Chapter 77 - Treasury Dual Control

Require two independent approvals for high-risk issuance/redemption/settlement overrides. Financial authorization begins with identity. Application users, devices, service identities, Fabric node identities, and signing keys have different trust purposes. The system must bind an authenticated human/device intent to a deterministic financial command without exposing high-value Fabric signing keys to browsers or terminals.

Design rule: Requester cannot self-approve

Implementation Objective

The required output is: Dual-control workflow. It must be version-controlled, reviewable, testable with the WAN disabled, and linked to the architecture/source-of-truth model. For a payment platform, every implementation decision should make it possible to explain who authorized an action, which facts were trusted, which controls ran, what financial journal committed, and how the result is recovered after failure.

Key Engineering Decisions

- Requester cannot self-approve
- Approval has amount/action scope and expiry
- Ledger command references approval ID/hash

Deep Implementation Guidance

Model identity proof, authentication, authorization, signing, and attribution separately. A Fabric client certificate proves an organizational identity, while the application must still prove which user/device requested the financial action and whether that action was permitted.

Data and Control Questions

- What is the authoritative source for every input used by this component?
- Which user, device, service, MSP, or administrator identity is allowed to initiate or approve it?
- Does the operation move value, reserve value, create a side effect, or only provide advisory information?
- What idempotency/replay key proves that a retry is the same business intent?
- What evidence is retained off-chain and what stable reference/hash belongs on the ledger?

Chapter 77 Implementation - Treasury Dual Control

Implementation Procedure

381. Define separate user, device, service, Fabric, and administrator identity models.
382. Authenticate locally and bind roles/permissions to stable IDs.
383. Verify signed device/user intent before creating a ledger command when required.
384. Protect keys using OS security or HSM/PKCS11 for high-value identities.
385. Test revocation, rotation, lockout, replay, and break-glass behavior.

Implementation Example

```
# Treasury Dual Control
INPUT  -> authenticate, validate schema, bind requestId/nonce
FACTS  -> read authoritative wallet/merchant/state data
AI     -> optional bounded recommendation with decisionId/version
POLICY -> enforce ownership, hard limits, approvals, risk fallback
LEDGER -> submit one idempotent deterministic transaction when value changes
EVENT  -> update projection/case/monitoring asynchronously
VERIFY -> discover committed result and retain correlationId + txId evidence
```

Verification Checklist

- The dual-control workflow exists and is version-controlled.
- The feature works with all public Internet/WAN connectivity disabled.
- Authorization, malformed input, duplicate retry, and failure behavior have explicit tests.
- If financial value changes, journal debits and credits balance exactly and historical entries remain immutable.
- If AI is involved, model/prompt/feature version and fallback behavior are recorded and deterministic policy remains final.
- A timeout/restart cannot create a duplicate financial result; request status can be rediscovered.
- Logs/evidence contain correlation references but not secrets or prohibited sensitive payment data.

Common Failure Modes

- Shared operator accounts.
- Browser/terminal holds high-value Fabric key.
- No revocation/rotation test.
- User-supplied identity trusted without authentication binding.

Expert Review Questions

- What happens if this component succeeds but the next component fails?
- How is the authorizing identity/device/key revoked or recovered?
- Can the operation be safely retried after the client loses the response?
- What happens if AI is unavailable, slow, or wrong?
- Can an auditor reconstruct debit, credit, actor, risk decision, approval, and Fabric txId from retained evidence?
- Which compensating transaction resolves a downstream operational error without rewriting history?

Chapter 78 - Audit Attribution

Make each financial action reconstructable to user, device, service identity, AI decision, approval, and Fabric transaction. Financial authorization begins with identity. Application users, devices, service identities, Fabric node identities, and signing keys have different trust purposes. The system must bind an authenticated human/device intent to a deterministic financial command without exposing high-value Fabric signing keys to browsers or terminals.

Design rule: Persist actorRef/deviceRef/requestId/decisionId/approvalId/txId

Implementation Objective

The required output is: Attribution schema. It must be version-controlled, reviewable, testable with the WAN disabled, and linked to the architecture/source-of-truth model. For a payment platform, every implementation decision should make it possible to explain who authorized an action, which facts were trusted, which controls ran, what financial journal committed, and how the result is recovered after failure.

Key Engineering Decisions

- Persist actorRef/deviceRef/requestId/decisionId/approvalId/txId
- Do not expose sensitive identity data unnecessarily
- Keep mapping off-chain

Deep Implementation Guidance

Model identity proof, authentication, authorization, signing, and attribution separately. A Fabric client certificate proves an organizational identity, while the application must still prove which user/device requested the financial action and whether that action was permitted.

Data and Control Questions

- What is the authoritative source for every input used by this component?
- Which user, device, service, MSP, or administrator identity is allowed to initiate or approve it?
- Does the operation move value, reserve value, create a side effect, or only provide advisory information?
- What idempotency/replay key proves that a retry is the same business intent?
- What evidence is retained off-chain and what stable reference/hash belongs on the ledger?

Chapter 78 Implementation - Audit Attribution

Implementation Procedure

386. Define separate user, device, service, Fabric, and administrator identity models.
387. Authenticate locally and bind roles/permissions to stable IDs.
388. Verify signed device/user intent before creating a ledger command when required.
389. Protect keys using OS security or HSM/PKCS11 for high-value identities.
390. Test revocation, rotation, lockout, replay, and break-glass behavior.

Implementation Example

```
# Audit Attribution
INPUT  -> authenticate, validate schema, bind requestId/nonce
FACTS  -> read authoritative wallet/merchant/state data
AI     -> optional bounded recommendation with decisionId/version
POLICY -> enforce ownership, hard limits, approvals, risk fallback
LEDGER -> submit one idempotent deterministic transaction when value changes
EVENT  -> update projection/case/monitoring asynchronously
VERIFY -> discover committed result and retain correlationId + txId evidence
```

Verification Checklist

- The attribution schema exists and is version-controlled.
- The feature works with all public Internet/WAN connectivity disabled.
- Authorization, malformed input, duplicate retry, and failure behavior have explicit tests.
- If financial value changes, journal debits and credits balance exactly and historical entries remain immutable.
- If AI is involved, model/prompt/feature version and fallback behavior are recorded and deterministic policy remains final.
- A timeout/restart cannot create a duplicate financial result; request status can be rediscovered.
- Logs/evidence contain correlation references but not secrets or prohibited sensitive payment data.

Common Failure Modes

- Shared operator accounts.
- Browser/terminal holds high-value Fabric key.
- No revocation/rotation test.
- User-supplied identity trusted without authentication binding.

Expert Review Questions

- What happens if this component succeeds but the next component fails?
- How is the authorizing identity/device/key revoked or recovered?
- Can the operation be safely retried after the client loses the response?
- What happens if AI is unavailable, slow, or wrong?
- Can an auditor reconstruct debit, credit, actor, risk decision, approval, and Fabric txId from retained evidence?
- Which compensating transaction resolves a downstream operational error without rewriting history?

Chapter 79 - Identity Recovery and Account Takeover Handling

Recover legitimate users while preventing attackers from bypassing wallet controls. Financial authorization begins with identity. Application users, devices, service identities, Fabric node identities, and signing keys have different trust purposes. The system must bind an authenticated human/device intent to a deterministic financial command without exposing high-value Fabric signing keys to browsers or terminals.

Design rule: Use verified local process

Implementation Objective

The required output is: Recovery runbook. It must be version-controlled, reviewable, testable with the WAN disabled, and linked to the architecture/source-of-truth model. For a payment platform, every implementation decision should make it possible to explain who authorized an action, which facts were trusted, which controls ran, what financial journal committed, and how the result is recovered after failure.

Key Engineering Decisions

- Use verified local process
- Freeze sensitive actions during recovery
- Invalidate old sessions/devices
- Review recent transactions

Deep Implementation Guidance

Model identity proof, authentication, authorization, signing, and attribution separately. A Fabric client certificate proves an organizational identity, while the application must still prove which user/device requested the financial action and whether that action was permitted.

Data and Control Questions

- What is the authoritative source for every input used by this component?
- Which user, device, service, MSP, or administrator identity is allowed to initiate or approve it?
- Does the operation move value, reserve value, create a side effect, or only provide advisory information?
- What idempotency/replay key proves that a retry is the same business intent?
- What evidence is retained off-chain and what stable reference/hash belongs on the ledger?

Chapter 79 Implementation - Identity Recovery and Account Takeover Handling

Implementation Procedure

391. Define separate user, device, service, Fabric, and administrator identity models.
392. Authenticate locally and bind roles/permissions to stable IDs.
393. Verify signed device/user intent before creating a ledger command when required.
394. Protect keys using OS security or HSM/PKCS11 for high-value identities.
395. Test revocation, rotation, lockout, replay, and break-glass behavior.

Implementation Example

```
# Identity Recovery and Account Takeover Handling
INPUT  -> authenticate, validate schema, bind requestId/nonce
FACTS  -> read authoritative wallet/merchant/state data
AI      -> optional bounded recommendation with decisionId/version
POLICY  -> enforce ownership, hard limits, approvals, risk fallback
LEDGER  -> submit one idempotent deterministic transaction when value changes
EVENT   -> update projection/case/monitoring asynchronously
VERIFY  -> discover committed result and retain correlationId + txId evidence
```

Verification Checklist

- The recovery runbook exists and is version-controlled.
- The feature works with all public Internet/WAN connectivity disabled.
- Authorization, malformed input, duplicate retry, and failure behavior have explicit tests.
- If financial value changes, journal debits and credits balance exactly and historical entries remain immutable.
- If AI is involved, model/prompt/feature version and fallback behavior are recorded and deterministic policy remains final.
- A timeout/restart cannot create a duplicate financial result; request status can be rediscovered.
- Logs/evidence contain correlation references but not secrets or prohibited sensitive payment data.

Common Failure Modes

- Shared operator accounts.
- Browser/terminal holds high-value Fabric key.
- No revocation/rotation test.
- User-supplied identity trusted without authentication binding.

Expert Review Questions

- What happens if this component succeeds but the next component fails?
- How is the authorizing identity/device/key revoked or recovered?
- Can the operation be safely retried after the client loses the response?
- What happens if AI is unavailable, slow, or wrong?
- Can an auditor reconstruct debit, credit, actor, risk decision, approval, and Fabric txId from retained evidence?
- Which compensating transaction resolves a downstream operational error without rewriting history?

Part VI - TypeScript Chaincode and Deterministic Financial Rules

Chaincode is the deterministic financial rules engine. It must validate identities, states, balances, limits, idempotency keys, asset codes, and balanced journal entries without network calls, AI inference, wall-clock assumptions, floating-point money, or mutable external dependencies.

Part focus: This part contains 26 implementation chapters. Each chapter includes design rules, implementation procedure, a practical artifact, verification, failure analysis, and expert review questions.

Chapter 80 - Chaincode Project Structure

Organize contracts, models, validation, accounting helpers, indexes, and tests. Chaincode is the deterministic financial rules engine. It must validate identities, states, balances, limits, idempotency keys, asset codes, and balanced journal entries without network calls, AI inference, wall-clock assumptions, floating-point money, or mutable external dependencies.

Design rule: Separate domain helpers from contract methods

Implementation Objective

The required output is: Repository structure. It must be version-controlled, reviewable, testable with the WAN disabled, and linked to the architecture/source-of-truth model. For a payment platform, every implementation decision should make it possible to explain who authorized an action, which facts were trusted, which controls ran, what financial journal committed, and how the result is recovered after failure.

Key Engineering Decisions

- Separate domain helpers from contract methods
- Keep schemas/version constants centralized
- Use deterministic serialization conventions

Deep Implementation Guidance

Keep all writes required for one logical financial action in one transaction when atomicity is required. Use deterministic validation and stable error codes. Never call an external API, database, clock service, random generator, or AI model from consensus-critical chaincode.

Data and Control Questions

- What is the authoritative source for every input used by this component?
- Which user, device, service, MSP, or administrator identity is allowed to initiate or approve it?
- Does the operation move value, reserve value, create a side effect, or only provide advisory information?
- What idempotency/replay key proves that a retry is the same business intent?
- What evidence is retained off-chain and what stable reference/hash belongs on the ledger?

Chapter 80 Implementation - Chaincode Project Structure

Implementation Procedure

396. Parse and validate every argument deterministically.
397. Authorize caller MSP/attributes and validate entity lifecycle state.
398. Check idempotency before any business write.
399. Write balanced journal/current state atomically and emit compact events.
400. Return stable error codes and test concurrent/MVCC conflict behavior.

Implementation Example

```
# Chaincode Project Structure
INPUT  -> authenticate, validate schema, bind requestId/nonce
FACTS  -> read authoritative wallet/merchant/state data
AI     -> optional bounded recommendation with decisionId/version
POLICY -> enforce ownership, hard limits, approvals, risk fallback
LEDGER -> submit one idempotent deterministic transaction when value changes
EVENT  -> update projection/case/monitoring asynchronously
VERIFY -> discover committed result and retain correlationId + txId evidence
```

Verification Checklist

- The repository structure exists and is version-controlled.
- The feature works with all public Internet/WAN connectivity disabled.
- Authorization, malformed input, duplicate retry, and failure behavior have explicit tests.
- If financial value changes, journal debits and credits balance exactly and historical entries remain immutable.
- If AI is involved, model/prompt/feature version and fallback behavior are recorded and deterministic policy remains final.
- A timeout/restart cannot create a duplicate financial result; request status can be rediscovered.
- Logs/evidence contain correlation references but not secrets or prohibited sensitive payment data.

Common Failure Modes

- External HTTP/AI call from chaincode.
- Non-deterministic timestamps/randomness.
- No duplicate-request guard.
- Balance checks split across non-atomic transactions.

Expert Review Questions

- What happens if this component succeeds but the next component fails?
- How is the authorizing identity/device/key revoked or recovered?
- Can the operation be safely retried after the client loses the response?
- What happens if AI is unavailable, slow, or wrong?
- Can an auditor reconstruct debit, credit, actor, risk decision, approval, and Fabric txId from retained evidence?
- Which compensating transaction resolves a downstream operational error without rewriting history?

Chapter 81 - Wallet State Model

Define on-chain current wallet state and immutable references. Chaincode is the deterministic financial rules engine. It must validate identities, states, balances, limits, idempotency keys, asset codes, and balanced journal entries without network calls, AI inference, wall-clock assumptions, floating-point money, or mutable external dependencies.

Design rule: walletId ownerRefHash asset type status available/reserved balance counters

Implementation Objective

The required output is: Wallet chaincode schema. It must be version-controlled, reviewable, testable with the WAN disabled, and linked to the architecture/source-of-truth model. For a payment platform, every implementation decision should make it possible to explain who authorized an action, which facts were trusted, which controls ran, what financial journal committed, and how the result is recovered after failure.

Key Engineering Decisions

- walletId ownerRefHash asset type status available/reserved balance counters
- Avoid PII
- Use explicit schemaVersion

Deep Implementation Guidance

Keep all writes required for one logical financial action in one transaction when atomicity is required. Use deterministic validation and stable error codes. Never call an external API, database, clock service, random generator, or AI model from consensus-critical chaincode.

Data and Control Questions

- What is the authoritative source for every input used by this component?
- Which user, device, service, MSP, or administrator identity is allowed to initiate or approve it?
- Does the operation move value, reserve value, create a side effect, or only provide advisory information?
- What idempotency/replay key proves that a retry is the same business intent?
- What evidence is retained off-chain and what stable reference/hash belongs on the ledger?

Chapter 81 Implementation - Wallet State Model

Implementation Procedure

401. Parse and validate every argument deterministically.
402. Authorize caller MSP/attributes and validate entity lifecycle state.
403. Check idempotency before any business write.
404. Write balanced journal/current state atomically and emit compact events.
405. Return stable error codes and test concurrent/MVCC conflict behavior.

Implementation Example

```
# Wallet State Model
INPUT  -> authenticate, validate schema, bind requestId/nonce
FACTS  -> read authoritative wallet/merchant/state data
AI      -> optional bounded recommendation with decisionId/version
POLICY  -> enforce ownership, hard limits, approvals, risk fallback
LEDGER  -> submit one idempotent deterministic transaction when value changes
EVENT   -> update projection/case/monitoring asynchronously
VERIFY  -> discover committed result and retain correlationId + txId evidence
```

Verification Checklist

- The wallet chaincode schema exists and is version-controlled.
- The feature works with all public Internet/WAN connectivity disabled.
- Authorization, malformed input, duplicate retry, and failure behavior have explicit tests.
- If financial value changes, journal debits and credits balance exactly and historical entries remain immutable.
- If AI is involved, model/prompt/feature version and fallback behavior are recorded and deterministic policy remains final.
- A timeout/restart cannot create a duplicate financial result; request status can be rediscovered.
- Logs/evidence contain correlation references but not secrets or prohibited sensitive payment data.

Common Failure Modes

- External HTTP/AI call from chaincode.
- Non-deterministic timestamps/randomness.
- No duplicate-request guard.
- Balance checks split across non-atomic transactions.

Expert Review Questions

- What happens if this component succeeds but the next component fails?
- How is the authorizing identity/device/key revoked or recovered?
- Can the operation be safely retried after the client loses the response?
- What happens if AI is unavailable, slow, or wrong?
- Can an auditor reconstruct debit, credit, actor, risk decision, approval, and Fabric txId from retained evidence?
- Which compensating transaction resolves a downstream operational error without rewriting history?

Chapter 82 - Journal Model

Represent every financial transaction as a balanced immutable journal. Chaincode is the deterministic financial rules engine. It must validate identities, states, balances, limits, idempotency keys, asset codes, and balanced journal entries without network calls, AI inference, wall-clock assumptions, floating-point money, or mutable external dependencies.

Design rule: journalId type asset requestId reference actorRef lines[] status

Implementation Objective

The required output is: Journal schema. It must be version-controlled, reviewable, testable with the WAN disabled, and linked to the architecture/source-of-truth model. For a payment platform, every implementation decision should make it possible to explain who authorized an action, which facts were trusted, which controls ran, what financial journal committed, and how the result is recovered after failure.

Key Engineering Decisions

- journalId type asset requestId reference actorRef lines[] status
- Each line has account/wallet debitMinor/creditMinor
- Sum debits equals credits

Deep Implementation Guidance

Keep all writes required for one logical financial action in one transaction when atomicity is required. Use deterministic validation and stable error codes. Never call an external API, database, clock service, random generator, or AI model from consensus-critical chaincode.

Data and Control Questions

- What is the authoritative source for every input used by this component?
- Which user, device, service, MSP, or administrator identity is allowed to initiate or approve it?
- Does the operation move value, reserve value, create a side effect, or only provide advisory information?
- What idempotency/replay key proves that a retry is the same business intent?
- What evidence is retained off-chain and what stable reference/hash belongs on the ledger?

Chapter 82 Implementation - Journal Model

Implementation Procedure

- 406. Parse and validate every argument deterministically.
- 407. Authorize caller MSP/attributes and validate entity lifecycle state.
- 408. Check idempotency before any business write.
- 409. Write balanced journal/current state atomically and emit compact events.
- 410. Return stable error codes and test concurrent/MVCC conflict behavior.

Implementation Example

```
# Journal Model
INPUT  -> authenticate, validate schema, bind requestId/nonce
FACTS  -> read authoritative wallet/merchant/state data
AI     -> optional bounded recommendation with decisionId/version
POLICY -> enforce ownership, hard limits, approvals, risk fallback
LEDGER -> submit one idempotent deterministic transaction when value changes
EVENT  -> update projection/case/monitoring asynchronously
VERIFY -> discover committed result and retain correlationId + txId evidence
```

Verification Checklist

- The journal schema exists and is version-controlled.
- The feature works with all public Internet/WAN connectivity disabled.
- Authorization, malformed input, duplicate retry, and failure behavior have explicit tests.
- If financial value changes, journal debits and credits balance exactly and historical entries remain immutable.
- If AI is involved, model/prompt/feature version and fallback behavior are recorded and deterministic policy remains final.
- A timeout/restart cannot create a duplicate financial result; request status can be rediscovered.
- Logs/evidence contain correlation references but not secrets or prohibited sensitive payment data.

Common Failure Modes

- External HTTP/AI call from chaincode.
- Non-deterministic timestamps/randomness.
- No duplicate-request guard.
- Balance checks split across non-atomic transactions.

Expert Review Questions

- What happens if this component succeeds but the next component fails?
- How is the authorizing identity/device/key revoked or recovered?
- Can the operation be safely retried after the client loses the response?
- What happens if AI is unavailable, slow, or wrong?
- Can an auditor reconstruct debit, credit, actor, risk decision, approval, and Fabric txId from retained evidence?
- Which compensating transaction resolves a downstream operational error without rewriting history?

Chapter 83 - Accounting Helper Functions

Centralize safe integer arithmetic and balancing checks. Chaincode is the deterministic financial rules engine. It must validate identities, states, balances, limits, idempotency keys, asset codes, and balanced journal entries without network calls, AI inference, wall-clock assumptions, floating-point money, or mutable external dependencies.

Design rule: Use Number safe integers or bigint serialization consistently

Implementation Objective

The required output is: Accounting library. It must be version-controlled, reviewable, testable with the WAN disabled, and linked to the architecture/source-of-truth model. For a payment platform, every implementation decision should make it possible to explain who authorized an action, which facts were trusted, which controls ran, what financial journal committed, and how the result is recovered after failure.

Key Engineering Decisions

- Use Number safe integers or bigint serialization consistently
- Reject negative/overflow amounts
- Assert asset consistency and balanced lines

Deep Implementation Guidance

Keep all writes required for one logical financial action in one transaction when atomicity is required. Use deterministic validation and stable error codes. Never call an external API, database, clock service, random generator, or AI model from consensus-critical chaincode.

Data and Control Questions

- What is the authoritative source for every input used by this component?
- Which user, device, service, MSP, or administrator identity is allowed to initiate or approve it?
- Does the operation move value, reserve value, create a side effect, or only provide advisory information?
- What idempotency/replay key proves that a retry is the same business intent?
- What evidence is retained off-chain and what stable reference/hash belongs on the ledger?

Chapter 83 Implementation - Accounting Helper Functions

Implementation Procedure

411. Parse and validate every argument deterministically.
412. Authorize caller MSP/attributes and validate entity lifecycle state.
413. Check idempotency before any business write.
414. Write balanced journal/current state atomically and emit compact events.
415. Return stable error codes and test concurrent/MVCC conflict behavior.

Implementation Example

```
# Accounting Helper Functions
INPUT  -> authenticate, validate schema, bind requestId/nonce
FACTS  -> read authoritative wallet/merchant/state data
AI     -> optional bounded recommendation with decisionId/version
POLICY -> enforce ownership, hard limits, approvals, risk fallback
LEDGER -> submit one idempotent deterministic transaction when value changes
EVENT  -> update projection/case/monitoring asynchronously
VERIFY -> discover committed result and retain correlationId + txId evidence
```

Verification Checklist

- The accounting library exists and is version-controlled.
- The feature works with all public Internet/WAN connectivity disabled.
- Authorization, malformed input, duplicate retry, and failure behavior have explicit tests.
- If financial value changes, journal debits and credits balance exactly and historical entries remain immutable.
- If AI is involved, model/prompt/feature version and fallback behavior are recorded and deterministic policy remains final.
- A timeout/restart cannot create a duplicate financial result; request status can be rediscovered.
- Logs/evidence contain correlation references but not secrets or prohibited sensitive payment data.

Common Failure Modes

- External HTTP/AI call from chaincode.
- Non-deterministic timestamps/randomness.
- No duplicate-request guard.
- Balance checks split across non-atomic transactions.

Expert Review Questions

- What happens if this component succeeds but the next component fails?
- How is the authorizing identity/device/key revoked or recovered?
- Can the operation be safely retried after the client loses the response?
- What happens if AI is unavailable, slow, or wrong?
- Can an auditor reconstruct debit, credit, actor, risk decision, approval, and Fabric txId from retained evidence?
- Which compensating transaction resolves a downstream operational error without rewriting history?

Chapter 84 - Authorization Helpers

Enforce caller organization/role/attributes before changing financial state. Chaincode is the deterministic financial rules engine. It must validate identities, states, balances, limits, idempotency keys, asset codes, and balanced journal entries without network calls, AI inference, wall-clock assumptions, floating-point money, or mutable external dependencies.

Design rule: Read MSP ID and attributes

Implementation Objective

The required output is: Authorization library. It must be version-controlled, reviewable, testable with the WAN disabled, and linked to the architecture/source-of-truth model. For a payment platform, every implementation decision should make it possible to explain who authorized an action, which facts were trusted, which controls ran, what financial journal committed, and how the result is recovered after failure.

Key Engineering Decisions

- Read MSP ID and attributes
- Map allowed roles per transaction
- Deny unknown callers
- Do not rely only on API

Deep Implementation Guidance

Keep all writes required for one logical financial action in one transaction when atomicity is required. Use deterministic validation and stable error codes. Never call an external API, database, clock service, random generator, or AI model from consensus-critical chaincode.

Data and Control Questions

- What is the authoritative source for every input used by this component?
- Which user, device, service, MSP, or administrator identity is allowed to initiate or approve it?
- Does the operation move value, reserve value, create a side effect, or only provide advisory information?
- What idempotency/replay key proves that a retry is the same business intent?
- What evidence is retained off-chain and what stable reference/hash belongs on the ledger?

Chapter 84 Implementation - Authorization Helpers

Implementation Procedure

- 416. Parse and validate every argument deterministically.
- 417. Authorize caller MSP/attributes and validate entity lifecycle state.
- 418. Check idempotency before any business write.
- 419. Write balanced journal/current state atomically and emit compact events.
- 420. Return stable error codes and test concurrent/MVCC conflict behavior.

Implementation Example

```
# Authorization Helpers
INPUT  -> authenticate, validate schema, bind requestId/nonce
FACTS  -> read authoritative wallet/merchant/state data
AI     -> optional bounded recommendation with decisionId/version
POLICY -> enforce ownership, hard limits, approvals, risk fallback
LEDGER -> submit one idempotent deterministic transaction when value changes
EVENT  -> update projection/case/monitoring asynchronously
VERIFY -> discover committed result and retain correlationId + txId evidence
```

Verification Checklist

- The authorization library exists and is version-controlled.
- The feature works with all public Internet/WAN connectivity disabled.
- Authorization, malformed input, duplicate retry, and failure behavior have explicit tests.
- If financial value changes, journal debits and credits balance exactly and historical entries remain immutable.
- If AI is involved, model/prompt/feature version and fallback behavior are recorded and deterministic policy remains final.
- A timeout/restart cannot create a duplicate financial result; request status can be rediscovered.
- Logs/evidence contain correlation references but not secrets or prohibited sensitive payment data.

Common Failure Modes

- External HTTP/AI call from chaincode.
- Non-deterministic timestamps/randomness.
- No duplicate-request guard.
- Balance checks split across non-atomic transactions.

Expert Review Questions

- What happens if this component succeeds but the next component fails?
- How is the authorizing identity/device/key revoked or recovered?
- Can the operation be safely retried after the client loses the response?
- What happens if AI is unavailable, slow, or wrong?
- Can an auditor reconstruct debit, credit, actor, risk decision, approval, and Fabric txId from retained evidence?
- Which compensating transaction resolves a downstream operational error without rewriting history?

Chapter 85 - CreateWallet Transaction

Create a unique wallet with controlled owner/type/asset/status. Chaincode is the deterministic financial rules engine. It must validate identities, states, balances, limits, idempotency keys, asset codes, and balanced journal entries without network calls, AI inference, wall-clock assumptions, floating-point money, or mutable external dependencies.

Design rule: Reject duplicate wallet

Implementation Objective

The required output is: CreateWallet chaincode. It must be version-controlled, reviewable, testable with the WAN disabled, and linked to the architecture/source-of-truth model. For a payment platform, every implementation decision should make it possible to explain who authorized an action, which facts were trusted, which controls ran, what financial journal committed, and how the result is recovered after failure.

Key Engineering Decisions

- Reject duplicate wallet
- Validate allowed type/asset
- Record creator and txId
- Initialize balances/counters

Deep Implementation Guidance

Keep all writes required for one logical financial action in one transaction when atomicity is required. Use deterministic validation and stable error codes. Never call an external API, database, clock service, random generator, or AI model from consensus-critical chaincode.

Data and Control Questions

- What is the authoritative source for every input used by this component?
- Which user, device, service, MSP, or administrator identity is allowed to initiate or approve it?
- Does the operation move value, reserve value, create a side effect, or only provide advisory information?
- What idempotency/replay key proves that a retry is the same business intent?
- What evidence is retained off-chain and what stable reference/hash belongs on the ledger?

Chapter 85 Implementation - CreateWallet Transaction

Implementation Procedure

421. Parse and validate every argument deterministically.
422. Authorize caller MSP/attributes and validate entity lifecycle state.
423. Check idempotency before any business write.
424. Write balanced journal/current state atomically and emit compact events.
425. Return stable error codes and test concurrent/MVCC conflict behavior.

Implementation Example

```
# CreateWallet Transaction
INPUT  -> authenticate, validate schema, bind requestId/nonce
FACTS  -> read authoritative wallet/merchant/state data
AI     -> optional bounded recommendation with decisionId/version
POLICY -> enforce ownership, hard limits, approvals, risk fallback
LEDGER -> submit one idempotent deterministic transaction when value changes
EVENT  -> update projection/case/monitoring asynchronously
VERIFY -> discover committed result and retain correlationId + txId evidence
```

Verification Checklist

- The createwallet chaincode exists and is version-controlled.
- The feature works with all public Internet/WAN connectivity disabled.
- Authorization, malformed input, duplicate retry, and failure behavior have explicit tests.
- If financial value changes, journal debits and credits balance exactly and historical entries remain immutable.
- If AI is involved, model/prompt/feature version and fallback behavior are recorded and deterministic policy remains final.
- A timeout/restart cannot create a duplicate financial result; request status can be rediscovered.
- Logs/evidence contain correlation references but not secrets or prohibited sensitive payment data.

Common Failure Modes

- External HTTP/AI call from chaincode.
- Non-deterministic timestamps/randomness.
- No duplicate-request guard.
- Balance checks split across non-atomic transactions.

Expert Review Questions

- What happens if this component succeeds but the next component fails?
- How is the authorizing identity/device/key revoked or recovered?
- Can the operation be safely retried after the client loses the response?
- What happens if AI is unavailable, slow, or wrong?
- Can an auditor reconstruct debit, credit, actor, risk decision, approval, and Fabric txId from retained evidence?
- Which compensating transaction resolves a downstream operational error without rewriting history?

Chapter 86 - Activate Suspend Freeze and Unfreeze Wallet

Control wallet availability without deleting it. Chaincode is the deterministic financial rules engine. It must validate identities, states, balances, limits, idempotency keys, asset codes, and balanced journal entries without network calls, AI inference, wall-clock assumptions, floating-point money, or mutable external dependencies.

Design rule: Separate operational suspension from security freeze

Implementation Objective

The required output is: Wallet status transactions. It must be version-controlled, reviewable, testable with the WAN disabled, and linked to the architecture/source-of-truth model. For a payment platform, every implementation decision should make it possible to explain who authorized an action, which facts were trusted, which controls ran, what financial journal committed, and how the result is recovered after failure.

Key Engineering Decisions

- Separate operational suspension from security freeze
- Only authorized roles may change state
- Record reason and case/approval reference

Deep Implementation Guidance

Keep all writes required for one logical financial action in one transaction when atomicity is required. Use deterministic validation and stable error codes. Never call an external API, database, clock service, random generator, or AI model from consensus-critical chaincode.

Data and Control Questions

- What is the authoritative source for every input used by this component?
- Which user, device, service, MSP, or administrator identity is allowed to initiate or approve it?
- Does the operation move value, reserve value, create a side effect, or only provide advisory information?
- What idempotency/replay key proves that a retry is the same business intent?
- What evidence is retained off-chain and what stable reference/hash belongs on the ledger?

Chapter 86 Implementation - Activate Suspend Freeze and Unfreeze Wallet

Implementation Procedure

- 426. Parse and validate every argument deterministically.
- 427. Authorize caller MSP/attributes and validate entity lifecycle state.
- 428. Check idempotency before any business write.
- 429. Write balanced journal/current state atomically and emit compact events.
- 430. Return stable error codes and test concurrent/MVCC conflict behavior.

Implementation Example

```
# Activate Suspend Freeze and Unfreeze Wallet
INPUT  -> authenticate, validate schema, bind requestId/nonce
FACTS  -> read authoritative wallet/merchant/state data
AI     -> optional bounded recommendation with decisionId/version
POLICY -> enforce ownership, hard limits, approvals, risk fallback
LEDGER -> submit one idempotent deterministic transaction when value changes
EVENT  -> update projection/case/monitoring asynchronously
VERIFY -> discover committed result and retain correlationId + txId evidence
```

Verification Checklist

- The wallet status transactions exists and is version-controlled.
- The feature works with all public Internet/WAN connectivity disabled.
- Authorization, malformed input, duplicate retry, and failure behavior have explicit tests.
- If financial value changes, journal debits and credits balance exactly and historical entries remain immutable.
- If AI is involved, model/prompt/feature version and fallback behavior are recorded and deterministic policy remains final.
- A timeout/restart cannot create a duplicate financial result; request status can be rediscovered.
- Logs/evidence contain correlation references but not secrets or prohibited sensitive payment data.

Common Failure Modes

- External HTTP/AI call from chaincode.
- Non-deterministic timestamps/randomness.
- No duplicate-request guard.
- Balance checks split across non-atomic transactions.

Expert Review Questions

- What happens if this component succeeds but the next component fails?
- How is the authorizing identity/device/key revoked or recovered?
- Can the operation be safely retried after the client loses the response?
- What happens if AI is unavailable, slow, or wrong?
- Can an auditor reconstruct debit, credit, actor, risk decision, approval, and Fabric txId from retained evidence?
- Which compensating transaction resolves a downstream operational error without rewriting history?

Chapter 87 - IssueValue Transaction

Create stored value through treasury-controlled balanced accounting. Chaincode is the deterministic financial rules engine. It must validate identities, states, balances, limits, idempotency keys, asset codes, and balanced journal entries without network calls, AI inference, wall-clock assumptions, floating-point money, or mutable external dependencies.

Design rule: Check treasury authority and approval

Implementation Objective

The required output is: IssueValue chaincode. It must be version-controlled, reviewable, testable with the WAN disabled, and linked to the architecture/source-of-truth model. For a payment platform, every implementation decision should make it possible to explain who authorized an action, which facts were trusted, which controls ran, what financial journal committed, and how the result is recovered after failure.

Key Engineering Decisions

- Check treasury authority and approval
- Debit issuance source/credit target wallet
- Enforce hard issuance limit
- Idempotent request key

Deep Implementation Guidance

Keep all writes required for one logical financial action in one transaction when atomicity is required. Use deterministic validation and stable error codes. Never call an external API, database, clock service, random generator, or AI model from consensus-critical chaincode.

Data and Control Questions

- What is the authoritative source for every input used by this component?
- Which user, device, service, MSP, or administrator identity is allowed to initiate or approve it?
- Does the operation move value, reserve value, create a side effect, or only provide advisory information?
- What idempotency/replay key proves that a retry is the same business intent?
- What evidence is retained off-chain and what stable reference/hash belongs on the ledger?

Chapter 87 Implementation - IssueValue Transaction

Implementation Procedure

431. Parse and validate every argument deterministically.
432. Authorize caller MSP/attributes and validate entity lifecycle state.
433. Check idempotency before any business write.
434. Write balanced journal/current state atomically and emit compact events.
435. Return stable error codes and test concurrent/MVCC conflict behavior.

Implementation Example

```
# IssueValue Transaction
INPUT  -> authenticate, validate schema, bind requestId/nonce
FACTS  -> read authoritative wallet/merchant/state data
AI     -> optional bounded recommendation with decisionId/version
POLICY -> enforce ownership, hard limits, approvals, risk fallback
LEDGER -> submit one idempotent deterministic transaction when value changes
EVENT  -> update projection/case/monitoring asynchronously
VERIFY -> discover committed result and retain correlationId + txId evidence
```

Verification Checklist

- The issuevalue chaincode exists and is version-controlled.
- The feature works with all public Internet/WAN connectivity disabled.
- Authorization, malformed input, duplicate retry, and failure behavior have explicit tests.
- If financial value changes, journal debits and credits balance exactly and historical entries remain immutable.
- If AI is involved, model/prompt/feature version and fallback behavior are recorded and deterministic policy remains final.
- A timeout/restart cannot create a duplicate financial result; request status can be rediscovered.
- Logs/evidence contain correlation references but not secrets or prohibited sensitive payment data.

Common Failure Modes

- External HTTP/AI call from chaincode.
- Non-deterministic timestamps/randomness.
- No duplicate-request guard.
- Balance checks split across non-atomic transactions.

Expert Review Questions

- What happens if this component succeeds but the next component fails?
- How is the authorizing identity/device/key revoked or recovered?
- Can the operation be safely retried after the client loses the response?
- What happens if AI is unavailable, slow, or wrong?
- Can an auditor reconstruct debit, credit, actor, risk decision, approval, and Fabric txId from retained evidence?
- Which compensating transaction resolves a downstream operational error without rewriting history?

Chapter 88 - RedeemValue Transaction

Remove stored value through controlled redemption. Chaincode is the deterministic financial rules engine. It must validate identities, states, balances, limits, idempotency keys, asset codes, and balanced journal entries without network calls, AI inference, wall-clock assumptions, floating-point money, or mutable external dependencies.

Design rule: Check balance/status/limits

Implementation Objective

The required output is: RedeemValue chaincode. It must be version-controlled, reviewable, testable with the WAN disabled, and linked to the architecture/source-of-truth model. For a payment platform, every implementation decision should make it possible to explain who authorized an action, which facts were trusted, which controls ran, what financial journal committed, and how the result is recovered after failure.

Key Engineering Decisions

- Check balance/status/limits
- Debit wallet/credit redemption account
- Link external payout reference if any
- Idempotent

Deep Implementation Guidance

Keep all writes required for one logical financial action in one transaction when atomicity is required. Use deterministic validation and stable error codes. Never call an external API, database, clock service, random generator, or AI model from consensus-critical chaincode.

Data and Control Questions

- What is the authoritative source for every input used by this component?
- Which user, device, service, MSP, or administrator identity is allowed to initiate or approve it?
- Does the operation move value, reserve value, create a side effect, or only provide advisory information?
- What idempotency/replay key proves that a retry is the same business intent?
- What evidence is retained off-chain and what stable reference/hash belongs on the ledger?

Chapter 88 Implementation - RedeemValue Transaction

Implementation Procedure

436. Parse and validate every argument deterministically.
437. Authorize caller MSP/attributes and validate entity lifecycle state.
438. Check idempotency before any business write.
439. Write balanced journal/current state atomically and emit compact events.
440. Return stable error codes and test concurrent/MVCC conflict behavior.

Implementation Example

```
# RedeemValue Transaction
INPUT  -> authenticate, validate schema, bind requestId/nonce
FACTS  -> read authoritative wallet/merchant/state data
AI     -> optional bounded recommendation with decisionId/version
POLICY -> enforce ownership, hard limits, approvals, risk fallback
LEDGER -> submit one idempotent deterministic transaction when value changes
EVENT  -> update projection/case/monitoring asynchronously
VERIFY -> discover committed result and retain correlationId + txId evidence
```

Verification Checklist

- The redeemvalue chaincode exists and is version-controlled.
- The feature works with all public Internet/WAN connectivity disabled.
- Authorization, malformed input, duplicate retry, and failure behavior have explicit tests.
- If financial value changes, journal debits and credits balance exactly and historical entries remain immutable.
- If AI is involved, model/prompt/feature version and fallback behavior are recorded and deterministic policy remains final.
- A timeout/restart cannot create a duplicate financial result; request status can be rediscovered.
- Logs/evidence contain correlation references but not secrets or prohibited sensitive payment data.

Common Failure Modes

- External HTTP/AI call from chaincode.
- Non-deterministic timestamps/randomness.
- No duplicate-request guard.
- Balance checks split across non-atomic transactions.

Expert Review Questions

- What happens if this component succeeds but the next component fails?
- How is the authorizing identity/device/key revoked or recovered?
- Can the operation be safely retried after the client loses the response?
- What happens if AI is unavailable, slow, or wrong?
- Can an auditor reconstruct debit, credit, actor, risk decision, approval, and Fabric txId from retained evidence?
- Which compensating transaction resolves a downstream operational error without rewriting history?

Chapter 89 - Transfer Transaction

Move funds wallet-to-wallet atomically. Chaincode is the deterministic financial rules engine. It must validate identities, states, balances, limits, idempotency keys, asset codes, and balanced journal entries without network calls, AI inference, wall-clock assumptions, floating-point money, or mutable external dependencies.

Design rule: Validate sender recipient asset status

Implementation Objective

The required output is: Transfer chaincode. It must be version-controlled, reviewable, testable with the WAN disabled, and linked to the architecture/source-of-truth model. For a payment platform, every implementation decision should make it possible to explain who authorized an action, which facts were trusted, which controls ran, what financial journal committed, and how the result is recovered after failure.

Key Engineering Decisions

- Validate sender recipient asset status
- Check available balance and limits
- Debit sender credit recipient
- Write journal and update balances atomically

Deep Implementation Guidance

Keep all writes required for one logical financial action in one transaction when atomicity is required. Use deterministic validation and stable error codes. Never call an external API, database, clock service, random generator, or AI model from consensus-critical chaincode.

Data and Control Questions

- What is the authoritative source for every input used by this component?
- Which user, device, service, MSP, or administrator identity is allowed to initiate or approve it?
- Does the operation move value, reserve value, create a side effect, or only provide advisory information?
- What idempotency/replay key proves that a retry is the same business intent?
- What evidence is retained off-chain and what stable reference/hash belongs on the ledger?

Chapter 89 Implementation - Transfer Transaction

Implementation Procedure

441. Parse and validate every argument deterministically.
442. Authorize caller MSP/attributes and validate entity lifecycle state.
443. Check idempotency before any business write.
444. Write balanced journal/current state atomically and emit compact events.
445. Return stable error codes and test concurrent/MVCC conflict behavior.

Implementation Example

```
@Transaction()
public async Transfer(ctx: Context, requestId: string, inputJson: string): Promise<string> {
  const prior = await ctx.stub.getState('REQUEST~${requestId}');
  if (prior.length) return prior.toString('utf8');

  const input = JSON.parse(inputJson);
  assertPositiveInt(input.amountMinor);
  const from = await this.mustGetWallet(ctx, input.fromWalletId);
  const to = await this.mustGetWallet(ctx, input.toWalletId);
  this.requireDebitAuthority(ctx, from);
  if (from.asset !== to.asset || from.asset !== input.asset) throw new Error('ASSET_MISMATCH');
  if (from.status !== 'ACTIVE' || to.status !== 'ACTIVE') throw new Error('WALLET_NOT_ACTIVE');
  if (from.availableMinor < input.amountMinor) throw new Error('INSUFFICIENT_FUNDS');

  from.availableMinor -= input.amountMinor;
  to.availableMinor += input.amountMinor;
  const journal = balancedJournal(requestId, input.asset, [
    debit(from.walletId, input.amountMinor),
    credit(to.walletId, input.amountMinor),
  ]);
  await this.putWallet(ctx, from); await this.putWallet(ctx, to);
  await ctx.stub.putState('JOURNAL~${journal.journalId}', Buffer.from(JSON.stringify(journal)));
  const result = JSON.stringify({requestId, journalId: journal.journalId});
  await ctx.stub.putState('REQUEST~${requestId}', Buffer.from(result));
  ctx.stub.setEvent('TransferCommitted', Buffer.from(result));
  return result;
}
```

Verification Checklist

- The transfer chaincode exists and is version-controlled.
- The feature works with all public Internet/WAN connectivity disabled.
- Authorization, malformed input, duplicate retry, and failure behavior have explicit tests.
- If financial value changes, journal debits and credits balance exactly and historical entries remain immutable.
- If AI is involved, model/prompt/feature version and fallback behavior are recorded and deterministic policy remains final.
- A timeout/restart cannot create a duplicate financial result; request status can be rediscovered.
- Logs/evidence contain correlation references but not secrets or prohibited sensitive payment data.

Common Failure Modes

- External HTTP/AI call from chaincode.
- Non-deterministic timestamps/randomness.
- No duplicate-request guard.

- Balance checks split across non-atomic transactions.

Expert Review Questions

- What happens if this component succeeds but the next component fails?
- How is the authorizing identity/device/key revoked or recovered?
- Can the operation be safely retried after the client loses the response?
- What happens if AI is unavailable, slow, or wrong?
- Can an auditor reconstruct debit, credit, actor, risk decision, approval, and Fabric txId from retained evidence?
- Which compensating transaction resolves a downstream operational error without rewriting history?

Chapter 90 - MerchantPayment Transaction

Transfer customer value to merchant with order/payment references. Chaincode is the deterministic financial rules engine. It must validate identities, states, balances, limits, idempotency keys, asset codes, and balanced journal entries without network calls, AI inference, wall-clock assumptions, floating-point money, or mutable external dependencies.

Design rule: Validate merchant ownership/store

Implementation Objective

The required output is: Payment chaincode. It must be version-controlled, reviewable, testable with the WAN disabled, and linked to the architecture/source-of-truth model. For a payment platform, every implementation decision should make it possible to explain who authorized an action, which facts were trusted, which controls ran, what financial journal committed, and how the result is recovered after failure.

Key Engineering Decisions

- Validate merchant ownership/store
- Calculate deterministic fee
- Write customer merchant fee lines
- Record riskDecisionId and authDigest reference

Deep Implementation Guidance

Keep all writes required for one logical financial action in one transaction when atomicity is required. Use deterministic validation and stable error codes. Never call an external API, database, clock service, random generator, or AI model from consensus-critical chaincode.

Data and Control Questions

- What is the authoritative source for every input used by this component?
- Which user, device, service, MSP, or administrator identity is allowed to initiate or approve it?
- Does the operation move value, reserve value, create a side effect, or only provide advisory information?
- What idempotency/replay key proves that a retry is the same business intent?
- What evidence is retained off-chain and what stable reference/hash belongs on the ledger?

Chapter 90 Implementation - MerchantPayment Transaction

Implementation Procedure

- 446. Parse and validate every argument deterministically.
- 447. Authorize caller MSP/attributes and validate entity lifecycle state.
- 448. Check idempotency before any business write.
- 449. Write balanced journal/current state atomically and emit compact events.
- 450. Return stable error codes and test concurrent/MVCC conflict behavior.

Implementation Example

```
# MerchantPayment Transaction
INPUT  -> authenticate, validate schema, bind requestId/nonce
FACTS  -> read authoritative wallet/merchant/state data
AI     -> optional bounded recommendation with decisionId/version
POLICY -> enforce ownership, hard limits, approvals, risk fallback
LEDGER -> submit one idempotent deterministic transaction when value changes
EVENT  -> update projection/case/monitoring asynchronously
VERIFY -> discover committed result and retain correlationId + txId evidence
```

Verification Checklist

- The payment chaincode exists and is version-controlled.
- The feature works with all public Internet/WAN connectivity disabled.
- Authorization, malformed input, duplicate retry, and failure behavior have explicit tests.
- If financial value changes, journal debits and credits balance exactly and historical entries remain immutable.
- If AI is involved, model/prompt/feature version and fallback behavior are recorded and deterministic policy remains final.
- A timeout/restart cannot create a duplicate financial result; request status can be rediscovered.
- Logs/evidence contain correlation references but not secrets or prohibited sensitive payment data.

Common Failure Modes

- External HTTP/AI call from chaincode.
- Non-deterministic timestamps/randomness.
- No duplicate-request guard.
- Balance checks split across non-atomic transactions.

Expert Review Questions

- What happens if this component succeeds but the next component fails?
- How is the authorizing identity/device/key revoked or recovered?
- Can the operation be safely retried after the client loses the response?
- What happens if AI is unavailable, slow, or wrong?
- Can an auditor reconstruct debit, credit, actor, risk decision, approval, and Fabric txId from retained evidence?
- Which compensating transaction resolves a downstream operational error without rewriting history?

Chapter 91 - CreateHold Transaction

Reserve spendable balance for later capture. Chaincode is the deterministic financial rules engine. It must validate identities, states, balances, limits, idempotency keys, asset codes, and balanced journal entries without network calls, AI inference, wall-clock assumptions, floating-point money, or mutable external dependencies.

Design rule: Move amount from available to reserved representation

Implementation Objective

The required output is: Hold chaincode. It must be version-controlled, reviewable, testable with the WAN disabled, and linked to the architecture/source-of-truth model. For a payment platform, every implementation decision should make it possible to explain who authorized an action, which facts were trusted, which controls ran, what financial journal committed, and how the result is recovered after failure.

Key Engineering Decisions

- Move amount from available to reserved representation
- Create unique hold state
- Validate expiry/reference
- Idempotent

Deep Implementation Guidance

Keep all writes required for one logical financial action in one transaction when atomicity is required. Use deterministic validation and stable error codes. Never call an external API, database, clock service, random generator, or AI model from consensus-critical chaincode.

Data and Control Questions

- What is the authoritative source for every input used by this component?
- Which user, device, service, MSP, or administrator identity is allowed to initiate or approve it?
- Does the operation move value, reserve value, create a side effect, or only provide advisory information?
- What idempotency/replay key proves that a retry is the same business intent?
- What evidence is retained off-chain and what stable reference/hash belongs on the ledger?

Chapter 91 Implementation - CreateHold Transaction

Implementation Procedure

451. Parse and validate every argument deterministically.
452. Authorize caller MSP/attributes and validate entity lifecycle state.
453. Check idempotency before any business write.
454. Write balanced journal/current state atomically and emit compact events.
455. Return stable error codes and test concurrent/MVCC conflict behavior.

Implementation Example

```
# CreateHold Transaction
INPUT  -> authenticate, validate schema, bind requestId/nonce
FACTS  -> read authoritative wallet/merchant/state data
AI     -> optional bounded recommendation with decisionId/version
POLICY -> enforce ownership, hard limits, approvals, risk fallback
LEDGER -> submit one idempotent deterministic transaction when value changes
EVENT  -> update projection/case/monitoring asynchronously
VERIFY -> discover committed result and retain correlationId + txId evidence
```

Verification Checklist

- The hold chaincode exists and is version-controlled.
- The feature works with all public Internet/WAN connectivity disabled.
- Authorization, malformed input, duplicate retry, and failure behavior have explicit tests.
- If financial value changes, journal debits and credits balance exactly and historical entries remain immutable.
- If AI is involved, model/prompt/feature version and fallback behavior are recorded and deterministic policy remains final.
- A timeout/restart cannot create a duplicate financial result; request status can be rediscovered.
- Logs/evidence contain correlation references but not secrets or prohibited sensitive payment data.

Common Failure Modes

- External HTTP/AI call from chaincode.
- Non-deterministic timestamps/randomness.
- No duplicate-request guard.
- Balance checks split across non-atomic transactions.

Expert Review Questions

- What happens if this component succeeds but the next component fails?
- How is the authorizing identity/device/key revoked or recovered?
- Can the operation be safely retried after the client loses the response?
- What happens if AI is unavailable, slow, or wrong?
- Can an auditor reconstruct debit, credit, actor, risk decision, approval, and Fabric txId from retained evidence?
- Which compensating transaction resolves a downstream operational error without rewriting history?

Chapter 92 - CaptureHold Transaction

Settle a hold to merchant and fees. Chaincode is the deterministic financial rules engine. It must validate identities, states, balances, limits, idempotency keys, asset codes, and balanced journal entries without network calls, AI inference, wall-clock assumptions, floating-point money, or mutable external dependencies.

Design rule: Check active hold and remaining amount

Implementation Objective

The required output is: CaptureHold chaincode. It must be version-controlled, reviewable, testable with the WAN disabled, and linked to the architecture/source-of-truth model. For a payment platform, every implementation decision should make it possible to explain who authorized an action, which facts were trusted, which controls ran, what financial journal committed, and how the result is recovered after failure.

Key Engineering Decisions

- Check active hold and remaining amount
- Allow full/partial capture according to policy
- Update reserved/balances/journal atomically

Deep Implementation Guidance

Keep all writes required for one logical financial action in one transaction when atomicity is required. Use deterministic validation and stable error codes. Never call an external API, database, clock service, random generator, or AI model from consensus-critical chaincode.

Data and Control Questions

- What is the authoritative source for every input used by this component?
- Which user, device, service, MSP, or administrator identity is allowed to initiate or approve it?
- Does the operation move value, reserve value, create a side effect, or only provide advisory information?
- What idempotency/replay key proves that a retry is the same business intent?
- What evidence is retained off-chain and what stable reference/hash belongs on the ledger?

Chapter 92 Implementation - CaptureHold Transaction

Implementation Procedure

- 456. Parse and validate every argument deterministically.
- 457. Authorize caller MSP/attributes and validate entity lifecycle state.
- 458. Check idempotency before any business write.
- 459. Write balanced journal/current state atomically and emit compact events.
- 460. Return stable error codes and test concurrent/MVCC conflict behavior.

Implementation Example

```
# CaptureHold Transaction
INPUT  -> authenticate, validate schema, bind requestId/nonce
FACTS  -> read authoritative wallet/merchant/state data
AI     -> optional bounded recommendation with decisionId/version
POLICY -> enforce ownership, hard limits, approvals, risk fallback
LEDGER -> submit one idempotent deterministic transaction when value changes
EVENT  -> update projection/case/monitoring asynchronously
VERIFY -> discover committed result and retain correlationId + txId evidence
```

Verification Checklist

- The capturehold chaincode exists and is version-controlled.
- The feature works with all public Internet/WAN connectivity disabled.
- Authorization, malformed input, duplicate retry, and failure behavior have explicit tests.
- If financial value changes, journal debits and credits balance exactly and historical entries remain immutable.
- If AI is involved, model/prompt/feature version and fallback behavior are recorded and deterministic policy remains final.
- A timeout/restart cannot create a duplicate financial result; request status can be rediscovered.
- Logs/evidence contain correlation references but not secrets or prohibited sensitive payment data.

Common Failure Modes

- External HTTP/AI call from chaincode.
- Non-deterministic timestamps/randomness.
- No duplicate-request guard.
- Balance checks split across non-atomic transactions.

Expert Review Questions

- What happens if this component succeeds but the next component fails?
- How is the authorizing identity/device/key revoked or recovered?
- Can the operation be safely retried after the client loses the response?
- What happens if AI is unavailable, slow, or wrong?
- Can an auditor reconstruct debit, credit, actor, risk decision, approval, and Fabric txId from retained evidence?
- Which compensating transaction resolves a downstream operational error without rewriting history?

Chapter 93 - ReleaseHold Transaction

Return uncaptured reserved value. Chaincode is the deterministic financial rules engine. It must validate identities, states, balances, limits, idempotency keys, asset codes, and balanced journal entries without network calls, AI inference, wall-clock assumptions, floating-point money, or mutable external dependencies.

Design rule: Only active remaining amount can release

Implementation Objective

The required output is: ReleaseHold chaincode. It must be version-controlled, reviewable, testable with the WAN disabled, and linked to the architecture/source-of-truth model. For a payment platform, every implementation decision should make it possible to explain who authorized an action, which facts were trusted, which controls ran, what financial journal committed, and how the result is recovered after failure.

Key Engineering Decisions

- Only active remaining amount can release
- Record reason/system expiry reference
- Prevent double release

Deep Implementation Guidance

Keep all writes required for one logical financial action in one transaction when atomicity is required. Use deterministic validation and stable error codes. Never call an external API, database, clock service, random generator, or AI model from consensus-critical chaincode.

Data and Control Questions

- What is the authoritative source for every input used by this component?
- Which user, device, service, MSP, or administrator identity is allowed to initiate or approve it?
- Does the operation move value, reserve value, create a side effect, or only provide advisory information?
- What idempotency/replay key proves that a retry is the same business intent?
- What evidence is retained off-chain and what stable reference/hash belongs on the ledger?

Chapter 93 Implementation - ReleaseHold Transaction

Implementation Procedure

461. Parse and validate every argument deterministically.
462. Authorize caller MSP/attributes and validate entity lifecycle state.
463. Check idempotency before any business write.
464. Write balanced journal/current state atomically and emit compact events.
465. Return stable error codes and test concurrent/MVCC conflict behavior.

Implementation Example

```
# ReleaseHold Transaction
INPUT  -> authenticate, validate schema, bind requestId/nonce
FACTS  -> read authoritative wallet/merchant/state data
AI     -> optional bounded recommendation with decisionId/version
POLICY -> enforce ownership, hard limits, approvals, risk fallback
LEDGER -> submit one idempotent deterministic transaction when value changes
EVENT  -> update projection/case/monitoring asynchronously
VERIFY -> discover committed result and retain correlationId + txId evidence
```

Verification Checklist

- The releasehold chaincode exists and is version-controlled.
- The feature works with all public Internet/WAN connectivity disabled.
- Authorization, malformed input, duplicate retry, and failure behavior have explicit tests.
- If financial value changes, journal debits and credits balance exactly and historical entries remain immutable.
- If AI is involved, model/prompt/feature version and fallback behavior are recorded and deterministic policy remains final.
- A timeout/restart cannot create a duplicate financial result; request status can be rediscovered.
- Logs/evidence contain correlation references but not secrets or prohibited sensitive payment data.

Common Failure Modes

- External HTTP/AI call from chaincode.
- Non-deterministic timestamps/randomness.
- No duplicate-request guard.
- Balance checks split across non-atomic transactions.

Expert Review Questions

- What happens if this component succeeds but the next component fails?
- How is the authorizing identity/device/key revoked or recovered?
- Can the operation be safely retried after the client loses the response?
- What happens if AI is unavailable, slow, or wrong?
- Can an auditor reconstruct debit, credit, actor, risk decision, approval, and Fabric txId from retained evidence?
- Which compensating transaction resolves a downstream operational error without rewriting history?

Chapter 94 - Refund Transaction

Return value against a prior merchant payment. Chaincode is the deterministic financial rules engine. It must validate identities, states, balances, limits, idempotency keys, asset codes, and balanced journal entries without network calls, AI inference, wall-clock assumptions, floating-point money, or mutable external dependencies.

Design rule: Validate original payment/refundable remainder

Implementation Objective

The required output is: Refund chaincode. It must be version-controlled, reviewable, testable with the WAN disabled, and linked to the architecture/source-of-truth model. For a payment platform, every implementation decision should make it possible to explain who authorized an action, which facts were trusted, which controls ran, what financial journal committed, and how the result is recovered after failure.

Key Engineering Decisions

- Validate original payment/refundable remainder
- Debit merchant/refund source and credit customer
- Record approval/reference
- Support partial refunds safely

Deep Implementation Guidance

Keep all writes required for one logical financial action in one transaction when atomicity is required. Use deterministic validation and stable error codes. Never call an external API, database, clock service, random generator, or AI model from consensus-critical chaincode.

Data and Control Questions

- What is the authoritative source for every input used by this component?
- Which user, device, service, MSP, or administrator identity is allowed to initiate or approve it?
- Does the operation move value, reserve value, create a side effect, or only provide advisory information?
- What idempotency/replay key proves that a retry is the same business intent?
- What evidence is retained off-chain and what stable reference/hash belongs on the ledger?

Chapter 94 Implementation - Refund Transaction

Implementation Procedure

- 466. Parse and validate every argument deterministically.
- 467. Authorize caller MSP/attributes and validate entity lifecycle state.
- 468. Check idempotency before any business write.
- 469. Write balanced journal/current state atomically and emit compact events.
- 470. Return stable error codes and test concurrent/MVCC conflict behavior.

Implementation Example

```
# Refund Transaction
INPUT  -> authenticate, validate schema, bind requestId/nonce
FACTS  -> read authoritative wallet/merchant/state data
AI     -> optional bounded recommendation with decisionId/version
POLICY -> enforce ownership, hard limits, approvals, risk fallback
LEDGER -> submit one idempotent deterministic transaction when value changes
EVENT  -> update projection/case/monitoring asynchronously
VERIFY -> discover committed result and retain correlationId + txId evidence
```

Verification Checklist

- The refund chaincode exists and is version-controlled.
- The feature works with all public Internet/WAN connectivity disabled.
- Authorization, malformed input, duplicate retry, and failure behavior have explicit tests.
- If financial value changes, journal debits and credits balance exactly and historical entries remain immutable.
- If AI is involved, model/prompt/feature version and fallback behavior are recorded and deterministic policy remains final.
- A timeout/restart cannot create a duplicate financial result; request status can be rediscovered.
- Logs/evidence contain correlation references but not secrets or prohibited sensitive payment data.

Common Failure Modes

- External HTTP/AI call from chaincode.
- Non-deterministic timestamps/randomness.
- No duplicate-request guard.
- Balance checks split across non-atomic transactions.

Expert Review Questions

- What happens if this component succeeds but the next component fails?
- How is the authorizing identity/device/key revoked or recovered?
- Can the operation be safely retried after the client loses the response?
- What happens if AI is unavailable, slow, or wrong?
- Can an auditor reconstruct debit, credit, actor, risk decision, approval, and Fabric txId from retained evidence?
- Which compensating transaction resolves a downstream operational error without rewriting history?

Chapter 95 - Reversal and Adjustment Transactions

Correct operational errors through new compensating journals. Chaincode is the deterministic financial rules engine. It must validate identities, states, balances, limits, idempotency keys, asset codes, and balanced journal entries without network calls, AI inference, wall-clock assumptions, floating-point money, or mutable external dependencies.

Design rule: Reference original journal

Implementation Objective

The required output is: Adjustment chaincode. It must be version-controlled, reviewable, testable with the WAN disabled, and linked to the architecture/source-of-truth model. For a payment platform, every implementation decision should make it possible to explain who authorized an action, which facts were trusted, which controls ran, what financial journal committed, and how the result is recovered after failure.

Key Engineering Decisions

- Reference original journal
- Require elevated role/approval
- Never mutate original lines
- Explain reason code

Deep Implementation Guidance

Keep all writes required for one logical financial action in one transaction when atomicity is required. Use deterministic validation and stable error codes. Never call an external API, database, clock service, random generator, or AI model from consensus-critical chaincode.

Data and Control Questions

- What is the authoritative source for every input used by this component?
- Which user, device, service, MSP, or administrator identity is allowed to initiate or approve it?
- Does the operation move value, reserve value, create a side effect, or only provide advisory information?
- What idempotency/replay key proves that a retry is the same business intent?
- What evidence is retained off-chain and what stable reference/hash belongs on the ledger?

Chapter 95 Implementation - Reversal and Adjustment Transactions

Implementation Procedure

471. Parse and validate every argument deterministically.
472. Authorize caller MSP/attributes and validate entity lifecycle state.
473. Check idempotency before any business write.
474. Write balanced journal/current state atomically and emit compact events.
475. Return stable error codes and test concurrent/MVCC conflict behavior.

Implementation Example

```
# Reversal and Adjustment Transactions
INPUT  -> authenticate, validate schema, bind requestId/nonce
FACTS  -> read authoritative wallet/merchant/state data
AI     -> optional bounded recommendation with decisionId/version
POLICY -> enforce ownership, hard limits, approvals, risk fallback
LEDGER -> submit one idempotent deterministic transaction when value changes
EVENT  -> update projection/case/monitoring asynchronously
VERIFY -> discover committed result and retain correlationId + txId evidence
```

Verification Checklist

- The adjustment chaincode exists and is version-controlled.
- The feature works with all public Internet/WAN connectivity disabled.
- Authorization, malformed input, duplicate retry, and failure behavior have explicit tests.
- If financial value changes, journal debits and credits balance exactly and historical entries remain immutable.
- If AI is involved, model/prompt/feature version and fallback behavior are recorded and deterministic policy remains final.
- A timeout/restart cannot create a duplicate financial result; request status can be rediscovered.
- Logs/evidence contain correlation references but not secrets or prohibited sensitive payment data.

Common Failure Modes

- External HTTP/AI call from chaincode.
- Non-deterministic timestamps/randomness.
- No duplicate-request guard.
- Balance checks split across non-atomic transactions.

Expert Review Questions

- What happens if this component succeeds but the next component fails?
- How is the authorizing identity/device/key revoked or recovered?
- Can the operation be safely retried after the client loses the response?
- What happens if AI is unavailable, slow, or wrong?
- Can an auditor reconstruct debit, credit, actor, risk decision, approval, and Fabric txId from retained evidence?
- Which compensating transaction resolves a downstream operational error without rewriting history?

Chapter 96 - Fee Calculation

Calculate fixed/percentage fees deterministically without floating point. Chaincode is the deterministic financial rules engine. It must validate identities, states, balances, limits, idempotency keys, asset codes, and balanced journal entries without network calls, AI inference, wall-clock assumptions, floating-point money, or mutable external dependencies.

Design rule: Use integer basis points/minor units

Implementation Objective

The required output is: Fee helper. It must be version-controlled, reviewable, testable with the WAN disabled, and linked to the architecture/source-of-truth model. For a payment platform, every implementation decision should make it possible to explain who authorized an action, which facts were trusted, which controls ran, what financial journal committed, and how the result is recovered after failure.

Key Engineering Decisions

- Use integer basis points/minor units
- Version fee policy
- Define rounding rule
- Persist calculated amount and policy version

Deep Implementation Guidance

Keep all writes required for one logical financial action in one transaction when atomicity is required. Use deterministic validation and stable error codes. Never call an external API, database, clock service, random generator, or AI model from consensus-critical chaincode.

Data and Control Questions

- What is the authoritative source for every input used by this component?
- Which user, device, service, MSP, or administrator identity is allowed to initiate or approve it?
- Does the operation move value, reserve value, create a side effect, or only provide advisory information?
- What idempotency/replay key proves that a retry is the same business intent?
- What evidence is retained off-chain and what stable reference/hash belongs on the ledger?

Chapter 96 Implementation - Fee Calculation

Implementation Procedure

- 476. Parse and validate every argument deterministically.
- 477. Authorize caller MSP/attributes and validate entity lifecycle state.
- 478. Check idempotency before any business write.
- 479. Write balanced journal/current state atomically and emit compact events.
- 480. Return stable error codes and test concurrent/MVCC conflict behavior.

Implementation Example

```
# Fee Calculation
INPUT  -> authenticate, validate schema, bind requestId/nonce
FACTS  -> read authoritative wallet/merchant/state data
AI     -> optional bounded recommendation with decisionId/version
POLICY -> enforce ownership, hard limits, approvals, risk fallback
LEDGER -> submit one idempotent deterministic transaction when value changes
EVENT  -> update projection/case/monitoring asynchronously
VERIFY -> discover committed result and retain correlationId + txId evidence
```

Verification Checklist

- The fee helper exists and is version-controlled.
- The feature works with all public Internet/WAN connectivity disabled.
- Authorization, malformed input, duplicate retry, and failure behavior have explicit tests.
- If financial value changes, journal debits and credits balance exactly and historical entries remain immutable.
- If AI is involved, model/prompt/feature version and fallback behavior are recorded and deterministic policy remains final.
- A timeout/restart cannot create a duplicate financial result; request status can be rediscovered.
- Logs/evidence contain correlation references but not secrets or prohibited sensitive payment data.

Common Failure Modes

- External HTTP/AI call from chaincode.
- Non-deterministic timestamps/randomness.
- No duplicate-request guard.
- Balance checks split across non-atomic transactions.

Expert Review Questions

- What happens if this component succeeds but the next component fails?
- How is the authorizing identity/device/key revoked or recovered?
- Can the operation be safely retried after the client loses the response?
- What happens if AI is unavailable, slow, or wrong?
- Can an auditor reconstruct debit, credit, actor, risk decision, approval, and Fabric txId from retained evidence?
- Which compensating transaction resolves a downstream operational error without rewriting history?

Chapter 97 - Hard Limit Enforcement

Enforce transaction and wallet limits independent of AI. Chaincode is the deterministic financial rules engine. It must validate identities, states, balances, limits, idempotency keys, asset codes, and balanced journal entries without network calls, AI inference, wall-clock assumptions, floating-point money, or mutable external dependencies.

Design rule: Use deterministic configured limits

Implementation Objective

The required output is: Limit helper. It must be version-controlled, reviewable, testable with the WAN disabled, and linked to the architecture/source-of-truth model. For a payment platform, every implementation decision should make it possible to explain who authorized an action, which facts were trusted, which controls ran, what financial journal committed, and how the result is recovered after failure.

Key Engineering Decisions

- Use deterministic configured limits
- Read relevant counters/state
- Reject before write
- AI recommendation cannot override hard maximum

Deep Implementation Guidance

Keep all writes required for one logical financial action in one transaction when atomicity is required. Use deterministic validation and stable error codes. Never call an external API, database, clock service, random generator, or AI model from consensus-critical chaincode.

Data and Control Questions

- What is the authoritative source for every input used by this component?
- Which user, device, service, MSP, or administrator identity is allowed to initiate or approve it?
- Does the operation move value, reserve value, create a side effect, or only provide advisory information?
- What idempotency/replay key proves that a retry is the same business intent?
- What evidence is retained off-chain and what stable reference/hash belongs on the ledger?

Chapter 97 Implementation - Hard Limit Enforcement

Implementation Procedure

481. Parse and validate every argument deterministically.
482. Authorize caller MSP/attributes and validate entity lifecycle state.
483. Check idempotency before any business write.
484. Write balanced journal/current state atomically and emit compact events.
485. Return stable error codes and test concurrent/MVCC conflict behavior.

Implementation Example

```
# Hard Limit Enforcement
INPUT  -> authenticate, validate schema, bind requestId/nonce
FACTS  -> read authoritative wallet/merchant/state data
AI     -> optional bounded recommendation with decisionId/version
POLICY -> enforce ownership, hard limits, approvals, risk fallback
LEDGER -> submit one idempotent deterministic transaction when value changes
EVENT  -> update projection/case/monitoring asynchronously
VERIFY -> discover committed result and retain correlationId + txId evidence
```

Verification Checklist

- The limit helper exists and is version-controlled.
- The feature works with all public Internet/WAN connectivity disabled.
- Authorization, malformed input, duplicate retry, and failure behavior have explicit tests.
- If financial value changes, journal debits and credits balance exactly and historical entries remain immutable.
- If AI is involved, model/prompt/feature version and fallback behavior are recorded and deterministic policy remains final.
- A timeout/restart cannot create a duplicate financial result; request status can be rediscovered.
- Logs/evidence contain correlation references but not secrets or prohibited sensitive payment data.

Common Failure Modes

- External HTTP/AI call from chaincode.
- Non-deterministic timestamps/randomness.
- No duplicate-request guard.
- Balance checks split across non-atomic transactions.

Expert Review Questions

- What happens if this component succeeds but the next component fails?
- How is the authorizing identity/device/key revoked or recovered?
- Can the operation be safely retried after the client loses the response?
- What happens if AI is unavailable, slow, or wrong?
- Can an auditor reconstruct debit, credit, actor, risk decision, approval, and Fabric txId from retained evidence?
- Which compensating transaction resolves a downstream operational error without rewriting history?

Chapter 98 - Daily and Rolling Counters

Track velocity facts needed for hard controls without nondeterministic queries. Chaincode is the deterministic financial rules engine. It must validate identities, states, balances, limits, idempotency keys, asset codes, and balanced journal entries without network calls, AI inference, wall-clock assumptions, floating-point money, or mutable external dependencies.

Design rule: Use deterministic bucket keys

Implementation Objective

The required output is: Counter state design. It must be version-controlled, reviewable, testable with the WAN disabled, and linked to the architecture/source-of-truth model. For a payment platform, every implementation decision should make it possible to explain who authorized an action, which facts were trusted, which controls ran, what financial journal committed, and how the result is recovered after failure.

Key Engineering Decisions

- Use deterministic bucket keys
- Update in same transaction
- Define business date source supplied/validated by application if needed

Deep Implementation Guidance

Keep all writes required for one logical financial action in one transaction when atomicity is required. Use deterministic validation and stable error codes. Never call an external API, database, clock service, random generator, or AI model from consensus-critical chaincode.

Data and Control Questions

- What is the authoritative source for every input used by this component?
- Which user, device, service, MSP, or administrator identity is allowed to initiate or approve it?
- Does the operation move value, reserve value, create a side effect, or only provide advisory information?
- What idempotency/replay key proves that a retry is the same business intent?
- What evidence is retained off-chain and what stable reference/hash belongs on the ledger?

Chapter 98 Implementation - Daily and Rolling Counters

Implementation Procedure

- 486. Parse and validate every argument deterministically.
- 487. Authorize caller MSP/attributes and validate entity lifecycle state.
- 488. Check idempotency before any business write.
- 489. Write balanced journal/current state atomically and emit compact events.
- 490. Return stable error codes and test concurrent/MVCC conflict behavior.

Implementation Example

```
# Daily and Rolling Counters
INPUT  -> authenticate, validate schema, bind requestId/nonce
FACTS  -> read authoritative wallet/merchant/state data
AI     -> optional bounded recommendation with decisionId/version
POLICY -> enforce ownership, hard limits, approvals, risk fallback
LEDGER -> submit one idempotent deterministic transaction when value changes
EVENT  -> update projection/case/monitoring asynchronously
VERIFY -> discover committed result and retain correlationId + txId evidence
```

Verification Checklist

- The counter state design exists and is version-controlled.
- The feature works with all public Internet/WAN connectivity disabled.
- Authorization, malformed input, duplicate retry, and failure behavior have explicit tests.
- If financial value changes, journal debits and credits balance exactly and historical entries remain immutable.
- If AI is involved, model/prompt/feature version and fallback behavior are recorded and deterministic policy remains final.
- A timeout/restart cannot create a duplicate financial result; request status can be rediscovered.
- Logs/evidence contain correlation references but not secrets or prohibited sensitive payment data.

Common Failure Modes

- External HTTP/AI call from chaincode.
- Non-deterministic timestamps/randomness.
- No duplicate-request guard.
- Balance checks split across non-atomic transactions.

Expert Review Questions

- What happens if this component succeeds but the next component fails?
- How is the authorizing identity/device/key revoked or recovered?
- Can the operation be safely retried after the client loses the response?
- What happens if AI is unavailable, slow, or wrong?
- Can an auditor reconstruct debit, credit, actor, risk decision, approval, and Fabric txId from retained evidence?
- Which compensating transaction resolves a downstream operational error without rewriting history?

Chapter 99 - Ledger-Level Idempotency

Make duplicate financial requests return the original committed result. Chaincode is the deterministic financial rules engine. It must validate identities, states, balances, limits, idempotency keys, asset codes, and balanced journal entries without network calls, AI inference, wall-clock assumptions, floating-point money, or mutable external dependencies.

Design rule: Key requestId globally or by wallet

Implementation Objective

The required output is: Idempotency pattern. It must be version-controlled, reviewable, testable with the WAN disabled, and linked to the architecture/source-of-truth model. For a payment platform, every implementation decision should make it possible to explain who authorized an action, which facts were trusted, which controls ran, what financial journal committed, and how the result is recovered after failure.

Key Engineering Decisions

- Key requestId globally or by wallet
- Store result/tx reference
- Check before all writes
- Never perform side effects before duplicate check

Deep Implementation Guidance

Keep all writes required for one logical financial action in one transaction when atomicity is required. Use deterministic validation and stable error codes. Never call an external API, database, clock service, random generator, or AI model from consensus-critical chaincode.

Data and Control Questions

- What is the authoritative source for every input used by this component?
- Which user, device, service, MSP, or administrator identity is allowed to initiate or approve it?
- Does the operation move value, reserve value, create a side effect, or only provide advisory information?
- What idempotency/replay key proves that a retry is the same business intent?
- What evidence is retained off-chain and what stable reference/hash belongs on the ledger?

Chapter 99 Implementation - Ledger-Level Idempotency

Implementation Procedure

491. Parse and validate every argument deterministically.
492. Authorize caller MSP/attributes and validate entity lifecycle state.
493. Check idempotency before any business write.
494. Write balanced journal/current state atomically and emit compact events.
495. Return stable error codes and test concurrent/MVCC conflict behavior.

Implementation Example

```
# Ledger-Level Idempotency
INPUT  -> authenticate, validate schema, bind requestId/nonce
FACTS  -> read authoritative wallet/merchant/state data
AI     -> optional bounded recommendation with decisionId/version
POLICY -> enforce ownership, hard limits, approvals, risk fallback
LEDGER -> submit one idempotent deterministic transaction when value changes
EVENT  -> update projection/case/monitoring asynchronously
VERIFY -> discover committed result and retain correlationId + txId evidence
```

Verification Checklist

- The idempotency pattern exists and is version-controlled.
- The feature works with all public Internet/WAN connectivity disabled.
- Authorization, malformed input, duplicate retry, and failure behavior have explicit tests.
- If financial value changes, journal debits and credits balance exactly and historical entries remain immutable.
- If AI is involved, model/prompt/feature version and fallback behavior are recorded and deterministic policy remains final.
- A timeout/restart cannot create a duplicate financial result; request status can be rediscovered.
- Logs/evidence contain correlation references but not secrets or prohibited sensitive payment data.

Common Failure Modes

- External HTTP/AI call from chaincode.
- Non-deterministic timestamps/randomness.
- No duplicate-request guard.
- Balance checks split across non-atomic transactions.

Expert Review Questions

- What happens if this component succeeds but the next component fails?
- How is the authorizing identity/device/key revoked or recovered?
- Can the operation be safely retried after the client loses the response?
- What happens if AI is unavailable, slow, or wrong?
- Can an auditor reconstruct debit, credit, actor, risk decision, approval, and Fabric txId from retained evidence?
- Which compensating transaction resolves a downstream operational error without rewriting history?

Chapter 100 - Signed Authorization Digest Verification Reference

Anchor proof that the backend validated a signed user/device intent. Chaincode is the deterministic financial rules engine. It must validate identities, states, balances, limits, idempotency keys, asset codes, and balanced journal entries without network calls, AI inference, wall-clock assumptions, floating-point money, or mutable external dependencies.

Design rule: Store SHA-256 digest and key/device version reference

Implementation Objective

The required output is: Authorization evidence fields. It must be version-controlled, reviewable, testable with the WAN disabled, and linked to the architecture/source-of-truth model. For a payment platform, every implementation decision should make it possible to explain who authorized an action, which facts were trusted, which controls ran, what financial journal committed, and how the result is recovered after failure.

Key Engineering Decisions

- Store SHA-256 digest and key/device version reference
- Chaincode can validate presence/format/policy
- Signature verification may remain in trusted backend unless chaincode crypto design is fully deterministic/supported

Deep Implementation Guidance

Keep all writes required for one logical financial action in one transaction when atomicity is required. Use deterministic validation and stable error codes. Never call an external API, database, clock service, random generator, or AI model from consensus-critical chaincode.

Data and Control Questions

- What is the authoritative source for every input used by this component?
- Which user, device, service, MSP, or administrator identity is allowed to initiate or approve it?
- Does the operation move value, reserve value, create a side effect, or only provide advisory information?
- What idempotency/replay key proves that a retry is the same business intent?
- What evidence is retained off-chain and what stable reference/hash belongs on the ledger?

Chapter 100 Implementation - Signed Authorization Digest Verification Reference

Implementation Procedure

496. Parse and validate every argument deterministically.
497. Authorize caller MSP/attributes and validate entity lifecycle state.
498. Check idempotency before any business write.
499. Write balanced journal/current state atomically and emit compact events.
500. Return stable error codes and test concurrent/MVCC conflict behavior.

Implementation Example

```
# Signed Authorization Digest Verification Reference
INPUT  -> authenticate, validate schema, bind requestId/nonce
FACTS  -> read authoritative wallet/merchant/state data
AI     -> optional bounded recommendation with decisionId/version
POLICY -> enforce ownership, hard limits, approvals, risk fallback
LEDGER -> submit one idempotent deterministic transaction when value changes
EVENT  -> update projection/case/monitoring asynchronously
VERIFY -> discover committed result and retain correlationId + txId evidence
```

Verification Checklist

- The authorization evidence fields exists and is version-controlled.
- The feature works with all public Internet/WAN connectivity disabled.
- Authorization, malformed input, duplicate retry, and failure behavior have explicit tests.
- If financial value changes, journal debits and credits balance exactly and historical entries remain immutable.
- If AI is involved, model/prompt/feature version and fallback behavior are recorded and deterministic policy remains final.
- A timeout/restart cannot create a duplicate financial result; request status can be rediscovered.
- Logs/evidence contain correlation references but not secrets or prohibited sensitive payment data.

Common Failure Modes

- External HTTP/AI call from chaincode.
- Non-deterministic timestamps/randomness.
- No duplicate-request guard.
- Balance checks split across non-atomic transactions.

Expert Review Questions

- What happens if this component succeeds but the next component fails?
- How is the authorizing identity/device/key revoked or recovered?
- Can the operation be safely retried after the client loses the response?
- What happens if AI is unavailable, slow, or wrong?
- Can an auditor reconstruct debit, credit, actor, risk decision, approval, and Fabric txId from retained evidence?
- Which compensating transaction resolves a downstream operational error without rewriting history?

Chapter 101 - Composite Keys and Indexes

Make deterministic key organization and queries efficient. Chaincode is the deterministic financial rules engine. It must validate identities, states, balances, limits, idempotency keys, asset codes, and balanced journal entries without network calls, AI inference, wall-clock assumptions, floating-point money, or mutable external dependencies.

Design rule: Use prefixes WALLET JOURNAL PAYMENT HOLD REQUEST

Implementation Objective

The required output is: Key strategy. It must be version-controlled, reviewable, testable with the WAN disabled, and linked to the architecture/source-of-truth model. For a payment platform, every implementation decision should make it possible to explain who authorized an action, which facts were trusted, which controls ran, what financial journal committed, and how the result is recovered after failure.

Key Engineering Decisions

- Use prefixes WALLET JOURNAL PAYMENT HOLD REQUEST
- Composite keys for owner/date/reference indexes
- Avoid unbounded scans

Deep Implementation Guidance

Keep all writes required for one logical financial action in one transaction when atomicity is required. Use deterministic validation and stable error codes. Never call an external API, database, clock service, random generator, or AI model from consensus-critical chaincode.

Data and Control Questions

- What is the authoritative source for every input used by this component?
- Which user, device, service, MSP, or administrator identity is allowed to initiate or approve it?
- Does the operation move value, reserve value, create a side effect, or only provide advisory information?
- What idempotency/replay key proves that a retry is the same business intent?
- What evidence is retained off-chain and what stable reference/hash belongs on the ledger?

Chapter 101 Implementation - Composite Keys and Indexes

Implementation Procedure

501. Parse and validate every argument deterministically.
502. Authorize caller MSP/attributes and validate entity lifecycle state.
503. Check idempotency before any business write.
504. Write balanced journal/current state atomically and emit compact events.
505. Return stable error codes and test concurrent/MVCC conflict behavior.

Implementation Example

```
# Composite Keys and Indexes
INPUT  -> authenticate, validate schema, bind requestId/nonce
FACTS  -> read authoritative wallet/merchant/state data
AI     -> optional bounded recommendation with decisionId/version
POLICY -> enforce ownership, hard limits, approvals, risk fallback
LEDGER -> submit one idempotent deterministic transaction when value changes
EVENT  -> update projection/case/monitoring asynchronously
VERIFY -> discover committed result and retain correlationId + txId evidence
```

Verification Checklist

- The key strategy exists and is version-controlled.
- The feature works with all public Internet/WAN connectivity disabled.
- Authorization, malformed input, duplicate retry, and failure behavior have explicit tests.
- If financial value changes, journal debits and credits balance exactly and historical entries remain immutable.
- If AI is involved, model/prompt/feature version and fallback behavior are recorded and deterministic policy remains final.
- A timeout/restart cannot create a duplicate financial result; request status can be rediscovered.
- Logs/evidence contain correlation references but not secrets or prohibited sensitive payment data.

Common Failure Modes

- External HTTP/AI call from chaincode.
- Non-deterministic timestamps/randomness.
- No duplicate-request guard.
- Balance checks split across non-atomic transactions.

Expert Review Questions

- What happens if this component succeeds but the next component fails?
- How is the authorizing identity/device/key revoked or recovered?
- Can the operation be safely retried after the client loses the response?
- What happens if AI is unavailable, slow, or wrong?
- Can an auditor reconstruct debit, credit, actor, risk decision, approval, and Fabric txId from retained evidence?
- Which compensating transaction resolves a downstream operational error without rewriting history?

Chapter 102 - Chaincode Events

Emit compact committed events for projections and operations. Chaincode is the deterministic financial rules engine. It must validate identities, states, balances, limits, idempotency keys, asset codes, and balanced journal entries without network calls, AI inference, wall-clock assumptions, floating-point money, or mutable external dependencies.

Design rule: Include IDs/type/status not sensitive payloads

Implementation Objective

The required output is: Event schema. It must be version-controlled, reviewable, testable with the WAN disabled, and linked to the architecture/source-of-truth model. For a payment platform, every implementation decision should make it possible to explain who authorized an action, which facts were trusted, which controls ran, what financial journal committed, and how the result is recovered after failure.

Key Engineering Decisions

- Include IDs/type/status not sensitive payloads
- Projection re-reads authoritative data if needed
- Version event schema

Deep Implementation Guidance

Keep all writes required for one logical financial action in one transaction when atomicity is required. Use deterministic validation and stable error codes. Never call an external API, database, clock service, random generator, or AI model from consensus-critical chaincode.

Data and Control Questions

- What is the authoritative source for every input used by this component?
- Which user, device, service, MSP, or administrator identity is allowed to initiate or approve it?
- Does the operation move value, reserve value, create a side effect, or only provide advisory information?
- What idempotency/replay key proves that a retry is the same business intent?
- What evidence is retained off-chain and what stable reference/hash belongs on the ledger?

Chapter 102 Implementation - Chaincode Events

Implementation Procedure

506. Parse and validate every argument deterministically.
507. Authorize caller MSP/attributes and validate entity lifecycle state.
508. Check idempotency before any business write.
509. Write balanced journal/current state atomically and emit compact events.
510. Return stable error codes and test concurrent/MVCC conflict behavior.

Implementation Example

```
# Chaincode Events
INPUT  -> authenticate, validate schema, bind requestId/nonce
FACTS  -> read authoritative wallet/merchant/state data
AI     -> optional bounded recommendation with decisionId/version
POLICY -> enforce ownership, hard limits, approvals, risk fallback
LEDGER -> submit one idempotent deterministic transaction when value changes
EVENT  -> update projection/case/monitoring asynchronously
VERIFY -> discover committed result and retain correlationId + txId evidence
```

Verification Checklist

- The event schema exists and is version-controlled.
- The feature works with all public Internet/WAN connectivity disabled.
- Authorization, malformed input, duplicate retry, and failure behavior have explicit tests.
- If financial value changes, journal debits and credits balance exactly and historical entries remain immutable.
- If AI is involved, model/prompt/feature version and fallback behavior are recorded and deterministic policy remains final.
- A timeout/restart cannot create a duplicate financial result; request status can be rediscovered.
- Logs/evidence contain correlation references but not secrets or prohibited sensitive payment data.

Common Failure Modes

- External HTTP/AI call from chaincode.
- Non-deterministic timestamps/randomness.
- No duplicate-request guard.
- Balance checks split across non-atomic transactions.

Expert Review Questions

- What happens if this component succeeds but the next component fails?
- How is the authorizing identity/device/key revoked or recovered?
- Can the operation be safely retried after the client loses the response?
- What happens if AI is unavailable, slow, or wrong?
- Can an auditor reconstruct debit, credit, actor, risk decision, approval, and Fabric txId from retained evidence?
- Which compensating transaction resolves a downstream operational error without rewriting history?

Chapter 103 - MVCC and Concurrent Spend

Handle two transactions spending the same balance safely. Chaincode is the deterministic financial rules engine. It must validate identities, states, balances, limits, idempotency keys, asset codes, and balanced journal entries without network calls, AI inference, wall-clock assumptions, floating-point money, or mutable external dependencies.

Design rule: Both simulations may read same version

Implementation Objective

The required output is: Concurrency design. It must be version-controlled, reviewable, testable with the WAN disabled, and linked to the architecture/source-of-truth model. For a payment platform, every implementation decision should make it possible to explain who authorized an action, which facts were trusted, which controls ran, what financial journal committed, and how the result is recovered after failure.

Key Engineering Decisions

- Both simulations may read same version
- only valid version commits
- Backend maps invalidation to retry/re-read
- Never retry blindly without re-evaluating balance

Deep Implementation Guidance

Keep all writes required for one logical financial action in one transaction when atomicity is required. Use deterministic validation and stable error codes. Never call an external API, database, clock service, random generator, or AI model from consensus-critical chaincode.

Data and Control Questions

- What is the authoritative source for every input used by this component?
- Which user, device, service, MSP, or administrator identity is allowed to initiate or approve it?
- Does the operation move value, reserve value, create a side effect, or only provide advisory information?
- What idempotency/replay key proves that a retry is the same business intent?
- What evidence is retained off-chain and what stable reference/hash belongs on the ledger?

Chapter 103 Implementation - MVCC and Concurrent Spend

Implementation Procedure

511. Parse and validate every argument deterministically.
512. Authorize caller MSP/attributes and validate entity lifecycle state.
513. Check idempotency before any business write.
514. Write balanced journal/current state atomically and emit compact events.
515. Return stable error codes and test concurrent/MVCC conflict behavior.

Implementation Example

```
# MVCC and Concurrent Spend
INPUT  -> authenticate, validate schema, bind requestId/nonce
FACTS  -> read authoritative wallet/merchant/state data
AI     -> optional bounded recommendation with decisionId/version
POLICY -> enforce ownership, hard limits, approvals, risk fallback
LEDGER -> submit one idempotent deterministic transaction when value changes
EVENT  -> update projection/case/monitoring asynchronously
VERIFY -> discover committed result and retain correlationId + txId evidence
```

Verification Checklist

- The concurrency design exists and is version-controlled.
- The feature works with all public Internet/WAN connectivity disabled.
- Authorization, malformed input, duplicate retry, and failure behavior have explicit tests.
- If financial value changes, journal debits and credits balance exactly and historical entries remain immutable.
- If AI is involved, model/prompt/feature version and fallback behavior are recorded and deterministic policy remains final.
- A timeout/restart cannot create a duplicate financial result; request status can be rediscovered.
- Logs/evidence contain correlation references but not secrets or prohibited sensitive payment data.

Common Failure Modes

- External HTTP/AI call from chaincode.
- Non-deterministic timestamps/randomness.
- No duplicate-request guard.
- Balance checks split across non-atomic transactions.

Expert Review Questions

- What happens if this component succeeds but the next component fails?
- How is the authorizing identity/device/key revoked or recovered?
- Can the operation be safely retried after the client loses the response?
- What happens if AI is unavailable, slow, or wrong?
- Can an auditor reconstruct debit, credit, actor, risk decision, approval, and Fabric txId from retained evidence?
- Which compensating transaction resolves a downstream operational error without rewriting history?

Chapter 104 - Stable Error Model

Return machine-mappable business errors. Chaincode is the deterministic financial rules engine. It must validate identities, states, balances, limits, idempotency keys, asset codes, and balanced journal entries without network calls, AI inference, wall-clock assumptions, floating-point money, or mutable external dependencies.

Design rule: INSUFFICIENT_FUNDS WALLET_FROZEN INVALID_STATE FORBIDDEN LIMIT_EXCEEDED
DUPLICATE/NOT_FOUND

Implementation Objective

The required output is: Error catalog. It must be version-controlled, reviewable, testable with the WAN disabled, and linked to the architecture/source-of-truth model. For a payment platform, every implementation decision should make it possible to explain who authorized an action, which facts were trusted, which controls ran, what financial journal committed, and how the result is recovered after failure.

Key Engineering Decisions

- INSUFFICIENT_FUNDS WALLET_FROZEN INVALID_STATE FORBIDDEN LIMIT_EXCEEDED
DUPLICATE/NOT_FOUND
- Do not expose secrets in errors

Deep Implementation Guidance

Keep all writes required for one logical financial action in one transaction when atomicity is required. Use deterministic validation and stable error codes. Never call an external API, database, clock service, random generator, or AI model from consensus-critical chaincode.

Data and Control Questions

- What is the authoritative source for every input used by this component?
- Which user, device, service, MSP, or administrator identity is allowed to initiate or approve it?
- Does the operation move value, reserve value, create a side effect, or only provide advisory information?
- What idempotency/replay key proves that a retry is the same business intent?
- What evidence is retained off-chain and what stable reference/hash belongs on the ledger?

Chapter 104 Implementation - Stable Error Model

Implementation Procedure

- 516. Parse and validate every argument deterministically.
- 517. Authorize caller MSP/attributes and validate entity lifecycle state.
- 518. Check idempotency before any business write.
- 519. Write balanced journal/current state atomically and emit compact events.
- 520. Return stable error codes and test concurrent/MVCC conflict behavior.

Implementation Example

```
# Stable Error Model
INPUT  -> authenticate, validate schema, bind requestId/nonce
FACTS  -> read authoritative wallet/merchant/state data
AI     -> optional bounded recommendation with decisionId/version
POLICY -> enforce ownership, hard limits, approvals, risk fallback
LEDGER -> submit one idempotent deterministic transaction when value changes
EVENT  -> update projection/case/monitoring asynchronously
VERIFY -> discover committed result and retain correlationId + txId evidence
```

Verification Checklist

- The error catalog exists and is version-controlled.
- The feature works with all public Internet/WAN connectivity disabled.
- Authorization, malformed input, duplicate retry, and failure behavior have explicit tests.
- If financial value changes, journal debits and credits balance exactly and historical entries remain immutable.
- If AI is involved, model/prompt/feature version and fallback behavior are recorded and deterministic policy remains final.
- A timeout/restart cannot create a duplicate financial result; request status can be rediscovered.
- Logs/evidence contain correlation references but not secrets or prohibited sensitive payment data.

Common Failure Modes

- External HTTP/AI call from chaincode.
- Non-deterministic timestamps/randomness.
- No duplicate-request guard.
- Balance checks split across non-atomic transactions.

Expert Review Questions

- What happens if this component succeeds but the next component fails?
- How is the authorizing identity/device/key revoked or recovered?
- Can the operation be safely retried after the client loses the response?
- What happens if AI is unavailable, slow, or wrong?
- Can an auditor reconstruct debit, credit, actor, risk decision, approval, and Fabric txId from retained evidence?
- Which compensating transaction resolves a downstream operational error without rewriting history?

Chapter 105 - Chaincode Upgrade and Schema Migration

Evolve financial rules without losing auditability. Chaincode is the deterministic financial rules engine. It must validate identities, states, balances, limits, idempotency keys, asset codes, and balanced journal entries without network calls, AI inference, wall-clock assumptions, floating-point money, or mutable external dependencies.

Design rule: Use lifecycle sequence/version

Implementation Objective

The required output is: Upgrade plan. It must be version-controlled, reviewable, testable with the WAN disabled, and linked to the architecture/source-of-truth model. For a payment platform, every implementation decision should make it possible to explain who authorized an action, which facts were trusted, which controls ran, what financial journal committed, and how the result is recovered after failure.

Key Engineering Decisions

- Use lifecycle sequence/version
- Backward-compatible state reading
- Explicit migration transactions/scripts
- Staging replay/evaluation before production

Deep Implementation Guidance

Keep all writes required for one logical financial action in one transaction when atomicity is required. Use deterministic validation and stable error codes. Never call an external API, database, clock service, random generator, or AI model from consensus-critical chaincode.

Data and Control Questions

- What is the authoritative source for every input used by this component?
- Which user, device, service, MSP, or administrator identity is allowed to initiate or approve it?
- Does the operation move value, reserve value, create a side effect, or only provide advisory information?
- What idempotency/replay key proves that a retry is the same business intent?
- What evidence is retained off-chain and what stable reference/hash belongs on the ledger?

Chapter 105 Implementation - Chaincode Upgrade and Schema Migration

Implementation Procedure

521. Parse and validate every argument deterministically.
522. Authorize caller MSP/attributes and validate entity lifecycle state.
523. Check idempotency before any business write.
524. Write balanced journal/current state atomically and emit compact events.
525. Return stable error codes and test concurrent/MVCC conflict behavior.

Implementation Example

```
# Chaincode Upgrade and Schema Migration
INPUT  -> authenticate, validate schema, bind requestId/nonce
FACTS  -> read authoritative wallet/merchant/state data
AI     -> optional bounded recommendation with decisionId/version
POLICY -> enforce ownership, hard limits, approvals, risk fallback
LEDGER -> submit one idempotent deterministic transaction when value changes
EVENT  -> update projection/case/monitoring asynchronously
VERIFY -> discover committed result and retain correlationId + txId evidence
```

Verification Checklist

- The upgrade plan exists and is version-controlled.
- The feature works with all public Internet/WAN connectivity disabled.
- Authorization, malformed input, duplicate retry, and failure behavior have explicit tests.
- If financial value changes, journal debits and credits balance exactly and historical entries remain immutable.
- If AI is involved, model/prompt/feature version and fallback behavior are recorded and deterministic policy remains final.
- A timeout/restart cannot create a duplicate financial result; request status can be rediscovered.
- Logs/evidence contain correlation references but not secrets or prohibited sensitive payment data.

Common Failure Modes

- External HTTP/AI call from chaincode.
- Non-deterministic timestamps/randomness.
- No duplicate-request guard.
- Balance checks split across non-atomic transactions.

Expert Review Questions

- What happens if this component succeeds but the next component fails?
- How is the authorizing identity/device/key revoked or recovered?
- Can the operation be safely retried after the client loses the response?
- What happens if AI is unavailable, slow, or wrong?
- Can an auditor reconstruct debit, credit, actor, risk decision, approval, and Fabric txId from retained evidence?
- Which compensating transaction resolves a downstream operational error without rewriting history?

Part VII - Node.js/TypeScript Backend and Integration APIs

The API layer authenticates users, validates request schemas, gathers authoritative facts, calls local AI when useful, applies deterministic policy, submits transactions through Fabric Gateway, waits for commit status, and exposes stable application errors. It must safely recover from lost HTTP responses and duplicate retries.

Part focus: This part contains 24 implementation chapters. Each chapter includes design rules, implementation procedure, a practical artifact, verification, failure analysis, and expert review questions.

Chapter 106 - Fabric Gateway Connection Helper

Connect the local backend to a trusted peer Gateway using pinned TLS and client identity. The API layer authenticates users, validates request schemas, gathers authoritative facts, calls local AI when useful, applies deterministic policy, submits transactions through Fabric Gateway, waits for commit status, and exposes stable application errors. It must safely recover from lost HTTP responses and duplicate retries.

Design rule: Load internal TLS root/client identity from protected paths

Implementation Objective

The required output is: Gateway module. It must be version-controlled, reviewable, testable with the WAN disabled, and linked to the architecture/source-of-truth model. For a payment platform, every implementation decision should make it possible to explain who authorized an action, which facts were trusted, which controls ran, what financial journal committed, and how the result is recovered after failure.

Key Engineering Decisions

- Load internal TLS root/client identity from protected paths
- Set evaluate/endorse/submit/commit deadlines
- Close connections cleanly

Deep Implementation Guidance

Treat the backend as a policy enforcement point, not a bypass around chaincode. A request may be lost after ledger submission, so recovery by requestId is mandatory. Never respond “failed” merely because the HTTP connection failed; discover the committed result first.

Data and Control Questions

- What is the authoritative source for every input used by this component?
- Which user, device, service, MSP, or administrator identity is allowed to initiate or approve it?
- Does the operation move value, reserve value, create a side effect, or only provide advisory information?
- What idempotency/replay key proves that a retry is the same business intent?
- What evidence is retained off-chain and what stable reference/hash belongs on the ledger?

Chapter 106 Implementation - Fabric Gateway Connection Helper

Implementation Procedure

526. Validate transport/schema/auth before calling Fabric or AI.
527. Read authoritative facts from ledger/projection according to the source-of-truth matrix.
528. Call local AI with bounded inputs and require structured output.
529. Apply deterministic policy/human approval before sensitive writes.
530. Submit through Gateway, wait for commit, log txId/correlationId, and make retries safe.

Implementation Example

```
# Fabric Gateway Connection Helper
INPUT  -> authenticate, validate schema, bind requestId/nonce
FACTS  -> read authoritative wallet/merchant/state data
AI     -> optional bounded recommendation with decisionId/version
POLICY -> enforce ownership, hard limits, approvals, risk fallback
LEDGER -> submit one idempotent deterministic transaction when value changes
EVENT  -> update projection/case/monitoring asynchronously
VERIFY -> discover committed result and retain correlationId + txId evidence
```

Verification Checklist

- The gateway module exists and is version-controlled.
- The feature works with all public Internet/WAN connectivity disabled.
- Authorization, malformed input, duplicate retry, and failure behavior have explicit tests.
- If financial value changes, journal debits and credits balance exactly and historical entries remain immutable.
- If AI is involved, model/prompt/feature version and fallback behavior are recorded and deterministic policy remains final.
- A timeout/restart cannot create a duplicate financial result; request status can be rediscovered.
- Logs/evidence contain correlation references but not secrets or prohibited sensitive payment data.

Common Failure Modes

- Returning success before commit status.
- Retry after timeout creates duplicate transfer.
- AI recommendation treated as authorization.
- Secrets/PAN/credentials logged.

Expert Review Questions

- What happens if this component succeeds but the next component fails?
- How is the authorizing identity/device/key revoked or recovered?
- Can the operation be safely retried after the client loses the response?
- What happens if AI is unavailable, slow, or wrong?
- Can an auditor reconstruct debit, credit, actor, risk decision, approval, and Fabric txId from retained evidence?
- Which compensating transaction resolves a downstream operational error without rewriting history?

Chapter 107 - Command vs Query Architecture

Separate write commands from high-volume operational reads. The API layer authenticates users, validates request schemas, gathers authoritative facts, calls local AI when useful, applies deterministic policy, submits transactions through Fabric Gateway, waits for commit status, and exposes stable application errors. It must safely recover from lost HTTP responses and duplicate retries.

Design rule: Financial commands submit to ledger

Implementation Objective

The required output is: CQRS design. It must be version-controlled, reviewable, testable with the WAN disabled, and linked to the architecture/source-of-truth model. For a payment platform, every implementation decision should make it possible to explain who authorized an action, which facts were trusted, which controls ran, what financial journal committed, and how the result is recovered after failure.

Key Engineering Decisions

- Financial commands submit to ledger
- Queries use ledger when authoritative proof is required and PostgreSQL projection for fast UI/search
- Document freshness expectations

Deep Implementation Guidance

Treat the backend as a policy enforcement point, not a bypass around chaincode. A request may be lost after ledger submission, so recovery by requestId is mandatory. Never respond “failed” merely because the HTTP connection failed; discover the committed result first.

Data and Control Questions

- What is the authoritative source for every input used by this component?
- Which user, device, service, MSP, or administrator identity is allowed to initiate or approve it?
- Does the operation move value, reserve value, create a side effect, or only provide advisory information?
- What idempotency/replay key proves that a retry is the same business intent?
- What evidence is retained off-chain and what stable reference/hash belongs on the ledger?

Chapter 107 Implementation - Command vs Query Architecture

Implementation Procedure

531. Validate transport/schema/auth before calling Fabric or AI.
532. Read authoritative facts from ledger/projection according to the source-of-truth matrix.
533. Call local AI with bounded inputs and require structured output.
534. Apply deterministic policy/human approval before sensitive writes.
535. Submit through Gateway, wait for commit, log txId/correlationId, and make retries safe.

Implementation Example

```
# Command vs Query Architecture
INPUT  -> authenticate, validate schema, bind requestId/nonce
FACTS  -> read authoritative wallet/merchant/state data
AI     -> optional bounded recommendation with decisionId/version
POLICY -> enforce ownership, hard limits, approvals, risk fallback
LEDGER -> submit one idempotent deterministic transaction when value changes
EVENT  -> update projection/case/monitoring asynchronously
VERIFY -> discover committed result and retain correlationId + txId evidence
```

Verification Checklist

- The cqrs design exists and is version-controlled.
- The feature works with all public Internet/WAN connectivity disabled.
- Authorization, malformed input, duplicate retry, and failure behavior have explicit tests.
- If financial value changes, journal debits and credits balance exactly and historical entries remain immutable.
- If AI is involved, model/prompt/feature version and fallback behavior are recorded and deterministic policy remains final.
- A timeout/restart cannot create a duplicate financial result; request status can be rediscovered.
- Logs/evidence contain correlation references but not secrets or prohibited sensitive payment data.

Common Failure Modes

- Returning success before commit status.
- Retry after timeout creates duplicate transfer.
- AI recommendation treated as authorization.
- Secrets/PAN/credentials logged.

Expert Review Questions

- What happens if this component succeeds but the next component fails?
- How is the authorizing identity/device/key revoked or recovered?
- Can the operation be safely retried after the client loses the response?
- What happens if AI is unavailable, slow, or wrong?
- Can an auditor reconstruct debit, credit, actor, risk decision, approval, and Fabric txId from retained evidence?
- Which compensating transaction resolves a downstream operational error without rewriting history?

Chapter 108 - REST API Standards

Create consistent local HTTP interfaces for wallet and merchant applications. The API layer authenticates users, validates request schemas, gathers authoritative facts, calls local AI when useful, applies deterministic policy, submits transactions through Fabric Gateway, waits for commit status, and exposes stable application errors. It must safely recover from lost HTTP responses and duplicate retries.

Design rule: Version endpoints

Implementation Objective

The required output is: API standard. It must be version-controlled, reviewable, testable with the WAN disabled, and linked to the architecture/source-of-truth model. For a payment platform, every implementation decision should make it possible to explain who authorized an action, which facts were trusted, which controls ran, what financial journal committed, and how the result is recovered after failure.

Key Engineering Decisions

- Version endpoints
- Use JSON schema validation
- Return correlationId/requestId/txId
- Use stable error codes

Deep Implementation Guidance

Treat the backend as a policy enforcement point, not a bypass around chaincode. A request may be lost after ledger submission, so recovery by requestId is mandatory. Never respond “failed” merely because the HTTP connection failed; discover the committed result first.

Data and Control Questions

- What is the authoritative source for every input used by this component?
- Which user, device, service, MSP, or administrator identity is allowed to initiate or approve it?
- Does the operation move value, reserve value, create a side effect, or only provide advisory information?
- What idempotency/replay key proves that a retry is the same business intent?
- What evidence is retained off-chain and what stable reference/hash belongs on the ledger?

Chapter 108 Implementation - REST API Standards

Implementation Procedure

536. Validate transport/schema/auth before calling Fabric or AI.
537. Read authoritative facts from ledger/projection according to the source-of-truth matrix.
538. Call local AI with bounded inputs and require structured output.
539. Apply deterministic policy/human approval before sensitive writes.
540. Submit through Gateway, wait for commit, log txId/correlationId, and make retries safe.

Implementation Example

```
# REST API Standards
INPUT  -> authenticate, validate schema, bind requestId/nonce
FACTS  -> read authoritative wallet/merchant/state data
AI      -> optional bounded recommendation with decisionId/version
POLICY  -> enforce ownership, hard limits, approvals, risk fallback
LEDGER  -> submit one idempotent deterministic transaction when value changes
EVENT   -> update projection/case/monitoring asynchronously
VERIFY  -> discover committed result and retain correlationId + txId evidence
```

Verification Checklist

- The api standard exists and is version-controlled.
- The feature works with all public Internet/WAN connectivity disabled.
- Authorization, malformed input, duplicate retry, and failure behavior have explicit tests.
- If financial value changes, journal debits and credits balance exactly and historical entries remain immutable.
- If AI is involved, model/prompt/feature version and fallback behavior are recorded and deterministic policy remains final.
- A timeout/restart cannot create a duplicate financial result; request status can be rediscovered.
- Logs/evidence contain correlation references but not secrets or prohibited sensitive payment data.

Common Failure Modes

- Returning success before commit status.
- Retry after timeout creates duplicate transfer.
- AI recommendation treated as authorization.
- Secrets/PAN/credentials logged.

Expert Review Questions

- What happens if this component succeeds but the next component fails?
- How is the authorizing identity/device/key revoked or recovered?
- Can the operation be safely retried after the client loses the response?
- What happens if AI is unavailable, slow, or wrong?
- Can an auditor reconstruct debit, credit, actor, risk decision, approval, and Fabric txId from retained evidence?
- Which compensating transaction resolves a downstream operational error without rewriting history?

Chapter 109 - Authentication Middleware

Authenticate local users/sessions before any wallet operation. The API layer authenticates users, validates request schemas, gathers authoritative facts, calls local AI when useful, applies deterministic policy, submits transactions through Fabric Gateway, waits for commit status, and exposes stable application errors. It must safely recover from lost HTTP responses and duplicate retries.

Design rule: Verify session/token locally

Implementation Objective

The required output is: Auth middleware. It must be version-controlled, reviewable, testable with the WAN disabled, and linked to the architecture/source-of-truth model. For a payment platform, every implementation decision should make it possible to explain who authorized an action, which facts were trusted, which controls ran, what financial journal committed, and how the result is recovered after failure.

Key Engineering Decisions

- Verify session/token locally
- Load roles/wallet bindings/device
- Deny expired/revoked user/device
- Never trust client-supplied role

Deep Implementation Guidance

Treat the backend as a policy enforcement point, not a bypass around chaincode. A request may be lost after ledger submission, so recovery by requestId is mandatory. Never respond “failed” merely because the HTTP connection failed; discover the committed result first.

Data and Control Questions

- What is the authoritative source for every input used by this component?
- Which user, device, service, MSP, or administrator identity is allowed to initiate or approve it?
- Does the operation move value, reserve value, create a side effect, or only provide advisory information?
- What idempotency/replay key proves that a retry is the same business intent?
- What evidence is retained off-chain and what stable reference/hash belongs on the ledger?

Chapter 109 Implementation - Authentication Middleware

Implementation Procedure

541. Validate transport/schema/auth before calling Fabric or AI.
542. Read authoritative facts from ledger/projection according to the source-of-truth matrix.
543. Call local AI with bounded inputs and require structured output.
544. Apply deterministic policy/human approval before sensitive writes.
545. Submit through Gateway, wait for commit, log txId/correlationId, and make retries safe.

Implementation Example

```
# Authentication Middleware
INPUT  -> authenticate, validate schema, bind requestId/nonce
FACTS  -> read authoritative wallet/merchant/state data
AI      -> optional bounded recommendation with decisionId/version
POLICY  -> enforce ownership, hard limits, approvals, risk fallback
LEDGER  -> submit one idempotent deterministic transaction when value changes
EVENT   -> update projection/case/monitoring asynchronously
VERIFY  -> discover committed result and retain correlationId + txId evidence
```

Verification Checklist

- The auth middleware exists and is version-controlled.
- The feature works with all public Internet/WAN connectivity disabled.
- Authorization, malformed input, duplicate retry, and failure behavior have explicit tests.
- If financial value changes, journal debits and credits balance exactly and historical entries remain immutable.
- If AI is involved, model/prompt/feature version and fallback behavior are recorded and deterministic policy remains final.
- A timeout/restart cannot create a duplicate financial result; request status can be rediscovered.
- Logs/evidence contain correlation references but not secrets or prohibited sensitive payment data.

Common Failure Modes

- Returning success before commit status.
- Retry after timeout creates duplicate transfer.
- AI recommendation treated as authorization.
- Secrets/PAN/credentials logged.

Expert Review Questions

- What happens if this component succeeds but the next component fails?
- How is the authorizing identity/device/key revoked or recovered?
- Can the operation be safely retried after the client loses the response?
- What happens if AI is unavailable, slow, or wrong?
- Can an auditor reconstruct debit, credit, actor, risk decision, approval, and Fabric txId from retained evidence?
- Which compensating transaction resolves a downstream operational error without rewriting history?

Chapter 110 - Authorization Service

Make business authorization explicit before risk scoring and ledger submission. The API layer authenticates users, validates request schemas, gathers authoritative facts, calls local AI when useful, applies deterministic policy, submits transactions through Fabric Gateway, waits for commit status, and exposes stable application errors. It must safely recover from lost HTTP responses and duplicate retries.

Design rule: Check wallet ownership merchant/store binding operation permission

Implementation Objective

The required output is: Authorization policy service. It must be version-controlled, reviewable, testable with the WAN disabled, and linked to the architecture/source-of-truth model. For a payment platform, every implementation decision should make it possible to explain who authorized an action, which facts were trusted, which controls ran, what financial journal committed, and how the result is recovered after failure.

Key Engineering Decisions

- Check wallet ownership merchant/store binding operation permission
- Hard-deny unauthorized requests
- Record policy decision

Deep Implementation Guidance

Treat the backend as a policy enforcement point, not a bypass around chaincode. A request may be lost after ledger submission, so recovery by requestId is mandatory. Never respond “failed” merely because the HTTP connection failed; discover the committed result first.

Data and Control Questions

- What is the authoritative source for every input used by this component?
- Which user, device, service, MSP, or administrator identity is allowed to initiate or approve it?
- Does the operation move value, reserve value, create a side effect, or only provide advisory information?
- What idempotency/replay key proves that a retry is the same business intent?
- What evidence is retained off-chain and what stable reference/hash belongs on the ledger?

Chapter 110 Implementation - Authorization Service

Implementation Procedure

546. Validate transport/schema/auth before calling Fabric or AI.
547. Read authoritative facts from ledger/projection according to the source-of-truth matrix.
548. Call local AI with bounded inputs and require structured output.
549. Apply deterministic policy/human approval before sensitive writes.
550. Submit through Gateway, wait for commit, log txId/correlationId, and make retries safe.

Implementation Example

```
# Authorization Service
INPUT  -> authenticate, validate schema, bind requestId/nonce
FACTS  -> read authoritative wallet/merchant/state data
AI     -> optional bounded recommendation with decisionId/version
POLICY -> enforce ownership, hard limits, approvals, risk fallback
LEDGER -> submit one idempotent deterministic transaction when value changes
EVENT  -> update projection/case/monitoring asynchronously
VERIFY -> discover committed result and retain correlationId + txId evidence
```

Verification Checklist

- The authorization policy service exists and is version-controlled.
- The feature works with all public Internet/WAN connectivity disabled.
- Authorization, malformed input, duplicate retry, and failure behavior have explicit tests.
- If financial value changes, journal debits and credits balance exactly and historical entries remain immutable.
- If AI is involved, model/prompt/feature version and fallback behavior are recorded and deterministic policy remains final.
- A timeout/restart cannot create a duplicate financial result; request status can be rediscovered.
- Logs/evidence contain correlation references but not secrets or prohibited sensitive payment data.

Common Failure Modes

- Returning success before commit status.
- Retry after timeout creates duplicate transfer.
- AI recommendation treated as authorization.
- Secrets/PAN/credentials logged.

Expert Review Questions

- What happens if this component succeeds but the next component fails?
- How is the authorizing identity/device/key revoked or recovered?
- Can the operation be safely retried after the client loses the response?
- What happens if AI is unavailable, slow, or wrong?
- Can an auditor reconstruct debit, credit, actor, risk decision, approval, and Fabric txId from retained evidence?
- Which compensating transaction resolves a downstream operational error without rewriting history?

Chapter 111 - Transaction Authorization Service

Verify signed device/user payment envelopes when enabled. The API layer authenticates users, validates request schemas, gathers authoritative facts, calls local AI when useful, applies deterministic policy, submits transactions through Fabric Gateway, waits for commit status, and exposes stable application errors. It must safely recover from lost HTTP responses and duplicate retries.

Design rule: Canonicalize fields

Implementation Objective

The required output is: Authorization verification module. It must be version-controlled, reviewable, testable with the WAN disabled, and linked to the architecture/source-of-truth model. For a payment platform, every implementation decision should make it possible to explain who authorized an action, which facts were trusted, which controls ran, what financial journal committed, and how the result is recovered after failure.

Key Engineering Decisions

- Canonicalize fields
- Verify signature/key version/nonce/requestId
- Check amount/destination binding
- Produce authDigest and evidence record

Deep Implementation Guidance

Treat the backend as a policy enforcement point, not a bypass around chaincode. A request may be lost after ledger submission, so recovery by requestId is mandatory. Never respond “failed” merely because the HTTP connection failed; discover the committed result first.

Data and Control Questions

- What is the authoritative source for every input used by this component?
- Which user, device, service, MSP, or administrator identity is allowed to initiate or approve it?
- Does the operation move value, reserve value, create a side effect, or only provide advisory information?
- What idempotency/replay key proves that a retry is the same business intent?
- What evidence is retained off-chain and what stable reference/hash belongs on the ledger?

Chapter 111 Implementation - Transaction Authorization Service

Implementation Procedure

551. Validate transport/schema/auth before calling Fabric or AI.
552. Read authoritative facts from ledger/projection according to the source-of-truth matrix.
553. Call local AI with bounded inputs and require structured output.
554. Apply deterministic policy/human approval before sensitive writes.
555. Submit through Gateway, wait for commit, log txId/correlationId, and make retries safe.

Implementation Example

```
# Transaction Authorization Service
INPUT  -> authenticate, validate schema, bind requestId/nonce
FACTS  -> read authoritative wallet/merchant/state data
AI     -> optional bounded recommendation with decisionId/version
POLICY -> enforce ownership, hard limits, approvals, risk fallback
LEDGER -> submit one idempotent deterministic transaction when value changes
EVENT  -> update projection/case/monitoring asynchronously
VERIFY -> discover committed result and retain correlationId + txId evidence
```

Verification Checklist

- The authorization verification module exists and is version-controlled.
- The feature works with all public Internet/WAN connectivity disabled.
- Authorization, malformed input, duplicate retry, and failure behavior have explicit tests.
- If financial value changes, journal debits and credits balance exactly and historical entries remain immutable.
- If AI is involved, model/prompt/feature version and fallback behavior are recorded and deterministic policy remains final.
- A timeout/restart cannot create a duplicate financial result; request status can be rediscovered.
- Logs/evidence contain correlation references but not secrets or prohibited sensitive payment data.

Common Failure Modes

- Returning success before commit status.
- Retry after timeout creates duplicate transfer.
- AI recommendation treated as authorization.
- Secrets/PAN/credentials logged.

Expert Review Questions

- What happens if this component succeeds but the next component fails?
- How is the authorizing identity/device/key revoked or recovered?
- Can the operation be safely retried after the client loses the response?
- What happens if AI is unavailable, slow, or wrong?
- Can an auditor reconstruct debit, credit, actor, risk decision, approval, and Fabric txId from retained evidence?
- Which compensating transaction resolves a downstream operational error without rewriting history?

Chapter 112 - AI Risk Client

Call the local risk service using bounded timeouts and strict schema. The API layer authenticates users, validates request schemas, gathers authoritative facts, calls local AI when useful, applies deterministic policy, submits transactions through Fabric Gateway, waits for commit status, and exposes stable application errors. It must safely recover from lost HTTP responses and duplicate retries.

Design rule: Send minimum required features

Implementation Objective

The required output is: Risk client. It must be version-controlled, reviewable, testable with the WAN disabled, and linked to the architecture/source-of-truth model. For a payment platform, every implementation decision should make it possible to explain who authorized an action, which facts were trusted, which controls ran, what financial journal committed, and how the result is recovered after failure.

Key Engineering Decisions

- Send minimum required features
- Require modelVersion/decisionId/recommendation/reasonCodes
- Timeout falls back to configured deterministic policy

Deep Implementation Guidance

Treat the backend as a policy enforcement point, not a bypass around chaincode. A request may be lost after ledger submission, so recovery by requestId is mandatory. Never respond “failed” merely because the HTTP connection failed; discover the committed result first.

Data and Control Questions

- What is the authoritative source for every input used by this component?
- Which user, device, service, MSP, or administrator identity is allowed to initiate or approve it?
- Does the operation move value, reserve value, create a side effect, or only provide advisory information?
- What idempotency/replay key proves that a retry is the same business intent?
- What evidence is retained off-chain and what stable reference/hash belongs on the ledger?

Chapter 112 Implementation - AI Risk Client

Implementation Procedure

- 556. Validate transport/schema/auth before calling Fabric or AI.
- 557. Read authoritative facts from ledger/projection according to the source-of-truth matrix.
- 558. Call local AI with bounded inputs and require structured output.
- 559. Apply deterministic policy/human approval before sensitive writes.
- 560. Submit through Gateway, wait for commit, log txId/correlationId, and make retries safe.

Implementation Example

```
# AI Risk Client
INPUT  -> authenticate, validate schema, bind requestId/nonce
FACTS  -> read authoritative wallet/merchant/state data
AI      -> optional bounded recommendation with decisionId/version
POLICY  -> enforce ownership, hard limits, approvals, risk fallback
LEDGER  -> submit one idempotent deterministic transaction when value changes
EVENT   -> update projection/case/monitoring asynchronously
VERIFY  -> discover committed result and retain correlationId + txId evidence
```

Verification Checklist

- The risk client exists and is version-controlled.
- The feature works with all public Internet/WAN connectivity disabled.
- Authorization, malformed input, duplicate retry, and failure behavior have explicit tests.
- If financial value changes, journal debits and credits balance exactly and historical entries remain immutable.
- If AI is involved, model/prompt/feature version and fallback behavior are recorded and deterministic policy remains final.
- A timeout/restart cannot create a duplicate financial result; request status can be rediscovered.
- Logs/evidence contain correlation references but not secrets or prohibited sensitive payment data.

Common Failure Modes

- Returning success before commit status.
- Retry after timeout creates duplicate transfer.
- AI recommendation treated as authorization.
- Secrets/PAN/credentials logged.

Expert Review Questions

- What happens if this component succeeds but the next component fails?
- How is the authorizing identity/device/key revoked or recovered?
- Can the operation be safely retried after the client loses the response?
- What happens if AI is unavailable, slow, or wrong?
- Can an auditor reconstruct debit, credit, actor, risk decision, approval, and Fabric txId from retained evidence?
- Which compensating transaction resolves a downstream operational error without rewriting history?

Chapter 113 - Deterministic Policy Gate

Combine hard limits, AI recommendation, approval status, and operational state. The API layer authenticates users, validates request schemas, gathers authoritative facts, calls local AI when useful, applies deterministic policy, submits transactions through Fabric Gateway, waits for commit status, and exposes stable application errors. It must safely recover from lost HTTP responses and duplicate retries.

Design rule: BLOCK hard denies immediately

Implementation Objective

The required output is: Policy gate. It must be version-controlled, reviewable, testable with the WAN disabled, and linked to the architecture/source-of-truth model. For a payment platform, every implementation decision should make it possible to explain who authorized an action, which facts were trusted, which controls ran, what financial journal committed, and how the result is recovered after failure.

Key Engineering Decisions

- BLOCK hard denies immediately
- REVIEW enters case queue
- ALLOW still must pass permissions/limits
- No LLM prose directly controls payment

Deep Implementation Guidance

Treat the backend as a policy enforcement point, not a bypass around chaincode. A request may be lost after ledger submission, so recovery by requestId is mandatory. Never respond “failed” merely because the HTTP connection failed; discover the committed result first.

Data and Control Questions

- What is the authoritative source for every input used by this component?
- Which user, device, service, MSP, or administrator identity is allowed to initiate or approve it?
- Does the operation move value, reserve value, create a side effect, or only provide advisory information?
- What idempotency/replay key proves that a retry is the same business intent?
- What evidence is retained off-chain and what stable reference/hash belongs on the ledger?

Chapter 113 Implementation - Deterministic Policy Gate

Implementation Procedure

561. Validate transport/schema/auth before calling Fabric or AI.
562. Read authoritative facts from ledger/projection according to the source-of-truth matrix.
563. Call local AI with bounded inputs and require structured output.
564. Apply deterministic policy/human approval before sensitive writes.
565. Submit through Gateway, wait for commit, log txId/correlationId, and make retries safe.

Implementation Example

```
async function authorizePayment(ctx: RequestContext, req: PaymentRequest) {
  enforceAuthenticated(ctx);
  enforceWalletOwnership(ctx.user, req.fromWalletId);
  enforceHardLimits(req);          // exact code rules

  const risk = await riskWithFallback(req);    // advisory AI
  if (risk.recommendation === 'BLOCK') return {status: 'REJECTED', risk};
  if (risk.recommendation === 'REVIEW') return createReviewCase(req, risk);

  const approval = requiresHumanApproval(req) ? await requireApproval(req) : undefined;
  return submitLedgerPayment(req, risk.decisionId, approval?.approvalId);
}
```

Verification Checklist

- The policy gate exists and is version-controlled.
- The feature works with all public Internet/WAN connectivity disabled.
- Authorization, malformed input, duplicate retry, and failure behavior have explicit tests.
- If financial value changes, journal debits and credits balance exactly and historical entries remain immutable.
- If AI is involved, model/prompt/feature version and fallback behavior are recorded and deterministic policy remains final.
- A timeout/restart cannot create a duplicate financial result; request status can be rediscovered.
- Logs/evidence contain correlation references but not secrets or prohibited sensitive payment data.

Common Failure Modes

- Returning success before commit status.
- Retry after timeout creates duplicate transfer.
- AI recommendation treated as authorization.
- Secrets/PAN/credentials logged.

Expert Review Questions

- What happens if this component succeeds but the next component fails?
- How is the authorizing identity/device/key revoked or recovered?
- Can the operation be safely retried after the client loses the response?
- What happens if AI is unavailable, slow, or wrong?
- Can an auditor reconstruct debit, credit, actor, risk decision, approval, and Fabric txId from retained evidence?
- Which compensating transaction resolves a downstream operational error without rewriting history?

Chapter 114 - Human Approval Service

Represent approvals as signed/time-bounded business objects. The API layer authenticates users, validates request schemas, gathers authoritative facts, calls local AI when useful, applies deterministic policy, submits transactions through Fabric Gateway, waits for commit status, and exposes stable application errors. It must safely recover from lost HTTP responses and duplicate retries.

Design rule: Requester and approver separation

Implementation Objective

The required output is: Approval API. It must be version-controlled, reviewable, testable with the WAN disabled, and linked to the architecture/source-of-truth model. For a payment platform, every implementation decision should make it possible to explain who authorized an action, which facts were trusted, which controls ran, what financial journal committed, and how the result is recovered after failure.

Key Engineering Decisions

- Requester and approver separation
- Scope action/amount/wallet/request
- Expire/revoke
- Anchor approval reference/hash in financial transaction

Deep Implementation Guidance

Treat the backend as a policy enforcement point, not a bypass around chaincode. A request may be lost after ledger submission, so recovery by requestId is mandatory. Never respond “failed” merely because the HTTP connection failed; discover the committed result first.

Data and Control Questions

- What is the authoritative source for every input used by this component?
- Which user, device, service, MSP, or administrator identity is allowed to initiate or approve it?
- Does the operation move value, reserve value, create a side effect, or only provide advisory information?
- What idempotency/replay key proves that a retry is the same business intent?
- What evidence is retained off-chain and what stable reference/hash belongs on the ledger?

Chapter 114 Implementation - Human Approval Service

Implementation Procedure

566. Validate transport/schema/auth before calling Fabric or AI.
567. Read authoritative facts from ledger/projection according to the source-of-truth matrix.
568. Call local AI with bounded inputs and require structured output.
569. Apply deterministic policy/human approval before sensitive writes.
570. Submit through Gateway, wait for commit, log txId/correlationId, and make retries safe.

Implementation Example

```
# Human Approval Service
INPUT  -> authenticate, validate schema, bind requestId/nonce
FACTS  -> read authoritative wallet/merchant/state data
AI     -> optional bounded recommendation with decisionId/version
POLICY -> enforce ownership, hard limits, approvals, risk fallback
LEDGER -> submit one idempotent deterministic transaction when value changes
EVENT  -> update projection/case/monitoring asynchronously
VERIFY -> discover committed result and retain correlationId + txId evidence
```

Verification Checklist

- The approval api exists and is version-controlled.
- The feature works with all public Internet/WAN connectivity disabled.
- Authorization, malformed input, duplicate retry, and failure behavior have explicit tests.
- If financial value changes, journal debits and credits balance exactly and historical entries remain immutable.
- If AI is involved, model/prompt/feature version and fallback behavior are recorded and deterministic policy remains final.
- A timeout/restart cannot create a duplicate financial result; request status can be rediscovered.
- Logs/evidence contain correlation references but not secrets or prohibited sensitive payment data.

Common Failure Modes

- Returning success before commit status.
- Retry after timeout creates duplicate transfer.
- AI recommendation treated as authorization.
- Secrets/PAN/credentials logged.

Expert Review Questions

- What happens if this component succeeds but the next component fails?
- How is the authorizing identity/device/key revoked or recovered?
- Can the operation be safely retried after the client loses the response?
- What happens if AI is unavailable, slow, or wrong?
- Can an auditor reconstruct debit, credit, actor, risk decision, approval, and Fabric txId from retained evidence?
- Which compensating transaction resolves a downstream operational error without rewriting history?

Chapter 115 - Wallet REST API

Expose create/read/status/balance/statement operations safely. The API layer authenticates users, validates request schemas, gathers authoritative facts, calls local AI when useful, applies deterministic policy, submits transactions through Fabric Gateway, waits for commit status, and exposes stable application errors. It must safely recover from lost HTTP responses and duplicate retries.

Design rule: Create controlled by role

Implementation Objective

The required output is: Wallet endpoints. It must be version-controlled, reviewable, testable with the WAN disabled, and linked to the architecture/source-of-truth model. For a payment platform, every implementation decision should make it possible to explain who authorized an action, which facts were trusted, which controls ran, what financial journal committed, and how the result is recovered after failure.

Key Engineering Decisions

- Create controlled by role
- Balance reads from projection with optional ledger verification
- Freeze/unfreeze requires reason and elevated permission

Deep Implementation Guidance

Treat the backend as a policy enforcement point, not a bypass around chaincode. A request may be lost after ledger submission, so recovery by requestId is mandatory. Never respond “failed” merely because the HTTP connection failed; discover the committed result first.

Data and Control Questions

- What is the authoritative source for every input used by this component?
- Which user, device, service, MSP, or administrator identity is allowed to initiate or approve it?
- Does the operation move value, reserve value, create a side effect, or only provide advisory information?
- What idempotency/replay key proves that a retry is the same business intent?
- What evidence is retained off-chain and what stable reference/hash belongs on the ledger?

Chapter 115 Implementation - Wallet REST API

Implementation Procedure

571. Validate transport/schema/auth before calling Fabric or AI.
572. Read authoritative facts from ledger/projection according to the source-of-truth matrix.
573. Call local AI with bounded inputs and require structured output.
574. Apply deterministic policy/human approval before sensitive writes.
575. Submit through Gateway, wait for commit, log txId/correlationId, and make retries safe.

Implementation Example

```
# Wallet REST API
INPUT  -> authenticate, validate schema, bind requestId/nonce
FACTS  -> read authoritative wallet/merchant/state data
AI      -> optional bounded recommendation with decisionId/version
POLICY  -> enforce ownership, hard limits, approvals, risk fallback
LEDGER  -> submit one idempotent deterministic transaction when value changes
EVENT   -> update projection/case/monitoring asynchronously
VERIFY  -> discover committed result and retain correlationId + txId evidence
```

Verification Checklist

- The wallet endpoints exists and is version-controlled.
- The feature works with all public Internet/WAN connectivity disabled.
- Authorization, malformed input, duplicate retry, and failure behavior have explicit tests.
- If financial value changes, journal debits and credits balance exactly and historical entries remain immutable.
- If AI is involved, model/prompt/feature version and fallback behavior are recorded and deterministic policy remains final.
- A timeout/restart cannot create a duplicate financial result; request status can be rediscovered.
- Logs/evidence contain correlation references but not secrets or prohibited sensitive payment data.

Common Failure Modes

- Returning success before commit status.
- Retry after timeout creates duplicate transfer.
- AI recommendation treated as authorization.
- Secrets/PAN/credentials logged.

Expert Review Questions

- What happens if this component succeeds but the next component fails?
- How is the authorizing identity/device/key revoked or recovered?
- Can the operation be safely retried after the client loses the response?
- What happens if AI is unavailable, slow, or wrong?
- Can an auditor reconstruct debit, credit, actor, risk decision, approval, and Fabric txId from retained evidence?
- Which compensating transaction resolves a downstream operational error without rewriting history?

Chapter 116 - Transfer REST API

Implement retry-safe P2P transfer. The API layer authenticates users, validates request schemas, gathers authoritative facts, calls local AI when useful, applies deterministic policy, submits transactions through Fabric Gateway, waits for commit status, and exposes stable application errors. It must safely recover from lost HTTP responses and duplicate retries.

Design rule: Require Idempotency-Key

Implementation Objective

The required output is: Transfer endpoint. It must be version-controlled, reviewable, testable with the WAN disabled, and linked to the architecture/source-of-truth model. For a payment platform, every implementation decision should make it possible to explain who authorized an action, which facts were trusted, which controls ran, what financial journal committed, and how the result is recovered after failure.

Key Engineering Decisions

- Require Idempotency-Key
- authenticate/authorize
- run risk/policy
- submit Transfer
- wait for commit
- return stable transaction result

Deep Implementation Guidance

Treat the backend as a policy enforcement point, not a bypass around chaincode. A request may be lost after ledger submission, so recovery by requestId is mandatory. Never respond “failed” merely because the HTTP connection failed; discover the committed result first.

Data and Control Questions

- What is the authoritative source for every input used by this component?
- Which user, device, service, MSP, or administrator identity is allowed to initiate or approve it?
- Does the operation move value, reserve value, create a side effect, or only provide advisory information?
- What idempotency/replay key proves that a retry is the same business intent?
- What evidence is retained off-chain and what stable reference/hash belongs on the ledger?

Chapter 116 Implementation - Transfer REST API

Implementation Procedure

576. Validate transport/schema/auth before calling Fabric or AI.
577. Read authoritative facts from ledger/projection according to the source-of-truth matrix.
578. Call local AI with bounded inputs and require structured output.
579. Apply deterministic policy/human approval before sensitive writes.
580. Submit through Gateway, wait for commit, log txId/correlationId, and make retries safe.

Implementation Example

```
# Transfer REST API
INPUT  -> authenticate, validate schema, bind requestId/nonce
FACTS  -> read authoritative wallet/merchant/state data
AI     -> optional bounded recommendation with decisionId/version
POLICY -> enforce ownership, hard limits, approvals, risk fallback
LEDGER -> submit one idempotent deterministic transaction when value changes
EVENT  -> update projection/case/monitoring asynchronously
VERIFY -> discover committed result and retain correlationId + txId evidence
```

Verification Checklist

- The transfer endpoint exists and is version-controlled.
- The feature works with all public Internet/WAN connectivity disabled.
- Authorization, malformed input, duplicate retry, and failure behavior have explicit tests.
- If financial value changes, journal debits and credits balance exactly and historical entries remain immutable.
- If AI is involved, model/prompt/feature version and fallback behavior are recorded and deterministic policy remains final.
- A timeout/restart cannot create a duplicate financial result; request status can be rediscovered.
- Logs/evidence contain correlation references but not secrets or prohibited sensitive payment data.

Common Failure Modes

- Returning success before commit status.
- Retry after timeout creates duplicate transfer.
- AI recommendation treated as authorization.
- Secrets/PAN/credentials logged.

Expert Review Questions

- What happens if this component succeeds but the next component fails?
- How is the authorizing identity/device/key revoked or recovered?
- Can the operation be safely retried after the client loses the response?
- What happens if AI is unavailable, slow, or wrong?
- Can an auditor reconstruct debit, credit, actor, risk decision, approval, and Fabric txId from retained evidence?
- Which compensating transaction resolves a downstream operational error without rewriting history?

Chapter 117 - Merchant Payment REST API

Implement QR/order-based merchant payment with local risk gate. The API layer authenticates users, validates request schemas, gathers authoritative facts, calls local AI when useful, applies deterministic policy, submits transactions through Fabric Gateway, waits for commit status, and exposes stable application errors. It must safely recover from lost HTTP responses and duplicate retries.

Design rule: Validate merchant/store and invoice/order reference

Implementation Objective

The required output is: Payment endpoint. It must be version-controlled, reviewable, testable with the WAN disabled, and linked to the architecture/source-of-truth model. For a payment platform, every implementation decision should make it possible to explain who authorized an action, which facts were trusted, which controls ran, what financial journal committed, and how the result is recovered after failure.

Key Engineering Decisions

- Validate merchant/store and invoice/order reference
- server owns amount validation when possible
- AI risk is advisory
- commit before success

Deep Implementation Guidance

Treat the backend as a policy enforcement point, not a bypass around chaincode. A request may be lost after ledger submission, so recovery by requestId is mandatory. Never respond “failed” merely because the HTTP connection failed; discover the committed result first.

Data and Control Questions

- What is the authoritative source for every input used by this component?
- Which user, device, service, MSP, or administrator identity is allowed to initiate or approve it?
- Does the operation move value, reserve value, create a side effect, or only provide advisory information?
- What idempotency/replay key proves that a retry is the same business intent?
- What evidence is retained off-chain and what stable reference/hash belongs on the ledger?

Chapter 117 Implementation - Merchant Payment REST API

Implementation Procedure

581. Validate transport/schema/auth before calling Fabric or AI.
582. Read authoritative facts from ledger/projection according to the source-of-truth matrix.
583. Call local AI with bounded inputs and require structured output.
584. Apply deterministic policy/human approval before sensitive writes.
585. Submit through Gateway, wait for commit, log txId/correlationId, and make retries safe.

Implementation Example

```
# Merchant Payment REST API
INPUT  -> authenticate, validate schema, bind requestId/nonce
FACTS  -> read authoritative wallet/merchant/state data
AI      -> optional bounded recommendation with decisionId/version
POLICY  -> enforce ownership, hard limits, approvals, risk fallback
LEDGER  -> submit one idempotent deterministic transaction when value changes
EVENT   -> update projection/case/monitoring asynchronously
VERIFY  -> discover committed result and retain correlationId + txId evidence
```

Verification Checklist

- The payment endpoint exists and is version-controlled.
- The feature works with all public Internet/WAN connectivity disabled.
- Authorization, malformed input, duplicate retry, and failure behavior have explicit tests.
- If financial value changes, journal debits and credits balance exactly and historical entries remain immutable.
- If AI is involved, model/prompt/feature version and fallback behavior are recorded and deterministic policy remains final.
- A timeout/restart cannot create a duplicate financial result; request status can be rediscovered.
- Logs/evidence contain correlation references but not secrets or prohibited sensitive payment data.

Common Failure Modes

- Returning success before commit status.
- Retry after timeout creates duplicate transfer.
- AI recommendation treated as authorization.
- Secrets/PAN/credentials logged.

Expert Review Questions

- What happens if this component succeeds but the next component fails?
- How is the authorizing identity/device/key revoked or recovered?
- Can the operation be safely retried after the client loses the response?
- What happens if AI is unavailable, slow, or wrong?
- Can an auditor reconstruct debit, credit, actor, risk decision, approval, and Fabric txId from retained evidence?
- Which compensating transaction resolves a downstream operational error without rewriting history?

Chapter 118 - Hold/Capture/Release REST API

Expose authorization-hold workflows for POS and reservations. The API layer authenticates users, validates request schemas, gathers authoritative facts, calls local AI when useful, applies deterministic policy, submits transactions through Fabric Gateway, waits for commit status, and exposes stable application errors. It must safely recover from lost HTTP responses and duplicate retries.

Design rule: State transitions validated server and chaincode

Implementation Objective

The required output is: Hold endpoints. It must be version-controlled, reviewable, testable with the WAN disabled, and linked to the architecture/source-of-truth model. For a payment platform, every implementation decision should make it possible to explain who authorized an action, which facts were trusted, which controls ran, what financial journal committed, and how the result is recovered after failure.

Key Engineering Decisions

- State transitions validated server and chaincode
- capture amount bounded by remaining hold
- release idempotent

Deep Implementation Guidance

Treat the backend as a policy enforcement point, not a bypass around chaincode. A request may be lost after ledger submission, so recovery by requestId is mandatory. Never respond “failed” merely because the HTTP connection failed; discover the committed result first.

Data and Control Questions

- What is the authoritative source for every input used by this component?
- Which user, device, service, MSP, or administrator identity is allowed to initiate or approve it?
- Does the operation move value, reserve value, create a side effect, or only provide advisory information?
- What idempotency/replay key proves that a retry is the same business intent?
- What evidence is retained off-chain and what stable reference/hash belongs on the ledger?

Chapter 118 Implementation - Hold/Capture/Release REST API

Implementation Procedure

586. Validate transport/schema/auth before calling Fabric or AI.
587. Read authoritative facts from ledger/projection according to the source-of-truth matrix.
588. Call local AI with bounded inputs and require structured output.
589. Apply deterministic policy/human approval before sensitive writes.
590. Submit through Gateway, wait for commit, log txId/correlationId, and make retries safe.

Implementation Example

```
# Hold/Capture/Release REST API
INPUT  -> authenticate, validate schema, bind requestId/nonce
FACTS  -> read authoritative wallet/merchant/state data
AI      -> optional bounded recommendation with decisionId/version
POLICY  -> enforce ownership, hard limits, approvals, risk fallback
LEDGER  -> submit one idempotent deterministic transaction when value changes
EVENT   -> update projection/case/monitoring asynchronously
VERIFY  -> discover committed result and retain correlationId + txId evidence
```

Verification Checklist

- The hold endpoints exists and is version-controlled.
- The feature works with all public Internet/WAN connectivity disabled.
- Authorization, malformed input, duplicate retry, and failure behavior have explicit tests.
- If financial value changes, journal debits and credits balance exactly and historical entries remain immutable.
- If AI is involved, model/prompt/feature version and fallback behavior are recorded and deterministic policy remains final.
- A timeout/restart cannot create a duplicate financial result; request status can be rediscovered.
- Logs/evidence contain correlation references but not secrets or prohibited sensitive payment data.

Common Failure Modes

- Returning success before commit status.
- Retry after timeout creates duplicate transfer.
- AI recommendation treated as authorization.
- Secrets/PAN/credentials logged.

Expert Review Questions

- What happens if this component succeeds but the next component fails?
- How is the authorizing identity/device/key revoked or recovered?
- Can the operation be safely retried after the client loses the response?
- What happens if AI is unavailable, slow, or wrong?
- Can an auditor reconstruct debit, credit, actor, risk decision, approval, and Fabric txId from retained evidence?
- Which compensating transaction resolves a downstream operational error without rewriting history?

Chapter 119 - Refund REST API

Implement controlled refund initiation and approval. The API layer authenticates users, validates request schemas, gathers authoritative facts, calls local AI when useful, applies deterministic policy, submits transactions through Fabric Gateway, waits for commit status, and exposes stable application errors. It must safely recover from lost HTTP responses and duplicate retries.

Design rule: Verify original payment and refundable amount

Implementation Objective

The required output is: Refund endpoint. It must be version-controlled, reviewable, testable with the WAN disabled, and linked to the architecture/source-of-truth model. For a payment platform, every implementation decision should make it possible to explain who authorized an action, which facts were trusted, which controls ran, what financial journal committed, and how the result is recovered after failure.

Key Engineering Decisions

- Verify original payment and refundable amount
- apply risk/approval thresholds
- submit immutable refund
- link case/reference

Deep Implementation Guidance

Treat the backend as a policy enforcement point, not a bypass around chaincode. A request may be lost after ledger submission, so recovery by requestId is mandatory. Never respond “failed” merely because the HTTP connection failed; discover the committed result first.

Data and Control Questions

- What is the authoritative source for every input used by this component?
- Which user, device, service, MSP, or administrator identity is allowed to initiate or approve it?
- Does the operation move value, reserve value, create a side effect, or only provide advisory information?
- What idempotency/replay key proves that a retry is the same business intent?
- What evidence is retained off-chain and what stable reference/hash belongs on the ledger?

Chapter 119 Implementation - Refund REST API

Implementation Procedure

591. Validate transport/schema/auth before calling Fabric or AI.
592. Read authoritative facts from ledger/projection according to the source-of-truth matrix.
593. Call local AI with bounded inputs and require structured output.
594. Apply deterministic policy/human approval before sensitive writes.
595. Submit through Gateway, wait for commit, log txId/correlationId, and make retries safe.

Implementation Example

```
# Refund REST API
INPUT  -> authenticate, validate schema, bind requestId/nonce
FACTS  -> read authoritative wallet/merchant/state data
AI     -> optional bounded recommendation with decisionId/version
POLICY -> enforce ownership, hard limits, approvals, risk fallback
LEDGER -> submit one idempotent deterministic transaction when value changes
EVENT  -> update projection/case/monitoring asynchronously
VERIFY -> discover committed result and retain correlationId + txId evidence
```

Verification Checklist

- The refund endpoint exists and is version-controlled.
- The feature works with all public Internet/WAN connectivity disabled.
- Authorization, malformed input, duplicate retry, and failure behavior have explicit tests.
- If financial value changes, journal debits and credits balance exactly and historical entries remain immutable.
- If AI is involved, model/prompt/feature version and fallback behavior are recorded and deterministic policy remains final.
- A timeout/restart cannot create a duplicate financial result; request status can be rediscovered.
- Logs/evidence contain correlation references but not secrets or prohibited sensitive payment data.

Common Failure Modes

- Returning success before commit status.
- Retry after timeout creates duplicate transfer.
- AI recommendation treated as authorization.
- Secrets/PAN/credentials logged.

Expert Review Questions

- What happens if this component succeeds but the next component fails?
- How is the authorizing identity/device/key revoked or recovered?
- Can the operation be safely retried after the client loses the response?
- What happens if AI is unavailable, slow, or wrong?
- Can an auditor reconstruct debit, credit, actor, risk decision, approval, and Fabric txId from retained evidence?
- Which compensating transaction resolves a downstream operational error without rewriting history?

Chapter 120 - Issue and Redeem REST API

Protect treasury creation/removal of value. The API layer authenticates users, validates request schemas, gathers authoritative facts, calls local AI when useful, applies deterministic policy, submits transactions through Fabric Gateway, waits for commit status, and exposes stable application errors. It must safely recover from lost HTTP responses and duplicate retries.

Design rule: Privileged auth and MFA

Implementation Objective

The required output is: Treasury endpoints. It must be version-controlled, reviewable, testable with the WAN disabled, and linked to the architecture/source-of-truth model. For a payment platform, every implementation decision should make it possible to explain who authorized an action, which facts were trusted, which controls ran, what financial journal committed, and how the result is recovered after failure.

Key Engineering Decisions

- Privileged auth and MFA
- dual approval over threshold
- never expose treasury Fabric key to UI
- reconciliation required

Deep Implementation Guidance

Treat the backend as a policy enforcement point, not a bypass around chaincode. A request may be lost after ledger submission, so recovery by requestId is mandatory. Never respond “failed” merely because the HTTP connection failed; discover the committed result first.

Data and Control Questions

- What is the authoritative source for every input used by this component?
- Which user, device, service, MSP, or administrator identity is allowed to initiate or approve it?
- Does the operation move value, reserve value, create a side effect, or only provide advisory information?
- What idempotency/replay key proves that a retry is the same business intent?
- What evidence is retained off-chain and what stable reference/hash belongs on the ledger?

Chapter 120 Implementation - Issue and Redeem REST API

Implementation Procedure

596. Validate transport/schema/auth before calling Fabric or AI.
597. Read authoritative facts from ledger/projection according to the source-of-truth matrix.
598. Call local AI with bounded inputs and require structured output.
599. Apply deterministic policy/human approval before sensitive writes.
600. Submit through Gateway, wait for commit, log txId/correlationId, and make retries safe.

Implementation Example

```
# Issue and Redeem REST API
INPUT  -> authenticate, validate schema, bind requestId/nonce
FACTS  -> read authoritative wallet/merchant/state data
AI     -> optional bounded recommendation with decisionId/version
POLICY -> enforce ownership, hard limits, approvals, risk fallback
LEDGER -> submit one idempotent deterministic transaction when value changes
EVENT  -> update projection/case/monitoring asynchronously
VERIFY -> discover committed result and retain correlationId + txId evidence
```

Verification Checklist

- The treasury endpoints exists and is version-controlled.
- The feature works with all public Internet/WAN connectivity disabled.
- Authorization, malformed input, duplicate retry, and failure behavior have explicit tests.
- If financial value changes, journal debits and credits balance exactly and historical entries remain immutable.
- If AI is involved, model/prompt/feature version and fallback behavior are recorded and deterministic policy remains final.
- A timeout/restart cannot create a duplicate financial result; request status can be rediscovered.
- Logs/evidence contain correlation references but not secrets or prohibited sensitive payment data.

Common Failure Modes

- Returning success before commit status.
- Retry after timeout creates duplicate transfer.
- AI recommendation treated as authorization.
- Secrets/PAN/credentials logged.

Expert Review Questions

- What happens if this component succeeds but the next component fails?
- How is the authorizing identity/device/key revoked or recovered?
- Can the operation be safely retried after the client loses the response?
- What happens if AI is unavailable, slow, or wrong?
- Can an auditor reconstruct debit, credit, actor, risk decision, approval, and Fabric txId from retained evidence?
- Which compensating transaction resolves a downstream operational error without rewriting history?

Chapter 121 - Transaction Status and Recovery API

Resolve uncertainty after client timeout/reconnect. The API layer authenticates users, validates request schemas, gathers authoritative facts, calls local AI when useful, applies deterministic policy, submits transactions through Fabric Gateway, waits for commit status, and exposes stable application errors. It must safely recover from lost HTTP responses and duplicate retries.

Design rule: Lookup by requestId/business reference

Implementation Objective

The required output is: Recovery endpoint. It must be version-controlled, reviewable, testable with the WAN disabled, and linked to the architecture/source-of-truth model. For a payment platform, every implementation decision should make it possible to explain who authorized an action, which facts were trusted, which controls ran, what financial journal committed, and how the result is recovered after failure.

Key Engineering Decisions

- Lookup by requestId/business reference
- return committed/rejected/pending/unknown
- prevent client from creating a second intent unnecessarily

Deep Implementation Guidance

Treat the backend as a policy enforcement point, not a bypass around chaincode. A request may be lost after ledger submission, so recovery by requestId is mandatory. Never respond “failed” merely because the HTTP connection failed; discover the committed result first.

Data and Control Questions

- What is the authoritative source for every input used by this component?
- Which user, device, service, MSP, or administrator identity is allowed to initiate or approve it?
- Does the operation move value, reserve value, create a side effect, or only provide advisory information?
- What idempotency/replay key proves that a retry is the same business intent?
- What evidence is retained off-chain and what stable reference/hash belongs on the ledger?

Chapter 121 Implementation - Transaction Status and Recovery API

Implementation Procedure

601. Validate transport/schema/auth before calling Fabric or AI.
602. Read authoritative facts from ledger/projection according to the source-of-truth matrix.
603. Call local AI with bounded inputs and require structured output.
604. Apply deterministic policy/human approval before sensitive writes.
605. Submit through Gateway, wait for commit, log txId/correlationId, and make retries safe.

Implementation Example

```
# Transaction Status and Recovery API
INPUT  -> authenticate, validate schema, bind requestId/nonce
FACTS  -> read authoritative wallet/merchant/state data
AI     -> optional bounded recommendation with decisionId/version
POLICY -> enforce ownership, hard limits, approvals, risk fallback
LEDGER -> submit one idempotent deterministic transaction when value changes
EVENT  -> update projection/case/monitoring asynchronously
VERIFY -> discover committed result and retain correlationId + txId evidence
```

Verification Checklist

- The recovery endpoint exists and is version-controlled.
- The feature works with all public Internet/WAN connectivity disabled.
- Authorization, malformed input, duplicate retry, and failure behavior have explicit tests.
- If financial value changes, journal debits and credits balance exactly and historical entries remain immutable.
- If AI is involved, model/prompt/feature version and fallback behavior are recorded and deterministic policy remains final.
- A timeout/restart cannot create a duplicate financial result; request status can be rediscovered.
- Logs/evidence contain correlation references but not secrets or prohibited sensitive payment data.

Common Failure Modes

- Returning success before commit status.
- Retry after timeout creates duplicate transfer.
- AI recommendation treated as authorization.
- Secrets/PAN/credentials logged.

Expert Review Questions

- What happens if this component succeeds but the next component fails?
- How is the authorizing identity/device/key revoked or recovered?
- Can the operation be safely retried after the client loses the response?
- What happens if AI is unavailable, slow, or wrong?
- Can an auditor reconstruct debit, credit, actor, risk decision, approval, and Fabric txId from retained evidence?
- Which compensating transaction resolves a downstream operational error without rewriting history?

Chapter 122 - PostgreSQL Projection Database

Maintain searchable current wallet/payment/merchant views from committed events. The API layer authenticates users, validates request schemas, gathers authoritative facts, calls local AI when useful, applies deterministic policy, submits transactions through Fabric Gateway, waits for commit status, and exposes stable application errors. It must safely recover from lost HTTP responses and duplicate retries.

Design rule: Store ledger tx/block references

Implementation Objective

The required output is: Projection schema. It must be version-controlled, reviewable, testable with the WAN disabled, and linked to the architecture/source-of-truth model. For a payment platform, every implementation decision should make it possible to explain who authorized an action, which facts were trusted, which controls ran, what financial journal committed, and how the result is recovered after failure.

Key Engineering Decisions

- Store ledger tx/block references
- tables are rebuildable
- do not treat projection write as the financial commit

Deep Implementation Guidance

Treat the backend as a policy enforcement point, not a bypass around chaincode. A request may be lost after ledger submission, so recovery by requestId is mandatory. Never respond “failed” merely because the HTTP connection failed; discover the committed result first.

Data and Control Questions

- What is the authoritative source for every input used by this component?
- Which user, device, service, MSP, or administrator identity is allowed to initiate or approve it?
- Does the operation move value, reserve value, create a side effect, or only provide advisory information?
- What idempotency/replay key proves that a retry is the same business intent?
- What evidence is retained off-chain and what stable reference/hash belongs on the ledger?

Chapter 122 Implementation - PostgreSQL Projection Database

Implementation Procedure

606. Validate transport/schema/auth before calling Fabric or AI.
607. Read authoritative facts from ledger/projection according to the source-of-truth matrix.
608. Call local AI with bounded inputs and require structured output.
609. Apply deterministic policy/human approval before sensitive writes.
610. Submit through Gateway, wait for commit, log txId/correlationId, and make retries safe.

Implementation Example

```
# PostgreSQL Projection Database
INPUT  -> authenticate, validate schema, bind requestId/nonce
FACTS  -> read authoritative wallet/merchant/state data
AI      -> optional bounded recommendation with decisionId/version
POLICY  -> enforce ownership, hard limits, approvals, risk fallback
LEDGER  -> submit one idempotent deterministic transaction when value changes
EVENT   -> update projection/case/monitoring asynchronously
VERIFY  -> discover committed result and retain correlationId + txId evidence
```

Verification Checklist

- The projection schema exists and is version-controlled.
- The feature works with all public Internet/WAN connectivity disabled.
- Authorization, malformed input, duplicate retry, and failure behavior have explicit tests.
- If financial value changes, journal debits and credits balance exactly and historical entries remain immutable.
- If AI is involved, model/prompt/feature version and fallback behavior are recorded and deterministic policy remains final.
- A timeout/restart cannot create a duplicate financial result; request status can be rediscovered.
- Logs/evidence contain correlation references but not secrets or prohibited sensitive payment data.

Common Failure Modes

- Returning success before commit status.
- Retry after timeout creates duplicate transfer.
- AI recommendation treated as authorization.
- Secrets/PAN/credentials logged.

Expert Review Questions

- What happens if this component succeeds but the next component fails?
- How is the authorizing identity/device/key revoked or recovered?
- Can the operation be safely retried after the client loses the response?
- What happens if AI is unavailable, slow, or wrong?
- Can an auditor reconstruct debit, credit, actor, risk decision, approval, and Fabric txId from retained evidence?
- Which compensating transaction resolves a downstream operational error without rewriting history?

Chapter 123 - Fabric Event Listener

Consume chaincode events and update projections idempotently. The API layer authenticates users, validates request schemas, gathers authoritative facts, calls local AI when useful, applies deterministic policy, submits transactions through Fabric Gateway, waits for commit status, and exposes stable application errors. It must safely recover from lost HTTP responses and duplicate retries.

Design rule: Checkpoint block position

Implementation Objective

The required output is: Projection worker. It must be version-controlled, reviewable, testable with the WAN disabled, and linked to the architecture/source-of-truth model. For a payment platform, every implementation decision should make it possible to explain who authorized an action, which facts were trusted, which controls ran, what financial journal committed, and how the result is recovered after failure.

Key Engineering Decisions

- Checkpoint block position
- deduplicate txId/event
- retry DB failures
- projection outage never rolls back ledger

Deep Implementation Guidance

Treat the backend as a policy enforcement point, not a bypass around chaincode. A request may be lost after ledger submission, so recovery by requestId is mandatory. Never respond “failed” merely because the HTTP connection failed; discover the committed result first.

Data and Control Questions

- What is the authoritative source for every input used by this component?
- Which user, device, service, MSP, or administrator identity is allowed to initiate or approve it?
- Does the operation move value, reserve value, create a side effect, or only provide advisory information?
- What idempotency/replay key proves that a retry is the same business intent?
- What evidence is retained off-chain and what stable reference/hash belongs on the ledger?

Chapter 123 Implementation - Fabric Event Listener

Implementation Procedure

611. Validate transport/schema/auth before calling Fabric or AI.
612. Read authoritative facts from ledger/projection according to the source-of-truth matrix.
613. Call local AI with bounded inputs and require structured output.
614. Apply deterministic policy/human approval before sensitive writes.
615. Submit through Gateway, wait for commit, log txId/correlationId, and make retries safe.

Implementation Example

```
# Fabric Event Listener
INPUT  -> authenticate, validate schema, bind requestId/nonce
FACTS  -> read authoritative wallet/merchant/state data
AI     -> optional bounded recommendation with decisionId/version
POLICY -> enforce ownership, hard limits, approvals, risk fallback
LEDGER -> submit one idempotent deterministic transaction when value changes
EVENT  -> update projection/case/monitoring asynchronously
VERIFY -> discover committed result and retain correlationId + txId evidence
```

Verification Checklist

- The projection worker exists and is version-controlled.
- The feature works with all public Internet/WAN connectivity disabled.
- Authorization, malformed input, duplicate retry, and failure behavior have explicit tests.
- If financial value changes, journal debits and credits balance exactly and historical entries remain immutable.
- If AI is involved, model/prompt/feature version and fallback behavior are recorded and deterministic policy remains final.
- A timeout/restart cannot create a duplicate financial result; request status can be rediscovered.
- Logs/evidence contain correlation references but not secrets or prohibited sensitive payment data.

Common Failure Modes

- Returning success before commit status.
- Retry after timeout creates duplicate transfer.
- AI recommendation treated as authorization.
- Secrets/PAN/credentials logged.

Expert Review Questions

- What happens if this component succeeds but the next component fails?
- How is the authorizing identity/device/key revoked or recovered?
- Can the operation be safely retried after the client loses the response?
- What happens if AI is unavailable, slow, or wrong?
- Can an auditor reconstruct debit, credit, actor, risk decision, approval, and Fabric txId from retained evidence?
- Which compensating transaction resolves a downstream operational error without rewriting history?

Chapter 124 - Outbox and Local Job Queue

Handle notifications, reports, AI jobs, and reconciliation without coupling them to ledger commit. The API layer authenticates users, validates request schemas, gathers authoritative facts, calls local AI when useful, applies deterministic policy, submits transactions through Fabric Gateway, waits for commit status, and exposes stable application errors. It must safely recover from lost HTTP responses and duplicate retries.

Design rule: Persist job intent locally

Implementation Objective

The required output is: Job/outbox design. It must be version-controlled, reviewable, testable with the WAN disabled, and linked to the architecture/source-of-truth model. For a payment platform, every implementation decision should make it possible to explain who authorized an action, which facts were trusted, which controls ran, what financial journal committed, and how the result is recovered after failure.

Key Engineering Decisions

- Persist job intent locally
- idempotent workers
- bounded retries/dead-letter state
- no public SaaS dependency

Deep Implementation Guidance

Treat the backend as a policy enforcement point, not a bypass around chaincode. A request may be lost after ledger submission, so recovery by requestId is mandatory. Never respond “failed” merely because the HTTP connection failed; discover the committed result first.

Data and Control Questions

- What is the authoritative source for every input used by this component?
- Which user, device, service, MSP, or administrator identity is allowed to initiate or approve it?
- Does the operation move value, reserve value, create a side effect, or only provide advisory information?
- What idempotency/replay key proves that a retry is the same business intent?
- What evidence is retained off-chain and what stable reference/hash belongs on the ledger?

Chapter 124 Implementation - Outbox and Local Job Queue

Implementation Procedure

616. Validate transport/schema/auth before calling Fabric or AI.
617. Read authoritative facts from ledger/projection according to the source-of-truth matrix.
618. Call local AI with bounded inputs and require structured output.
619. Apply deterministic policy/human approval before sensitive writes.
620. Submit through Gateway, wait for commit, log txId/correlationId, and make retries safe.

Implementation Example

```
# Outbox and Local Job Queue
INPUT  -> authenticate, validate schema, bind requestId/nonce
FACTS  -> read authoritative wallet/merchant/state data
AI     -> optional bounded recommendation with decisionId/version
POLICY -> enforce ownership, hard limits, approvals, risk fallback
LEDGER -> submit one idempotent deterministic transaction when value changes
EVENT  -> update projection/case/monitoring asynchronously
VERIFY -> discover committed result and retain correlationId + txId evidence
```

Verification Checklist

- The job/outbox design exists and is version-controlled.
- The feature works with all public Internet/WAN connectivity disabled.
- Authorization, malformed input, duplicate retry, and failure behavior have explicit tests.
- If financial value changes, journal debits and credits balance exactly and historical entries remain immutable.
- If AI is involved, model/prompt/feature version and fallback behavior are recorded and deterministic policy remains final.
- A timeout/restart cannot create a duplicate financial result; request status can be rediscovered.
- Logs/evidence contain correlation references but not secrets or prohibited sensitive payment data.

Common Failure Modes

- Returning success before commit status.
- Retry after timeout creates duplicate transfer.
- AI recommendation treated as authorization.
- Secrets/PAN/credentials logged.

Expert Review Questions

- What happens if this component succeeds but the next component fails?
- How is the authorizing identity/device/key revoked or recovered?
- Can the operation be safely retried after the client loses the response?
- What happens if AI is unavailable, slow, or wrong?
- Can an auditor reconstruct debit, credit, actor, risk decision, approval, and Fabric txId from retained evidence?
- Which compensating transaction resolves a downstream operational error without rewriting history?

Chapter 125 - Merchant and Wallet Notifications

Generate local UI/intranet/SMS-gateway notifications without changing financial truth. The API layer authenticates users, validates request schemas, gathers authoritative facts, calls local AI when useful, applies deterministic policy, submits transactions through Fabric Gateway, waits for commit status, and exposes stable application errors. It must safely recover from lost HTTP responses and duplicate retries.

Design rule: Notification failure is non-financial

Implementation Objective

The required output is: Notification service. It must be version-controlled, reviewable, testable with the WAN disabled, and linked to the architecture/source-of-truth model. For a payment platform, every implementation decision should make it possible to explain who authorized an action, which facts were trusted, which controls ran, what financial journal committed, and how the result is recovered after failure.

Key Engineering Decisions

- Notification failure is non-financial
- include safe reference not secret data
- allow replay from event/job record

Deep Implementation Guidance

Treat the backend as a policy enforcement point, not a bypass around chaincode. A request may be lost after ledger submission, so recovery by requestId is mandatory. Never respond “failed” merely because the HTTP connection failed; discover the committed result first.

Data and Control Questions

- What is the authoritative source for every input used by this component?
- Which user, device, service, MSP, or administrator identity is allowed to initiate or approve it?
- Does the operation move value, reserve value, create a side effect, or only provide advisory information?
- What idempotency/replay key proves that a retry is the same business intent?
- What evidence is retained off-chain and what stable reference/hash belongs on the ledger?

Chapter 125 Implementation - Merchant and Wallet Notifications

Implementation Procedure

621. Validate transport/schema/auth before calling Fabric or AI.
622. Read authoritative facts from ledger/projection according to the source-of-truth matrix.
623. Call local AI with bounded inputs and require structured output.
624. Apply deterministic policy/human approval before sensitive writes.
625. Submit through Gateway, wait for commit, log txId/correlationId, and make retries safe.

Implementation Example

```
# Merchant and Wallet Notifications
INPUT  -> authenticate, validate schema, bind requestId/nonce
FACTS  -> read authoritative wallet/merchant/state data
AI     -> optional bounded recommendation with decisionId/version
POLICY -> enforce ownership, hard limits, approvals, risk fallback
LEDGER -> submit one idempotent deterministic transaction when value changes
EVENT  -> update projection/case/monitoring asynchronously
VERIFY -> discover committed result and retain correlationId + txId evidence
```

Verification Checklist

- The notification service exists and is version-controlled.
- The feature works with all public Internet/WAN connectivity disabled.
- Authorization, malformed input, duplicate retry, and failure behavior have explicit tests.
- If financial value changes, journal debits and credits balance exactly and historical entries remain immutable.
- If AI is involved, model/prompt/feature version and fallback behavior are recorded and deterministic policy remains final.
- A timeout/restart cannot create a duplicate financial result; request status can be rediscovered.
- Logs/evidence contain correlation references but not secrets or prohibited sensitive payment data.

Common Failure Modes

- Returning success before commit status.
- Retry after timeout creates duplicate transfer.
- AI recommendation treated as authorization.
- Secrets/PAN/credentials logged.

Expert Review Questions

- What happens if this component succeeds but the next component fails?
- How is the authorizing identity/device/key revoked or recovered?
- Can the operation be safely retried after the client loses the response?
- What happens if AI is unavailable, slow, or wrong?
- Can an auditor reconstruct debit, credit, actor, risk decision, approval, and Fabric txId from retained evidence?
- Which compensating transaction resolves a downstream operational error without rewriting history?

Chapter 126 - Error Mapping

Translate Fabric/AI/DB/business failures into stable API responses. The API layer authenticates users, validates request schemas, gathers authoritative facts, calls local AI when useful, applies deterministic policy, submits transactions through Fabric Gateway, waits for commit status, and exposes stable application errors. It must safely recover from lost HTTP responses and duplicate retries.

Design rule: Differentiate rejected vs pending/unknown

Implementation Objective

The required output is: Error catalog. It must be version-controlled, reviewable, testable with the WAN disabled, and linked to the architecture/source-of-truth model. For a payment platform, every implementation decision should make it possible to explain who authorized an action, which facts were trusted, which controls ran, what financial journal committed, and how the result is recovered after failure.

Key Engineering Decisions

- Differentiate rejected vs pending/unknown
- never return 500 for deterministic business rejection
- include support correlation ID

Deep Implementation Guidance

Treat the backend as a policy enforcement point, not a bypass around chaincode. A request may be lost after ledger submission, so recovery by requestId is mandatory. Never respond “failed” merely because the HTTP connection failed; discover the committed result first.

Data and Control Questions

- What is the authoritative source for every input used by this component?
- Which user, device, service, MSP, or administrator identity is allowed to initiate or approve it?
- Does the operation move value, reserve value, create a side effect, or only provide advisory information?
- What idempotency/replay key proves that a retry is the same business intent?
- What evidence is retained off-chain and what stable reference/hash belongs on the ledger?

Chapter 126 Implementation - Error Mapping

Implementation Procedure

- 626. Validate transport/schema/auth before calling Fabric or AI.
- 627. Read authoritative facts from ledger/projection according to the source-of-truth matrix.
- 628. Call local AI with bounded inputs and require structured output.
- 629. Apply deterministic policy/human approval before sensitive writes.
- 630. Submit through Gateway, wait for commit, log txId/correlationId, and make retries safe.

Implementation Example

```
# Error Mapping
INPUT  -> authenticate, validate schema, bind requestId/nonce
FACTS  -> read authoritative wallet/merchant/state data
AI     -> optional bounded recommendation with decisionId/version
POLICY -> enforce ownership, hard limits, approvals, risk fallback
LEDGER -> submit one idempotent deterministic transaction when value changes
EVENT  -> update projection/case/monitoring asynchronously
VERIFY -> discover committed result and retain correlationId + txId evidence
```

Verification Checklist

- The error catalog exists and is version-controlled.
- The feature works with all public Internet/WAN connectivity disabled.
- Authorization, malformed input, duplicate retry, and failure behavior have explicit tests.
- If financial value changes, journal debits and credits balance exactly and historical entries remain immutable.
- If AI is involved, model/prompt/feature version and fallback behavior are recorded and deterministic policy remains final.
- A timeout/restart cannot create a duplicate financial result; request status can be rediscovered.
- Logs/evidence contain correlation references but not secrets or prohibited sensitive payment data.

Common Failure Modes

- Returning success before commit status.
- Retry after timeout creates duplicate transfer.
- AI recommendation treated as authorization.
- Secrets/PAN/credentials logged.

Expert Review Questions

- What happens if this component succeeds but the next component fails?
- How is the authorizing identity/device/key revoked or recovered?
- Can the operation be safely retried after the client loses the response?
- What happens if AI is unavailable, slow, or wrong?
- Can an auditor reconstruct debit, credit, actor, risk decision, approval, and Fabric txId from retained evidence?
- Which compensating transaction resolves a downstream operational error without rewriting history?

Chapter 127 - Audit API

Expose controlled transaction/journal/risk/approval evidence to auditors. The API layer authenticates users, validates request schemas, gathers authoritative facts, calls local AI when useful, applies deterministic policy, submits transactions through Fabric Gateway, waits for commit status, and exposes stable application errors. It must safely recover from lost HTTP responses and duplicate retries.

Design rule: Read-only role

Implementation Objective

The required output is: Audit endpoints. It must be version-controlled, reviewable, testable with the WAN disabled, and linked to the architecture/source-of-truth model. For a payment platform, every implementation decision should make it possible to explain who authorized an action, which facts were trusted, which controls ran, what financial journal committed, and how the result is recovered after failure.

Key Engineering Decisions

- Read-only role
- filter by tx/request/wallet/date
- redact PII according to role
- include ledger and AI version references

Deep Implementation Guidance

Treat the backend as a policy enforcement point, not a bypass around chaincode. A request may be lost after ledger submission, so recovery by requestId is mandatory. Never respond “failed” merely because the HTTP connection failed; discover the committed result first.

Data and Control Questions

- What is the authoritative source for every input used by this component?
- Which user, device, service, MSP, or administrator identity is allowed to initiate or approve it?
- Does the operation move value, reserve value, create a side effect, or only provide advisory information?
- What idempotency/replay key proves that a retry is the same business intent?
- What evidence is retained off-chain and what stable reference/hash belongs on the ledger?

Chapter 127 Implementation - Audit API

Implementation Procedure

631. Validate transport/schema/auth before calling Fabric or AI.
632. Read authoritative facts from ledger/projection according to the source-of-truth matrix.
633. Call local AI with bounded inputs and require structured output.
634. Apply deterministic policy/human approval before sensitive writes.
635. Submit through Gateway, wait for commit, log txId/correlationId, and make retries safe.

Implementation Example

```
# Audit API
INPUT  -> authenticate, validate schema, bind requestId/nonce
FACTS  -> read authoritative wallet/merchant/state data
AI     -> optional bounded recommendation with decisionId/version
POLICY -> enforce ownership, hard limits, approvals, risk fallback
LEDGER -> submit one idempotent deterministic transaction when value changes
EVENT  -> update projection/case/monitoring asynchronously
VERIFY -> discover committed result and retain correlationId + txId evidence
```

Verification Checklist

- The audit endpoints exists and is version-controlled.
- The feature works with all public Internet/WAN connectivity disabled.
- Authorization, malformed input, duplicate retry, and failure behavior have explicit tests.
- If financial value changes, journal debits and credits balance exactly and historical entries remain immutable.
- If AI is involved, model/prompt/feature version and fallback behavior are recorded and deterministic policy remains final.
- A timeout/restart cannot create a duplicate financial result; request status can be rediscovered.
- Logs/evidence contain correlation references but not secrets or prohibited sensitive payment data.

Common Failure Modes

- Returning success before commit status.
- Retry after timeout creates duplicate transfer.
- AI recommendation treated as authorization.
- Secrets/PAN/credentials logged.

Expert Review Questions

- What happens if this component succeeds but the next component fails?
- How is the authorizing identity/device/key revoked or recovered?
- Can the operation be safely retried after the client loses the response?
- What happens if AI is unavailable, slow, or wrong?
- Can an auditor reconstruct debit, credit, actor, risk decision, approval, and Fabric txId from retained evidence?
- Which compensating transaction resolves a downstream operational error without rewriting history?

Chapter 128 - Health and Readiness API

Report local dependency health for operators/load balancers. The API layer authenticates users, validates request schemas, gathers authoritative facts, calls local AI when useful, applies deterministic policy, submits transactions through Fabric Gateway, waits for commit status, and exposes stable application errors. It must safely recover from lost HTTP responses and duplicate retries.

Design rule: Separate liveness/readiness

Implementation Objective

The required output is: Health endpoints. It must be version-controlled, reviewable, testable with the WAN disabled, and linked to the architecture/source-of-truth model. For a payment platform, every implementation decision should make it possible to explain who authorized an action, which facts were trusted, which controls ran, what financial journal committed, and how the result is recovered after failure.

Key Engineering Decisions

- Separate liveness/readiness
- check peer/Gateway DB risk/LLM as appropriate
- AI degradation should be distinct from ledger unavailable

Deep Implementation Guidance

Treat the backend as a policy enforcement point, not a bypass around chaincode. A request may be lost after ledger submission, so recovery by requestId is mandatory. Never respond “failed” merely because the HTTP connection failed; discover the committed result first.

Data and Control Questions

- What is the authoritative source for every input used by this component?
- Which user, device, service, MSP, or administrator identity is allowed to initiate or approve it?
- Does the operation move value, reserve value, create a side effect, or only provide advisory information?
- What idempotency/replay key proves that a retry is the same business intent?
- What evidence is retained off-chain and what stable reference/hash belongs on the ledger?

Chapter 128 Implementation - Health and Readiness API

Implementation Procedure

636. Validate transport/schema/auth before calling Fabric or AI.
637. Read authoritative facts from ledger/projection according to the source-of-truth matrix.
638. Call local AI with bounded inputs and require structured output.
639. Apply deterministic policy/human approval before sensitive writes.
640. Submit through Gateway, wait for commit, log txId/correlationId, and make retries safe.

Implementation Example

```
# Health and Readiness API
INPUT  -> authenticate, validate schema, bind requestId/nonce
FACTS  -> read authoritative wallet/merchant/state data
AI      -> optional bounded recommendation with decisionId/version
POLICY  -> enforce ownership, hard limits, approvals, risk fallback
LEDGER  -> submit one idempotent deterministic transaction when value changes
EVENT   -> update projection/case/monitoring asynchronously
VERIFY  -> discover committed result and retain correlationId + txId evidence
```

Verification Checklist

- The health endpoints exists and is version-controlled.
- The feature works with all public Internet/WAN connectivity disabled.
- Authorization, malformed input, duplicate retry, and failure behavior have explicit tests.
- If financial value changes, journal debits and credits balance exactly and historical entries remain immutable.
- If AI is involved, model/prompt/feature version and fallback behavior are recorded and deterministic policy remains final.
- A timeout/restart cannot create a duplicate financial result; request status can be rediscovered.
- Logs/evidence contain correlation references but not secrets or prohibited sensitive payment data.

Common Failure Modes

- Returning success before commit status.
- Retry after timeout creates duplicate transfer.
- AI recommendation treated as authorization.
- Secrets/PAN/credentials logged.

Expert Review Questions

- What happens if this component succeeds but the next component fails?
- How is the authorizing identity/device/key revoked or recovered?
- Can the operation be safely retried after the client loses the response?
- What happens if AI is unavailable, slow, or wrong?
- Can an auditor reconstruct debit, credit, actor, risk decision, approval, and Fabric txId from retained evidence?
- Which compensating transaction resolves a downstream operational error without rewriting history?

Chapter 129 - Configuration and Secret Loading

Keep environment-specific values out of source and logs. The API layer authenticates users, validates request schemas, gathers authoritative facts, calls local AI when useful, applies deterministic policy, submits transactions through Fabric Gateway, waits for commit status, and exposes stable application errors. It must safely recover from lost HTTP responses and duplicate retries.

Design rule: Use local secret files/HSM/env injection

Implementation Objective

The required output is: Config module. It must be version-controlled, reviewable, testable with the WAN disabled, and linked to the architecture/source-of-truth model. For a payment platform, every implementation decision should make it possible to explain who authorized an action, which facts were trusted, which controls ran, what financial journal committed, and how the result is recovered after failure.

Key Engineering Decisions

- Use local secret files/HSM/env injection
- validate at startup
- never default silently to public endpoints

Deep Implementation Guidance

Treat the backend as a policy enforcement point, not a bypass around chaincode. A request may be lost after ledger submission, so recovery by requestId is mandatory. Never respond “failed” merely because the HTTP connection failed; discover the committed result first.

Data and Control Questions

- What is the authoritative source for every input used by this component?
- Which user, device, service, MSP, or administrator identity is allowed to initiate or approve it?
- Does the operation move value, reserve value, create a side effect, or only provide advisory information?
- What idempotency/replay key proves that a retry is the same business intent?
- What evidence is retained off-chain and what stable reference/hash belongs on the ledger?

Chapter 129 Implementation - Configuration and Secret Loading

Implementation Procedure

641. Validate transport/schema/auth before calling Fabric or AI.
642. Read authoritative facts from ledger/projection according to the source-of-truth matrix.
643. Call local AI with bounded inputs and require structured output.
644. Apply deterministic policy/human approval before sensitive writes.
645. Submit through Gateway, wait for commit, log txId/correlationId, and make retries safe.

Implementation Example

```
# Configuration and Secret Loading
INPUT  -> authenticate, validate schema, bind requestId/nonce
FACTS  -> read authoritative wallet/merchant/state data
AI      -> optional bounded recommendation with decisionId/version
POLICY  -> enforce ownership, hard limits, approvals, risk fallback
LEDGER  -> submit one idempotent deterministic transaction when value changes
EVENT   -> update projection/case/monitoring asynchronously
VERIFY  -> discover committed result and retain correlationId + txId evidence
```

Verification Checklist

- The config module exists and is version-controlled.
- The feature works with all public Internet/WAN connectivity disabled.
- Authorization, malformed input, duplicate retry, and failure behavior have explicit tests.
- If financial value changes, journal debits and credits balance exactly and historical entries remain immutable.
- If AI is involved, model/prompt/feature version and fallback behavior are recorded and deterministic policy remains final.
- A timeout/restart cannot create a duplicate financial result; request status can be rediscovered.
- Logs/evidence contain correlation references but not secrets or prohibited sensitive payment data.

Common Failure Modes

- Returning success before commit status.
- Retry after timeout creates duplicate transfer.
- AI recommendation treated as authorization.
- Secrets/PAN/credentials logged.

Expert Review Questions

- What happens if this component succeeds but the next component fails?
- How is the authorizing identity/device/key revoked or recovered?
- Can the operation be safely retried after the client loses the response?
- What happens if AI is unavailable, slow, or wrong?
- Can an auditor reconstruct debit, credit, actor, risk decision, approval, and Fabric txId from retained evidence?
- Which compensating transaction resolves a downstream operational error without rewriting history?

Part VIII - Wallet, Merchant POS, and Operator User Interfaces

Wallet and merchant applications should make transaction state explicit: draft, authorized, risk-reviewed, submitted, committed, rejected, reversed, or pending human review. A polished UI never substitutes for server-side authorization and ledger enforcement.

Part focus: This part contains 16 implementation chapters. Each chapter includes design rules, implementation procedure, a practical artifact, verification, failure analysis, and expert review questions.

Chapter 130 - LAN-Only Frontend Architecture

Build web/POS clients with no CDN/cloud runtime dependency. Wallet and merchant applications should make transaction state explicit: draft, authorized, risk-reviewed, submitted, committed, rejected, reversed, or pending human review. A polished UI never substitutes for server-side authorization and ledger enforcement.

Design rule: Bundle JS CSS fonts icons locally

Implementation Objective

The required output is: Frontend deployment. It must be version-controlled, reviewable, testable with the WAN disabled, and linked to the architecture/source-of-truth model. For a payment platform, every implementation decision should make it possible to explain who authorized an action, which facts were trusted, which controls ran, what financial journal committed, and how the result is recovered after failure.

Key Engineering Decisions

- Bundle JS CSS fonts icons locally
- serve from internal reverse proxy
- use internal API/DNS only

Deep Implementation Guidance

Financial interfaces should be explicit about authorization and finality. Users need to distinguish “request accepted for review,” “submitted,” “committed,” “rejected,” and “status unknown.” This reduces support incidents and duplicate retries.

Data and Control Questions

- What is the authoritative source for every input used by this component?
- Which user, device, service, MSP, or administrator identity is allowed to initiate or approve it?
- Does the operation move value, reserve value, create a side effect, or only provide advisory information?
- What idempotency/replay key proves that a retry is the same business intent?
- What evidence is retained off-chain and what stable reference/hash belongs on the ledger?

Chapter 130 Implementation - LAN-Only Frontend Architecture

Implementation Procedure

- 646. Render role-aware screens but enforce permissions server-side.
- 647. Generate one idempotency key per financial intent.
- 648. Show pending/committed/rejected/review states explicitly.
- 649. Recover after refresh/reconnect by querying requestId/transaction status.
- 650. Keep all JS/CSS/fonts/assets local and show service-health degradation clearly.

Implementation Example

```
# LAN-Only Frontend Architecture
INPUT  -> authenticate, validate schema, bind requestId/nonce
FACTS  -> read authoritative wallet/merchant/state data
AI     -> optional bounded recommendation with decisionId/version
POLICY -> enforce ownership, hard limits, approvals, risk fallback
LEDGER -> submit one idempotent deterministic transaction when value changes
EVENT  -> update projection/case/monitoring asynchronously
VERIFY -> discover committed result and retain correlationId + txId evidence
```

Verification Checklist

- The frontend deployment exists and is version-controlled.
- The feature works with all public Internet/WAN connectivity disabled.
- Authorization, malformed input, duplicate retry, and failure behavior have explicit tests.
- If financial value changes, journal debits and credits balance exactly and historical entries remain immutable.
- If AI is involved, model/prompt/feature version and fallback behavior are recorded and deterministic policy remains final.
- A timeout/restart cannot create a duplicate financial result; request status can be rediscovered.
- Logs/evidence contain correlation references but not secrets or prohibited sensitive payment data.

Common Failure Modes

- Double-submit.
- Ambiguous pending state.
- Offline runtime assets loaded from CDN.
- Client-calculated totals trusted by server.

Expert Review Questions

- What happens if this component succeeds but the next component fails?
- How is the authorizing identity/device/key revoked or recovered?
- Can the operation be safely retried after the client loses the response?
- What happens if AI is unavailable, slow, or wrong?
- Can an auditor reconstruct debit, credit, actor, risk decision, approval, and Fabric txId from retained evidence?
- Which compensating transaction resolves a downstream operational error without rewriting history?

Chapter 131 - Customer Wallet Dashboard

Show balances, holds, recent transactions, status, and security alerts clearly. Wallet and merchant applications should make transaction state explicit: draft, authorized, risk-reviewed, submitted, committed, rejected, reversed, or pending human review. A polished UI never substitutes for server-side authorization and ledger enforcement.

Design rule: Display asset and available/reserved separately

Implementation Objective

The required output is: Wallet dashboard. It must be version-controlled, reviewable, testable with the WAN disabled, and linked to the architecture/source-of-truth model. For a payment platform, every implementation decision should make it possible to explain who authorized an action, which facts were trusted, which controls ran, what financial journal committed, and how the result is recovered after failure.

Key Engineering Decisions

- Display asset and available/reserved separately
- show projection timestamp
- support ledger verification view for disputes

Deep Implementation Guidance

Financial interfaces should be explicit about authorization and finality. Users need to distinguish “request accepted for review,” “submitted,” “committed,” “rejected,” and “status unknown.” This reduces support incidents and duplicate retries.

Data and Control Questions

- What is the authoritative source for every input used by this component?
- Which user, device, service, MSP, or administrator identity is allowed to initiate or approve it?
- Does the operation move value, reserve value, create a side effect, or only provide advisory information?
- What idempotency/replay key proves that a retry is the same business intent?
- What evidence is retained off-chain and what stable reference/hash belongs on the ledger?

Chapter 131 Implementation - Customer Wallet Dashboard

Implementation Procedure

651. Render role-aware screens but enforce permissions server-side.
652. Generate one idempotency key per financial intent.
653. Show pending/committed/rejected/review states explicitly.
654. Recover after refresh/reconnect by querying requestId/transaction status.
655. Keep all JS/CSS/fonts/assets local and show service-health degradation clearly.

Implementation Example

```
# Customer Wallet Dashboard
INPUT  -> authenticate, validate schema, bind requestId/nonce
FACTS  -> read authoritative wallet/merchant/state data
AI     -> optional bounded recommendation with decisionId/version
POLICY -> enforce ownership, hard limits, approvals, risk fallback
LEDGER -> submit one idempotent deterministic transaction when value changes
EVENT  -> update projection/case/monitoring asynchronously
VERIFY -> discover committed result and retain correlationId + txId evidence
```

Verification Checklist

- The wallet dashboard exists and is version-controlled.
- The feature works with all public Internet/WAN connectivity disabled.
- Authorization, malformed input, duplicate retry, and failure behavior have explicit tests.
- If financial value changes, journal debits and credits balance exactly and historical entries remain immutable.
- If AI is involved, model/prompt/feature version and fallback behavior are recorded and deterministic policy remains final.
- A timeout/restart cannot create a duplicate financial result; request status can be rediscovered.
- Logs/evidence contain correlation references but not secrets or prohibited sensitive payment data.

Common Failure Modes

- Double-submit.
- Ambiguous pending state.
- Offline runtime assets loaded from CDN.
- Client-calculated totals trusted by server.

Expert Review Questions

- What happens if this component succeeds but the next component fails?
- How is the authorizing identity/device/key revoked or recovered?
- Can the operation be safely retried after the client loses the response?
- What happens if AI is unavailable, slow, or wrong?
- Can an auditor reconstruct debit, credit, actor, risk decision, approval, and Fabric txId from retained evidence?
- Which compensating transaction resolves a downstream operational error without rewriting history?

Chapter 132 - P2P Transfer UX

Reduce wrong-recipient and duplicate-transfer risk. Wallet and merchant applications should make transaction state explicit: draft, authorized, risk-reviewed, submitted, committed, rejected, reversed, or pending human review. A polished UI never substitutes for server-side authorization and ledger enforcement.

Design rule: Resolve recipient locally

Implementation Objective

The required output is: Transfer screen. It must be version-controlled, reviewable, testable with the WAN disabled, and linked to the architecture/source-of-truth model. For a payment platform, every implementation decision should make it possible to explain who authorized an action, which facts were trusted, which controls ran, what financial journal committed, and how the result is recovered after failure.

Key Engineering Decisions

- Resolve recipient locally
- confirm amount/asset/recipient
- generate requestId once
- show pending commit and final reference

Deep Implementation Guidance

Financial interfaces should be explicit about authorization and finality. Users need to distinguish “request accepted for review,” “submitted,” “committed,” “rejected,” and “status unknown.” This reduces support incidents and duplicate retries.

Data and Control Questions

- What is the authoritative source for every input used by this component?
- Which user, device, service, MSP, or administrator identity is allowed to initiate or approve it?
- Does the operation move value, reserve value, create a side effect, or only provide advisory information?
- What idempotency/replay key proves that a retry is the same business intent?
- What evidence is retained off-chain and what stable reference/hash belongs on the ledger?

Chapter 132 Implementation - P2P Transfer UX

Implementation Procedure

- 656. Render role-aware screens but enforce permissions server-side.
- 657. Generate one idempotency key per financial intent.
- 658. Show pending/committed/rejected/review states explicitly.
- 659. Recover after refresh/reconnect by querying requestId/transaction status.
- 660. Keep all JS/CSS/fonts/assets local and show service-health degradation clearly.

Implementation Example

```
# P2P Transfer UX
INPUT  -> authenticate, validate schema, bind requestId/nonce
FACTS  -> read authoritative wallet/merchant/state data
AI     -> optional bounded recommendation with decisionId/version
POLICY -> enforce ownership, hard limits, approvals, risk fallback
LEDGER -> submit one idempotent deterministic transaction when value changes
EVENT  -> update projection/case/monitoring asynchronously
VERIFY -> discover committed result and retain correlationId + txId evidence
```

Verification Checklist

- The transfer screen exists and is version-controlled.
- The feature works with all public Internet/WAN connectivity disabled.
- Authorization, malformed input, duplicate retry, and failure behavior have explicit tests.
- If financial value changes, journal debits and credits balance exactly and historical entries remain immutable.
- If AI is involved, model/prompt/feature version and fallback behavior are recorded and deterministic policy remains final.
- A timeout/restart cannot create a duplicate financial result; request status can be rediscovered.
- Logs/evidence contain correlation references but not secrets or prohibited sensitive payment data.

Common Failure Modes

- Double-submit.
- Ambiguous pending state.
- Offline runtime assets loaded from CDN.
- Client-calculated totals trusted by server.

Expert Review Questions

- What happens if this component succeeds but the next component fails?
- How is the authorizing identity/device/key revoked or recovered?
- Can the operation be safely retried after the client loses the response?
- What happens if AI is unavailable, slow, or wrong?
- Can an auditor reconstruct debit, credit, actor, risk decision, approval, and Fabric txId from retained evidence?
- Which compensating transaction resolves a downstream operational error without rewriting history?

Chapter 133 - QR Payment Format

Define a local QR payload for merchant/store/invoice/payment intent. Wallet and merchant applications should make transaction state explicit: draft, authorized, risk-reviewed, submitted, committed, rejected, reversed, or pending human review. A polished UI never substitutes for server-side authorization and ledger enforcement.

Design rule: Use versioned URI/JSON payload

Implementation Objective

The required output is: QR specification. It must be version-controlled, reviewable, testable with the WAN disabled, and linked to the architecture/source-of-truth model. For a payment platform, every implementation decision should make it possible to explain who authorized an action, which facts were trusted, which controls ran, what financial journal committed, and how the result is recovered after failure.

Key Engineering Decisions

- Use versioned URI/JSON payload
- do not embed secrets
- sign merchant QR if authenticity needed
- validate expiry/amount binding

Deep Implementation Guidance

Financial interfaces should be explicit about authorization and finality. Users need to distinguish “request accepted for review,” “submitted,” “committed,” “rejected,” and “status unknown.” This reduces support incidents and duplicate retries.

Data and Control Questions

- What is the authoritative source for every input used by this component?
- Which user, device, service, MSP, or administrator identity is allowed to initiate or approve it?
- Does the operation move value, reserve value, create a side effect, or only provide advisory information?
- What idempotency/replay key proves that a retry is the same business intent?
- What evidence is retained off-chain and what stable reference/hash belongs on the ledger?

Chapter 133 Implementation - QR Payment Format

Implementation Procedure

661. Render role-aware screens but enforce permissions server-side.
662. Generate one idempotency key per financial intent.
663. Show pending/committed/rejected/review states explicitly.
664. Recover after refresh/reconnect by querying requestId/transaction status.
665. Keep all JS/CSS/fonts/assets local and show service-health degradation clearly.

Implementation Example

```
# QR Payment Format
INPUT  -> authenticate, validate schema, bind requestId/nonce
FACTS  -> read authoritative wallet/merchant/state data
AI     -> optional bounded recommendation with decisionId/version
POLICY -> enforce ownership, hard limits, approvals, risk fallback
LEDGER -> submit one idempotent deterministic transaction when value changes
EVENT  -> update projection/case/monitoring asynchronously
VERIFY -> discover committed result and retain correlationId + txId evidence
```

Verification Checklist

- The qr specification exists and is version-controlled.
- The feature works with all public Internet/WAN connectivity disabled.
- Authorization, malformed input, duplicate retry, and failure behavior have explicit tests.
- If financial value changes, journal debits and credits balance exactly and historical entries remain immutable.
- If AI is involved, model/prompt/feature version and fallback behavior are recorded and deterministic policy remains final.
- A timeout/restart cannot create a duplicate financial result; request status can be rediscovered.
- Logs/evidence contain correlation references but not secrets or prohibited sensitive payment data.

Common Failure Modes

- Double-submit.
- Ambiguous pending state.
- Offline runtime assets loaded from CDN.
- Client-calculated totals trusted by server.

Expert Review Questions

- What happens if this component succeeds but the next component fails?
- How is the authorizing identity/device/key revoked or recovered?
- Can the operation be safely retried after the client loses the response?
- What happens if AI is unavailable, slow, or wrong?
- Can an auditor reconstruct debit, credit, actor, risk decision, approval, and Fabric txId from retained evidence?
- Which compensating transaction resolves a downstream operational error without rewriting history?

Chapter 134 - Merchant POS Payment Flow

Implement scan/amount/authorize/risk/commit/receipt flow. Wallet and merchant applications should make transaction state explicit: draft, authorized, risk-reviewed, submitted, committed, rejected, reversed, or pending human review. A polished UI never substitutes for server-side authorization and ledger enforcement.

Design rule: Terminal never sees Fabric admin key

Implementation Objective

The required output is: POS state machine. It must be version-controlled, reviewable, testable with the WAN disabled, and linked to the architecture/source-of-truth model. For a payment platform, every implementation decision should make it possible to explain who authorized an action, which facts were trusted, which controls ran, what financial journal committed, and how the result is recovered after failure.

Key Engineering Decisions

- Terminal never sees Fabric admin key
- disable repeat submit
- show review/rejected/pending/committed states

Deep Implementation Guidance

Financial interfaces should be explicit about authorization and finality. Users need to distinguish “request accepted for review,” “submitted,” “committed,” “rejected,” and “status unknown.” This reduces support incidents and duplicate retries.

Data and Control Questions

- What is the authoritative source for every input used by this component?
- Which user, device, service, MSP, or administrator identity is allowed to initiate or approve it?
- Does the operation move value, reserve value, create a side effect, or only provide advisory information?
- What idempotency/replay key proves that a retry is the same business intent?
- What evidence is retained off-chain and what stable reference/hash belongs on the ledger?

Chapter 134 Implementation - Merchant POS Payment Flow

Implementation Procedure

- 666. Render role-aware screens but enforce permissions server-side.
- 667. Generate one idempotency key per financial intent.
- 668. Show pending/committed/rejected/review states explicitly.
- 669. Recover after refresh/reconnect by querying requestId/transaction status.
- 670. Keep all JS/CSS/fonts/assets local and show service-health degradation clearly.

Implementation Example

```
# Merchant POS Payment Flow
INPUT  -> authenticate, validate schema, bind requestId/nonce
FACTS  -> read authoritative wallet/merchant/state data
AI     -> optional bounded recommendation with decisionId/version
POLICY -> enforce ownership, hard limits, approvals, risk fallback
LEDGER -> submit one idempotent deterministic transaction when value changes
EVENT  -> update projection/case/monitoring asynchronously
VERIFY -> discover committed result and retain correlationId + txId evidence
```

Verification Checklist

- The pos state machine exists and is version-controlled.
- The feature works with all public Internet/WAN connectivity disabled.
- Authorization, malformed input, duplicate retry, and failure behavior have explicit tests.
- If financial value changes, journal debits and credits balance exactly and historical entries remain immutable.
- If AI is involved, model/prompt/feature version and fallback behavior are recorded and deterministic policy remains final.
- A timeout/restart cannot create a duplicate financial result; request status can be rediscovered.
- Logs/evidence contain correlation references but not secrets or prohibited sensitive payment data.

Common Failure Modes

- Double-submit.
- Ambiguous pending state.
- Offline runtime assets loaded from CDN.
- Client-calculated totals trusted by server.

Expert Review Questions

- What happens if this component succeeds but the next component fails?
- How is the authorizing identity/device/key revoked or recovered?
- Can the operation be safely retried after the client loses the response?
- What happens if AI is unavailable, slow, or wrong?
- Can an auditor reconstruct debit, credit, actor, risk decision, approval, and Fabric txId from retained evidence?
- Which compensating transaction resolves a downstream operational error without rewriting history?

Chapter 135 - Hold and Capture UI

Support reservation/preauthorization use cases without confusing held and spent value. Wallet and merchant applications should make transaction state explicit: draft, authorized, risk-reviewed, submitted, committed, rejected, reversed, or pending human review. A polished UI never substitutes for server-side authorization and ledger enforcement.

Design rule: Show hold amount/expiry/status

Implementation Objective

The required output is: Hold UI. It must be version-controlled, reviewable, testable with the WAN disabled, and linked to the architecture/source-of-truth model. For a payment platform, every implementation decision should make it possible to explain who authorized an action, which facts were trusted, which controls ran, what financial journal committed, and how the result is recovered after failure.

Key Engineering Decisions

- Show hold amount/expiry/status
- capture only permitted amount
- release controls require role

Deep Implementation Guidance

Financial interfaces should be explicit about authorization and finality. Users need to distinguish “request accepted for review,” “submitted,” “committed,” “rejected,” and “status unknown.” This reduces support incidents and duplicate retries.

Data and Control Questions

- What is the authoritative source for every input used by this component?
- Which user, device, service, MSP, or administrator identity is allowed to initiate or approve it?
- Does the operation move value, reserve value, create a side effect, or only provide advisory information?
- What idempotency/replay key proves that a retry is the same business intent?
- What evidence is retained off-chain and what stable reference/hash belongs on the ledger?

Chapter 135 Implementation - Hold and Capture UI

Implementation Procedure

- 671. Render role-aware screens but enforce permissions server-side.
- 672. Generate one idempotency key per financial intent.
- 673. Show pending/committed/rejected/review states explicitly.
- 674. Recover after refresh/reconnect by querying requestId/transaction status.
- 675. Keep all JS/CSS/fonts/assets local and show service-health degradation clearly.

Implementation Example

```
# Hold and Capture UI
INPUT  -> authenticate, validate schema, bind requestId/nonce
FACTS  -> read authoritative wallet/merchant/state data
AI     -> optional bounded recommendation with decisionId/version
POLICY -> enforce ownership, hard limits, approvals, risk fallback
LEDGER -> submit one idempotent deterministic transaction when value changes
EVENT  -> update projection/case/monitoring asynchronously
VERIFY -> discover committed result and retain correlationId + txId evidence
```

Verification Checklist

- The hold ui exists and is version-controlled.
- The feature works with all public Internet/WAN connectivity disabled.
- Authorization, malformed input, duplicate retry, and failure behavior have explicit tests.
- If financial value changes, journal debits and credits balance exactly and historical entries remain immutable.
- If AI is involved, model/prompt/feature version and fallback behavior are recorded and deterministic policy remains final.
- A timeout/restart cannot create a duplicate financial result; request status can be rediscovered.
- Logs/evidence contain correlation references but not secrets or prohibited sensitive payment data.

Common Failure Modes

- Double-submit.
- Ambiguous pending state.
- Offline runtime assets loaded from CDN.
- Client-calculated totals trusted by server.

Expert Review Questions

- What happens if this component succeeds but the next component fails?
- How is the authorizing identity/device/key revoked or recovered?
- Can the operation be safely retried after the client loses the response?
- What happens if AI is unavailable, slow, or wrong?
- Can an auditor reconstruct debit, credit, actor, risk decision, approval, and Fabric txId from retained evidence?
- Which compensating transaction resolves a downstream operational error without rewriting history?

Chapter 136 - Refund and Dispute UI

Guide staff through original payment lookup, reason, evidence, approval, and refund. Wallet and merchant applications should make transaction state explicit: draft, authorized, risk-reviewed, submitted, committed, rejected, reversed, or pending human review. A polished UI never substitutes for server-side authorization and ledger enforcement.

Design rule: Never type arbitrary destination wallet when refund should go to original payer

Implementation Objective

The required output is: Refund workflow UI. It must be version-controlled, reviewable, testable with the WAN disabled, and linked to the architecture/source-of-truth model. For a payment platform, every implementation decision should make it possible to explain who authorized an action, which facts were trusted, which controls ran, what financial journal committed, and how the result is recovered after failure.

Key Engineering Decisions

- Never type arbitrary destination wallet when refund should go to original payer
- show refundable remainder
- require case/approval when policy says

Deep Implementation Guidance

Financial interfaces should be explicit about authorization and finality. Users need to distinguish “request accepted for review,” “submitted,” “committed,” “rejected,” and “status unknown.” This reduces support incidents and duplicate retries.

Data and Control Questions

- What is the authoritative source for every input used by this component?
- Which user, device, service, MSP, or administrator identity is allowed to initiate or approve it?
- Does the operation move value, reserve value, create a side effect, or only provide advisory information?
- What idempotency/replay key proves that a retry is the same business intent?
- What evidence is retained off-chain and what stable reference/hash belongs on the ledger?

Chapter 136 Implementation - Refund and Dispute UI

Implementation Procedure

- 676. Render role-aware screens but enforce permissions server-side.
- 677. Generate one idempotency key per financial intent.
- 678. Show pending/committed/rejected/review states explicitly.
- 679. Recover after refresh/reconnect by querying requestId/transaction status.
- 680. Keep all JS/CSS/fonts/assets local and show service-health degradation clearly.

Implementation Example

```
# Refund and Dispute UI
INPUT  -> authenticate, validate schema, bind requestId/nonce
FACTS  -> read authoritative wallet/merchant/state data
AI      -> optional bounded recommendation with decisionId/version
POLICY  -> enforce ownership, hard limits, approvals, risk fallback
LEDGER  -> submit one idempotent deterministic transaction when value changes
EVENT   -> update projection/case/monitoring asynchronously
VERIFY  -> discover committed result and retain correlationId + txId evidence
```

Verification Checklist

- The refund workflow ui exists and is version-controlled.
- The feature works with all public Internet/WAN connectivity disabled.
- Authorization, malformed input, duplicate retry, and failure behavior have explicit tests.
- If financial value changes, journal debits and credits balance exactly and historical entries remain immutable.
- If AI is involved, model/prompt/feature version and fallback behavior are recorded and deterministic policy remains final.
- A timeout/restart cannot create a duplicate financial result; request status can be rediscovered.
- Logs/evidence contain correlation references but not secrets or prohibited sensitive payment data.

Common Failure Modes

- Double-submit.
- Ambiguous pending state.
- Offline runtime assets loaded from CDN.
- Client-calculated totals trusted by server.

Expert Review Questions

- What happens if this component succeeds but the next component fails?
- How is the authorizing identity/device/key revoked or recovered?
- Can the operation be safely retried after the client loses the response?
- What happens if AI is unavailable, slow, or wrong?
- Can an auditor reconstruct debit, credit, actor, risk decision, approval, and Fabric txId from retained evidence?
- Which compensating transaction resolves a downstream operational error without rewriting history?

Chapter 137 - Treasury Console

Provide tightly controlled issuance/redemption/settlement functions. Wallet and merchant applications should make transaction state explicit: draft, authorized, risk-reviewed, submitted, committed, rejected, reversed, or pending human review. A polished UI never substitutes for server-side authorization and ledger enforcement.

Design rule: MFA and dual approval

Implementation Objective

The required output is: Treasury UI. It must be version-controlled, reviewable, testable with the WAN disabled, and linked to the architecture/source-of-truth model. For a payment platform, every implementation decision should make it possible to explain who authorized an action, which facts were trusted, which controls ran, what financial journal committed, and how the result is recovered after failure.

Key Engineering Decisions

- MFA and dual approval
- show before/after balances
- no bulk operation without preview and reconciliation report

Deep Implementation Guidance

Financial interfaces should be explicit about authorization and finality. Users need to distinguish “request accepted for review,” “submitted,” “committed,” “rejected,” and “status unknown.” This reduces support incidents and duplicate retries.

Data and Control Questions

- What is the authoritative source for every input used by this component?
- Which user, device, service, MSP, or administrator identity is allowed to initiate or approve it?
- Does the operation move value, reserve value, create a side effect, or only provide advisory information?
- What idempotency/replay key proves that a retry is the same business intent?
- What evidence is retained off-chain and what stable reference/hash belongs on the ledger?

Chapter 137 Implementation - Treasury Console

Implementation Procedure

681. Render role-aware screens but enforce permissions server-side.
682. Generate one idempotency key per financial intent.
683. Show pending/committed/rejected/review states explicitly.
684. Recover after refresh/reconnect by querying requestId/transaction status.
685. Keep all JS/CSS/fonts/assets local and show service-health degradation clearly.

Implementation Example

```
# Treasury Console
INPUT  -> authenticate, validate schema, bind requestId/nonce
FACTS  -> read authoritative wallet/merchant/state data
AI     -> optional bounded recommendation with decisionId/version
POLICY -> enforce ownership, hard limits, approvals, risk fallback
LEDGER -> submit one idempotent deterministic transaction when value changes
EVENT  -> update projection/case/monitoring asynchronously
VERIFY -> discover committed result and retain correlationId + txId evidence
```

Verification Checklist

- The treasury ui exists and is version-controlled.
- The feature works with all public Internet/WAN connectivity disabled.
- Authorization, malformed input, duplicate retry, and failure behavior have explicit tests.
- If financial value changes, journal debits and credits balance exactly and historical entries remain immutable.
- If AI is involved, model/prompt/feature version and fallback behavior are recorded and deterministic policy remains final.
- A timeout/restart cannot create a duplicate financial result; request status can be rediscovered.
- Logs/evidence contain correlation references but not secrets or prohibited sensitive payment data.

Common Failure Modes

- Double-submit.
- Ambiguous pending state.
- Offline runtime assets loaded from CDN.
- Client-calculated totals trusted by server.

Expert Review Questions

- What happens if this component succeeds but the next component fails?
- How is the authorizing identity/device/key revoked or recovered?
- Can the operation be safely retried after the client loses the response?
- What happens if AI is unavailable, slow, or wrong?
- Can an auditor reconstruct debit, credit, actor, risk decision, approval, and Fabric txId from retained evidence?
- Which compensating transaction resolves a downstream operational error without rewriting history?

Chapter 138 - Merchant Settlement Dashboard

Show merchant payable balance, batches, fees, exceptions, and settlement status. Wallet and merchant applications should make transaction state explicit: draft, authorized, risk-reviewed, submitted, committed, rejected, reversed, or pending human review. A polished UI never substitutes for server-side authorization and ledger enforcement.

Design rule: Display source transaction range

Implementation Objective

The required output is: Settlement UI. It must be version-controlled, reviewable, testable with the WAN disabled, and linked to the architecture/source-of-truth model. For a payment platform, every implementation decision should make it possible to explain who authorized an action, which facts were trusted, which controls ran, what financial journal committed, and how the result is recovered after failure.

Key Engineering Decisions

- Display source transaction range
- require approval for manual adjustment
- anchor batch report hash

Deep Implementation Guidance

Financial interfaces should be explicit about authorization and finality. Users need to distinguish “request accepted for review,” “submitted,” “committed,” “rejected,” and “status unknown.” This reduces support incidents and duplicate retries.

Data and Control Questions

- What is the authoritative source for every input used by this component?
- Which user, device, service, MSP, or administrator identity is allowed to initiate or approve it?
- Does the operation move value, reserve value, create a side effect, or only provide advisory information?
- What idempotency/replay key proves that a retry is the same business intent?
- What evidence is retained off-chain and what stable reference/hash belongs on the ledger?

Chapter 138 Implementation - Merchant Settlement Dashboard

Implementation Procedure

- 686. Render role-aware screens but enforce permissions server-side.
- 687. Generate one idempotency key per financial intent.
- 688. Show pending/committed/rejected/review states explicitly.
- 689. Recover after refresh/reconnect by querying requestId/transaction status.
- 690. Keep all JS/CSS/fonts/assets local and show service-health degradation clearly.

Implementation Example

```
# Merchant Settlement Dashboard
INPUT  -> authenticate, validate schema, bind requestId/nonce
FACTS  -> read authoritative wallet/merchant/state data
AI     -> optional bounded recommendation with decisionId/version
POLICY -> enforce ownership, hard limits, approvals, risk fallback
LEDGER -> submit one idempotent deterministic transaction when value changes
EVENT  -> update projection/case/monitoring asynchronously
VERIFY -> discover committed result and retain correlationId + txId evidence
```

Verification Checklist

- The settlement ui exists and is version-controlled.
- The feature works with all public Internet/WAN connectivity disabled.
- Authorization, malformed input, duplicate retry, and failure behavior have explicit tests.
- If financial value changes, journal debits and credits balance exactly and historical entries remain immutable.
- If AI is involved, model/prompt/feature version and fallback behavior are recorded and deterministic policy remains final.
- A timeout/restart cannot create a duplicate financial result; request status can be rediscovered.
- Logs/evidence contain correlation references but not secrets or prohibited sensitive payment data.

Common Failure Modes

- Double-submit.
- Ambiguous pending state.
- Offline runtime assets loaded from CDN.
- Client-calculated totals trusted by server.

Expert Review Questions

- What happens if this component succeeds but the next component fails?
- How is the authorizing identity/device/key revoked or recovered?
- Can the operation be safely retried after the client loses the response?
- What happens if AI is unavailable, slow, or wrong?
- Can an auditor reconstruct debit, credit, actor, risk decision, approval, and Fabric txId from retained evidence?
- Which compensating transaction resolves a downstream operational error without rewriting history?

Chapter 139 - Fraud Review Queue

Present AI/rules evidence for REVIEW decisions without asking analyst to trust a single score. Wallet and merchant applications should make transaction state explicit: draft, authorized, risk-reviewed, submitted, committed, rejected, reversed, or pending human review. A polished UI never substitutes for server-side authorization and ledger enforcement.

Design rule: Show reason codes/features/model version/relevant history

Implementation Objective

The required output is: Review console. It must be version-controlled, reviewable, testable with the WAN disabled, and linked to the architecture/source-of-truth model. For a payment platform, every implementation decision should make it possible to explain who authorized an action, which facts were trusted, which controls ran, what financial journal committed, and how the result is recovered after failure.

Key Engineering Decisions

- Show reason codes/features/model version/relevant history
- allow approve/reject/escalate with reason
- record reviewer identity

Deep Implementation Guidance

Financial interfaces should be explicit about authorization and finality. Users need to distinguish “request accepted for review,” “submitted,” “committed,” “rejected,” and “status unknown.” This reduces support incidents and duplicate retries.

Data and Control Questions

- What is the authoritative source for every input used by this component?
- Which user, device, service, MSP, or administrator identity is allowed to initiate or approve it?
- Does the operation move value, reserve value, create a side effect, or only provide advisory information?
- What idempotency/replay key proves that a retry is the same business intent?
- What evidence is retained off-chain and what stable reference/hash belongs on the ledger?

Chapter 139 Implementation - Fraud Review Queue

Implementation Procedure

691. Render role-aware screens but enforce permissions server-side.
692. Generate one idempotency key per financial intent.
693. Show pending/committed/rejected/review states explicitly.
694. Recover after refresh/reconnect by querying requestId/transaction status.
695. Keep all JS/CSS/fonts/assets local and show service-health degradation clearly.

Implementation Example

```
# Fraud Review Queue
INPUT  -> authenticate, validate schema, bind requestId/nonce
FACTS  -> read authoritative wallet/merchant/state data
AI     -> optional bounded recommendation with decisionId/version
POLICY -> enforce ownership, hard limits, approvals, risk fallback
LEDGER -> submit one idempotent deterministic transaction when value changes
EVENT  -> update projection/case/monitoring asynchronously
VERIFY -> discover committed result and retain correlationId + txId evidence
```

Verification Checklist

- The review console exists and is version-controlled.
- The feature works with all public Internet/WAN connectivity disabled.
- Authorization, malformed input, duplicate retry, and failure behavior have explicit tests.
- If financial value changes, journal debits and credits balance exactly and historical entries remain immutable.
- If AI is involved, model/prompt/feature version and fallback behavior are recorded and deterministic policy remains final.
- A timeout/restart cannot create a duplicate financial result; request status can be rediscovered.
- Logs/evidence contain correlation references but not secrets or prohibited sensitive payment data.

Common Failure Modes

- Double-submit.
- Ambiguous pending state.
- Offline runtime assets loaded from CDN.
- Client-calculated totals trusted by server.

Expert Review Questions

- What happens if this component succeeds but the next component fails?
- How is the authorizing identity/device/key revoked or recovered?
- Can the operation be safely retried after the client loses the response?
- What happens if AI is unavailable, slow, or wrong?
- Can an auditor reconstruct debit, credit, actor, risk decision, approval, and Fabric txId from retained evidence?
- Which compensating transaction resolves a downstream operational error without rewriting history?

Chapter 140 - Security Operations Dashboard

Show frozen wallets, revoked devices, unusual failures, and model/service health. Wallet and merchant applications should make transaction state explicit: draft, authorized, risk-reviewed, submitted, committed, rejected, reversed, or pending human review. A polished UI never substitutes for server-side authorization and ledger enforcement.

Design rule: Prioritize actionable alerts

Implementation Objective

The required output is: Security dashboard. It must be version-controlled, reviewable, testable with the WAN disabled, and linked to the architecture/source-of-truth model. For a payment platform, every implementation decision should make it possible to explain who authorized an action, which facts were trusted, which controls ran, what financial journal committed, and how the result is recovered after failure.

Key Engineering Decisions

- Prioritize actionable alerts
- link to evidence
- separate security freeze from ordinary wallet suspension

Deep Implementation Guidance

Financial interfaces should be explicit about authorization and finality. Users need to distinguish “request accepted for review,” “submitted,” “committed,” “rejected,” and “status unknown.” This reduces support incidents and duplicate retries.

Data and Control Questions

- What is the authoritative source for every input used by this component?
- Which user, device, service, MSP, or administrator identity is allowed to initiate or approve it?
- Does the operation move value, reserve value, create a side effect, or only provide advisory information?
- What idempotency/replay key proves that a retry is the same business intent?
- What evidence is retained off-chain and what stable reference/hash belongs on the ledger?

Chapter 140 Implementation - Security Operations Dashboard

Implementation Procedure

696. Render role-aware screens but enforce permissions server-side.
697. Generate one idempotency key per financial intent.
698. Show pending/committed/rejected/review states explicitly.
699. Recover after refresh/reconnect by querying requestId/transaction status.
700. Keep all JS/CSS/fonts/assets local and show service-health degradation clearly.

Implementation Example

```
# Security Operations Dashboard
INPUT  -> authenticate, validate schema, bind requestId/nonce
FACTS  -> read authoritative wallet/merchant/state data
AI     -> optional bounded recommendation with decisionId/version
POLICY -> enforce ownership, hard limits, approvals, risk fallback
LEDGER -> submit one idempotent deterministic transaction when value changes
EVENT  -> update projection/case/monitoring asynchronously
VERIFY -> discover committed result and retain correlationId + txId evidence
```

Verification Checklist

- The security dashboard exists and is version-controlled.
- The feature works with all public Internet/WAN connectivity disabled.
- Authorization, malformed input, duplicate retry, and failure behavior have explicit tests.
- If financial value changes, journal debits and credits balance exactly and historical entries remain immutable.
- If AI is involved, model/prompt/feature version and fallback behavior are recorded and deterministic policy remains final.
- A timeout/restart cannot create a duplicate financial result; request status can be rediscovered.
- Logs/evidence contain correlation references but not secrets or prohibited sensitive payment data.

Common Failure Modes

- Double-submit.
- Ambiguous pending state.
- Offline runtime assets loaded from CDN.
- Client-calculated totals trusted by server.

Expert Review Questions

- What happens if this component succeeds but the next component fails?
- How is the authorizing identity/device/key revoked or recovered?
- Can the operation be safely retried after the client loses the response?
- What happens if AI is unavailable, slow, or wrong?
- Can an auditor reconstruct debit, credit, actor, risk decision, approval, and Fabric txId from retained evidence?
- Which compensating transaction resolves a downstream operational error without rewriting history?

Chapter 141 - Audit Explorer

Allow authorized auditors to trace journal/payment/risk/approval history. Wallet and merchant applications should make transaction state explicit: draft, authorized, risk-reviewed, submitted, committed, rejected, reversed, or pending human review. A polished UI never substitutes for server-side authorization and ledger enforcement.

Design rule: Search by txId requestId wallet reference merchant case

Implementation Objective

The required output is: Audit UI. It must be version-controlled, reviewable, testable with the WAN disabled, and linked to the architecture/source-of-truth model. For a payment platform, every implementation decision should make it possible to explain who authorized an action, which facts were trusted, which controls ran, what financial journal committed, and how the result is recovered after failure.

Key Engineering Decisions

- Search by txId requestId wallet reference merchant case
- show balanced lines and immutable references
- redact PII

Deep Implementation Guidance

Financial interfaces should be explicit about authorization and finality. Users need to distinguish “request accepted for review,” “submitted,” “committed,” “rejected,” and “status unknown.” This reduces support incidents and duplicate retries.

Data and Control Questions

- What is the authoritative source for every input used by this component?
- Which user, device, service, MSP, or administrator identity is allowed to initiate or approve it?
- Does the operation move value, reserve value, create a side effect, or only provide advisory information?
- What idempotency/replay key proves that a retry is the same business intent?
- What evidence is retained off-chain and what stable reference/hash belongs on the ledger?

Chapter 141 Implementation - Audit Explorer

Implementation Procedure

701. Render role-aware screens but enforce permissions server-side.
702. Generate one idempotency key per financial intent.
703. Show pending/committed/rejected/review states explicitly.
704. Recover after refresh/reconnect by querying requestId/transaction status.
705. Keep all JS/CSS/fonts/assets local and show service-health degradation clearly.

Implementation Example

```
# Audit Explorer
INPUT  -> authenticate, validate schema, bind requestId/nonce
FACTS  -> read authoritative wallet/merchant/state data
AI     -> optional bounded recommendation with decisionId/version
POLICY -> enforce ownership, hard limits, approvals, risk fallback
LEDGER -> submit one idempotent deterministic transaction when value changes
EVENT  -> update projection/case/monitoring asynchronously
VERIFY -> discover committed result and retain correlationId + txId evidence
```

Verification Checklist

- The audit ui exists and is version-controlled.
- The feature works with all public Internet/WAN connectivity disabled.
- Authorization, malformed input, duplicate retry, and failure behavior have explicit tests.
- If financial value changes, journal debits and credits balance exactly and historical entries remain immutable.
- If AI is involved, model/prompt/feature version and fallback behavior are recorded and deterministic policy remains final.
- A timeout/restart cannot create a duplicate financial result; request status can be rediscovered.
- Logs/evidence contain correlation references but not secrets or prohibited sensitive payment data.

Common Failure Modes

- Double-submit.
- Ambiguous pending state.
- Offline runtime assets loaded from CDN.
- Client-calculated totals trusted by server.

Expert Review Questions

- What happens if this component succeeds but the next component fails?
- How is the authorizing identity/device/key revoked or recovered?
- Can the operation be safely retried after the client loses the response?
- What happens if AI is unavailable, slow, or wrong?
- Can an auditor reconstruct debit, credit, actor, risk decision, approval, and Fabric txId from retained evidence?
- Which compensating transaction resolves a downstream operational error without rewriting history?

Chapter 142 - Transaction Receipt

Generate a local receipt with verifiable reference/hash. Wallet and merchant applications should make transaction state explicit: draft, authorized, risk-reviewed, submitted, committed, rejected, reversed, or pending human review. A polished UI never substitutes for server-side authorization and ledger enforcement.

Design rule: Include transaction/business references, amount/asset, merchant, time, status

Implementation Objective

The required output is: Receipt format. It must be version-controlled, reviewable, testable with the WAN disabled, and linked to the architecture/source-of-truth model. For a payment platform, every implementation decision should make it possible to explain who authorized an action, which facts were trusted, which controls ran, what financial journal committed, and how the result is recovered after failure.

Key Engineering Decisions

- Include transaction/business references, amount/asset, merchant, time, status
- avoid secrets
- optionally anchor receipt/document SHA-256

Deep Implementation Guidance

Financial interfaces should be explicit about authorization and finality. Users need to distinguish “request accepted for review,” “submitted,” “committed,” “rejected,” and “status unknown.” This reduces support incidents and duplicate retries.

Data and Control Questions

- What is the authoritative source for every input used by this component?
- Which user, device, service, MSP, or administrator identity is allowed to initiate or approve it?
- Does the operation move value, reserve value, create a side effect, or only provide advisory information?
- What idempotency/replay key proves that a retry is the same business intent?
- What evidence is retained off-chain and what stable reference/hash belongs on the ledger?

Chapter 142 Implementation - Transaction Receipt

Implementation Procedure

- 706. Render role-aware screens but enforce permissions server-side.
- 707. Generate one idempotency key per financial intent.
- 708. Show pending/committed/rejected/review states explicitly.
- 709. Recover after refresh/reconnect by querying requestId/transaction status.
- 710. Keep all JS/CSS/fonts/assets local and show service-health degradation clearly.

Implementation Example

```
# Transaction Receipt
INPUT  -> authenticate, validate schema, bind requestId/nonce
FACTS  -> read authoritative wallet/merchant/state data
AI     -> optional bounded recommendation with decisionId/version
POLICY -> enforce ownership, hard limits, approvals, risk fallback
LEDGER -> submit one idempotent deterministic transaction when value changes
EVENT  -> update projection/case/monitoring asynchronously
VERIFY -> discover committed result and retain correlationId + txId evidence
```

Verification Checklist

- The receipt format exists and is version-controlled.
- The feature works with all public Internet/WAN connectivity disabled.
- Authorization, malformed input, duplicate retry, and failure behavior have explicit tests.
- If financial value changes, journal debits and credits balance exactly and historical entries remain immutable.
- If AI is involved, model/prompt/feature version and fallback behavior are recorded and deterministic policy remains final.
- A timeout/restart cannot create a duplicate financial result; request status can be rediscovered.
- Logs/evidence contain correlation references but not secrets or prohibited sensitive payment data.

Common Failure Modes

- Double-submit.
- Ambiguous pending state.
- Offline runtime assets loaded from CDN.
- Client-calculated totals trusted by server.

Expert Review Questions

- What happens if this component succeeds but the next component fails?
- How is the authorizing identity/device/key revoked or recovered?
- Can the operation be safely retried after the client loses the response?
- What happens if AI is unavailable, slow, or wrong?
- Can an auditor reconstruct debit, credit, actor, risk decision, approval, and Fabric txId from retained evidence?
- Which compensating transaction resolves a downstream operational error without rewriting history?

Chapter 143 - Pending/Unknown Transaction Recovery UX

Handle browser/network timeout without encouraging duplicate payment. Wallet and merchant applications should make transaction state explicit: draft, authorized, risk-reviewed, submitted, committed, rejected, reversed, or pending human review. A polished UI never substitutes for server-side authorization and ledger enforcement.

Design rule: Query by requestId

Implementation Objective

The required output is: Recovery screen. It must be version-controlled, reviewable, testable with the WAN disabled, and linked to the architecture/source-of-truth model. For a payment platform, every implementation decision should make it possible to explain who authorized an action, which facts were trusted, which controls ran, what financial journal committed, and how the result is recovered after failure.

Key Engineering Decisions

- Query by requestId
- explain pending vs rejected
- allow retry only when policy says no committed result exists

Deep Implementation Guidance

Financial interfaces should be explicit about authorization and finality. Users need to distinguish “request accepted for review,” “submitted,” “committed,” “rejected,” and “status unknown.” This reduces support incidents and duplicate retries.

Data and Control Questions

- What is the authoritative source for every input used by this component?
- Which user, device, service, MSP, or administrator identity is allowed to initiate or approve it?
- Does the operation move value, reserve value, create a side effect, or only provide advisory information?
- What idempotency/replay key proves that a retry is the same business intent?
- What evidence is retained off-chain and what stable reference/hash belongs on the ledger?

Chapter 143 Implementation - Pending/Unknown Transaction Recovery UX

Implementation Procedure

- 711. Render role-aware screens but enforce permissions server-side.
- 712. Generate one idempotency key per financial intent.
- 713. Show pending/committed/rejected/review states explicitly.
- 714. Recover after refresh/reconnect by querying requestId/transaction status.
- 715. Keep all JS/CSS/fonts/assets local and show service-health degradation clearly.

Implementation Example

```
# Pending/Unknown Transaction Recovery UX
INPUT  -> authenticate, validate schema, bind requestId/nonce
FACTS  -> read authoritative wallet/merchant/state data
AI     -> optional bounded recommendation with decisionId/version
POLICY -> enforce ownership, hard limits, approvals, risk fallback
LEDGER -> submit one idempotent deterministic transaction when value changes
EVENT  -> update projection/case/monitoring asynchronously
VERIFY -> discover committed result and retain correlationId + txId evidence
```

Verification Checklist

- The recovery screen exists and is version-controlled.
- The feature works with all public Internet/WAN connectivity disabled.
- Authorization, malformed input, duplicate retry, and failure behavior have explicit tests.
- If financial value changes, journal debits and credits balance exactly and historical entries remain immutable.
- If AI is involved, model/prompt/feature version and fallback behavior are recorded and deterministic policy remains final.
- A timeout/restart cannot create a duplicate financial result; request status can be rediscovered.
- Logs/evidence contain correlation references but not secrets or prohibited sensitive payment data.

Common Failure Modes

- Double-submit.
- Ambiguous pending state.
- Offline runtime assets loaded from CDN.
- Client-calculated totals trusted by server.

Expert Review Questions

- What happens if this component succeeds but the next component fails?
- How is the authorizing identity/device/key revoked or recovered?
- Can the operation be safely retried after the client loses the response?
- What happens if AI is unavailable, slow, or wrong?
- Can an auditor reconstruct debit, credit, actor, risk decision, approval, and Fabric txId from retained evidence?
- Which compensating transaction resolves a downstream operational error without rewriting history?

Chapter 144 - Accessibility and Operator Error Prevention

Design financial screens to reduce accidental actions. Wallet and merchant applications should make transaction state explicit: draft, authorized, risk-reviewed, submitted, committed, rejected, reversed, or pending human review. A polished UI never substitutes for server-side authorization and ledger enforcement.

Design rule: Clear amounts/asset

Implementation Objective

The required output is: UX safety checklist. It must be version-controlled, reviewable, testable with the WAN disabled, and linked to the architecture/source-of-truth model. For a payment platform, every implementation decision should make it possible to explain who authorized an action, which facts were trusted, which controls ran, what financial journal committed, and how the result is recovered after failure.

Key Engineering Decisions

- Clear amounts/asset
- large confirmation for irreversible actions
- keyboard/scanner support
- role-aware warnings

Deep Implementation Guidance

Financial interfaces should be explicit about authorization and finality. Users need to distinguish “request accepted for review,” “submitted,” “committed,” “rejected,” and “status unknown.” This reduces support incidents and duplicate retries.

Data and Control Questions

- What is the authoritative source for every input used by this component?
- Which user, device, service, MSP, or administrator identity is allowed to initiate or approve it?
- Does the operation move value, reserve value, create a side effect, or only provide advisory information?
- What idempotency/replay key proves that a retry is the same business intent?
- What evidence is retained off-chain and what stable reference/hash belongs on the ledger?

Chapter 144 Implementation - Accessibility and Operator Error Prevention

Implementation Procedure

- 716. Render role-aware screens but enforce permissions server-side.
- 717. Generate one idempotency key per financial intent.
- 718. Show pending/committed/rejected/review states explicitly.
- 719. Recover after refresh/reconnect by querying requestId/transaction status.
- 720. Keep all JS/CSS/fonts/assets local and show service-health degradation clearly.

Implementation Example

```
# Accessibility and Operator Error Prevention
INPUT  -> authenticate, validate schema, bind requestId/nonce
FACTS  -> read authoritative wallet/merchant/state data
AI     -> optional bounded recommendation with decisionId/version
POLICY -> enforce ownership, hard limits, approvals, risk fallback
LEDGER -> submit one idempotent deterministic transaction when value changes
EVENT  -> update projection/case/monitoring asynchronously
VERIFY -> discover committed result and retain correlationId + txId evidence
```

Verification Checklist

- The ux safety checklist exists and is version-controlled.
- The feature works with all public Internet/WAN connectivity disabled.
- Authorization, malformed input, duplicate retry, and failure behavior have explicit tests.
- If financial value changes, journal debits and credits balance exactly and historical entries remain immutable.
- If AI is involved, model/prompt/feature version and fallback behavior are recorded and deterministic policy remains final.
- A timeout/restart cannot create a duplicate financial result; request status can be rediscovered.
- Logs/evidence contain correlation references but not secrets or prohibited sensitive payment data.

Common Failure Modes

- Double-submit.
- Ambiguous pending state.
- Offline runtime assets loaded from CDN.
- Client-calculated totals trusted by server.

Expert Review Questions

- What happens if this component succeeds but the next component fails?
- How is the authorizing identity/device/key revoked or recovered?
- Can the operation be safely retried after the client loses the response?
- What happens if AI is unavailable, slow, or wrong?
- Can an auditor reconstruct debit, credit, actor, risk decision, approval, and Fabric txId from retained evidence?
- Which compensating transaction resolves a downstream operational error without rewriting history?

Chapter 145 - Degraded Mode UX

Communicate local component failures safely. Wallet and merchant applications should make transaction state explicit: draft, authorized, risk-reviewed, submitted, committed, rejected, reversed, or pending human review. A polished UI never substitutes for server-side authorization and ledger enforcement.

Design rule: If AI unavailable use configured rules/review policy

Implementation Objective

The required output is: Degradation matrix. It must be version-controlled, reviewable, testable with the WAN disabled, and linked to the architecture/source-of-truth model. For a payment platform, every implementation decision should make it possible to explain who authorized an action, which facts were trusted, which controls ran, what financial journal committed, and how the result is recovered after failure.

Key Engineering Decisions

- If AI unavailable use configured rules/review policy
- If DB projection unavailable query limited ledger/read state
- If ordering quorum unavailable disable new writes

Deep Implementation Guidance

Financial interfaces should be explicit about authorization and finality. Users need to distinguish “request accepted for review,” “submitted,” “committed,” “rejected,” and “status unknown.” This reduces support incidents and duplicate retries.

Data and Control Questions

- What is the authoritative source for every input used by this component?
- Which user, device, service, MSP, or administrator identity is allowed to initiate or approve it?
- Does the operation move value, reserve value, create a side effect, or only provide advisory information?
- What idempotency/replay key proves that a retry is the same business intent?
- What evidence is retained off-chain and what stable reference/hash belongs on the ledger?

Chapter 145 Implementation - Degraded Mode UX

Implementation Procedure

721. Render role-aware screens but enforce permissions server-side.
722. Generate one idempotency key per financial intent.
723. Show pending/committed/rejected/review states explicitly.
724. Recover after refresh/reconnect by querying requestId/transaction status.
725. Keep all JS/CSS/fonts/assets local and show service-health degradation clearly.

Implementation Example

```
# Degraded Mode UX
INPUT  -> authenticate, validate schema, bind requestId/nonce
FACTS  -> read authoritative wallet/merchant/state data
AI     -> optional bounded recommendation with decisionId/version
POLICY -> enforce ownership, hard limits, approvals, risk fallback
LEDGER -> submit one idempotent deterministic transaction when value changes
EVENT  -> update projection/case/monitoring asynchronously
VERIFY -> discover committed result and retain correlationId + txId evidence
```

Verification Checklist

- The degradation matrix exists and is version-controlled.
- The feature works with all public Internet/WAN connectivity disabled.
- Authorization, malformed input, duplicate retry, and failure behavior have explicit tests.
- If financial value changes, journal debits and credits balance exactly and historical entries remain immutable.
- If AI is involved, model/prompt/feature version and fallback behavior are recorded and deterministic policy remains final.
- A timeout/restart cannot create a duplicate financial result; request status can be rediscovered.
- Logs/evidence contain correlation references but not secrets or prohibited sensitive payment data.

Common Failure Modes

- Double-submit.
- Ambiguous pending state.
- Offline runtime assets loaded from CDN.
- Client-calculated totals trusted by server.

Expert Review Questions

- What happens if this component succeeds but the next component fails?
- How is the authorizing identity/device/key revoked or recovered?
- Can the operation be safely retried after the client loses the response?
- What happens if AI is unavailable, slow, or wrong?
- Can an auditor reconstruct debit, credit, actor, risk decision, approval, and Fabric txId from retained evidence?
- Which compensating transaction resolves a downstream operational error without rewriting history?

Part IX - AI Architecture, Governance, and Decision Contracts

AI operates beside the ledger, not inside it. Fraud models, anomaly detectors, local LLMs, and RAG assistants produce versioned recommendations and evidence. Deterministic policy and human approval decide whether sensitive financial actions may proceed.

Part focus: This part contains 15 implementation chapters. Each chapter includes design rules, implementation procedure, a practical artifact, verification, failure analysis, and expert review questions.

Chapter 146 - Why AI Must Stay Outside Chaincode

Keep financial consensus deterministic while still using local intelligence. AI operates beside the ledger, not inside it. Fraud models, anomaly detectors, local LLMs, and RAG assistants produce versioned recommendations and evidence. Deterministic policy and human approval decide whether sensitive financial actions may proceed.

Design rule: No model inference/HTTP calls from chaincode

Implementation Objective

The required output is: AI boundary ADR. It must be version-controlled, reviewable, testable with the WAN disabled, and linked to the architecture/source-of-truth model. For a payment platform, every implementation decision should make it possible to explain who authorized an action, which facts were trusted, which controls ran, what financial journal committed, and how the result is recovered after failure.

Key Engineering Decisions

- No model inference/HTTP calls from chaincode
- AI result is an input reference/evidence to the deterministic transaction path
- Ledger commit remains reproducible

Deep Implementation Guidance

Every AI output that can influence workflow routing needs a strict contract, version, evaluation result, timeout, fallback, and owner. Record enough evidence to reproduce which model/rules/prompts were used without putting sensitive feature vectors into immutable ledger history.

Data and Control Questions

- What is the authoritative source for every input used by this component?
- Which user, device, service, MSP, or administrator identity is allowed to initiate or approve it?
- Does the operation move value, reserve value, create a side effect, or only provide advisory information?
- What idempotency/replay key proves that a retry is the same business intent?
- What evidence is retained off-chain and what stable reference/hash belongs on the ledger?

Chapter 146 Implementation - Why AI Must Stay Outside Chaincode

Implementation Procedure

- 726. Define the exact advisory decision, input features, schema, owner, and fallback.
- 727. Pin model/prompt/feature versions and record SHA-256 hashes.
- 728. Run in shadow mode and evaluate before enforcement.
- 729. Apply deterministic thresholds and human review for consequential outcomes.
- 730. Log decisionId, evidence, recommendation, final policy result, and ledger txId.

Implementation Example

```
# Why AI Must Stay Outside Chaincode
INPUT  -> authenticate, validate schema, bind requestId/nonce
FACTS  -> read authoritative wallet/merchant/state data
AI     -> optional bounded recommendation with decisionId/version
POLICY -> enforce ownership, hard limits, approvals, risk fallback
LEDGER -> submit one idempotent deterministic transaction when value changes
EVENT  -> update projection/case/monitoring asynchronously
VERIFY -> discover committed result and retain correlationId + txId evidence
```

Verification Checklist

- The ai boundary adr exists and is version-controlled.
- The feature works with all public Internet/WAN connectivity disabled.
- Authorization, malformed input, duplicate retry, and failure behavior have explicit tests.
- If financial value changes, journal debits and credits balance exactly and historical entries remain immutable.
- If AI is involved, model/prompt/feature version and fallback behavior are recorded and deterministic policy remains final.
- A timeout/restart cannot create a duplicate financial result; request status can be rediscovered.
- Logs/evidence contain correlation references but not secrets or prohibited sensitive payment data.

Common Failure Modes

- No shadow evaluation.
- Model output unversioned.
- Model failure blocks every payment without fallback.
- Recommendation directly triggers high-risk ledger write.

Expert Review Questions

- What happens if this component succeeds but the next component fails?
- How is the authorizing identity/device/key revoked or recovered?
- Can the operation be safely retried after the client loses the response?
- What happens if AI is unavailable, slow, or wrong?
- Can an auditor reconstruct debit, credit, actor, risk decision, approval, and Fabric txId from retained evidence?
- Which compensating transaction resolves a downstream operational error without rewriting history?

Chapter 147 - Offline AI Topology

Deploy risk, embeddings, LLM, model registry, and evaluation services entirely on the LAN. AI operates beside the ledger, not inside it. Fraud models, anomaly detectors, local LLMs, and RAG assistants produce versioned recommendations and evidence. Deterministic policy and human approval decide whether sensitive financial actions may proceed.

Design rule: Risk service close to API

Implementation Objective

The required output is: AI architecture diagram. It must be version-controlled, reviewable, testable with the WAN disabled, and linked to the architecture/source-of-truth model. For a payment platform, every implementation decision should make it possible to explain who authorized an action, which facts were trusted, which controls ran, what financial journal committed, and how the result is recovered after failure.

Key Engineering Decisions

- Risk service close to API
- LLM isolated by network policy
- model files in approved local repository
- pgvector stores embeddings locally

Deep Implementation Guidance

Every AI output that can influence workflow routing needs a strict contract, version, evaluation result, timeout, fallback, and owner. Record enough evidence to reproduce which model/rules/prompts were used without putting sensitive feature vectors into immutable ledger history.

Data and Control Questions

- What is the authoritative source for every input used by this component?
- Which user, device, service, MSP, or administrator identity is allowed to initiate or approve it?
- Does the operation move value, reserve value, create a side effect, or only provide advisory information?
- What idempotency/replay key proves that a retry is the same business intent?
- What evidence is retained off-chain and what stable reference/hash belongs on the ledger?

Chapter 147 Implementation - Offline AI Topology

Implementation Procedure

731. Define the exact advisory decision, input features, schema, owner, and fallback.
732. Pin model/prompt/feature versions and record SHA-256 hashes.
733. Run in shadow mode and evaluate before enforcement.
734. Apply deterministic thresholds and human review for consequential outcomes.
735. Log decisionId, evidence, recommendation, final policy result, and ledger txId.

Implementation Example

```
# Offline AI Topology
INPUT  -> authenticate, validate schema, bind requestId/nonce
FACTS  -> read authoritative wallet/merchant/state data
AI     -> optional bounded recommendation with decisionId/version
POLICY -> enforce ownership, hard limits, approvals, risk fallback
LEDGER -> submit one idempotent deterministic transaction when value changes
EVENT  -> update projection/case/monitoring asynchronously
VERIFY -> discover committed result and retain correlationId + txId evidence
```

Verification Checklist

- The ai architecture diagram exists and is version-controlled.
- The feature works with all public Internet/WAN connectivity disabled.
- Authorization, malformed input, duplicate retry, and failure behavior have explicit tests.
- If financial value changes, journal debits and credits balance exactly and historical entries remain immutable.
- If AI is involved, model/prompt/feature version and fallback behavior are recorded and deterministic policy remains final.
- A timeout/restart cannot create a duplicate financial result; request status can be rediscovered.
- Logs/evidence contain correlation references but not secrets or prohibited sensitive payment data.

Common Failure Modes

- No shadow evaluation.
- Model output unversioned.
- Model failure blocks every payment without fallback.
- Recommendation directly triggers high-risk ledger write.

Expert Review Questions

- What happens if this component succeeds but the next component fails?
- How is the authorizing identity/device/key revoked or recovered?
- Can the operation be safely retried after the client loses the response?
- What happens if AI is unavailable, slow, or wrong?
- Can an auditor reconstruct debit, credit, actor, risk decision, approval, and Fabric txId from retained evidence?
- Which compensating transaction resolves a downstream operational error without rewriting history?

Chapter 148 - AI Decision Contract

Standardize every risk recommendation as machine-validated structured data. AI operates beside the ledger, not inside it. Fraud models, anomaly detectors, local LLMs, and RAG assistants produce versioned recommendations and evidence. Deterministic policy and human approval decide whether sensitive financial actions may proceed.

Design rule: decisionId modelVersion featureVersion riskScore recommendation reasonCodes evidenceRefs createdAt

Implementation Objective

The required output is: Risk decision schema. It must be version-controlled, reviewable, testable with the WAN disabled, and linked to the architecture/source-of-truth model. For a payment platform, every implementation decision should make it possible to explain who authorized an action, which facts were trusted, which controls ran, what financial journal committed, and how the result is recovered after failure.

Key Engineering Decisions

- decisionId modelVersion featureVersion riskScore recommendation reasonCodes evidenceRefs createdAt
- No free-form prose in policy gate

Deep Implementation Guidance

Every AI output that can influence workflow routing needs a strict contract, version, evaluation result, timeout, fallback, and owner. Record enough evidence to reproduce which model/rules/prompts were used without putting sensitive feature vectors into immutable ledger history.

Data and Control Questions

- What is the authoritative source for every input used by this component?
- Which user, device, service, MSP, or administrator identity is allowed to initiate or approve it?
- Does the operation move value, reserve value, create a side effect, or only provide advisory information?
- What idempotency/replay key proves that a retry is the same business intent?
- What evidence is retained off-chain and what stable reference/hash belongs on the ledger?

Chapter 148 Implementation - AI Decision Contract

Implementation Procedure

- 736. Define the exact advisory decision, input features, schema, owner, and fallback.
- 737. Pin model/prompt/feature versions and record SHA-256 hashes.
- 738. Run in shadow mode and evaluate before enforcement.
- 739. Apply deterministic thresholds and human review for consequential outcomes.
- 740. Log decisionId, evidence, recommendation, final policy result, and ledger txId.

Implementation Example

```
# AI Decision Contract
INPUT  -> authenticate, validate schema, bind requestId/nonce
FACTS  -> read authoritative wallet/merchant/state data
AI     -> optional bounded recommendation with decisionId/version
POLICY -> enforce ownership, hard limits, approvals, risk fallback
LEDGER -> submit one idempotent deterministic transaction when value changes
EVENT  -> update projection/case/monitoring asynchronously
VERIFY -> discover committed result and retain correlationId + txId evidence
```

Verification Checklist

- The risk decision schema exists and is version-controlled.
- The feature works with all public Internet/WAN connectivity disabled.
- Authorization, malformed input, duplicate retry, and failure behavior have explicit tests.
- If financial value changes, journal debits and credits balance exactly and historical entries remain immutable.
- If AI is involved, model/prompt/feature version and fallback behavior are recorded and deterministic policy remains final.
- A timeout/restart cannot create a duplicate financial result; request status can be rediscovered.
- Logs/evidence contain correlation references but not secrets or prohibited sensitive payment data.

Common Failure Modes

- No shadow evaluation.
- Model output unversioned.
- Model failure blocks every payment without fallback.
- Recommendation directly triggers high-risk ledger write.

Expert Review Questions

- What happens if this component succeeds but the next component fails?
- How is the authorizing identity/device/key revoked or recovered?
- Can the operation be safely retried after the client loses the response?
- What happens if AI is unavailable, slow, or wrong?
- Can an auditor reconstruct debit, credit, actor, risk decision, approval, and Fabric txId from retained evidence?
- Which compensating transaction resolves a downstream operational error without rewriting history?

Chapter 149 - Feature Store Boundary

Define which features may be used, how they are computed, and how they are versioned. AI operates beside the ledger, not inside it. Fraud models, anomaly detectors, local LLMs, and RAG assistants produce versioned recommendations and evidence. Deterministic policy and human approval decide whether sensitive financial actions may proceed.

Design rule: Use authoritative transaction/device/merchant facts

Implementation Objective

The required output is: Feature contract. It must be version-controlled, reviewable, testable with the WAN disabled, and linked to the architecture/source-of-truth model. For a payment platform, every implementation decision should make it possible to explain who authorized an action, which facts were trusted, which controls ran, what financial journal committed, and how the result is recovered after failure.

Key Engineering Decisions

- Use authoritative transaction/device/merchant facts
- avoid secret/irrelevant PII
- record featureVersion and computation time

Deep Implementation Guidance

Every AI output that can influence workflow routing needs a strict contract, version, evaluation result, timeout, fallback, and owner. Record enough evidence to reproduce which model/rules/prompts were used without putting sensitive feature vectors into immutable ledger history.

Data and Control Questions

- What is the authoritative source for every input used by this component?
- Which user, device, service, MSP, or administrator identity is allowed to initiate or approve it?
- Does the operation move value, reserve value, create a side effect, or only provide advisory information?
- What idempotency/replay key proves that a retry is the same business intent?
- What evidence is retained off-chain and what stable reference/hash belongs on the ledger?

Chapter 149 Implementation - Feature Store Boundary

Implementation Procedure

741. Define the exact advisory decision, input features, schema, owner, and fallback.
742. Pin model/prompt/feature versions and record SHA-256 hashes.
743. Run in shadow mode and evaluate before enforcement.
744. Apply deterministic thresholds and human review for consequential outcomes.
745. Log decisionId, evidence, recommendation, final policy result, and ledger txId.

Implementation Example

```
# Feature Store Boundary
INPUT  -> authenticate, validate schema, bind requestId/nonce
FACTS  -> read authoritative wallet/merchant/state data
AI     -> optional bounded recommendation with decisionId/version
POLICY -> enforce ownership, hard limits, approvals, risk fallback
LEDGER -> submit one idempotent deterministic transaction when value changes
EVENT  -> update projection/case/monitoring asynchronously
VERIFY -> discover committed result and retain correlationId + txId evidence
```

Verification Checklist

- The feature contract exists and is version-controlled.
- The feature works with all public Internet/WAN connectivity disabled.
- Authorization, malformed input, duplicate retry, and failure behavior have explicit tests.
- If financial value changes, journal debits and credits balance exactly and historical entries remain immutable.
- If AI is involved, model/prompt/feature version and fallback behavior are recorded and deterministic policy remains final.
- A timeout/restart cannot create a duplicate financial result; request status can be rediscovered.
- Logs/evidence contain correlation references but not secrets or prohibited sensitive payment data.

Common Failure Modes

- No shadow evaluation.
- Model output unversioned.
- Model failure blocks every payment without fallback.
- Recommendation directly triggers high-risk ledger write.

Expert Review Questions

- What happens if this component succeeds but the next component fails?
- How is the authorizing identity/device/key revoked or recovered?
- Can the operation be safely retried after the client loses the response?
- What happens if AI is unavailable, slow, or wrong?
- Can an auditor reconstruct debit, credit, actor, risk decision, approval, and Fabric txId from retained evidence?
- Which compensating transaction resolves a downstream operational error without rewriting history?

Chapter 150 - Model Registry and SHA-256 Anchoring

Make model promotion auditable without putting model binaries on-chain. AI operates beside the ledger, not inside it. Fraud models, anomaly detectors, local LLMs, and RAG assistants produce versioned recommendations and evidence. Deterministic policy and human approval decide whether sensitive financial actions may proceed.

Design rule: Store file hash/version/training metadata/eval result/approver

Implementation Objective

The required output is: Model registry. It must be version-controlled, reviewable, testable with the WAN disabled, and linked to the architecture/source-of-truth model. For a payment platform, every implementation decision should make it possible to explain who authorized an action, which facts were trusted, which controls ran, what financial journal committed, and how the result is recovered after failure.

Key Engineering Decisions

- Store file hash/version/training metadata/eval result/approver
- optionally anchor approved hash to ledger/audit record
- serve only approved models

Deep Implementation Guidance

Every AI output that can influence workflow routing needs a strict contract, version, evaluation result, timeout, fallback, and owner. Record enough evidence to reproduce which model/rules/prompts were used without putting sensitive feature vectors into immutable ledger history.

Data and Control Questions

- What is the authoritative source for every input used by this component?
- Which user, device, service, MSP, or administrator identity is allowed to initiate or approve it?
- Does the operation move value, reserve value, create a side effect, or only provide advisory information?
- What idempotency/replay key proves that a retry is the same business intent?
- What evidence is retained off-chain and what stable reference/hash belongs on the ledger?

Chapter 150 Implementation - Model Registry and SHA-256 Anchoring

Implementation Procedure

- 746. Define the exact advisory decision, input features, schema, owner, and fallback.
- 747. Pin model/prompt/feature versions and record SHA-256 hashes.
- 748. Run in shadow mode and evaluate before enforcement.
- 749. Apply deterministic thresholds and human review for consequential outcomes.
- 750. Log decisionId, evidence, recommendation, final policy result, and ledger txId.

Implementation Example

```
# Model Registry and SHA-256 Anchoring
INPUT  -> authenticate, validate schema, bind requestId/nonce
FACTS  -> read authoritative wallet/merchant/state data
AI     -> optional bounded recommendation with decisionId/version
POLICY -> enforce ownership, hard limits, approvals, risk fallback
LEDGER -> submit one idempotent deterministic transaction when value changes
EVENT  -> update projection/case/monitoring asynchronously
VERIFY -> discover committed result and retain correlationId + txId evidence
```

Verification Checklist

- The model registry exists and is version-controlled.
- The feature works with all public Internet/WAN connectivity disabled.
- Authorization, malformed input, duplicate retry, and failure behavior have explicit tests.
- If financial value changes, journal debits and credits balance exactly and historical entries remain immutable.
- If AI is involved, model/prompt/feature version and fallback behavior are recorded and deterministic policy remains final.
- A timeout/restart cannot create a duplicate financial result; request status can be rediscovered.
- Logs/evidence contain correlation references but not secrets or prohibited sensitive payment data.

Common Failure Modes

- No shadow evaluation.
- Model output unversioned.
- Model failure blocks every payment without fallback.
- Recommendation directly triggers high-risk ledger write.

Expert Review Questions

- What happens if this component succeeds but the next component fails?
- How is the authorizing identity/device/key revoked or recovered?
- Can the operation be safely retried after the client loses the response?
- What happens if AI is unavailable, slow, or wrong?
- Can an auditor reconstruct debit, credit, actor, risk decision, approval, and Fabric txId from retained evidence?
- Which compensating transaction resolves a downstream operational error without rewriting history?

Chapter 151 - Prompt and RAG Version Registry

Treat prompts, system instructions, schemas, embedding model, and retrieval configuration as deployable artifacts. AI operates beside the ledger, not inside it. Fraud models, anomaly detectors, local LLMs, and RAG assistants produce versioned recommendations and evidence. Deterministic policy and human approval decide whether sensitive financial actions may proceed.

Design rule: Version prompt and schema together

Implementation Objective

The required output is: Prompt/RAG registry. It must be version-controlled, reviewable, testable with the WAN disabled, and linked to the architecture/source-of-truth model. For a payment platform, every implementation decision should make it possible to explain who authorized an action, which facts were trusted, which controls ran, what financial journal committed, and how the result is recovered after failure.

Key Engineering Decisions

- Version prompt and schema together
- hash knowledge corpus snapshot
- record evaluation before promotion

Deep Implementation Guidance

Every AI output that can influence workflow routing needs a strict contract, version, evaluation result, timeout, fallback, and owner. Record enough evidence to reproduce which model/rules/prompts were used without putting sensitive feature vectors into immutable ledger history.

Data and Control Questions

- What is the authoritative source for every input used by this component?
- Which user, device, service, MSP, or administrator identity is allowed to initiate or approve it?
- Does the operation move value, reserve value, create a side effect, or only provide advisory information?
- What idempotency/replay key proves that a retry is the same business intent?
- What evidence is retained off-chain and what stable reference/hash belongs on the ledger?

Chapter 151 Implementation - Prompt and RAG Version Registry

Implementation Procedure

751. Define the exact advisory decision, input features, schema, owner, and fallback.
752. Pin model/prompt/feature versions and record SHA-256 hashes.
753. Run in shadow mode and evaluate before enforcement.
754. Apply deterministic thresholds and human review for consequential outcomes.
755. Log decisionId, evidence, recommendation, final policy result, and ledger txId.

Implementation Example

```
# Prompt and RAG Version Registry
INPUT  -> authenticate, validate schema, bind requestId/nonce
FACTS  -> read authoritative wallet/merchant/state data
AI     -> optional bounded recommendation with decisionId/version
POLICY -> enforce ownership, hard limits, approvals, risk fallback
LEDGER -> submit one idempotent deterministic transaction when value changes
EVENT  -> update projection/case/monitoring asynchronously
VERIFY -> discover committed result and retain correlationId + txId evidence
```

Verification Checklist

- The prompt/rag registry exists and is version-controlled.
- The feature works with all public Internet/WAN connectivity disabled.
- Authorization, malformed input, duplicate retry, and failure behavior have explicit tests.
- If financial value changes, journal debits and credits balance exactly and historical entries remain immutable.
- If AI is involved, model/prompt/feature version and fallback behavior are recorded and deterministic policy remains final.
- A timeout/restart cannot create a duplicate financial result; request status can be rediscovered.
- Logs/evidence contain correlation references but not secrets or prohibited sensitive payment data.

Common Failure Modes

- No shadow evaluation.
- Model output unversioned.
- Model failure blocks every payment without fallback.
- Recommendation directly triggers high-risk ledger write.

Expert Review Questions

- What happens if this component succeeds but the next component fails?
- How is the authorizing identity/device/key revoked or recovered?
- Can the operation be safely retried after the client loses the response?
- What happens if AI is unavailable, slow, or wrong?
- Can an auditor reconstruct debit, credit, actor, risk decision, approval, and Fabric txId from retained evidence?
- Which compensating transaction resolves a downstream operational error without rewriting history?

Chapter 152 - Shadow Mode

Evaluate AI recommendations without allowing them to affect money movement. AI operates beside the ledger, not inside it. Fraud models, anomaly detectors, local LLMs, and RAG assistants produce versioned recommendations and evidence. Deterministic policy and human approval decide whether sensitive financial actions may proceed.

Design rule: Run model on live-like decisions

Implementation Objective

The required output is: Shadow deployment. It must be version-controlled, reviewable, testable with the WAN disabled, and linked to the architecture/source-of-truth model. For a payment platform, every implementation decision should make it possible to explain who authorized an action, which facts were trusted, which controls ran, what financial journal committed, and how the result is recovered after failure.

Key Engineering Decisions

- Run model on live-like decisions
- record what it would do
- compare to rules/analyst outcome
- do not block/approve based on model

Deep Implementation Guidance

Every AI output that can influence workflow routing needs a strict contract, version, evaluation result, timeout, fallback, and owner. Record enough evidence to reproduce which model/rules/prompts were used without putting sensitive feature vectors into immutable ledger history.

Data and Control Questions

- What is the authoritative source for every input used by this component?
- Which user, device, service, MSP, or administrator identity is allowed to initiate or approve it?
- Does the operation move value, reserve value, create a side effect, or only provide advisory information?
- What idempotency/replay key proves that a retry is the same business intent?
- What evidence is retained off-chain and what stable reference/hash belongs on the ledger?

Chapter 152 Implementation - Shadow Mode

Implementation Procedure

- 756. Define the exact advisory decision, input features, schema, owner, and fallback.
- 757. Pin model/prompt/feature versions and record SHA-256 hashes.
- 758. Run in shadow mode and evaluate before enforcement.
- 759. Apply deterministic thresholds and human review for consequential outcomes.
- 760. Log decisionId, evidence, recommendation, final policy result, and ledger txId.

Implementation Example

```
# Shadow Mode
INPUT  -> authenticate, validate schema, bind requestId/nonce
FACTS  -> read authoritative wallet/merchant/state data
AI     -> optional bounded recommendation with decisionId/version
POLICY -> enforce ownership, hard limits, approvals, risk fallback
LEDGER -> submit one idempotent deterministic transaction when value changes
EVENT  -> update projection/case/monitoring asynchronously
VERIFY -> discover committed result and retain correlationId + txId evidence
```

Verification Checklist

- The shadow deployment exists and is version-controlled.
- The feature works with all public Internet/WAN connectivity disabled.
- Authorization, malformed input, duplicate retry, and failure behavior have explicit tests.
- If financial value changes, journal debits and credits balance exactly and historical entries remain immutable.
- If AI is involved, model/prompt/feature version and fallback behavior are recorded and deterministic policy remains final.
- A timeout/restart cannot create a duplicate financial result; request status can be rediscovered.
- Logs/evidence contain correlation references but not secrets or prohibited sensitive payment data.

Common Failure Modes

- No shadow evaluation.
- Model output unversioned.
- Model failure blocks every payment without fallback.
- Recommendation directly triggers high-risk ledger write.

Expert Review Questions

- What happens if this component succeeds but the next component fails?
- How is the authorizing identity/device/key revoked or recovered?
- Can the operation be safely retried after the client loses the response?
- What happens if AI is unavailable, slow, or wrong?
- Can an auditor reconstruct debit, credit, actor, risk decision, approval, and Fabric txId from retained evidence?
- Which compensating transaction resolves a downstream operational error without rewriting history?

Chapter 153 - Advisory Mode

Use AI to prioritize review while deterministic policy remains final. AI operates beside the ledger, not inside it. Fraud models, anomaly detectors, local LLMs, and RAG assistants produce versioned recommendations and evidence. Deterministic policy and human approval decide whether sensitive financial actions may proceed.

Design rule: ALLOW/REVIEW/BLOCK are recommendations

Implementation Objective

The required output is: Advisory policy. It must be version-controlled, reviewable, testable with the WAN disabled, and linked to the architecture/source-of-truth model. For a payment platform, every implementation decision should make it possible to explain who authorized an action, which facts were trusted, which controls ran, what financial journal committed, and how the result is recovered after failure.

Key Engineering Decisions

- ALLOW/REVIEW/BLOCK are recommendations
- hard rules can override
- analyst sees reason codes and model version

Deep Implementation Guidance

Every AI output that can influence workflow routing needs a strict contract, version, evaluation result, timeout, fallback, and owner. Record enough evidence to reproduce which model/rules/prompts were used without putting sensitive feature vectors into immutable ledger history.

Data and Control Questions

- What is the authoritative source for every input used by this component?
- Which user, device, service, MSP, or administrator identity is allowed to initiate or approve it?
- Does the operation move value, reserve value, create a side effect, or only provide advisory information?
- What idempotency/replay key proves that a retry is the same business intent?
- What evidence is retained off-chain and what stable reference/hash belongs on the ledger?

Chapter 153 Implementation - Advisory Mode

Implementation Procedure

- 761. Define the exact advisory decision, input features, schema, owner, and fallback.
- 762. Pin model/prompt/feature versions and record SHA-256 hashes.
- 763. Run in shadow mode and evaluate before enforcement.
- 764. Apply deterministic thresholds and human review for consequential outcomes.
- 765. Log decisionId, evidence, recommendation, final policy result, and ledger txId.

Implementation Example

```
# Advisory Mode
INPUT  -> authenticate, validate schema, bind requestId/nonce
FACTS  -> read authoritative wallet/merchant/state data
AI      -> optional bounded recommendation with decisionId/version
POLICY  -> enforce ownership, hard limits, approvals, risk fallback
LEDGER  -> submit one idempotent deterministic transaction when value changes
EVENT   -> update projection/case/monitoring asynchronously
VERIFY  -> discover committed result and retain correlationId + txId evidence
```

Verification Checklist

- The advisory policy exists and is version-controlled.
- The feature works with all public Internet/WAN connectivity disabled.
- Authorization, malformed input, duplicate retry, and failure behavior have explicit tests.
- If financial value changes, journal debits and credits balance exactly and historical entries remain immutable.
- If AI is involved, model/prompt/feature version and fallback behavior are recorded and deterministic policy remains final.
- A timeout/restart cannot create a duplicate financial result; request status can be rediscovered.
- Logs/evidence contain correlation references but not secrets or prohibited sensitive payment data.

Common Failure Modes

- No shadow evaluation.
- Model output unversioned.
- Model failure blocks every payment without fallback.
- Recommendation directly triggers high-risk ledger write.

Expert Review Questions

- What happens if this component succeeds but the next component fails?
- How is the authorizing identity/device/key revoked or recovered?
- Can the operation be safely retried after the client loses the response?
- What happens if AI is unavailable, slow, or wrong?
- Can an auditor reconstruct debit, credit, actor, risk decision, approval, and Fabric txId from retained evidence?
- Which compensating transaction resolves a downstream operational error without rewriting history?

Chapter 154 - Controlled Enforcement

Allow proven AI to influence bounded low-risk decisions under policy. AI operates beside the ledger, not inside it. Fraud models, anomaly detectors, local LLMs, and RAG assistants produce versioned recommendations and evidence. Deterministic policy and human approval decide whether sensitive financial actions may proceed.

Design rule: Define exact scope and thresholds

Implementation Objective

The required output is: Enforcement gate. It must be version-controlled, reviewable, testable with the WAN disabled, and linked to the architecture/source-of-truth model. For a payment platform, every implementation decision should make it possible to explain who authorized an action, which facts were trusted, which controls ran, what financial journal committed, and how the result is recovered after failure.

Key Engineering Decisions

- Define exact scope and thresholds
- maintain hard deny limits
- fallback on service/model uncertainty
- measure false positives continuously

Deep Implementation Guidance

Every AI output that can influence workflow routing needs a strict contract, version, evaluation result, timeout, fallback, and owner. Record enough evidence to reproduce which model/rules/prompts were used without putting sensitive feature vectors into immutable ledger history.

Data and Control Questions

- What is the authoritative source for every input used by this component?
- Which user, device, service, MSP, or administrator identity is allowed to initiate or approve it?
- Does the operation move value, reserve value, create a side effect, or only provide advisory information?
- What idempotency/replay key proves that a retry is the same business intent?
- What evidence is retained off-chain and what stable reference/hash belongs on the ledger?

Chapter 154 Implementation - Controlled Enforcement

Implementation Procedure

- 766. Define the exact advisory decision, input features, schema, owner, and fallback.
- 767. Pin model/prompt/feature versions and record SHA-256 hashes.
- 768. Run in shadow mode and evaluate before enforcement.
- 769. Apply deterministic thresholds and human review for consequential outcomes.
- 770. Log decisionId, evidence, recommendation, final policy result, and ledger txId.

Implementation Example

```
# Controlled Enforcement
INPUT  -> authenticate, validate schema, bind requestId/nonce
FACTS  -> read authoritative wallet/merchant/state data
AI     -> optional bounded recommendation with decisionId/version
POLICY -> enforce ownership, hard limits, approvals, risk fallback
LEDGER -> submit one idempotent deterministic transaction when value changes
EVENT  -> update projection/case/monitoring asynchronously
VERIFY -> discover committed result and retain correlationId + txId evidence
```

Verification Checklist

- The enforcement gate exists and is version-controlled.
- The feature works with all public Internet/WAN connectivity disabled.
- Authorization, malformed input, duplicate retry, and failure behavior have explicit tests.
- If financial value changes, journal debits and credits balance exactly and historical entries remain immutable.
- If AI is involved, model/prompt/feature version and fallback behavior are recorded and deterministic policy remains final.
- A timeout/restart cannot create a duplicate financial result; request status can be rediscovered.
- Logs/evidence contain correlation references but not secrets or prohibited sensitive payment data.

Common Failure Modes

- No shadow evaluation.
- Model output unversioned.
- Model failure blocks every payment without fallback.
- Recommendation directly triggers high-risk ledger write.

Expert Review Questions

- What happens if this component succeeds but the next component fails?
- How is the authorizing identity/device/key revoked or recovered?
- Can the operation be safely retried after the client loses the response?
- What happens if AI is unavailable, slow, or wrong?
- Can an auditor reconstruct debit, credit, actor, risk decision, approval, and Fabric txId from retained evidence?
- Which compensating transaction resolves a downstream operational error without rewriting history?

Chapter 155 - Human Review Queue

Pause consequential ambiguous decisions for accountable human action. AI operates beside the ledger, not inside it. Fraud models, anomaly detectors, local LLMs, and RAG assistants produce versioned recommendations and evidence. Deterministic policy and human approval decide whether sensitive financial actions may proceed.

Design rule: Review item contains immutable request snapshot/evidence

Implementation Objective

The required output is: Review workflow. It must be version-controlled, reviewable, testable with the WAN disabled, and linked to the architecture/source-of-truth model. For a payment platform, every implementation decision should make it possible to explain who authorized an action, which facts were trusted, which controls ran, what financial journal committed, and how the result is recovered after failure.

Key Engineering Decisions

- Review item contains immutable request snapshot/evidence
- analyst decision reason required
- approval expires and is linked to resulting ledger command

Deep Implementation Guidance

Every AI output that can influence workflow routing needs a strict contract, version, evaluation result, timeout, fallback, and owner. Record enough evidence to reproduce which model/rules/prompts were used without putting sensitive feature vectors into immutable ledger history.

Data and Control Questions

- What is the authoritative source for every input used by this component?
- Which user, device, service, MSP, or administrator identity is allowed to initiate or approve it?
- Does the operation move value, reserve value, create a side effect, or only provide advisory information?
- What idempotency/replay key proves that a retry is the same business intent?
- What evidence is retained off-chain and what stable reference/hash belongs on the ledger?

Chapter 155 Implementation - Human Review Queue

Implementation Procedure

- 771. Define the exact advisory decision, input features, schema, owner, and fallback.
- 772. Pin model/prompt/feature versions and record SHA-256 hashes.
- 773. Run in shadow mode and evaluate before enforcement.
- 774. Apply deterministic thresholds and human review for consequential outcomes.
- 775. Log decisionId, evidence, recommendation, final policy result, and ledger txId.

Implementation Example

```
# Human Review Queue
INPUT  -> authenticate, validate schema, bind requestId/nonce
FACTS  -> read authoritative wallet/merchant/state data
AI     -> optional bounded recommendation with decisionId/version
POLICY -> enforce ownership, hard limits, approvals, risk fallback
LEDGER -> submit one idempotent deterministic transaction when value changes
EVENT  -> update projection/case/monitoring asynchronously
VERIFY -> discover committed result and retain correlationId + txId evidence
```

Verification Checklist

- The review workflow exists and is version-controlled.
- The feature works with all public Internet/WAN connectivity disabled.
- Authorization, malformed input, duplicate retry, and failure behavior have explicit tests.
- If financial value changes, journal debits and credits balance exactly and historical entries remain immutable.
- If AI is involved, model/prompt/feature version and fallback behavior are recorded and deterministic policy remains final.
- A timeout/restart cannot create a duplicate financial result; request status can be rediscovered.
- Logs/evidence contain correlation references but not secrets or prohibited sensitive payment data.

Common Failure Modes

- No shadow evaluation.
- Model output unversioned.
- Model failure blocks every payment without fallback.
- Recommendation directly triggers high-risk ledger write.

Expert Review Questions

- What happens if this component succeeds but the next component fails?
- How is the authorizing identity/device/key revoked or recovered?
- Can the operation be safely retried after the client loses the response?
- What happens if AI is unavailable, slow, or wrong?
- Can an auditor reconstruct debit, credit, actor, risk decision, approval, and Fabric txId from retained evidence?
- Which compensating transaction resolves a downstream operational error without rewriting history?

Chapter 156 - AI Fallback Matrix

Define what happens when model/LLM/vector DB is unavailable or slow. AI operates beside the ledger, not inside it. Fraud models, anomaly detectors, local LLMs, and RAG assistants produce versioned recommendations and evidence. Deterministic policy and human approval decide whether sensitive financial actions may proceed.

Design rule: Core accounting continues with deterministic controls

Implementation Objective

The required output is: Fallback policy. It must be version-controlled, reviewable, testable with the WAN disabled, and linked to the architecture/source-of-truth model. For a payment platform, every implementation decision should make it possible to explain who authorized an action, which facts were trusted, which controls ran, what financial journal committed, and how the result is recovered after failure.

Key Engineering Decisions

- Core accounting continues with deterministic controls
- high-risk may route to REVIEW
- support assistant can degrade without blocking payments

Deep Implementation Guidance

Every AI output that can influence workflow routing needs a strict contract, version, evaluation result, timeout, fallback, and owner. Record enough evidence to reproduce which model/rules/prompts were used without putting sensitive feature vectors into immutable ledger history.

Data and Control Questions

- What is the authoritative source for every input used by this component?
- Which user, device, service, MSP, or administrator identity is allowed to initiate or approve it?
- Does the operation move value, reserve value, create a side effect, or only provide advisory information?
- What idempotency/replay key proves that a retry is the same business intent?
- What evidence is retained off-chain and what stable reference/hash belongs on the ledger?

Chapter 156 Implementation - AI Fallback Matrix

Implementation Procedure

- 776. Define the exact advisory decision, input features, schema, owner, and fallback.
- 777. Pin model/prompt/feature versions and record SHA-256 hashes.
- 778. Run in shadow mode and evaluate before enforcement.
- 779. Apply deterministic thresholds and human review for consequential outcomes.
- 780. Log decisionId, evidence, recommendation, final policy result, and ledger txId.

Implementation Example

```
# AI Fallback Matrix
INPUT  -> authenticate, validate schema, bind requestId/nonce
FACTS  -> read authoritative wallet/merchant/state data
AI      -> optional bounded recommendation with decisionId/version
POLICY  -> enforce ownership, hard limits, approvals, risk fallback
LEDGER  -> submit one idempotent deterministic transaction when value changes
EVENT   -> update projection/case/monitoring asynchronously
VERIFY  -> discover committed result and retain correlationId + txId evidence
```

Verification Checklist

- The fallback policy exists and is version-controlled.
- The feature works with all public Internet/WAN connectivity disabled.
- Authorization, malformed input, duplicate retry, and failure behavior have explicit tests.
- If financial value changes, journal debits and credits balance exactly and historical entries remain immutable.
- If AI is involved, model/prompt/feature version and fallback behavior are recorded and deterministic policy remains final.
- A timeout/restart cannot create a duplicate financial result; request status can be rediscovered.
- Logs/evidence contain correlation references but not secrets or prohibited sensitive payment data.

Common Failure Modes

- No shadow evaluation.
- Model output unversioned.
- Model failure blocks every payment without fallback.
- Recommendation directly triggers high-risk ledger write.

Expert Review Questions

- What happens if this component succeeds but the next component fails?
- How is the authorizing identity/device/key revoked or recovered?
- Can the operation be safely retried after the client loses the response?
- What happens if AI is unavailable, slow, or wrong?
- Can an auditor reconstruct debit, credit, actor, risk decision, approval, and Fabric txId from retained evidence?
- Which compensating transaction resolves a downstream operational error without rewriting history?

Chapter 157 - AI Data Retention

Limit storage of model inputs/outputs to what operations and audit need. AI operates beside the ledger, not inside it. Fraud models, anomaly detectors, local LLMs, and RAG assistants produce versioned recommendations and evidence. Deterministic policy and human approval decide whether sensitive financial actions may proceed.

Design rule: Minimize PII

Implementation Objective

The required output is: Retention policy. It must be version-controlled, reviewable, testable with the WAN disabled, and linked to the architecture/source-of-truth model. For a payment platform, every implementation decision should make it possible to explain who authorized an action, which facts were trusted, which controls ran, what financial journal committed, and how the result is recovered after failure.

Key Engineering Decisions

- Minimize PII
- separate training dataset from production logs
- define deletion/retention for off-chain AI records

Deep Implementation Guidance

Every AI output that can influence workflow routing needs a strict contract, version, evaluation result, timeout, fallback, and owner. Record enough evidence to reproduce which model/rules/prompts were used without putting sensitive feature vectors into immutable ledger history.

Data and Control Questions

- What is the authoritative source for every input used by this component?
- Which user, device, service, MSP, or administrator identity is allowed to initiate or approve it?
- Does the operation move value, reserve value, create a side effect, or only provide advisory information?
- What idempotency/replay key proves that a retry is the same business intent?
- What evidence is retained off-chain and what stable reference/hash belongs on the ledger?

Chapter 157 Implementation - AI Data Retention

Implementation Procedure

781. Define the exact advisory decision, input features, schema, owner, and fallback.
782. Pin model/prompt/feature versions and record SHA-256 hashes.
783. Run in shadow mode and evaluate before enforcement.
784. Apply deterministic thresholds and human review for consequential outcomes.
785. Log decisionId, evidence, recommendation, final policy result, and ledger txId.

Implementation Example

```
# AI Data Retention
INPUT  -> authenticate, validate schema, bind requestId/nonce
FACTS  -> read authoritative wallet/merchant/state data
AI     -> optional bounded recommendation with decisionId/version
POLICY -> enforce ownership, hard limits, approvals, risk fallback
LEDGER -> submit one idempotent deterministic transaction when value changes
EVENT  -> update projection/case/monitoring asynchronously
VERIFY -> discover committed result and retain correlationId + txId evidence
```

Verification Checklist

- The retention policy exists and is version-controlled.
- The feature works with all public Internet/WAN connectivity disabled.
- Authorization, malformed input, duplicate retry, and failure behavior have explicit tests.
- If financial value changes, journal debits and credits balance exactly and historical entries remain immutable.
- If AI is involved, model/prompt/feature version and fallback behavior are recorded and deterministic policy remains final.
- A timeout/restart cannot create a duplicate financial result; request status can be rediscovered.
- Logs/evidence contain correlation references but not secrets or prohibited sensitive payment data.

Common Failure Modes

- No shadow evaluation.
- Model output unversioned.
- Model failure blocks every payment without fallback.
- Recommendation directly triggers high-risk ledger write.

Expert Review Questions

- What happens if this component succeeds but the next component fails?
- How is the authorizing identity/device/key revoked or recovered?
- Can the operation be safely retried after the client loses the response?
- What happens if AI is unavailable, slow, or wrong?
- Can an auditor reconstruct debit, credit, actor, risk decision, approval, and Fabric txId from retained evidence?
- Which compensating transaction resolves a downstream operational error without rewriting history?

Chapter 158 - AI Change Management

Promote model/prompt/feature changes like financial software changes. AI operates beside the ledger, not inside it. Fraud models, anomaly detectors, local LLMs, and RAG assistants produce versioned recommendations and evidence. Deterministic policy and human approval decide whether sensitive financial actions may proceed.

Design rule: Staging eval

Implementation Objective

The required output is: AI release workflow. It must be version-controlled, reviewable, testable with the WAN disabled, and linked to the architecture/source-of-truth model. For a payment platform, every implementation decision should make it possible to explain who authorized an action, which facts were trusted, which controls ran, what financial journal committed, and how the result is recovered after failure.

Key Engineering Decisions

- Staging eval
- security/adversarial tests
- approval
- version pin
- canary/shadow
- rollback package

Deep Implementation Guidance

Every AI output that can influence workflow routing needs a strict contract, version, evaluation result, timeout, fallback, and owner. Record enough evidence to reproduce which model/rules/prompts were used without putting sensitive feature vectors into immutable ledger history.

Data and Control Questions

- What is the authoritative source for every input used by this component?
- Which user, device, service, MSP, or administrator identity is allowed to initiate or approve it?
- Does the operation move value, reserve value, create a side effect, or only provide advisory information?
- What idempotency/replay key proves that a retry is the same business intent?
- What evidence is retained off-chain and what stable reference/hash belongs on the ledger?

Chapter 158 Implementation - AI Change Management

Implementation Procedure

- 786. Define the exact advisory decision, input features, schema, owner, and fallback.
- 787. Pin model/prompt/feature versions and record SHA-256 hashes.
- 788. Run in shadow mode and evaluate before enforcement.
- 789. Apply deterministic thresholds and human review for consequential outcomes.
- 790. Log decisionId, evidence, recommendation, final policy result, and ledger txId.

Implementation Example

```
# AI Change Management
INPUT  -> authenticate, validate schema, bind requestId/nonce
FACTS  -> read authoritative wallet/merchant/state data
AI     -> optional bounded recommendation with decisionId/version
POLICY -> enforce ownership, hard limits, approvals, risk fallback
LEDGER -> submit one idempotent deterministic transaction when value changes
EVENT  -> update projection/case/monitoring asynchronously
VERIFY -> discover committed result and retain correlationId + txId evidence
```

Verification Checklist

- The ai release workflow exists and is version-controlled.
- The feature works with all public Internet/WAN connectivity disabled.
- Authorization, malformed input, duplicate retry, and failure behavior have explicit tests.
- If financial value changes, journal debits and credits balance exactly and historical entries remain immutable.
- If AI is involved, model/prompt/feature version and fallback behavior are recorded and deterministic policy remains final.
- A timeout/restart cannot create a duplicate financial result; request status can be rediscovered.
- Logs/evidence contain correlation references but not secrets or prohibited sensitive payment data.

Common Failure Modes

- No shadow evaluation.
- Model output unversioned.
- Model failure blocks every payment without fallback.
- Recommendation directly triggers high-risk ledger write.

Expert Review Questions

- What happens if this component succeeds but the next component fails?
- How is the authorizing identity/device/key revoked or recovered?
- Can the operation be safely retried after the client loses the response?
- What happens if AI is unavailable, slow, or wrong?
- Can an auditor reconstruct debit, credit, actor, risk decision, approval, and Fabric txId from retained evidence?
- Which compensating transaction resolves a downstream operational error without rewriting history?

Chapter 159 - AI Governance Using NIST AI RMF Concepts

Organize governance, mapping, measurement, and management of AI risk. AI operates beside the ledger, not inside it. Fraud models, anomaly detectors, local LLMs, and RAG assistants produce versioned recommendations and evidence. Deterministic policy and human approval decide whether sensitive financial actions may proceed.

Design rule: Assign owners

Implementation Objective

The required output is: AI governance register. It must be version-controlled, reviewable, testable with the WAN disabled, and linked to the architecture/source-of-truth model. For a payment platform, every implementation decision should make it possible to explain who authorized an action, which facts were trusted, which controls ran, what financial journal committed, and how the result is recovered after failure.

Key Engineering Decisions

- Assign owners
- document intended use and unacceptable use
- measure model/workflow risks
- maintain response/rollback plans

Deep Implementation Guidance

Every AI output that can influence workflow routing needs a strict contract, version, evaluation result, timeout, fallback, and owner. Record enough evidence to reproduce which model/rules/prompts were used without putting sensitive feature vectors into immutable ledger history.

Data and Control Questions

- What is the authoritative source for every input used by this component?
- Which user, device, service, MSP, or administrator identity is allowed to initiate or approve it?
- Does the operation move value, reserve value, create a side effect, or only provide advisory information?
- What idempotency/replay key proves that a retry is the same business intent?
- What evidence is retained off-chain and what stable reference/hash belongs on the ledger?

Chapter 159 Implementation - AI Governance Using NIST AI RMF Concepts

Implementation Procedure

791. Define the exact advisory decision, input features, schema, owner, and fallback.
792. Pin model/prompt/feature versions and record SHA-256 hashes.
793. Run in shadow mode and evaluate before enforcement.
794. Apply deterministic thresholds and human review for consequential outcomes.
795. Log decisionId, evidence, recommendation, final policy result, and ledger txId.

Implementation Example

```
# AI Governance Using NIST AI RMF Concepts
INPUT  -> authenticate, validate schema, bind requestId/nonce
FACTS  -> read authoritative wallet/merchant/state data
AI     -> optional bounded recommendation with decisionId/version
POLICY -> enforce ownership, hard limits, approvals, risk fallback
LEDGER -> submit one idempotent deterministic transaction when value changes
EVENT  -> update projection/case/monitoring asynchronously
VERIFY -> discover committed result and retain correlationId + txId evidence
```

Verification Checklist

- The ai governance register exists and is version-controlled.
- The feature works with all public Internet/WAN connectivity disabled.
- Authorization, malformed input, duplicate retry, and failure behavior have explicit tests.
- If financial value changes, journal debits and credits balance exactly and historical entries remain immutable.
- If AI is involved, model/prompt/feature version and fallback behavior are recorded and deterministic policy remains final.
- A timeout/restart cannot create a duplicate financial result; request status can be rediscovered.
- Logs/evidence contain correlation references but not secrets or prohibited sensitive payment data.

Common Failure Modes

- No shadow evaluation.
- Model output unversioned.
- Model failure blocks every payment without fallback.
- Recommendation directly triggers high-risk ledger write.

Expert Review Questions

- What happens if this component succeeds but the next component fails?
- How is the authorizing identity/device/key revoked or recovered?
- Can the operation be safely retried after the client loses the response?
- What happens if AI is unavailable, slow, or wrong?
- Can an auditor reconstruct debit, credit, actor, risk decision, approval, and Fabric txId from retained evidence?
- Which compensating transaction resolves a downstream operational error without rewriting history?

Chapter 160 - AI Evidence Linked to Ledger

Allow auditors to reconstruct the advisory context for a payment without storing sensitive model internals on-chain. AI operates beside the ledger, not inside it. Fraud models, anomaly detectors, local LLMs, and RAG assistants produce versioned recommendations and evidence. Deterministic policy and human approval decide whether sensitive financial actions may proceed.

Design rule: Store decisionId/modelVersion/risk recommendation/authDigest/approvalRef on ledger as needed

Implementation Objective

The required output is: Evidence linkage design. It must be version-controlled, reviewable, testable with the WAN disabled, and linked to the architecture/source-of-truth model. For a payment platform, every implementation decision should make it possible to explain who authorized an action, which facts were trusted, which controls ran, what financial journal committed, and how the result is recovered after failure.

Key Engineering Decisions

- Store decisionId/modelVersion/risk recommendation/authDigest/approvalRef on ledger as needed
- detailed features remain access-controlled off-chain

Deep Implementation Guidance

Every AI output that can influence workflow routing needs a strict contract, version, evaluation result, timeout, fallback, and owner. Record enough evidence to reproduce which model/rules/prompts were used without putting sensitive feature vectors into immutable ledger history.

Data and Control Questions

- What is the authoritative source for every input used by this component?
- Which user, device, service, MSP, or administrator identity is allowed to initiate or approve it?
- Does the operation move value, reserve value, create a side effect, or only provide advisory information?
- What idempotency/replay key proves that a retry is the same business intent?
- What evidence is retained off-chain and what stable reference/hash belongs on the ledger?

Chapter 160 Implementation - AI Evidence Linked to Ledger

Implementation Procedure

796. Define the exact advisory decision, input features, schema, owner, and fallback.
797. Pin model/prompt/feature versions and record SHA-256 hashes.
798. Run in shadow mode and evaluate before enforcement.
799. Apply deterministic thresholds and human review for consequential outcomes.
800. Log decisionId, evidence, recommendation, final policy result, and ledger txId.

Implementation Example

```
# AI Evidence Linked to Ledger
INPUT  -> authenticate, validate schema, bind requestId/nonce
FACTS  -> read authoritative wallet/merchant/state data
AI      -> optional bounded recommendation with decisionId/version
POLICY  -> enforce ownership, hard limits, approvals, risk fallback
LEDGER  -> submit one idempotent deterministic transaction when value changes
EVENT   -> update projection/case/monitoring asynchronously
VERIFY  -> discover committed result and retain correlationId + txId evidence
```

Verification Checklist

- The evidence linkage design exists and is version-controlled.
- The feature works with all public Internet/WAN connectivity disabled.
- Authorization, malformed input, duplicate retry, and failure behavior have explicit tests.
- If financial value changes, journal debits and credits balance exactly and historical entries remain immutable.
- If AI is involved, model/prompt/feature version and fallback behavior are recorded and deterministic policy remains final.
- A timeout/restart cannot create a duplicate financial result; request status can be rediscovered.
- Logs/evidence contain correlation references but not secrets or prohibited sensitive payment data.

Common Failure Modes

- No shadow evaluation.
- Model output unversioned.
- Model failure blocks every payment without fallback.
- Recommendation directly triggers high-risk ledger write.

Expert Review Questions

- What happens if this component succeeds but the next component fails?
- How is the authorizing identity/device/key revoked or recovered?
- Can the operation be safely retried after the client loses the response?
- What happens if AI is unavailable, slow, or wrong?
- Can an auditor reconstruct debit, credit, actor, risk decision, approval, and Fabric txId from retained evidence?
- Which compensating transaction resolves a downstream operational error without rewriting history?

Part X - Fraud Detection, Anomaly Models, and Risk Engineering

Fraud engineering combines deterministic rules, behavioral features, statistical/ML models, explainable reason codes, calibrated thresholds, and a human-review path. The target is not maximum blocking; it is measurable risk reduction at an acceptable false-positive cost.

Part focus: This part contains 23 implementation chapters. Each chapter includes design rules, implementation procedure, a practical artifact, verification, failure analysis, and expert review questions.

Chapter 161 - Deterministic Fraud Rules Baseline

Create transparent controls that remain available if ML is disabled. Fraud engineering combines deterministic rules, behavioral features, statistical/ML models, explainable reason codes, calibrated thresholds, and a human-review path. The target is not maximum blocking; it is measurable risk reduction at an acceptable false-positive cost.

Design rule: Velocity thresholds

Implementation Objective

The required output is: Rules catalog. It must be version-controlled, reviewable, testable with the WAN disabled, and linked to the architecture/source-of-truth model. For a payment platform, every implementation decision should make it possible to explain who authorized an action, which facts were trusted, which controls ran, what financial journal committed, and how the result is recovered after failure.

Key Engineering Decisions

- Velocity thresholds
- new-device limits
- failed-auth counters
- high-value/new-recipient review
- merchant refund anomaly
- freeze lists

Deep Implementation Guidance

Measure the operational system, not only the model. Review capacity, threshold volume, false-positive cost, analyst overrides, latency, drift, and fraud outcomes all determine whether a “better” model is actually safer in production.

Data and Control Questions

- What is the authoritative source for every input used by this component?
- Which user, device, service, MSP, or administrator identity is allowed to initiate or approve it?
- Does the operation move value, reserve value, create a side effect, or only provide advisory information?
- What idempotency/replay key proves that a retry is the same business intent?
- What evidence is retained off-chain and what stable reference/hash belongs on the ledger?

Chapter 161 Implementation - Deterministic Fraud Rules Baseline

Implementation Procedure

801. Start with transparent deterministic fraud rules and measurable features.
802. Build labeled datasets without leakage and split by time/entity as appropriate.
803. Train/evaluate model with precision, recall, false-positive cost, and calibration.
804. Serve locally with a strict schema and bounded latency.
805. Monitor drift/override rates and retain manual fallback when model service is unavailable.

Implementation Example

```
# Deterministic Fraud Rules Baseline
INPUT  -> authenticate, validate schema, bind requestId/nonce
FACTS  -> read authoritative wallet/merchant/state data
AI     -> optional bounded recommendation with decisionId/version
POLICY -> enforce ownership, hard limits, approvals, risk fallback
LEDGER -> submit one idempotent deterministic transaction when value changes
EVENT  -> update projection/case/monitoring asynchronously
VERIFY -> discover committed result and retain correlationId + txId evidence
```

Verification Checklist

- The rules catalog exists and is version-controlled.
- The feature works with all public Internet/WAN connectivity disabled.
- Authorization, malformed input, duplicate retry, and failure behavior have explicit tests.
- If financial value changes, journal debits and credits balance exactly and historical entries remain immutable.
- If AI is involved, model/prompt/feature version and fallback behavior are recorded and deterministic policy remains final.
- A timeout/restart cannot create a duplicate financial result; request status can be rediscovered.
- Logs/evidence contain correlation references but not secrets or prohibited sensitive payment data.

Common Failure Modes

- Data leakage in training.
- Threshold chosen only for headline accuracy.
- No explainable reason codes.
- Drift ignored until customer complaints rise.

Expert Review Questions

- What happens if this component succeeds but the next component fails?
- How is the authorizing identity/device/key revoked or recovered?
- Can the operation be safely retried after the client loses the response?
- What happens if AI is unavailable, slow, or wrong?
- Can an auditor reconstruct debit, credit, actor, risk decision, approval, and Fabric txId from retained evidence?
- Which compensating transaction resolves a downstream operational error without rewriting history?

Chapter 162 - Transaction Feature Engineering

Build a versioned numerical/categorical feature set from authoritative local data. Fraud engineering combines deterministic rules, behavioral features, statistical/ML models, explainable reason codes, calibrated thresholds, and a human-review path. The target is not maximum blocking; it is measurable risk reduction at an acceptable false-positive cost.

Design rule: amount relative to user baseline

Implementation Objective

The required output is: Feature dictionary. It must be version-controlled, reviewable, testable with the WAN disabled, and linked to the architecture/source-of-truth model. For a payment platform, every implementation decision should make it possible to explain who authorized an action, which facts were trusted, which controls ran, what financial journal committed, and how the result is recovered after failure.

Key Engineering Decisions

- amount relative to user baseline
- hour/day
- wallet age
- recipient age
- transaction counts/sums
- device novelty
- merchant risk
- failed auth

Deep Implementation Guidance

Measure the operational system, not only the model. Review capacity, threshold volume, false-positive cost, analyst overrides, latency, drift, and fraud outcomes all determine whether a “better” model is actually safer in production.

Data and Control Questions

- What is the authoritative source for every input used by this component?
- Which user, device, service, MSP, or administrator identity is allowed to initiate or approve it?
- Does the operation move value, reserve value, create a side effect, or only provide advisory information?
- What idempotency/replay key proves that a retry is the same business intent?
- What evidence is retained off-chain and what stable reference/hash belongs on the ledger?

Chapter 162 Implementation - Transaction Feature Engineering

Implementation Procedure

806. Start with transparent deterministic fraud rules and measurable features.
807. Build labeled datasets without leakage and split by time/entity as appropriate.
808. Train/evaluate model with precision, recall, false-positive cost, and calibration.
809. Serve locally with a strict schema and bounded latency.
810. Monitor drift/override rates and retain manual fallback when model service is unavailable.

Implementation Example

```
# Transaction Feature Engineering
INPUT  -> authenticate, validate schema, bind requestId/nonce
FACTS  -> read authoritative wallet/merchant/state data
AI     -> optional bounded recommendation with decisionId/version
POLICY -> enforce ownership, hard limits, approvals, risk fallback
LEDGER -> submit one idempotent deterministic transaction when value changes
EVENT  -> update projection/case/monitoring asynchronously
VERIFY -> discover committed result and retain correlationId + txId evidence
```

Verification Checklist

- The feature dictionary exists and is version-controlled.
- The feature works with all public Internet/WAN connectivity disabled.
- Authorization, malformed input, duplicate retry, and failure behavior have explicit tests.
- If financial value changes, journal debits and credits balance exactly and historical entries remain immutable.
- If AI is involved, model/prompt/feature version and fallback behavior are recorded and deterministic policy remains final.
- A timeout/restart cannot create a duplicate financial result; request status can be rediscovered.
- Logs/evidence contain correlation references but not secrets or prohibited sensitive payment data.

Common Failure Modes

- Data leakage in training.
- Threshold chosen only for headline accuracy.
- No explainable reason codes.
- Drift ignored until customer complaints rise.

Expert Review Questions

- What happens if this component succeeds but the next component fails?
- How is the authorizing identity/device/key revoked or recovered?
- Can the operation be safely retried after the client loses the response?
- What happens if AI is unavailable, slow, or wrong?
- Can an auditor reconstruct debit, credit, actor, risk decision, approval, and Fabric txId from retained evidence?
- Which compensating transaction resolves a downstream operational error without rewriting history?

Chapter 163 - Velocity Features

Measure rapid sequences that may indicate takeover or abuse. Fraud engineering combines deterministic rules, behavioral features, statistical/ML models, explainable reason codes, calibrated thresholds, and a human-review path. The target is not maximum blocking; it is measurable risk reduction at an acceptable false-positive cost.

Design rule: Count/sum in 1m 10m 1h 24h windows

Implementation Objective

The required output is: Velocity feature set. It must be version-controlled, reviewable, testable with the WAN disabled, and linked to the architecture/source-of-truth model. For a payment platform, every implementation decision should make it possible to explain who authorized an action, which facts were trusted, which controls ran, what financial journal committed, and how the result is recovered after failure.

Key Engineering Decisions

- Count/sum in 1m 10m 1h 24h windows
- distinct recipients/merchants
- decline/retry count
- compute without future leakage

Deep Implementation Guidance

Measure the operational system, not only the model. Review capacity, threshold volume, false-positive cost, analyst overrides, latency, drift, and fraud outcomes all determine whether a “better” model is actually safer in production.

Data and Control Questions

- What is the authoritative source for every input used by this component?
- Which user, device, service, MSP, or administrator identity is allowed to initiate or approve it?
- Does the operation move value, reserve value, create a side effect, or only provide advisory information?
- What idempotency/replay key proves that a retry is the same business intent?
- What evidence is retained off-chain and what stable reference/hash belongs on the ledger?

Chapter 163 Implementation - Velocity Features

Implementation Procedure

811. Start with transparent deterministic fraud rules and measurable features.
812. Build labeled datasets without leakage and split by time/entity as appropriate.
813. Train/evaluate model with precision, recall, false-positive cost, and calibration.
814. Serve locally with a strict schema and bounded latency.
815. Monitor drift/override rates and retain manual fallback when model service is unavailable.

Implementation Example

```
# Velocity Features
INPUT  -> authenticate, validate schema, bind requestId/nonce
FACTS  -> read authoritative wallet/merchant/state data
AI     -> optional bounded recommendation with decisionId/version
POLICY -> enforce ownership, hard limits, approvals, risk fallback
LEDGER -> submit one idempotent deterministic transaction when value changes
EVENT  -> update projection/case/monitoring asynchronously
VERIFY -> discover committed result and retain correlationId + txId evidence
```

Verification Checklist

- The velocity feature set exists and is version-controlled.
- The feature works with all public Internet/WAN connectivity disabled.
- Authorization, malformed input, duplicate retry, and failure behavior have explicit tests.
- If financial value changes, journal debits and credits balance exactly and historical entries remain immutable.
- If AI is involved, model/prompt/feature version and fallback behavior are recorded and deterministic policy remains final.
- A timeout/restart cannot create a duplicate financial result; request status can be rediscovered.
- Logs/evidence contain correlation references but not secrets or prohibited sensitive payment data.

Common Failure Modes

- Data leakage in training.
- Threshold chosen only for headline accuracy.
- No explainable reason codes.
- Drift ignored until customer complaints rise.

Expert Review Questions

- What happens if this component succeeds but the next component fails?
- How is the authorizing identity/device/key revoked or recovered?
- Can the operation be safely retried after the client loses the response?
- What happens if AI is unavailable, slow, or wrong?
- Can an auditor reconstruct debit, credit, actor, risk decision, approval, and Fabric txId from retained evidence?
- Which compensating transaction resolves a downstream operational error without rewriting history?

Chapter 164 - Device and Session Risk Features

Use local device history as risk evidence. Fraud engineering combines deterministic rules, behavioral features, statistical/ML models, explainable reason codes, calibrated thresholds, and a human-review path. The target is not maximum blocking; it is measurable risk reduction at an acceptable false-positive cost.

Design rule: new device

Implementation Objective

The required output is: Device features. It must be version-controlled, reviewable, testable with the WAN disabled, and linked to the architecture/source-of-truth model. For a payment platform, every implementation decision should make it possible to explain who authorized an action, which facts were trusted, which controls ran, what financial journal committed, and how the result is recovered after failure.

Key Engineering Decisions

- new device
- recent re-registration
- IP/VLAN/terminal mismatch
- failed login history
- device reputation status

Deep Implementation Guidance

Measure the operational system, not only the model. Review capacity, threshold volume, false-positive cost, analyst overrides, latency, drift, and fraud outcomes all determine whether a “better” model is actually safer in production.

Data and Control Questions

- What is the authoritative source for every input used by this component?
- Which user, device, service, MSP, or administrator identity is allowed to initiate or approve it?
- Does the operation move value, reserve value, create a side effect, or only provide advisory information?
- What idempotency/replay key proves that a retry is the same business intent?
- What evidence is retained off-chain and what stable reference/hash belongs on the ledger?

Chapter 164 Implementation - Device and Session Risk Features

Implementation Procedure

816. Start with transparent deterministic fraud rules and measurable features.
817. Build labeled datasets without leakage and split by time/entity as appropriate.
818. Train/evaluate model with precision, recall, false-positive cost, and calibration.
819. Serve locally with a strict schema and bounded latency.
820. Monitor drift/override rates and retain manual fallback when model service is unavailable.

Implementation Example

```
# Device and Session Risk Features
INPUT  -> authenticate, validate schema, bind requestId/nonce
FACTS  -> read authoritative wallet/merchant/state data
AI     -> optional bounded recommendation with decisionId/version
POLICY -> enforce ownership, hard limits, approvals, risk fallback
LEDGER -> submit one idempotent deterministic transaction when value changes
EVENT  -> update projection/case/monitoring asynchronously
VERIFY -> discover committed result and retain correlationId + txId evidence
```

Verification Checklist

- The device features exists and is version-controlled.
- The feature works with all public Internet/WAN connectivity disabled.
- Authorization, malformed input, duplicate retry, and failure behavior have explicit tests.
- If financial value changes, journal debits and credits balance exactly and historical entries remain immutable.
- If AI is involved, model/prompt/feature version and fallback behavior are recorded and deterministic policy remains final.
- A timeout/restart cannot create a duplicate financial result; request status can be rediscovered.
- Logs/evidence contain correlation references but not secrets or prohibited sensitive payment data.

Common Failure Modes

- Data leakage in training.
- Threshold chosen only for headline accuracy.
- No explainable reason codes.
- Drift ignored until customer complaints rise.

Expert Review Questions

- What happens if this component succeeds but the next component fails?
- How is the authorizing identity/device/key revoked or recovered?
- Can the operation be safely retried after the client loses the response?
- What happens if AI is unavailable, slow, or wrong?
- Can an auditor reconstruct debit, credit, actor, risk decision, approval, and Fabric txId from retained evidence?
- Which compensating transaction resolves a downstream operational error without rewriting history?

Chapter 165 - Recipient and Graph Features

Detect risky new destinations and circular/rapid value movement. Fraud engineering combines deterministic rules, behavioral features, statistical/ML models, explainable reason codes, calibrated thresholds, and a human-review path. The target is not maximum blocking; it is measurable risk reduction at an acceptable false-positive cost.

Design rule: recipient age

Implementation Objective

The required output is: Graph feature design. It must be version-controlled, reviewable, testable with the WAN disabled, and linked to the architecture/source-of-truth model. For a payment platform, every implementation decision should make it possible to explain who authorized an action, which facts were trusted, which controls ran, what financial journal committed, and how the result is recovered after failure.

Key Engineering Decisions

- recipient age
- first-time recipient
- fan-out/fan-in
- short cycles
- shared-device clusters
- keep graph analysis advisory initially

Deep Implementation Guidance

Measure the operational system, not only the model. Review capacity, threshold volume, false-positive cost, analyst overrides, latency, drift, and fraud outcomes all determine whether a “better” model is actually safer in production.

Data and Control Questions

- What is the authoritative source for every input used by this component?
- Which user, device, service, MSP, or administrator identity is allowed to initiate or approve it?
- Does the operation move value, reserve value, create a side effect, or only provide advisory information?
- What idempotency/replay key proves that a retry is the same business intent?
- What evidence is retained off-chain and what stable reference/hash belongs on the ledger?

Chapter 165 Implementation - Recipient and Graph Features

Implementation Procedure

821. Start with transparent deterministic fraud rules and measurable features.
822. Build labeled datasets without leakage and split by time/entity as appropriate.
823. Train/evaluate model with precision, recall, false-positive cost, and calibration.
824. Serve locally with a strict schema and bounded latency.
825. Monitor drift/override rates and retain manual fallback when model service is unavailable.

Implementation Example

```
# Recipient and Graph Features
INPUT  -> authenticate, validate schema, bind requestId/nonce
FACTS  -> read authoritative wallet/merchant/state data
AI      -> optional bounded recommendation with decisionId/version
POLICY  -> enforce ownership, hard limits, approvals, risk fallback
LEDGER  -> submit one idempotent deterministic transaction when value changes
EVENT   -> update projection/case/monitoring asynchronously
VERIFY  -> discover committed result and retain correlationId + txId evidence
```

Verification Checklist

- The graph feature design exists and is version-controlled.
- The feature works with all public Internet/WAN connectivity disabled.
- Authorization, malformed input, duplicate retry, and failure behavior have explicit tests.
- If financial value changes, journal debits and credits balance exactly and historical entries remain immutable.
- If AI is involved, model/prompt/feature version and fallback behavior are recorded and deterministic policy remains final.
- A timeout/restart cannot create a duplicate financial result; request status can be rediscovered.
- Logs/evidence contain correlation references but not secrets or prohibited sensitive payment data.

Common Failure Modes

- Data leakage in training.
- Threshold chosen only for headline accuracy.
- No explainable reason codes.
- Drift ignored until customer complaints rise.

Expert Review Questions

- What happens if this component succeeds but the next component fails?
- How is the authorizing identity/device/key revoked or recovered?
- Can the operation be safely retried after the client loses the response?
- What happens if AI is unavailable, slow, or wrong?
- Can an auditor reconstruct debit, credit, actor, risk decision, approval, and Fabric txId from retained evidence?
- Which compensating transaction resolves a downstream operational error without rewriting history?

Chapter 166 - Merchant Risk Features

Identify abnormal merchant behavior. Fraud engineering combines deterministic rules, behavioral features, statistical/ML models, explainable reason codes, calibrated thresholds, and a human-review path. The target is not maximum blocking; it is measurable risk reduction at an acceptable false-positive cost.

Design rule: refund ratio

Implementation Objective

The required output is: Merchant feature set. It must be version-controlled, reviewable, testable with the WAN disabled, and linked to the architecture/source-of-truth model. For a payment platform, every implementation decision should make it possible to explain who authorized an action, which facts were trusted, which controls ran, what financial journal committed, and how the result is recovered after failure.

Key Engineering Decisions

- refund ratio
- chargeback/dispute rate if available
- average amount shift
- rapid settlement
- unusual customer concentration

Deep Implementation Guidance

Measure the operational system, not only the model. Review capacity, threshold volume, false-positive cost, analyst overrides, latency, drift, and fraud outcomes all determine whether a “better” model is actually safer in production.

Data and Control Questions

- What is the authoritative source for every input used by this component?
- Which user, device, service, MSP, or administrator identity is allowed to initiate or approve it?
- Does the operation move value, reserve value, create a side effect, or only provide advisory information?
- What idempotency/replay key proves that a retry is the same business intent?
- What evidence is retained off-chain and what stable reference/hash belongs on the ledger?

Chapter 166 Implementation - Merchant Risk Features

Implementation Procedure

826. Start with transparent deterministic fraud rules and measurable features.
827. Build labeled datasets without leakage and split by time/entity as appropriate.
828. Train/evaluate model with precision, recall, false-positive cost, and calibration.
829. Serve locally with a strict schema and bounded latency.
830. Monitor drift/override rates and retain manual fallback when model service is unavailable.

Implementation Example

```
# Merchant Risk Features
INPUT  -> authenticate, validate schema, bind requestId/nonce
FACTS  -> read authoritative wallet/merchant/state data
AI     -> optional bounded recommendation with decisionId/version
POLICY -> enforce ownership, hard limits, approvals, risk fallback
LEDGER -> submit one idempotent deterministic transaction when value changes
EVENT  -> update projection/case/monitoring asynchronously
VERIFY -> discover committed result and retain correlationId + txId evidence
```

Verification Checklist

- The merchant feature set exists and is version-controlled.
- The feature works with all public Internet/WAN connectivity disabled.
- Authorization, malformed input, duplicate retry, and failure behavior have explicit tests.
- If financial value changes, journal debits and credits balance exactly and historical entries remain immutable.
- If AI is involved, model/prompt/feature version and fallback behavior are recorded and deterministic policy remains final.
- A timeout/restart cannot create a duplicate financial result; request status can be rediscovered.
- Logs/evidence contain correlation references but not secrets or prohibited sensitive payment data.

Common Failure Modes

- Data leakage in training.
- Threshold chosen only for headline accuracy.
- No explainable reason codes.
- Drift ignored until customer complaints rise.

Expert Review Questions

- What happens if this component succeeds but the next component fails?
- How is the authorizing identity/device/key revoked or recovered?
- Can the operation be safely retried after the client loses the response?
- What happens if AI is unavailable, slow, or wrong?
- Can an auditor reconstruct debit, credit, actor, risk decision, approval, and Fabric txId from retained evidence?
- Which compensating transaction resolves a downstream operational error without rewriting history?

Chapter 167 - Supervised Classification

Train a labeled fraud-risk model from historical reviewed outcomes. Fraud engineering combines deterministic rules, behavioral features, statistical/ML models, explainable reason codes, calibrated thresholds, and a human-review path. The target is not maximum blocking; it is measurable risk reduction at an acceptable false-positive cost.

Design rule: Time-aware splits

Implementation Objective

The required output is: Training pipeline. It must be version-controlled, reviewable, testable with the WAN disabled, and linked to the architecture/source-of-truth model. For a payment platform, every implementation decision should make it possible to explain who authorized an action, which facts were trusted, which controls ran, what financial journal committed, and how the result is recovered after failure.

Key Engineering Decisions

- Time-aware splits
- class imbalance handling
- prevent leakage
- calibrate probabilities
- document label definition

Deep Implementation Guidance

Measure the operational system, not only the model. Review capacity, threshold volume, false-positive cost, analyst overrides, latency, drift, and fraud outcomes all determine whether a “better” model is actually safer in production.

Data and Control Questions

- What is the authoritative source for every input used by this component?
- Which user, device, service, MSP, or administrator identity is allowed to initiate or approve it?
- Does the operation move value, reserve value, create a side effect, or only provide advisory information?
- What idempotency/replay key proves that a retry is the same business intent?
- What evidence is retained off-chain and what stable reference/hash belongs on the ledger?

Chapter 167 Implementation - Supervised Classification

Implementation Procedure

831. Start with transparent deterministic fraud rules and measurable features.
832. Build labeled datasets without leakage and split by time/entity as appropriate.
833. Train/evaluate model with precision, recall, false-positive cost, and calibration.
834. Serve locally with a strict schema and bounded latency.
835. Monitor drift/override rates and retain manual fallback when model service is unavailable.

Implementation Example

```
# Supervised Classification
INPUT  -> authenticate, validate schema, bind requestId/nonce
FACTS  -> read authoritative wallet/merchant/state data
AI     -> optional bounded recommendation with decisionId/version
POLICY -> enforce ownership, hard limits, approvals, risk fallback
LEDGER -> submit one idempotent deterministic transaction when value changes
EVENT  -> update projection/case/monitoring asynchronously
VERIFY -> discover committed result and retain correlationId + txId evidence
```

Verification Checklist

- The training pipeline exists and is version-controlled.
- The feature works with all public Internet/WAN connectivity disabled.
- Authorization, malformed input, duplicate retry, and failure behavior have explicit tests.
- If financial value changes, journal debits and credits balance exactly and historical entries remain immutable.
- If AI is involved, model/prompt/feature version and fallback behavior are recorded and deterministic policy remains final.
- A timeout/restart cannot create a duplicate financial result; request status can be rediscovered.
- Logs/evidence contain correlation references but not secrets or prohibited sensitive payment data.

Common Failure Modes

- Data leakage in training.
- Threshold chosen only for headline accuracy.
- No explainable reason codes.
- Drift ignored until customer complaints rise.

Expert Review Questions

- What happens if this component succeeds but the next component fails?
- How is the authorizing identity/device/key revoked or recovered?
- Can the operation be safely retried after the client loses the response?
- What happens if AI is unavailable, slow, or wrong?
- Can an auditor reconstruct debit, credit, actor, risk decision, approval, and Fabric txId from retained evidence?
- Which compensating transaction resolves a downstream operational error without rewriting history?

Chapter 168 - Anomaly Detection

Use unsupervised/semi-supervised models to surface novel patterns. Fraud engineering combines deterministic rules, behavioral features, statistical/ML models, explainable reason codes, calibrated thresholds, and a human-review path. The target is not maximum blocking; it is measurable risk reduction at an acceptable false-positive cost.

Design rule: Train on stable baseline

Implementation Objective

The required output is: Anomaly pipeline. It must be version-controlled, reviewable, testable with the WAN disabled, and linked to the architecture/source-of-truth model. For a payment platform, every implementation decision should make it possible to explain who authorized an action, which facts were trusted, which controls ran, what financial journal committed, and how the result is recovered after failure.

Key Engineering Decisions

- Train on stable baseline
- score deviations
- do not equate anomaly with fraud
- route high anomaly to review

Deep Implementation Guidance

Measure the operational system, not only the model. Review capacity, threshold volume, false-positive cost, analyst overrides, latency, drift, and fraud outcomes all determine whether a “better” model is actually safer in production.

Data and Control Questions

- What is the authoritative source for every input used by this component?
- Which user, device, service, MSP, or administrator identity is allowed to initiate or approve it?
- Does the operation move value, reserve value, create a side effect, or only provide advisory information?
- What idempotency/replay key proves that a retry is the same business intent?
- What evidence is retained off-chain and what stable reference/hash belongs on the ledger?

Chapter 168 Implementation - Anomaly Detection

Implementation Procedure

- 836. Start with transparent deterministic fraud rules and measurable features.
- 837. Build labeled datasets without leakage and split by time/entity as appropriate.
- 838. Train/evaluate model with precision, recall, false-positive cost, and calibration.
- 839. Serve locally with a strict schema and bounded latency.
- 840. Monitor drift/override rates and retain manual fallback when model service is unavailable.

Implementation Example

```
# Anomaly Detection
INPUT  -> authenticate, validate schema, bind requestId/nonce
FACTS  -> read authoritative wallet/merchant/state data
AI     -> optional bounded recommendation with decisionId/version
POLICY -> enforce ownership, hard limits, approvals, risk fallback
LEDGER -> submit one idempotent deterministic transaction when value changes
EVENT  -> update projection/case/monitoring asynchronously
VERIFY -> discover committed result and retain correlationId + txId evidence
```

Verification Checklist

- The anomaly pipeline exists and is version-controlled.
- The feature works with all public Internet/WAN connectivity disabled.
- Authorization, malformed input, duplicate retry, and failure behavior have explicit tests.
- If financial value changes, journal debits and credits balance exactly and historical entries remain immutable.
- If AI is involved, model/prompt/feature version and fallback behavior are recorded and deterministic policy remains final.
- A timeout/restart cannot create a duplicate financial result; request status can be rediscovered.
- Logs/evidence contain correlation references but not secrets or prohibited sensitive payment data.

Common Failure Modes

- Data leakage in training.
- Threshold chosen only for headline accuracy.
- No explainable reason codes.
- Drift ignored until customer complaints rise.

Expert Review Questions

- What happens if this component succeeds but the next component fails?
- How is the authorizing identity/device/key revoked or recovered?
- Can the operation be safely retried after the client loses the response?
- What happens if AI is unavailable, slow, or wrong?
- Can an auditor reconstruct debit, credit, actor, risk decision, approval, and Fabric txId from retained evidence?
- Which compensating transaction resolves a downstream operational error without rewriting history?

Chapter 169 - Baseline Logistic/Tree Models

Start with interpretable models before complex architectures. Fraud engineering combines deterministic rules, behavioral features, statistical/ML models, explainable reason codes, calibrated thresholds, and a human-review path. The target is not maximum blocking; it is measurable risk reduction at an acceptable false-positive cost.

Design rule: Logistic regression for transparency

Implementation Objective

The required output is: Model comparison. It must be version-controlled, reviewable, testable with the WAN disabled, and linked to the architecture/source-of-truth model. For a payment platform, every implementation decision should make it possible to explain who authorized an action, which facts were trusted, which controls ran, what financial journal committed, and how the result is recovered after failure.

Key Engineering Decisions

- Logistic regression for transparency
- tree/boosting for nonlinear patterns
- compare recall/precision/latency/explainability

Deep Implementation Guidance

Measure the operational system, not only the model. Review capacity, threshold volume, false-positive cost, analyst overrides, latency, drift, and fraud outcomes all determine whether a “better” model is actually safer in production.

Data and Control Questions

- What is the authoritative source for every input used by this component?
- Which user, device, service, MSP, or administrator identity is allowed to initiate or approve it?
- Does the operation move value, reserve value, create a side effect, or only provide advisory information?
- What idempotency/replay key proves that a retry is the same business intent?
- What evidence is retained off-chain and what stable reference/hash belongs on the ledger?

Chapter 169 Implementation - Baseline Logistic/Tree Models

Implementation Procedure

841. Start with transparent deterministic fraud rules and measurable features.
842. Build labeled datasets without leakage and split by time/entity as appropriate.
843. Train/evaluate model with precision, recall, false-positive cost, and calibration.
844. Serve locally with a strict schema and bounded latency.
845. Monitor drift/override rates and retain manual fallback when model service is unavailable.

Implementation Example

```
# Baseline Logistic/Tree Models
INPUT  -> authenticate, validate schema, bind requestId/nonce
FACTS  -> read authoritative wallet/merchant/state data
AI     -> optional bounded recommendation with decisionId/version
POLICY -> enforce ownership, hard limits, approvals, risk fallback
LEDGER -> submit one idempotent deterministic transaction when value changes
EVENT  -> update projection/case/monitoring asynchronously
VERIFY -> discover committed result and retain correlationId + txId evidence
```

Verification Checklist

- The model comparison exists and is version-controlled.
- The feature works with all public Internet/WAN connectivity disabled.
- Authorization, malformed input, duplicate retry, and failure behavior have explicit tests.
- If financial value changes, journal debits and credits balance exactly and historical entries remain immutable.
- If AI is involved, model/prompt/feature version and fallback behavior are recorded and deterministic policy remains final.
- A timeout/restart cannot create a duplicate financial result; request status can be rediscovered.
- Logs/evidence contain correlation references but not secrets or prohibited sensitive payment data.

Common Failure Modes

- Data leakage in training.
- Threshold chosen only for headline accuracy.
- No explainable reason codes.
- Drift ignored until customer complaints rise.

Expert Review Questions

- What happens if this component succeeds but the next component fails?
- How is the authorizing identity/device/key revoked or recovered?
- Can the operation be safely retried after the client loses the response?
- What happens if AI is unavailable, slow, or wrong?
- Can an auditor reconstruct debit, credit, actor, risk decision, approval, and Fabric txId from retained evidence?
- Which compensating transaction resolves a downstream operational error without rewriting history?

Chapter 170 - Threshold Calibration

Choose ALLOW/REVIEW/BLOCK thresholds from business cost, not model accuracy alone. Fraud engineering combines deterministic rules, behavioral features, statistical/ML models, explainable reason codes, calibrated thresholds, and a human-review path. The target is not maximum blocking; it is measurable risk reduction at an acceptable false-positive cost.

Design rule: Quantify false positive cost and missed fraud cost

Implementation Objective

The required output is: Threshold worksheet. It must be version-controlled, reviewable, testable with the WAN disabled, and linked to the architecture/source-of-truth model. For a payment platform, every implementation decision should make it possible to explain who authorized an action, which facts were trusted, which controls ran, what financial journal committed, and how the result is recovered after failure.

Key Engineering Decisions

- Quantify false positive cost and missed fraud cost
- separate review and block thresholds
- validate by segment

Deep Implementation Guidance

Measure the operational system, not only the model. Review capacity, threshold volume, false-positive cost, analyst overrides, latency, drift, and fraud outcomes all determine whether a “better” model is actually safer in production.

Data and Control Questions

- What is the authoritative source for every input used by this component?
- Which user, device, service, MSP, or administrator identity is allowed to initiate or approve it?
- Does the operation move value, reserve value, create a side effect, or only provide advisory information?
- What idempotency/replay key proves that a retry is the same business intent?
- What evidence is retained off-chain and what stable reference/hash belongs on the ledger?

Chapter 170 Implementation - Threshold Calibration

Implementation Procedure

- 846. Start with transparent deterministic fraud rules and measurable features.
- 847. Build labeled datasets without leakage and split by time/entity as appropriate.
- 848. Train/evaluate model with precision, recall, false-positive cost, and calibration.
- 849. Serve locally with a strict schema and bounded latency.
- 850. Monitor drift/override rates and retain manual fallback when model service is unavailable.

Implementation Example

```
# Threshold Calibration
INPUT  -> authenticate, validate schema, bind requestId/nonce
FACTS  -> read authoritative wallet/merchant/state data
AI     -> optional bounded recommendation with decisionId/version
POLICY -> enforce ownership, hard limits, approvals, risk fallback
LEDGER -> submit one idempotent deterministic transaction when value changes
EVENT  -> update projection/case/monitoring asynchronously
VERIFY -> discover committed result and retain correlationId + txId evidence
```

Verification Checklist

- The threshold worksheet exists and is version-controlled.
- The feature works with all public Internet/WAN connectivity disabled.
- Authorization, malformed input, duplicate retry, and failure behavior have explicit tests.
- If financial value changes, journal debits and credits balance exactly and historical entries remain immutable.
- If AI is involved, model/prompt/feature version and fallback behavior are recorded and deterministic policy remains final.
- A timeout/restart cannot create a duplicate financial result; request status can be rediscovered.
- Logs/evidence contain correlation references but not secrets or prohibited sensitive payment data.

Common Failure Modes

- Data leakage in training.
- Threshold chosen only for headline accuracy.
- No explainable reason codes.
- Drift ignored until customer complaints rise.

Expert Review Questions

- What happens if this component succeeds but the next component fails?
- How is the authorizing identity/device/key revoked or recovered?
- Can the operation be safely retried after the client loses the response?
- What happens if AI is unavailable, slow, or wrong?
- Can an auditor reconstruct debit, credit, actor, risk decision, approval, and Fabric txId from retained evidence?
- Which compensating transaction resolves a downstream operational error without rewriting history?

Chapter 171 - Precision Recall and PR Curves

Evaluate highly imbalanced fraud tasks correctly. Fraud engineering combines deterministic rules, behavioral features, statistical/ML models, explainable reason codes, calibrated thresholds, and a human-review path. The target is not maximum blocking; it is measurable risk reduction at an acceptable false-positive cost.

Design rule: Report precision/recall/F1/PR-AUC/confusion matrix

Implementation Objective

The required output is: Evaluation report. It must be version-controlled, reviewable, testable with the WAN disabled, and linked to the architecture/source-of-truth model. For a payment platform, every implementation decision should make it possible to explain who authorized an action, which facts were trusted, which controls ran, what financial journal committed, and how the result is recovered after failure.

Key Engineering Decisions

- Report precision/recall/F1/PR-AUC/confusion matrix
- include volume at threshold
- avoid headline accuracy

Deep Implementation Guidance

Measure the operational system, not only the model. Review capacity, threshold volume, false-positive cost, analyst overrides, latency, drift, and fraud outcomes all determine whether a “better” model is actually safer in production.

Data and Control Questions

- What is the authoritative source for every input used by this component?
- Which user, device, service, MSP, or administrator identity is allowed to initiate or approve it?
- Does the operation move value, reserve value, create a side effect, or only provide advisory information?
- What idempotency/replay key proves that a retry is the same business intent?
- What evidence is retained off-chain and what stable reference/hash belongs on the ledger?

Chapter 171 Implementation - Precision Recall and PR Curves

Implementation Procedure

851. Start with transparent deterministic fraud rules and measurable features.
852. Build labeled datasets without leakage and split by time/entity as appropriate.
853. Train/evaluate model with precision, recall, false-positive cost, and calibration.
854. Serve locally with a strict schema and bounded latency.
855. Monitor drift/override rates and retain manual fallback when model service is unavailable.

Implementation Example

```
# Precision Recall and PR Curves
INPUT  -> authenticate, validate schema, bind requestId/nonce
FACTS  -> read authoritative wallet/merchant/state data
AI     -> optional bounded recommendation with decisionId/version
POLICY -> enforce ownership, hard limits, approvals, risk fallback
LEDGER -> submit one idempotent deterministic transaction when value changes
EVENT  -> update projection/case/monitoring asynchronously
VERIFY -> discover committed result and retain correlationId + txId evidence
```

Verification Checklist

- The evaluation report exists and is version-controlled.
- The feature works with all public Internet/WAN connectivity disabled.
- Authorization, malformed input, duplicate retry, and failure behavior have explicit tests.
- If financial value changes, journal debits and credits balance exactly and historical entries remain immutable.
- If AI is involved, model/prompt/feature version and fallback behavior are recorded and deterministic policy remains final.
- A timeout/restart cannot create a duplicate financial result; request status can be rediscovered.
- Logs/evidence contain correlation references but not secrets or prohibited sensitive payment data.

Common Failure Modes

- Data leakage in training.
- Threshold chosen only for headline accuracy.
- No explainable reason codes.
- Drift ignored until customer complaints rise.

Expert Review Questions

- What happens if this component succeeds but the next component fails?
- How is the authorizing identity/device/key revoked or recovered?
- Can the operation be safely retried after the client loses the response?
- What happens if AI is unavailable, slow, or wrong?
- Can an auditor reconstruct debit, credit, actor, risk decision, approval, and Fabric txId from retained evidence?
- Which compensating transaction resolves a downstream operational error without rewriting history?

Chapter 172 - Calibration and Expected Loss

Make risk scores meaningful enough for policy decisions. Fraud engineering combines deterministic rules, behavioral features, statistical/ML models, explainable reason codes, calibrated thresholds, and a human-review path. The target is not maximum blocking; it is measurable risk reduction at an acceptable false-positive cost.

Design rule: Reliability/calibration plot

Implementation Objective

The required output is: Calibration report. It must be version-controlled, reviewable, testable with the WAN disabled, and linked to the architecture/source-of-truth model. For a payment platform, every implementation decision should make it possible to explain who authorized an action, which facts were trusted, which controls ran, what financial journal committed, and how the result is recovered after failure.

Key Engineering Decisions

- Reliability/calibration plot
- expected cost by threshold
- monitor post-deployment drift

Deep Implementation Guidance

Measure the operational system, not only the model. Review capacity, threshold volume, false-positive cost, analyst overrides, latency, drift, and fraud outcomes all determine whether a “better” model is actually safer in production.

Data and Control Questions

- What is the authoritative source for every input used by this component?
- Which user, device, service, MSP, or administrator identity is allowed to initiate or approve it?
- Does the operation move value, reserve value, create a side effect, or only provide advisory information?
- What idempotency/replay key proves that a retry is the same business intent?
- What evidence is retained off-chain and what stable reference/hash belongs on the ledger?

Chapter 172 Implementation - Calibration and Expected Loss

Implementation Procedure

- 856. Start with transparent deterministic fraud rules and measurable features.
- 857. Build labeled datasets without leakage and split by time/entity as appropriate.
- 858. Train/evaluate model with precision, recall, false-positive cost, and calibration.
- 859. Serve locally with a strict schema and bounded latency.
- 860. Monitor drift/override rates and retain manual fallback when model service is unavailable.

Implementation Example

```
# Calibration and Expected Loss
INPUT  -> authenticate, validate schema, bind requestId/nonce
FACTS  -> read authoritative wallet/merchant/state data
AI     -> optional bounded recommendation with decisionId/version
POLICY -> enforce ownership, hard limits, approvals, risk fallback
LEDGER -> submit one idempotent deterministic transaction when value changes
EVENT  -> update projection/case/monitoring asynchronously
VERIFY -> discover committed result and retain correlationId + txId evidence
```

Verification Checklist

- The calibration report exists and is version-controlled.
- The feature works with all public Internet/WAN connectivity disabled.
- Authorization, malformed input, duplicate retry, and failure behavior have explicit tests.
- If financial value changes, journal debits and credits balance exactly and historical entries remain immutable.
- If AI is involved, model/prompt/feature version and fallback behavior are recorded and deterministic policy remains final.
- A timeout/restart cannot create a duplicate financial result; request status can be rediscovered.
- Logs/evidence contain correlation references but not secrets or prohibited sensitive payment data.

Common Failure Modes

- Data leakage in training.
- Threshold chosen only for headline accuracy.
- No explainable reason codes.
- Drift ignored until customer complaints rise.

Expert Review Questions

- What happens if this component succeeds but the next component fails?
- How is the authorizing identity/device/key revoked or recovered?
- Can the operation be safely retried after the client loses the response?
- What happens if AI is unavailable, slow, or wrong?
- Can an auditor reconstruct debit, credit, actor, risk decision, approval, and Fabric txId from retained evidence?
- Which compensating transaction resolves a downstream operational error without rewriting history?

Chapter 173 - Reason Codes and Explainability

Provide actionable evidence to analysts and customer-support operators. Fraud engineering combines deterministic rules, behavioral features, statistical/ML models, explainable reason codes, calibrated thresholds, and a human-review path. The target is not maximum blocking; it is measurable risk reduction at an acceptable false-positive cost.

Design rule: Map dominant features/rules to stable codes

Implementation Objective

The required output is: Reason-code standard. It must be version-controlled, reviewable, testable with the WAN disabled, and linked to the architecture/source-of-truth model. For a payment platform, every implementation decision should make it possible to explain who authorized an action, which facts were trusted, which controls ran, what financial journal committed, and how the result is recovered after failure.

Key Engineering Decisions

- Map dominant features/rules to stable codes
- avoid unsupported causal claims
- log model and feature versions

Deep Implementation Guidance

Measure the operational system, not only the model. Review capacity, threshold volume, false-positive cost, analyst overrides, latency, drift, and fraud outcomes all determine whether a “better” model is actually safer in production.

Data and Control Questions

- What is the authoritative source for every input used by this component?
- Which user, device, service, MSP, or administrator identity is allowed to initiate or approve it?
- Does the operation move value, reserve value, create a side effect, or only provide advisory information?
- What idempotency/replay key proves that a retry is the same business intent?
- What evidence is retained off-chain and what stable reference/hash belongs on the ledger?

Chapter 173 Implementation - Reason Codes and Explainability

Implementation Procedure

861. Start with transparent deterministic fraud rules and measurable features.
862. Build labeled datasets without leakage and split by time/entity as appropriate.
863. Train/evaluate model with precision, recall, false-positive cost, and calibration.
864. Serve locally with a strict schema and bounded latency.
865. Monitor drift/override rates and retain manual fallback when model service is unavailable.

Implementation Example

```
# Reason Codes and Explainability
INPUT  -> authenticate, validate schema, bind requestId/nonce
FACTS  -> read authoritative wallet/merchant/state data
AI     -> optional bounded recommendation with decisionId/version
POLICY -> enforce ownership, hard limits, approvals, risk fallback
LEDGER -> submit one idempotent deterministic transaction when value changes
EVENT  -> update projection/case/monitoring asynchronously
VERIFY -> discover committed result and retain correlationId + txId evidence
```

Verification Checklist

- The reason-code standard exists and is version-controlled.
- The feature works with all public Internet/WAN connectivity disabled.
- Authorization, malformed input, duplicate retry, and failure behavior have explicit tests.
- If financial value changes, journal debits and credits balance exactly and historical entries remain immutable.
- If AI is involved, model/prompt/feature version and fallback behavior are recorded and deterministic policy remains final.
- A timeout/restart cannot create a duplicate financial result; request status can be rediscovered.
- Logs/evidence contain correlation references but not secrets or prohibited sensitive payment data.

Common Failure Modes

- Data leakage in training.
- Threshold chosen only for headline accuracy.
- No explainable reason codes.
- Drift ignored until customer complaints rise.

Expert Review Questions

- What happens if this component succeeds but the next component fails?
- How is the authorizing identity/device/key revoked or recovered?
- Can the operation be safely retried after the client loses the response?
- What happens if AI is unavailable, slow, or wrong?
- Can an auditor reconstruct debit, credit, actor, risk decision, approval, and Fabric txId from retained evidence?
- Which compensating transaction resolves a downstream operational error without rewriting history?

Chapter 174 - False Positive Management

Prevent overly aggressive fraud controls from destroying usability. Fraud engineering combines deterministic rules, behavioral features, statistical/ML models, explainable reason codes, calibrated thresholds, and a human-review path. The target is not maximum blocking; it is measurable risk reduction at an acceptable false-positive cost.

Design rule: Track review approval rate

Implementation Objective

The required output is: False-positive program. It must be version-controlled, reviewable, testable with the WAN disabled, and linked to the architecture/source-of-truth model. For a payment platform, every implementation decision should make it possible to explain who authorized an action, which facts were trusted, which controls ran, what financial journal committed, and how the result is recovered after failure.

Key Engineering Decisions

- Track review approval rate
- customer friction
- segment-specific errors
- analyst overrides
- adjust thresholds under change control

Deep Implementation Guidance

Measure the operational system, not only the model. Review capacity, threshold volume, false-positive cost, analyst overrides, latency, drift, and fraud outcomes all determine whether a “better” model is actually safer in production.

Data and Control Questions

- What is the authoritative source for every input used by this component?
- Which user, device, service, MSP, or administrator identity is allowed to initiate or approve it?
- Does the operation move value, reserve value, create a side effect, or only provide advisory information?
- What idempotency/replay key proves that a retry is the same business intent?
- What evidence is retained off-chain and what stable reference/hash belongs on the ledger?

Chapter 174 Implementation - False Positive Management

Implementation Procedure

- 866. Start with transparent deterministic fraud rules and measurable features.
- 867. Build labeled datasets without leakage and split by time/entity as appropriate.
- 868. Train/evaluate model with precision, recall, false-positive cost, and calibration.
- 869. Serve locally with a strict schema and bounded latency.
- 870. Monitor drift/override rates and retain manual fallback when model service is unavailable.

Implementation Example

```
# False Positive Management
INPUT  -> authenticate, validate schema, bind requestId/nonce
FACTS  -> read authoritative wallet/merchant/state data
AI     -> optional bounded recommendation with decisionId/version
POLICY -> enforce ownership, hard limits, approvals, risk fallback
LEDGER -> submit one idempotent deterministic transaction when value changes
EVENT  -> update projection/case/monitoring asynchronously
VERIFY -> discover committed result and retain correlationId + txId evidence
```

Verification Checklist

- The false-positive program exists and is version-controlled.
- The feature works with all public Internet/WAN connectivity disabled.
- Authorization, malformed input, duplicate retry, and failure behavior have explicit tests.
- If financial value changes, journal debits and credits balance exactly and historical entries remain immutable.
- If AI is involved, model/prompt/feature version and fallback behavior are recorded and deterministic policy remains final.
- A timeout/restart cannot create a duplicate financial result; request status can be rediscovered.
- Logs/evidence contain correlation references but not secrets or prohibited sensitive payment data.

Common Failure Modes

- Data leakage in training.
- Threshold chosen only for headline accuracy.
- No explainable reason codes.
- Drift ignored until customer complaints rise.

Expert Review Questions

- What happens if this component succeeds but the next component fails?
- How is the authorizing identity/device/key revoked or recovered?
- Can the operation be safely retried after the client loses the response?
- What happens if AI is unavailable, slow, or wrong?
- Can an auditor reconstruct debit, credit, actor, risk decision, approval, and Fabric txId from retained evidence?
- Which compensating transaction resolves a downstream operational error without rewriting history?

Chapter 175 - Model Drift Monitoring

Detect when behavior/data distribution changes. Fraud engineering combines deterministic rules, behavioral features, statistical/ML models, explainable reason codes, calibrated thresholds, and a human-review path. The target is not maximum blocking; it is measurable risk reduction at an acceptable false-positive cost.

Design rule: Feature distribution shifts

Implementation Objective

The required output is: Drift dashboard. It must be version-controlled, reviewable, testable with the WAN disabled, and linked to the architecture/source-of-truth model. For a payment platform, every implementation decision should make it possible to explain who authorized an action, which facts were trusted, which controls ran, what financial journal committed, and how the result is recovered after failure.

Key Engineering Decisions

- Feature distribution shifts
- score distribution
- approval/rejection rate
- confirmed fraud rate
- model age

Deep Implementation Guidance

Measure the operational system, not only the model. Review capacity, threshold volume, false-positive cost, analyst overrides, latency, drift, and fraud outcomes all determine whether a “better” model is actually safer in production.

Data and Control Questions

- What is the authoritative source for every input used by this component?
- Which user, device, service, MSP, or administrator identity is allowed to initiate or approve it?
- Does the operation move value, reserve value, create a side effect, or only provide advisory information?
- What idempotency/replay key proves that a retry is the same business intent?
- What evidence is retained off-chain and what stable reference/hash belongs on the ledger?

Chapter 175 Implementation - Model Drift Monitoring

Implementation Procedure

871. Start with transparent deterministic fraud rules and measurable features.
872. Build labeled datasets without leakage and split by time/entity as appropriate.
873. Train/evaluate model with precision, recall, false-positive cost, and calibration.
874. Serve locally with a strict schema and bounded latency.
875. Monitor drift/override rates and retain manual fallback when model service is unavailable.

Implementation Example

```
# Model Drift Monitoring
INPUT  -> authenticate, validate schema, bind requestId/nonce
FACTS  -> read authoritative wallet/merchant/state data
AI     -> optional bounded recommendation with decisionId/version
POLICY -> enforce ownership, hard limits, approvals, risk fallback
LEDGER -> submit one idempotent deterministic transaction when value changes
EVENT  -> update projection/case/monitoring asynchronously
VERIFY -> discover committed result and retain correlationId + txId evidence
```

Verification Checklist

- The drift dashboard exists and is version-controlled.
- The feature works with all public Internet/WAN connectivity disabled.
- Authorization, malformed input, duplicate retry, and failure behavior have explicit tests.
- If financial value changes, journal debits and credits balance exactly and historical entries remain immutable.
- If AI is involved, model/prompt/feature version and fallback behavior are recorded and deterministic policy remains final.
- A timeout/restart cannot create a duplicate financial result; request status can be rediscovered.
- Logs/evidence contain correlation references but not secrets or prohibited sensitive payment data.

Common Failure Modes

- Data leakage in training.
- Threshold chosen only for headline accuracy.
- No explainable reason codes.
- Drift ignored until customer complaints rise.

Expert Review Questions

- What happens if this component succeeds but the next component fails?
- How is the authorizing identity/device/key revoked or recovered?
- Can the operation be safely retried after the client loses the response?
- What happens if AI is unavailable, slow, or wrong?
- Can an auditor reconstruct debit, credit, actor, risk decision, approval, and Fabric txId from retained evidence?
- Which compensating transaction resolves a downstream operational error without rewriting history?

Chapter 176 - Offline Training Pipeline

Train/retrain without production Internet dependency. Fraud engineering combines deterministic rules, behavioral features, statistical/ML models, explainable reason codes, calibrated thresholds, and a human-review path. The target is not maximum blocking; it is measurable risk reduction at an acceptable false-positive cost.

Design rule: Export approved pseudonymized dataset

Implementation Objective

The required output is: Training runbook. It must be version-controlled, reviewable, testable with the WAN disabled, and linked to the architecture/source-of-truth model. For a payment platform, every implementation decision should make it possible to explain who authorized an action, which facts were trusted, which controls ran, what financial journal committed, and how the result is recovered after failure.

Key Engineering Decisions

- Export approved pseudonymized dataset
- pin Python packages
- record seed/code/data hash
- produce signed model/eval bundle

Deep Implementation Guidance

Measure the operational system, not only the model. Review capacity, threshold volume, false-positive cost, analyst overrides, latency, drift, and fraud outcomes all determine whether a “better” model is actually safer in production.

Data and Control Questions

- What is the authoritative source for every input used by this component?
- Which user, device, service, MSP, or administrator identity is allowed to initiate or approve it?
- Does the operation move value, reserve value, create a side effect, or only provide advisory information?
- What idempotency/replay key proves that a retry is the same business intent?
- What evidence is retained off-chain and what stable reference/hash belongs on the ledger?

Chapter 176 Implementation - Offline Training Pipeline

Implementation Procedure

- 876. Start with transparent deterministic fraud rules and measurable features.
- 877. Build labeled datasets without leakage and split by time/entity as appropriate.
- 878. Train/evaluate model with precision, recall, false-positive cost, and calibration.
- 879. Serve locally with a strict schema and bounded latency.
- 880. Monitor drift/override rates and retain manual fallback when model service is unavailable.

Implementation Example

```
# Offline Training Pipeline
INPUT  -> authenticate, validate schema, bind requestId/nonce
FACTS  -> read authoritative wallet/merchant/state data
AI     -> optional bounded recommendation with decisionId/version
POLICY -> enforce ownership, hard limits, approvals, risk fallback
LEDGER -> submit one idempotent deterministic transaction when value changes
EVENT  -> update projection/case/monitoring asynchronously
VERIFY -> discover committed result and retain correlationId + txId evidence
```

Verification Checklist

- The training runbook exists and is version-controlled.
- The feature works with all public Internet/WAN connectivity disabled.
- Authorization, malformed input, duplicate retry, and failure behavior have explicit tests.
- If financial value changes, journal debits and credits balance exactly and historical entries remain immutable.
- If AI is involved, model/prompt/feature version and fallback behavior are recorded and deterministic policy remains final.
- A timeout/restart cannot create a duplicate financial result; request status can be rediscovered.
- Logs/evidence contain correlation references but not secrets or prohibited sensitive payment data.

Common Failure Modes

- Data leakage in training.
- Threshold chosen only for headline accuracy.
- No explainable reason codes.
- Drift ignored until customer complaints rise.

Expert Review Questions

- What happens if this component succeeds but the next component fails?
- How is the authorizing identity/device/key revoked or recovered?
- Can the operation be safely retried after the client loses the response?
- What happens if AI is unavailable, slow, or wrong?
- Can an auditor reconstruct debit, credit, actor, risk decision, approval, and Fabric txId from retained evidence?
- Which compensating transaction resolves a downstream operational error without rewriting history?

Chapter 177 - ONNX Model Export

Package approved models for consistent local inference. Fraud engineering combines deterministic rules, behavioral features, statistical/ML models, explainable reason codes, calibrated thresholds, and a human-review path. The target is not maximum blocking; it is measurable risk reduction at an acceptable false-positive cost.

Design rule: Export from training framework

Implementation Objective

The required output is: ONNX artifact. It must be version-controlled, reviewable, testable with the WAN disabled, and linked to the architecture/source-of-truth model. For a payment platform, every implementation decision should make it possible to explain who authorized an action, which facts were trusted, which controls ran, what financial journal committed, and how the result is recovered after failure.

Key Engineering Decisions

- Export from training framework
- verify numerical parity
- hash file
- store model metadata

Deep Implementation Guidance

Measure the operational system, not only the model. Review capacity, threshold volume, false-positive cost, analyst overrides, latency, drift, and fraud outcomes all determine whether a “better” model is actually safer in production.

Data and Control Questions

- What is the authoritative source for every input used by this component?
- Which user, device, service, MSP, or administrator identity is allowed to initiate or approve it?
- Does the operation move value, reserve value, create a side effect, or only provide advisory information?
- What idempotency/replay key proves that a retry is the same business intent?
- What evidence is retained off-chain and what stable reference/hash belongs on the ledger?

Chapter 177 Implementation - ONNX Model Export

Implementation Procedure

881. Start with transparent deterministic fraud rules and measurable features.
882. Build labeled datasets without leakage and split by time/entity as appropriate.
883. Train/evaluate model with precision, recall, false-positive cost, and calibration.
884. Serve locally with a strict schema and bounded latency.
885. Monitor drift/override rates and retain manual fallback when model service is unavailable.

Implementation Example

```
# ONNX Model Export
INPUT  -> authenticate, validate schema, bind requestId/nonce
FACTS  -> read authoritative wallet/merchant/state data
AI     -> optional bounded recommendation with decisionId/version
POLICY -> enforce ownership, hard limits, approvals, risk fallback
LEDGER -> submit one idempotent deterministic transaction when value changes
EVENT  -> update projection/case/monitoring asynchronously
VERIFY -> discover committed result and retain correlationId + txId evidence
```

Verification Checklist

- The onnx artifact exists and is version-controlled.
- The feature works with all public Internet/WAN connectivity disabled.
- Authorization, malformed input, duplicate retry, and failure behavior have explicit tests.
- If financial value changes, journal debits and credits balance exactly and historical entries remain immutable.
- If AI is involved, model/prompt/feature version and fallback behavior are recorded and deterministic policy remains final.
- A timeout/restart cannot create a duplicate financial result; request status can be rediscovered.
- Logs/evidence contain correlation references but not secrets or prohibited sensitive payment data.

Common Failure Modes

- Data leakage in training.
- Threshold chosen only for headline accuracy.
- No explainable reason codes.
- Drift ignored until customer complaints rise.

Expert Review Questions

- What happens if this component succeeds but the next component fails?
- How is the authorizing identity/device/key revoked or recovered?
- Can the operation be safely retried after the client loses the response?
- What happens if AI is unavailable, slow, or wrong?
- Can an auditor reconstruct debit, credit, actor, risk decision, approval, and Fabric txId from retained evidence?
- Which compensating transaction resolves a downstream operational error without rewriting history?

Chapter 178 - ONNX Runtime Risk Service

Serve low-latency local inference through a strict API. Fraud engineering combines deterministic rules, behavioral features, statistical/ML models, explainable reason codes, calibrated thresholds, and a human-review path. The target is not maximum blocking; it is measurable risk reduction at an acceptable false-positive cost.

Design rule: FastAPI/local service

Implementation Objective

The required output is: Risk service. It must be version-controlled, reviewable, testable with the WAN disabled, and linked to the architecture/source-of-truth model. For a payment platform, every implementation decision should make it possible to explain who authorized an action, which facts were trusted, which controls ran, what financial journal committed, and how the result is recovered after failure.

Key Engineering Decisions

- FastAPI/local service
- validate feature vector/schema
- bounded timeout
- health endpoint
- return structured decision

Deep Implementation Guidance

Measure the operational system, not only the model. Review capacity, threshold volume, false-positive cost, analyst overrides, latency, drift, and fraud outcomes all determine whether a “better” model is actually safer in production.

Data and Control Questions

- What is the authoritative source for every input used by this component?
- Which user, device, service, MSP, or administrator identity is allowed to initiate or approve it?
- Does the operation move value, reserve value, create a side effect, or only provide advisory information?
- What idempotency/replay key proves that a retry is the same business intent?
- What evidence is retained off-chain and what stable reference/hash belongs on the ledger?

Chapter 178 Implementation - ONNX Runtime Risk Service

Implementation Procedure

886. Start with transparent deterministic fraud rules and measurable features.
887. Build labeled datasets without leakage and split by time/entity as appropriate.
888. Train/evaluate model with precision, recall, false-positive cost, and calibration.
889. Serve locally with a strict schema and bounded latency.
890. Monitor drift/override rates and retain manual fallback when model service is unavailable.

Implementation Example

```
from fastapi import FastAPI
from pydantic import BaseModel, Field
import numpy as np
import onnxruntime as ort
import uuid

app = FastAPI()
session = ort.InferenceSession('/opt/models/risk-v7.onnx', providers=['CPUExecutionProvider'])

class RiskRequest(BaseModel):
    requestId: str
    amountMinor: int = Field(gt=0)
    features: list[float]

@app.post('/risk')
def risk(req: RiskRequest):
    x = np.asarray([req.features], dtype=np.float32)
    score = float(session.run(None, {session.get_inputs()[0].name: x})[0].reshape(-1)[0])
    recommendation = 'BLOCK' if score >= 0.95 else ('REVIEW' if score >= 0.70 else 'ALLOW')
    return {
        'decisionId': str(uuid.uuid4()),
        'modelVersion': 'risk-v7',
        'featureVersion': 'fv3',
        'riskScore': score,
        'recommendation': recommendation,
        'reasonCodes': []
    }
```

Verification Checklist

- The risk service exists and is version-controlled.
- The feature works with all public Internet/WAN connectivity disabled.
- Authorization, malformed input, duplicate retry, and failure behavior have explicit tests.
- If financial value changes, journal debits and credits balance exactly and historical entries remain immutable.
- If AI is involved, model/prompt/feature version and fallback behavior are recorded and deterministic policy remains final.
- A timeout/restart cannot create a duplicate financial result; request status can be rediscovered.
- Logs/evidence contain correlation references but not secrets or prohibited sensitive payment data.

Common Failure Modes

- Data leakage in training.
- Threshold chosen only for headline accuracy.
- No explainable reason codes.

- Drift ignored until customer complaints rise.

Expert Review Questions

- What happens if this component succeeds but the next component fails?
- How is the authorizing identity/device/key revoked or recovered?
- Can the operation be safely retried after the client loses the response?
- What happens if AI is unavailable, slow, or wrong?
- Can an auditor reconstruct debit, credit, actor, risk decision, approval, and Fabric txId from retained evidence?
- Which compensating transaction resolves a downstream operational error without rewriting history?

Chapter 179 - Model Service High Availability

Prevent risk service maintenance from becoming a payment outage. Fraud engineering combines deterministic rules, behavioral features, statistical/ML models, explainable reason codes, calibrated thresholds, and a human-review path. The target is not maximum blocking; it is measurable risk reduction at an acceptable false-positive cost.

Design rule: Two local instances or quick failover

Implementation Objective

The required output is: AI HA plan. It must be version-controlled, reviewable, testable with the WAN disabled, and linked to the architecture/source-of-truth model. For a payment platform, every implementation decision should make it possible to explain who authorized an action, which facts were trusted, which controls ran, what financial journal committed, and how the result is recovered after failure.

Key Engineering Decisions

- Two local instances or quick failover
- policy defines fallback
- keep deterministic rules in API
- monitor latency/errors

Deep Implementation Guidance

Measure the operational system, not only the model. Review capacity, threshold volume, false-positive cost, analyst overrides, latency, drift, and fraud outcomes all determine whether a “better” model is actually safer in production.

Data and Control Questions

- What is the authoritative source for every input used by this component?
- Which user, device, service, MSP, or administrator identity is allowed to initiate or approve it?
- Does the operation move value, reserve value, create a side effect, or only provide advisory information?
- What idempotency/replay key proves that a retry is the same business intent?
- What evidence is retained off-chain and what stable reference/hash belongs on the ledger?

Chapter 179 Implementation - Model Service High Availability

Implementation Procedure

891. Start with transparent deterministic fraud rules and measurable features.
892. Build labeled datasets without leakage and split by time/entity as appropriate.
893. Train/evaluate model with precision, recall, false-positive cost, and calibration.
894. Serve locally with a strict schema and bounded latency.
895. Monitor drift/override rates and retain manual fallback when model service is unavailable.

Implementation Example

```
# Model Service High Availability
INPUT  -> authenticate, validate schema, bind requestId/nonce
FACTS  -> read authoritative wallet/merchant/state data
AI     -> optional bounded recommendation with decisionId/version
POLICY -> enforce ownership, hard limits, approvals, risk fallback
LEDGER -> submit one idempotent deterministic transaction when value changes
EVENT  -> update projection/case/monitoring asynchronously
VERIFY -> discover committed result and retain correlationId + txId evidence
```

Verification Checklist

- The ai ha plan exists and is version-controlled.
- The feature works with all public Internet/WAN connectivity disabled.
- Authorization, malformed input, duplicate retry, and failure behavior have explicit tests.
- If financial value changes, journal debits and credits balance exactly and historical entries remain immutable.
- If AI is involved, model/prompt/feature version and fallback behavior are recorded and deterministic policy remains final.
- A timeout/restart cannot create a duplicate financial result; request status can be rediscovered.
- Logs/evidence contain correlation references but not secrets or prohibited sensitive payment data.

Common Failure Modes

- Data leakage in training.
- Threshold chosen only for headline accuracy.
- No explainable reason codes.
- Drift ignored until customer complaints rise.

Expert Review Questions

- What happens if this component succeeds but the next component fails?
- How is the authorizing identity/device/key revoked or recovered?
- Can the operation be safely retried after the client loses the response?
- What happens if AI is unavailable, slow, or wrong?
- Can an auditor reconstruct debit, credit, actor, risk decision, approval, and Fabric txId from retained evidence?
- Which compensating transaction resolves a downstream operational error without rewriting history?

Chapter 180 - Rules Plus Model Ensemble

Combine transparent rules with model score safely. Fraud engineering combines deterministic rules, behavioral features, statistical/ML models, explainable reason codes, calibrated thresholds, and a human-review path. The target is not maximum blocking; it is measurable risk reduction at an acceptable false-positive cost.

Design rule: Hard rule can force BLOCK/REVIEW

Implementation Objective

The required output is: Risk combination policy. It must be version-controlled, reviewable, testable with the WAN disabled, and linked to the architecture/source-of-truth model. For a payment platform, every implementation decision should make it possible to explain who authorized an action, which facts were trusted, which controls ran, what financial journal committed, and how the result is recovered after failure.

Key Engineering Decisions

- Hard rule can force BLOCK/REVIEW
- model influences soft threshold
- never let model override hard security denial

Deep Implementation Guidance

Measure the operational system, not only the model. Review capacity, threshold volume, false-positive cost, analyst overrides, latency, drift, and fraud outcomes all determine whether a “better” model is actually safer in production.

Data and Control Questions

- What is the authoritative source for every input used by this component?
- Which user, device, service, MSP, or administrator identity is allowed to initiate or approve it?
- Does the operation move value, reserve value, create a side effect, or only provide advisory information?
- What idempotency/replay key proves that a retry is the same business intent?
- What evidence is retained off-chain and what stable reference/hash belongs on the ledger?

Chapter 180 Implementation - Rules Plus Model Ensemble

Implementation Procedure

896. Start with transparent deterministic fraud rules and measurable features.
897. Build labeled datasets without leakage and split by time/entity as appropriate.
898. Train/evaluate model with precision, recall, false-positive cost, and calibration.
899. Serve locally with a strict schema and bounded latency.
900. Monitor drift/override rates and retain manual fallback when model service is unavailable.

Implementation Example

```
# Rules Plus Model Ensemble
INPUT  -> authenticate, validate schema, bind requestId/nonce
FACTS  -> read authoritative wallet/merchant/state data
AI     -> optional bounded recommendation with decisionId/version
POLICY -> enforce ownership, hard limits, approvals, risk fallback
LEDGER -> submit one idempotent deterministic transaction when value changes
EVENT  -> update projection/case/monitoring asynchronously
VERIFY -> discover committed result and retain correlationId + txId evidence
```

Verification Checklist

- The risk combination policy exists and is version-controlled.
- The feature works with all public Internet/WAN connectivity disabled.
- Authorization, malformed input, duplicate retry, and failure behavior have explicit tests.
- If financial value changes, journal debits and credits balance exactly and historical entries remain immutable.
- If AI is involved, model/prompt/feature version and fallback behavior are recorded and deterministic policy remains final.
- A timeout/restart cannot create a duplicate financial result; request status can be rediscovered.
- Logs/evidence contain correlation references but not secrets or prohibited sensitive payment data.

Common Failure Modes

- Data leakage in training.
- Threshold chosen only for headline accuracy.
- No explainable reason codes.
- Drift ignored until customer complaints rise.

Expert Review Questions

- What happens if this component succeeds but the next component fails?
- How is the authorizing identity/device/key revoked or recovered?
- Can the operation be safely retried after the client loses the response?
- What happens if AI is unavailable, slow, or wrong?
- Can an auditor reconstruct debit, credit, actor, risk decision, approval, and Fabric txId from retained evidence?
- Which compensating transaction resolves a downstream operational error without rewriting history?

Chapter 181 - Manual Review Feedback Loop

Capture analyst outcomes for evaluation/retraining without contaminating labels. Fraud engineering combines deterministic rules, behavioral features, statistical/ML models, explainable reason codes, calibrated thresholds, and a human-review path. The target is not maximum blocking; it is measurable risk reduction at an acceptable false-positive cost.

Design rule: Record review action/reason/evidence

Implementation Objective

The required output is: Feedback schema. It must be version-controlled, reviewable, testable with the WAN disabled, and linked to the architecture/source-of-truth model. For a payment platform, every implementation decision should make it possible to explain who authorized an action, which facts were trusted, which controls ran, what financial journal committed, and how the result is recovered after failure.

Key Engineering Decisions

- Record review action/reason/evidence
- separate provisional from confirmed fraud label
- version labels

Deep Implementation Guidance

Measure the operational system, not only the model. Review capacity, threshold volume, false-positive cost, analyst overrides, latency, drift, and fraud outcomes all determine whether a “better” model is actually safer in production.

Data and Control Questions

- What is the authoritative source for every input used by this component?
- Which user, device, service, MSP, or administrator identity is allowed to initiate or approve it?
- Does the operation move value, reserve value, create a side effect, or only provide advisory information?
- What idempotency/replay key proves that a retry is the same business intent?
- What evidence is retained off-chain and what stable reference/hash belongs on the ledger?

Chapter 181 Implementation - Manual Review Feedback Loop

Implementation Procedure

901. Start with transparent deterministic fraud rules and measurable features.
902. Build labeled datasets without leakage and split by time/entity as appropriate.
903. Train/evaluate model with precision, recall, false-positive cost, and calibration.
904. Serve locally with a strict schema and bounded latency.
905. Monitor drift/override rates and retain manual fallback when model service is unavailable.

Implementation Example

```
# Manual Review Feedback Loop
INPUT  -> authenticate, validate schema, bind requestId/nonce
FACTS  -> read authoritative wallet/merchant/state data
AI     -> optional bounded recommendation with decisionId/version
POLICY -> enforce ownership, hard limits, approvals, risk fallback
LEDGER -> submit one idempotent deterministic transaction when value changes
EVENT  -> update projection/case/monitoring asynchronously
VERIFY -> discover committed result and retain correlationId + txId evidence
```

Verification Checklist

- The feedback schema exists and is version-controlled.
- The feature works with all public Internet/WAN connectivity disabled.
- Authorization, malformed input, duplicate retry, and failure behavior have explicit tests.
- If financial value changes, journal debits and credits balance exactly and historical entries remain immutable.
- If AI is involved, model/prompt/feature version and fallback behavior are recorded and deterministic policy remains final.
- A timeout/restart cannot create a duplicate financial result; request status can be rediscovered.
- Logs/evidence contain correlation references but not secrets or prohibited sensitive payment data.

Common Failure Modes

- Data leakage in training.
- Threshold chosen only for headline accuracy.
- No explainable reason codes.
- Drift ignored until customer complaints rise.

Expert Review Questions

- What happens if this component succeeds but the next component fails?
- How is the authorizing identity/device/key revoked or recovered?
- Can the operation be safely retried after the client loses the response?
- What happens if AI is unavailable, slow, or wrong?
- Can an auditor reconstruct debit, credit, actor, risk decision, approval, and Fabric txId from retained evidence?
- Which compensating transaction resolves a downstream operational error without rewriting history?

Chapter 182 - Fraud Decision Audit Trail

Link each evaluated transaction to the exact rule/model context. Fraud engineering combines deterministic rules, behavioral features, statistical/ML models, explainable reason codes, calibrated thresholds, and a human-review path. The target is not maximum blocking; it is measurable risk reduction at an acceptable false-positive cost.

Design rule: requestId decisionId ruleSetVersion modelVersion featureVersion score recommendation reasonCodes analyst outcome txId

Implementation Objective

The required output is: Risk audit schema. It must be version-controlled, reviewable, testable with the WAN disabled, and linked to the architecture/source-of-truth model. For a payment platform, every implementation decision should make it possible to explain who authorized an action, which facts were trusted, which controls ran, what financial journal committed, and how the result is recovered after failure.

Key Engineering Decisions

- requestId decisionId ruleSetVersion modelVersion featureVersion score recommendation reasonCodes analyst outcome txId

Deep Implementation Guidance

Measure the operational system, not only the model. Review capacity, threshold volume, false-positive cost, analyst overrides, latency, drift, and fraud outcomes all determine whether a “better” model is actually safer in production.

Data and Control Questions

- What is the authoritative source for every input used by this component?
- Which user, device, service, MSP, or administrator identity is allowed to initiate or approve it?
- Does the operation move value, reserve value, create a side effect, or only provide advisory information?
- What idempotency/replay key proves that a retry is the same business intent?
- What evidence is retained off-chain and what stable reference/hash belongs on the ledger?

Chapter 182 Implementation - Fraud Decision Audit Trail

Implementation Procedure

906. Start with transparent deterministic fraud rules and measurable features.
907. Build labeled datasets without leakage and split by time/entity as appropriate.
908. Train/evaluate model with precision, recall, false-positive cost, and calibration.
909. Serve locally with a strict schema and bounded latency.
910. Monitor drift/override rates and retain manual fallback when model service is unavailable.

Implementation Example

```
# Fraud Decision Audit Trail
INPUT  -> authenticate, validate schema, bind requestId/nonce
FACTS  -> read authoritative wallet/merchant/state data
AI     -> optional bounded recommendation with decisionId/version
POLICY -> enforce ownership, hard limits, approvals, risk fallback
LEDGER -> submit one idempotent deterministic transaction when value changes
EVENT  -> update projection/case/monitoring asynchronously
VERIFY -> discover committed result and retain correlationId + txId evidence
```

Verification Checklist

- The risk audit schema exists and is version-controlled.
- The feature works with all public Internet/WAN connectivity disabled.
- Authorization, malformed input, duplicate retry, and failure behavior have explicit tests.
- If financial value changes, journal debits and credits balance exactly and historical entries remain immutable.
- If AI is involved, model/prompt/feature version and fallback behavior are recorded and deterministic policy remains final.
- A timeout/restart cannot create a duplicate financial result; request status can be rediscovered.
- Logs/evidence contain correlation references but not secrets or prohibited sensitive payment data.

Common Failure Modes

- Data leakage in training.
- Threshold chosen only for headline accuracy.
- No explainable reason codes.
- Drift ignored until customer complaints rise.

Expert Review Questions

- What happens if this component succeeds but the next component fails?
- How is the authorizing identity/device/key revoked or recovered?
- Can the operation be safely retried after the client loses the response?
- What happens if AI is unavailable, slow, or wrong?
- Can an auditor reconstruct debit, credit, actor, risk decision, approval, and Fabric txId from retained evidence?
- Which compensating transaction resolves a downstream operational error without rewriting history?

Chapter 183 - Adversarial Fraud Testing

Test obvious evasion and edge patterns before enforcement. Fraud engineering combines deterministic rules, behavioral features, statistical/ML models, explainable reason codes, calibrated thresholds, and a human-review path. The target is not maximum blocking; it is measurable risk reduction at an acceptable false-positive cost.

Design rule: Split amounts

Implementation Objective

The required output is: Fraud red-team cases. It must be version-controlled, reviewable, testable with the WAN disabled, and linked to the architecture/source-of-truth model. For a payment platform, every implementation decision should make it possible to explain who authorized an action, which facts were trusted, which controls ran, what financial journal committed, and how the result is recovered after failure.

Key Engineering Decisions

- Split amounts
- recipient rotation
- new device
- rapid reversals
- merchant collusion simulation
- model unavailable/poisoned input

Deep Implementation Guidance

Measure the operational system, not only the model. Review capacity, threshold volume, false-positive cost, analyst overrides, latency, drift, and fraud outcomes all determine whether a “better” model is actually safer in production.

Data and Control Questions

- What is the authoritative source for every input used by this component?
- Which user, device, service, MSP, or administrator identity is allowed to initiate or approve it?
- Does the operation move value, reserve value, create a side effect, or only provide advisory information?
- What idempotency/replay key proves that a retry is the same business intent?
- What evidence is retained off-chain and what stable reference/hash belongs on the ledger?

Chapter 183 Implementation - Adversarial Fraud Testing

Implementation Procedure

911. Start with transparent deterministic fraud rules and measurable features.
912. Build labeled datasets without leakage and split by time/entity as appropriate.
913. Train/evaluate model with precision, recall, false-positive cost, and calibration.
914. Serve locally with a strict schema and bounded latency.
915. Monitor drift/override rates and retain manual fallback when model service is unavailable.

Implementation Example

```
# Adversarial Fraud Testing
INPUT  -> authenticate, validate schema, bind requestId/nonce
FACTS  -> read authoritative wallet/merchant/state data
AI     -> optional bounded recommendation with decisionId/version
POLICY -> enforce ownership, hard limits, approvals, risk fallback
LEDGER -> submit one idempotent deterministic transaction when value changes
EVENT  -> update projection/case/monitoring asynchronously
VERIFY -> discover committed result and retain correlationId + txId evidence
```

Verification Checklist

- The fraud red-team cases exists and is version-controlled.
- The feature works with all public Internet/WAN connectivity disabled.
- Authorization, malformed input, duplicate retry, and failure behavior have explicit tests.
- If financial value changes, journal debits and credits balance exactly and historical entries remain immutable.
- If AI is involved, model/prompt/feature version and fallback behavior are recorded and deterministic policy remains final.
- A timeout/restart cannot create a duplicate financial result; request status can be rediscovered.
- Logs/evidence contain correlation references but not secrets or prohibited sensitive payment data.

Common Failure Modes

- Data leakage in training.
- Threshold chosen only for headline accuracy.
- No explainable reason codes.
- Drift ignored until customer complaints rise.

Expert Review Questions

- What happens if this component succeeds but the next component fails?
- How is the authorizing identity/device/key revoked or recovered?
- Can the operation be safely retried after the client loses the response?
- What happens if AI is unavailable, slow, or wrong?
- Can an auditor reconstruct debit, credit, actor, risk decision, approval, and Fabric txId from retained evidence?
- Which compensating transaction resolves a downstream operational error without rewriting history?

Part XI - Local LLM, RAG, Support, and Operations AI

A local LLM can explain transactions, summarize disputes, answer policy questions through RAG, and assist operators. It should be treated as an untrusted interpreter: schema-constrained outputs, evidence references, prompt-injection defenses, least-privilege tools, and no independent financial signing authority.

Part focus: This part contains 20 implementation chapters. Each chapter includes design rules, implementation procedure, a practical artifact, verification, failure analysis, and expert review questions.

Chapter 184 - Local llama.cpp Server

Run an approved GGUF language model entirely on LAN infrastructure. A local LLM can explain transactions, summarize disputes, answer policy questions through RAG, and assist operators. It should be treated as an untrusted interpreter: schema-constrained outputs, evidence references, prompt-injection defenses, least-privilege tools, and no independent financial signing authority.

Design rule: Bind only internal interface

Implementation Objective

The required output is: LLM serving runbook. It must be version-controlled, reviewable, testable with the WAN disabled, and linked to the architecture/source-of-truth model. For a payment platform, every implementation decision should make it possible to explain who authorized an action, which facts were trusted, which controls ran, what financial journal committed, and how the result is recovered after failure.

Key Engineering Decisions

- Bind only internal interface
- pin model hash and server version
- size context and concurrency
- disable public network access

Deep Implementation Guidance

RAG and local LLMs are useful because they turn approved documents and verified transaction facts into readable assistance. They remain probabilistic. The application must perform access control before retrieval, constrain tools, validate outputs, and use no-answer behavior when evidence is insufficient.

Data and Control Questions

- What is the authoritative source for every input used by this component?
- Which user, device, service, MSP, or administrator identity is allowed to initiate or approve it?
- Does the operation move value, reserve value, create a side effect, or only provide advisory information?
- What idempotency/replay key proves that a retry is the same business intent?
- What evidence is retained off-chain and what stable reference/hash belongs on the ledger?

Chapter 184 Implementation - Local llama.cpp Server

Implementation Procedure

916. Host approved model files locally and expose only LAN endpoints.
917. Use RAG for policy/operational knowledge and return evidence references.
918. Constrain outputs with JSON schema for machine-consumed tasks.
919. Treat retrieved/user content as untrusted and never reveal secrets/system prompts.
920. Evaluate groundedness, refusal/no-answer behavior, injection resistance, and latency.

Implementation Example

```
# Example local-only LLM server. Model file is pre-approved and copied into the LAN.
./llama-server \
-m /opt/models/approved-assistant.gguf \
--host 10.20.30.40 \
--port 8080

# Application base URL:
# http://10.20.30.40:8080/v1
# Firewall blocks any WAN egress from the LLM host.
```

Verification Checklist

- The llm serving runbook exists and is version-controlled.
- The feature works with all public Internet/WAN connectivity disabled.
- Authorization, malformed input, duplicate retry, and failure behavior have explicit tests.
- If financial value changes, journal debits and credits balance exactly and historical entries remain immutable.
- If AI is involved, model/prompt/feature version and fallback behavior are recorded and deterministic policy remains final.
- A timeout/restart cannot create a duplicate financial result; request status can be rediscovered.
- Logs/evidence contain correlation references but not secrets or prohibited sensitive payment data.

Common Failure Modes

- Prompt injection from user/RAG content.
- Hallucinated policy presented as fact.
- Unrestricted financial tools.
- No evidence/no-answer behavior.

Expert Review Questions

- What happens if this component succeeds but the next component fails?
- How is the authorizing identity/device/key revoked or recovered?
- Can the operation be safely retried after the client loses the response?
- What happens if AI is unavailable, slow, or wrong?
- Can an auditor reconstruct debit, credit, actor, risk decision, approval, and Fabric txId from retained evidence?
- Which compensating transaction resolves a downstream operational error without rewriting history?

Chapter 185 - OpenAI-Compatible Local API Pattern

Use a familiar API surface while keeping inference local. A local LLM can explain transactions, summarize disputes, answer policy questions through RAG, and assist operators. It should be treated as an untrusted interpreter: schema-constrained outputs, evidence references, prompt-injection defenses, least-privilege tools, and no independent financial signing authority.

Design rule: Point base URL to LAN llama-server

Implementation Objective

The required output is: Local LLM client module. It must be version-controlled, reviewable, testable with the WAN disabled, and linked to the architecture/source-of-truth model. For a payment platform, every implementation decision should make it possible to explain who authorized an action, which facts were trusted, which controls ran, what financial journal committed, and how the result is recovered after failure.

Key Engineering Decisions

- Point base URL to LAN llama-server
- set timeouts
- use structured response formats where supported
- do not accidentally fall back to cloud endpoint

Deep Implementation Guidance

RAG and local LLMs are useful because they turn approved documents and verified transaction facts into readable assistance. They remain probabilistic. The application must perform access control before retrieval, constrain tools, validate outputs, and use no-answer behavior when evidence is insufficient.

Data and Control Questions

- What is the authoritative source for every input used by this component?
- Which user, device, service, MSP, or administrator identity is allowed to initiate or approve it?
- Does the operation move value, reserve value, create a side effect, or only provide advisory information?
- What idempotency/replay key proves that a retry is the same business intent?
- What evidence is retained off-chain and what stable reference/hash belongs on the ledger?

Chapter 185 Implementation - OpenAI-Compatible Local API Pattern

Implementation Procedure

921. Host approved model files locally and expose only LAN endpoints.
922. Use RAG for policy/operational knowledge and return evidence references.
923. Constrain outputs with JSON schema for machine-consumed tasks.
924. Treat retrieved/user content as untrusted and never reveal secrets/system prompts.
925. Evaluate groundedness, refusal/no-answer behavior, injection resistance, and latency.

Implementation Example

```
# OpenAI-Compatible Local API Pattern
INPUT  -> authenticate, validate schema, bind requestId/nonce
FACTS  -> read authoritative wallet/merchant/state data
AI     -> optional bounded recommendation with decisionId/version
POLICY -> enforce ownership, hard limits, approvals, risk fallback
LEDGER -> submit one idempotent deterministic transaction when value changes
EVENT  -> update projection/case/monitoring asynchronously
VERIFY -> discover committed result and retain correlationId + txId evidence
```

Verification Checklist

- The local llm client module exists and is version-controlled.
- The feature works with all public Internet/WAN connectivity disabled.
- Authorization, malformed input, duplicate retry, and failure behavior have explicit tests.
- If financial value changes, journal debits and credits balance exactly and historical entries remain immutable.
- If AI is involved, model/prompt/feature version and fallback behavior are recorded and deterministic policy remains final.
- A timeout/restart cannot create a duplicate financial result; request status can be rediscovered.
- Logs/evidence contain correlation references but not secrets or prohibited sensitive payment data.

Common Failure Modes

- Prompt injection from user/RAG content.
- Hallucinated policy presented as fact.
- Unrestricted financial tools.
- No evidence/no-answer behavior.

Expert Review Questions

- What happens if this component succeeds but the next component fails?
- How is the authorizing identity/device/key revoked or recovered?
- Can the operation be safely retried after the client loses the response?
- What happens if AI is unavailable, slow, or wrong?
- Can an auditor reconstruct debit, credit, actor, risk decision, approval, and Fabric txId from retained evidence?
- Which compensating transaction resolves a downstream operational error without rewriting history?

Chapter 186 - Schema-Constrained LLM Outputs

Make machine-consumed AI results predictable and reject invalid structure. A local LLM can explain transactions, summarize disputes, answer policy questions through RAG, and assist operators. It should be treated as an untrusted interpreter: schema-constrained outputs, evidence references, prompt-injection defenses, least-privilege tools, and no independent financial signing authority.

Design rule: Use enums and required fields

Implementation Objective

The required output is: JSON schema contract. It must be version-controlled, reviewable, testable with the WAN disabled, and linked to the architecture/source-of-truth model. For a payment platform, every implementation decision should make it possible to explain who authorized an action, which facts were trusted, which controls ran, what financial journal committed, and how the result is recovered after failure.

Key Engineering Decisions

- Use enums and required fields
- validate again in application
- never parse critical authorization from prose

Deep Implementation Guidance

RAG and local LLMs are useful because they turn approved documents and verified transaction facts into readable assistance. They remain probabilistic. The application must perform access control before retrieval, constrain tools, validate outputs, and use no-answer behavior when evidence is insufficient.

Data and Control Questions

- What is the authoritative source for every input used by this component?
- Which user, device, service, MSP, or administrator identity is allowed to initiate or approve it?
- Does the operation move value, reserve value, create a side effect, or only provide advisory information?
- What idempotency/replay key proves that a retry is the same business intent?
- What evidence is retained off-chain and what stable reference/hash belongs on the ledger?

Chapter 186 Implementation - Schema-Constrained LLM Outputs

Implementation Procedure

926. Host approved model files locally and expose only LAN endpoints.
927. Use RAG for policy/operational knowledge and return evidence references.
928. Constrain outputs with JSON schema for machine-consumed tasks.
929. Treat retrieved/user content as untrusted and never reveal secrets/system prompts.
930. Evaluate groundedness, refusal/no-answer behavior, injection resistance, and latency.

Implementation Example

```
{
  "type": "json_schema",
  "json_schema": {
    "name": "support_result",
    "strict": true,
    "schema": {
      "type": "object",
      "properties": {
        "summary": {"type": "string"},
        "category": {"type": "string", "enum": ["payment_pending", "refund", "access", "fraud_concern", "merchant", "other"]},
        "requiresHumanReview": {"type": "boolean"},
        "evidenceRefs": {"type": "array", "items": {"type": "string"}}
      },
      "required": ["summary", "category", "requiresHumanReview", "evidenceRefs"],
      "additionalProperties": false
    }
  }
}
```

Verification Checklist

- The json schema contract exists and is version-controlled.
- The feature works with all public Internet/WAN connectivity disabled.
- Authorization, malformed input, duplicate retry, and failure behavior have explicit tests.
- If financial value changes, journal debits and credits balance exactly and historical entries remain immutable.
- If AI is involved, model/prompt/feature version and fallback behavior are recorded and deterministic policy remains final.
- A timeout/restart cannot create a duplicate financial result; request status can be rediscovered.
- Logs/evidence contain correlation references but not secrets or prohibited sensitive payment data.

Common Failure Modes

- Prompt injection from user/RAG content.
- Hallucinated policy presented as fact.
- Unrestricted financial tools.
- No evidence/no-answer behavior.

Expert Review Questions

- What happens if this component succeeds but the next component fails?
- How is the authorizing identity/device/key revoked or recovered?
- Can the operation be safely retried after the client loses the response?
- What happens if AI is unavailable, slow, or wrong?

- Can an auditor reconstruct debit, credit, actor, risk decision, approval, and Fabric txId from retained evidence?
- Which compensating transaction resolves a downstream operational error without rewriting history?

Chapter 187 - Local Embeddings Service

Generate embeddings on the LAN for RAG and similarity tasks. A local LLM can explain transactions, summarize disputes, answer policy questions through RAG, and assist operators. It should be treated as an untrusted interpreter: schema-constrained outputs, evidence references, prompt-injection defenses, least-privilege tools, and no independent financial signing authority.

Design rule: Pin embedding model/version/dimension

Implementation Objective

The required output is: Embedding service. It must be version-controlled, reviewable, testable with the WAN disabled, and linked to the architecture/source-of-truth model. For a payment platform, every implementation decision should make it possible to explain who authorized an action, which facts were trusted, which controls ran, what financial journal committed, and how the result is recovered after failure.

Key Engineering Decisions

- Pin embedding model/version/dimension
- normalize consistently
- batch offline ingestion
- record model hash

Deep Implementation Guidance

RAG and local LLMs are useful because they turn approved documents and verified transaction facts into readable assistance. They remain probabilistic. The application must perform access control before retrieval, constrain tools, validate outputs, and use no-answer behavior when evidence is insufficient.

Data and Control Questions

- What is the authoritative source for every input used by this component?
- Which user, device, service, MSP, or administrator identity is allowed to initiate or approve it?
- Does the operation move value, reserve value, create a side effect, or only provide advisory information?
- What idempotency/replay key proves that a retry is the same business intent?
- What evidence is retained off-chain and what stable reference/hash belongs on the ledger?

Chapter 187 Implementation - Local Embeddings Service

Implementation Procedure

931. Host approved model files locally and expose only LAN endpoints.
932. Use RAG for policy/operational knowledge and return evidence references.
933. Constrain outputs with JSON schema for machine-consumed tasks.
934. Treat retrieved/user content as untrusted and never reveal secrets/system prompts.
935. Evaluate groundedness, refusal/no-answer behavior, injection resistance, and latency.

Implementation Example

```
# Local Embeddings Service
INPUT  -> authenticate, validate schema, bind requestId/nonce
FACTS  -> read authoritative wallet/merchant/state data
AI     -> optional bounded recommendation with decisionId/version
POLICY -> enforce ownership, hard limits, approvals, risk fallback
LEDGER -> submit one idempotent deterministic transaction when value changes
EVENT  -> update projection/case/monitoring asynchronously
VERIFY -> discover committed result and retain correlationId + txId evidence
```

Verification Checklist

- The embedding service exists and is version-controlled.
- The feature works with all public Internet/WAN connectivity disabled.
- Authorization, malformed input, duplicate retry, and failure behavior have explicit tests.
- If financial value changes, journal debits and credits balance exactly and historical entries remain immutable.
- If AI is involved, model/prompt/feature version and fallback behavior are recorded and deterministic policy remains final.
- A timeout/restart cannot create a duplicate financial result; request status can be rediscovered.
- Logs/evidence contain correlation references but not secrets or prohibited sensitive payment data.

Common Failure Modes

- Prompt injection from user/RAG content.
- Hallucinated policy presented as fact.
- Unrestricted financial tools.
- No evidence/no-answer behavior.

Expert Review Questions

- What happens if this component succeeds but the next component fails?
- How is the authorizing identity/device/key revoked or recovered?
- Can the operation be safely retried after the client loses the response?
- What happens if AI is unavailable, slow, or wrong?
- Can an auditor reconstruct debit, credit, actor, risk decision, approval, and Fabric txId from retained evidence?
- Which compensating transaction resolves a downstream operational error without rewriting history?

Chapter 188 - PostgreSQL plus pgvector

Store policy/document embeddings in the existing local database where appropriate. A local LLM can explain transactions, summarize disputes, answer policy questions through RAG, and assist operators. It should be treated as an untrusted interpreter: schema-constrained outputs, evidence references, prompt-injection defenses, least-privilege tools, and no independent financial signing authority.

Design rule: Use vector column and metadata

Implementation Objective

The required output is: Vector schema. It must be version-controlled, reviewable, testable with the WAN disabled, and linked to the architecture/source-of-truth model. For a payment platform, every implementation decision should make it possible to explain who authorized an action, which facts were trusted, which controls ran, what financial journal committed, and how the result is recovered after failure.

Key Engineering Decisions

- Use vector column and metadata
- filter by role/document status/version
- index only after measuring scale needs

Deep Implementation Guidance

RAG and local LLMs are useful because they turn approved documents and verified transaction facts into readable assistance. They remain probabilistic. The application must perform access control before retrieval, constrain tools, validate outputs, and use no-answer behavior when evidence is insufficient.

Data and Control Questions

- What is the authoritative source for every input used by this component?
- Which user, device, service, MSP, or administrator identity is allowed to initiate or approve it?
- Does the operation move value, reserve value, create a side effect, or only provide advisory information?
- What idempotency/replay key proves that a retry is the same business intent?
- What evidence is retained off-chain and what stable reference/hash belongs on the ledger?

Chapter 188 Implementation - PostgreSQL plus pgvector

Implementation Procedure

936. Host approved model files locally and expose only LAN endpoints.
937. Use RAG for policy/operational knowledge and return evidence references.
938. Constrain outputs with JSON schema for machine-consumed tasks.
939. Treat retrieved/user content as untrusted and never reveal secrets/system prompts.
940. Evaluate groundedness, refusal/no-answer behavior, injection resistance, and latency.

Implementation Example

```
CREATE EXTENSION IF NOT EXISTS vector;

CREATE TABLE knowledge_chunk (
  chunk_id text PRIMARY KEY,
  source_id text NOT NULL,
  source_version text NOT NULL,
  access_role text NOT NULL,
  content text NOT NULL,
  embedding vector(768) NOT NULL,
  active boolean NOT NULL DEFAULT true
);

-- Conceptual cosine-distance retrieval; filter authorization BEFORE returning content.
SELECT chunk_id, source_id, content
FROM knowledge_chunk
WHERE active = true AND access_role = ANY($2)
ORDER BY embedding <=> $1
LIMIT 6;
```

Verification Checklist

- The vector schema exists and is version-controlled.
- The feature works with all public Internet/WAN connectivity disabled.
- Authorization, malformed input, duplicate retry, and failure behavior have explicit tests.
- If financial value changes, journal debits and credits balance exactly and historical entries remain immutable.
- If AI is involved, model/prompt/feature version and fallback behavior are recorded and deterministic policy remains final.
- A timeout/restart cannot create a duplicate financial result; request status can be rediscovered.
- Logs/evidence contain correlation references but not secrets or prohibited sensitive payment data.

Common Failure Modes

- Prompt injection from user/RAG content.
- Hallucinated policy presented as fact.
- Unrestricted financial tools.
- No evidence/no-answer behavior.

Expert Review Questions

- What happens if this component succeeds but the next component fails?
- How is the authorizing identity/device/key revoked or recovered?
- Can the operation be safely retried after the client loses the response?
- What happens if AI is unavailable, slow, or wrong?

- Can an auditor reconstruct debit, credit, actor, risk decision, approval, and Fabric txId from retained evidence?
- Which compensating transaction resolves a downstream operational error without rewriting history?

Chapter 189 - Knowledge Corpus Governance

Control what documents can influence the local assistant. A local LLM can explain transactions, summarize disputes, answer policy questions through RAG, and assist operators. It should be treated as an untrusted interpreter: schema-constrained outputs, evidence references, prompt-injection defenses, least-privilege tools, and no independent financial signing authority.

Design rule: Approved source owner/version/effective date/access level

Implementation Objective

The required output is: Corpus register. It must be version-controlled, reviewable, testable with the WAN disabled, and linked to the architecture/source-of-truth model. For a payment platform, every implementation decision should make it possible to explain who authorized an action, which facts were trusted, which controls ran, what financial journal committed, and how the result is recovered after failure.

Key Engineering Decisions

- Approved source owner/version/effective date/access level
- remove obsolete policy from active index
- retain archival evidence separately

Deep Implementation Guidance

RAG and local LLMs are useful because they turn approved documents and verified transaction facts into readable assistance. They remain probabilistic. The application must perform access control before retrieval, constrain tools, validate outputs, and use no-answer behavior when evidence is insufficient.

Data and Control Questions

- What is the authoritative source for every input used by this component?
- Which user, device, service, MSP, or administrator identity is allowed to initiate or approve it?
- Does the operation move value, reserve value, create a side effect, or only provide advisory information?
- What idempotency/replay key proves that a retry is the same business intent?
- What evidence is retained off-chain and what stable reference/hash belongs on the ledger?

Chapter 189 Implementation - Knowledge Corpus Governance

Implementation Procedure

941. Host approved model files locally and expose only LAN endpoints.
942. Use RAG for policy/operational knowledge and return evidence references.
943. Constrain outputs with JSON schema for machine-consumed tasks.
944. Treat retrieved/user content as untrusted and never reveal secrets/system prompts.
945. Evaluate groundedness, refusal/no-answer behavior, injection resistance, and latency.

Implementation Example

```
# Knowledge Corpus Governance
INPUT  -> authenticate, validate schema, bind requestId/nonce
FACTS  -> read authoritative wallet/merchant/state data
AI     -> optional bounded recommendation with decisionId/version
POLICY -> enforce ownership, hard limits, approvals, risk fallback
LEDGER -> submit one idempotent deterministic transaction when value changes
EVENT  -> update projection/case/monitoring asynchronously
VERIFY -> discover committed result and retain correlationId + txId evidence
```

Verification Checklist

- The corpus register exists and is version-controlled.
- The feature works with all public Internet/WAN connectivity disabled.
- Authorization, malformed input, duplicate retry, and failure behavior have explicit tests.
- If financial value changes, journal debits and credits balance exactly and historical entries remain immutable.
- If AI is involved, model/prompt/feature version and fallback behavior are recorded and deterministic policy remains final.
- A timeout/restart cannot create a duplicate financial result; request status can be rediscovered.
- Logs/evidence contain correlation references but not secrets or prohibited sensitive payment data.

Common Failure Modes

- Prompt injection from user/RAG content.
- Hallucinated policy presented as fact.
- Unrestricted financial tools.
- No evidence/no-answer behavior.

Expert Review Questions

- What happens if this component succeeds but the next component fails?
- How is the authorizing identity/device/key revoked or recovered?
- Can the operation be safely retried after the client loses the response?
- What happens if AI is unavailable, slow, or wrong?
- Can an auditor reconstruct debit, credit, actor, risk decision, approval, and Fabric txId from retained evidence?
- Which compensating transaction resolves a downstream operational error without rewriting history?

Chapter 190 - Document Parsing and Chunking

Prepare local policies, procedures, manuals, and FAQs for reliable retrieval. A local LLM can explain transactions, summarize disputes, answer policy questions through RAG, and assist operators. It should be treated as an untrusted interpreter: schema-constrained outputs, evidence references, prompt-injection defenses, least-privilege tools, and no independent financial signing authority.

Design rule: Preserve headings/source/page

Implementation Objective

The required output is: Ingestion pipeline. It must be version-controlled, reviewable, testable with the WAN disabled, and linked to the architecture/source-of-truth model. For a payment platform, every implementation decision should make it possible to explain who authorized an action, which facts were trusted, which controls ran, what financial journal committed, and how the result is recovered after failure.

Key Engineering Decisions

- Preserve headings/source/page
- use structure-aware chunks
- avoid mixing unrelated sections
- store provenance metadata

Deep Implementation Guidance

RAG and local LLMs are useful because they turn approved documents and verified transaction facts into readable assistance. They remain probabilistic. The application must perform access control before retrieval, constrain tools, validate outputs, and use no-answer behavior when evidence is insufficient.

Data and Control Questions

- What is the authoritative source for every input used by this component?
- Which user, device, service, MSP, or administrator identity is allowed to initiate or approve it?
- Does the operation move value, reserve value, create a side effect, or only provide advisory information?
- What idempotency/replay key proves that a retry is the same business intent?
- What evidence is retained off-chain and what stable reference/hash belongs on the ledger?

Chapter 190 Implementation - Document Parsing and Chunking

Implementation Procedure

- 946. Host approved model files locally and expose only LAN endpoints.
- 947. Use RAG for policy/operational knowledge and return evidence references.
- 948. Constrain outputs with JSON schema for machine-consumed tasks.
- 949. Treat retrieved/user content as untrusted and never reveal secrets/system prompts.
- 950. Evaluate groundedness, refusal/no-answer behavior, injection resistance, and latency.

Implementation Example

```
# Document Parsing and Chunking
INPUT  -> authenticate, validate schema, bind requestId/nonce
FACTS  -> read authoritative wallet/merchant/state data
AI     -> optional bounded recommendation with decisionId/version
POLICY -> enforce ownership, hard limits, approvals, risk fallback
LEDGER -> submit one idempotent deterministic transaction when value changes
EVENT  -> update projection/case/monitoring asynchronously
VERIFY -> discover committed result and retain correlationId + txId evidence
```

Verification Checklist

- The ingestion pipeline exists and is version-controlled.
- The feature works with all public Internet/WAN connectivity disabled.
- Authorization, malformed input, duplicate retry, and failure behavior have explicit tests.
- If financial value changes, journal debits and credits balance exactly and historical entries remain immutable.
- If AI is involved, model/prompt/feature version and fallback behavior are recorded and deterministic policy remains final.
- A timeout/restart cannot create a duplicate financial result; request status can be rediscovered.
- Logs/evidence contain correlation references but not secrets or prohibited sensitive payment data.

Common Failure Modes

- Prompt injection from user/RAG content.
- Hallucinated policy presented as fact.
- Unrestricted financial tools.
- No evidence/no-answer behavior.

Expert Review Questions

- What happens if this component succeeds but the next component fails?
- How is the authorizing identity/device/key revoked or recovered?
- Can the operation be safely retried after the client loses the response?
- What happens if AI is unavailable, slow, or wrong?
- Can an auditor reconstruct debit, credit, actor, risk decision, approval, and Fabric txId from retained evidence?
- Which compensating transaction resolves a downstream operational error without rewriting history?

Chapter 191 - Policy RAG Assistant

Answer wallet/payment policy questions with retrieved evidence rather than model memory. A local LLM can explain transactions, summarize disputes, answer policy questions through RAG, and assist operators. It should be treated as an untrusted interpreter: schema-constrained outputs, evidence references, prompt-injection defenses, least-privilege tools, and no independent financial signing authority.

Design rule: Retrieve permitted chunks

Implementation Objective

The required output is: RAG workflow. It must be version-controlled, reviewable, testable with the WAN disabled, and linked to the architecture/source-of-truth model. For a payment platform, every implementation decision should make it possible to explain who authorized an action, which facts were trusted, which controls ran, what financial journal committed, and how the result is recovered after failure.

Key Engineering Decisions

- Retrieve permitted chunks
- instruct model to answer only from evidence
- return source references
- no-answer if evidence insufficient

Deep Implementation Guidance

RAG and local LLMs are useful because they turn approved documents and verified transaction facts into readable assistance. They remain probabilistic. The application must perform access control before retrieval, constrain tools, validate outputs, and use no-answer behavior when evidence is insufficient.

Data and Control Questions

- What is the authoritative source for every input used by this component?
- Which user, device, service, MSP, or administrator identity is allowed to initiate or approve it?
- Does the operation move value, reserve value, create a side effect, or only provide advisory information?
- What idempotency/replay key proves that a retry is the same business intent?
- What evidence is retained off-chain and what stable reference/hash belongs on the ledger?

Chapter 191 Implementation - Policy RAG Assistant

Implementation Procedure

951. Host approved model files locally and expose only LAN endpoints.
952. Use RAG for policy/operational knowledge and return evidence references.
953. Constrain outputs with JSON schema for machine-consumed tasks.
954. Treat retrieved/user content as untrusted and never reveal secrets/system prompts.
955. Evaluate groundedness, refusal/no-answer behavior, injection resistance, and latency.

Implementation Example

```
# Policy RAG Assistant
INPUT  -> authenticate, validate schema, bind requestId/nonce
FACTS  -> read authoritative wallet/merchant/state data
AI     -> optional bounded recommendation with decisionId/version
POLICY -> enforce ownership, hard limits, approvals, risk fallback
LEDGER -> submit one idempotent deterministic transaction when value changes
EVENT  -> update projection/case/monitoring asynchronously
VERIFY -> discover committed result and retain correlationId + txId evidence
```

Verification Checklist

- The rag workflow exists and is version-controlled.
- The feature works with all public Internet/WAN connectivity disabled.
- Authorization, malformed input, duplicate retry, and failure behavior have explicit tests.
- If financial value changes, journal debits and credits balance exactly and historical entries remain immutable.
- If AI is involved, model/prompt/feature version and fallback behavior are recorded and deterministic policy remains final.
- A timeout/restart cannot create a duplicate financial result; request status can be rediscovered.
- Logs/evidence contain correlation references but not secrets or prohibited sensitive payment data.

Common Failure Modes

- Prompt injection from user/RAG content.
- Hallucinated policy presented as fact.
- Unrestricted financial tools.
- No evidence/no-answer behavior.

Expert Review Questions

- What happens if this component succeeds but the next component fails?
- How is the authorizing identity/device/key revoked or recovered?
- Can the operation be safely retried after the client loses the response?
- What happens if AI is unavailable, slow, or wrong?
- Can an auditor reconstruct debit, credit, actor, risk decision, approval, and Fabric txId from retained evidence?
- Which compensating transaction resolves a downstream operational error without rewriting history?

Chapter 192 - Dispute Summarizer

Summarize transaction/case evidence for staff without changing the dispute outcome. A local LLM can explain transactions, summarize disputes, answer policy questions through RAG, and assist operators. It should be treated as an untrusted interpreter: schema-constrained outputs, evidence references, prompt-injection defenses, least-privilege tools, and no independent financial signing authority.

Design rule: Input only authorized case evidence

Implementation Objective

The required output is: Summarization workflow. It must be version-controlled, reviewable, testable with the WAN disabled, and linked to the architecture/source-of-truth model. For a payment platform, every implementation decision should make it possible to explain who authorized an action, which facts were trusted, which controls ran, what financial journal committed, and how the result is recovered after failure.

Key Engineering Decisions

- Input only authorized case evidence
- distinguish facts from claims
- link evidence IDs
- human owns resolution

Deep Implementation Guidance

RAG and local LLMs are useful because they turn approved documents and verified transaction facts into readable assistance. They remain probabilistic. The application must perform access control before retrieval, constrain tools, validate outputs, and use no-answer behavior when evidence is insufficient.

Data and Control Questions

- What is the authoritative source for every input used by this component?
- Which user, device, service, MSP, or administrator identity is allowed to initiate or approve it?
- Does the operation move value, reserve value, create a side effect, or only provide advisory information?
- What idempotency/replay key proves that a retry is the same business intent?
- What evidence is retained off-chain and what stable reference/hash belongs on the ledger?

Chapter 192 Implementation - Dispute Summarizer

Implementation Procedure

- 956. Host approved model files locally and expose only LAN endpoints.
- 957. Use RAG for policy/operational knowledge and return evidence references.
- 958. Constrain outputs with JSON schema for machine-consumed tasks.
- 959. Treat retrieved/user content as untrusted and never reveal secrets/system prompts.
- 960. Evaluate groundedness, refusal/no-answer behavior, injection resistance, and latency.

Implementation Example

```
# Dispute Summarizer
INPUT  -> authenticate, validate schema, bind requestId/nonce
FACTS  -> read authoritative wallet/merchant/state data
AI     -> optional bounded recommendation with decisionId/version
POLICY -> enforce ownership, hard limits, approvals, risk fallback
LEDGER -> submit one idempotent deterministic transaction when value changes
EVENT  -> update projection/case/monitoring asynchronously
VERIFY -> discover committed result and retain correlationId + txId evidence
```

Verification Checklist

- The summarization workflow exists and is version-controlled.
- The feature works with all public Internet/WAN connectivity disabled.
- Authorization, malformed input, duplicate retry, and failure behavior have explicit tests.
- If financial value changes, journal debits and credits balance exactly and historical entries remain immutable.
- If AI is involved, model/prompt/feature version and fallback behavior are recorded and deterministic policy remains final.
- A timeout/restart cannot create a duplicate financial result; request status can be rediscovered.
- Logs/evidence contain correlation references but not secrets or prohibited sensitive payment data.

Common Failure Modes

- Prompt injection from user/RAG content.
- Hallucinated policy presented as fact.
- Unrestricted financial tools.
- No evidence/no-answer behavior.

Expert Review Questions

- What happens if this component succeeds but the next component fails?
- How is the authorizing identity/device/key revoked or recovered?
- Can the operation be safely retried after the client loses the response?
- What happens if AI is unavailable, slow, or wrong?
- Can an auditor reconstruct debit, credit, actor, risk decision, approval, and Fabric txId from retained evidence?
- Which compensating transaction resolves a downstream operational error without rewriting history?

Chapter 193 - Transaction Explanation Assistant

Translate journal/payment states into customer-friendly language. A local LLM can explain transactions, summarize disputes, answer policy questions through RAG, and assist operators. It should be treated as an untrusted interpreter: schema-constrained outputs, evidence references, prompt-injection defenses, least-privilege tools, and no independent financial signing authority.

Design rule: Read verified transaction facts

Implementation Objective

The required output is: Explanation workflow. It must be version-controlled, reviewable, testable with the WAN disabled, and linked to the architecture/source-of-truth model. For a payment platform, every implementation decision should make it possible to explain who authorized an action, which facts were trusted, which controls ran, what financial journal committed, and how the result is recovered after failure.

Key Engineering Decisions

- Read verified transaction facts
- avoid claiming fraud/guilt
- explain pending/hold/refund states accurately
- cite transaction reference

Deep Implementation Guidance

RAG and local LLMs are useful because they turn approved documents and verified transaction facts into readable assistance. They remain probabilistic. The application must perform access control before retrieval, constrain tools, validate outputs, and use no-answer behavior when evidence is insufficient.

Data and Control Questions

- What is the authoritative source for every input used by this component?
- Which user, device, service, MSP, or administrator identity is allowed to initiate or approve it?
- Does the operation move value, reserve value, create a side effect, or only provide advisory information?
- What idempotency/replay key proves that a retry is the same business intent?
- What evidence is retained off-chain and what stable reference/hash belongs on the ledger?

Chapter 193 Implementation - Transaction Explanation Assistant

Implementation Procedure

961. Host approved model files locally and expose only LAN endpoints.
962. Use RAG for policy/operational knowledge and return evidence references.
963. Constrain outputs with JSON schema for machine-consumed tasks.
964. Treat retrieved/user content as untrusted and never reveal secrets/system prompts.
965. Evaluate groundedness, refusal/no-answer behavior, injection resistance, and latency.

Implementation Example

```
# Transaction Explanation Assistant
INPUT  -> authenticate, validate schema, bind requestId/nonce
FACTS  -> read authoritative wallet/merchant/state data
AI     -> optional bounded recommendation with decisionId/version
POLICY -> enforce ownership, hard limits, approvals, risk fallback
LEDGER -> submit one idempotent deterministic transaction when value changes
EVENT  -> update projection/case/monitoring asynchronously
VERIFY -> discover committed result and retain correlationId + txId evidence
```

Verification Checklist

- The explanation workflow exists and is version-controlled.
- The feature works with all public Internet/WAN connectivity disabled.
- Authorization, malformed input, duplicate retry, and failure behavior have explicit tests.
- If financial value changes, journal debits and credits balance exactly and historical entries remain immutable.
- If AI is involved, model/prompt/feature version and fallback behavior are recorded and deterministic policy remains final.
- A timeout/restart cannot create a duplicate financial result; request status can be rediscovered.
- Logs/evidence contain correlation references but not secrets or prohibited sensitive payment data.

Common Failure Modes

- Prompt injection from user/RAG content.
- Hallucinated policy presented as fact.
- Unrestricted financial tools.
- No evidence/no-answer behavior.

Expert Review Questions

- What happens if this component succeeds but the next component fails?
- How is the authorizing identity/device/key revoked or recovered?
- Can the operation be safely retried after the client loses the response?
- What happens if AI is unavailable, slow, or wrong?
- Can an auditor reconstruct debit, credit, actor, risk decision, approval, and Fabric txId from retained evidence?
- Which compensating transaction resolves a downstream operational error without rewriting history?

Chapter 194 - Operations Copilot

Help operators interpret runbooks, alerts, and known troubleshooting procedures. A local LLM can explain transactions, summarize disputes, answer policy questions through RAG, and assist operators. It should be treated as an untrusted interpreter: schema-constrained outputs, evidence references, prompt-injection defenses, least-privilege tools, and no independent financial signing authority.

Design rule: Read-only tools initially

Implementation Objective

The required output is: Ops RAG assistant. It must be version-controlled, reviewable, testable with the WAN disabled, and linked to the architecture/source-of-truth model. For a payment platform, every implementation decision should make it possible to explain who authorized an action, which facts were trusted, which controls ran, what financial journal committed, and how the result is recovered after failure.

Key Engineering Decisions

- Read-only tools initially
- retrieve approved runbooks
- never execute shell/admin commands directly without separate controlled workflow

Deep Implementation Guidance

RAG and local LLMs are useful because they turn approved documents and verified transaction facts into readable assistance. They remain probabilistic. The application must perform access control before retrieval, constrain tools, validate outputs, and use no-answer behavior when evidence is insufficient.

Data and Control Questions

- What is the authoritative source for every input used by this component?
- Which user, device, service, MSP, or administrator identity is allowed to initiate or approve it?
- Does the operation move value, reserve value, create a side effect, or only provide advisory information?
- What idempotency/replay key proves that a retry is the same business intent?
- What evidence is retained off-chain and what stable reference/hash belongs on the ledger?

Chapter 194 Implementation - Operations Copilot

Implementation Procedure

- 966. Host approved model files locally and expose only LAN endpoints.
- 967. Use RAG for policy/operational knowledge and return evidence references.
- 968. Constrain outputs with JSON schema for machine-consumed tasks.
- 969. Treat retrieved/user content as untrusted and never reveal secrets/system prompts.
- 970. Evaluate groundedness, refusal/no-answer behavior, injection resistance, and latency.

Implementation Example

```
# Operations Copilot
INPUT  -> authenticate, validate schema, bind requestId/nonce
FACTS  -> read authoritative wallet/merchant/state data
AI     -> optional bounded recommendation with decisionId/version
POLICY -> enforce ownership, hard limits, approvals, risk fallback
LEDGER -> submit one idempotent deterministic transaction when value changes
EVENT  -> update projection/case/monitoring asynchronously
VERIFY -> discover committed result and retain correlationId + txId evidence
```

Verification Checklist

- The ops rag assistant exists and is version-controlled.
- The feature works with all public Internet/WAN connectivity disabled.
- Authorization, malformed input, duplicate retry, and failure behavior have explicit tests.
- If financial value changes, journal debits and credits balance exactly and historical entries remain immutable.
- If AI is involved, model/prompt/feature version and fallback behavior are recorded and deterministic policy remains final.
- A timeout/restart cannot create a duplicate financial result; request status can be rediscovered.
- Logs/evidence contain correlation references but not secrets or prohibited sensitive payment data.

Common Failure Modes

- Prompt injection from user/RAG content.
- Hallucinated policy presented as fact.
- Unrestricted financial tools.
- No evidence/no-answer behavior.

Expert Review Questions

- What happens if this component succeeds but the next component fails?
- How is the authorizing identity/device/key revoked or recovered?
- Can the operation be safely retried after the client loses the response?
- What happens if AI is unavailable, slow, or wrong?
- Can an auditor reconstruct debit, credit, actor, risk decision, approval, and Fabric txId from retained evidence?
- Which compensating transaction resolves a downstream operational error without rewriting history?

Chapter 195 - AI Support Classification

Classify support requests into payment pending, refund, access, fraud concern, merchant issue, or other. A local LLM can explain transactions, summarize disputes, answer policy questions through RAG, and assist operators. It should be treated as an untrusted interpreter: schema-constrained outputs, evidence references, prompt-injection defenses, least-privilege tools, and no independent financial signing authority.

Design rule: Structured category/priority/entities

Implementation Objective

The required output is: Support classifier. It must be version-controlled, reviewable, testable with the WAN disabled, and linked to the architecture/source-of-truth model. For a payment platform, every implementation decision should make it possible to explain who authorized an action, which facts were trusted, which controls ran, what financial journal committed, and how the result is recovered after failure.

Key Engineering Decisions

- Structured category/priority/entities
- do not infer transaction outcome without live lookup
- route sensitive cases to human

Deep Implementation Guidance

RAG and local LLMs are useful because they turn approved documents and verified transaction facts into readable assistance. They remain probabilistic. The application must perform access control before retrieval, constrain tools, validate outputs, and use no-answer behavior when evidence is insufficient.

Data and Control Questions

- What is the authoritative source for every input used by this component?
- Which user, device, service, MSP, or administrator identity is allowed to initiate or approve it?
- Does the operation move value, reserve value, create a side effect, or only provide advisory information?
- What idempotency/replay key proves that a retry is the same business intent?
- What evidence is retained off-chain and what stable reference/hash belongs on the ledger?

Chapter 195 Implementation - AI Support Classification

Implementation Procedure

971. Host approved model files locally and expose only LAN endpoints.
972. Use RAG for policy/operational knowledge and return evidence references.
973. Constrain outputs with JSON schema for machine-consumed tasks.
974. Treat retrieved/user content as untrusted and never reveal secrets/system prompts.
975. Evaluate groundedness, refusal/no-answer behavior, injection resistance, and latency.

Implementation Example

```
# AI Support Classification
INPUT  -> authenticate, validate schema, bind requestId/nonce
FACTS  -> read authoritative wallet/merchant/state data
AI     -> optional bounded recommendation with decisionId/version
POLICY -> enforce ownership, hard limits, approvals, risk fallback
LEDGER -> submit one idempotent deterministic transaction when value changes
EVENT  -> update projection/case/monitoring asynchronously
VERIFY -> discover committed result and retain correlationId + txId evidence
```

Verification Checklist

- The support classifier exists and is version-controlled.
- The feature works with all public Internet/WAN connectivity disabled.
- Authorization, malformed input, duplicate retry, and failure behavior have explicit tests.
- If financial value changes, journal debits and credits balance exactly and historical entries remain immutable.
- If AI is involved, model/prompt/feature version and fallback behavior are recorded and deterministic policy remains final.
- A timeout/restart cannot create a duplicate financial result; request status can be rediscovered.
- Logs/evidence contain correlation references but not secrets or prohibited sensitive payment data.

Common Failure Modes

- Prompt injection from user/RAG content.
- Hallucinated policy presented as fact.
- Unrestricted financial tools.
- No evidence/no-answer behavior.

Expert Review Questions

- What happens if this component succeeds but the next component fails?
- How is the authorizing identity/device/key revoked or recovered?
- Can the operation be safely retried after the client loses the response?
- What happens if AI is unavailable, slow, or wrong?
- Can an auditor reconstruct debit, credit, actor, risk decision, approval, and Fabric txId from retained evidence?
- Which compensating transaction resolves a downstream operational error without rewriting history?

Chapter 196 - LLM Tool Policy

Limit which local tools the assistant may call and what effects they can have. A local LLM can explain transactions, summarize disputes, answer policy questions through RAG, and assist operators. It should be treated as an untrusted interpreter: schema-constrained outputs, evidence references, prompt-injection defenses, least-privilege tools, and no independent financial signing authority.

Design rule: Prefer read-only transaction lookup and policy search

Implementation Objective

The required output is: Tool permission matrix. It must be version-controlled, reviewable, testable with the WAN disabled, and linked to the architecture/source-of-truth model. For a payment platform, every implementation decision should make it possible to explain who authorized an action, which facts were trusted, which controls ran, what financial journal committed, and how the result is recovered after failure.

Key Engineering Decisions

- Prefer read-only transaction lookup and policy search
- financial write tools unavailable to general assistant
- human approval for any future sensitive tool

Deep Implementation Guidance

RAG and local LLMs are useful because they turn approved documents and verified transaction facts into readable assistance. They remain probabilistic. The application must perform access control before retrieval, constrain tools, validate outputs, and use no-answer behavior when evidence is insufficient.

Data and Control Questions

- What is the authoritative source for every input used by this component?
- Which user, device, service, MSP, or administrator identity is allowed to initiate or approve it?
- Does the operation move value, reserve value, create a side effect, or only provide advisory information?
- What idempotency/replay key proves that a retry is the same business intent?
- What evidence is retained off-chain and what stable reference/hash belongs on the ledger?

Chapter 196 Implementation - LLM Tool Policy

Implementation Procedure

- 976. Host approved model files locally and expose only LAN endpoints.
- 977. Use RAG for policy/operational knowledge and return evidence references.
- 978. Constrain outputs with JSON schema for machine-consumed tasks.
- 979. Treat retrieved/user content as untrusted and never reveal secrets/system prompts.
- 980. Evaluate groundedness, refusal/no-answer behavior, injection resistance, and latency.

Implementation Example

```
# LLM Tool Policy
INPUT  -> authenticate, validate schema, bind requestId/nonce
FACTS  -> read authoritative wallet/merchant/state data
AI     -> optional bounded recommendation with decisionId/version
POLICY -> enforce ownership, hard limits, approvals, risk fallback
LEDGER -> submit one idempotent deterministic transaction when value changes
EVENT  -> update projection/case/monitoring asynchronously
VERIFY -> discover committed result and retain correlationId + txId evidence
```

Verification Checklist

- The tool permission matrix exists and is version-controlled.
- The feature works with all public Internet/WAN connectivity disabled.
- Authorization, malformed input, duplicate retry, and failure behavior have explicit tests.
- If financial value changes, journal debits and credits balance exactly and historical entries remain immutable.
- If AI is involved, model/prompt/feature version and fallback behavior are recorded and deterministic policy remains final.
- A timeout/restart cannot create a duplicate financial result; request status can be rediscovered.
- Logs/evidence contain correlation references but not secrets or prohibited sensitive payment data.

Common Failure Modes

- Prompt injection from user/RAG content.
- Hallucinated policy presented as fact.
- Unrestricted financial tools.
- No evidence/no-answer behavior.

Expert Review Questions

- What happens if this component succeeds but the next component fails?
- How is the authorizing identity/device/key revoked or recovered?
- Can the operation be safely retried after the client loses the response?
- What happens if AI is unavailable, slow, or wrong?
- Can an auditor reconstruct debit, credit, actor, risk decision, approval, and Fabric txId from retained evidence?
- Which compensating transaction resolves a downstream operational error without rewriting history?

Chapter 197 - Prompt Injection Defense

Treat users, merchant text, receipts, and retrieved documents as untrusted instructions. A local LLM can explain transactions, summarize disputes, answer policy questions through RAG, and assist operators. It should be treated as an untrusted interpreter: schema-constrained outputs, evidence references, prompt-injection defenses, least-privilege tools, and no independent financial signing authority.

Design rule: System/tool policies cannot be overridden by data

Implementation Objective

The required output is: Injection controls. It must be version-controlled, reviewable, testable with the WAN disabled, and linked to the architecture/source-of-truth model. For a payment platform, every implementation decision should make it possible to explain who authorized an action, which facts were trusted, which controls ran, what financial journal committed, and how the result is recovered after failure.

Key Engineering Decisions

- System/tool policies cannot be overridden by data
- strip/label untrusted content
- allowlist tools
- do not expose secrets/prompts

Deep Implementation Guidance

RAG and local LLMs are useful because they turn approved documents and verified transaction facts into readable assistance. They remain probabilistic. The application must perform access control before retrieval, constrain tools, validate outputs, and use no-answer behavior when evidence is insufficient.

Data and Control Questions

- What is the authoritative source for every input used by this component?
- Which user, device, service, MSP, or administrator identity is allowed to initiate or approve it?
- Does the operation move value, reserve value, create a side effect, or only provide advisory information?
- What idempotency/replay key proves that a retry is the same business intent?
- What evidence is retained off-chain and what stable reference/hash belongs on the ledger?

Chapter 197 Implementation - Prompt Injection Defense

Implementation Procedure

981. Host approved model files locally and expose only LAN endpoints.
982. Use RAG for policy/operational knowledge and return evidence references.
983. Constrain outputs with JSON schema for machine-consumed tasks.
984. Treat retrieved/user content as untrusted and never reveal secrets/system prompts.
985. Evaluate groundedness, refusal/no-answer behavior, injection resistance, and latency.

Implementation Example

```
# Prompt Injection Defense
INPUT  -> authenticate, validate schema, bind requestId/nonce
FACTS  -> read authoritative wallet/merchant/state data
AI     -> optional bounded recommendation with decisionId/version
POLICY -> enforce ownership, hard limits, approvals, risk fallback
LEDGER -> submit one idempotent deterministic transaction when value changes
EVENT  -> update projection/case/monitoring asynchronously
VERIFY -> discover committed result and retain correlationId + txId evidence
```

Verification Checklist

- The injection controls exists and is version-controlled.
- The feature works with all public Internet/WAN connectivity disabled.
- Authorization, malformed input, duplicate retry, and failure behavior have explicit tests.
- If financial value changes, journal debits and credits balance exactly and historical entries remain immutable.
- If AI is involved, model/prompt/feature version and fallback behavior are recorded and deterministic policy remains final.
- A timeout/restart cannot create a duplicate financial result; request status can be rediscovered.
- Logs/evidence contain correlation references but not secrets or prohibited sensitive payment data.

Common Failure Modes

- Prompt injection from user/RAG content.
- Hallucinated policy presented as fact.
- Unrestricted financial tools.
- No evidence/no-answer behavior.

Expert Review Questions

- What happens if this component succeeds but the next component fails?
- How is the authorizing identity/device/key revoked or recovered?
- Can the operation be safely retried after the client loses the response?
- What happens if AI is unavailable, slow, or wrong?
- Can an auditor reconstruct debit, credit, actor, risk decision, approval, and Fabric txId from retained evidence?
- Which compensating transaction resolves a downstream operational error without rewriting history?

Chapter 198 - RAG Access Control

Prevent retrieval of policy/case content a user is not authorized to see. A local LLM can explain transactions, summarize disputes, answer policy questions through RAG, and assist operators. It should be treated as an untrusted interpreter: schema-constrained outputs, evidence references, prompt-injection defenses, least-privilege tools, and no independent financial signing authority.

Design rule: Filter by role/tenant/case membership before similarity ranking

Implementation Objective

The required output is: Retrieval authorization. It must be version-controlled, reviewable, testable with the WAN disabled, and linked to the architecture/source-of-truth model. For a payment platform, every implementation decision should make it possible to explain who authorized an action, which facts were trusted, which controls ran, what financial journal committed, and how the result is recovered after failure.

Key Engineering Decisions

- Filter by role/tenant/case membership before similarity ranking
- do not rely on LLM refusal after retrieval

Deep Implementation Guidance

RAG and local LLMs are useful because they turn approved documents and verified transaction facts into readable assistance. They remain probabilistic. The application must perform access control before retrieval, constrain tools, validate outputs, and use no-answer behavior when evidence is insufficient.

Data and Control Questions

- What is the authoritative source for every input used by this component?
- Which user, device, service, MSP, or administrator identity is allowed to initiate or approve it?
- Does the operation move value, reserve value, create a side effect, or only provide advisory information?
- What idempotency/replay key proves that a retry is the same business intent?
- What evidence is retained off-chain and what stable reference/hash belongs on the ledger?

Chapter 198 Implementation - RAG Access Control

Implementation Procedure

- 986. Host approved model files locally and expose only LAN endpoints.
- 987. Use RAG for policy/operational knowledge and return evidence references.
- 988. Constrain outputs with JSON schema for machine-consumed tasks.
- 989. Treat retrieved/user content as untrusted and never reveal secrets/system prompts.
- 990. Evaluate groundedness, refusal/no-answer behavior, injection resistance, and latency.

Implementation Example

```
# RAG Access Control
INPUT  -> authenticate, validate schema, bind requestId/nonce
FACTS  -> read authoritative wallet/merchant/state data
AI     -> optional bounded recommendation with decisionId/version
POLICY -> enforce ownership, hard limits, approvals, risk fallback
LEDGER -> submit one idempotent deterministic transaction when value changes
EVENT  -> update projection/case/monitoring asynchronously
VERIFY -> discover committed result and retain correlationId + txId evidence
```

Verification Checklist

- The retrieval authorization exists and is version-controlled.
- The feature works with all public Internet/WAN connectivity disabled.
- Authorization, malformed input, duplicate retry, and failure behavior have explicit tests.
- If financial value changes, journal debits and credits balance exactly and historical entries remain immutable.
- If AI is involved, model/prompt/feature version and fallback behavior are recorded and deterministic policy remains final.
- A timeout/restart cannot create a duplicate financial result; request status can be rediscovered.
- Logs/evidence contain correlation references but not secrets or prohibited sensitive payment data.

Common Failure Modes

- Prompt injection from user/RAG content.
- Hallucinated policy presented as fact.
- Unrestricted financial tools.
- No evidence/no-answer behavior.

Expert Review Questions

- What happens if this component succeeds but the next component fails?
- How is the authorizing identity/device/key revoked or recovered?
- Can the operation be safely retried after the client loses the response?
- What happens if AI is unavailable, slow, or wrong?
- Can an auditor reconstruct debit, credit, actor, risk decision, approval, and Fabric txId from retained evidence?
- Which compensating transaction resolves a downstream operational error without rewriting history?

Chapter 199 - RAG Evaluation

Measure retrieval and answer quality with repeatable local datasets. A local LLM can explain transactions, summarize disputes, answer policy questions through RAG, and assist operators. It should be treated as an untrusted interpreter: schema-constrained outputs, evidence references, prompt-injection defenses, least-privilege tools, and no independent financial signing authority.

Design rule: Hit/Recall@K/MRR as useful

Implementation Objective

The required output is: RAG eval suite. It must be version-controlled, reviewable, testable with the WAN disabled, and linked to the architecture/source-of-truth model. For a payment platform, every implementation decision should make it possible to explain who authorized an action, which facts were trusted, which controls ran, what financial journal committed, and how the result is recovered after failure.

Key Engineering Decisions

- Hit/Recall@K/MRR as useful
- groundedness/correctness/completeness/citation quality
- negative/no-answer cases

Deep Implementation Guidance

RAG and local LLMs are useful because they turn approved documents and verified transaction facts into readable assistance. They remain probabilistic. The application must perform access control before retrieval, constrain tools, validate outputs, and use no-answer behavior when evidence is insufficient.

Data and Control Questions

- What is the authoritative source for every input used by this component?
- Which user, device, service, MSP, or administrator identity is allowed to initiate or approve it?
- Does the operation move value, reserve value, create a side effect, or only provide advisory information?
- What idempotency/replay key proves that a retry is the same business intent?
- What evidence is retained off-chain and what stable reference/hash belongs on the ledger?

Chapter 199 Implementation - RAG Evaluation

Implementation Procedure

991. Host approved model files locally and expose only LAN endpoints.
992. Use RAG for policy/operational knowledge and return evidence references.
993. Constrain outputs with JSON schema for machine-consumed tasks.
994. Treat retrieved/user content as untrusted and never reveal secrets/system prompts.
995. Evaluate groundedness, refusal/no-answer behavior, injection resistance, and latency.

Implementation Example

```
# RAG Evaluation
INPUT  -> authenticate, validate schema, bind requestId/nonce
FACTS  -> read authoritative wallet/merchant/state data
AI     -> optional bounded recommendation with decisionId/version
POLICY -> enforce ownership, hard limits, approvals, risk fallback
LEDGER -> submit one idempotent deterministic transaction when value changes
EVENT  -> update projection/case/monitoring asynchronously
VERIFY -> discover committed result and retain correlationId + txId evidence
```

Verification Checklist

- The rag eval suite exists and is version-controlled.
- The feature works with all public Internet/WAN connectivity disabled.
- Authorization, malformed input, duplicate retry, and failure behavior have explicit tests.
- If financial value changes, journal debits and credits balance exactly and historical entries remain immutable.
- If AI is involved, model/prompt/feature version and fallback behavior are recorded and deterministic policy remains final.
- A timeout/restart cannot create a duplicate financial result; request status can be rediscovered.
- Logs/evidence contain correlation references but not secrets or prohibited sensitive payment data.

Common Failure Modes

- Prompt injection from user/RAG content.
- Hallucinated policy presented as fact.
- Unrestricted financial tools.
- No evidence/no-answer behavior.

Expert Review Questions

- What happens if this component succeeds but the next component fails?
- How is the authorizing identity/device/key revoked or recovered?
- Can the operation be safely retried after the client loses the response?
- What happens if AI is unavailable, slow, or wrong?
- Can an auditor reconstruct debit, credit, actor, risk decision, approval, and Fabric txId from retained evidence?
- Which compensating transaction resolves a downstream operational error without rewriting history?

Chapter 200 - LLM Safety Evaluation

Test support/ops assistant against harmful or misleading behavior. A local LLM can explain transactions, summarize disputes, answer policy questions through RAG, and assist operators. It should be treated as an untrusted interpreter: schema-constrained outputs, evidence references, prompt-injection defenses, least-privilege tools, and no independent financial signing authority.

Design rule: Prompt injection

Implementation Objective

The required output is: LLM eval suite. It must be version-controlled, reviewable, testable with the WAN disabled, and linked to the architecture/source-of-truth model. For a payment platform, every implementation decision should make it possible to explain who authorized an action, which facts were trusted, which controls ran, what financial journal committed, and how the result is recovered after failure.

Key Engineering Decisions

- Prompt injection
- data exfiltration
- unsupported financial advice
- tool misuse
- hallucinated payment status
- secret-reveal attempts

Deep Implementation Guidance

RAG and local LLMs are useful because they turn approved documents and verified transaction facts into readable assistance. They remain probabilistic. The application must perform access control before retrieval, constrain tools, validate outputs, and use no-answer behavior when evidence is insufficient.

Data and Control Questions

- What is the authoritative source for every input used by this component?
- Which user, device, service, MSP, or administrator identity is allowed to initiate or approve it?
- Does the operation move value, reserve value, create a side effect, or only provide advisory information?
- What idempotency/replay key proves that a retry is the same business intent?
- What evidence is retained off-chain and what stable reference/hash belongs on the ledger?

Chapter 200 Implementation - LLM Safety Evaluation

Implementation Procedure

996. Host approved model files locally and expose only LAN endpoints.
997. Use RAG for policy/operational knowledge and return evidence references.
998. Constrain outputs with JSON schema for machine-consumed tasks.
999. Treat retrieved/user content as untrusted and never reveal secrets/system prompts.
1000. Evaluate groundedness, refusal/no-answer behavior, injection resistance, and latency.

Implementation Example

```
# LLM Safety Evaluation
INPUT  -> authenticate, validate schema, bind requestId/nonce
FACTS  -> read authoritative wallet/merchant/state data
AI     -> optional bounded recommendation with decisionId/version
POLICY -> enforce ownership, hard limits, approvals, risk fallback
LEDGER -> submit one idempotent deterministic transaction when value changes
EVENT  -> update projection/case/monitoring asynchronously
VERIFY -> discover committed result and retain correlationId + txId evidence
```

Verification Checklist

- The llm eval suite exists and is version-controlled.
- The feature works with all public Internet/WAN connectivity disabled.
- Authorization, malformed input, duplicate retry, and failure behavior have explicit tests.
- If financial value changes, journal debits and credits balance exactly and historical entries remain immutable.
- If AI is involved, model/prompt/feature version and fallback behavior are recorded and deterministic policy remains final.
- A timeout/restart cannot create a duplicate financial result; request status can be rediscovered.
- Logs/evidence contain correlation references but not secrets or prohibited sensitive payment data.

Common Failure Modes

- Prompt injection from user/RAG content.
- Hallucinated policy presented as fact.
- Unrestricted financial tools.
- No evidence/no-answer behavior.

Expert Review Questions

- What happens if this component succeeds but the next component fails?
- How is the authorizing identity/device/key revoked or recovered?
- Can the operation be safely retried after the client loses the response?
- What happens if AI is unavailable, slow, or wrong?
- Can an auditor reconstruct debit, credit, actor, risk decision, approval, and Fabric txId from retained evidence?
- Which compensating transaction resolves a downstream operational error without rewriting history?

Chapter 201 - Prompt and Model Rollback

Restore a known-good local assistant quickly after regression. A local LLM can explain transactions, summarize disputes, answer policy questions through RAG, and assist operators. It should be treated as an untrusted interpreter: schema-constrained outputs, evidence references, prompt-injection defenses, least-privilege tools, and no independent financial signing authority.

Design rule: Keep previous prompt/schema/model bundle

Implementation Objective

The required output is: Rollback procedure. It must be version-controlled, reviewable, testable with the WAN disabled, and linked to the architecture/source-of-truth model. For a payment platform, every implementation decision should make it possible to explain who authorized an action, which facts were trusted, which controls ran, what financial journal committed, and how the result is recovered after failure.

Key Engineering Decisions

- Keep previous prompt/schema/model bundle
- switch service alias/config
- run smoke/eval suite
- record rollback reason

Deep Implementation Guidance

RAG and local LLMs are useful because they turn approved documents and verified transaction facts into readable assistance. They remain probabilistic. The application must perform access control before retrieval, constrain tools, validate outputs, and use no-answer behavior when evidence is insufficient.

Data and Control Questions

- What is the authoritative source for every input used by this component?
- Which user, device, service, MSP, or administrator identity is allowed to initiate or approve it?
- Does the operation move value, reserve value, create a side effect, or only provide advisory information?
- What idempotency/replay key proves that a retry is the same business intent?
- What evidence is retained off-chain and what stable reference/hash belongs on the ledger?

Chapter 201 Implementation - Prompt and Model Rollback

Implementation Procedure

1001. Host approved model files locally and expose only LAN endpoints.
1002. Use RAG for policy/operational knowledge and return evidence references.
1003. Constrain outputs with JSON schema for machine-consumed tasks.
1004. Treat retrieved/user content as untrusted and never reveal secrets/system prompts.
1005. Evaluate groundedness, refusal/no-answer behavior, injection resistance, and latency.

Implementation Example

```
# Prompt and Model Rollback
INPUT  -> authenticate, validate schema, bind requestId/nonce
FACTS  -> read authoritative wallet/merchant/state data
AI     -> optional bounded recommendation with decisionId/version
POLICY -> enforce ownership, hard limits, approvals, risk fallback
LEDGER -> submit one idempotent deterministic transaction when value changes
EVENT  -> update projection/case/monitoring asynchronously
VERIFY -> discover committed result and retain correlationId + txId evidence
```

Verification Checklist

- The rollback procedure exists and is version-controlled.
- The feature works with all public Internet/WAN connectivity disabled.
- Authorization, malformed input, duplicate retry, and failure behavior have explicit tests.
- If financial value changes, journal debits and credits balance exactly and historical entries remain immutable.
- If AI is involved, model/prompt/feature version and fallback behavior are recorded and deterministic policy remains final.
- A timeout/restart cannot create a duplicate financial result; request status can be rediscovered.
- Logs/evidence contain correlation references but not secrets or prohibited sensitive payment data.

Common Failure Modes

- Prompt injection from user/RAG content.
- Hallucinated policy presented as fact.
- Unrestricted financial tools.
- No evidence/no-answer behavior.

Expert Review Questions

- What happens if this component succeeds but the next component fails?
- How is the authorizing identity/device/key revoked or recovered?
- Can the operation be safely retried after the client loses the response?
- What happens if AI is unavailable, slow, or wrong?
- Can an auditor reconstruct debit, credit, actor, risk decision, approval, and Fabric txId from retained evidence?
- Which compensating transaction resolves a downstream operational error without rewriting history?

Chapter 202 - LLM Data Retention and Privacy

Avoid creating a second uncontrolled archive of financial conversations. A local LLM can explain transactions, summarize disputes, answer policy questions through RAG, and assist operators. It should be treated as an untrusted interpreter: schema-constrained outputs, evidence references, prompt-injection defenses, least-privilege tools, and no independent financial signing authority.

Design rule: Store minimum prompt/response evidence

Implementation Objective

The required output is: LLM retention policy. It must be version-controlled, reviewable, testable with the WAN disabled, and linked to the architecture/source-of-truth model. For a payment platform, every implementation decision should make it possible to explain who authorized an action, which facts were trusted, which controls ran, what financial journal committed, and how the result is recovered after failure.

Key Engineering Decisions

- Store minimum prompt/response evidence
- redact secrets
- separate model training consent
- apply off-chain deletion/retention rules

Deep Implementation Guidance

RAG and local LLMs are useful because they turn approved documents and verified transaction facts into readable assistance. They remain probabilistic. The application must perform access control before retrieval, constrain tools, validate outputs, and use no-answer behavior when evidence is insufficient.

Data and Control Questions

- What is the authoritative source for every input used by this component?
- Which user, device, service, MSP, or administrator identity is allowed to initiate or approve it?
- Does the operation move value, reserve value, create a side effect, or only provide advisory information?
- What idempotency/replay key proves that a retry is the same business intent?
- What evidence is retained off-chain and what stable reference/hash belongs on the ledger?

Chapter 202 Implementation - LLM Data Retention and Privacy

Implementation Procedure

1006. Host approved model files locally and expose only LAN endpoints.
1007. Use RAG for policy/operational knowledge and return evidence references.
1008. Constrain outputs with JSON schema for machine-consumed tasks.
1009. Treat retrieved/user content as untrusted and never reveal secrets/system prompts.
1010. Evaluate groundedness, refusal/no-answer behavior, injection resistance, and latency.

Implementation Example

```
# LLM Data Retention and Privacy
INPUT  -> authenticate, validate schema, bind requestId/nonce
FACTS  -> read authoritative wallet/merchant/state data
AI     -> optional bounded recommendation with decisionId/version
POLICY -> enforce ownership, hard limits, approvals, risk fallback
LEDGER -> submit one idempotent deterministic transaction when value changes
EVENT  -> update projection/case/monitoring asynchronously
VERIFY -> discover committed result and retain correlationId + txId evidence
```

Verification Checklist

- The llm retention policy exists and is version-controlled.
- The feature works with all public Internet/WAN connectivity disabled.
- Authorization, malformed input, duplicate retry, and failure behavior have explicit tests.
- If financial value changes, journal debits and credits balance exactly and historical entries remain immutable.
- If AI is involved, model/prompt/feature version and fallback behavior are recorded and deterministic policy remains final.
- A timeout/restart cannot create a duplicate financial result; request status can be rediscovered.
- Logs/evidence contain correlation references but not secrets or prohibited sensitive payment data.

Common Failure Modes

- Prompt injection from user/RAG content.
- Hallucinated policy presented as fact.
- Unrestricted financial tools.
- No evidence/no-answer behavior.

Expert Review Questions

- What happens if this component succeeds but the next component fails?
- How is the authorizing identity/device/key revoked or recovered?
- Can the operation be safely retried after the client loses the response?
- What happens if AI is unavailable, slow, or wrong?
- Can an auditor reconstruct debit, credit, actor, risk decision, approval, and Fabric txId from retained evidence?
- Which compensating transaction resolves a downstream operational error without rewriting history?

Chapter 203 - Air-Gapped Model Import

Bring approved models into the LAN safely. A local LLM can explain transactions, summarize disputes, answer policy questions through RAG, and assist operators. It should be treated as an untrusted interpreter: schema-constrained outputs, evidence references, prompt-injection defenses, least-privilege tools, and no independent financial signing authority.

Design rule: Acquire on staging workstation

Implementation Objective

The required output is: Model import SOP. It must be version-controlled, reviewable, testable with the WAN disabled, and linked to the architecture/source-of-truth model. For a payment platform, every implementation decision should make it possible to explain who authorized an action, which facts were trusted, which controls ran, what financial journal committed, and how the result is recovered after failure.

Key Engineering Decisions

- Acquire on staging workstation
- verify publisher/hash/license
- malware scan
- internal approval
- copy to model registry
- block unapproved files

Deep Implementation Guidance

RAG and local LLMs are useful because they turn approved documents and verified transaction facts into readable assistance. They remain probabilistic. The application must perform access control before retrieval, constrain tools, validate outputs, and use no-answer behavior when evidence is insufficient.

Data and Control Questions

- What is the authoritative source for every input used by this component?
- Which user, device, service, MSP, or administrator identity is allowed to initiate or approve it?
- Does the operation move value, reserve value, create a side effect, or only provide advisory information?
- What idempotency/replay key proves that a retry is the same business intent?
- What evidence is retained off-chain and what stable reference/hash belongs on the ledger?

Chapter 203 Implementation - Air-Gapped Model Import

Implementation Procedure

1011. Host approved model files locally and expose only LAN endpoints.
1012. Use RAG for policy/operational knowledge and return evidence references.
1013. Constrain outputs with JSON schema for machine-consumed tasks.
1014. Treat retrieved/user content as untrusted and never reveal secrets/system prompts.
1015. Evaluate groundedness, refusal/no-answer behavior, injection resistance, and latency.

Implementation Example

```
# Air-Gapped Model Import
INPUT  -> authenticate, validate schema, bind requestId/nonce
FACTS  -> read authoritative wallet/merchant/state data
AI     -> optional bounded recommendation with decisionId/version
POLICY -> enforce ownership, hard limits, approvals, risk fallback
LEDGER -> submit one idempotent deterministic transaction when value changes
EVENT  -> update projection/case/monitoring asynchronously
VERIFY -> discover committed result and retain correlationId + txId evidence
```

Verification Checklist

- The model import sop exists and is version-controlled.
- The feature works with all public Internet/WAN connectivity disabled.
- Authorization, malformed input, duplicate retry, and failure behavior have explicit tests.
- If financial value changes, journal debits and credits balance exactly and historical entries remain immutable.
- If AI is involved, model/prompt/feature version and fallback behavior are recorded and deterministic policy remains final.
- A timeout/restart cannot create a duplicate financial result; request status can be rediscovered.
- Logs/evidence contain correlation references but not secrets or prohibited sensitive payment data.

Common Failure Modes

- Prompt injection from user/RAG content.
- Hallucinated policy presented as fact.
- Unrestricted financial tools.
- No evidence/no-answer behavior.

Expert Review Questions

- What happens if this component succeeds but the next component fails?
- How is the authorizing identity/device/key revoked or recovered?
- Can the operation be safely retried after the client loses the response?
- What happens if AI is unavailable, slow, or wrong?
- Can an auditor reconstruct debit, credit, actor, risk decision, approval, and Fabric txId from retained evidence?
- Which compensating transaction resolves a downstream operational error without rewriting history?

Part XII - Security, Privacy, and Financial Abuse Resistance

Offline does not mean secure. Insider abuse, stolen credentials, removable-media malware, endpoint compromise, replay, model tampering, configuration drift, and physical access remain serious threats. Use layered controls and preserve evidence across identity, application, ledger, model, and infrastructure layers.

Part focus: This part contains 24 implementation chapters. Each chapter includes design rules, implementation procedure, a practical artifact, verification, failure analysis, and expert review questions.

Chapter 204 - Security Threat Model

Model insider, endpoint, key, network, database, Fabric, AI, and physical threats. Offline does not mean secure. Insider abuse, stolen credentials, removable-media malware, endpoint compromise, replay, model tampering, configuration drift, and physical access remain serious threats. Use layered controls and preserve evidence across identity, application, ledger, model, and infrastructure layers.

Design rule: Assets/actors/entry points/impact/controls

Implementation Objective

The required output is: Threat register. It must be version-controlled, reviewable, testable with the WAN disabled, and linked to the architecture/source-of-truth model. For a payment platform, every implementation decision should make it possible to explain who authorized an action, which facts were trusted, which controls ran, what financial journal committed, and how the result is recovered after failure.

Key Engineering Decisions

- Assets/actors/entry points/impact/controls
- include compromised privileged operator and malicious merchant
- review after architecture change

Deep Implementation Guidance

Apply defense in depth across physical access, network segmentation, host hardening, identity/key lifecycle, application authorization, ledger policies, data privacy, AI model integrity, and incident response. Offline systems are still exposed to insiders and removable-media supply-chain risk.

Data and Control Questions

- What is the authoritative source for every input used by this component?
- Which user, device, service, MSP, or administrator identity is allowed to initiate or approve it?
- Does the operation move value, reserve value, create a side effect, or only provide advisory information?
- What idempotency/replay key proves that a retry is the same business intent?
- What evidence is retained off-chain and what stable reference/hash belongs on the ledger?

Chapter 204 Implementation - Security Threat Model

Implementation Procedure

- 1016. Maintain an explicit threat model and asset/key inventory.
- 1017. Segment networks, minimize privileges, harden endpoints, and protect secrets.
- 1018. Encrypt sensitive off-chain data and minimize immutable PII.
- 1019. Test replay, auth bypass, prompt injection, model tampering, and malicious update media.
- 1020. Maintain kill switches, certificate revocation, evidence preservation, and recovery playbooks.

Implementation Example

```
# Security Threat Model
INPUT  -> authenticate, validate schema, bind requestId/nonce
FACTS  -> read authoritative wallet/merchant/state data
AI     -> optional bounded recommendation with decisionId/version
POLICY -> enforce ownership, hard limits, approvals, risk fallback
LEDGER -> submit one idempotent deterministic transaction when value changes
EVENT  -> update projection/case/monitoring asynchronously
VERIFY -> discover committed result and retain correlationId + txId evidence
```

Verification Checklist

- The threat register exists and is version-controlled.
- The feature works with all public Internet/WAN connectivity disabled.
- Authorization, malformed input, duplicate retry, and failure behavior have explicit tests.
- If financial value changes, journal debits and credits balance exactly and historical entries remain immutable.
- If AI is involved, model/prompt/feature version and fallback behavior are recorded and deterministic policy remains final.
- A timeout/restart cannot create a duplicate financial result; request status can be rediscovered.
- Logs/evidence contain correlation references but not secrets or prohibited sensitive payment data.

Common Failure Modes

- PII/card secrets placed on immutable ledger.
- Excessive network/admin privileges.
- Unsigned offline update bundle.
- No incident write-disable/revocation path.

Expert Review Questions

- What happens if this component succeeds but the next component fails?
- How is the authorizing identity/device/key revoked or recovered?
- Can the operation be safely retried after the client loses the response?
- What happens if AI is unavailable, slow, or wrong?
- Can an auditor reconstruct debit, credit, actor, risk decision, approval, and Fabric txId from retained evidence?
- Which compensating transaction resolves a downstream operational error without rewriting history?

Chapter 205 - Network Least Privilege

Minimize reachable services and management surfaces. Offline does not mean secure. Insider abuse, stolen credentials, removable-media malware, endpoint compromise, replay, model tampering, configuration drift, and physical access remain serious threats. Use layered controls and preserve evidence across identity, application, ledger, model, and infrastructure layers.

Design rule: Client to API only

Implementation Objective

The required output is: Firewall policy. It must be version-controlled, reviewable, testable with the WAN disabled, and linked to the architecture/source-of-truth model. For a payment platform, every implementation decision should make it possible to explain who authorized an action, which facts were trusted, which controls ran, what financial journal committed, and how the result is recovered after failure.

Key Engineering Decisions

- Client to API only
- API to required peers/DB/AI
- peer/orderer/CA management separated
- deny lateral movement by default

Deep Implementation Guidance

Apply defense in depth across physical access, network segmentation, host hardening, identity/key lifecycle, application authorization, ledger policies, data privacy, AI model integrity, and incident response. Offline systems are still exposed to insiders and removable-media supply-chain risk.

Data and Control Questions

- What is the authoritative source for every input used by this component?
- Which user, device, service, MSP, or administrator identity is allowed to initiate or approve it?
- Does the operation move value, reserve value, create a side effect, or only provide advisory information?
- What idempotency/replay key proves that a retry is the same business intent?
- What evidence is retained off-chain and what stable reference/hash belongs on the ledger?

Chapter 205 Implementation - Network Least Privilege

Implementation Procedure

1021. Maintain an explicit threat model and asset/key inventory.
1022. Segment networks, minimize privileges, harden endpoints, and protect secrets.
1023. Encrypt sensitive off-chain data and minimize immutable PII.
1024. Test replay, auth bypass, prompt injection, model tampering, and malicious update media.
1025. Maintain kill switches, certificate revocation, evidence preservation, and recovery playbooks.

Implementation Example

```
# Network Least Privilege
INPUT  -> authenticate, validate schema, bind requestId/nonce
FACTS  -> read authoritative wallet/merchant/state data
AI     -> optional bounded recommendation with decisionId/version
POLICY -> enforce ownership, hard limits, approvals, risk fallback
LEDGER -> submit one idempotent deterministic transaction when value changes
EVENT  -> update projection/case/monitoring asynchronously
VERIFY -> discover committed result and retain correlationId + txId evidence
```

Verification Checklist

- The firewall policy exists and is version-controlled.
- The feature works with all public Internet/WAN connectivity disabled.
- Authorization, malformed input, duplicate retry, and failure behavior have explicit tests.
- If financial value changes, journal debits and credits balance exactly and historical entries remain immutable.
- If AI is involved, model/prompt/feature version and fallback behavior are recorded and deterministic policy remains final.
- A timeout/restart cannot create a duplicate financial result; request status can be rediscovered.
- Logs/evidence contain correlation references but not secrets or prohibited sensitive payment data.

Common Failure Modes

- PII/card secrets placed on immutable ledger.
- Excessive network/admin privileges.
- Unsigned offline update bundle.
- No incident write-disable/revocation path.

Expert Review Questions

- What happens if this component succeeds but the next component fails?
- How is the authorizing identity/device/key revoked or recovered?
- Can the operation be safely retried after the client loses the response?
- What happens if AI is unavailable, slow, or wrong?
- Can an auditor reconstruct debit, credit, actor, risk decision, approval, and Fabric txId from retained evidence?
- Which compensating transaction resolves a downstream operational error without rewriting history?

Chapter 206 - Host Hardening

Reduce attack surface on Linux/Windows servers and terminals. Offline does not mean secure. Insider abuse, stolen credentials, removable-media malware, endpoint compromise, replay, model tampering, configuration drift, and physical access remain serious threats. Use layered controls and preserve evidence across identity, application, ledger, model, and infrastructure layers.

Design rule: Patch from approved bundle

Implementation Objective

The required output is: Hardening baseline. It must be version-controlled, reviewable, testable with the WAN disabled, and linked to the architecture/source-of-truth model. For a payment platform, every implementation decision should make it possible to explain who authorized an action, which facts were trusted, which controls ran, what financial journal committed, and how the result is recovered after failure.

Key Engineering Decisions

- Patch from approved bundle
- disable unused services
- restrict local admins
- disk encryption where appropriate
- secure boot/logging options

Deep Implementation Guidance

Apply defense in depth across physical access, network segmentation, host hardening, identity/key lifecycle, application authorization, ledger policies, data privacy, AI model integrity, and incident response. Offline systems are still exposed to insiders and removable-media supply-chain risk.

Data and Control Questions

- What is the authoritative source for every input used by this component?
- Which user, device, service, MSP, or administrator identity is allowed to initiate or approve it?
- Does the operation move value, reserve value, create a side effect, or only provide advisory information?
- What idempotency/replay key proves that a retry is the same business intent?
- What evidence is retained off-chain and what stable reference/hash belongs on the ledger?

Chapter 206 Implementation - Host Hardening

Implementation Procedure

- 1026. Maintain an explicit threat model and asset/key inventory.
- 1027. Segment networks, minimize privileges, harden endpoints, and protect secrets.
- 1028. Encrypt sensitive off-chain data and minimize immutable PII.
- 1029. Test replay, auth bypass, prompt injection, model tampering, and malicious update media.
- 1030. Maintain kill switches, certificate revocation, evidence preservation, and recovery playbooks.

Implementation Example

```
# Host Hardening
INPUT  -> authenticate, validate schema, bind requestId/nonce
FACTS  -> read authoritative wallet/merchant/state data
AI     -> optional bounded recommendation with decisionId/version
POLICY -> enforce ownership, hard limits, approvals, risk fallback
LEDGER -> submit one idempotent deterministic transaction when value changes
EVENT  -> update projection/case/monitoring asynchronously
VERIFY -> discover committed result and retain correlationId + txId evidence
```

Verification Checklist

- The hardening baseline exists and is version-controlled.
- The feature works with all public Internet/WAN connectivity disabled.
- Authorization, malformed input, duplicate retry, and failure behavior have explicit tests.
- If financial value changes, journal debits and credits balance exactly and historical entries remain immutable.
- If AI is involved, model/prompt/feature version and fallback behavior are recorded and deterministic policy remains final.
- A timeout/restart cannot create a duplicate financial result; request status can be rediscovered.
- Logs/evidence contain correlation references but not secrets or prohibited sensitive payment data.

Common Failure Modes

- PII/card secrets placed on immutable ledger.
- Excessive network/admin privileges.
- Unsigned offline update bundle.
- No incident write-disable/revocation path.

Expert Review Questions

- What happens if this component succeeds but the next component fails?
- How is the authorizing identity/device/key revoked or recovered?
- Can the operation be safely retried after the client loses the response?
- What happens if AI is unavailable, slow, or wrong?
- Can an auditor reconstruct debit, credit, actor, risk decision, approval, and Fabric txId from retained evidence?
- Which compensating transaction resolves a downstream operational error without rewriting history?

Chapter 207 - Secret Management

Protect database, TLS, API, HSM, and application secrets. Offline does not mean secure. Insider abuse, stolen credentials, removable-media malware, endpoint compromise, replay, model tampering, configuration drift, and physical access remain serious threats. Use layered controls and preserve evidence across identity, application, ledger, model, and infrastructure layers.

Design rule: No secrets in source/container image/log

Implementation Objective

The required output is: Secret inventory. It must be version-controlled, reviewable, testable with the WAN disabled, and linked to the architecture/source-of-truth model. For a payment platform, every implementation decision should make it possible to explain who authorized an action, which facts were trusted, which controls ran, what financial journal committed, and how the result is recovered after failure.

Key Engineering Decisions

- No secrets in source/container image/log
- restrict read permissions
- rotate
- backup only encrypted/controlled

Deep Implementation Guidance

Apply defense in depth across physical access, network segmentation, host hardening, identity/key lifecycle, application authorization, ledger policies, data privacy, AI model integrity, and incident response. Offline systems are still exposed to insiders and removable-media supply-chain risk.

Data and Control Questions

- What is the authoritative source for every input used by this component?
- Which user, device, service, MSP, or administrator identity is allowed to initiate or approve it?
- Does the operation move value, reserve value, create a side effect, or only provide advisory information?
- What idempotency/replay key proves that a retry is the same business intent?
- What evidence is retained off-chain and what stable reference/hash belongs on the ledger?

Chapter 207 Implementation - Secret Management

Implementation Procedure

1031. Maintain an explicit threat model and asset/key inventory.
1032. Segment networks, minimize privileges, harden endpoints, and protect secrets.
1033. Encrypt sensitive off-chain data and minimize immutable PII.
1034. Test replay, auth bypass, prompt injection, model tampering, and malicious update media.
1035. Maintain kill switches, certificate revocation, evidence preservation, and recovery playbooks.

Implementation Example

```
# Secret Management
INPUT  -> authenticate, validate schema, bind requestId/nonce
FACTS  -> read authoritative wallet/merchant/state data
AI     -> optional bounded recommendation with decisionId/version
POLICY -> enforce ownership, hard limits, approvals, risk fallback
LEDGER -> submit one idempotent deterministic transaction when value changes
EVENT  -> update projection/case/monitoring asynchronously
VERIFY -> discover committed result and retain correlationId + txId evidence
```

Verification Checklist

- The secret inventory exists and is version-controlled.
- The feature works with all public Internet/WAN connectivity disabled.
- Authorization, malformed input, duplicate retry, and failure behavior have explicit tests.
- If financial value changes, journal debits and credits balance exactly and historical entries remain immutable.
- If AI is involved, model/prompt/feature version and fallback behavior are recorded and deterministic policy remains final.
- A timeout/restart cannot create a duplicate financial result; request status can be rediscovered.
- Logs/evidence contain correlation references but not secrets or prohibited sensitive payment data.

Common Failure Modes

- PII/card secrets placed on immutable ledger.
- Excessive network/admin privileges.
- Unsigned offline update bundle.
- No incident write-disable/revocation path.

Expert Review Questions

- What happens if this component succeeds but the next component fails?
- How is the authorizing identity/device/key revoked or recovered?
- Can the operation be safely retried after the client loses the response?
- What happens if AI is unavailable, slow, or wrong?
- Can an auditor reconstruct debit, credit, actor, risk decision, approval, and Fabric txId from retained evidence?
- Which compensating transaction resolves a downstream operational error without rewriting history?

Chapter 208 - Fabric Key Management

Protect enrollment and node/client signing identities according to financial impact. Offline does not mean secure. Insider abuse, stolen credentials, removable-media malware, endpoint compromise, replay, model tampering, configuration drift, and physical access remain serious threats. Use layered controls and preserve evidence across identity, application, ledger, model, and infrastructure layers.

Design rule: Inventory key owner/purpose/location

Implementation Objective

The required output is: Fabric key SOP. It must be version-controlled, reviewable, testable with the WAN disabled, and linked to the architecture/source-of-truth model. For a payment platform, every implementation decision should make it possible to explain who authorized an action, which facts were trusted, which controls ran, what financial journal committed, and how the result is recovered after failure.

Key Engineering Decisions

- Inventory key owner/purpose/location
- HSM for high-value identities where justified
- revocation and replacement drill

Deep Implementation Guidance

Apply defense in depth across physical access, network segmentation, host hardening, identity/key lifecycle, application authorization, ledger policies, data privacy, AI model integrity, and incident response. Offline systems are still exposed to insiders and removable-media supply-chain risk.

Data and Control Questions

- What is the authoritative source for every input used by this component?
- Which user, device, service, MSP, or administrator identity is allowed to initiate or approve it?
- Does the operation move value, reserve value, create a side effect, or only provide advisory information?
- What idempotency/replay key proves that a retry is the same business intent?
- What evidence is retained off-chain and what stable reference/hash belongs on the ledger?

Chapter 208 Implementation - Fabric Key Management

Implementation Procedure

- 1036. Maintain an explicit threat model and asset/key inventory.
- 1037. Segment networks, minimize privileges, harden endpoints, and protect secrets.
- 1038. Encrypt sensitive off-chain data and minimize immutable PII.
- 1039. Test replay, auth bypass, prompt injection, model tampering, and malicious update media.
- 1040. Maintain kill switches, certificate revocation, evidence preservation, and recovery playbooks.

Implementation Example

```
# Fabric Key Management
INPUT  -> authenticate, validate schema, bind requestId/nonce
FACTS  -> read authoritative wallet/merchant/state data
AI     -> optional bounded recommendation with decisionId/version
POLICY -> enforce ownership, hard limits, approvals, risk fallback
LEDGER -> submit one idempotent deterministic transaction when value changes
EVENT  -> update projection/case/monitoring asynchronously
VERIFY -> discover committed result and retain correlationId + txId evidence
```

Verification Checklist

- The fabric key sop exists and is version-controlled.
- The feature works with all public Internet/WAN connectivity disabled.
- Authorization, malformed input, duplicate retry, and failure behavior have explicit tests.
- If financial value changes, journal debits and credits balance exactly and historical entries remain immutable.
- If AI is involved, model/prompt/feature version and fallback behavior are recorded and deterministic policy remains final.
- A timeout/restart cannot create a duplicate financial result; request status can be rediscovered.
- Logs/evidence contain correlation references but not secrets or prohibited sensitive payment data.

Common Failure Modes

- PII/card secrets placed on immutable ledger.
- Excessive network/admin privileges.
- Unsigned offline update bundle.
- No incident write-disable/revocation path.

Expert Review Questions

- What happens if this component succeeds but the next component fails?
- How is the authorizing identity/device/key revoked or recovered?
- Can the operation be safely retried after the client loses the response?
- What happens if AI is unavailable, slow, or wrong?
- Can an auditor reconstruct debit, credit, actor, risk decision, approval, and Fabric txId from retained evidence?
- Which compensating transaction resolves a downstream operational error without rewriting history?

Chapter 209 - NIST Key Management Alignment

Apply lifecycle thinking to cryptographic material. Offline does not mean secure. Insider abuse, stolen credentials, removable-media malware, endpoint compromise, replay, model tampering, configuration drift, and physical access remain serious threats. Use layered controls and preserve evidence across identity, application, ledger, model, and infrastructure layers.

Design rule: Generation distribution storage use rotation revocation archival destruction

Implementation Objective

The required output is: Key lifecycle checklist. It must be version-controlled, reviewable, testable with the WAN disabled, and linked to the architecture/source-of-truth model. For a payment platform, every implementation decision should make it possible to explain who authorized an action, which facts were trusted, which controls ran, what financial journal committed, and how the result is recovered after failure.

Key Engineering Decisions

- Generation distribution storage use rotation revocation archival destruction
- document algorithm/key-size policy
- separate duties

Deep Implementation Guidance

Apply defense in depth across physical access, network segmentation, host hardening, identity/key lifecycle, application authorization, ledger policies, data privacy, AI model integrity, and incident response. Offline systems are still exposed to insiders and removable-media supply-chain risk.

Data and Control Questions

- What is the authoritative source for every input used by this component?
- Which user, device, service, MSP, or administrator identity is allowed to initiate or approve it?
- Does the operation move value, reserve value, create a side effect, or only provide advisory information?
- What idempotency/replay key proves that a retry is the same business intent?
- What evidence is retained off-chain and what stable reference/hash belongs on the ledger?

Chapter 209 Implementation - NIST Key Management Alignment

Implementation Procedure

1041. Maintain an explicit threat model and asset/key inventory.
1042. Segment networks, minimize privileges, harden endpoints, and protect secrets.
1043. Encrypt sensitive off-chain data and minimize immutable PII.
1044. Test replay, auth bypass, prompt injection, model tampering, and malicious update media.
1045. Maintain kill switches, certificate revocation, evidence preservation, and recovery playbooks.

Implementation Example

```
# NIST Key Management Alignment
INPUT  -> authenticate, validate schema, bind requestId/nonce
FACTS  -> read authoritative wallet/merchant/state data
AI     -> optional bounded recommendation with decisionId/version
POLICY -> enforce ownership, hard limits, approvals, risk fallback
LEDGER -> submit one idempotent deterministic transaction when value changes
EVENT  -> update projection/case/monitoring asynchronously
VERIFY -> discover committed result and retain correlationId + txId evidence
```

Verification Checklist

- The key lifecycle checklist exists and is version-controlled.
- The feature works with all public Internet/WAN connectivity disabled.
- Authorization, malformed input, duplicate retry, and failure behavior have explicit tests.
- If financial value changes, journal debits and credits balance exactly and historical entries remain immutable.
- If AI is involved, model/prompt/feature version and fallback behavior are recorded and deterministic policy remains final.
- A timeout/restart cannot create a duplicate financial result; request status can be rediscovered.
- Logs/evidence contain correlation references but not secrets or prohibited sensitive payment data.

Common Failure Modes

- PII/card secrets placed on immutable ledger.
- Excessive network/admin privileges.
- Unsigned offline update bundle.
- No incident write-disable/revocation path.

Expert Review Questions

- What happens if this component succeeds but the next component fails?
- How is the authorizing identity/device/key revoked or recovered?
- Can the operation be safely retried after the client loses the response?
- What happens if AI is unavailable, slow, or wrong?
- Can an auditor reconstruct debit, credit, actor, risk decision, approval, and Fabric txId from retained evidence?
- Which compensating transaction resolves a downstream operational error without rewriting history?

Chapter 210 - Application Security Verification

Use a structured web/API security standard such as OWASP ASVS concepts. Offline does not mean secure. Insider abuse, stolen credentials, removable-media malware, endpoint compromise, replay, model tampering, configuration drift, and physical access remain serious threats. Use layered controls and preserve evidence across identity, application, ledger, model, and infrastructure layers.

Design rule: Auth/session/access control/input validation/crypto/logging/config/API

Implementation Objective

The required output is: ASVS verification plan. It must be version-controlled, reviewable, testable with the WAN disabled, and linked to the architecture/source-of-truth model. For a payment platform, every implementation decision should make it possible to explain who authorized an action, which facts were trusted, which controls ran, what financial journal committed, and how the result is recovered after failure.

Key Engineering Decisions

- Auth/session/access control/input validation/crypto/logging/config/API
- test server-side authorization and business logic

Deep Implementation Guidance

Apply defense in depth across physical access, network segmentation, host hardening, identity/key lifecycle, application authorization, ledger policies, data privacy, AI model integrity, and incident response. Offline systems are still exposed to insiders and removable-media supply-chain risk.

Data and Control Questions

- What is the authoritative source for every input used by this component?
- Which user, device, service, MSP, or administrator identity is allowed to initiate or approve it?
- Does the operation move value, reserve value, create a side effect, or only provide advisory information?
- What idempotency/replay key proves that a retry is the same business intent?
- What evidence is retained off-chain and what stable reference/hash belongs on the ledger?

Chapter 210 Implementation - Application Security Verification

Implementation Procedure

- 1046. Maintain an explicit threat model and asset/key inventory.
- 1047. Segment networks, minimize privileges, harden endpoints, and protect secrets.
- 1048. Encrypt sensitive off-chain data and minimize immutable PII.
- 1049. Test replay, auth bypass, prompt injection, model tampering, and malicious update media.
- 1050. Maintain kill switches, certificate revocation, evidence preservation, and recovery playbooks.

Implementation Example

```
# Application Security Verification
INPUT  -> authenticate, validate schema, bind requestId/nonce
FACTS  -> read authoritative wallet/merchant/state data
AI      -> optional bounded recommendation with decisionId/version
POLICY  -> enforce ownership, hard limits, approvals, risk fallback
LEDGER  -> submit one idempotent deterministic transaction when value changes
EVENT   -> update projection/case/monitoring asynchronously
VERIFY  -> discover committed result and retain correlationId + txId evidence
```

Verification Checklist

- The asvs verification plan exists and is version-controlled.
- The feature works with all public Internet/WAN connectivity disabled.
- Authorization, malformed input, duplicate retry, and failure behavior have explicit tests.
- If financial value changes, journal debits and credits balance exactly and historical entries remain immutable.
- If AI is involved, model/prompt/feature version and fallback behavior are recorded and deterministic policy remains final.
- A timeout/restart cannot create a duplicate financial result; request status can be rediscovered.
- Logs/evidence contain correlation references but not secrets or prohibited sensitive payment data.

Common Failure Modes

- PII/card secrets placed on immutable ledger.
- Excessive network/admin privileges.
- Unsigned offline update bundle.
- No incident write-disable/revocation path.

Expert Review Questions

- What happens if this component succeeds but the next component fails?
- How is the authorizing identity/device/key revoked or recovered?
- Can the operation be safely retried after the client loses the response?
- What happens if AI is unavailable, slow, or wrong?
- Can an auditor reconstruct debit, credit, actor, risk decision, approval, and Fabric txId from retained evidence?
- Which compensating transaction resolves a downstream operational error without rewriting history?

Chapter 211 - OWASP LLM Security Verification

Apply LLM-specific controls to local assistants even without Internet exposure. Offline does not mean secure. Insider abuse, stolen credentials, removable-media malware, endpoint compromise, replay, model tampering, configuration drift, and physical access remain serious threats. Use layered controls and preserve evidence across identity, application, ledger, model, and infrastructure layers.

Design rule: Prompt injection/data leakage/tool abuse/model integrity/memory/storage/integration/monitoring risks

Implementation Objective

The required output is: LLM security checklist. It must be version-controlled, reviewable, testable with the WAN disabled, and linked to the architecture/source-of-truth model. For a payment platform, every implementation decision should make it possible to explain who authorized an action, which facts were trusted, which controls ran, what financial journal committed, and how the result is recovered after failure.

Key Engineering Decisions

- Prompt injection/data leakage/tool abuse/model integrity/memory/storage/integration/monitoring risks
- verify each control

Deep Implementation Guidance

Apply defense in depth across physical access, network segmentation, host hardening, identity/key lifecycle, application authorization, ledger policies, data privacy, AI model integrity, and incident response. Offline systems are still exposed to insiders and removable-media supply-chain risk.

Data and Control Questions

- What is the authoritative source for every input used by this component?
- Which user, device, service, MSP, or administrator identity is allowed to initiate or approve it?
- Does the operation move value, reserve value, create a side effect, or only provide advisory information?
- What idempotency/replay key proves that a retry is the same business intent?
- What evidence is retained off-chain and what stable reference/hash belongs on the ledger?

Chapter 211 Implementation - OWASP LLM Security Verification

Implementation Procedure

1051. Maintain an explicit threat model and asset/key inventory.
1052. Segment networks, minimize privileges, harden endpoints, and protect secrets.
1053. Encrypt sensitive off-chain data and minimize immutable PII.
1054. Test replay, auth bypass, prompt injection, model tampering, and malicious update media.
1055. Maintain kill switches, certificate revocation, evidence preservation, and recovery playbooks.

Implementation Example

```
# OWASP LLM Security Verification
INPUT  -> authenticate, validate schema, bind requestId/nonce
FACTS  -> read authoritative wallet/merchant/state data
AI     -> optional bounded recommendation with decisionId/version
POLICY -> enforce ownership, hard limits, approvals, risk fallback
LEDGER -> submit one idempotent deterministic transaction when value changes
EVENT  -> update projection/case/monitoring asynchronously
VERIFY -> discover committed result and retain correlationId + txId evidence
```

Verification Checklist

- The llm security checklist exists and is version-controlled.
- The feature works with all public Internet/WAN connectivity disabled.
- Authorization, malformed input, duplicate retry, and failure behavior have explicit tests.
- If financial value changes, journal debits and credits balance exactly and historical entries remain immutable.
- If AI is involved, model/prompt/feature version and fallback behavior are recorded and deterministic policy remains final.
- A timeout/restart cannot create a duplicate financial result; request status can be rediscovered.
- Logs/evidence contain correlation references but not secrets or prohibited sensitive payment data.

Common Failure Modes

- PII/card secrets placed on immutable ledger.
- Excessive network/admin privileges.
- Unsigned offline update bundle.
- No incident write-disable/revocation path.

Expert Review Questions

- What happens if this component succeeds but the next component fails?
- How is the authorizing identity/device/key revoked or recovered?
- Can the operation be safely retried after the client loses the response?
- What happens if AI is unavailable, slow, or wrong?
- Can an auditor reconstruct debit, credit, actor, risk decision, approval, and Fabric txId from retained evidence?
- Which compensating transaction resolves a downstream operational error without rewriting history?

Chapter 212 - NIST AI Risk Management Profile

Use govern-map-measure-manage discipline for local fraud/LLM features. Offline does not mean secure. Insider abuse, stolen credentials, removable-media malware, endpoint compromise, replay, model tampering, configuration drift, and physical access remain serious threats. Use layered controls and preserve evidence across identity, application, ledger, model, and infrastructure layers.

Design rule: Document context/impact

Implementation Objective

The required output is: AI risk register. It must be version-controlled, reviewable, testable with the WAN disabled, and linked to the architecture/source-of-truth model. For a payment platform, every implementation decision should make it possible to explain who authorized an action, which facts were trusted, which controls ran, what financial journal committed, and how the result is recovered after failure.

Key Engineering Decisions

- Document context/impact
- measure quality/safety
- assign owners
- manage residual risk and rollback

Deep Implementation Guidance

Apply defense in depth across physical access, network segmentation, host hardening, identity/key lifecycle, application authorization, ledger policies, data privacy, AI model integrity, and incident response. Offline systems are still exposed to insiders and removable-media supply-chain risk.

Data and Control Questions

- What is the authoritative source for every input used by this component?
- Which user, device, service, MSP, or administrator identity is allowed to initiate or approve it?
- Does the operation move value, reserve value, create a side effect, or only provide advisory information?
- What idempotency/replay key proves that a retry is the same business intent?
- What evidence is retained off-chain and what stable reference/hash belongs on the ledger?

Chapter 212 Implementation - NIST AI Risk Management Profile

Implementation Procedure

- 1056. Maintain an explicit threat model and asset/key inventory.
- 1057. Segment networks, minimize privileges, harden endpoints, and protect secrets.
- 1058. Encrypt sensitive off-chain data and minimize immutable PII.
- 1059. Test replay, auth bypass, prompt injection, model tampering, and malicious update media.
- 1060. Maintain kill switches, certificate revocation, evidence preservation, and recovery playbooks.

Implementation Example

```
# NIST AI Risk Management Profile
INPUT  -> authenticate, validate schema, bind requestId/nonce
FACTS  -> read authoritative wallet/merchant/state data
AI      -> optional bounded recommendation with decisionId/version
POLICY  -> enforce ownership, hard limits, approvals, risk fallback
LEDGER  -> submit one idempotent deterministic transaction when value changes
EVENT   -> update projection/case/monitoring asynchronously
VERIFY  -> discover committed result and retain correlationId + txId evidence
```

Verification Checklist

- The ai risk register exists and is version-controlled.
- The feature works with all public Internet/WAN connectivity disabled.
- Authorization, malformed input, duplicate retry, and failure behavior have explicit tests.
- If financial value changes, journal debits and credits balance exactly and historical entries remain immutable.
- If AI is involved, model/prompt/feature version and fallback behavior are recorded and deterministic policy remains final.
- A timeout/restart cannot create a duplicate financial result; request status can be rediscovered.
- Logs/evidence contain correlation references but not secrets or prohibited sensitive payment data.

Common Failure Modes

- PII/card secrets placed on immutable ledger.
- Excessive network/admin privileges.
- Unsigned offline update bundle.
- No incident write-disable/revocation path.

Expert Review Questions

- What happens if this component succeeds but the next component fails?
- How is the authorizing identity/device/key revoked or recovered?
- Can the operation be safely retried after the client loses the response?
- What happens if AI is unavailable, slow, or wrong?
- Can an auditor reconstruct debit, credit, actor, risk decision, approval, and Fabric txId from retained evidence?
- Which compensating transaction resolves a downstream operational error without rewriting history?

Chapter 213 - PII Minimization

Keep names/contact/identity documents and sensitive case evidence off immutable ledger. Offline does not mean secure. Insider abuse, stolen credentials, removable-media malware, endpoint compromise, replay, model tampering, configuration drift, and physical access remain serious threats. Use layered controls and preserve evidence across identity, application, ledger, model, and infrastructure layers.

Design rule: Ledger uses opaque refs/hashes

Implementation Objective

The required output is: PII map. It must be version-controlled, reviewable, testable with the WAN disabled, and linked to the architecture/source-of-truth model. For a payment platform, every implementation decision should make it possible to explain who authorized an action, which facts were trusted, which controls ran, what financial journal committed, and how the result is recovered after failure.

Key Engineering Decisions

- Ledger uses opaque refs/hashes
- database encrypts/access-controls PII
- retention/deletion remains possible off-chain

Deep Implementation Guidance

Apply defense in depth across physical access, network segmentation, host hardening, identity/key lifecycle, application authorization, ledger policies, data privacy, AI model integrity, and incident response. Offline systems are still exposed to insiders and removable-media supply-chain risk.

Data and Control Questions

- What is the authoritative source for every input used by this component?
- Which user, device, service, MSP, or administrator identity is allowed to initiate or approve it?
- Does the operation move value, reserve value, create a side effect, or only provide advisory information?
- What idempotency/replay key proves that a retry is the same business intent?
- What evidence is retained off-chain and what stable reference/hash belongs on the ledger?

Chapter 213 Implementation - PII Minimization

Implementation Procedure

1061. Maintain an explicit threat model and asset/key inventory.
1062. Segment networks, minimize privileges, harden endpoints, and protect secrets.
1063. Encrypt sensitive off-chain data and minimize immutable PII.
1064. Test replay, auth bypass, prompt injection, model tampering, and malicious update media.
1065. Maintain kill switches, certificate revocation, evidence preservation, and recovery playbooks.

Implementation Example

```
# PII Minimization
INPUT  -> authenticate, validate schema, bind requestId/nonce
FACTS  -> read authoritative wallet/merchant/state data
AI     -> optional bounded recommendation with decisionId/version
POLICY -> enforce ownership, hard limits, approvals, risk fallback
LEDGER -> submit one idempotent deterministic transaction when value changes
EVENT  -> update projection/case/monitoring asynchronously
VERIFY -> discover committed result and retain correlationId + txId evidence
```

Verification Checklist

- The pii map exists and is version-controlled.
- The feature works with all public Internet/WAN connectivity disabled.
- Authorization, malformed input, duplicate retry, and failure behavior have explicit tests.
- If financial value changes, journal debits and credits balance exactly and historical entries remain immutable.
- If AI is involved, model/prompt/feature version and fallback behavior are recorded and deterministic policy remains final.
- A timeout/restart cannot create a duplicate financial result; request status can be rediscovered.
- Logs/evidence contain correlation references but not secrets or prohibited sensitive payment data.

Common Failure Modes

- PII/card secrets placed on immutable ledger.
- Excessive network/admin privileges.
- Unsigned offline update bundle.
- No incident write-disable/revocation path.

Expert Review Questions

- What happens if this component succeeds but the next component fails?
- How is the authorizing identity/device/key revoked or recovered?
- Can the operation be safely retried after the client loses the response?
- What happens if AI is unavailable, slow, or wrong?
- Can an auditor reconstruct debit, credit, actor, risk decision, approval, and Fabric txId from retained evidence?
- Which compensating transaction resolves a downstream operational error without rewriting history?

Chapter 214 - Payment Card Data Boundary

Avoid bringing PAN/CVV into this architecture unless a real card-processing requirement exists. Offline does not mean secure. Insider abuse, stolen credentials, removable-media malware, endpoint compromise, replay, model tampering, configuration drift, and physical access remain serious threats. Use layered controls and preserve evidence across identity, application, ledger, model, and infrastructure layers.

Design rule: Closed-loop wallet avoids cardholder-data storage

Implementation Objective

The required output is: PCI boundary statement. It must be version-controlled, reviewable, testable with the WAN disabled, and linked to the architecture/source-of-truth model. For a payment platform, every implementation decision should make it possible to explain who authorized an action, which facts were trusted, which controls ran, what financial journal committed, and how the result is recovered after failure.

Key Engineering Decisions

- Closed-loop wallet avoids cardholder-data storage
- If card data is processed transmitted or stored, treat PCI DSS scope separately
- never place PAN/CVV on-chain

Deep Implementation Guidance

Apply defense in depth across physical access, network segmentation, host hardening, identity/key lifecycle, application authorization, ledger policies, data privacy, AI model integrity, and incident response. Offline systems are still exposed to insiders and removable-media supply-chain risk.

Data and Control Questions

- What is the authoritative source for every input used by this component?
- Which user, device, service, MSP, or administrator identity is allowed to initiate or approve it?
- Does the operation move value, reserve value, create a side effect, or only provide advisory information?
- What idempotency/replay key proves that a retry is the same business intent?
- What evidence is retained off-chain and what stable reference/hash belongs on the ledger?

Chapter 214 Implementation - Payment Card Data Boundary

Implementation Procedure

- 1066. Maintain an explicit threat model and asset/key inventory.
- 1067. Segment networks, minimize privileges, harden endpoints, and protect secrets.
- 1068. Encrypt sensitive off-chain data and minimize immutable PII.
- 1069. Test replay, auth bypass, prompt injection, model tampering, and malicious update media.
- 1070. Maintain kill switches, certificate revocation, evidence preservation, and recovery playbooks.

Implementation Example

```
# Payment Card Data Boundary
INPUT  -> authenticate, validate schema, bind requestId/nonce
FACTS  -> read authoritative wallet/merchant/state data
AI     -> optional bounded recommendation with decisionId/version
POLICY -> enforce ownership, hard limits, approvals, risk fallback
LEDGER -> submit one idempotent deterministic transaction when value changes
EVENT  -> update projection/case/monitoring asynchronously
VERIFY -> discover committed result and retain correlationId + txId evidence
```

Verification Checklist

- The pci boundary statement exists and is version-controlled.
- The feature works with all public Internet/WAN connectivity disabled.
- Authorization, malformed input, duplicate retry, and failure behavior have explicit tests.
- If financial value changes, journal debits and credits balance exactly and historical entries remain immutable.
- If AI is involved, model/prompt/feature version and fallback behavior are recorded and deterministic policy remains final.
- A timeout/restart cannot create a duplicate financial result; request status can be rediscovered.
- Logs/evidence contain correlation references but not secrets or prohibited sensitive payment data.

Common Failure Modes

- PII/card secrets placed on immutable ledger.
- Excessive network/admin privileges.
- Unsigned offline update bundle.
- No incident write-disable/revocation path.

Expert Review Questions

- What happens if this component succeeds but the next component fails?
- How is the authorizing identity/device/key revoked or recovered?
- Can the operation be safely retried after the client loses the response?
- What happens if AI is unavailable, slow, or wrong?
- Can an auditor reconstruct debit, credit, actor, risk decision, approval, and Fabric txId from retained evidence?
- Which compensating transaction resolves a downstream operational error without rewriting history?

Chapter 215 - Replay and Request Forgery Protection

Prevent captured transaction intents from being reused or altered. Offline does not mean secure. Insider abuse, stolen credentials, removable-media malware, endpoint compromise, replay, model tampering, configuration drift, and physical access remain serious threats. Use layered controls and preserve evidence across identity, application, ledger, model, and infrastructure layers.

Design rule: TLS

Implementation Objective

The required output is: Request security design. It must be version-controlled, reviewable, testable with the WAN disabled, and linked to the architecture/source-of-truth model. For a payment platform, every implementation decision should make it possible to explain who authorized an action, which facts were trusted, which controls ran, what financial journal committed, and how the result is recovered after failure.

Key Engineering Decisions

- TLS
- session auth
- signed envelope where used
- nonce/requestId
- destination/amount binding
- idempotency

Deep Implementation Guidance

Apply defense in depth across physical access, network segmentation, host hardening, identity/key lifecycle, application authorization, ledger policies, data privacy, AI model integrity, and incident response. Offline systems are still exposed to insiders and removable-media supply-chain risk.

Data and Control Questions

- What is the authoritative source for every input used by this component?
- Which user, device, service, MSP, or administrator identity is allowed to initiate or approve it?
- Does the operation move value, reserve value, create a side effect, or only provide advisory information?
- What idempotency/replay key proves that a retry is the same business intent?
- What evidence is retained off-chain and what stable reference/hash belongs on the ledger?

Chapter 215 Implementation - Replay and Request Forgery Protection

Implementation Procedure

1071. Maintain an explicit threat model and asset/key inventory.
1072. Segment networks, minimize privileges, harden endpoints, and protect secrets.
1073. Encrypt sensitive off-chain data and minimize immutable PII.
1074. Test replay, auth bypass, prompt injection, model tampering, and malicious update media.
1075. Maintain kill switches, certificate revocation, evidence preservation, and recovery playbooks.

Implementation Example

```
# Replay and Request Forgery Protection
INPUT  -> authenticate, validate schema, bind requestId/nonce
FACTS  -> read authoritative wallet/merchant/state data
AI     -> optional bounded recommendation with decisionId/version
POLICY -> enforce ownership, hard limits, approvals, risk fallback
LEDGER -> submit one idempotent deterministic transaction when value changes
EVENT  -> update projection/case/monitoring asynchronously
VERIFY -> discover committed result and retain correlationId + txId evidence
```

Verification Checklist

- The request security design exists and is version-controlled.
- The feature works with all public Internet/WAN connectivity disabled.
- Authorization, malformed input, duplicate retry, and failure behavior have explicit tests.
- If financial value changes, journal debits and credits balance exactly and historical entries remain immutable.
- If AI is involved, model/prompt/feature version and fallback behavior are recorded and deterministic policy remains final.
- A timeout/restart cannot create a duplicate financial result; request status can be rediscovered.
- Logs/evidence contain correlation references but not secrets or prohibited sensitive payment data.

Common Failure Modes

- PII/card secrets placed on immutable ledger.
- Excessive network/admin privileges.
- Unsigned offline update bundle.
- No incident write-disable/revocation path.

Expert Review Questions

- What happens if this component succeeds but the next component fails?
- How is the authorizing identity/device/key revoked or recovered?
- Can the operation be safely retried after the client loses the response?
- What happens if AI is unavailable, slow, or wrong?
- Can an auditor reconstruct debit, credit, actor, risk decision, approval, and Fabric txId from retained evidence?
- Which compensating transaction resolves a downstream operational error without rewriting history?

Chapter 216 - Transaction Tampering Controls

Ensure browser/POS cannot change server-owned merchant/amount/fee facts silently. Offline does not mean secure. Insider abuse, stolen credentials, removable-media malware, endpoint compromise, replay, model tampering, configuration drift, and physical access remain serious threats. Use layered controls and preserve evidence across identity, application, ledger, model, and infrastructure layers.

Design rule: Server re-resolves merchant/order/price

Implementation Objective

The required output is: Integrity controls. It must be version-controlled, reviewable, testable with the WAN disabled, and linked to the architecture/source-of-truth model. For a payment platform, every implementation decision should make it possible to explain who authorized an action, which facts were trusted, which controls ran, what financial journal committed, and how the result is recovered after failure.

Key Engineering Decisions

- Server re-resolves merchant/order/price
- chaincode revalidates asset/status/balance/fee
- authorization digest binds user intent

Deep Implementation Guidance

Apply defense in depth across physical access, network segmentation, host hardening, identity/key lifecycle, application authorization, ledger policies, data privacy, AI model integrity, and incident response. Offline systems are still exposed to insiders and removable-media supply-chain risk.

Data and Control Questions

- What is the authoritative source for every input used by this component?
- Which user, device, service, MSP, or administrator identity is allowed to initiate or approve it?
- Does the operation move value, reserve value, create a side effect, or only provide advisory information?
- What idempotency/replay key proves that a retry is the same business intent?
- What evidence is retained off-chain and what stable reference/hash belongs on the ledger?

Chapter 216 Implementation - Transaction Tampering Controls

Implementation Procedure

- 1076. Maintain an explicit threat model and asset/key inventory.
- 1077. Segment networks, minimize privileges, harden endpoints, and protect secrets.
- 1078. Encrypt sensitive off-chain data and minimize immutable PII.
- 1079. Test replay, auth bypass, prompt injection, model tampering, and malicious update media.
- 1080. Maintain kill switches, certificate revocation, evidence preservation, and recovery playbooks.

Implementation Example

```
# Transaction Tampering Controls
INPUT  -> authenticate, validate schema, bind requestId/nonce
FACTS  -> read authoritative wallet/merchant/state data
AI     -> optional bounded recommendation with decisionId/version
POLICY -> enforce ownership, hard limits, approvals, risk fallback
LEDGER -> submit one idempotent deterministic transaction when value changes
EVENT  -> update projection/case/monitoring asynchronously
VERIFY -> discover committed result and retain correlationId + txId evidence
```

Verification Checklist

- The integrity controls exists and is version-controlled.
- The feature works with all public Internet/WAN connectivity disabled.
- Authorization, malformed input, duplicate retry, and failure behavior have explicit tests.
- If financial value changes, journal debits and credits balance exactly and historical entries remain immutable.
- If AI is involved, model/prompt/feature version and fallback behavior are recorded and deterministic policy remains final.
- A timeout/restart cannot create a duplicate financial result; request status can be rediscovered.
- Logs/evidence contain correlation references but not secrets or prohibited sensitive payment data.

Common Failure Modes

- PII/card secrets placed on immutable ledger.
- Excessive network/admin privileges.
- Unsigned offline update bundle.
- No incident write-disable/revocation path.

Expert Review Questions

- What happens if this component succeeds but the next component fails?
- How is the authorizing identity/device/key revoked or recovered?
- Can the operation be safely retried after the client loses the response?
- What happens if AI is unavailable, slow, or wrong?
- Can an auditor reconstruct debit, credit, actor, risk decision, approval, and Fabric txId from retained evidence?
- Which compensating transaction resolves a downstream operational error without rewriting history?

Chapter 217 - Database Tamper Detection

Detect or recover from projection/case/AI data manipulation. Offline does not mean secure. Insider abuse, stolen credentials, removable-media malware, endpoint compromise, replay, model tampering, configuration drift, and physical access remain serious threats. Use layered controls and preserve evidence across identity, application, ledger, model, and infrastructure layers.

Design rule: Ledger remains authoritative for financial entries

Implementation Objective

The required output is: DB integrity plan. It must be version-controlled, reviewable, testable with the WAN disabled, and linked to the architecture/source-of-truth model. For a payment platform, every implementation decision should make it possible to explain who authorized an action, which facts were trusted, which controls ran, what financial journal committed, and how the result is recovered after failure.

Key Engineering Decisions

- Ledger remains authoritative for financial entries
- regular reconciliation
- restricted DB admins
- backup/log/audit changes

Deep Implementation Guidance

Apply defense in depth across physical access, network segmentation, host hardening, identity/key lifecycle, application authorization, ledger policies, data privacy, AI model integrity, and incident response. Offline systems are still exposed to insiders and removable-media supply-chain risk.

Data and Control Questions

- What is the authoritative source for every input used by this component?
- Which user, device, service, MSP, or administrator identity is allowed to initiate or approve it?
- Does the operation move value, reserve value, create a side effect, or only provide advisory information?
- What idempotency/replay key proves that a retry is the same business intent?
- What evidence is retained off-chain and what stable reference/hash belongs on the ledger?

Chapter 217 Implementation - Database Tamper Detection

Implementation Procedure

1081. Maintain an explicit threat model and asset/key inventory.
1082. Segment networks, minimize privileges, harden endpoints, and protect secrets.
1083. Encrypt sensitive off-chain data and minimize immutable PII.
1084. Test replay, auth bypass, prompt injection, model tampering, and malicious update media.
1085. Maintain kill switches, certificate revocation, evidence preservation, and recovery playbooks.

Implementation Example

```
# Database Tamper Detection
INPUT  -> authenticate, validate schema, bind requestId/nonce
FACTS  -> read authoritative wallet/merchant/state data
AI     -> optional bounded recommendation with decisionId/version
POLICY -> enforce ownership, hard limits, approvals, risk fallback
LEDGER -> submit one idempotent deterministic transaction when value changes
EVENT  -> update projection/case/monitoring asynchronously
VERIFY -> discover committed result and retain correlationId + txId evidence
```

Verification Checklist

- The db integrity plan exists and is version-controlled.
- The feature works with all public Internet/WAN connectivity disabled.
- Authorization, malformed input, duplicate retry, and failure behavior have explicit tests.
- If financial value changes, journal debits and credits balance exactly and historical entries remain immutable.
- If AI is involved, model/prompt/feature version and fallback behavior are recorded and deterministic policy remains final.
- A timeout/restart cannot create a duplicate financial result; request status can be rediscovered.
- Logs/evidence contain correlation references but not secrets or prohibited sensitive payment data.

Common Failure Modes

- PII/card secrets placed on immutable ledger.
- Excessive network/admin privileges.
- Unsigned offline update bundle.
- No incident write-disable/revocation path.

Expert Review Questions

- What happens if this component succeeds but the next component fails?
- How is the authorizing identity/device/key revoked or recovered?
- Can the operation be safely retried after the client loses the response?
- What happens if AI is unavailable, slow, or wrong?
- Can an auditor reconstruct debit, credit, actor, risk decision, approval, and Fabric txId from retained evidence?
- Which compensating transaction resolves a downstream operational error without rewriting history?

Chapter 218 - Model Tamper Detection

Prevent an unapproved model or prompt from silently changing fraud decisions. Offline does not mean secure. Insider abuse, stolen credentials, removable-media malware, endpoint compromise, replay, model tampering, configuration drift, and physical access remain serious threats. Use layered controls and preserve evidence across identity, application, ledger, model, and infrastructure layers.

Design rule: Hash model/prompt/schema

Implementation Objective

The required output is: AI integrity controls. It must be version-controlled, reviewable, testable with the WAN disabled, and linked to the architecture/source-of-truth model. For a payment platform, every implementation decision should make it possible to explain who authorized an action, which facts were trusted, which controls ran, what financial journal committed, and how the result is recovered after failure.

Key Engineering Decisions

- Hash model/prompt/schema
- registry allowlist
- startup verification
- deployment approval
- alert on unknown hash

Deep Implementation Guidance

Apply defense in depth across physical access, network segmentation, host hardening, identity/key lifecycle, application authorization, ledger policies, data privacy, AI model integrity, and incident response. Offline systems are still exposed to insiders and removable-media supply-chain risk.

Data and Control Questions

- What is the authoritative source for every input used by this component?
- Which user, device, service, MSP, or administrator identity is allowed to initiate or approve it?
- Does the operation move value, reserve value, create a side effect, or only provide advisory information?
- What idempotency/replay key proves that a retry is the same business intent?
- What evidence is retained off-chain and what stable reference/hash belongs on the ledger?

Chapter 218 Implementation - Model Tamper Detection

Implementation Procedure

1086. Maintain an explicit threat model and asset/key inventory.
1087. Segment networks, minimize privileges, harden endpoints, and protect secrets.
1088. Encrypt sensitive off-chain data and minimize immutable PII.
1089. Test replay, auth bypass, prompt injection, model tampering, and malicious update media.
1090. Maintain kill switches, certificate revocation, evidence preservation, and recovery playbooks.

Implementation Example

```
# Model Tamper Detection
INPUT  -> authenticate, validate schema, bind requestId/nonce
FACTS  -> read authoritative wallet/merchant/state data
AI     -> optional bounded recommendation with decisionId/version
POLICY -> enforce ownership, hard limits, approvals, risk fallback
LEDGER -> submit one idempotent deterministic transaction when value changes
EVENT  -> update projection/case/monitoring asynchronously
VERIFY -> discover committed result and retain correlationId + txId evidence
```

Verification Checklist

- The ai integrity controls exists and is version-controlled.
- The feature works with all public Internet/WAN connectivity disabled.
- Authorization, malformed input, duplicate retry, and failure behavior have explicit tests.
- If financial value changes, journal debits and credits balance exactly and historical entries remain immutable.
- If AI is involved, model/prompt/feature version and fallback behavior are recorded and deterministic policy remains final.
- A timeout/restart cannot create a duplicate financial result; request status can be rediscovered.
- Logs/evidence contain correlation references but not secrets or prohibited sensitive payment data.

Common Failure Modes

- PII/card secrets placed on immutable ledger.
- Excessive network/admin privileges.
- Unsigned offline update bundle.
- No incident write-disable/revocation path.

Expert Review Questions

- What happens if this component succeeds but the next component fails?
- How is the authorizing identity/device/key revoked or recovered?
- Can the operation be safely retried after the client loses the response?
- What happens if AI is unavailable, slow, or wrong?
- Can an auditor reconstruct debit, credit, actor, risk decision, approval, and Fabric txId from retained evidence?
- Which compensating transaction resolves a downstream operational error without rewriting history?

Chapter 219 - Training Data Poisoning Controls

Protect offline retraining data and labels. Offline does not mean secure. Insider abuse, stolen credentials, removable-media malware, endpoint compromise, replay, model tampering, configuration drift, and physical access remain serious threats. Use layered controls and preserve evidence across identity, application, ledger, model, and infrastructure layers.

Design rule: Signed/export manifests

Implementation Objective

The required output is: ML data security. It must be version-controlled, reviewable, testable with the WAN disabled, and linked to the architecture/source-of-truth model. For a payment platform, every implementation decision should make it possible to explain who authorized an action, which facts were trusted, which controls ran, what financial journal committed, and how the result is recovered after failure.

Key Engineering Decisions

- Signed/export manifests
- restricted label edits
- dataset hash
- review anomalous bulk changes
- keep raw source lineage

Deep Implementation Guidance

Apply defense in depth across physical access, network segmentation, host hardening, identity/key lifecycle, application authorization, ledger policies, data privacy, AI model integrity, and incident response. Offline systems are still exposed to insiders and removable-media supply-chain risk.

Data and Control Questions

- What is the authoritative source for every input used by this component?
- Which user, device, service, MSP, or administrator identity is allowed to initiate or approve it?
- Does the operation move value, reserve value, create a side effect, or only provide advisory information?
- What idempotency/replay key proves that a retry is the same business intent?
- What evidence is retained off-chain and what stable reference/hash belongs on the ledger?

Chapter 219 Implementation - Training Data Poisoning Controls

Implementation Procedure

1091. Maintain an explicit threat model and asset/key inventory.
1092. Segment networks, minimize privileges, harden endpoints, and protect secrets.
1093. Encrypt sensitive off-chain data and minimize immutable PII.
1094. Test replay, auth bypass, prompt injection, model tampering, and malicious update media.
1095. Maintain kill switches, certificate revocation, evidence preservation, and recovery playbooks.

Implementation Example

```
# Training Data Poisoning Controls
INPUT  -> authenticate, validate schema, bind requestId/nonce
FACTS  -> read authoritative wallet/merchant/state data
AI     -> optional bounded recommendation with decisionId/version
POLICY -> enforce ownership, hard limits, approvals, risk fallback
LEDGER -> submit one idempotent deterministic transaction when value changes
EVENT  -> update projection/case/monitoring asynchronously
VERIFY -> discover committed result and retain correlationId + txId evidence
```

Verification Checklist

- The ml data security exists and is version-controlled.
- The feature works with all public Internet/WAN connectivity disabled.
- Authorization, malformed input, duplicate retry, and failure behavior have explicit tests.
- If financial value changes, journal debits and credits balance exactly and historical entries remain immutable.
- If AI is involved, model/prompt/feature version and fallback behavior are recorded and deterministic policy remains final.
- A timeout/restart cannot create a duplicate financial result; request status can be rediscovered.
- Logs/evidence contain correlation references but not secrets or prohibited sensitive payment data.

Common Failure Modes

- PII/card secrets placed on immutable ledger.
- Excessive network/admin privileges.
- Unsigned offline update bundle.
- No incident write-disable/revocation path.

Expert Review Questions

- What happens if this component succeeds but the next component fails?
- How is the authorizing identity/device/key revoked or recovered?
- Can the operation be safely retried after the client loses the response?
- What happens if AI is unavailable, slow, or wrong?
- Can an auditor reconstruct debit, credit, actor, risk decision, approval, and Fabric txId from retained evidence?
- Which compensating transaction resolves a downstream operational error without rewriting history?

Chapter 220 - Prompt Injection and Data Exfiltration

Prevent local LLM from following malicious embedded instructions. Offline does not mean secure. Insider abuse, stolen credentials, removable-media malware, endpoint compromise, replay, model tampering, configuration drift, and physical access remain serious threats. Use layered controls and preserve evidence across identity, application, ledger, model, and infrastructure layers.

Design rule: No secrets in prompt

Implementation Objective

The required output is: LLM isolation. It must be version-controlled, reviewable, testable with the WAN disabled, and linked to the architecture/source-of-truth model. For a payment platform, every implementation decision should make it possible to explain who authorized an action, which facts were trusted, which controls ran, what financial journal committed, and how the result is recovered after failure.

Key Engineering Decisions

- No secrets in prompt
- retrieval ACL before model
- tool allowlist
- structured outputs
- security evals
- network egress blocked

Deep Implementation Guidance

Apply defense in depth across physical access, network segmentation, host hardening, identity/key lifecycle, application authorization, ledger policies, data privacy, AI model integrity, and incident response. Offline systems are still exposed to insiders and removable-media supply-chain risk.

Data and Control Questions

- What is the authoritative source for every input used by this component?
- Which user, device, service, MSP, or administrator identity is allowed to initiate or approve it?
- Does the operation move value, reserve value, create a side effect, or only provide advisory information?
- What idempotency/replay key proves that a retry is the same business intent?
- What evidence is retained off-chain and what stable reference/hash belongs on the ledger?

Chapter 220 Implementation - Prompt Injection and Data Exfiltration

Implementation Procedure

1096. Maintain an explicit threat model and asset/key inventory.
1097. Segment networks, minimize privileges, harden endpoints, and protect secrets.
1098. Encrypt sensitive off-chain data and minimize immutable PII.
1099. Test replay, auth bypass, prompt injection, model tampering, and malicious update media.
1100. Maintain kill switches, certificate revocation, evidence preservation, and recovery playbooks.

Implementation Example

```
# Prompt Injection and Data Exfiltration
INPUT  -> authenticate, validate schema, bind requestId/nonce
FACTS  -> read authoritative wallet/merchant/state data
AI     -> optional bounded recommendation with decisionId/version
POLICY -> enforce ownership, hard limits, approvals, risk fallback
LEDGER -> submit one idempotent deterministic transaction when value changes
EVENT  -> update projection/case/monitoring asynchronously
VERIFY -> discover committed result and retain correlationId + txId evidence
```

Verification Checklist

- The llm isolation exists and is version-controlled.
- The feature works with all public Internet/WAN connectivity disabled.
- Authorization, malformed input, duplicate retry, and failure behavior have explicit tests.
- If financial value changes, journal debits and credits balance exactly and historical entries remain immutable.
- If AI is involved, model/prompt/feature version and fallback behavior are recorded and deterministic policy remains final.
- A timeout/restart cannot create a duplicate financial result; request status can be rediscovered.
- Logs/evidence contain correlation references but not secrets or prohibited sensitive payment data.

Common Failure Modes

- PII/card secrets placed on immutable ledger.
- Excessive network/admin privileges.
- Unsigned offline update bundle.
- No incident write-disable/revocation path.

Expert Review Questions

- What happens if this component succeeds but the next component fails?
- How is the authorizing identity/device/key revoked or recovered?
- Can the operation be safely retried after the client loses the response?
- What happens if AI is unavailable, slow, or wrong?
- Can an auditor reconstruct debit, credit, actor, risk decision, approval, and Fabric txId from retained evidence?
- Which compensating transaction resolves a downstream operational error without rewriting history?

Chapter 221 - Insider Fraud and Separation of Duties

Reduce risk that one operator can create and hide value movements. Offline does not mean secure. Insider abuse, stolen credentials, removable-media malware, endpoint compromise, replay, model tampering, configuration drift, and physical access remain serious threats. Use layered controls and preserve evidence across identity, application, ledger, model, and infrastructure layers.

Design rule: Treasury requester/approver split

Implementation Objective

The required output is: SoD matrix. It must be version-controlled, reviewable, testable with the WAN disabled, and linked to the architecture/source-of-truth model. For a payment platform, every implementation decision should make it possible to explain who authorized an action, which facts were trusted, which controls ran, what financial journal committed, and how the result is recovered after failure.

Key Engineering Decisions

- Treasury requester/approver split
- AuditOrg independent
- DB admin cannot alter ledger
- break-glass use reviewed

Deep Implementation Guidance

Apply defense in depth across physical access, network segmentation, host hardening, identity/key lifecycle, application authorization, ledger policies, data privacy, AI model integrity, and incident response. Offline systems are still exposed to insiders and removable-media supply-chain risk.

Data and Control Questions

- What is the authoritative source for every input used by this component?
- Which user, device, service, MSP, or administrator identity is allowed to initiate or approve it?
- Does the operation move value, reserve value, create a side effect, or only provide advisory information?
- What idempotency/replay key proves that a retry is the same business intent?
- What evidence is retained off-chain and what stable reference/hash belongs on the ledger?

Chapter 221 Implementation - Insider Fraud and Separation of Duties

Implementation Procedure

1101. Maintain an explicit threat model and asset/key inventory.
1102. Segment networks, minimize privileges, harden endpoints, and protect secrets.
1103. Encrypt sensitive off-chain data and minimize immutable PII.
1104. Test replay, auth bypass, prompt injection, model tampering, and malicious update media.
1105. Maintain kill switches, certificate revocation, evidence preservation, and recovery playbooks.

Implementation Example

```
# Insider Fraud and Separation of Duties
INPUT  -> authenticate, validate schema, bind requestId/nonce
FACTS  -> read authoritative wallet/merchant/state data
AI     -> optional bounded recommendation with decisionId/version
POLICY -> enforce ownership, hard limits, approvals, risk fallback
LEDGER -> submit one idempotent deterministic transaction when value changes
EVENT  -> update projection/case/monitoring asynchronously
VERIFY -> discover committed result and retain correlationId + txId evidence
```

Verification Checklist

- The sod matrix exists and is version-controlled.
- The feature works with all public Internet/WAN connectivity disabled.
- Authorization, malformed input, duplicate retry, and failure behavior have explicit tests.
- If financial value changes, journal debits and credits balance exactly and historical entries remain immutable.
- If AI is involved, model/prompt/feature version and fallback behavior are recorded and deterministic policy remains final.
- A timeout/restart cannot create a duplicate financial result; request status can be rediscovered.
- Logs/evidence contain correlation references but not secrets or prohibited sensitive payment data.

Common Failure Modes

- PII/card secrets placed on immutable ledger.
- Excessive network/admin privileges.
- Unsigned offline update bundle.
- No incident write-disable/revocation path.

Expert Review Questions

- What happens if this component succeeds but the next component fails?
- How is the authorizing identity/device/key revoked or recovered?
- Can the operation be safely retried after the client loses the response?
- What happens if AI is unavailable, slow, or wrong?
- Can an auditor reconstruct debit, credit, actor, risk decision, approval, and Fabric txId from retained evidence?
- Which compensating transaction resolves a downstream operational error without rewriting history?

Chapter 222 - Physical Security

Protect LAN servers, HSMs, backup media, POS devices, and network equipment. Offline does not mean secure. Insider abuse, stolen credentials, removable-media malware, endpoint compromise, replay, model tampering, configuration drift, and physical access remain serious threats. Use layered controls and preserve evidence across identity, application, ledger, model, and infrastructure layers.

Design rule: Locked rooms/racks

Implementation Objective

The required output is: Physical control plan. It must be version-controlled, reviewable, testable with the WAN disabled, and linked to the architecture/source-of-truth model. For a payment platform, every implementation decision should make it possible to explain who authorized an action, which facts were trusted, which controls ran, what financial journal committed, and how the result is recovered after failure.

Key Engineering Decisions

- Locked rooms/racks
- asset inventory
- tamper response
- controlled USB/removable media
- UPS/environment monitoring

Deep Implementation Guidance

Apply defense in depth across physical access, network segmentation, host hardening, identity/key lifecycle, application authorization, ledger policies, data privacy, AI model integrity, and incident response. Offline systems are still exposed to insiders and removable-media supply-chain risk.

Data and Control Questions

- What is the authoritative source for every input used by this component?
- Which user, device, service, MSP, or administrator identity is allowed to initiate or approve it?
- Does the operation move value, reserve value, create a side effect, or only provide advisory information?
- What idempotency/replay key proves that a retry is the same business intent?
- What evidence is retained off-chain and what stable reference/hash belongs on the ledger?

Chapter 222 Implementation - Physical Security

Implementation Procedure

- 1106. Maintain an explicit threat model and asset/key inventory.
- 1107. Segment networks, minimize privileges, harden endpoints, and protect secrets.
- 1108. Encrypt sensitive off-chain data and minimize immutable PII.
- 1109. Test replay, auth bypass, prompt injection, model tampering, and malicious update media.
- 1110. Maintain kill switches, certificate revocation, evidence preservation, and recovery playbooks.

Implementation Example

```
# Physical Security
INPUT  -> authenticate, validate schema, bind requestId/nonce
FACTS  -> read authoritative wallet/merchant/state data
AI     -> optional bounded recommendation with decisionId/version
POLICY -> enforce ownership, hard limits, approvals, risk fallback
LEDGER -> submit one idempotent deterministic transaction when value changes
EVENT  -> update projection/case/monitoring asynchronously
VERIFY -> discover committed result and retain correlationId + txId evidence
```

Verification Checklist

- The physical control plan exists and is version-controlled.
- The feature works with all public Internet/WAN connectivity disabled.
- Authorization, malformed input, duplicate retry, and failure behavior have explicit tests.
- If financial value changes, journal debits and credits balance exactly and historical entries remain immutable.
- If AI is involved, model/prompt/feature version and fallback behavior are recorded and deterministic policy remains final.
- A timeout/restart cannot create a duplicate financial result; request status can be rediscovered.
- Logs/evidence contain correlation references but not secrets or prohibited sensitive payment data.

Common Failure Modes

- PII/card secrets placed on immutable ledger.
- Excessive network/admin privileges.
- Unsigned offline update bundle.
- No incident write-disable/revocation path.

Expert Review Questions

- What happens if this component succeeds but the next component fails?
- How is the authorizing identity/device/key revoked or recovered?
- Can the operation be safely retried after the client loses the response?
- What happens if AI is unavailable, slow, or wrong?
- Can an auditor reconstruct debit, credit, actor, risk decision, approval, and Fabric txId from retained evidence?
- Which compensating transaction resolves a downstream operational error without rewriting history?

Chapter 223 - Audit Logging

Preserve security and financial evidence without logging secrets. Offline does not mean secure. Insider abuse, stolen credentials, removable-media malware, endpoint compromise, replay, model tampering, configuration drift, and physical access remain serious threats. Use layered controls and preserve evidence across identity, application, ledger, model, and infrastructure layers.

Design rule: Actor/device/request/decision/approval/tx/result

Implementation Objective

The required output is: Audit log schema. It must be version-controlled, reviewable, testable with the WAN disabled, and linked to the architecture/source-of-truth model. For a payment platform, every implementation decision should make it possible to explain who authorized an action, which facts were trusted, which controls ran, what financial journal committed, and how the result is recovered after failure.

Key Engineering Decisions

- Actor/device/request/decision/approval/tx/result
- central local logging
- time sync
- access controls
- retention

Deep Implementation Guidance

Apply defense in depth across physical access, network segmentation, host hardening, identity/key lifecycle, application authorization, ledger policies, data privacy, AI model integrity, and incident response. Offline systems are still exposed to insiders and removable-media supply-chain risk.

Data and Control Questions

- What is the authoritative source for every input used by this component?
- Which user, device, service, MSP, or administrator identity is allowed to initiate or approve it?
- Does the operation move value, reserve value, create a side effect, or only provide advisory information?
- What idempotency/replay key proves that a retry is the same business intent?
- What evidence is retained off-chain and what stable reference/hash belongs on the ledger?

Chapter 223 Implementation - Audit Logging

Implementation Procedure

- 1111. Maintain an explicit threat model and asset/key inventory.
- 1112. Segment networks, minimize privileges, harden endpoints, and protect secrets.
- 1113. Encrypt sensitive off-chain data and minimize immutable PII.
- 1114. Test replay, auth bypass, prompt injection, model tampering, and malicious update media.
- 1115. Maintain kill switches, certificate revocation, evidence preservation, and recovery playbooks.

Implementation Example

```
# Audit Logging
INPUT  -> authenticate, validate schema, bind requestId/nonce
FACTS  -> read authoritative wallet/merchant/state data
AI     -> optional bounded recommendation with decisionId/version
POLICY -> enforce ownership, hard limits, approvals, risk fallback
LEDGER -> submit one idempotent deterministic transaction when value changes
EVENT  -> update projection/case/monitoring asynchronously
VERIFY -> discover committed result and retain correlationId + txId evidence
```

Verification Checklist

- The audit log schema exists and is version-controlled.
- The feature works with all public Internet/WAN connectivity disabled.
- Authorization, malformed input, duplicate retry, and failure behavior have explicit tests.
- If financial value changes, journal debits and credits balance exactly and historical entries remain immutable.
- If AI is involved, model/prompt/feature version and fallback behavior are recorded and deterministic policy remains final.
- A timeout/restart cannot create a duplicate financial result; request status can be rediscovered.
- Logs/evidence contain correlation references but not secrets or prohibited sensitive payment data.

Common Failure Modes

- PII/card secrets placed on immutable ledger.
- Excessive network/admin privileges.
- Unsigned offline update bundle.
- No incident write-disable/revocation path.

Expert Review Questions

- What happens if this component succeeds but the next component fails?
- How is the authorizing identity/device/key revoked or recovered?
- Can the operation be safely retried after the client loses the response?
- What happens if AI is unavailable, slow, or wrong?
- Can an auditor reconstruct debit, credit, actor, risk decision, approval, and Fabric txId from retained evidence?
- Which compensating transaction resolves a downstream operational error without rewriting history?

Chapter 224 - Incident Kill Switches

Stop risky writes while preserving reads/evidence during compromise. Offline does not mean secure. Insider abuse, stolen credentials, removable-media malware, endpoint compromise, replay, model tampering, configuration drift, and physical access remain serious threats. Use layered controls and preserve evidence across identity, application, ledger, model, and infrastructure layers.

Design rule: Disable selected API write routes

Implementation Objective

The required output is: Kill-switch design. It must be version-controlled, reviewable, testable with the WAN disabled, and linked to the architecture/source-of-truth model. For a payment platform, every implementation decision should make it possible to explain who authorized an action, which facts were trusted, which controls ran, what financial journal committed, and how the result is recovered after failure.

Key Engineering Decisions

- Disable selected API write routes
- freeze wallets/merchant where policy
- revoke identities
- stop model enforcement
- preserve logs before cleanup

Deep Implementation Guidance

Apply defense in depth across physical access, network segmentation, host hardening, identity/key lifecycle, application authorization, ledger policies, data privacy, AI model integrity, and incident response. Offline systems are still exposed to insiders and removable-media supply-chain risk.

Data and Control Questions

- What is the authoritative source for every input used by this component?
- Which user, device, service, MSP, or administrator identity is allowed to initiate or approve it?
- Does the operation move value, reserve value, create a side effect, or only provide advisory information?
- What idempotency/replay key proves that a retry is the same business intent?
- What evidence is retained off-chain and what stable reference/hash belongs on the ledger?

Chapter 224 Implementation - Incident Kill Switches

Implementation Procedure

- 1116. Maintain an explicit threat model and asset/key inventory.
- 1117. Segment networks, minimize privileges, harden endpoints, and protect secrets.
- 1118. Encrypt sensitive off-chain data and minimize immutable PII.
- 1119. Test replay, auth bypass, prompt injection, model tampering, and malicious update media.
- 1120. Maintain kill switches, certificate revocation, evidence preservation, and recovery playbooks.

Implementation Example

```
# Incident Kill Switches
INPUT  -> authenticate, validate schema, bind requestId/nonce
FACTS  -> read authoritative wallet/merchant/state data
AI     -> optional bounded recommendation with decisionId/version
POLICY -> enforce ownership, hard limits, approvals, risk fallback
LEDGER -> submit one idempotent deterministic transaction when value changes
EVENT  -> update projection/case/monitoring asynchronously
VERIFY -> discover committed result and retain correlationId + txId evidence
```

Verification Checklist

- The kill-switch design exists and is version-controlled.
- The feature works with all public Internet/WAN connectivity disabled.
- Authorization, malformed input, duplicate retry, and failure behavior have explicit tests.
- If financial value changes, journal debits and credits balance exactly and historical entries remain immutable.
- If AI is involved, model/prompt/feature version and fallback behavior are recorded and deterministic policy remains final.
- A timeout/restart cannot create a duplicate financial result; request status can be rediscovered.
- Logs/evidence contain correlation references but not secrets or prohibited sensitive payment data.

Common Failure Modes

- PII/card secrets placed on immutable ledger.
- Excessive network/admin privileges.
- Unsigned offline update bundle.
- No incident write-disable/revocation path.

Expert Review Questions

- What happens if this component succeeds but the next component fails?
- How is the authorizing identity/device/key revoked or recovered?
- Can the operation be safely retried after the client loses the response?
- What happens if AI is unavailable, slow, or wrong?
- Can an auditor reconstruct debit, credit, actor, risk decision, approval, and Fabric txId from retained evidence?
- Which compensating transaction resolves a downstream operational error without rewriting history?

Chapter 225 - Security Incident Response

Define detection, containment, evidence, recovery, and postmortem workflow. Offline does not mean secure. Insider abuse, stolen credentials, removable-media malware, endpoint compromise, replay, model tampering, configuration drift, and physical access remain serious threats. Use layered controls and preserve evidence across identity, application, ledger, model, and infrastructure layers.

Design rule: Assign incident commander

Implementation Objective

The required output is: IR runbook. It must be version-controlled, reviewable, testable with the WAN disabled, and linked to the architecture/source-of-truth model. For a payment platform, every implementation decision should make it possible to explain who authorized an action, which facts were trusted, which controls ran, what financial journal committed, and how the result is recovered after failure.

Key Engineering Decisions

- Assign incident commander
- capture volatile evidence
- revoke/rotate
- reconcile financial state
- restore from known-good
- document impact

Deep Implementation Guidance

Apply defense in depth across physical access, network segmentation, host hardening, identity/key lifecycle, application authorization, ledger policies, data privacy, AI model integrity, and incident response. Offline systems are still exposed to insiders and removable-media supply-chain risk.

Data and Control Questions

- What is the authoritative source for every input used by this component?
- Which user, device, service, MSP, or administrator identity is allowed to initiate or approve it?
- Does the operation move value, reserve value, create a side effect, or only provide advisory information?
- What idempotency/replay key proves that a retry is the same business intent?
- What evidence is retained off-chain and what stable reference/hash belongs on the ledger?

Chapter 225 Implementation - Security Incident Response

Implementation Procedure

- 1121. Maintain an explicit threat model and asset/key inventory.
- 1122. Segment networks, minimize privileges, harden endpoints, and protect secrets.
- 1123. Encrypt sensitive off-chain data and minimize immutable PII.
- 1124. Test replay, auth bypass, prompt injection, model tampering, and malicious update media.
- 1125. Maintain kill switches, certificate revocation, evidence preservation, and recovery playbooks.

Implementation Example

```
# Security Incident Response
INPUT  -> authenticate, validate schema, bind requestId/nonce
FACTS  -> read authoritative wallet/merchant/state data
AI      -> optional bounded recommendation with decisionId/version
POLICY  -> enforce ownership, hard limits, approvals, risk fallback
LEDGER  -> submit one idempotent deterministic transaction when value changes
EVENT   -> update projection/case/monitoring asynchronously
VERIFY  -> discover committed result and retain correlationId + txId evidence
```

Verification Checklist

- The ir runbook exists and is version-controlled.
- The feature works with all public Internet/WAN connectivity disabled.
- Authorization, malformed input, duplicate retry, and failure behavior have explicit tests.
- If financial value changes, journal debits and credits balance exactly and historical entries remain immutable.
- If AI is involved, model/prompt/feature version and fallback behavior are recorded and deterministic policy remains final.
- A timeout/restart cannot create a duplicate financial result; request status can be rediscovered.
- Logs/evidence contain correlation references but not secrets or prohibited sensitive payment data.

Common Failure Modes

- PII/card secrets placed on immutable ledger.
- Excessive network/admin privileges.
- Unsigned offline update bundle.
- No incident write-disable/revocation path.

Expert Review Questions

- What happens if this component succeeds but the next component fails?
- How is the authorizing identity/device/key revoked or recovered?
- Can the operation be safely retried after the client loses the response?
- What happens if AI is unavailable, slow, or wrong?
- Can an auditor reconstruct debit, credit, actor, risk decision, approval, and Fabric txId from retained evidence?
- Which compensating transaction resolves a downstream operational error without rewriting history?

Chapter 226 - Offline Software Update Security

Prevent supply-chain compromise via removable media and stale artifacts. Offline does not mean secure. Insider abuse, stolen credentials, removable-media malware, endpoint compromise, replay, model tampering, configuration drift, and physical access remain serious threats. Use layered controls and preserve evidence across identity, application, ledger, model, and infrastructure layers.

Design rule: Verify vendor signature/hash

Implementation Objective

The required output is: Update security SOP. It must be version-controlled, reviewable, testable with the WAN disabled, and linked to the architecture/source-of-truth model. For a payment platform, every implementation decision should make it possible to explain who authorized an action, which facts were trusted, which controls ran, what financial journal committed, and how the result is recovered after failure.

Key Engineering Decisions

- Verify vendor signature/hash
- internal manifest
- malware scan
- staging test
- dual approval for critical updates
- rollback bundle

Deep Implementation Guidance

Apply defense in depth across physical access, network segmentation, host hardening, identity/key lifecycle, application authorization, ledger policies, data privacy, AI model integrity, and incident response. Offline systems are still exposed to insiders and removable-media supply-chain risk.

Data and Control Questions

- What is the authoritative source for every input used by this component?
- Which user, device, service, MSP, or administrator identity is allowed to initiate or approve it?
- Does the operation move value, reserve value, create a side effect, or only provide advisory information?
- What idempotency/replay key proves that a retry is the same business intent?
- What evidence is retained off-chain and what stable reference/hash belongs on the ledger?

Chapter 226 Implementation - Offline Software Update Security

Implementation Procedure

- 1126. Maintain an explicit threat model and asset/key inventory.
- 1127. Segment networks, minimize privileges, harden endpoints, and protect secrets.
- 1128. Encrypt sensitive off-chain data and minimize immutable PII.
- 1129. Test replay, auth bypass, prompt injection, model tampering, and malicious update media.
- 1130. Maintain kill switches, certificate revocation, evidence preservation, and recovery playbooks.

Implementation Example

```
# Offline Software Update Security
INPUT  -> authenticate, validate schema, bind requestId/nonce
FACTS  -> read authoritative wallet/merchant/state data
AI     -> optional bounded recommendation with decisionId/version
POLICY -> enforce ownership, hard limits, approvals, risk fallback
LEDGER -> submit one idempotent deterministic transaction when value changes
EVENT  -> update projection/case/monitoring asynchronously
VERIFY -> discover committed result and retain correlationId + txId evidence
```

Verification Checklist

- The update security sop exists and is version-controlled.
- The feature works with all public Internet/WAN connectivity disabled.
- Authorization, malformed input, duplicate retry, and failure behavior have explicit tests.
- If financial value changes, journal debits and credits balance exactly and historical entries remain immutable.
- If AI is involved, model/prompt/feature version and fallback behavior are recorded and deterministic policy remains final.
- A timeout/restart cannot create a duplicate financial result; request status can be rediscovered.
- Logs/evidence contain correlation references but not secrets or prohibited sensitive payment data.

Common Failure Modes

- PII/card secrets placed on immutable ledger.
- Excessive network/admin privileges.
- Unsigned offline update bundle.
- No incident write-disable/revocation path.

Expert Review Questions

- What happens if this component succeeds but the next component fails?
- How is the authorizing identity/device/key revoked or recovered?
- Can the operation be safely retried after the client loses the response?
- What happens if AI is unavailable, slow, or wrong?
- Can an auditor reconstruct debit, credit, actor, risk decision, approval, and Fabric txId from retained evidence?
- Which compensating transaction resolves a downstream operational error without rewriting history?

Chapter 227 - Backup Security

Protect backups that contain keys, PII, models, and ledger/database state. Offline does not mean secure. Insider abuse, stolen credentials, removable-media malware, endpoint compromise, replay, model tampering, configuration drift, and physical access remain serious threats. Use layered controls and preserve evidence across identity, application, ledger, model, and infrastructure layers.

Design rule: Encrypt

Implementation Objective

The required output is: Backup security policy. It must be version-controlled, reviewable, testable with the WAN disabled, and linked to the architecture/source-of-truth model. For a payment platform, every implementation decision should make it possible to explain who authorized an action, which facts were trusted, which controls ran, what financial journal committed, and how the result is recovered after failure.

Key Engineering Decisions

- Encrypt
- restrict operators
- separate key material
- immutable/offline copy
- restore test
- document destruction/retention

Deep Implementation Guidance

Apply defense in depth across physical access, network segmentation, host hardening, identity/key lifecycle, application authorization, ledger policies, data privacy, AI model integrity, and incident response. Offline systems are still exposed to insiders and removable-media supply-chain risk.

Data and Control Questions

- What is the authoritative source for every input used by this component?
- Which user, device, service, MSP, or administrator identity is allowed to initiate or approve it?
- Does the operation move value, reserve value, create a side effect, or only provide advisory information?
- What idempotency/replay key proves that a retry is the same business intent?
- What evidence is retained off-chain and what stable reference/hash belongs on the ledger?

Chapter 227 Implementation - Backup Security

Implementation Procedure

- 1131. Maintain an explicit threat model and asset/key inventory.
- 1132. Segment networks, minimize privileges, harden endpoints, and protect secrets.
- 1133. Encrypt sensitive off-chain data and minimize immutable PII.
- 1134. Test replay, auth bypass, prompt injection, model tampering, and malicious update media.
- 1135. Maintain kill switches, certificate revocation, evidence preservation, and recovery playbooks.

Implementation Example

```
# Backup Security
INPUT  -> authenticate, validate schema, bind requestId/nonce
FACTS  -> read authoritative wallet/merchant/state data
AI     -> optional bounded recommendation with decisionId/version
POLICY -> enforce ownership, hard limits, approvals, risk fallback
LEDGER -> submit one idempotent deterministic transaction when value changes
EVENT  -> update projection/case/monitoring asynchronously
VERIFY -> discover committed result and retain correlationId + txId evidence
```

Verification Checklist

- The backup security policy exists and is version-controlled.
- The feature works with all public Internet/WAN connectivity disabled.
- Authorization, malformed input, duplicate retry, and failure behavior have explicit tests.
- If financial value changes, journal debits and credits balance exactly and historical entries remain immutable.
- If AI is involved, model/prompt/feature version and fallback behavior are recorded and deterministic policy remains final.
- A timeout/restart cannot create a duplicate financial result; request status can be rediscovered.
- Logs/evidence contain correlation references but not secrets or prohibited sensitive payment data.

Common Failure Modes

- PII/card secrets placed on immutable ledger.
- Excessive network/admin privileges.
- Unsigned offline update bundle.
- No incident write-disable/revocation path.

Expert Review Questions

- What happens if this component succeeds but the next component fails?
- How is the authorizing identity/device/key revoked or recovered?
- Can the operation be safely retried after the client loses the response?
- What happens if AI is unavailable, slow, or wrong?
- Can an auditor reconstruct debit, credit, actor, risk decision, approval, and Fabric txId from retained evidence?
- Which compensating transaction resolves a downstream operational error without rewriting history?

Part XIII - Testing, Evaluation, and Quality Engineering

Financial correctness must be tested as invariants, not only API responses. Test balanced journals, conservation of value, authorization, idempotency, concurrent spend, quorum loss, model failure, projection rebuild, restore, and human-approval workflows with traceable evidence.

Part focus: This part contains 23 implementation chapters. Each chapter includes design rules, implementation procedure, a practical artifact, verification, failure analysis, and expert review questions.

Chapter 228 - Accounting Invariant Unit Tests

Prove every chaincode financial path preserves balance and state rules. Financial correctness must be tested as invariants, not only API responses. Test balanced journals, conservation of value, authorization, idempotency, concurrent spend, quorum loss, model failure, projection rebuild, restore, and human-approval workflows with traceable evidence.

Design rule: Sum debits credits

Implementation Objective

The required output is: Accounting test suite. It must be version-controlled, reviewable, testable with the WAN disabled, and linked to the architecture/source-of-truth model. For a payment platform, every implementation decision should make it possible to explain who authorized an action, which facts were trusted, which controls ran, what financial journal committed, and how the result is recovered after failure.

Key Engineering Decisions

- Sum debits credits
- no unauthorized creation
- no negative available balance unless allowed
- refund/capture limits

Deep Implementation Guidance

A test passes only when financial state, journal balance, transaction validity, events, projection state, AI/approval evidence, and visible API outcome all match expectations. Retain requestId/decisionId/approvalId/txId so failures can be reconstructed.

Data and Control Questions

- What is the authoritative source for every input used by this component?
- Which user, device, service, MSP, or administrator identity is allowed to initiate or approve it?
- Does the operation move value, reserve value, create a side effect, or only provide advisory information?
- What idempotency/replay key proves that a retry is the same business intent?
- What evidence is retained off-chain and what stable reference/hash belongs on the ledger?

Chapter 228 Implementation - Accounting Invariant Unit Tests

Implementation Procedure

- 1136. Define invariant-level expected outcomes and retain transaction IDs.
- 1137. Test normal, boundary, malformed, unauthorized, duplicate, and concurrent requests.
- 1138. Inject peer/orderer/database/AI/network failures.
- 1139. Reconcile ledger, projections, model decisions, approvals, and visible API outcome.
- 1140. Run backup/restore, certificate rotation, and production rehearsal before go-live.

Implementation Example

```
# Accounting Invariant Unit Tests
INPUT  -> authenticate, validate schema, bind requestId/nonce
FACTS  -> read authoritative wallet/merchant/state data
AI      -> optional bounded recommendation with decisionId/version
POLICY  -> enforce ownership, hard limits, approvals, risk fallback
LEDGER  -> submit one idempotent deterministic transaction when value changes
EVENT   -> update projection/case/monitoring asynchronously
VERIFY  -> discover committed result and retain correlationId + txId evidence
```

Verification Checklist

- The accounting test suite exists and is version-controlled.
- The feature works with all public Internet/WAN connectivity disabled.
- Authorization, malformed input, duplicate retry, and failure behavior have explicit tests.
- If financial value changes, journal debits and credits balance exactly and historical entries remain immutable.
- If AI is involved, model/prompt/feature version and fallback behavior are recorded and deterministic policy remains final.
- A timeout/restart cannot create a duplicate financial result; request status can be rediscovered.
- Logs/evidence contain correlation references but not secrets or prohibited sensitive payment data.

Common Failure Modes

- Only happy-path demos.
- No concurrent spend/idempotency tests.
- Model tested separately but not in workflow.
- Backup exists but restore never proven.

Expert Review Questions

- What happens if this component succeeds but the next component fails?
- How is the authorizing identity/device/key revoked or recovered?
- Can the operation be safely retried after the client loses the response?
- What happens if AI is unavailable, slow, or wrong?
- Can an auditor reconstruct debit, credit, actor, risk decision, approval, and Fabric txId from retained evidence?
- Which compensating transaction resolves a downstream operational error without rewriting history?

Chapter 229 - Chaincode Unit Tests

Test wallet lifecycle, journals, holds, refunds, freezes, errors, and idempotency. Financial correctness must be tested as invariants, not only API responses. Test balanced journals, conservation of value, authorization, idempotency, concurrent spend, quorum loss, model failure, projection rebuild, restore, and human-approval workflows with traceable evidence.

Design rule: Use deterministic mocks/test network

Implementation Objective

The required output is: Chaincode unit suite. It must be version-controlled, reviewable, testable with the WAN disabled, and linked to the architecture/source-of-truth model. For a payment platform, every implementation decision should make it possible to explain who authorized an action, which facts were trusted, which controls ran, what financial journal committed, and how the result is recovered after failure.

Key Engineering Decisions

- Use deterministic mocks/test network
- assert state and events
- test every stable error code

Deep Implementation Guidance

A test passes only when financial state, journal balance, transaction validity, events, projection state, AI/approval evidence, and visible API outcome all match expectations. Retain requestId/decisionId/approvalId/txId so failures can be reconstructed.

Data and Control Questions

- What is the authoritative source for every input used by this component?
- Which user, device, service, MSP, or administrator identity is allowed to initiate or approve it?
- Does the operation move value, reserve value, create a side effect, or only provide advisory information?
- What idempotency/replay key proves that a retry is the same business intent?
- What evidence is retained off-chain and what stable reference/hash belongs on the ledger?

Chapter 229 Implementation - Chaincode Unit Tests

Implementation Procedure

1141. Define invariant-level expected outcomes and retain transaction IDs.
1142. Test normal, boundary, malformed, unauthorized, duplicate, and concurrent requests.
1143. Inject peer/orderer/database/AI/network failures.
1144. Reconcile ledger, projections, model decisions, approvals, and visible API outcome.
1145. Run backup/restore, certificate rotation, and production rehearsal before go-live.

Implementation Example

```
# Chaincode Unit Tests
INPUT  -> authenticate, validate schema, bind requestId/nonce
FACTS  -> read authoritative wallet/merchant/state data
AI      -> optional bounded recommendation with decisionId/version
POLICY -> enforce ownership, hard limits, approvals, risk fallback
LEDGER  -> submit one idempotent deterministic transaction when value changes
EVENT   -> update projection/case/monitoring asynchronously
VERIFY  -> discover committed result and retain correlationId + txId evidence
```

Verification Checklist

- The chaincode unit suite exists and is version-controlled.
- The feature works with all public Internet/WAN connectivity disabled.
- Authorization, malformed input, duplicate retry, and failure behavior have explicit tests.
- If financial value changes, journal debits and credits balance exactly and historical entries remain immutable.
- If AI is involved, model/prompt/feature version and fallback behavior are recorded and deterministic policy remains final.
- A timeout/restart cannot create a duplicate financial result; request status can be rediscovered.
- Logs/evidence contain correlation references but not secrets or prohibited sensitive payment data.

Common Failure Modes

- Only happy-path demos.
- No concurrent spend/idempotency tests.
- Model tested separately but not in workflow.
- Backup exists but restore never proven.

Expert Review Questions

- What happens if this component succeeds but the next component fails?
- How is the authorizing identity/device/key revoked or recovered?
- Can the operation be safely retried after the client loses the response?
- What happens if AI is unavailable, slow, or wrong?
- Can an auditor reconstruct debit, credit, actor, risk decision, approval, and Fabric txId from retained evidence?
- Which compensating transaction resolves a downstream operational error without rewriting history?

Chapter 230 - Fabric Integration Tests

Verify real endorsement, ordering, validation, commit status, and events. Financial correctness must be tested as invariants, not only API responses. Test balanced journals, conservation of value, authorization, idempotency, concurrent spend, quorum loss, model failure, projection rebuild, restore, and human-approval workflows with traceable evidence.

Design rule: Submit through Gateway

Implementation Objective

The required output is: Integration suite. It must be version-controlled, reviewable, testable with the WAN disabled, and linked to the architecture/source-of-truth model. For a payment platform, every implementation decision should make it possible to explain who authorized an action, which facts were trusted, which controls ran, what financial journal committed, and how the result is recovered after failure.

Key Engineering Decisions

- Submit through Gateway
- check ledger values and tx validation
- confirm event projection
- retain tx IDs

Deep Implementation Guidance

A test passes only when financial state, journal balance, transaction validity, events, projection state, AI/approval evidence, and visible API outcome all match expectations. Retain requestId/decisionId/approvalId/txId so failures can be reconstructed.

Data and Control Questions

- What is the authoritative source for every input used by this component?
- Which user, device, service, MSP, or administrator identity is allowed to initiate or approve it?
- Does the operation move value, reserve value, create a side effect, or only provide advisory information?
- What idempotency/replay key proves that a retry is the same business intent?
- What evidence is retained off-chain and what stable reference/hash belongs on the ledger?

Chapter 230 Implementation - Fabric Integration Tests

Implementation Procedure

- 1146. Define invariant-level expected outcomes and retain transaction IDs.
- 1147. Test normal, boundary, malformed, unauthorized, duplicate, and concurrent requests.
- 1148. Inject peer/orderer/database/AI/network failures.
- 1149. Reconcile ledger, projections, model decisions, approvals, and visible API outcome.
- 1150. Run backup/restore, certificate rotation, and production rehearsal before go-live.

Implementation Example

```
# Fabric Integration Tests
INPUT  -> authenticate, validate schema, bind requestId/nonce
FACTS  -> read authoritative wallet/merchant/state data
AI     -> optional bounded recommendation with decisionId/version
POLICY -> enforce ownership, hard limits, approvals, risk fallback
LEDGER -> submit one idempotent deterministic transaction when value changes
EVENT  -> update projection/case/monitoring asynchronously
VERIFY -> discover committed result and retain correlationId + txId evidence
```

Verification Checklist

- The integration suite exists and is version-controlled.
- The feature works with all public Internet/WAN connectivity disabled.
- Authorization, malformed input, duplicate retry, and failure behavior have explicit tests.
- If financial value changes, journal debits and credits balance exactly and historical entries remain immutable.
- If AI is involved, model/prompt/feature version and fallback behavior are recorded and deterministic policy remains final.
- A timeout/restart cannot create a duplicate financial result; request status can be rediscovered.
- Logs/evidence contain correlation references but not secrets or prohibited sensitive payment data.

Common Failure Modes

- Only happy-path demos.
- No concurrent spend/idempotency tests.
- Model tested separately but not in workflow.
- Backup exists but restore never proven.

Expert Review Questions

- What happens if this component succeeds but the next component fails?
- How is the authorizing identity/device/key revoked or recovered?
- Can the operation be safely retried after the client loses the response?
- What happens if AI is unavailable, slow, or wrong?
- Can an auditor reconstruct debit, credit, actor, risk decision, approval, and Fabric txId from retained evidence?
- Which compensating transaction resolves a downstream operational error without rewriting history?

Chapter 231 - Concurrent Double-Spend Tests

Attempt simultaneous transfers/payments against the same available balance. Financial correctness must be tested as invariants, not only API responses. Test balanced journals, conservation of value, authorization, idempotency, concurrent spend, quorum loss, model failure, projection rebuild, restore, and human-approval workflows with traceable evidence.

Design rule: Only transactions consistent with available balance commit

Implementation Objective

The required output is: Concurrency suite. It must be version-controlled, reviewable, testable with the WAN disabled, and linked to the architecture/source-of-truth model. For a payment platform, every implementation decision should make it possible to explain who authorized an action, which facts were trusted, which controls ran, what financial journal committed, and how the result is recovered after failure.

Key Engineering Decisions

- Only transactions consistent with available balance commit
- MVCC/revalidation errors handled
- no retry creates overspend

Deep Implementation Guidance

A test passes only when financial state, journal balance, transaction validity, events, projection state, AI/approval evidence, and visible API outcome all match expectations. Retain requestId/decisionId/approvalId/txId so failures can be reconstructed.

Data and Control Questions

- What is the authoritative source for every input used by this component?
- Which user, device, service, MSP, or administrator identity is allowed to initiate or approve it?
- Does the operation move value, reserve value, create a side effect, or only provide advisory information?
- What idempotency/replay key proves that a retry is the same business intent?
- What evidence is retained off-chain and what stable reference/hash belongs on the ledger?

Chapter 231 Implementation - Concurrent Double-Spend Tests

Implementation Procedure

1151. Define invariant-level expected outcomes and retain transaction IDs.
1152. Test normal, boundary, malformed, unauthorized, duplicate, and concurrent requests.
1153. Inject peer/orderer/database/AI/network failures.
1154. Reconcile ledger, projections, model decisions, approvals, and visible API outcome.
1155. Run backup/restore, certificate rotation, and production rehearsal before go-live.

Implementation Example

```
# Concurrent Double-Spend Tests
INPUT  -> authenticate, validate schema, bind requestId/nonce
FACTS  -> read authoritative wallet/merchant/state data
AI     -> optional bounded recommendation with decisionId/version
POLICY -> enforce ownership, hard limits, approvals, risk fallback
LEDGER -> submit one idempotent deterministic transaction when value changes
EVENT  -> update projection/case/monitoring asynchronously
VERIFY -> discover committed result and retain correlationId + txId evidence
```

Verification Checklist

- The concurrency suite exists and is version-controlled.
- The feature works with all public Internet/WAN connectivity disabled.
- Authorization, malformed input, duplicate retry, and failure behavior have explicit tests.
- If financial value changes, journal debits and credits balance exactly and historical entries remain immutable.
- If AI is involved, model/prompt/feature version and fallback behavior are recorded and deterministic policy remains final.
- A timeout/restart cannot create a duplicate financial result; request status can be rediscovered.
- Logs/evidence contain correlation references but not secrets or prohibited sensitive payment data.

Common Failure Modes

- Only happy-path demos.
- No concurrent spend/idempotency tests.
- Model tested separately but not in workflow.
- Backup exists but restore never proven.

Expert Review Questions

- What happens if this component succeeds but the next component fails?
- How is the authorizing identity/device/key revoked or recovered?
- Can the operation be safely retried after the client loses the response?
- What happens if AI is unavailable, slow, or wrong?
- Can an auditor reconstruct debit, credit, actor, risk decision, approval, and Fabric txId from retained evidence?
- Which compensating transaction resolves a downstream operational error without rewriting history?

Chapter 232 - Idempotency Tests

Repeat the same request after success, timeout, reconnect, and API restart. Financial correctness must be tested as invariants, not only API responses. Test balanced journals, conservation of value, authorization, idempotency, concurrent spend, quorum loss, model failure, projection rebuild, restore, and human-approval workflows with traceable evidence.

Design rule: Same requestId returns original outcome

Implementation Objective

The required output is: Duplicate-request suite. It must be version-controlled, reviewable, testable with the WAN disabled, and linked to the architecture/source-of-truth model. For a payment platform, every implementation decision should make it possible to explain who authorized an action, which facts were trusted, which controls ran, what financial journal committed, and how the result is recovered after failure.

Key Engineering Decisions

- Same requestId returns original outcome
- no duplicate journal/payment/refund
- status recovery works

Deep Implementation Guidance

A test passes only when financial state, journal balance, transaction validity, events, projection state, AI/approval evidence, and visible API outcome all match expectations. Retain requestId/decisionId/approvalId/txId so failures can be reconstructed.

Data and Control Questions

- What is the authoritative source for every input used by this component?
- Which user, device, service, MSP, or administrator identity is allowed to initiate or approve it?
- Does the operation move value, reserve value, create a side effect, or only provide advisory information?
- What idempotency/replay key proves that a retry is the same business intent?
- What evidence is retained off-chain and what stable reference/hash belongs on the ledger?

Chapter 232 Implementation - Idempotency Tests

Implementation Procedure

- 1156. Define invariant-level expected outcomes and retain transaction IDs.
- 1157. Test normal, boundary, malformed, unauthorized, duplicate, and concurrent requests.
- 1158. Inject peer/orderer/database/AI/network failures.
- 1159. Reconcile ledger, projections, model decisions, approvals, and visible API outcome.
- 1160. Run backup/restore, certificate rotation, and production rehearsal before go-live.

Implementation Example

```
# Idempotency Tests
INPUT  -> authenticate, validate schema, bind requestId/nonce
FACTS  -> read authoritative wallet/merchant/state data
AI     -> optional bounded recommendation with decisionId/version
POLICY -> enforce ownership, hard limits, approvals, risk fallback
LEDGER -> submit one idempotent deterministic transaction when value changes
EVENT  -> update projection/case/monitoring asynchronously
VERIFY -> discover committed result and retain correlationId + txId evidence
```

Verification Checklist

- The duplicate-request suite exists and is version-controlled.
- The feature works with all public Internet/WAN connectivity disabled.
- Authorization, malformed input, duplicate retry, and failure behavior have explicit tests.
- If financial value changes, journal debits and credits balance exactly and historical entries remain immutable.
- If AI is involved, model/prompt/feature version and fallback behavior are recorded and deterministic policy remains final.
- A timeout/restart cannot create a duplicate financial result; request status can be rediscovered.
- Logs/evidence contain correlation references but not secrets or prohibited sensitive payment data.

Common Failure Modes

- Only happy-path demos.
- No concurrent spend/idempotency tests.
- Model tested separately but not in workflow.
- Backup exists but restore never proven.

Expert Review Questions

- What happens if this component succeeds but the next component fails?
- How is the authorizing identity/device/key revoked or recovered?
- Can the operation be safely retried after the client loses the response?
- What happens if AI is unavailable, slow, or wrong?
- Can an auditor reconstruct debit, credit, actor, risk decision, approval, and Fabric txId from retained evidence?
- Which compensating transaction resolves a downstream operational error without rewriting history?

Chapter 233 - Authorization Tests

Prove users/devices/services/orgs cannot cross role/wallet/merchant boundaries. Financial correctness must be tested as invariants, not only API responses. Test balanced journals, conservation of value, authorization, idempotency, concurrent spend, quorum loss, model failure, projection rebuild, restore, and human-approval workflows with traceable evidence.

Design rule: Customer cannot issue

Implementation Objective

The required output is: Authorization matrix test. It must be version-controlled, reviewable, testable with the WAN disabled, and linked to the architecture/source-of-truth model. For a payment platform, every implementation decision should make it possible to explain who authorized an action, which facts were trusted, which controls ran, what financial journal committed, and how the result is recovered after failure.

Key Engineering Decisions

- Customer cannot issue
- merchant cannot spend customer wallet
- auditor read-only
- revoked device denied

Deep Implementation Guidance

A test passes only when financial state, journal balance, transaction validity, events, projection state, AI/approval evidence, and visible API outcome all match expectations. Retain requestId/decisionId/approvalId/txId so failures can be reconstructed.

Data and Control Questions

- What is the authoritative source for every input used by this component?
- Which user, device, service, MSP, or administrator identity is allowed to initiate or approve it?
- Does the operation move value, reserve value, create a side effect, or only provide advisory information?
- What idempotency/replay key proves that a retry is the same business intent?
- What evidence is retained off-chain and what stable reference/hash belongs on the ledger?

Chapter 233 Implementation - Authorization Tests

Implementation Procedure

- 1161. Define invariant-level expected outcomes and retain transaction IDs.
- 1162. Test normal, boundary, malformed, unauthorized, duplicate, and concurrent requests.
- 1163. Inject peer/orderer/database/AI/network failures.
- 1164. Reconcile ledger, projections, model decisions, approvals, and visible API outcome.
- 1165. Run backup/restore, certificate rotation, and production rehearsal before go-live.

Implementation Example

```
# Authorization Tests
INPUT  -> authenticate, validate schema, bind requestId/nonce
FACTS  -> read authoritative wallet/merchant/state data
AI     -> optional bounded recommendation with decisionId/version
POLICY -> enforce ownership, hard limits, approvals, risk fallback
LEDGER -> submit one idempotent deterministic transaction when value changes
EVENT  -> update projection/case/monitoring asynchronously
VERIFY -> discover committed result and retain correlationId + txId evidence
```

Verification Checklist

- The authorization matrix test exists and is version-controlled.
- The feature works with all public Internet/WAN connectivity disabled.
- Authorization, malformed input, duplicate retry, and failure behavior have explicit tests.
- If financial value changes, journal debits and credits balance exactly and historical entries remain immutable.
- If AI is involved, model/prompt/feature version and fallback behavior are recorded and deterministic policy remains final.
- A timeout/restart cannot create a duplicate financial result; request status can be rediscovered.
- Logs/evidence contain correlation references but not secrets or prohibited sensitive payment data.

Common Failure Modes

- Only happy-path demos.
- No concurrent spend/idempotency tests.
- Model tested separately but not in workflow.
- Backup exists but restore never proven.

Expert Review Questions

- What happens if this component succeeds but the next component fails?
- How is the authorizing identity/device/key revoked or recovered?
- Can the operation be safely retried after the client loses the response?
- What happens if AI is unavailable, slow, or wrong?
- Can an auditor reconstruct debit, credit, actor, risk decision, approval, and Fabric txId from retained evidence?
- Which compensating transaction resolves a downstream operational error without rewriting history?

Chapter 234 - Raft Quorum and Orderer Fault Tests

Verify expected behavior when one or more orderers stop. Financial correctness must be tested as invariants, not only API responses. Test balanced journals, conservation of value, authorization, idempotency, concurrent spend, quorum loss, model failure, projection rebuild, restore, and human-approval workflows with traceable evidence.

Design rule: One of three down continues

Implementation Objective

The required output is: Ordering fault suite. It must be version-controlled, reviewable, testable with the WAN disabled, and linked to the architecture/source-of-truth model. For a payment platform, every implementation decision should make it possible to explain who authorized an action, which facts were trusted, which controls ran, what financial journal committed, and how the result is recovered after failure.

Key Engineering Decisions

- One of three down continues
- two down stops new ordering
- recovery catches up
- no duplicate transactions

Deep Implementation Guidance

A test passes only when financial state, journal balance, transaction validity, events, projection state, AI/approval evidence, and visible API outcome all match expectations. Retain requestId/decisionId/approvalId/txId so failures can be reconstructed.

Data and Control Questions

- What is the authoritative source for every input used by this component?
- Which user, device, service, MSP, or administrator identity is allowed to initiate or approve it?
- Does the operation move value, reserve value, create a side effect, or only provide advisory information?
- What idempotency/replay key proves that a retry is the same business intent?
- What evidence is retained off-chain and what stable reference/hash belongs on the ledger?

Chapter 234 Implementation - Raft Quorum and Orderer Fault Tests

Implementation Procedure

- 1166. Define invariant-level expected outcomes and retain transaction IDs.
- 1167. Test normal, boundary, malformed, unauthorized, duplicate, and concurrent requests.
- 1168. Inject peer/orderer/database/AI/network failures.
- 1169. Reconcile ledger, projections, model decisions, approvals, and visible API outcome.
- 1170. Run backup/restore, certificate rotation, and production rehearsal before go-live.

Implementation Example

```
# Raft Quorum and Orderer Fault Tests
INPUT  -> authenticate, validate schema, bind requestId/nonce
FACTS  -> read authoritative wallet/merchant/state data
AI     -> optional bounded recommendation with decisionId/version
POLICY -> enforce ownership, hard limits, approvals, risk fallback
LEDGER -> submit one idempotent deterministic transaction when value changes
EVENT  -> update projection/case/monitoring asynchronously
VERIFY -> discover committed result and retain correlationId + txId evidence
```

Verification Checklist

- The ordering fault suite exists and is version-controlled.
- The feature works with all public Internet/WAN connectivity disabled.
- Authorization, malformed input, duplicate retry, and failure behavior have explicit tests.
- If financial value changes, journal debits and credits balance exactly and historical entries remain immutable.
- If AI is involved, model/prompt/feature version and fallback behavior are recorded and deterministic policy remains final.
- A timeout/restart cannot create a duplicate financial result; request status can be rediscovered.
- Logs/evidence contain correlation references but not secrets or prohibited sensitive payment data.

Common Failure Modes

- Only happy-path demos.
- No concurrent spend/idempotency tests.
- Model tested separately but not in workflow.
- Backup exists but restore never proven.

Expert Review Questions

- What happens if this component succeeds but the next component fails?
- How is the authorizing identity/device/key revoked or recovered?
- Can the operation be safely retried after the client loses the response?
- What happens if AI is unavailable, slow, or wrong?
- Can an auditor reconstruct debit, credit, actor, risk decision, approval, and Fabric txId from retained evidence?
- Which compensating transaction resolves a downstream operational error without rewriting history?

Chapter 235 - Peer and CouchDB Fault Tests

Verify reads/writes/recovery under peer or state DB failure. Financial correctness must be tested as invariants, not only API responses. Test balanced journals, conservation of value, authorization, idempotency, concurrent spend, quorum loss, model failure, projection rebuild, restore, and human-approval workflows with traceable evidence.

Design rule: Gateway failover strategy known

Implementation Objective

The required output is: Peer fault suite. It must be version-controlled, reviewable, testable with the WAN disabled, and linked to the architecture/source-of-truth model. For a payment platform, every implementation decision should make it possible to explain who authorized an action, which facts were trusted, which controls ran, what financial journal committed, and how the result is recovered after failure.

Key Engineering Decisions

- Gateway failover strategy known
- peer catches up
- ledger integrity preserved
- operations report degradation

Deep Implementation Guidance

A test passes only when financial state, journal balance, transaction validity, events, projection state, AI/approval evidence, and visible API outcome all match expectations. Retain requestId/decisionId/approvalId/txId so failures can be reconstructed.

Data and Control Questions

- What is the authoritative source for every input used by this component?
- Which user, device, service, MSP, or administrator identity is allowed to initiate or approve it?
- Does the operation move value, reserve value, create a side effect, or only provide advisory information?
- What idempotency/replay key proves that a retry is the same business intent?
- What evidence is retained off-chain and what stable reference/hash belongs on the ledger?

Chapter 235 Implementation - Peer and CouchDB Fault Tests

Implementation Procedure

- 1171. Define invariant-level expected outcomes and retain transaction IDs.
- 1172. Test normal, boundary, malformed, unauthorized, duplicate, and concurrent requests.
- 1173. Inject peer/orderer/database/AI/network failures.
- 1174. Reconcile ledger, projections, model decisions, approvals, and visible API outcome.
- 1175. Run backup/restore, certificate rotation, and production rehearsal before go-live.

Implementation Example

```
# Peer and CouchDB Fault Tests
INPUT  -> authenticate, validate schema, bind requestId/nonce
FACTS  -> read authoritative wallet/merchant/state data
AI      -> optional bounded recommendation with decisionId/version
POLICY  -> enforce ownership, hard limits, approvals, risk fallback
LEDGER  -> submit one idempotent deterministic transaction when value changes
EVENT   -> update projection/case/monitoring asynchronously
VERIFY  -> discover committed result and retain correlationId + txId evidence
```

Verification Checklist

- The peer fault suite exists and is version-controlled.
- The feature works with all public Internet/WAN connectivity disabled.
- Authorization, malformed input, duplicate retry, and failure behavior have explicit tests.
- If financial value changes, journal debits and credits balance exactly and historical entries remain immutable.
- If AI is involved, model/prompt/feature version and fallback behavior are recorded and deterministic policy remains final.
- A timeout/restart cannot create a duplicate financial result; request status can be rediscovered.
- Logs/evidence contain correlation references but not secrets or prohibited sensitive payment data.

Common Failure Modes

- Only happy-path demos.
- No concurrent spend/idempotency tests.
- Model tested separately but not in workflow.
- Backup exists but restore never proven.

Expert Review Questions

- What happens if this component succeeds but the next component fails?
- How is the authorizing identity/device/key revoked or recovered?
- Can the operation be safely retried after the client loses the response?
- What happens if AI is unavailable, slow, or wrong?
- Can an auditor reconstruct debit, credit, actor, risk decision, approval, and Fabric txId from retained evidence?
- Which compensating transaction resolves a downstream operational error without rewriting history?

Chapter 236 - PostgreSQL Projection Failure Tests

Prove financial commits remain safe if read-model update fails. Financial correctness must be tested as invariants, not only API responses. Test balanced journals, conservation of value, authorization, idempotency, concurrent spend, quorum loss, model failure, projection rebuild, restore, and human-approval workflows with traceable evidence.

Design rule: Ledger commits

Implementation Objective

The required output is: Projection fault suite. It must be version-controlled, reviewable, testable with the WAN disabled, and linked to the architecture/source-of-truth model. For a payment platform, every implementation decision should make it possible to explain who authorized an action, which facts were trusted, which controls ran, what financial journal committed, and how the result is recovered after failure.

Key Engineering Decisions

- Ledger commits
- event worker retries
- checkpoint resumes
- projection can rebuild
- UI indicates freshness/degradation

Deep Implementation Guidance

A test passes only when financial state, journal balance, transaction validity, events, projection state, AI/approval evidence, and visible API outcome all match expectations. Retain requestId/decisionId/approvalId/txId so failures can be reconstructed.

Data and Control Questions

- What is the authoritative source for every input used by this component?
- Which user, device, service, MSP, or administrator identity is allowed to initiate or approve it?
- Does the operation move value, reserve value, create a side effect, or only provide advisory information?
- What idempotency/replay key proves that a retry is the same business intent?
- What evidence is retained off-chain and what stable reference/hash belongs on the ledger?

Chapter 236 Implementation - PostgreSQL Projection Failure Tests

Implementation Procedure

1176. Define invariant-level expected outcomes and retain transaction IDs.
1177. Test normal, boundary, malformed, unauthorized, duplicate, and concurrent requests.
1178. Inject peer/orderer/database/AI/network failures.
1179. Reconcile ledger, projections, model decisions, approvals, and visible API outcome.
1180. Run backup/restore, certificate rotation, and production rehearsal before go-live.

Implementation Example

```
# PostgreSQL Projection Failure Tests
INPUT  -> authenticate, validate schema, bind requestId/nonce
FACTS  -> read authoritative wallet/merchant/state data
AI     -> optional bounded recommendation with decisionId/version
POLICY -> enforce ownership, hard limits, approvals, risk fallback
LEDGER -> submit one idempotent deterministic transaction when value changes
EVENT  -> update projection/case/monitoring asynchronously
VERIFY -> discover committed result and retain correlationId + txId evidence
```

Verification Checklist

- The projection fault suite exists and is version-controlled.
- The feature works with all public Internet/WAN connectivity disabled.
- Authorization, malformed input, duplicate retry, and failure behavior have explicit tests.
- If financial value changes, journal debits and credits balance exactly and historical entries remain immutable.
- If AI is involved, model/prompt/feature version and fallback behavior are recorded and deterministic policy remains final.
- A timeout/restart cannot create a duplicate financial result; request status can be rediscovered.
- Logs/evidence contain correlation references but not secrets or prohibited sensitive payment data.

Common Failure Modes

- Only happy-path demos.
- No concurrent spend/idempotency tests.
- Model tested separately but not in workflow.
- Backup exists but restore never proven.

Expert Review Questions

- What happens if this component succeeds but the next component fails?
- How is the authorizing identity/device/key revoked or recovered?
- Can the operation be safely retried after the client loses the response?
- What happens if AI is unavailable, slow, or wrong?
- Can an auditor reconstruct debit, credit, actor, risk decision, approval, and Fabric txId from retained evidence?
- Which compensating transaction resolves a downstream operational error without rewriting history?

Chapter 237 - AI Service Failure Tests

Prove local risk/LLM outages follow explicit fallback policy. Financial correctness must be tested as invariants, not only API responses. Test balanced journals, conservation of value, authorization, idempotency, concurrent spend, quorum loss, model failure, projection rebuild, restore, and human-approval workflows with traceable evidence.

Design rule: Risk timeout routes to rules/review according to policy

Implementation Objective

The required output is: AI fault suite. It must be version-controlled, reviewable, testable with the WAN disabled, and linked to the architecture/source-of-truth model. For a payment platform, every implementation decision should make it possible to explain who authorized an action, which facts were trusted, which controls ran, what financial journal committed, and how the result is recovered after failure.

Key Engineering Decisions

- Risk timeout routes to rules/review according to policy
- LLM outage does not stop core payments
- health alerts fire

Deep Implementation Guidance

A test passes only when financial state, journal balance, transaction validity, events, projection state, AI/approval evidence, and visible API outcome all match expectations. Retain requestId/decisionId/approvalId/txId so failures can be reconstructed.

Data and Control Questions

- What is the authoritative source for every input used by this component?
- Which user, device, service, MSP, or administrator identity is allowed to initiate or approve it?
- Does the operation move value, reserve value, create a side effect, or only provide advisory information?
- What idempotency/replay key proves that a retry is the same business intent?
- What evidence is retained off-chain and what stable reference/hash belongs on the ledger?

Chapter 237 Implementation - AI Service Failure Tests

Implementation Procedure

1181. Define invariant-level expected outcomes and retain transaction IDs.
1182. Test normal, boundary, malformed, unauthorized, duplicate, and concurrent requests.
1183. Inject peer/orderer/database/AI/network failures.
1184. Reconcile ledger, projections, model decisions, approvals, and visible API outcome.
1185. Run backup/restore, certificate rotation, and production rehearsal before go-live.

Implementation Example

```
# AI Service Failure Tests
INPUT  -> authenticate, validate schema, bind requestId/nonce
FACTS  -> read authoritative wallet/merchant/state data
AI      -> optional bounded recommendation with decisionId/version
POLICY -> enforce ownership, hard limits, approvals, risk fallback
LEDGER -> submit one idempotent deterministic transaction when value changes
EVENT  -> update projection/case/monitoring asynchronously
VERIFY -> discover committed result and retain correlationId + txId evidence
```

Verification Checklist

- The ai fault suite exists and is version-controlled.
- The feature works with all public Internet/WAN connectivity disabled.
- Authorization, malformed input, duplicate retry, and failure behavior have explicit tests.
- If financial value changes, journal debits and credits balance exactly and historical entries remain immutable.
- If AI is involved, model/prompt/feature version and fallback behavior are recorded and deterministic policy remains final.
- A timeout/restart cannot create a duplicate financial result; request status can be rediscovered.
- Logs/evidence contain correlation references but not secrets or prohibited sensitive payment data.

Common Failure Modes

- Only happy-path demos.
- No concurrent spend/idempotency tests.
- Model tested separately but not in workflow.
- Backup exists but restore never proven.

Expert Review Questions

- What happens if this component succeeds but the next component fails?
- How is the authorizing identity/device/key revoked or recovered?
- Can the operation be safely retried after the client loses the response?
- What happens if AI is unavailable, slow, or wrong?
- Can an auditor reconstruct debit, credit, actor, risk decision, approval, and Fabric txId from retained evidence?
- Which compensating transaction resolves a downstream operational error without rewriting history?

Chapter 238 - Fraud Model Offline Evaluation

Evaluate candidate models against frozen labeled datasets. Financial correctness must be tested as invariants, not only API responses. Test balanced journals, conservation of value, authorization, idempotency, concurrent spend, quorum loss, model failure, projection rebuild, restore, and human-approval workflows with traceable evidence.

Design rule: Precision recall PR-AUC calibration false-positive cost

Implementation Objective

The required output is: Fraud eval report. It must be version-controlled, reviewable, testable with the WAN disabled, and linked to the architecture/source-of-truth model. For a payment platform, every implementation decision should make it possible to explain who authorized an action, which facts were trusted, which controls ran, what financial journal committed, and how the result is recovered after failure.

Key Engineering Decisions

- Precision recall PR-AUC calibration false-positive cost
- segment/time slices
- compare to current model/rules

Deep Implementation Guidance

A test passes only when financial state, journal balance, transaction validity, events, projection state, AI/approval evidence, and visible API outcome all match expectations. Retain requestId/decisionId/approvalId/txId so failures can be reconstructed.

Data and Control Questions

- What is the authoritative source for every input used by this component?
- Which user, device, service, MSP, or administrator identity is allowed to initiate or approve it?
- Does the operation move value, reserve value, create a side effect, or only provide advisory information?
- What idempotency/replay key proves that a retry is the same business intent?
- What evidence is retained off-chain and what stable reference/hash belongs on the ledger?

Chapter 238 Implementation - Fraud Model Offline Evaluation

Implementation Procedure

- 1186. Define invariant-level expected outcomes and retain transaction IDs.
- 1187. Test normal, boundary, malformed, unauthorized, duplicate, and concurrent requests.
- 1188. Inject peer/orderer/database/AI/network failures.
- 1189. Reconcile ledger, projections, model decisions, approvals, and visible API outcome.
- 1190. Run backup/restore, certificate rotation, and production rehearsal before go-live.

Implementation Example

```
# Fraud Model Offline Evaluation
INPUT  -> authenticate, validate schema, bind requestId/nonce
FACTS  -> read authoritative wallet/merchant/state data
AI      -> optional bounded recommendation with decisionId/version
POLICY  -> enforce ownership, hard limits, approvals, risk fallback
LEDGER  -> submit one idempotent deterministic transaction when value changes
EVENT   -> update projection/case/monitoring asynchronously
VERIFY  -> discover committed result and retain correlationId + txId evidence
```

Verification Checklist

- The fraud eval report exists and is version-controlled.
- The feature works with all public Internet/WAN connectivity disabled.
- Authorization, malformed input, duplicate retry, and failure behavior have explicit tests.
- If financial value changes, journal debits and credits balance exactly and historical entries remain immutable.
- If AI is involved, model/prompt/feature version and fallback behavior are recorded and deterministic policy remains final.
- A timeout/restart cannot create a duplicate financial result; request status can be rediscovered.
- Logs/evidence contain correlation references but not secrets or prohibited sensitive payment data.

Common Failure Modes

- Only happy-path demos.
- No concurrent spend/idempotency tests.
- Model tested separately but not in workflow.
- Backup exists but restore never proven.

Expert Review Questions

- What happens if this component succeeds but the next component fails?
- How is the authorizing identity/device/key revoked or recovered?
- Can the operation be safely retried after the client loses the response?
- What happens if AI is unavailable, slow, or wrong?
- Can an auditor reconstruct debit, credit, actor, risk decision, approval, and Fabric txId from retained evidence?
- Which compensating transaction resolves a downstream operational error without rewriting history?

Chapter 239 - Shadow Mode Comparison

Compare model recommendation with real policy/analyst outcomes without impact. Financial correctness must be tested as invariants, not only API responses. Test balanced journals, conservation of value, authorization, idempotency, concurrent spend, quorum loss, model failure, projection rebuild, restore, and human-approval workflows with traceable evidence.

Design rule: Decision-volume by recommendation

Implementation Objective

The required output is: Shadow report. It must be version-controlled, reviewable, testable with the WAN disabled, and linked to the architecture/source-of-truth model. For a payment platform, every implementation decision should make it possible to explain who authorized an action, which facts were trusted, which controls ran, what financial journal committed, and how the result is recovered after failure.

Key Engineering Decisions

- Decision-volume by recommendation
- override/confirmed fraud metrics
- latency/error rate
- threshold proposal

Deep Implementation Guidance

A test passes only when financial state, journal balance, transaction validity, events, projection state, AI/approval evidence, and visible API outcome all match expectations. Retain requestId/decisionId/approvalId/txId so failures can be reconstructed.

Data and Control Questions

- What is the authoritative source for every input used by this component?
- Which user, device, service, MSP, or administrator identity is allowed to initiate or approve it?
- Does the operation move value, reserve value, create a side effect, or only provide advisory information?
- What idempotency/replay key proves that a retry is the same business intent?
- What evidence is retained off-chain and what stable reference/hash belongs on the ledger?

Chapter 239 Implementation - Shadow Mode Comparison

Implementation Procedure

1191. Define invariant-level expected outcomes and retain transaction IDs.
1192. Test normal, boundary, malformed, unauthorized, duplicate, and concurrent requests.
1193. Inject peer/orderer/database/AI/network failures.
1194. Reconcile ledger, projections, model decisions, approvals, and visible API outcome.
1195. Run backup/restore, certificate rotation, and production rehearsal before go-live.

Implementation Example

```
# Shadow Mode Comparison
INPUT  -> authenticate, validate schema, bind requestId/nonce
FACTS  -> read authoritative wallet/merchant/state data
AI      -> optional bounded recommendation with decisionId/version
POLICY  -> enforce ownership, hard limits, approvals, risk fallback
LEDGER  -> submit one idempotent deterministic transaction when value changes
EVENT   -> update projection/case/monitoring asynchronously
VERIFY  -> discover committed result and retain correlationId + txId evidence
```

Verification Checklist

- The shadow report exists and is version-controlled.
- The feature works with all public Internet/WAN connectivity disabled.
- Authorization, malformed input, duplicate retry, and failure behavior have explicit tests.
- If financial value changes, journal debits and credits balance exactly and historical entries remain immutable.
- If AI is involved, model/prompt/feature version and fallback behavior are recorded and deterministic policy remains final.
- A timeout/restart cannot create a duplicate financial result; request status can be rediscovered.
- Logs/evidence contain correlation references but not secrets or prohibited sensitive payment data.

Common Failure Modes

- Only happy-path demos.
- No concurrent spend/idempotency tests.
- Model tested separately but not in workflow.
- Backup exists but restore never proven.

Expert Review Questions

- What happens if this component succeeds but the next component fails?
- How is the authorizing identity/device/key revoked or recovered?
- Can the operation be safely retried after the client loses the response?
- What happens if AI is unavailable, slow, or wrong?
- Can an auditor reconstruct debit, credit, actor, risk decision, approval, and Fabric txId from retained evidence?
- Which compensating transaction resolves a downstream operational error without rewriting history?

Chapter 240 - LLM and RAG Evaluation

Measure local assistant evidence quality and unsafe behavior. Financial correctness must be tested as invariants, not only API responses. Test balanced journals, conservation of value, authorization, idempotency, concurrent spend, quorum loss, model failure, projection rebuild, restore, and human-approval workflows with traceable evidence.

Design rule: Retrieval recall

Implementation Objective

The required output is: LLM/RAG eval report. It must be version-controlled, reviewable, testable with the WAN disabled, and linked to the architecture/source-of-truth model. For a payment platform, every implementation decision should make it possible to explain who authorized an action, which facts were trusted, which controls ran, what financial journal committed, and how the result is recovered after failure.

Key Engineering Decisions

- Retrieval recall
- groundedness
- citation
- no-answer
- prompt injection
- tool misuse
- transaction-status hallucination

Deep Implementation Guidance

A test passes only when financial state, journal balance, transaction validity, events, projection state, AI/approval evidence, and visible API outcome all match expectations. Retain requestId/decisionId/approvalId/txId so failures can be reconstructed.

Data and Control Questions

- What is the authoritative source for every input used by this component?
- Which user, device, service, MSP, or administrator identity is allowed to initiate or approve it?
- Does the operation move value, reserve value, create a side effect, or only provide advisory information?
- What idempotency/replay key proves that a retry is the same business intent?
- What evidence is retained off-chain and what stable reference/hash belongs on the ledger?

Chapter 240 Implementation - LLM and RAG Evaluation

Implementation Procedure

1196. Define invariant-level expected outcomes and retain transaction IDs.
1197. Test normal, boundary, malformed, unauthorized, duplicate, and concurrent requests.
1198. Inject peer/orderer/database/AI/network failures.
1199. Reconcile ledger, projections, model decisions, approvals, and visible API outcome.
1200. Run backup/restore, certificate rotation, and production rehearsal before go-live.

Implementation Example

```
# LLM and RAG Evaluation
INPUT  -> authenticate, validate schema, bind requestId/nonce
FACTS  -> read authoritative wallet/merchant/state data
AI     -> optional bounded recommendation with decisionId/version
POLICY -> enforce ownership, hard limits, approvals, risk fallback
LEDGER -> submit one idempotent deterministic transaction when value changes
EVENT  -> update projection/case/monitoring asynchronously
VERIFY -> discover committed result and retain correlationId + txId evidence
```

Verification Checklist

- The llm/rag eval report exists and is version-controlled.
- The feature works with all public Internet/WAN connectivity disabled.
- Authorization, malformed input, duplicate retry, and failure behavior have explicit tests.
- If financial value changes, journal debits and credits balance exactly and historical entries remain immutable.
- If AI is involved, model/prompt/feature version and fallback behavior are recorded and deterministic policy remains final.
- A timeout/restart cannot create a duplicate financial result; request status can be rediscovered.
- Logs/evidence contain correlation references but not secrets or prohibited sensitive payment data.

Common Failure Modes

- Only happy-path demos.
- No concurrent spend/idempotency tests.
- Model tested separately but not in workflow.
- Backup exists but restore never proven.

Expert Review Questions

- What happens if this component succeeds but the next component fails?
- How is the authorizing identity/device/key revoked or recovered?
- Can the operation be safely retried after the client loses the response?
- What happens if AI is unavailable, slow, or wrong?
- Can an auditor reconstruct debit, credit, actor, risk decision, approval, and Fabric txId from retained evidence?
- Which compensating transaction resolves a downstream operational error without rewriting history?

Chapter 241 - Adversarial AI Tests

Attack model inputs, RAG documents, prompts, schemas, and tool boundaries. Financial correctness must be tested as invariants, not only API responses. Test balanced journals, conservation of value, authorization, idempotency, concurrent spend, quorum loss, model failure, projection rebuild, restore, and human-approval workflows with traceable evidence.

Design rule: Injection

Implementation Objective

The required output is: AI red-team suite. It must be version-controlled, reviewable, testable with the WAN disabled, and linked to the architecture/source-of-truth model. For a payment platform, every implementation decision should make it possible to explain who authorized an action, which facts were trusted, which controls ran, what financial journal committed, and how the result is recovered after failure.

Key Engineering Decisions

- Injection
- malformed JSON
- oversized input
- poisoned document
- secret request
- financial write attempt

Deep Implementation Guidance

A test passes only when financial state, journal balance, transaction validity, events, projection state, AI/approval evidence, and visible API outcome all match expectations. Retain requestId/decisionId/approvalId/txId so failures can be reconstructed.

Data and Control Questions

- What is the authoritative source for every input used by this component?
- Which user, device, service, MSP, or administrator identity is allowed to initiate or approve it?
- Does the operation move value, reserve value, create a side effect, or only provide advisory information?
- What idempotency/replay key proves that a retry is the same business intent?
- What evidence is retained off-chain and what stable reference/hash belongs on the ledger?

Chapter 241 Implementation - Adversarial AI Tests

Implementation Procedure

1201. Define invariant-level expected outcomes and retain transaction IDs.
1202. Test normal, boundary, malformed, unauthorized, duplicate, and concurrent requests.
1203. Inject peer/orderer/database/AI/network failures.
1204. Reconcile ledger, projections, model decisions, approvals, and visible API outcome.
1205. Run backup/restore, certificate rotation, and production rehearsal before go-live.

Implementation Example

```
# Adversarial AI Tests
INPUT  -> authenticate, validate schema, bind requestId/nonce
FACTS  -> read authoritative wallet/merchant/state data
AI     -> optional bounded recommendation with decisionId/version
POLICY -> enforce ownership, hard limits, approvals, risk fallback
LEDGER -> submit one idempotent deterministic transaction when value changes
EVENT  -> update projection/case/monitoring asynchronously
VERIFY -> discover committed result and retain correlationId + txId evidence
```

Verification Checklist

- The ai red-team suite exists and is version-controlled.
- The feature works with all public Internet/WAN connectivity disabled.
- Authorization, malformed input, duplicate retry, and failure behavior have explicit tests.
- If financial value changes, journal debits and credits balance exactly and historical entries remain immutable.
- If AI is involved, model/prompt/feature version and fallback behavior are recorded and deterministic policy remains final.
- A timeout/restart cannot create a duplicate financial result; request status can be rediscovered.
- Logs/evidence contain correlation references but not secrets or prohibited sensitive payment data.

Common Failure Modes

- Only happy-path demos.
- No concurrent spend/idempotency tests.
- Model tested separately but not in workflow.
- Backup exists but restore never proven.

Expert Review Questions

- What happens if this component succeeds but the next component fails?
- How is the authorizing identity/device/key revoked or recovered?
- Can the operation be safely retried after the client loses the response?
- What happens if AI is unavailable, slow, or wrong?
- Can an auditor reconstruct debit, credit, actor, risk decision, approval, and Fabric txId from retained evidence?
- Which compensating transaction resolves a downstream operational error without rewriting history?

Chapter 242 - REST API Contract Tests

Validate schemas, errors, idempotency, auth, timeouts, and status recovery. Financial correctness must be tested as invariants, not only API responses. Test balanced journals, conservation of value, authorization, idempotency, concurrent spend, quorum loss, model failure, projection rebuild, restore, and human-approval workflows with traceable evidence.

Design rule: Malformed/unknown fields

Implementation Objective

The required output is: API contract suite. It must be version-controlled, reviewable, testable with the WAN disabled, and linked to the architecture/source-of-truth model. For a payment platform, every implementation decision should make it possible to explain who authorized an action, which facts were trusted, which controls ran, what financial journal committed, and how the result is recovered after failure.

Key Engineering Decisions

- Malformed/unknown fields
- stable status codes
- correlation ID
- no secret leakage
- pending vs rejected semantics

Deep Implementation Guidance

A test passes only when financial state, journal balance, transaction validity, events, projection state, AI/approval evidence, and visible API outcome all match expectations. Retain requestId/decisionId/approvalId/txId so failures can be reconstructed.

Data and Control Questions

- What is the authoritative source for every input used by this component?
- Which user, device, service, MSP, or administrator identity is allowed to initiate or approve it?
- Does the operation move value, reserve value, create a side effect, or only provide advisory information?
- What idempotency/replay key proves that a retry is the same business intent?
- What evidence is retained off-chain and what stable reference/hash belongs on the ledger?

Chapter 242 Implementation - REST API Contract Tests

Implementation Procedure

1206. Define invariant-level expected outcomes and retain transaction IDs.
1207. Test normal, boundary, malformed, unauthorized, duplicate, and concurrent requests.
1208. Inject peer/orderer/database/AI/network failures.
1209. Reconcile ledger, projections, model decisions, approvals, and visible API outcome.
1210. Run backup/restore, certificate rotation, and production rehearsal before go-live.

Implementation Example

```
# REST API Contract Tests
INPUT  -> authenticate, validate schema, bind requestId/nonce
FACTS  -> read authoritative wallet/merchant/state data
AI     -> optional bounded recommendation with decisionId/version
POLICY -> enforce ownership, hard limits, approvals, risk fallback
LEDGER -> submit one idempotent deterministic transaction when value changes
EVENT  -> update projection/case/monitoring asynchronously
VERIFY -> discover committed result and retain correlationId + txId evidence
```

Verification Checklist

- The api contract suite exists and is version-controlled.
- The feature works with all public Internet/WAN connectivity disabled.
- Authorization, malformed input, duplicate retry, and failure behavior have explicit tests.
- If financial value changes, journal debits and credits balance exactly and historical entries remain immutable.
- If AI is involved, model/prompt/feature version and fallback behavior are recorded and deterministic policy remains final.
- A timeout/restart cannot create a duplicate financial result; request status can be rediscovered.
- Logs/evidence contain correlation references but not secrets or prohibited sensitive payment data.

Common Failure Modes

- Only happy-path demos.
- No concurrent spend/idempotency tests.
- Model tested separately but not in workflow.
- Backup exists but restore never proven.

Expert Review Questions

- What happens if this component succeeds but the next component fails?
- How is the authorizing identity/device/key revoked or recovered?
- Can the operation be safely retried after the client loses the response?
- What happens if AI is unavailable, slow, or wrong?
- Can an auditor reconstruct debit, credit, actor, risk decision, approval, and Fabric txId from retained evidence?
- Which compensating transaction resolves a downstream operational error without rewriting history?

Chapter 243 - End-to-End Payment Tests

Exercise UI-to-API-to-risk-to-Fabric-to-projection-to-receipt flows. Financial correctness must be tested as invariants, not only API responses. Test balanced journals, conservation of value, authorization, idempotency, concurrent spend, quorum loss, model failure, projection rebuild, restore, and human-approval workflows with traceable evidence.

Design rule: Customer P2P

Implementation Objective

The required output is: E2E suite. It must be version-controlled, reviewable, testable with the WAN disabled, and linked to the architecture/source-of-truth model. For a payment platform, every implementation decision should make it possible to explain who authorized an action, which facts were trusted, which controls ran, what financial journal committed, and how the result is recovered after failure.

Key Engineering Decisions

- Customer P2P
- merchant pay
- hold/capture/release
- refund
- freeze
- AI review
- recovery after timeout

Deep Implementation Guidance

A test passes only when financial state, journal balance, transaction validity, events, projection state, AI/approval evidence, and visible API outcome all match expectations. Retain requestId/decisionId/approvalId/txId so failures can be reconstructed.

Data and Control Questions

- What is the authoritative source for every input used by this component?
- Which user, device, service, MSP, or administrator identity is allowed to initiate or approve it?
- Does the operation move value, reserve value, create a side effect, or only provide advisory information?
- What idempotency/replay key proves that a retry is the same business intent?
- What evidence is retained off-chain and what stable reference/hash belongs on the ledger?

Chapter 243 Implementation - End-to-End Payment Tests

Implementation Procedure

1211. Define invariant-level expected outcomes and retain transaction IDs.
1212. Test normal, boundary, malformed, unauthorized, duplicate, and concurrent requests.
1213. Inject peer/orderer/database/AI/network failures.
1214. Reconcile ledger, projections, model decisions, approvals, and visible API outcome.
1215. Run backup/restore, certificate rotation, and production rehearsal before go-live.

Implementation Example

```
# End-to-End Payment Tests
INPUT  -> authenticate, validate schema, bind requestId/nonce
FACTS  -> read authoritative wallet/merchant/state data
AI     -> optional bounded recommendation with decisionId/version
POLICY -> enforce ownership, hard limits, approvals, risk fallback
LEDGER -> submit one idempotent deterministic transaction when value changes
EVENT  -> update projection/case/monitoring asynchronously
VERIFY -> discover committed result and retain correlationId + txId evidence
```

Verification Checklist

- The e2e suite exists and is version-controlled.
- The feature works with all public Internet/WAN connectivity disabled.
- Authorization, malformed input, duplicate retry, and failure behavior have explicit tests.
- If financial value changes, journal debits and credits balance exactly and historical entries remain immutable.
- If AI is involved, model/prompt/feature version and fallback behavior are recorded and deterministic policy remains final.
- A timeout/restart cannot create a duplicate financial result; request status can be rediscovered.
- Logs/evidence contain correlation references but not secrets or prohibited sensitive payment data.

Common Failure Modes

- Only happy-path demos.
- No concurrent spend/idempotency tests.
- Model tested separately but not in workflow.
- Backup exists but restore never proven.

Expert Review Questions

- What happens if this component succeeds but the next component fails?
- How is the authorizing identity/device/key revoked or recovered?
- Can the operation be safely retried after the client loses the response?
- What happens if AI is unavailable, slow, or wrong?
- Can an auditor reconstruct debit, credit, actor, risk decision, approval, and Fabric txId from retained evidence?
- Which compensating transaction resolves a downstream operational error without rewriting history?

Chapter 244 - Reconciliation Tests

Recompute expected balances and compare ledger/projection/report state. Financial correctness must be tested as invariants, not only API responses. Test balanced journals, conservation of value, authorization, idempotency, concurrent spend, quorum loss, model failure, projection rebuild, restore, and human-approval workflows with traceable evidence.

Design rule: All journal lines balance

Implementation Objective

The required output is: Reconciliation suite. It must be version-controlled, reviewable, testable with the WAN disabled, and linked to the architecture/source-of-truth model. For a payment platform, every implementation decision should make it possible to explain who authorized an action, which facts were trusted, which controls ran, what financial journal committed, and how the result is recovered after failure.

Key Engineering Decisions

- All journal lines balance
- wallet totals reconcile
- settlement batches sum
- exceptions visible and unresolved differences fail test

Deep Implementation Guidance

A test passes only when financial state, journal balance, transaction validity, events, projection state, AI/approval evidence, and visible API outcome all match expectations. Retain requestId/decisionId/approvalId/txId so failures can be reconstructed.

Data and Control Questions

- What is the authoritative source for every input used by this component?
- Which user, device, service, MSP, or administrator identity is allowed to initiate or approve it?
- Does the operation move value, reserve value, create a side effect, or only provide advisory information?
- What idempotency/replay key proves that a retry is the same business intent?
- What evidence is retained off-chain and what stable reference/hash belongs on the ledger?

Chapter 244 Implementation - Reconciliation Tests

Implementation Procedure

- 1216. Define invariant-level expected outcomes and retain transaction IDs.
- 1217. Test normal, boundary, malformed, unauthorized, duplicate, and concurrent requests.
- 1218. Inject peer/orderer/database/AI/network failures.
- 1219. Reconcile ledger, projections, model decisions, approvals, and visible API outcome.
- 1220. Run backup/restore, certificate rotation, and production rehearsal before go-live.

Implementation Example

```
# Reconciliation Tests
INPUT  -> authenticate, validate schema, bind requestId/nonce
FACTS  -> read authoritative wallet/merchant/state data
AI     -> optional bounded recommendation with decisionId/version
POLICY -> enforce ownership, hard limits, approvals, risk fallback
LEDGER -> submit one idempotent deterministic transaction when value changes
EVENT  -> update projection/case/monitoring asynchronously
VERIFY -> discover committed result and retain correlationId + txId evidence
```

Verification Checklist

- The reconciliation suite exists and is version-controlled.
- The feature works with all public Internet/WAN connectivity disabled.
- Authorization, malformed input, duplicate retry, and failure behavior have explicit tests.
- If financial value changes, journal debits and credits balance exactly and historical entries remain immutable.
- If AI is involved, model/prompt/feature version and fallback behavior are recorded and deterministic policy remains final.
- A timeout/restart cannot create a duplicate financial result; request status can be rediscovered.
- Logs/evidence contain correlation references but not secrets or prohibited sensitive payment data.

Common Failure Modes

- Only happy-path demos.
- No concurrent spend/idempotency tests.
- Model tested separately but not in workflow.
- Backup exists but restore never proven.

Expert Review Questions

- What happens if this component succeeds but the next component fails?
- How is the authorizing identity/device/key revoked or recovered?
- Can the operation be safely retried after the client loses the response?
- What happens if AI is unavailable, slow, or wrong?
- Can an auditor reconstruct debit, credit, actor, risk decision, approval, and Fabric txId from retained evidence?
- Which compensating transaction resolves a downstream operational error without rewriting history?

Chapter 245 - Backup and Restore Drill

Restore the full local platform on isolated hardware. Financial correctness must be tested as invariants, not only API responses. Test balanced journals, conservation of value, authorization, idempotency, concurrent spend, quorum loss, model failure, projection rebuild, restore, and human-approval workflows with traceable evidence.

Design rule: Restore Fabric/orderer/peer data according to supported procedure

Implementation Objective

The required output is: DR evidence pack. It must be version-controlled, reviewable, testable with the WAN disabled, and linked to the architecture/source-of-truth model. For a payment platform, every implementation decision should make it possible to explain who authorized an action, which facts were trusted, which controls ran, what financial journal committed, and how the result is recovered after failure.

Key Engineering Decisions

- Restore Fabric/orderer/peer data according to supported procedure
- restore DB/config/models
- reconcile
- prove new payment after recovery

Deep Implementation Guidance

A test passes only when financial state, journal balance, transaction validity, events, projection state, AI/approval evidence, and visible API outcome all match expectations. Retain requestId/decisionId/approvalId/txId so failures can be reconstructed.

Data and Control Questions

- What is the authoritative source for every input used by this component?
- Which user, device, service, MSP, or administrator identity is allowed to initiate or approve it?
- Does the operation move value, reserve value, create a side effect, or only provide advisory information?
- What idempotency/replay key proves that a retry is the same business intent?
- What evidence is retained off-chain and what stable reference/hash belongs on the ledger?

Chapter 245 Implementation - Backup and Restore Drill

Implementation Procedure

1221. Define invariant-level expected outcomes and retain transaction IDs.
1222. Test normal, boundary, malformed, unauthorized, duplicate, and concurrent requests.
1223. Inject peer/orderer/database/AI/network failures.
1224. Reconcile ledger, projections, model decisions, approvals, and visible API outcome.
1225. Run backup/restore, certificate rotation, and production rehearsal before go-live.

Implementation Example

```
# Backup and Restore Drill
INPUT  -> authenticate, validate schema, bind requestId/nonce
FACTS  -> read authoritative wallet/merchant/state data
AI      -> optional bounded recommendation with decisionId/version
POLICY  -> enforce ownership, hard limits, approvals, risk fallback
LEDGER  -> submit one idempotent deterministic transaction when value changes
EVENT   -> update projection/case/monitoring asynchronously
VERIFY  -> discover committed result and retain correlationId + txId evidence
```

Verification Checklist

- The dr evidence pack exists and is version-controlled.
- The feature works with all public Internet/WAN connectivity disabled.
- Authorization, malformed input, duplicate retry, and failure behavior have explicit tests.
- If financial value changes, journal debits and credits balance exactly and historical entries remain immutable.
- If AI is involved, model/prompt/feature version and fallback behavior are recorded and deterministic policy remains final.
- A timeout/restart cannot create a duplicate financial result; request status can be rediscovered.
- Logs/evidence contain correlation references but not secrets or prohibited sensitive payment data.

Common Failure Modes

- Only happy-path demos.
- No concurrent spend/idempotency tests.
- Model tested separately but not in workflow.
- Backup exists but restore never proven.

Expert Review Questions

- What happens if this component succeeds but the next component fails?
- How is the authorizing identity/device/key revoked or recovered?
- Can the operation be safely retried after the client loses the response?
- What happens if AI is unavailable, slow, or wrong?
- Can an auditor reconstruct debit, credit, actor, risk decision, approval, and Fabric txId from retained evidence?
- Which compensating transaction resolves a downstream operational error without rewriting history?

Chapter 246 - Certificate Rotation Drill

Renew/revoke identities without unexpected payment outage. Financial correctness must be tested as invariants, not only API responses. Test balanced journals, conservation of value, authorization, idempotency, concurrent spend, quorum loss, model failure, projection rebuild, restore, and human-approval workflows with traceable evidence.

Design rule: Rotate client/node certs

Implementation Objective

The required output is: Cert drill. It must be version-controlled, reviewable, testable with the WAN disabled, and linked to the architecture/source-of-truth model. For a payment platform, every implementation decision should make it possible to explain who authorized an action, which facts were trusted, which controls ran, what financial journal committed, and how the result is recovered after failure.

Key Engineering Decisions

- Rotate client/node certs
- update trust
- restart as required
- verify old revoked identity denied
- health checks pass

Deep Implementation Guidance

A test passes only when financial state, journal balance, transaction validity, events, projection state, AI/approval evidence, and visible API outcome all match expectations. Retain requestId/decisionId/approvalId/txId so failures can be reconstructed.

Data and Control Questions

- What is the authoritative source for every input used by this component?
- Which user, device, service, MSP, or administrator identity is allowed to initiate or approve it?
- Does the operation move value, reserve value, create a side effect, or only provide advisory information?
- What idempotency/replay key proves that a retry is the same business intent?
- What evidence is retained off-chain and what stable reference/hash belongs on the ledger?

Chapter 246 Implementation - Certificate Rotation Drill

Implementation Procedure

- 1226. Define invariant-level expected outcomes and retain transaction IDs.
- 1227. Test normal, boundary, malformed, unauthorized, duplicate, and concurrent requests.
- 1228. Inject peer/orderer/database/AI/network failures.
- 1229. Reconcile ledger, projections, model decisions, approvals, and visible API outcome.
- 1230. Run backup/restore, certificate rotation, and production rehearsal before go-live.

Implementation Example

```
# Certificate Rotation Drill
INPUT  -> authenticate, validate schema, bind requestId/nonce
FACTS  -> read authoritative wallet/merchant/state data
AI      -> optional bounded recommendation with decisionId/version
POLICY  -> enforce ownership, hard limits, approvals, risk fallback
LEDGER  -> submit one idempotent deterministic transaction when value changes
EVENT   -> update projection/case/monitoring asynchronously
VERIFY  -> discover committed result and retain correlationId + txId evidence
```

Verification Checklist

- The cert drill exists and is version-controlled.
- The feature works with all public Internet/WAN connectivity disabled.
- Authorization, malformed input, duplicate retry, and failure behavior have explicit tests.
- If financial value changes, journal debits and credits balance exactly and historical entries remain immutable.
- If AI is involved, model/prompt/feature version and fallback behavior are recorded and deterministic policy remains final.
- A timeout/restart cannot create a duplicate financial result; request status can be rediscovered.
- Logs/evidence contain correlation references but not secrets or prohibited sensitive payment data.

Common Failure Modes

- Only happy-path demos.
- No concurrent spend/idempotency tests.
- Model tested separately but not in workflow.
- Backup exists but restore never proven.

Expert Review Questions

- What happens if this component succeeds but the next component fails?
- How is the authorizing identity/device/key revoked or recovered?
- Can the operation be safely retried after the client loses the response?
- What happens if AI is unavailable, slow, or wrong?
- Can an auditor reconstruct debit, credit, actor, risk decision, approval, and Fabric txId from retained evidence?
- Which compensating transaction resolves a downstream operational error without rewriting history?

Chapter 247 - Model Rollback Drill

Return to previous approved fraud/LLM model quickly. Financial correctness must be tested as invariants, not only API responses. Test balanced journals, conservation of value, authorization, idempotency, concurrent spend, quorum loss, model failure, projection rebuild, restore, and human-approval workflows with traceable evidence.

Design rule: Switch registry/config

Implementation Objective

The required output is: AI rollback test. It must be version-controlled, reviewable, testable with the WAN disabled, and linked to the architecture/source-of-truth model. For a payment platform, every implementation decision should make it possible to explain who authorized an action, which facts were trusted, which controls ran, what financial journal committed, and how the result is recovered after failure.

Key Engineering Decisions

- Switch registry/config
- verify hash
- run smoke/eval
- record decision
- no ledger ambiguity

Deep Implementation Guidance

A test passes only when financial state, journal balance, transaction validity, events, projection state, AI/approval evidence, and visible API outcome all match expectations. Retain requestId/decisionId/approvalId/txId so failures can be reconstructed.

Data and Control Questions

- What is the authoritative source for every input used by this component?
- Which user, device, service, MSP, or administrator identity is allowed to initiate or approve it?
- Does the operation move value, reserve value, create a side effect, or only provide advisory information?
- What idempotency/replay key proves that a retry is the same business intent?
- What evidence is retained off-chain and what stable reference/hash belongs on the ledger?

Chapter 247 Implementation - Model Rollback Drill

Implementation Procedure

1231. Define invariant-level expected outcomes and retain transaction IDs.
1232. Test normal, boundary, malformed, unauthorized, duplicate, and concurrent requests.
1233. Inject peer/orderer/database/AI/network failures.
1234. Reconcile ledger, projections, model decisions, approvals, and visible API outcome.
1235. Run backup/restore, certificate rotation, and production rehearsal before go-live.

Implementation Example

```
# Model Rollback Drill
INPUT  -> authenticate, validate schema, bind requestId/nonce
FACTS  -> read authoritative wallet/merchant/state data
AI     -> optional bounded recommendation with decisionId/version
POLICY -> enforce ownership, hard limits, approvals, risk fallback
LEDGER -> submit one idempotent deterministic transaction when value changes
EVENT  -> update projection/case/monitoring asynchronously
VERIFY -> discover committed result and retain correlationId + txId evidence
```

Verification Checklist

- The ai rollback test exists and is version-controlled.
- The feature works with all public Internet/WAN connectivity disabled.
- Authorization, malformed input, duplicate retry, and failure behavior have explicit tests.
- If financial value changes, journal debits and credits balance exactly and historical entries remain immutable.
- If AI is involved, model/prompt/feature version and fallback behavior are recorded and deterministic policy remains final.
- A timeout/restart cannot create a duplicate financial result; request status can be rediscovered.
- Logs/evidence contain correlation references but not secrets or prohibited sensitive payment data.

Common Failure Modes

- Only happy-path demos.
- No concurrent spend/idempotency tests.
- Model tested separately but not in workflow.
- Backup exists but restore never proven.

Expert Review Questions

- What happens if this component succeeds but the next component fails?
- How is the authorizing identity/device/key revoked or recovered?
- Can the operation be safely retried after the client loses the response?
- What happens if AI is unavailable, slow, or wrong?
- Can an auditor reconstruct debit, credit, actor, risk decision, approval, and Fabric txId from retained evidence?
- Which compensating transaction resolves a downstream operational error without rewriting history?

Chapter 248 - Performance and Soak Tests

Measure realistic transaction and AI workloads over sustained periods. Financial correctness must be tested as invariants, not only API responses. Test balanced journals, conservation of value, authorization, idempotency, concurrent spend, quorum loss, model failure, projection rebuild, restore, and human-approval workflows with traceable evidence.

Design rule: p50/p95/p99 API and commit latency

Implementation Objective

The required output is: Performance report. It must be version-controlled, reviewable, testable with the WAN disabled, and linked to the architecture/source-of-truth model. For a payment platform, every implementation decision should make it possible to explain who authorized an action, which facts were trusted, which controls ran, what financial journal committed, and how the result is recovered after failure.

Key Engineering Decisions

- p50/p95/p99 API and commit latency
- throughput
- CPU/RAM/storage
- AI inference latency
- error/queue growth

Deep Implementation Guidance

A test passes only when financial state, journal balance, transaction validity, events, projection state, AI/approval evidence, and visible API outcome all match expectations. Retain requestId/decisionId/approvalId/txId so failures can be reconstructed.

Data and Control Questions

- What is the authoritative source for every input used by this component?
- Which user, device, service, MSP, or administrator identity is allowed to initiate or approve it?
- Does the operation move value, reserve value, create a side effect, or only provide advisory information?
- What idempotency/replay key proves that a retry is the same business intent?
- What evidence is retained off-chain and what stable reference/hash belongs on the ledger?

Chapter 248 Implementation - Performance and Soak Tests

Implementation Procedure

- 1236. Define invariant-level expected outcomes and retain transaction IDs.
- 1237. Test normal, boundary, malformed, unauthorized, duplicate, and concurrent requests.
- 1238. Inject peer/orderer/database/AI/network failures.
- 1239. Reconcile ledger, projections, model decisions, approvals, and visible API outcome.
- 1240. Run backup/restore, certificate rotation, and production rehearsal before go-live.

Implementation Example

```
# Performance and Soak Tests
INPUT  -> authenticate, validate schema, bind requestId/nonce
FACTS  -> read authoritative wallet/merchant/state data
AI      -> optional bounded recommendation with decisionId/version
POLICY  -> enforce ownership, hard limits, approvals, risk fallback
LEDGER  -> submit one idempotent deterministic transaction when value changes
EVENT   -> update projection/case/monitoring asynchronously
VERIFY  -> discover committed result and retain correlationId + txId evidence
```

Verification Checklist

- The performance report exists and is version-controlled.
- The feature works with all public Internet/WAN connectivity disabled.
- Authorization, malformed input, duplicate retry, and failure behavior have explicit tests.
- If financial value changes, journal debits and credits balance exactly and historical entries remain immutable.
- If AI is involved, model/prompt/feature version and fallback behavior are recorded and deterministic policy remains final.
- A timeout/restart cannot create a duplicate financial result; request status can be rediscovered.
- Logs/evidence contain correlation references but not secrets or prohibited sensitive payment data.

Common Failure Modes

- Only happy-path demos.
- No concurrent spend/idempotency tests.
- Model tested separately but not in workflow.
- Backup exists but restore never proven.

Expert Review Questions

- What happens if this component succeeds but the next component fails?
- How is the authorizing identity/device/key revoked or recovered?
- Can the operation be safely retried after the client loses the response?
- What happens if AI is unavailable, slow, or wrong?
- Can an auditor reconstruct debit, credit, actor, risk decision, approval, and Fabric txId from retained evidence?
- Which compensating transaction resolves a downstream operational error without rewriting history?

Chapter 249 - User Acceptance Tests

Validate business usability, reconciliation, support, and exception handling. Financial correctness must be tested as invariants, not only API responses. Test balanced journals, conservation of value, authorization, idempotency, concurrent spend, quorum loss, model failure, projection rebuild, restore, and human-approval workflows with traceable evidence.

Design rule: Customer/merchant/treasury/auditor scenarios

Implementation Objective

The required output is: UAT pack. It must be version-controlled, reviewable, testable with the WAN disabled, and linked to the architecture/source-of-truth model. For a payment platform, every implementation decision should make it possible to explain who authorized an action, which facts were trusted, which controls ran, what financial journal committed, and how the result is recovered after failure.

Key Engineering Decisions

- Customer/merchant/treasury/auditor scenarios
- clear pending/review states
- receipt/report verification
- operator sign-off

Deep Implementation Guidance

A test passes only when financial state, journal balance, transaction validity, events, projection state, AI/approval evidence, and visible API outcome all match expectations. Retain requestId/decisionId/approvalId/txId so failures can be reconstructed.

Data and Control Questions

- What is the authoritative source for every input used by this component?
- Which user, device, service, MSP, or administrator identity is allowed to initiate or approve it?
- Does the operation move value, reserve value, create a side effect, or only provide advisory information?
- What idempotency/replay key proves that a retry is the same business intent?
- What evidence is retained off-chain and what stable reference/hash belongs on the ledger?

Chapter 249 Implementation - User Acceptance Tests

Implementation Procedure

1241. Define invariant-level expected outcomes and retain transaction IDs.
1242. Test normal, boundary, malformed, unauthorized, duplicate, and concurrent requests.
1243. Inject peer/orderer/database/AI/network failures.
1244. Reconcile ledger, projections, model decisions, approvals, and visible API outcome.
1245. Run backup/restore, certificate rotation, and production rehearsal before go-live.

Implementation Example

```
# User Acceptance Tests
INPUT  -> authenticate, validate schema, bind requestId/nonce
FACTS  -> read authoritative wallet/merchant/state data
AI     -> optional bounded recommendation with decisionId/version
POLICY -> enforce ownership, hard limits, approvals, risk fallback
LEDGER -> submit one idempotent deterministic transaction when value changes
EVENT  -> update projection/case/monitoring asynchronously
VERIFY -> discover committed result and retain correlationId + txId evidence
```

Verification Checklist

- The uat pack exists and is version-controlled.
- The feature works with all public Internet/WAN connectivity disabled.
- Authorization, malformed input, duplicate retry, and failure behavior have explicit tests.
- If financial value changes, journal debits and credits balance exactly and historical entries remain immutable.
- If AI is involved, model/prompt/feature version and fallback behavior are recorded and deterministic policy remains final.
- A timeout/restart cannot create a duplicate financial result; request status can be rediscovered.
- Logs/evidence contain correlation references but not secrets or prohibited sensitive payment data.

Common Failure Modes

- Only happy-path demos.
- No concurrent spend/idempotency tests.
- Model tested separately but not in workflow.
- Backup exists but restore never proven.

Expert Review Questions

- What happens if this component succeeds but the next component fails?
- How is the authorizing identity/device/key revoked or recovered?
- Can the operation be safely retried after the client loses the response?
- What happens if AI is unavailable, slow, or wrong?
- Can an auditor reconstruct debit, credit, actor, risk decision, approval, and Fabric txId from retained evidence?
- Which compensating transaction resolves a downstream operational error without rewriting history?

Chapter 250 - Production Rehearsal

Run the planned production release/rollback/runbooks before go-live. Financial correctness must be tested as invariants, not only API responses. Test balanced journals, conservation of value, authorization, idempotency, concurrent spend, quorum loss, model failure, projection rebuild, restore, and human-approval workflows with traceable evidence.

Design rule: Clean deployment

Implementation Objective

The required output is: Rehearsal evidence. It must be version-controlled, reviewable, testable with the WAN disabled, and linked to the architecture/source-of-truth model. For a payment platform, every implementation decision should make it possible to explain who authorized an action, which facts were trusted, which controls ran, what financial journal committed, and how the result is recovered after failure.

Key Engineering Decisions

- Clean deployment
- offline bundle
- backup
- restore checkpoint
- failover
- incident simulation
- operator independently executes

Deep Implementation Guidance

A test passes only when financial state, journal balance, transaction validity, events, projection state, AI/approval evidence, and visible API outcome all match expectations. Retain requestId/decisionId/approvalId/txId so failures can be reconstructed.

Data and Control Questions

- What is the authoritative source for every input used by this component?
- Which user, device, service, MSP, or administrator identity is allowed to initiate or approve it?
- Does the operation move value, reserve value, create a side effect, or only provide advisory information?
- What idempotency/replay key proves that a retry is the same business intent?
- What evidence is retained off-chain and what stable reference/hash belongs on the ledger?

Chapter 250 Implementation - Production Rehearsal

Implementation Procedure

1246. Define invariant-level expected outcomes and retain transaction IDs.
1247. Test normal, boundary, malformed, unauthorized, duplicate, and concurrent requests.
1248. Inject peer/orderer/database/AI/network failures.
1249. Reconcile ledger, projections, model decisions, approvals, and visible API outcome.
1250. Run backup/restore, certificate rotation, and production rehearsal before go-live.

Implementation Example

```
# Production Rehearsal
INPUT  -> authenticate, validate schema, bind requestId/nonce
FACTS  -> read authoritative wallet/merchant/state data
AI      -> optional bounded recommendation with decisionId/version
POLICY  -> enforce ownership, hard limits, approvals, risk fallback
LEDGER  -> submit one idempotent deterministic transaction when value changes
EVENT   -> update projection/case/monitoring asynchronously
VERIFY  -> discover committed result and retain correlationId + txId evidence
```

Verification Checklist

- The rehearsal evidence exists and is version-controlled.
- The feature works with all public Internet/WAN connectivity disabled.
- Authorization, malformed input, duplicate retry, and failure behavior have explicit tests.
- If financial value changes, journal debits and credits balance exactly and historical entries remain immutable.
- If AI is involved, model/prompt/feature version and fallback behavior are recorded and deterministic policy remains final.
- A timeout/restart cannot create a duplicate financial result; request status can be rediscovered.
- Logs/evidence contain correlation references but not secrets or prohibited sensitive payment data.

Common Failure Modes

- Only happy-path demos.
- No concurrent spend/idempotency tests.
- Model tested separately but not in workflow.
- Backup exists but restore never proven.

Expert Review Questions

- What happens if this component succeeds but the next component fails?
- How is the authorizing identity/device/key revoked or recovered?
- Can the operation be safely retried after the client loses the response?
- What happens if AI is unavailable, slow, or wrong?
- Can an auditor reconstruct debit, credit, actor, risk decision, approval, and Fabric txId from retained evidence?
- Which compensating transaction resolves a downstream operational error without rewriting history?

Part XIV - Production Operations, Backup, Recovery, and Change Management

A production runbook must work when the original developer is absent and the Internet is unavailable. Operators need exact startup/shutdown, health, reconciliation, backup, restore, certificate rotation, model rollback, chaincode upgrade, incident, and offline patching procedures.

Part focus: This part contains 20 implementation chapters. Each chapter includes design rules, implementation procedure, a practical artifact, verification, failure analysis, and expert review questions.

Chapter 251 - Startup Procedure

Bring the local platform up in dependency order with verification. A production runbook must work when the original developer is absent and the Internet is unavailable. Operators need exact startup/shutdown, health, reconciliation, backup, restore, certificate rotation, model rollback, chaincode upgrade, incident, and offline patching procedures.

Design rule: Network/DNS/time

Implementation Objective

The required output is: Startup runbook. It must be version-controlled, reviewable, testable with the WAN disabled, and linked to the architecture/source-of-truth model. For a payment platform, every implementation decision should make it possible to explain who authorized an action, which facts were trusted, which controls ran, what financial journal committed, and how the result is recovered after failure.

Key Engineering Decisions

- Network/DNS/time
- orderers
- peers/CouchDB
- PostgreSQL
- API/projection
- AI
- UI
- health and test transaction

Deep Implementation Guidance

Runbooks must be executable by another trained operator. Include prechecks, exact commands, expected output, stop conditions, rollback, verification, and evidence to retain. Periodically drill restore, revocation, model rollback, and quorum loss.

Data and Control Questions

- What is the authoritative source for every input used by this component?
- Which user, device, service, MSP, or administrator identity is allowed to initiate or approve it?
- Does the operation move value, reserve value, create a side effect, or only provide advisory information?
- What idempotency/replay key proves that a retry is the same business intent?
- What evidence is retained off-chain and what stable reference/hash belongs on the ledger?

Chapter 251 Implementation - Startup Procedure

Implementation Procedure

1251. Create operator runbooks with prechecks, exact commands, expected output, rollback, and verification.
1252. Monitor quorum, ledger height, certificate expiry, reconciliation, API latency, model health, and storage.
1253. Back up and restore on a schedule; drill it.
1254. Promote models/chaincode/config with signed manifests and rollback points.
1255. Review incidents, overrides, fraud metrics, and access logs routinely.

Implementation Example

```
# Startup Procedure
INPUT  -> authenticate, validate schema, bind requestId/nonce
FACTS  -> read authoritative wallet/merchant/state data
AI      -> optional bounded recommendation with decisionId/version
POLICY -> enforce ownership, hard limits, approvals, risk fallback
LEDGER  -> submit one idempotent deterministic transaction when value changes
EVENT   -> update projection/case/monitoring asynchronously
VERIFY  -> discover committed result and retain correlationId + txId evidence
```

Verification Checklist

- The startup runbook exists and is version-controlled.
- The feature works with all public Internet/WAN connectivity disabled.
- Authorization, malformed input, duplicate retry, and failure behavior have explicit tests.
- If financial value changes, journal debits and credits balance exactly and historical entries remain immutable.
- If AI is involved, model/prompt/feature version and fallback behavior are recorded and deterministic policy remains final.
- A timeout/restart cannot create a duplicate financial result; request status can be rediscovered.
- Logs/evidence contain correlation references but not secrets or prohibited sensitive payment data.

Common Failure Modes

- Runbooks depend on one developer.
- Certificate/model/version inventory stale.
- Reconciliation alerts ignored.
- No rollback for model or chaincode deployment.

Expert Review Questions

- What happens if this component succeeds but the next component fails?
- How is the authorizing identity/device/key revoked or recovered?
- Can the operation be safely retried after the client loses the response?
- What happens if AI is unavailable, slow, or wrong?
- Can an auditor reconstruct debit, credit, actor, risk decision, approval, and Fabric txId from retained evidence?
- Which compensating transaction resolves a downstream operational error without rewriting history?

Chapter 252 - Shutdown Procedure

Stop services without corrupting data or losing operational evidence. A production runbook must work when the original developer is absent and the Internet is unavailable. Operators need exact startup/shutdown, health, reconciliation, backup, restore, certificate rotation, model rollback, chaincode upgrade, incident, and offline patching procedures.

Design rule: Quiesce writes

Implementation Objective

The required output is: Shutdown runbook. It must be version-controlled, reviewable, testable with the WAN disabled, and linked to the architecture/source-of-truth model. For a payment platform, every implementation decision should make it possible to explain who authorized an action, which facts were trusted, which controls ran, what financial journal committed, and how the result is recovered after failure.

Key Engineering Decisions

- Quiesce writes
- drain jobs
- record ledger/projection checkpoint
- backup if required
- stop dependent services in safe order

Deep Implementation Guidance

Runbooks must be executable by another trained operator. Include prechecks, exact commands, expected output, stop conditions, rollback, verification, and evidence to retain. Periodically drill restore, revocation, model rollback, and quorum loss.

Data and Control Questions

- What is the authoritative source for every input used by this component?
- Which user, device, service, MSP, or administrator identity is allowed to initiate or approve it?
- Does the operation move value, reserve value, create a side effect, or only provide advisory information?
- What idempotency/replay key proves that a retry is the same business intent?
- What evidence is retained off-chain and what stable reference/hash belongs on the ledger?

Chapter 252 Implementation - Shutdown Procedure

Implementation Procedure

1256. Create operator runbooks with prechecks, exact commands, expected output, rollback, and verification.
1257. Monitor quorum, ledger height, certificate expiry, reconciliation, API latency, model health, and storage.
1258. Back up and restore on a schedule; drill it.
1259. Promote models/chaincode/config with signed manifests and rollback points.
1260. Review incidents, overrides, fraud metrics, and access logs routinely.

Implementation Example

```
# Shutdown Procedure
INPUT  -> authenticate, validate schema, bind requestId/nonce
FACTS  -> read authoritative wallet/merchant/state data
AI      -> optional bounded recommendation with decisionId/version
POLICY  -> enforce ownership, hard limits, approvals, risk fallback
LEDGER  -> submit one idempotent deterministic transaction when value changes
EVENT   -> update projection/case/monitoring asynchronously
VERIFY  -> discover committed result and retain correlationId + txId evidence
```

Verification Checklist

- The shutdown runbook exists and is version-controlled.
- The feature works with all public Internet/WAN connectivity disabled.
- Authorization, malformed input, duplicate retry, and failure behavior have explicit tests.
- If financial value changes, journal debits and credits balance exactly and historical entries remain immutable.
- If AI is involved, model/prompt/feature version and fallback behavior are recorded and deterministic policy remains final.
- A timeout/restart cannot create a duplicate financial result; request status can be rediscovered.
- Logs/evidence contain correlation references but not secrets or prohibited sensitive payment data.

Common Failure Modes

- Runbooks depend on one developer.
- Certificate/model/version inventory stale.
- Reconciliation alerts ignored.
- No rollback for model or chaincode deployment.

Expert Review Questions

- What happens if this component succeeds but the next component fails?
- How is the authorizing identity/device/key revoked or recovered?
- Can the operation be safely retried after the client loses the response?
- What happens if AI is unavailable, slow, or wrong?
- Can an auditor reconstruct debit, credit, actor, risk decision, approval, and Fabric txId from retained evidence?
- Which compensating transaction resolves a downstream operational error without rewriting history?

Chapter 253 - Daily Health Checks

Give operators a short routine that catches common failures early. A production runbook must work when the original developer is absent and the Internet is unavailable. Operators need exact startup/shutdown, health, reconciliation, backup, restore, certificate rotation, model rollback, chaincode upgrade, incident, and offline patching procedures.

Design rule: Raft quorum

Implementation Objective

The required output is: Daily checklist. It must be version-controlled, reviewable, testable with the WAN disabled, and linked to the architecture/source-of-truth model. For a payment platform, every implementation decision should make it possible to explain who authorized an action, which facts were trusted, which controls ran, what financial journal committed, and how the result is recovered after failure.

Key Engineering Decisions

- Raft quorum
- peer heights
- certificate expiry
- DB replication/storage
- API latency
- AI model hash/health
- reconciliation exceptions

Deep Implementation Guidance

Runbooks must be executable by another trained operator. Include prechecks, exact commands, expected output, stop conditions, rollback, verification, and evidence to retain. Periodically drill restore, revocation, model rollback, and quorum loss.

Data and Control Questions

- What is the authoritative source for every input used by this component?
- Which user, device, service, MSP, or administrator identity is allowed to initiate or approve it?
- Does the operation move value, reserve value, create a side effect, or only provide advisory information?
- What idempotency/replay key proves that a retry is the same business intent?
- What evidence is retained off-chain and what stable reference/hash belongs on the ledger?

Chapter 253 Implementation - Daily Health Checks

Implementation Procedure

1261. Create operator runbooks with prechecks, exact commands, expected output, rollback, and verification.
1262. Monitor quorum, ledger height, certificate expiry, reconciliation, API latency, model health, and storage.
1263. Back up and restore on a schedule; drill it.
1264. Promote models/chaincode/config with signed manifests and rollback points.
1265. Review incidents, overrides, fraud metrics, and access logs routinely.

Implementation Example

```
# Daily Health Checks
INPUT  -> authenticate, validate schema, bind requestId/nonce
FACTS  -> read authoritative wallet/merchant/state data
AI     -> optional bounded recommendation with decisionId/version
POLICY -> enforce ownership, hard limits, approvals, risk fallback
LEDGER -> submit one idempotent deterministic transaction when value changes
EVENT  -> update projection/case/monitoring asynchronously
VERIFY -> discover committed result and retain correlationId + txId evidence
```

Verification Checklist

- The daily checklist exists and is version-controlled.
- The feature works with all public Internet/WAN connectivity disabled.
- Authorization, malformed input, duplicate retry, and failure behavior have explicit tests.
- If financial value changes, journal debits and credits balance exactly and historical entries remain immutable.
- If AI is involved, model/prompt/feature version and fallback behavior are recorded and deterministic policy remains final.
- A timeout/restart cannot create a duplicate financial result; request status can be rediscovered.
- Logs/evidence contain correlation references but not secrets or prohibited sensitive payment data.

Common Failure Modes

- Runbooks depend on one developer.
- Certificate/model/version inventory stale.
- Reconciliation alerts ignored.
- No rollback for model or chaincode deployment.

Expert Review Questions

- What happens if this component succeeds but the next component fails?
- How is the authorizing identity/device/key revoked or recovered?
- Can the operation be safely retried after the client loses the response?
- What happens if AI is unavailable, slow, or wrong?
- Can an auditor reconstruct debit, credit, actor, risk decision, approval, and Fabric txId from retained evidence?
- Which compensating transaction resolves a downstream operational error without rewriting history?

Chapter 254 - Financial Reconciliation Monitoring

Continuously detect disagreement between ledger, projections, settlements, and reports. A production runbook must work when the original developer is absent and the Internet is unavailable. Operators need exact startup/shutdown, health, reconciliation, backup, restore, certificate rotation, model rollback, chaincode upgrade, incident, and offline patching procedures.

Design rule: Journal invariants

Implementation Objective

The required output is: Reconciliation dashboard. It must be version-controlled, reviewable, testable with the WAN disabled, and linked to the architecture/source-of-truth model. For a payment platform, every implementation decision should make it possible to explain who authorized an action, which facts were trusted, which controls ran, what financial journal committed, and how the result is recovered after failure.

Key Engineering Decisions

- Journal invariants
- projection lag
- settlement totals
- unresolved suspense
- alert thresholds and owner

Deep Implementation Guidance

Runbooks must be executable by another trained operator. Include prechecks, exact commands, expected output, stop conditions, rollback, verification, and evidence to retain. Periodically drill restore, revocation, model rollback, and quorum loss.

Data and Control Questions

- What is the authoritative source for every input used by this component?
- Which user, device, service, MSP, or administrator identity is allowed to initiate or approve it?
- Does the operation move value, reserve value, create a side effect, or only provide advisory information?
- What idempotency/replay key proves that a retry is the same business intent?
- What evidence is retained off-chain and what stable reference/hash belongs on the ledger?

Chapter 254 Implementation - Financial Reconciliation Monitoring

Implementation Procedure

1266. Create operator runbooks with prechecks, exact commands, expected output, rollback, and verification.
1267. Monitor quorum, ledger height, certificate expiry, reconciliation, API latency, model health, and storage.
1268. Back up and restore on a schedule; drill it.
1269. Promote models/chaincode/config with signed manifests and rollback points.
1270. Review incidents, overrides, fraud metrics, and access logs routinely.

Implementation Example

```
# Financial Reconciliation Monitoring
INPUT  -> authenticate, validate schema, bind requestId/nonce
FACTS  -> read authoritative wallet/merchant/state data
AI     -> optional bounded recommendation with decisionId/version
POLICY -> enforce ownership, hard limits, approvals, risk fallback
LEDGER -> submit one idempotent deterministic transaction when value changes
EVENT  -> update projection/case/monitoring asynchronously
VERIFY -> discover committed result and retain correlationId + txId evidence
```

Verification Checklist

- The reconciliation dashboard exists and is version-controlled.
- The feature works with all public Internet/WAN connectivity disabled.
- Authorization, malformed input, duplicate retry, and failure behavior have explicit tests.
- If financial value changes, journal debits and credits balance exactly and historical entries remain immutable.
- If AI is involved, model/prompt/feature version and fallback behavior are recorded and deterministic policy remains final.
- A timeout/restart cannot create a duplicate financial result; request status can be rediscovered.
- Logs/evidence contain correlation references but not secrets or prohibited sensitive payment data.

Common Failure Modes

- Runbooks depend on one developer.
- Certificate/model/version inventory stale.
- Reconciliation alerts ignored.
- No rollback for model or chaincode deployment.

Expert Review Questions

- What happens if this component succeeds but the next component fails?
- How is the authorizing identity/device/key revoked or recovered?
- Can the operation be safely retried after the client loses the response?
- What happens if AI is unavailable, slow, or wrong?
- Can an auditor reconstruct debit, credit, actor, risk decision, approval, and Fabric txId from retained evidence?
- Which compensating transaction resolves a downstream operational error without rewriting history?

Chapter 255 - Backup Architecture

Define what must be backed up and how often. A production runbook must work when the original developer is absent and the Internet is unavailable. Operators need exact startup/shutdown, health, reconciliation, backup, restore, certificate rotation, model rollback, chaincode upgrade, incident, and offline patching procedures.

Design rule: Fabric data/config

Implementation Objective

The required output is: Backup matrix. It must be version-controlled, reviewable, testable with the WAN disabled, and linked to the architecture/source-of-truth model. For a payment platform, every implementation decision should make it possible to explain who authorized an action, which facts were trusted, which controls ran, what financial journal committed, and how the result is recovered after failure.

Key Engineering Decisions

- Fabric data/config
- PostgreSQL
- CA/config/critical keys according to policy
- model registry
- prompts
- receipts
- logs/runbooks

Deep Implementation Guidance

Runbooks must be executable by another trained operator. Include prechecks, exact commands, expected output, stop conditions, rollback, verification, and evidence to retain. Periodically drill restore, revocation, model rollback, and quorum loss.

Data and Control Questions

- What is the authoritative source for every input used by this component?
- Which user, device, service, MSP, or administrator identity is allowed to initiate or approve it?
- Does the operation move value, reserve value, create a side effect, or only provide advisory information?
- What idempotency/replay key proves that a retry is the same business intent?
- What evidence is retained off-chain and what stable reference/hash belongs on the ledger?

Chapter 255 Implementation - Backup Architecture

Implementation Procedure

1271. Create operator runbooks with prechecks, exact commands, expected output, rollback, and verification.
1272. Monitor quorum, ledger height, certificate expiry, reconciliation, API latency, model health, and storage.
1273. Back up and restore on a schedule; drill it.
1274. Promote models/chaincode/config with signed manifests and rollback points.
1275. Review incidents, overrides, fraud metrics, and access logs routinely.

Implementation Example

```
# Backup Architecture
INPUT  -> authenticate, validate schema, bind requestId/nonce
FACTS  -> read authoritative wallet/merchant/state data
AI      -> optional bounded recommendation with decisionId/version
POLICY -> enforce ownership, hard limits, approvals, risk fallback
LEDGER -> submit one idempotent deterministic transaction when value changes
EVENT  -> update projection/case/monitoring asynchronously
VERIFY -> discover committed result and retain correlationId + txId evidence
```

Verification Checklist

- The backup matrix exists and is version-controlled.
- The feature works with all public Internet/WAN connectivity disabled.
- Authorization, malformed input, duplicate retry, and failure behavior have explicit tests.
- If financial value changes, journal debits and credits balance exactly and historical entries remain immutable.
- If AI is involved, model/prompt/feature version and fallback behavior are recorded and deterministic policy remains final.
- A timeout/restart cannot create a duplicate financial result; request status can be rediscovered.
- Logs/evidence contain correlation references but not secrets or prohibited sensitive payment data.

Common Failure Modes

- Runbooks depend on one developer.
- Certificate/model/version inventory stale.
- Reconciliation alerts ignored.
- No rollback for model or chaincode deployment.

Expert Review Questions

- What happens if this component succeeds but the next component fails?
- How is the authorizing identity/device/key revoked or recovered?
- Can the operation be safely retried after the client loses the response?
- What happens if AI is unavailable, slow, or wrong?
- Can an auditor reconstruct debit, credit, actor, risk decision, approval, and Fabric txId from retained evidence?
- Which compensating transaction resolves a downstream operational error without rewriting history?

Chapter 256 - Fabric Recovery Procedure

Recover peers/orderers according to supported Fabric operational practices. A production runbook must work when the original developer is absent and the Internet is unavailable. Operators need exact startup/shutdown, health, reconciliation, backup, restore, certificate rotation, model rollback, chaincode upgrade, incident, and offline patching procedures.

Design rule: Use tested backup/snapshot strategy

Implementation Objective

The required output is: Fabric recovery runbook. It must be version-controlled, reviewable, testable with the WAN disabled, and linked to the architecture/source-of-truth model. For a payment platform, every implementation decision should make it possible to explain who authorized an action, which facts were trusted, which controls ran, what financial journal committed, and how the result is recovered after failure.

Key Engineering Decisions

- Use tested backup/snapshot strategy
- preserve channel/config material
- verify ledger height/hash consistency
- rejoin/catch up where appropriate

Deep Implementation Guidance

Runbooks must be executable by another trained operator. Include prechecks, exact commands, expected output, stop conditions, rollback, verification, and evidence to retain. Periodically drill restore, revocation, model rollback, and quorum loss.

Data and Control Questions

- What is the authoritative source for every input used by this component?
- Which user, device, service, MSP, or administrator identity is allowed to initiate or approve it?
- Does the operation move value, reserve value, create a side effect, or only provide advisory information?
- What idempotency/replay key proves that a retry is the same business intent?
- What evidence is retained off-chain and what stable reference/hash belongs on the ledger?

Chapter 256 Implementation - Fabric Recovery Procedure

Implementation Procedure

1276. Create operator runbooks with prechecks, exact commands, expected output, rollback, and verification.
1277. Monitor quorum, ledger height, certificate expiry, reconciliation, API latency, model health, and storage.
1278. Back up and restore on a schedule; drill it.
1279. Promote models/chaincode/config with signed manifests and rollback points.
1280. Review incidents, overrides, fraud metrics, and access logs routinely.

Implementation Example

```
# Fabric Recovery Procedure
INPUT  -> authenticate, validate schema, bind requestId/nonce
FACTS  -> read authoritative wallet/merchant/state data
AI      -> optional bounded recommendation with decisionId/version
POLICY -> enforce ownership, hard limits, approvals, risk fallback
LEDGER  -> submit one idempotent deterministic transaction when value changes
EVENT   -> update projection/case/monitoring asynchronously
VERIFY  -> discover committed result and retain correlationId + txId evidence
```

Verification Checklist

- The fabric recovery runbook exists and is version-controlled.
- The feature works with all public Internet/WAN connectivity disabled.
- Authorization, malformed input, duplicate retry, and failure behavior have explicit tests.
- If financial value changes, journal debits and credits balance exactly and historical entries remain immutable.
- If AI is involved, model/prompt/feature version and fallback behavior are recorded and deterministic policy remains final.
- A timeout/restart cannot create a duplicate financial result; request status can be rediscovered.
- Logs/evidence contain correlation references but not secrets or prohibited sensitive payment data.

Common Failure Modes

- Runbooks depend on one developer.
- Certificate/model/version inventory stale.
- Reconciliation alerts ignored.
- No rollback for model or chaincode deployment.

Expert Review Questions

- What happens if this component succeeds but the next component fails?
- How is the authorizing identity/device/key revoked or recovered?
- Can the operation be safely retried after the client loses the response?
- What happens if AI is unavailable, slow, or wrong?
- Can an auditor reconstruct debit, credit, actor, risk decision, approval, and Fabric txId from retained evidence?
- Which compensating transaction resolves a downstream operational error without rewriting history?

Chapter 257 - PostgreSQL Restore and Projection Rebuild

Recover operational reads without corrupting ledger truth. A production runbook must work when the original developer is absent and the Internet is unavailable. Operators need exact startup/shutdown, health, reconciliation, backup, restore, certificate rotation, model rollback, chaincode upgrade, incident, and offline patching procedures.

Design rule: Restore DB backup or rebuild financial projections from ledger/events/export

Implementation Objective

The required output is: DB recovery runbook. It must be version-controlled, reviewable, testable with the WAN disabled, and linked to the architecture/source-of-truth model. For a payment platform, every implementation decision should make it possible to explain who authorized an action, which facts were trusted, which controls ran, what financial journal committed, and how the result is recovered after failure.

Key Engineering Decisions

- Restore DB backup or rebuild financial projections from ledger/events/export
- reconcile before resuming normal operations

Deep Implementation Guidance

Runbooks must be executable by another trained operator. Include prechecks, exact commands, expected output, stop conditions, rollback, verification, and evidence to retain. Periodically drill restore, revocation, model rollback, and quorum loss.

Data and Control Questions

- What is the authoritative source for every input used by this component?
- Which user, device, service, MSP, or administrator identity is allowed to initiate or approve it?
- Does the operation move value, reserve value, create a side effect, or only provide advisory information?
- What idempotency/replay key proves that a retry is the same business intent?
- What evidence is retained off-chain and what stable reference/hash belongs on the ledger?

Chapter 257 Implementation - PostgreSQL Restore and Projection Rebuild

Implementation Procedure

1281. Create operator runbooks with prechecks, exact commands, expected output, rollback, and verification.
1282. Monitor quorum, ledger height, certificate expiry, reconciliation, API latency, model health, and storage.
1283. Back up and restore on a schedule; drill it.
1284. Promote models/chaincode/config with signed manifests and rollback points.
1285. Review incidents, overrides, fraud metrics, and access logs routinely.

Implementation Example

```
# PostgreSQL Restore and Projection Rebuild
INPUT  -> authenticate, validate schema, bind requestId/nonce
FACTS  -> read authoritative wallet/merchant/state data
AI     -> optional bounded recommendation with decisionId/version
POLICY -> enforce ownership, hard limits, approvals, risk fallback
LEDGER -> submit one idempotent deterministic transaction when value changes
EVENT  -> update projection/case/monitoring asynchronously
VERIFY -> discover committed result and retain correlationId + txId evidence
```

Verification Checklist

- The db recovery runbook exists and is version-controlled.
- The feature works with all public Internet/WAN connectivity disabled.
- Authorization, malformed input, duplicate retry, and failure behavior have explicit tests.
- If financial value changes, journal debits and credits balance exactly and historical entries remain immutable.
- If AI is involved, model/prompt/feature version and fallback behavior are recorded and deterministic policy remains final.
- A timeout/restart cannot create a duplicate financial result; request status can be rediscovered.
- Logs/evidence contain correlation references but not secrets or prohibited sensitive payment data.

Common Failure Modes

- Runbooks depend on one developer.
- Certificate/model/version inventory stale.
- Reconciliation alerts ignored.
- No rollback for model or chaincode deployment.

Expert Review Questions

- What happens if this component succeeds but the next component fails?
- How is the authorizing identity/device/key revoked or recovered?
- Can the operation be safely retried after the client loses the response?
- What happens if AI is unavailable, slow, or wrong?
- Can an auditor reconstruct debit, credit, actor, risk decision, approval, and Fabric txId from retained evidence?
- Which compensating transaction resolves a downstream operational error without rewriting history?

Chapter 258 - Certificate Rotation Operations

Renew local TLS/enrollment identities on a calendar. A production runbook must work when the original developer is absent and the Internet is unavailable. Operators need exact startup/shutdown, health, reconciliation, backup, restore, certificate rotation, model rollback, chaincode upgrade, incident, and offline patching procedures.

Design rule: Inventory expiries

Implementation Objective

The required output is: Cert rotation schedule. It must be version-controlled, reviewable, testable with the WAN disabled, and linked to the architecture/source-of-truth model. For a payment platform, every implementation decision should make it possible to explain who authorized an action, which facts were trusted, which controls ran, what financial journal committed, and how the result is recovered after failure.

Key Engineering Decisions

- Inventory expiries
- stage new certs
- deploy in controlled order
- verify trust/health
- remove/revoke old material

Deep Implementation Guidance

Runbooks must be executable by another trained operator. Include prechecks, exact commands, expected output, stop conditions, rollback, verification, and evidence to retain. Periodically drill restore, revocation, model rollback, and quorum loss.

Data and Control Questions

- What is the authoritative source for every input used by this component?
- Which user, device, service, MSP, or administrator identity is allowed to initiate or approve it?
- Does the operation move value, reserve value, create a side effect, or only provide advisory information?
- What idempotency/replay key proves that a retry is the same business intent?
- What evidence is retained off-chain and what stable reference/hash belongs on the ledger?

Chapter 258 Implementation - Certificate Rotation Operations

Implementation Procedure

1286. Create operator runbooks with prechecks, exact commands, expected output, rollback, and verification.
1287. Monitor quorum, ledger height, certificate expiry, reconciliation, API latency, model health, and storage.
1288. Back up and restore on a schedule; drill it.
1289. Promote models/chaincode/config with signed manifests and rollback points.
1290. Review incidents, overrides, fraud metrics, and access logs routinely.

Implementation Example

```
# Certificate Rotation Operations
INPUT  -> authenticate, validate schema, bind requestId/nonce
FACTS  -> read authoritative wallet/merchant/state data
AI      -> optional bounded recommendation with decisionId/version
POLICY  -> enforce ownership, hard limits, approvals, risk fallback
LEDGER  -> submit one idempotent deterministic transaction when value changes
EVENT   -> update projection/case/monitoring asynchronously
VERIFY  -> discover committed result and retain correlationId + txId evidence
```

Verification Checklist

- The cert rotation schedule exists and is version-controlled.
- The feature works with all public Internet/WAN connectivity disabled.
- Authorization, malformed input, duplicate retry, and failure behavior have explicit tests.
- If financial value changes, journal debits and credits balance exactly and historical entries remain immutable.
- If AI is involved, model/prompt/feature version and fallback behavior are recorded and deterministic policy remains final.
- A timeout/restart cannot create a duplicate financial result; request status can be rediscovered.
- Logs/evidence contain correlation references but not secrets or prohibited sensitive payment data.

Common Failure Modes

- Runbooks depend on one developer.
- Certificate/model/version inventory stale.
- Reconciliation alerts ignored.
- No rollback for model or chaincode deployment.

Expert Review Questions

- What happens if this component succeeds but the next component fails?
- How is the authorizing identity/device/key revoked or recovered?
- Can the operation be safely retried after the client loses the response?
- What happens if AI is unavailable, slow, or wrong?
- Can an auditor reconstruct debit, credit, actor, risk decision, approval, and Fabric txId from retained evidence?
- Which compensating transaction resolves a downstream operational error without rewriting history?

Chapter 259 - Chaincode Upgrade Operations

Promote new deterministic financial rules safely. A production runbook must work when the original developer is absent and the Internet is unavailable. Operators need exact startup/shutdown, health, reconciliation, backup, restore, certificate rotation, model rollback, chaincode upgrade, incident, and offline patching procedures.

Design rule: Package/hash

Implementation Objective

The required output is: Chaincode release runbook. It must be version-controlled, reviewable, testable with the WAN disabled, and linked to the architecture/source-of-truth model. For a payment platform, every implementation decision should make it possible to explain who authorized an action, which facts were trusted, which controls ran, what financial journal committed, and how the result is recovered after failure.

Key Engineering Decisions

- Package/hash
- multi-org approvals
- commit sequence
- smoke and invariant tests
- rollback/forward-fix plan

Deep Implementation Guidance

Runbooks must be executable by another trained operator. Include prechecks, exact commands, expected output, stop conditions, rollback, verification, and evidence to retain. Periodically drill restore, revocation, model rollback, and quorum loss.

Data and Control Questions

- What is the authoritative source for every input used by this component?
- Which user, device, service, MSP, or administrator identity is allowed to initiate or approve it?
- Does the operation move value, reserve value, create a side effect, or only provide advisory information?
- What idempotency/replay key proves that a retry is the same business intent?
- What evidence is retained off-chain and what stable reference/hash belongs on the ledger?

Chapter 259 Implementation - Chaincode Upgrade Operations

Implementation Procedure

1291. Create operator runbooks with prechecks, exact commands, expected output, rollback, and verification.
1292. Monitor quorum, ledger height, certificate expiry, reconciliation, API latency, model health, and storage.
1293. Back up and restore on a schedule; drill it.
1294. Promote models/chaincode/config with signed manifests and rollback points.
1295. Review incidents, overrides, fraud metrics, and access logs routinely.

Implementation Example

```
# Chaincode Upgrade Operations
INPUT  -> authenticate, validate schema, bind requestId/nonce
FACTS  -> read authoritative wallet/merchant/state data
AI     -> optional bounded recommendation with decisionId/version
POLICY -> enforce ownership, hard limits, approvals, risk fallback
LEDGER -> submit one idempotent deterministic transaction when value changes
EVENT  -> update projection/case/monitoring asynchronously
VERIFY -> discover committed result and retain correlationId + txId evidence
```

Verification Checklist

- The chaincode release runbook exists and is version-controlled.
- The feature works with all public Internet/WAN connectivity disabled.
- Authorization, malformed input, duplicate retry, and failure behavior have explicit tests.
- If financial value changes, journal debits and credits balance exactly and historical entries remain immutable.
- If AI is involved, model/prompt/feature version and fallback behavior are recorded and deterministic policy remains final.
- A timeout/restart cannot create a duplicate financial result; request status can be rediscovered.
- Logs/evidence contain correlation references but not secrets or prohibited sensitive payment data.

Common Failure Modes

- Runbooks depend on one developer.
- Certificate/model/version inventory stale.
- Reconciliation alerts ignored.
- No rollback for model or chaincode deployment.

Expert Review Questions

- What happens if this component succeeds but the next component fails?
- How is the authorizing identity/device/key revoked or recovered?
- Can the operation be safely retried after the client loses the response?
- What happens if AI is unavailable, slow, or wrong?
- Can an auditor reconstruct debit, credit, actor, risk decision, approval, and Fabric txId from retained evidence?
- Which compensating transaction resolves a downstream operational error without rewriting history?

Chapter 260 - API and UI Release Operations

Deploy local application releases with zero public dependency. A production runbook must work when the original developer is absent and the Internet is unavailable. Operators need exact startup/shutdown, health, reconciliation, backup, restore, certificate rotation, model rollback, chaincode upgrade, incident, and offline patching procedures.

Design rule: Signed/checksummed bundle

Implementation Objective

The required output is: App release runbook. It must be version-controlled, reviewable, testable with the WAN disabled, and linked to the architecture/source-of-truth model. For a payment platform, every implementation decision should make it possible to explain who authorized an action, which facts were trusted, which controls ran, what financial journal committed, and how the result is recovered after failure.

Key Engineering Decisions

- Signed/checksummed bundle
- database migration plan
- blue/green or maintenance window
- health and rollback

Deep Implementation Guidance

Runbooks must be executable by another trained operator. Include prechecks, exact commands, expected output, stop conditions, rollback, verification, and evidence to retain. Periodically drill restore, revocation, model rollback, and quorum loss.

Data and Control Questions

- What is the authoritative source for every input used by this component?
- Which user, device, service, MSP, or administrator identity is allowed to initiate or approve it?
- Does the operation move value, reserve value, create a side effect, or only provide advisory information?
- What idempotency/replay key proves that a retry is the same business intent?
- What evidence is retained off-chain and what stable reference/hash belongs on the ledger?

Chapter 260 Implementation - API and UI Release Operations

Implementation Procedure

1296. Create operator runbooks with prechecks, exact commands, expected output, rollback, and verification.
1297. Monitor quorum, ledger height, certificate expiry, reconciliation, API latency, model health, and storage.
1298. Back up and restore on a schedule; drill it.
1299. Promote models/chaincode/config with signed manifests and rollback points.
1300. Review incidents, overrides, fraud metrics, and access logs routinely.

Implementation Example

```
# API and UI Release Operations
INPUT  -> authenticate, validate schema, bind requestId/nonce
FACTS  -> read authoritative wallet/merchant/state data
AI     -> optional bounded recommendation with decisionId/version
POLICY -> enforce ownership, hard limits, approvals, risk fallback
LEDGER -> submit one idempotent deterministic transaction when value changes
EVENT  -> update projection/case/monitoring asynchronously
VERIFY -> discover committed result and retain correlationId + txId evidence
```

Verification Checklist

- The app release runbook exists and is version-controlled.
- The feature works with all public Internet/WAN connectivity disabled.
- Authorization, malformed input, duplicate retry, and failure behavior have explicit tests.
- If financial value changes, journal debits and credits balance exactly and historical entries remain immutable.
- If AI is involved, model/prompt/feature version and fallback behavior are recorded and deterministic policy remains final.
- A timeout/restart cannot create a duplicate financial result; request status can be rediscovered.
- Logs/evidence contain correlation references but not secrets or prohibited sensitive payment data.

Common Failure Modes

- Runbooks depend on one developer.
- Certificate/model/version inventory stale.
- Reconciliation alerts ignored.
- No rollback for model or chaincode deployment.

Expert Review Questions

- What happens if this component succeeds but the next component fails?
- How is the authorizing identity/device/key revoked or recovered?
- Can the operation be safely retried after the client loses the response?
- What happens if AI is unavailable, slow, or wrong?
- Can an auditor reconstruct debit, credit, actor, risk decision, approval, and Fabric txId from retained evidence?
- Which compensating transaction resolves a downstream operational error without rewriting history?

Chapter 261 - Fraud Model Promotion

Move candidate ML model from evaluation to shadow/advisory/enforcement. A production runbook must work when the original developer is absent and the Internet is unavailable. Operators need exact startup/shutdown, health, reconciliation, backup, restore, certificate rotation, model rollback, chaincode upgrade, incident, and offline patching procedures.

Design rule: Eval threshold

Implementation Objective

The required output is: Model promotion gate. It must be version-controlled, reviewable, testable with the WAN disabled, and linked to the architecture/source-of-truth model. For a payment platform, every implementation decision should make it possible to explain who authorized an action, which facts were trusted, which controls ran, what financial journal committed, and how the result is recovered after failure.

Key Engineering Decisions

- Eval threshold
- security review
- business owner approval
- hash registry
- canary/shadow
- rollback switch

Deep Implementation Guidance

Runbooks must be executable by another trained operator. Include prechecks, exact commands, expected output, stop conditions, rollback, verification, and evidence to retain. Periodically drill restore, revocation, model rollback, and quorum loss.

Data and Control Questions

- What is the authoritative source for every input used by this component?
- Which user, device, service, MSP, or administrator identity is allowed to initiate or approve it?
- Does the operation move value, reserve value, create a side effect, or only provide advisory information?
- What idempotency/replay key proves that a retry is the same business intent?
- What evidence is retained off-chain and what stable reference/hash belongs on the ledger?

Chapter 261 Implementation - Fraud Model Promotion

Implementation Procedure

1301. Create operator runbooks with prechecks, exact commands, expected output, rollback, and verification.
1302. Monitor quorum, ledger height, certificate expiry, reconciliation, API latency, model health, and storage.
1303. Back up and restore on a schedule; drill it.
1304. Promote models/chaincode/config with signed manifests and rollback points.
1305. Review incidents, overrides, fraud metrics, and access logs routinely.

Implementation Example

```
# Fraud Model Promotion
INPUT  -> authenticate, validate schema, bind requestId/nonce
FACTS  -> read authoritative wallet/merchant/state data
AI     -> optional bounded recommendation with decisionId/version
POLICY -> enforce ownership, hard limits, approvals, risk fallback
LEDGER -> submit one idempotent deterministic transaction when value changes
EVENT  -> update projection/case/monitoring asynchronously
VERIFY -> discover committed result and retain correlationId + txId evidence
```

Verification Checklist

- The model promotion gate exists and is version-controlled.
- The feature works with all public Internet/WAN connectivity disabled.
- Authorization, malformed input, duplicate retry, and failure behavior have explicit tests.
- If financial value changes, journal debits and credits balance exactly and historical entries remain immutable.
- If AI is involved, model/prompt/feature version and fallback behavior are recorded and deterministic policy remains final.
- A timeout/restart cannot create a duplicate financial result; request status can be rediscovered.
- Logs/evidence contain correlation references but not secrets or prohibited sensitive payment data.

Common Failure Modes

- Runbooks depend on one developer.
- Certificate/model/version inventory stale.
- Reconciliation alerts ignored.
- No rollback for model or chaincode deployment.

Expert Review Questions

- What happens if this component succeeds but the next component fails?
- How is the authorizing identity/device/key revoked or recovered?
- Can the operation be safely retried after the client loses the response?
- What happens if AI is unavailable, slow, or wrong?
- Can an auditor reconstruct debit, credit, actor, risk decision, approval, and Fabric txId from retained evidence?
- Which compensating transaction resolves a downstream operational error without rewriting history?

Chapter 262 - LLM/Prompt/RAG Promotion

Update local assistant components under change control. A production runbook must work when the original developer is absent and the Internet is unavailable. Operators need exact startup/shutdown, health, reconciliation, backup, restore, certificate rotation, model rollback, chaincode upgrade, incident, and offline patching procedures.

Design rule: Corpus/prompt/model/schema hashes

Implementation Objective

The required output is: LLM release gate. It must be version-controlled, reviewable, testable with the WAN disabled, and linked to the architecture/source-of-truth model. For a payment platform, every implementation decision should make it possible to explain who authorized an action, which facts were trusted, which controls ran, what financial journal committed, and how the result is recovered after failure.

Key Engineering Decisions

- Corpus/prompt/model/schema hashes
- RAG/safety eval
- resource test
- approval
- rollback bundle

Deep Implementation Guidance

Runbooks must be executable by another trained operator. Include prechecks, exact commands, expected output, stop conditions, rollback, verification, and evidence to retain. Periodically drill restore, revocation, model rollback, and quorum loss.

Data and Control Questions

- What is the authoritative source for every input used by this component?
- Which user, device, service, MSP, or administrator identity is allowed to initiate or approve it?
- Does the operation move value, reserve value, create a side effect, or only provide advisory information?
- What idempotency/replay key proves that a retry is the same business intent?
- What evidence is retained off-chain and what stable reference/hash belongs on the ledger?

Chapter 262 Implementation - LLM/Prompt/RAG Promotion

Implementation Procedure

1306. Create operator runbooks with prechecks, exact commands, expected output, rollback, and verification.
1307. Monitor quorum, ledger height, certificate expiry, reconciliation, API latency, model health, and storage.
1308. Back up and restore on a schedule; drill it.
1309. Promote models/chaincode/config with signed manifests and rollback points.
1310. Review incidents, overrides, fraud metrics, and access logs routinely.

Implementation Example

```
# LLM/Prompt/RAG Promotion
INPUT  -> authenticate, validate schema, bind requestId/nonce
FACTS  -> read authoritative wallet/merchant/state data
AI     -> optional bounded recommendation with decisionId/version
POLICY -> enforce ownership, hard limits, approvals, risk fallback
LEDGER -> submit one idempotent deterministic transaction when value changes
EVENT  -> update projection/case/monitoring asynchronously
VERIFY -> discover committed result and retain correlationId + txId evidence
```

Verification Checklist

- The llm release gate exists and is version-controlled.
- The feature works with all public Internet/WAN connectivity disabled.
- Authorization, malformed input, duplicate retry, and failure behavior have explicit tests.
- If financial value changes, journal debits and credits balance exactly and historical entries remain immutable.
- If AI is involved, model/prompt/feature version and fallback behavior are recorded and deterministic policy remains final.
- A timeout/restart cannot create a duplicate financial result; request status can be rediscovered.
- Logs/evidence contain correlation references but not secrets or prohibited sensitive payment data.

Common Failure Modes

- Runbooks depend on one developer.
- Certificate/model/version inventory stale.
- Reconciliation alerts ignored.
- No rollback for model or chaincode deployment.

Expert Review Questions

- What happens if this component succeeds but the next component fails?
- How is the authorizing identity/device/key revoked or recovered?
- Can the operation be safely retried after the client loses the response?
- What happens if AI is unavailable, slow, or wrong?
- Can an auditor reconstruct debit, credit, actor, risk decision, approval, and Fabric txId from retained evidence?
- Which compensating transaction resolves a downstream operational error without rewriting history?

Chapter 263 - Incident Operations

Coordinate write controls, revocation, reconciliation, and evidence during incidents. A production runbook must work when the original developer is absent and the Internet is unavailable. Operators need exact startup/shutdown, health, reconciliation, backup, restore, certificate rotation, model rollback, chaincode upgrade, incident, and offline patching procedures.

Design rule: Classify severity

Implementation Objective

The required output is: Incident operator runbook. It must be version-controlled, reviewable, testable with the WAN disabled, and linked to the architecture/source-of-truth model. For a payment platform, every implementation decision should make it possible to explain who authorized an action, which facts were trusted, which controls ran, what financial journal committed, and how the result is recovered after failure.

Key Engineering Decisions

- Classify severity
- disable affected writes
- freeze specific entities
- capture evidence
- communicate locally
- recover and reconcile

Deep Implementation Guidance

Runbooks must be executable by another trained operator. Include prechecks, exact commands, expected output, stop conditions, rollback, verification, and evidence to retain. Periodically drill restore, revocation, model rollback, and quorum loss.

Data and Control Questions

- What is the authoritative source for every input used by this component?
- Which user, device, service, MSP, or administrator identity is allowed to initiate or approve it?
- Does the operation move value, reserve value, create a side effect, or only provide advisory information?
- What idempotency/replay key proves that a retry is the same business intent?
- What evidence is retained off-chain and what stable reference/hash belongs on the ledger?

Chapter 263 Implementation - Incident Operations

Implementation Procedure

1311. Create operator runbooks with prechecks, exact commands, expected output, rollback, and verification.
1312. Monitor quorum, ledger height, certificate expiry, reconciliation, API latency, model health, and storage.
1313. Back up and restore on a schedule; drill it.
1314. Promote models/chaincode/config with signed manifests and rollback points.
1315. Review incidents, overrides, fraud metrics, and access logs routinely.

Implementation Example

```
# Incident Operations
INPUT  -> authenticate, validate schema, bind requestId/nonce
FACTS  -> read authoritative wallet/merchant/state data
AI      -> optional bounded recommendation with decisionId/version
POLICY  -> enforce ownership, hard limits, approvals, risk fallback
LEDGER  -> submit one idempotent deterministic transaction when value changes
EVENT   -> update projection/case/monitoring asynchronously
VERIFY  -> discover committed result and retain correlationId + txId evidence
```

Verification Checklist

- The incident operator runbook exists and is version-controlled.
- The feature works with all public Internet/WAN connectivity disabled.
- Authorization, malformed input, duplicate retry, and failure behavior have explicit tests.
- If financial value changes, journal debits and credits balance exactly and historical entries remain immutable.
- If AI is involved, model/prompt/feature version and fallback behavior are recorded and deterministic policy remains final.
- A timeout/restart cannot create a duplicate financial result; request status can be rediscovered.
- Logs/evidence contain correlation references but not secrets or prohibited sensitive payment data.

Common Failure Modes

- Runbooks depend on one developer.
- Certificate/model/version inventory stale.
- Reconciliation alerts ignored.
- No rollback for model or chaincode deployment.

Expert Review Questions

- What happens if this component succeeds but the next component fails?
- How is the authorizing identity/device/key revoked or recovered?
- Can the operation be safely retried after the client loses the response?
- What happens if AI is unavailable, slow, or wrong?
- Can an auditor reconstruct debit, credit, actor, risk decision, approval, and Fabric txId from retained evidence?
- Which compensating transaction resolves a downstream operational error without rewriting history?

Chapter 264 - Offline Patch Management

Apply OS/container/package/model updates without Internet access. A production runbook must work when the original developer is absent and the Internet is unavailable. Operators need exact startup/shutdown, health, reconciliation, backup, restore, certificate rotation, model rollback, chaincode upgrade, incident, and offline patching procedures.

Design rule: Acquire/verify in staging

Implementation Objective

The required output is: Patch runbook. It must be version-controlled, reviewable, testable with the WAN disabled, and linked to the architecture/source-of-truth model. For a payment platform, every implementation decision should make it possible to explain who authorized an action, which facts were trusted, which controls ran, what financial journal committed, and how the result is recovered after failure.

Key Engineering Decisions

- Acquire/verify in staging
- create internal release manifest
- test
- backup
- apply
- verify
- retain rollback artifacts

Deep Implementation Guidance

Runbooks must be executable by another trained operator. Include prechecks, exact commands, expected output, stop conditions, rollback, verification, and evidence to retain. Periodically drill restore, revocation, model rollback, and quorum loss.

Data and Control Questions

- What is the authoritative source for every input used by this component?
- Which user, device, service, MSP, or administrator identity is allowed to initiate or approve it?
- Does the operation move value, reserve value, create a side effect, or only provide advisory information?
- What idempotency/replay key proves that a retry is the same business intent?
- What evidence is retained off-chain and what stable reference/hash belongs on the ledger?

Chapter 264 Implementation - Offline Patch Management

Implementation Procedure

1316. Create operator runbooks with prechecks, exact commands, expected output, rollback, and verification.
1317. Monitor quorum, ledger height, certificate expiry, reconciliation, API latency, model health, and storage.
1318. Back up and restore on a schedule; drill it.
1319. Promote models/chaincode/config with signed manifests and rollback points.
1320. Review incidents, overrides, fraud metrics, and access logs routinely.

Implementation Example

```
# Offline Patch Management
INPUT  -> authenticate, validate schema, bind requestId/nonce
FACTS  -> read authoritative wallet/merchant/state data
AI     -> optional bounded recommendation with decisionId/version
POLICY -> enforce ownership, hard limits, approvals, risk fallback
LEDGER -> submit one idempotent deterministic transaction when value changes
EVENT  -> update projection/case/monitoring asynchronously
VERIFY -> discover committed result and retain correlationId + txId evidence
```

Verification Checklist

- The patch runbook exists and is version-controlled.
- The feature works with all public Internet/WAN connectivity disabled.
- Authorization, malformed input, duplicate retry, and failure behavior have explicit tests.
- If financial value changes, journal debits and credits balance exactly and historical entries remain immutable.
- If AI is involved, model/prompt/feature version and fallback behavior are recorded and deterministic policy remains final.
- A timeout/restart cannot create a duplicate financial result; request status can be rediscovered.
- Logs/evidence contain correlation references but not secrets or prohibited sensitive payment data.

Common Failure Modes

- Runbooks depend on one developer.
- Certificate/model/version inventory stale.
- Reconciliation alerts ignored.
- No rollback for model or chaincode deployment.

Expert Review Questions

- What happens if this component succeeds but the next component fails?
- How is the authorizing identity/device/key revoked or recovered?
- Can the operation be safely retried after the client loses the response?
- What happens if AI is unavailable, slow, or wrong?
- Can an auditor reconstruct debit, credit, actor, risk decision, approval, and Fabric txId from retained evidence?
- Which compensating transaction resolves a downstream operational error without rewriting history?

Chapter 265 - Capacity Monitoring

Predict storage/compute exhaustion before it stops payments. A production runbook must work when the original developer is absent and the Internet is unavailable. Operators need exact startup/shutdown, health, reconciliation, backup, restore, certificate rotation, model rollback, chaincode upgrade, incident, and offline patching procedures.

Design rule: Ledger growth

Implementation Objective

The required output is: Capacity dashboard. It must be version-controlled, reviewable, testable with the WAN disabled, and linked to the architecture/source-of-truth model. For a payment platform, every implementation decision should make it possible to explain who authorized an action, which facts were trusted, which controls ran, what financial journal committed, and how the result is recovered after failure.

Key Engineering Decisions

- Ledger growth
- CouchDB/Postgres disk
- logs
- model storage
- RAM/CPU
- AI queues
- backup capacity

Deep Implementation Guidance

Runbooks must be executable by another trained operator. Include prechecks, exact commands, expected output, stop conditions, rollback, verification, and evidence to retain. Periodically drill restore, revocation, model rollback, and quorum loss.

Data and Control Questions

- What is the authoritative source for every input used by this component?
- Which user, device, service, MSP, or administrator identity is allowed to initiate or approve it?
- Does the operation move value, reserve value, create a side effect, or only provide advisory information?
- What idempotency/replay key proves that a retry is the same business intent?
- What evidence is retained off-chain and what stable reference/hash belongs on the ledger?

Chapter 265 Implementation - Capacity Monitoring

Implementation Procedure

1321. Create operator runbooks with prechecks, exact commands, expected output, rollback, and verification.
1322. Monitor quorum, ledger height, certificate expiry, reconciliation, API latency, model health, and storage.
1323. Back up and restore on a schedule; drill it.
1324. Promote models/chaincode/config with signed manifests and rollback points.
1325. Review incidents, overrides, fraud metrics, and access logs routinely.

Implementation Example

```
# Capacity Monitoring
INPUT  -> authenticate, validate schema, bind requestId/nonce
FACTS  -> read authoritative wallet/merchant/state data
AI     -> optional bounded recommendation with decisionId/version
POLICY -> enforce ownership, hard limits, approvals, risk fallback
LEDGER -> submit one idempotent deterministic transaction when value changes
EVENT  -> update projection/case/monitoring asynchronously
VERIFY -> discover committed result and retain correlationId + txId evidence
```

Verification Checklist

- The capacity dashboard exists and is version-controlled.
- The feature works with all public Internet/WAN connectivity disabled.
- Authorization, malformed input, duplicate retry, and failure behavior have explicit tests.
- If financial value changes, journal debits and credits balance exactly and historical entries remain immutable.
- If AI is involved, model/prompt/feature version and fallback behavior are recorded and deterministic policy remains final.
- A timeout/restart cannot create a duplicate financial result; request status can be rediscovered.
- Logs/evidence contain correlation references but not secrets or prohibited sensitive payment data.

Common Failure Modes

- Runbooks depend on one developer.
- Certificate/model/version inventory stale.
- Reconciliation alerts ignored.
- No rollback for model or chaincode deployment.

Expert Review Questions

- What happens if this component succeeds but the next component fails?
- How is the authorizing identity/device/key revoked or recovered?
- Can the operation be safely retried after the client loses the response?
- What happens if AI is unavailable, slow, or wrong?
- Can an auditor reconstruct debit, credit, actor, risk decision, approval, and Fabric txId from retained evidence?
- Which compensating transaction resolves a downstream operational error without rewriting history?

Chapter 266 - Operator Access Review

Remove stale privileges and prove separation of duties. A production runbook must work when the original developer is absent and the Internet is unavailable. Operators need exact startup/shutdown, health, reconciliation, backup, restore, certificate rotation, model rollback, chaincode upgrade, incident, and offline patching procedures.

Design rule: Quarterly/monthly review

Implementation Objective

The required output is: Access review. It must be version-controlled, reviewable, testable with the WAN disabled, and linked to the architecture/source-of-truth model. For a payment platform, every implementation decision should make it possible to explain who authorized an action, which facts were trusted, which controls ran, what financial journal committed, and how the result is recovered after failure.

Key Engineering Decisions

- Quarterly/monthly review
- user/device/service/admin roles
- HSM operators
- break-glass sealed credentials
- auditor sign-off

Deep Implementation Guidance

Runbooks must be executable by another trained operator. Include prechecks, exact commands, expected output, stop conditions, rollback, verification, and evidence to retain. Periodically drill restore, revocation, model rollback, and quorum loss.

Data and Control Questions

- What is the authoritative source for every input used by this component?
- Which user, device, service, MSP, or administrator identity is allowed to initiate or approve it?
- Does the operation move value, reserve value, create a side effect, or only provide advisory information?
- What idempotency/replay key proves that a retry is the same business intent?
- What evidence is retained off-chain and what stable reference/hash belongs on the ledger?

Chapter 266 Implementation - Operator Access Review

Implementation Procedure

1326. Create operator runbooks with prechecks, exact commands, expected output, rollback, and verification.
1327. Monitor quorum, ledger height, certificate expiry, reconciliation, API latency, model health, and storage.
1328. Back up and restore on a schedule; drill it.
1329. Promote models/chaincode/config with signed manifests and rollback points.
1330. Review incidents, overrides, fraud metrics, and access logs routinely.

Implementation Example

```
# Operator Access Review
INPUT  -> authenticate, validate schema, bind requestId/nonce
FACTS  -> read authoritative wallet/merchant/state data
AI      -> optional bounded recommendation with decisionId/version
POLICY -> enforce ownership, hard limits, approvals, risk fallback
LEDGER  -> submit one idempotent deterministic transaction when value changes
EVENT   -> update projection/case/monitoring asynchronously
VERIFY  -> discover committed result and retain correlationId + txId evidence
```

Verification Checklist

- The access review exists and is version-controlled.
- The feature works with all public Internet/WAN connectivity disabled.
- Authorization, malformed input, duplicate retry, and failure behavior have explicit tests.
- If financial value changes, journal debits and credits balance exactly and historical entries remain immutable.
- If AI is involved, model/prompt/feature version and fallback behavior are recorded and deterministic policy remains final.
- A timeout/restart cannot create a duplicate financial result; request status can be rediscovered.
- Logs/evidence contain correlation references but not secrets or prohibited sensitive payment data.

Common Failure Modes

- Runbooks depend on one developer.
- Certificate/model/version inventory stale.
- Reconciliation alerts ignored.
- No rollback for model or chaincode deployment.

Expert Review Questions

- What happens if this component succeeds but the next component fails?
- How is the authorizing identity/device/key revoked or recovered?
- Can the operation be safely retried after the client loses the response?
- What happens if AI is unavailable, slow, or wrong?
- Can an auditor reconstruct debit, credit, actor, risk decision, approval, and Fabric txId from retained evidence?
- Which compensating transaction resolves a downstream operational error without rewriting history?

Chapter 267 - Key and Secret Inventory Review

Ensure every active secret/key has owner, purpose, expiry/rotation, backup status. A production runbook must work when the original developer is absent and the Internet is unavailable. Operators need exact startup/shutdown, health, reconciliation, backup, restore, certificate rotation, model rollback, chaincode upgrade, incident, and offline patching procedures.

Design rule: Fabric certs

Implementation Objective

The required output is: Key inventory report. It must be version-controlled, reviewable, testable with the WAN disabled, and linked to the architecture/source-of-truth model. For a payment platform, every implementation decision should make it possible to explain who authorized an action, which facts were trusted, which controls ran, what financial journal committed, and how the result is recovered after failure.

Key Engineering Decisions

- Fabric certs
- TLS keys
- DB secrets
- device keys
- HSM objects
- backup encryption keys
- model signing keys if used

Deep Implementation Guidance

Runbooks must be executable by another trained operator. Include prechecks, exact commands, expected output, stop conditions, rollback, verification, and evidence to retain. Periodically drill restore, revocation, model rollback, and quorum loss.

Data and Control Questions

- What is the authoritative source for every input used by this component?
- Which user, device, service, MSP, or administrator identity is allowed to initiate or approve it?
- Does the operation move value, reserve value, create a side effect, or only provide advisory information?
- What idempotency/replay key proves that a retry is the same business intent?
- What evidence is retained off-chain and what stable reference/hash belongs on the ledger?

Chapter 267 Implementation - Key and Secret Inventory Review

Implementation Procedure

1331. Create operator runbooks with prechecks, exact commands, expected output, rollback, and verification.
1332. Monitor quorum, ledger height, certificate expiry, reconciliation, API latency, model health, and storage.
1333. Back up and restore on a schedule; drill it.
1334. Promote models/chaincode/config with signed manifests and rollback points.
1335. Review incidents, overrides, fraud metrics, and access logs routinely.

Implementation Example

```
# Key and Secret Inventory Review
INPUT  -> authenticate, validate schema, bind requestId/nonce
FACTS  -> read authoritative wallet/merchant/state data
AI      -> optional bounded recommendation with decisionId/version
POLICY  -> enforce ownership, hard limits, approvals, risk fallback
LEDGER  -> submit one idempotent deterministic transaction when value changes
EVENT   -> update projection/case/monitoring asynchronously
VERIFY  -> discover committed result and retain correlationId + txId evidence
```

Verification Checklist

- The key inventory report exists and is version-controlled.
- The feature works with all public Internet/WAN connectivity disabled.
- Authorization, malformed input, duplicate retry, and failure behavior have explicit tests.
- If financial value changes, journal debits and credits balance exactly and historical entries remain immutable.
- If AI is involved, model/prompt/feature version and fallback behavior are recorded and deterministic policy remains final.
- A timeout/restart cannot create a duplicate financial result; request status can be rediscovered.
- Logs/evidence contain correlation references but not secrets or prohibited sensitive payment data.

Common Failure Modes

- Runbooks depend on one developer.
- Certificate/model/version inventory stale.
- Reconciliation alerts ignored.
- No rollback for model or chaincode deployment.

Expert Review Questions

- What happens if this component succeeds but the next component fails?
- How is the authorizing identity/device/key revoked or recovered?
- Can the operation be safely retried after the client loses the response?
- What happens if AI is unavailable, slow, or wrong?
- Can an auditor reconstruct debit, credit, actor, risk decision, approval, and Fabric txId from retained evidence?
- Which compensating transaction resolves a downstream operational error without rewriting history?

Chapter 268 - Daily Close Runbook

Produce financial/audit evidence and investigate exceptions. A production runbook must work when the original developer is absent and the Internet is unavailable. Operators need exact startup/shutdown, health, reconciliation, backup, restore, certificate rotation, model rollback, chaincode upgrade, incident, and offline patching procedures.

Design rule: Run reconciliation

Implementation Objective

The required output is: Close procedure. It must be version-controlled, reviewable, testable with the WAN disabled, and linked to the architecture/source-of-truth model. For a payment platform, every implementation decision should make it possible to explain who authorized an action, which facts were trusted, which controls ran, what financial journal committed, and how the result is recovered after failure.

Key Engineering Decisions

- Run reconciliation
- settlement summary
- AI review exceptions
- sign/hash report package
- escalate unresolved mismatch

Deep Implementation Guidance

Runbooks must be executable by another trained operator. Include prechecks, exact commands, expected output, stop conditions, rollback, verification, and evidence to retain. Periodically drill restore, revocation, model rollback, and quorum loss.

Data and Control Questions

- What is the authoritative source for every input used by this component?
- Which user, device, service, MSP, or administrator identity is allowed to initiate or approve it?
- Does the operation move value, reserve value, create a side effect, or only provide advisory information?
- What idempotency/replay key proves that a retry is the same business intent?
- What evidence is retained off-chain and what stable reference/hash belongs on the ledger?

Chapter 268 Implementation - Daily Close Runbook

Implementation Procedure

- 1336. Create operator runbooks with prechecks, exact commands, expected output, rollback, and verification.
- 1337. Monitor quorum, ledger height, certificate expiry, reconciliation, API latency, model health, and storage.
- 1338. Back up and restore on a schedule; drill it.
- 1339. Promote models/chaincode/config with signed manifests and rollback points.
- 1340. Review incidents, overrides, fraud metrics, and access logs routinely.

Implementation Example

```
# Daily Close Runbook
INPUT  -> authenticate, validate schema, bind requestId/nonce
FACTS  -> read authoritative wallet/merchant/state data
AI      -> optional bounded recommendation with decisionId/version
POLICY  -> enforce ownership, hard limits, approvals, risk fallback
LEDGER  -> submit one idempotent deterministic transaction when value changes
EVENT   -> update projection/case/monitoring asynchronously
VERIFY  -> discover committed result and retain correlationId + txId evidence
```

Verification Checklist

- The close procedure exists and is version-controlled.
- The feature works with all public Internet/WAN connectivity disabled.
- Authorization, malformed input, duplicate retry, and failure behavior have explicit tests.
- If financial value changes, journal debits and credits balance exactly and historical entries remain immutable.
- If AI is involved, model/prompt/feature version and fallback behavior are recorded and deterministic policy remains final.
- A timeout/restart cannot create a duplicate financial result; request status can be rediscovered.
- Logs/evidence contain correlation references but not secrets or prohibited sensitive payment data.

Common Failure Modes

- Runbooks depend on one developer.
- Certificate/model/version inventory stale.
- Reconciliation alerts ignored.
- No rollback for model or chaincode deployment.

Expert Review Questions

- What happens if this component succeeds but the next component fails?
- How is the authorizing identity/device/key revoked or recovered?
- Can the operation be safely retried after the client loses the response?
- What happens if AI is unavailable, slow, or wrong?
- Can an auditor reconstruct debit, credit, actor, risk decision, approval, and Fabric txId from retained evidence?
- Which compensating transaction resolves a downstream operational error without rewriting history?

Chapter 269 - Disaster Recovery Exercise

Practice complete site/host recovery before an emergency. A production runbook must work when the original developer is absent and the Internet is unavailable. Operators need exact startup/shutdown, health, reconciliation, backup, restore, certificate rotation, model rollback, chaincode upgrade, incident, and offline patching procedures.

Design rule: Declare scenario

Implementation Objective

The required output is: DR drill. It must be version-controlled, reviewable, testable with the WAN disabled, and linked to the architecture/source-of-truth model. For a payment platform, every implementation decision should make it possible to explain who authorized an action, which facts were trusted, which controls ran, what financial journal committed, and how the result is recovered after failure.

Key Engineering Decisions

- Declare scenario
- restore network/services/data/identities/models
- reconcile
- measure RTO/RPO
- update gaps

Deep Implementation Guidance

Runbooks must be executable by another trained operator. Include prechecks, exact commands, expected output, stop conditions, rollback, verification, and evidence to retain. Periodically drill restore, revocation, model rollback, and quorum loss.

Data and Control Questions

- What is the authoritative source for every input used by this component?
- Which user, device, service, MSP, or administrator identity is allowed to initiate or approve it?
- Does the operation move value, reserve value, create a side effect, or only provide advisory information?
- What idempotency/replay key proves that a retry is the same business intent?
- What evidence is retained off-chain and what stable reference/hash belongs on the ledger?

Chapter 269 Implementation - Disaster Recovery Exercise

Implementation Procedure

1341. Create operator runbooks with prechecks, exact commands, expected output, rollback, and verification.
1342. Monitor quorum, ledger height, certificate expiry, reconciliation, API latency, model health, and storage.
1343. Back up and restore on a schedule; drill it.
1344. Promote models/chaincode/config with signed manifests and rollback points.
1345. Review incidents, overrides, fraud metrics, and access logs routinely.

Implementation Example

```
# Disaster Recovery Exercise
INPUT  -> authenticate, validate schema, bind requestId/nonce
FACTS  -> read authoritative wallet/merchant/state data
AI     -> optional bounded recommendation with decisionId/version
POLICY -> enforce ownership, hard limits, approvals, risk fallback
LEDGER -> submit one idempotent deterministic transaction when value changes
EVENT  -> update projection/case/monitoring asynchronously
VERIFY -> discover committed result and retain correlationId + txId evidence
```

Verification Checklist

- The dr drill exists and is version-controlled.
- The feature works with all public Internet/WAN connectivity disabled.
- Authorization, malformed input, duplicate retry, and failure behavior have explicit tests.
- If financial value changes, journal debits and credits balance exactly and historical entries remain immutable.
- If AI is involved, model/prompt/feature version and fallback behavior are recorded and deterministic policy remains final.
- A timeout/restart cannot create a duplicate financial result; request status can be rediscovered.
- Logs/evidence contain correlation references but not secrets or prohibited sensitive payment data.

Common Failure Modes

- Runbooks depend on one developer.
- Certificate/model/version inventory stale.
- Reconciliation alerts ignored.
- No rollback for model or chaincode deployment.

Expert Review Questions

- What happens if this component succeeds but the next component fails?
- How is the authorizing identity/device/key revoked or recovered?
- Can the operation be safely retried after the client loses the response?
- What happens if AI is unavailable, slow, or wrong?
- Can an auditor reconstruct debit, credit, actor, risk decision, approval, and Fabric txId from retained evidence?
- Which compensating transaction resolves a downstream operational error without rewriting history?

Chapter 270 - Decommissioning

Retire hosts, devices, wallets, models, and keys without losing required audit evidence. A production runbook must work when the original developer is absent and the Internet is unavailable. Operators need exact startup/shutdown, health, reconciliation, backup, restore, certificate rotation, model rollback, chaincode upgrade, incident, and offline patching procedures.

Design rule: Revoke identity

Implementation Objective

The required output is: Decommission SOP. It must be version-controlled, reviewable, testable with the WAN disabled, and linked to the architecture/source-of-truth model. For a payment platform, every implementation decision should make it possible to explain who authorized an action, which facts were trusted, which controls ran, what financial journal committed, and how the result is recovered after failure.

Key Engineering Decisions

- Revoke identity
- remove secrets
- wipe media
- retain required ledger/audit backups
- update inventories

Deep Implementation Guidance

Runbooks must be executable by another trained operator. Include prechecks, exact commands, expected output, stop conditions, rollback, verification, and evidence to retain. Periodically drill restore, revocation, model rollback, and quorum loss.

Data and Control Questions

- What is the authoritative source for every input used by this component?
- Which user, device, service, MSP, or administrator identity is allowed to initiate or approve it?
- Does the operation move value, reserve value, create a side effect, or only provide advisory information?
- What idempotency/replay key proves that a retry is the same business intent?
- What evidence is retained off-chain and what stable reference/hash belongs on the ledger?

Chapter 270 Implementation - Decommissioning

Implementation Procedure

1346. Create operator runbooks with prechecks, exact commands, expected output, rollback, and verification.
1347. Monitor quorum, ledger height, certificate expiry, reconciliation, API latency, model health, and storage.
1348. Back up and restore on a schedule; drill it.
1349. Promote models/chaincode/config with signed manifests and rollback points.
1350. Review incidents, overrides, fraud metrics, and access logs routinely.

Implementation Example

```
# Decommissioning
INPUT  -> authenticate, validate schema, bind requestId/nonce
FACTS  -> read authoritative wallet/merchant/state data
AI      -> optional bounded recommendation with decisionId/version
POLICY  -> enforce ownership, hard limits, approvals, risk fallback
LEDGER  -> submit one idempotent deterministic transaction when value changes
EVENT   -> update projection/case/monitoring asynchronously
VERIFY  -> discover committed result and retain correlationId + txId evidence
```

Verification Checklist

- The decommission sop exists and is version-controlled.
- The feature works with all public Internet/WAN connectivity disabled.
- Authorization, malformed input, duplicate retry, and failure behavior have explicit tests.
- If financial value changes, journal debits and credits balance exactly and historical entries remain immutable.
- If AI is involved, model/prompt/feature version and fallback behavior are recorded and deterministic policy remains final.
- A timeout/restart cannot create a duplicate financial result; request status can be rediscovered.
- Logs/evidence contain correlation references but not secrets or prohibited sensitive payment data.

Common Failure Modes

- Runbooks depend on one developer.
- Certificate/model/version inventory stale.
- Reconciliation alerts ignored.
- No rollback for model or chaincode deployment.

Expert Review Questions

- What happens if this component succeeds but the next component fails?
- How is the authorizing identity/device/key revoked or recovered?
- Can the operation be safely retried after the client loses the response?
- What happens if AI is unavailable, slow, or wrong?
- Can an auditor reconstruct debit, credit, actor, risk decision, approval, and Fabric txId from retained evidence?
- Which compensating transaction resolves a downstream operational error without rewriting history?

Part XV - Performance, High Availability, Multi-Site, and Advanced Design

Scale only after measuring the bottleneck. Fabric ordering, peer endorsement, CouchDB access, PostgreSQL projections, AI inference, local LLM context, and UI concurrency all have different capacity characteristics. Add complexity only for a measured availability, governance, security, or performance need.

Part focus: This part contains 19 implementation chapters. Each chapter includes design rules, implementation procedure, a practical artifact, verification, failure analysis, and expert review questions.

Chapter 271 - Performance Measurement Methodology

Measure the end-to-end financial path before tuning. Scale only after measuring the bottleneck. Fabric ordering, peer endorsement, CouchDB access, PostgreSQL projections, AI inference, local LLM context, and UI concurrency all have different capacity characteristics. Add complexity only for a measured availability, governance, security, or performance need.

Design rule: Separate API auth/risk/Gateway endorsement/order/commit/projection latency

Implementation Objective

The required output is: Benchmark plan. It must be version-controlled, reviewable, testable with the WAN disabled, and linked to the architecture/source-of-truth model. For a payment platform, every implementation decision should make it possible to explain who authorized an action, which facts were trusted, which controls ran, what financial journal committed, and how the result is recovered after failure.

Key Engineering Decisions

- Separate API auth/risk/Gateway endorsement/order/commit/projection latency
- use realistic payload/concurrency
- report percentiles

Deep Implementation Guidance

Prefer the simplest topology that satisfies the trust and recovery model. Every new site, asset, peer, channel, HSM, model, queue, and HA layer adds operational failure modes. Require measured benefit and a tested rollback plan.

Data and Control Questions

- What is the authoritative source for every input used by this component?
- Which user, device, service, MSP, or administrator identity is allowed to initiate or approve it?
- Does the operation move value, reserve value, create a side effect, or only provide advisory information?
- What idempotency/replay key proves that a retry is the same business intent?
- What evidence is retained off-chain and what stable reference/hash belongs on the ledger?

Chapter 271 Implementation - Performance Measurement Methodology

Implementation Procedure

1351. Baseline end-to-end latency and throughput before tuning.
1352. Measure each component separately and identify the dominant bottleneck.
1353. Use HA/redundancy only where the business RTO/availability requires it.
1354. Test scale changes under realistic concurrency and failure.
1355. Keep migration/rollback evidence for topology, Fabric, model, and database upgrades.

Implementation Example

```
# Performance Measurement Methodology
INPUT  -> authenticate, validate schema, bind requestId/nonce
FACTS  -> read authoritative wallet/merchant/state data
AI     -> optional bounded recommendation with decisionId/version
POLICY -> enforce ownership, hard limits, approvals, risk fallback
LEDGER -> submit one idempotent deterministic transaction when value changes
EVENT  -> update projection/case/monitoring asynchronously
VERIFY -> discover committed result and retain correlationId + txId evidence
```

Verification Checklist

- The benchmark plan exists and is version-controlled.
- The feature works with all public Internet/WAN connectivity disabled.
- Authorization, malformed input, duplicate retry, and failure behavior have explicit tests.
- If financial value changes, journal debits and credits balance exactly and historical entries remain immutable.
- If AI is involved, model/prompt/feature version and fallback behavior are recorded and deterministic policy remains final.
- A timeout/restart cannot create a duplicate financial result; request status can be rediscovered.
- Logs/evidence contain correlation references but not secrets or prohibited sensitive payment data.

Common Failure Modes

- Premature multi-channel/multi-agent complexity.
- Capacity claims without measurement.
- Disconnected-spend mode without strict risk caps.
- Migration without rollback rehearsal.

Expert Review Questions

- What happens if this component succeeds but the next component fails?
- How is the authorizing identity/device/key revoked or recovered?
- Can the operation be safely retried after the client loses the response?
- What happens if AI is unavailable, slow, or wrong?
- Can an auditor reconstruct debit, credit, actor, risk decision, approval, and Fabric txId from retained evidence?
- Which compensating transaction resolves a downstream operational error without rewriting history?

Chapter 272 - Fabric Throughput Tuning

Tune block, endorsement, peer, CouchDB, and resource settings only from evidence. Scale only after measuring the bottleneck. Fabric ordering, peer endorsement, CouchDB access, PostgreSQL projections, AI inference, local LLM context, and UI concurrency all have different capacity characteristics. Add complexity only for a measured availability, governance, security, or performance need.

Design rule: Measure bottleneck

Implementation Objective

The required output is: Fabric tuning plan. It must be version-controlled, reviewable, testable with the WAN disabled, and linked to the architecture/source-of-truth model. For a payment platform, every implementation decision should make it possible to explain who authorized an action, which facts were trusted, which controls ran, what financial journal committed, and how the result is recovered after failure.

Key Engineering Decisions

- Measure bottleneck
- avoid huge blocks for latency-sensitive payments
- index state queries
- dedicate resources

Deep Implementation Guidance

Prefer the simplest topology that satisfies the trust and recovery model. Every new site, asset, peer, channel, HSM, model, queue, and HA layer adds operational failure modes. Require measured benefit and a tested rollback plan.

Data and Control Questions

- What is the authoritative source for every input used by this component?
- Which user, device, service, MSP, or administrator identity is allowed to initiate or approve it?
- Does the operation move value, reserve value, create a side effect, or only provide advisory information?
- What idempotency/replay key proves that a retry is the same business intent?
- What evidence is retained off-chain and what stable reference/hash belongs on the ledger?

Chapter 272 Implementation - Fabric Throughput Tuning

Implementation Procedure

1356. Baseline end-to-end latency and throughput before tuning.
1357. Measure each component separately and identify the dominant bottleneck.
1358. Use HA/redundancy only where the business RTO/availability requires it.
1359. Test scale changes under realistic concurrency and failure.
1360. Keep migration/rollback evidence for topology, Fabric, model, and database upgrades.

Implementation Example

```
# Fabric Throughput Tuning
INPUT  -> authenticate, validate schema, bind requestId/nonce
FACTS  -> read authoritative wallet/merchant/state data
AI     -> optional bounded recommendation with decisionId/version
POLICY -> enforce ownership, hard limits, approvals, risk fallback
LEDGER -> submit one idempotent deterministic transaction when value changes
EVENT  -> update projection/case/monitoring asynchronously
VERIFY -> discover committed result and retain correlationId + txId evidence
```

Verification Checklist

- The fabric tuning plan exists and is version-controlled.
- The feature works with all public Internet/WAN connectivity disabled.
- Authorization, malformed input, duplicate retry, and failure behavior have explicit tests.
- If financial value changes, journal debits and credits balance exactly and historical entries remain immutable.
- If AI is involved, model/prompt/feature version and fallback behavior are recorded and deterministic policy remains final.
- A timeout/restart cannot create a duplicate financial result; request status can be rediscovered.
- Logs/evidence contain correlation references but not secrets or prohibited sensitive payment data.

Common Failure Modes

- Premature multi-channel/multi-agent complexity.
- Capacity claims without measurement.
- Disconnected-spend mode without strict risk caps.
- Migration without rollback rehearsal.

Expert Review Questions

- What happens if this component succeeds but the next component fails?
- How is the authorizing identity/device/key revoked or recovered?
- Can the operation be safely retried after the client loses the response?
- What happens if AI is unavailable, slow, or wrong?
- Can an auditor reconstruct debit, credit, actor, risk decision, approval, and Fabric txId from retained evidence?
- Which compensating transaction resolves a downstream operational error without rewriting history?

Chapter 273 - API Scaling

Scale stateless application components without duplicating financial actions. Scale only after measuring the bottleneck. Fabric ordering, peer endorsement, CouchDB access, PostgreSQL projections, AI inference, local LLM context, and UI concurrency all have different capacity characteristics. Add complexity only for a measured availability, governance, security, or performance need.

Design rule: Multiple API instances behind local load balancer

Implementation Objective

The required output is: API HA design. It must be version-controlled, reviewable, testable with the WAN disabled, and linked to the architecture/source-of-truth model. For a payment platform, every implementation decision should make it possible to explain who authorized an action, which facts were trusted, which controls ran, what financial journal committed, and how the result is recovered after failure.

Key Engineering Decisions

- Multiple API instances behind local load balancer
- shared session/idempotency store
- each uses Gateway safely
- health/readiness

Deep Implementation Guidance

Prefer the simplest topology that satisfies the trust and recovery model. Every new site, asset, peer, channel, HSM, model, queue, and HA layer adds operational failure modes. Require measured benefit and a tested rollback plan.

Data and Control Questions

- What is the authoritative source for every input used by this component?
- Which user, device, service, MSP, or administrator identity is allowed to initiate or approve it?
- Does the operation move value, reserve value, create a side effect, or only provide advisory information?
- What idempotency/replay key proves that a retry is the same business intent?
- What evidence is retained off-chain and what stable reference/hash belongs on the ledger?

Chapter 273 Implementation - API Scaling

Implementation Procedure

1361. Baseline end-to-end latency and throughput before tuning.
1362. Measure each component separately and identify the dominant bottleneck.
1363. Use HA/redundancy only where the business RTO/availability requires it.
1364. Test scale changes under realistic concurrency and failure.
1365. Keep migration/rollback evidence for topology, Fabric, model, and database upgrades.

Implementation Example

```
# API Scaling
INPUT  -> authenticate, validate schema, bind requestId/nonce
FACTS  -> read authoritative wallet/merchant/state data
AI      -> optional bounded recommendation with decisionId/version
POLICY  -> enforce ownership, hard limits, approvals, risk fallback
LEDGER  -> submit one idempotent deterministic transaction when value changes
EVENT   -> update projection/case/monitoring asynchronously
VERIFY  -> discover committed result and retain correlationId + txId evidence
```

Verification Checklist

- The api ha design exists and is version-controlled.
- The feature works with all public Internet/WAN connectivity disabled.
- Authorization, malformed input, duplicate retry, and failure behavior have explicit tests.
- If financial value changes, journal debits and credits balance exactly and historical entries remain immutable.
- If AI is involved, model/prompt/feature version and fallback behavior are recorded and deterministic policy remains final.
- A timeout/restart cannot create a duplicate financial result; request status can be rediscovered.
- Logs/evidence contain correlation references but not secrets or prohibited sensitive payment data.

Common Failure Modes

- Premature multi-channel/multi-agent complexity.
- Capacity claims without measurement.
- Disconnected-spend mode without strict risk caps.
- Migration without rollback rehearsal.

Expert Review Questions

- What happens if this component succeeds but the next component fails?
- How is the authorizing identity/device/key revoked or recovered?
- Can the operation be safely retried after the client loses the response?
- What happens if AI is unavailable, slow, or wrong?
- Can an auditor reconstruct debit, credit, actor, risk decision, approval, and Fabric txId from retained evidence?
- Which compensating transaction resolves a downstream operational error without rewriting history?

Chapter 274 - PostgreSQL High Availability

Choose local database redundancy according to RTO/RPO and operator skill. Scale only after measuring the bottleneck. Fabric ordering, peer endorsement, CouchDB access, PostgreSQL projections, AI inference, local LLM context, and UI concurrency all have different capacity characteristics. Add complexity only for a measured availability, governance, security, or performance need.

Design rule: Primary/standby if needed

Implementation Objective

The required output is: DB HA ADR. It must be version-controlled, reviewable, testable with the WAN disabled, and linked to the architecture/source-of-truth model. For a payment platform, every implementation decision should make it possible to explain who authorized an action, which facts were trusted, which controls ran, what financial journal committed, and how the result is recovered after failure.

Key Engineering Decisions

- Primary/standby if needed
- backups still required
- projection can rebuild
- PII/case data may require stronger recovery than projection tables

Deep Implementation Guidance

Prefer the simplest topology that satisfies the trust and recovery model. Every new site, asset, peer, channel, HSM, model, queue, and HA layer adds operational failure modes. Require measured benefit and a tested rollback plan.

Data and Control Questions

- What is the authoritative source for every input used by this component?
- Which user, device, service, MSP, or administrator identity is allowed to initiate or approve it?
- Does the operation move value, reserve value, create a side effect, or only provide advisory information?
- What idempotency/replay key proves that a retry is the same business intent?
- What evidence is retained off-chain and what stable reference/hash belongs on the ledger?

Chapter 274 Implementation - PostgreSQL High Availability

Implementation Procedure

- 1366. Baseline end-to-end latency and throughput before tuning.
- 1367. Measure each component separately and identify the dominant bottleneck.
- 1368. Use HA/redundancy only where the business RTO/availability requires it.
- 1369. Test scale changes under realistic concurrency and failure.
- 1370. Keep migration/rollback evidence for topology, Fabric, model, and database upgrades.

Implementation Example

```
# PostgreSQL High Availability
INPUT  -> authenticate, validate schema, bind requestId/nonce
FACTS  -> read authoritative wallet/merchant/state data
AI      -> optional bounded recommendation with decisionId/version
POLICY  -> enforce ownership, hard limits, approvals, risk fallback
LEDGER  -> submit one idempotent deterministic transaction when value changes
EVENT   -> update projection/case/monitoring asynchronously
VERIFY  -> discover committed result and retain correlationId + txId evidence
```

Verification Checklist

- The db ha adr exists and is version-controlled.
- The feature works with all public Internet/WAN connectivity disabled.
- Authorization, malformed input, duplicate retry, and failure behavior have explicit tests.
- If financial value changes, journal debits and credits balance exactly and historical entries remain immutable.
- If AI is involved, model/prompt/feature version and fallback behavior are recorded and deterministic policy remains final.
- A timeout/restart cannot create a duplicate financial result; request status can be rediscovered.
- Logs/evidence contain correlation references but not secrets or prohibited sensitive payment data.

Common Failure Modes

- Premature multi-channel/multi-agent complexity.
- Capacity claims without measurement.
- Disconnected-spend mode without strict risk caps.
- Migration without rollback rehearsal.

Expert Review Questions

- What happens if this component succeeds but the next component fails?
- How is the authorizing identity/device/key revoked or recovered?
- Can the operation be safely retried after the client loses the response?
- What happens if AI is unavailable, slow, or wrong?
- Can an auditor reconstruct debit, credit, actor, risk decision, approval, and Fabric txId from retained evidence?
- Which compensating transaction resolves a downstream operational error without rewriting history?

Chapter 275 - AI Inference Capacity

Size fraud inference so risk decisions stay within payment latency budget. Scale only after measuring the bottleneck. Fabric ordering, peer endorsement, CouchDB access, PostgreSQL projections, AI inference, local LLM context, and UI concurrency all have different capacity characteristics. Add complexity only for a measured availability, governance, security, or performance need.

Design rule: Batching usually not on critical path

Implementation Objective

The required output is: Risk capacity plan. It must be version-controlled, reviewable, testable with the WAN disabled, and linked to the architecture/source-of-truth model. For a payment platform, every implementation decision should make it possible to explain who authorized an action, which facts were trusted, which controls ran, what financial journal committed, and how the result is recovered after failure.

Key Engineering Decisions

- Batching usually not on critical path
- use lightweight model
- CPU vs GPU benchmark
- bounded queue
- fallback on overload

Deep Implementation Guidance

Prefer the simplest topology that satisfies the trust and recovery model. Every new site, asset, peer, channel, HSM, model, queue, and HA layer adds operational failure modes. Require measured benefit and a tested rollback plan.

Data and Control Questions

- What is the authoritative source for every input used by this component?
- Which user, device, service, MSP, or administrator identity is allowed to initiate or approve it?
- Does the operation move value, reserve value, create a side effect, or only provide advisory information?
- What idempotency/replay key proves that a retry is the same business intent?
- What evidence is retained off-chain and what stable reference/hash belongs on the ledger?

Chapter 275 Implementation - AI Inference Capacity

Implementation Procedure

1371. Baseline end-to-end latency and throughput before tuning.
1372. Measure each component separately and identify the dominant bottleneck.
1373. Use HA/redundancy only where the business RTO/availability requires it.
1374. Test scale changes under realistic concurrency and failure.
1375. Keep migration/rollback evidence for topology, Fabric, model, and database upgrades.

Implementation Example

```
# AI Inference Capacity
INPUT  -> authenticate, validate schema, bind requestId/nonce
FACTS  -> read authoritative wallet/merchant/state data
AI     -> optional bounded recommendation with decisionId/version
POLICY -> enforce ownership, hard limits, approvals, risk fallback
LEDGER -> submit one idempotent deterministic transaction when value changes
EVENT  -> update projection/case/monitoring asynchronously
VERIFY -> discover committed result and retain correlationId + txId evidence
```

Verification Checklist

- The risk capacity plan exists and is version-controlled.
- The feature works with all public Internet/WAN connectivity disabled.
- Authorization, malformed input, duplicate retry, and failure behavior have explicit tests.
- If financial value changes, journal debits and credits balance exactly and historical entries remain immutable.
- If AI is involved, model/prompt/feature version and fallback behavior are recorded and deterministic policy remains final.
- A timeout/restart cannot create a duplicate financial result; request status can be rediscovered.
- Logs/evidence contain correlation references but not secrets or prohibited sensitive payment data.

Common Failure Modes

- Premature multi-channel/multi-agent complexity.
- Capacity claims without measurement.
- Disconnected-spend mode without strict risk caps.
- Migration without rollback rehearsal.

Expert Review Questions

- What happens if this component succeeds but the next component fails?
- How is the authorizing identity/device/key revoked or recovered?
- Can the operation be safely retried after the client loses the response?
- What happens if AI is unavailable, slow, or wrong?
- Can an auditor reconstruct debit, credit, actor, risk decision, approval, and Fabric txId from retained evidence?
- Which compensating transaction resolves a downstream operational error without rewriting history?

Chapter 276 - Local LLM Capacity Planning

Prevent support/ops LLM workloads from harming payment infrastructure. Scale only after measuring the bottleneck. Fabric ordering, peer endorsement, CouchDB access, PostgreSQL projections, AI inference, local LLM context, and UI concurrency all have different capacity characteristics. Add complexity only for a measured availability, governance, security, or performance need.

Design rule: Separate host/GPU where possible

Implementation Objective

The required output is: LLM capacity plan. It must be version-controlled, reviewable, testable with the WAN disabled, and linked to the architecture/source-of-truth model. For a payment platform, every implementation decision should make it possible to explain who authorized an action, which facts were trusted, which controls ran, what financial journal committed, and how the result is recovered after failure.

Key Engineering Decisions

- Separate host/GPU where possible
- limit concurrency/context
- priority queue
- payment path never waits on general assistant

Deep Implementation Guidance

Prefer the simplest topology that satisfies the trust and recovery model. Every new site, asset, peer, channel, HSM, model, queue, and HA layer adds operational failure modes. Require measured benefit and a tested rollback plan.

Data and Control Questions

- What is the authoritative source for every input used by this component?
- Which user, device, service, MSP, or administrator identity is allowed to initiate or approve it?
- Does the operation move value, reserve value, create a side effect, or only provide advisory information?
- What idempotency/replay key proves that a retry is the same business intent?
- What evidence is retained off-chain and what stable reference/hash belongs on the ledger?

Chapter 276 Implementation - Local LLM Capacity Planning

Implementation Procedure

- 1376. Baseline end-to-end latency and throughput before tuning.
- 1377. Measure each component separately and identify the dominant bottleneck.
- 1378. Use HA/redundancy only where the business RTO/availability requires it.
- 1379. Test scale changes under realistic concurrency and failure.
- 1380. Keep migration/rollback evidence for topology, Fabric, model, and database upgrades.

Implementation Example

```
# Local LLM Capacity Planning
INPUT  -> authenticate, validate schema, bind requestId/nonce
FACTS  -> read authoritative wallet/merchant/state data
AI     -> optional bounded recommendation with decisionId/version
POLICY -> enforce ownership, hard limits, approvals, risk fallback
LEDGER -> submit one idempotent deterministic transaction when value changes
EVENT  -> update projection/case/monitoring asynchronously
VERIFY -> discover committed result and retain correlationId + txId evidence
```

Verification Checklist

- The llm capacity plan exists and is version-controlled.
- The feature works with all public Internet/WAN connectivity disabled.
- Authorization, malformed input, duplicate retry, and failure behavior have explicit tests.
- If financial value changes, journal debits and credits balance exactly and historical entries remain immutable.
- If AI is involved, model/prompt/feature version and fallback behavior are recorded and deterministic policy remains final.
- A timeout/restart cannot create a duplicate financial result; request status can be rediscovered.
- Logs/evidence contain correlation references but not secrets or prohibited sensitive payment data.

Common Failure Modes

- Premature multi-channel/multi-agent complexity.
- Capacity claims without measurement.
- Disconnected-spend mode without strict risk caps.
- Migration without rollback rehearsal.

Expert Review Questions

- What happens if this component succeeds but the next component fails?
- How is the authorizing identity/device/key revoked or recovered?
- Can the operation be safely retried after the client loses the response?
- What happens if AI is unavailable, slow, or wrong?
- Can an auditor reconstruct debit, credit, actor, risk decision, approval, and Fabric txId from retained evidence?
- Which compensating transaction resolves a downstream operational error without rewriting history?

Chapter 277 - Multi-Branch LAN/WAN Design

Extend to branches while retaining controlled local operation. Scale only after measuring the bottleneck. Fabric ordering, peer endorsement, CouchDB access, PostgreSQL projections, AI inference, local LLM context, and UI concurrency all have different capacity characteristics. Add complexity only for a measured availability, governance, security, or performance need.

Design rule: Define whether branches share Fabric channel over private WAN or run local domains

Implementation Objective

The required output is: Branch architecture ADR. It must be version-controlled, reviewable, testable with the WAN disabled, and linked to the architecture/source-of-truth model. For a payment platform, every implementation decision should make it possible to explain who authorized an action, which facts were trusted, which controls ran, what financial journal committed, and how the result is recovered after failure.

Key Engineering Decisions

- Define whether branches share Fabric channel over private WAN or run local domains
- plan partitions/reconciliation
- do not assume Internet

Deep Implementation Guidance

Prefer the simplest topology that satisfies the trust and recovery model. Every new site, asset, peer, channel, HSM, model, queue, and HA layer adds operational failure modes. Require measured benefit and a tested rollback plan.

Data and Control Questions

- What is the authoritative source for every input used by this component?
- Which user, device, service, MSP, or administrator identity is allowed to initiate or approve it?
- Does the operation move value, reserve value, create a side effect, or only provide advisory information?
- What idempotency/replay key proves that a retry is the same business intent?
- What evidence is retained off-chain and what stable reference/hash belongs on the ledger?

Chapter 277 Implementation - Multi-Branch LAN/WAN Design

Implementation Procedure

1381. Baseline end-to-end latency and throughput before tuning.
1382. Measure each component separately and identify the dominant bottleneck.
1383. Use HA/redundancy only where the business RTO/availability requires it.
1384. Test scale changes under realistic concurrency and failure.
1385. Keep migration/rollback evidence for topology, Fabric, model, and database upgrades.

Implementation Example

```
# Multi-Branch LAN/WAN Design
INPUT  -> authenticate, validate schema, bind requestId/nonce
FACTS  -> read authoritative wallet/merchant/state data
AI      -> optional bounded recommendation with decisionId/version
POLICY  -> enforce ownership, hard limits, approvals, risk fallback
LEDGER  -> submit one idempotent deterministic transaction when value changes
EVENT   -> update projection/case/monitoring asynchronously
VERIFY  -> discover committed result and retain correlationId + txId evidence
```

Verification Checklist

- The branch architecture adr exists and is version-controlled.
- The feature works with all public Internet/WAN connectivity disabled.
- Authorization, malformed input, duplicate retry, and failure behavior have explicit tests.
- If financial value changes, journal debits and credits balance exactly and historical entries remain immutable.
- If AI is involved, model/prompt/feature version and fallback behavior are recorded and deterministic policy remains final.
- A timeout/restart cannot create a duplicate financial result; request status can be rediscovered.
- Logs/evidence contain correlation references but not secrets or prohibited sensitive payment data.

Common Failure Modes

- Premature multi-channel/multi-agent complexity.
- Capacity claims without measurement.
- Disconnected-spend mode without strict risk caps.
- Migration without rollback rehearsal.

Expert Review Questions

- What happens if this component succeeds but the next component fails?
- How is the authorizing identity/device/key revoked or recovered?
- Can the operation be safely retried after the client loses the response?
- What happens if AI is unavailable, slow, or wrong?
- Can an auditor reconstruct debit, credit, actor, risk decision, approval, and Fabric txId from retained evidence?
- Which compensating transaction resolves a downstream operational error without rewriting history?

Chapter 278 - Disconnected Branch Voucher Mode

If true site partition spending is required, use capped pre-authorized value rather than pretending consensus continues. Scale only after measuring the bottleneck. Fabric ordering, peer endorsement, CouchDB access, PostgreSQL projections, AI inference, local LLM context, and UI concurrency all have different capacity characteristics. Add complexity only for a measured availability, governance, security, or performance need.

Design rule: Issue bounded offline vouchers/allowance

Implementation Objective

The required output is: Partition-risk design. It must be version-controlled, reviewable, testable with the WAN disabled, and linked to the architecture/source-of-truth model. For a payment platform, every implementation decision should make it possible to explain who authorized an action, which facts were trusted, which controls ran, what financial journal committed, and how the result is recovered after failure.

Key Engineering Decisions

- Issue bounded offline vouchers/allowance
- prevent unlimited double spend
- expire/reconcile after reconnection
- business accepts residual risk explicitly

Deep Implementation Guidance

Prefer the simplest topology that satisfies the trust and recovery model. Every new site, asset, peer, channel, HSM, model, queue, and HA layer adds operational failure modes. Require measured benefit and a tested rollback plan.

Data and Control Questions

- What is the authoritative source for every input used by this component?
- Which user, device, service, MSP, or administrator identity is allowed to initiate or approve it?
- Does the operation move value, reserve value, create a side effect, or only provide advisory information?
- What idempotency/replay key proves that a retry is the same business intent?
- What evidence is retained off-chain and what stable reference/hash belongs on the ledger?

Chapter 278 Implementation - Disconnected Branch Voucher Mode

Implementation Procedure

1386. Baseline end-to-end latency and throughput before tuning.
1387. Measure each component separately and identify the dominant bottleneck.
1388. Use HA/redundancy only where the business RTO/availability requires it.
1389. Test scale changes under realistic concurrency and failure.
1390. Keep migration/rollback evidence for topology, Fabric, model, and database upgrades.

Implementation Example

```
# Disconnected Branch Voucher Mode
INPUT  -> authenticate, validate schema, bind requestId/nonce
FACTS  -> read authoritative wallet/merchant/state data
AI     -> optional bounded recommendation with decisionId/version
POLICY -> enforce ownership, hard limits, approvals, risk fallback
LEDGER -> submit one idempotent deterministic transaction when value changes
EVENT  -> update projection/case/monitoring asynchronously
VERIFY -> discover committed result and retain correlationId + txId evidence
```

Verification Checklist

- The partition-risk design exists and is version-controlled.
- The feature works with all public Internet/WAN connectivity disabled.
- Authorization, malformed input, duplicate retry, and failure behavior have explicit tests.
- If financial value changes, journal debits and credits balance exactly and historical entries remain immutable.
- If AI is involved, model/prompt/feature version and fallback behavior are recorded and deterministic policy remains final.
- A timeout/restart cannot create a duplicate financial result; request status can be rediscovered.
- Logs/evidence contain correlation references but not secrets or prohibited sensitive payment data.

Common Failure Modes

- Premature multi-channel/multi-agent complexity.
- Capacity claims without measurement.
- Disconnected-spend mode without strict risk caps.
- Migration without rollback rehearsal.

Expert Review Questions

- What happens if this component succeeds but the next component fails?
- How is the authorizing identity/device/key revoked or recovered?
- Can the operation be safely retried after the client loses the response?
- What happens if AI is unavailable, slow, or wrong?
- Can an auditor reconstruct debit, credit, actor, risk decision, approval, and Fabric txId from retained evidence?
- Which compensating transaction resolves a downstream operational error without rewriting history?

Chapter 279 - Multi-Asset Wallet

Support more than one internal asset without accidental conversion or commingling. Scale only after measuring the bottleneck. Fabric ordering, peer endorsement, CouchDB access, PostgreSQL projections, AI inference, local LLM context, and UI concurrency all have different capacity characteristics. Add complexity only for a measured availability, governance, security, or performance need.

Design rule: Every wallet/journal line has asset

Implementation Objective

The required output is: Multi-asset design. It must be version-controlled, reviewable, testable with the WAN disabled, and linked to the architecture/source-of-truth model. For a payment platform, every implementation decision should make it possible to explain who authorized an action, which facts were trusted, which controls ran, what financial journal committed, and how the result is recovered after failure.

Key Engineering Decisions

- Every wallet/journal line has asset
- no cross-asset balancing
- conversion is explicit transaction with rate/reference/authorization

Deep Implementation Guidance

Prefer the simplest topology that satisfies the trust and recovery model. Every new site, asset, peer, channel, HSM, model, queue, and HA layer adds operational failure modes. Require measured benefit and a tested rollback plan.

Data and Control Questions

- What is the authoritative source for every input used by this component?
- Which user, device, service, MSP, or administrator identity is allowed to initiate or approve it?
- Does the operation move value, reserve value, create a side effect, or only provide advisory information?
- What idempotency/replay key proves that a retry is the same business intent?
- What evidence is retained off-chain and what stable reference/hash belongs on the ledger?

Chapter 279 Implementation - Multi-Asset Wallet

Implementation Procedure

1391. Baseline end-to-end latency and throughput before tuning.
1392. Measure each component separately and identify the dominant bottleneck.
1393. Use HA/redundancy only where the business RTO/availability requires it.
1394. Test scale changes under realistic concurrency and failure.
1395. Keep migration/rollback evidence for topology, Fabric, model, and database upgrades.

Implementation Example

```
# Multi-Asset Wallet
INPUT  -> authenticate, validate schema, bind requestId/nonce
FACTS  -> read authoritative wallet/merchant/state data
AI      -> optional bounded recommendation with decisionId/version
POLICY  -> enforce ownership, hard limits, approvals, risk fallback
LEDGER  -> submit one idempotent deterministic transaction when value changes
EVENT   -> update projection/case/monitoring asynchronously
VERIFY  -> discover committed result and retain correlationId + txId evidence
```

Verification Checklist

- The multi-asset design exists and is version-controlled.
- The feature works with all public Internet/WAN connectivity disabled.
- Authorization, malformed input, duplicate retry, and failure behavior have explicit tests.
- If financial value changes, journal debits and credits balance exactly and historical entries remain immutable.
- If AI is involved, model/prompt/feature version and fallback behavior are recorded and deterministic policy remains final.
- A timeout/restart cannot create a duplicate financial result; request status can be rediscovered.
- Logs/evidence contain correlation references but not secrets or prohibited sensitive payment data.

Common Failure Modes

- Premature multi-channel/multi-agent complexity.
- Capacity claims without measurement.
- Disconnected-spend mode without strict risk caps.
- Migration without rollback rehearsal.

Expert Review Questions

- What happens if this component succeeds but the next component fails?
- How is the authorizing identity/device/key revoked or recovered?
- Can the operation be safely retried after the client loses the response?
- What happens if AI is unavailable, slow, or wrong?
- Can an auditor reconstruct debit, credit, actor, risk decision, approval, and Fabric txId from retained evidence?
- Which compensating transaction resolves a downstream operational error without rewriting history?

Chapter 280 - Advanced Settlement Networks

Model multiple merchants/treasury accounts while retaining balanced accounting. Scale only after measuring the bottleneck. Fabric ordering, peer endorsement, CouchDB access, PostgreSQL projections, AI inference, local LLM context, and UI concurrency all have different capacity characteristics. Add complexity only for a measured availability, governance, security, or performance need.

Design rule: Batch by merchant/period/asset

Implementation Objective

The required output is: Settlement architecture. It must be version-controlled, reviewable, testable with the WAN disabled, and linked to the architecture/source-of-truth model. For a payment platform, every implementation decision should make it possible to explain who authorized an action, which facts were trusted, which controls ran, what financial journal committed, and how the result is recovered after failure.

Key Engineering Decisions

- Batch by merchant/period/asset
- fees explicit
- exceptions to suspense
- auditable approval

Deep Implementation Guidance

Prefer the simplest topology that satisfies the trust and recovery model. Every new site, asset, peer, channel, HSM, model, queue, and HA layer adds operational failure modes. Require measured benefit and a tested rollback plan.

Data and Control Questions

- What is the authoritative source for every input used by this component?
- Which user, device, service, MSP, or administrator identity is allowed to initiate or approve it?
- Does the operation move value, reserve value, create a side effect, or only provide advisory information?
- What idempotency/replay key proves that a retry is the same business intent?
- What evidence is retained off-chain and what stable reference/hash belongs on the ledger?

Chapter 280 Implementation - Advanced Settlement Networks

Implementation Procedure

- 1396. Baseline end-to-end latency and throughput before tuning.
- 1397. Measure each component separately and identify the dominant bottleneck.
- 1398. Use HA/redundancy only where the business RTO/availability requires it.
- 1399. Test scale changes under realistic concurrency and failure.
- 1400. Keep migration/rollback evidence for topology, Fabric, model, and database upgrades.

Implementation Example

```
# Advanced Settlement Networks
INPUT  -> authenticate, validate schema, bind requestId/nonce
FACTS  -> read authoritative wallet/merchant/state data
AI     -> optional bounded recommendation with decisionId/version
POLICY -> enforce ownership, hard limits, approvals, risk fallback
LEDGER -> submit one idempotent deterministic transaction when value changes
EVENT  -> update projection/case/monitoring asynchronously
VERIFY -> discover committed result and retain correlationId + txId evidence
```

Verification Checklist

- The settlement architecture exists and is version-controlled.
- The feature works with all public Internet/WAN connectivity disabled.
- Authorization, malformed input, duplicate retry, and failure behavior have explicit tests.
- If financial value changes, journal debits and credits balance exactly and historical entries remain immutable.
- If AI is involved, model/prompt/feature version and fallback behavior are recorded and deterministic policy remains final.
- A timeout/restart cannot create a duplicate financial result; request status can be rediscovered.
- Logs/evidence contain correlation references but not secrets or prohibited sensitive payment data.

Common Failure Modes

- Premature multi-channel/multi-agent complexity.
- Capacity claims without measurement.
- Disconnected-spend mode without strict risk caps.
- Migration without rollback rehearsal.

Expert Review Questions

- What happens if this component succeeds but the next component fails?
- How is the authorizing identity/device/key revoked or recovered?
- Can the operation be safely retried after the client loses the response?
- What happens if AI is unavailable, slow, or wrong?
- Can an auditor reconstruct debit, credit, actor, risk decision, approval, and Fabric txId from retained evidence?
- Which compensating transaction resolves a downstream operational error without rewriting history?

Chapter 281 - Fabric 3.x and SmartBFT Upgrade Path

Plan optional migration when Byzantine fault tolerance is a real requirement. Scale only after measuring the bottleneck. Fabric ordering, peer endorsement, CouchDB access, PostgreSQL projections, AI inference, local LLM context, and UI concurrency all have different capacity characteristics. Add complexity only for a measured availability, governance, security, or performance need.

Design rule: Validate supported upgrade path/current docs

Implementation Objective

The required output is: Upgrade ADR. It must be version-controlled, reviewable, testable with the WAN disabled, and linked to the architecture/source-of-truth model. For a payment platform, every implementation decision should make it possible to explain who authorized an action, which facts were trusted, which controls ran, what financial journal committed, and how the result is recovered after failure.

Key Engineering Decisions

- Validate supported upgrade path/current docs
- understand stronger topology/operational cost
- test app/chaincode compatibility

Deep Implementation Guidance

Prefer the simplest topology that satisfies the trust and recovery model. Every new site, asset, peer, channel, HSM, model, queue, and HA layer adds operational failure modes. Require measured benefit and a tested rollback plan.

Data and Control Questions

- What is the authoritative source for every input used by this component?
- Which user, device, service, MSP, or administrator identity is allowed to initiate or approve it?
- Does the operation move value, reserve value, create a side effect, or only provide advisory information?
- What idempotency/replay key proves that a retry is the same business intent?
- What evidence is retained off-chain and what stable reference/hash belongs on the ledger?

Chapter 281 Implementation - Fabric 3.x and SmartBFT Upgrade Path

Implementation Procedure

1401. Baseline end-to-end latency and throughput before tuning.
1402. Measure each component separately and identify the dominant bottleneck.
1403. Use HA/redundancy only where the business RTO/availability requires it.
1404. Test scale changes under realistic concurrency and failure.
1405. Keep migration/rollback evidence for topology, Fabric, model, and database upgrades.

Implementation Example

```
# Fabric 3.x and SmartBFT Upgrade Path
INPUT  -> authenticate, validate schema, bind requestId/nonce
FACTS  -> read authoritative wallet/merchant/state data
AI     -> optional bounded recommendation with decisionId/version
POLICY -> enforce ownership, hard limits, approvals, risk fallback
LEDGER -> submit one idempotent deterministic transaction when value changes
EVENT  -> update projection/case/monitoring asynchronously
VERIFY -> discover committed result and retain correlationId + txId evidence
```

Verification Checklist

- The upgrade adr exists and is version-controlled.
- The feature works with all public Internet/WAN connectivity disabled.
- Authorization, malformed input, duplicate retry, and failure behavior have explicit tests.
- If financial value changes, journal debits and credits balance exactly and historical entries remain immutable.
- If AI is involved, model/prompt/feature version and fallback behavior are recorded and deterministic policy remains final.
- A timeout/restart cannot create a duplicate financial result; request status can be rediscovered.
- Logs/evidence contain correlation references but not secrets or prohibited sensitive payment data.

Common Failure Modes

- Premature multi-channel/multi-agent complexity.
- Capacity claims without measurement.
- Disconnected-spend mode without strict risk caps.
- Migration without rollback rehearsal.

Expert Review Questions

- What happens if this component succeeds but the next component fails?
- How is the authorizing identity/device/key revoked or recovered?
- Can the operation be safely retried after the client loses the response?
- What happens if AI is unavailable, slow, or wrong?
- Can an auditor reconstruct debit, credit, actor, risk decision, approval, and Fabric txId from retained evidence?
- Which compensating transaction resolves a downstream operational error without rewriting history?

Chapter 282 - Hardware Security Module Expansion

Move more sensitive signing operations into HSM-backed workflows. Scale only after measuring the bottleneck. Fabric ordering, peer endorsement, CouchDB access, PostgreSQL projections, AI inference, local LLM context, and UI concurrency all have different capacity characteristics. Add complexity only for a measured availability, governance, security, or performance need.

Design rule: Prioritize treasury/service/admin signing

Implementation Objective

The required output is: HSM roadmap. It must be version-controlled, reviewable, testable with the WAN disabled, and linked to the architecture/source-of-truth model. For a payment platform, every implementation decision should make it possible to explain who authorized an action, which facts were trusted, which controls ran, what financial journal committed, and how the result is recovered after failure.

Key Engineering Decisions

- Prioritize treasury/service/admin signing
- dual/operator controls
- capacity/HA
- vendor replacement and backup

Deep Implementation Guidance

Prefer the simplest topology that satisfies the trust and recovery model. Every new site, asset, peer, channel, HSM, model, queue, and HA layer adds operational failure modes. Require measured benefit and a tested rollback plan.

Data and Control Questions

- What is the authoritative source for every input used by this component?
- Which user, device, service, MSP, or administrator identity is allowed to initiate or approve it?
- Does the operation move value, reserve value, create a side effect, or only provide advisory information?
- What idempotency/replay key proves that a retry is the same business intent?
- What evidence is retained off-chain and what stable reference/hash belongs on the ledger?

Chapter 282 Implementation - Hardware Security Module Expansion

Implementation Procedure

1406. Baseline end-to-end latency and throughput before tuning.
1407. Measure each component separately and identify the dominant bottleneck.
1408. Use HA/redundancy only where the business RTO/availability requires it.
1409. Test scale changes under realistic concurrency and failure.
1410. Keep migration/rollback evidence for topology, Fabric, model, and database upgrades.

Implementation Example

```
# Hardware Security Module Expansion
INPUT  -> authenticate, validate schema, bind requestId/nonce
FACTS  -> read authoritative wallet/merchant/state data
AI     -> optional bounded recommendation with decisionId/version
POLICY -> enforce ownership, hard limits, approvals, risk fallback
LEDGER -> submit one idempotent deterministic transaction when value changes
EVENT  -> update projection/case/monitoring asynchronously
VERIFY -> discover committed result and retain correlationId + txId evidence
```

Verification Checklist

- The hsm roadmap exists and is version-controlled.
- The feature works with all public Internet/WAN connectivity disabled.
- Authorization, malformed input, duplicate retry, and failure behavior have explicit tests.
- If financial value changes, journal debits and credits balance exactly and historical entries remain immutable.
- If AI is involved, model/prompt/feature version and fallback behavior are recorded and deterministic policy remains final.
- A timeout/restart cannot create a duplicate financial result; request status can be rediscovered.
- Logs/evidence contain correlation references but not secrets or prohibited sensitive payment data.

Common Failure Modes

- Premature multi-channel/multi-agent complexity.
- Capacity claims without measurement.
- Disconnected-spend mode without strict risk caps.
- Migration without rollback rehearsal.

Expert Review Questions

- What happens if this component succeeds but the next component fails?
- How is the authorizing identity/device/key revoked or recovered?
- Can the operation be safely retried after the client loses the response?
- What happens if AI is unavailable, slow, or wrong?
- Can an auditor reconstruct debit, credit, actor, risk decision, approval, and Fabric txId from retained evidence?
- Which compensating transaction resolves a downstream operational error without rewriting history?

Chapter 283 - Ledger Evidence Anchoring for AI Releases

Use hashes to prove which model/rules/prompt package was approved at a point in time. Scale only after measuring the bottleneck. Fabric ordering, peer endorsement, CouchDB access, PostgreSQL projections, AI inference, local LLM context, and UI concurrency all have different capacity characteristics. Add complexity only for a measured availability, governance, security, or performance need.

Design rule: Hash manifest not binary

Implementation Objective

The required output is: Release evidence transaction. It must be version-controlled, reviewable, testable with the WAN disabled, and linked to the architecture/source-of-truth model. For a payment platform, every implementation decision should make it possible to explain who authorized an action, which facts were trusted, which controls ran, what financial journal committed, and how the result is recovered after failure.

Key Engineering Decisions

- Hash manifest not binary
- include version/approver/date/reference
- off-chain registry stores detailed artifact

Deep Implementation Guidance

Prefer the simplest topology that satisfies the trust and recovery model. Every new site, asset, peer, channel, HSM, model, queue, and HA layer adds operational failure modes. Require measured benefit and a tested rollback plan.

Data and Control Questions

- What is the authoritative source for every input used by this component?
- Which user, device, service, MSP, or administrator identity is allowed to initiate or approve it?
- Does the operation move value, reserve value, create a side effect, or only provide advisory information?
- What idempotency/replay key proves that a retry is the same business intent?
- What evidence is retained off-chain and what stable reference/hash belongs on the ledger?

Chapter 283 Implementation - Ledger Evidence Anchoring for AI Releases

Implementation Procedure

1411. Baseline end-to-end latency and throughput before tuning.
1412. Measure each component separately and identify the dominant bottleneck.
1413. Use HA/redundancy only where the business RTO/availability requires it.
1414. Test scale changes under realistic concurrency and failure.
1415. Keep migration/rollback evidence for topology, Fabric, model, and database upgrades.

Implementation Example

```
# Ledger Evidence Anchoring for AI Releases
INPUT  -> authenticate, validate schema, bind requestId/nonce
FACTS  -> read authoritative wallet/merchant/state data
AI     -> optional bounded recommendation with decisionId/version
POLICY -> enforce ownership, hard limits, approvals, risk fallback
LEDGER -> submit one idempotent deterministic transaction when value changes
EVENT  -> update projection/case/monitoring asynchronously
VERIFY -> discover committed result and retain correlationId + txId evidence
```

Verification Checklist

- The release evidence transaction exists and is version-controlled.
- The feature works with all public Internet/WAN connectivity disabled.
- Authorization, malformed input, duplicate retry, and failure behavior have explicit tests.
- If financial value changes, journal debits and credits balance exactly and historical entries remain immutable.
- If AI is involved, model/prompt/feature version and fallback behavior are recorded and deterministic policy remains final.
- A timeout/restart cannot create a duplicate financial result; request status can be rediscovered.
- Logs/evidence contain correlation references but not secrets or prohibited sensitive payment data.

Common Failure Modes

- Premature multi-channel/multi-agent complexity.
- Capacity claims without measurement.
- Disconnected-spend mode without strict risk caps.
- Migration without rollback rehearsal.

Expert Review Questions

- What happens if this component succeeds but the next component fails?
- How is the authorizing identity/device/key revoked or recovered?
- Can the operation be safely retried after the client loses the response?
- What happens if AI is unavailable, slow, or wrong?
- Can an auditor reconstruct debit, credit, actor, risk decision, approval, and Fabric txId from retained evidence?
- Which compensating transaction resolves a downstream operational error without rewriting history?

Chapter 284 - Local Analytics Warehouse

Build historical BI without forcing analytics queries onto operational peers. Scale only after measuring the bottleneck. Fabric ordering, peer endorsement, CouchDB access, PostgreSQL projections, AI inference, local LLM context, and UI concurrency all have different capacity characteristics. Add complexity only for a measured availability, governance, security, or performance need.

Design rule: Export/projection to local warehouse

Implementation Objective

The required output is: Analytics design. It must be version-controlled, reviewable, testable with the WAN disabled, and linked to the architecture/source-of-truth model. For a payment platform, every implementation decision should make it possible to explain who authorized an action, which facts were trusted, which controls ran, what financial journal committed, and how the result is recovered after failure.

Key Engineering Decisions

- Export/projection to local warehouse
- read-only
- reconcile to ledger
- apply privacy retention and access controls

Deep Implementation Guidance

Prefer the simplest topology that satisfies the trust and recovery model. Every new site, asset, peer, channel, HSM, model, queue, and HA layer adds operational failure modes. Require measured benefit and a tested rollback plan.

Data and Control Questions

- What is the authoritative source for every input used by this component?
- Which user, device, service, MSP, or administrator identity is allowed to initiate or approve it?
- Does the operation move value, reserve value, create a side effect, or only provide advisory information?
- What idempotency/replay key proves that a retry is the same business intent?
- What evidence is retained off-chain and what stable reference/hash belongs on the ledger?

Chapter 284 Implementation - Local Analytics Warehouse

Implementation Procedure

- 1416. Baseline end-to-end latency and throughput before tuning.
- 1417. Measure each component separately and identify the dominant bottleneck.
- 1418. Use HA/redundancy only where the business RTO/availability requires it.
- 1419. Test scale changes under realistic concurrency and failure.
- 1420. Keep migration/rollback evidence for topology, Fabric, model, and database upgrades.

Implementation Example

```
# Local Analytics Warehouse
INPUT  -> authenticate, validate schema, bind requestId/nonce
FACTS  -> read authoritative wallet/merchant/state data
AI     -> optional bounded recommendation with decisionId/version
POLICY -> enforce ownership, hard limits, approvals, risk fallback
LEDGER -> submit one idempotent deterministic transaction when value changes
EVENT  -> update projection/case/monitoring asynchronously
VERIFY -> discover committed result and retain correlationId + txId evidence
```

Verification Checklist

- The analytics design exists and is version-controlled.
- The feature works with all public Internet/WAN connectivity disabled.
- Authorization, malformed input, duplicate retry, and failure behavior have explicit tests.
- If financial value changes, journal debits and credits balance exactly and historical entries remain immutable.
- If AI is involved, model/prompt/feature version and fallback behavior are recorded and deterministic policy remains final.
- A timeout/restart cannot create a duplicate financial result; request status can be rediscovered.
- Logs/evidence contain correlation references but not secrets or prohibited sensitive payment data.

Common Failure Modes

- Premature multi-channel/multi-agent complexity.
- Capacity claims without measurement.
- Disconnected-spend mode without strict risk caps.
- Migration without rollback rehearsal.

Expert Review Questions

- What happens if this component succeeds but the next component fails?
- How is the authorizing identity/device/key revoked or recovered?
- Can the operation be safely retried after the client loses the response?
- What happens if AI is unavailable, slow, or wrong?
- Can an auditor reconstruct debit, credit, actor, risk decision, approval, and Fabric txId from retained evidence?
- Which compensating transaction resolves a downstream operational error without rewriting history?

Chapter 285 - Graph Analytics for Fraud

Run local graph analysis over transaction projections for investigator support. Scale only after measuring the bottleneck. Fabric ordering, peer endorsement, CouchDB access, PostgreSQL projections, AI inference, local LLM context, and UI concurrency all have different capacity characteristics. Add complexity only for a measured availability, governance, security, or performance need.

Design rule: Use pseudonymous wallet/device/merchant nodes

Implementation Objective

The required output is: Graph analytics design. It must be version-controlled, reviewable, testable with the WAN disabled, and linked to the architecture/source-of-truth model. For a payment platform, every implementation decision should make it possible to explain who authorized an action, which facts were trusted, which controls ran, what financial journal committed, and how the result is recovered after failure.

Key Engineering Decisions

- Use pseudonymous wallet/device/merchant nodes
- compute community/cycle/fanout features
- advisory and explainable

Deep Implementation Guidance

Prefer the simplest topology that satisfies the trust and recovery model. Every new site, asset, peer, channel, HSM, model, queue, and HA layer adds operational failure modes. Require measured benefit and a tested rollback plan.

Data and Control Questions

- What is the authoritative source for every input used by this component?
- Which user, device, service, MSP, or administrator identity is allowed to initiate or approve it?
- Does the operation move value, reserve value, create a side effect, or only provide advisory information?
- What idempotency/replay key proves that a retry is the same business intent?
- What evidence is retained off-chain and what stable reference/hash belongs on the ledger?

Chapter 285 Implementation - Graph Analytics for Fraud

Implementation Procedure

- 1421. Baseline end-to-end latency and throughput before tuning.
- 1422. Measure each component separately and identify the dominant bottleneck.
- 1423. Use HA/redundancy only where the business RTO/availability requires it.
- 1424. Test scale changes under realistic concurrency and failure.
- 1425. Keep migration/rollback evidence for topology, Fabric, model, and database upgrades.

Implementation Example

```
# Graph Analytics for Fraud
INPUT  -> authenticate, validate schema, bind requestId/nonce
FACTS  -> read authoritative wallet/merchant/state data
AI     -> optional bounded recommendation with decisionId/version
POLICY -> enforce ownership, hard limits, approvals, risk fallback
LEDGER -> submit one idempotent deterministic transaction when value changes
EVENT  -> update projection/case/monitoring asynchronously
VERIFY -> discover committed result and retain correlationId + txId evidence
```

Verification Checklist

- The graph analytics design exists and is version-controlled.
- The feature works with all public Internet/WAN connectivity disabled.
- Authorization, malformed input, duplicate retry, and failure behavior have explicit tests.
- If financial value changes, journal debits and credits balance exactly and historical entries remain immutable.
- If AI is involved, model/prompt/feature version and fallback behavior are recorded and deterministic policy remains final.
- A timeout/restart cannot create a duplicate financial result; request status can be rediscovered.
- Logs/evidence contain correlation references but not secrets or prohibited sensitive payment data.

Common Failure Modes

- Premature multi-channel/multi-agent complexity.
- Capacity claims without measurement.
- Disconnected-spend mode without strict risk caps.
- Migration without rollback rehearsal.

Expert Review Questions

- What happens if this component succeeds but the next component fails?
- How is the authorizing identity/device/key revoked or recovered?
- Can the operation be safely retried after the client loses the response?
- What happens if AI is unavailable, slow, or wrong?
- Can an auditor reconstruct debit, credit, actor, risk decision, approval, and Fabric txId from retained evidence?
- Which compensating transaction resolves a downstream operational error without rewriting history?

Chapter 286 - Privacy-Preserving Analytics Options

Reduce exposure of customer identity during model development and reporting. Scale only after measuring the bottleneck. Fabric ordering, peer endorsement, CouchDB access, PostgreSQL projections, AI inference, local LLM context, and UI concurrency all have different capacity characteristics. Add complexity only for a measured availability, governance, security, or performance need.

Design rule: Pseudonymize identifiers

Implementation Objective

The required output is: Privacy design. It must be version-controlled, reviewable, testable with the WAN disabled, and linked to the architecture/source-of-truth model. For a payment platform, every implementation decision should make it possible to explain who authorized an action, which facts were trusted, which controls ran, what financial journal committed, and how the result is recovered after failure.

Key Engineering Decisions

- Pseudonymize identifiers
- aggregate where possible
- separate mapping keys
- limit raw narrative access

Deep Implementation Guidance

Prefer the simplest topology that satisfies the trust and recovery model. Every new site, asset, peer, channel, HSM, model, queue, and HA layer adds operational failure modes. Require measured benefit and a tested rollback plan.

Data and Control Questions

- What is the authoritative source for every input used by this component?
- Which user, device, service, MSP, or administrator identity is allowed to initiate or approve it?
- Does the operation move value, reserve value, create a side effect, or only provide advisory information?
- What idempotency/replay key proves that a retry is the same business intent?
- What evidence is retained off-chain and what stable reference/hash belongs on the ledger?

Chapter 286 Implementation - Privacy-Preserving Analytics Options

Implementation Procedure

- 1426. Baseline end-to-end latency and throughput before tuning.
- 1427. Measure each component separately and identify the dominant bottleneck.
- 1428. Use HA/redundancy only where the business RTO/availability requires it.
- 1429. Test scale changes under realistic concurrency and failure.
- 1430. Keep migration/rollback evidence for topology, Fabric, model, and database upgrades.

Implementation Example

```
# Privacy-Preserving Analytics Options
INPUT  -> authenticate, validate schema, bind requestId/nonce
FACTS  -> read authoritative wallet/merchant/state data
AI     -> optional bounded recommendation with decisionId/version
POLICY -> enforce ownership, hard limits, approvals, risk fallback
LEDGER -> submit one idempotent deterministic transaction when value changes
EVENT  -> update projection/case/monitoring asynchronously
VERIFY -> discover committed result and retain correlationId + txId evidence
```

Verification Checklist

- The privacy design exists and is version-controlled.
- The feature works with all public Internet/WAN connectivity disabled.
- Authorization, malformed input, duplicate retry, and failure behavior have explicit tests.
- If financial value changes, journal debits and credits balance exactly and historical entries remain immutable.
- If AI is involved, model/prompt/feature version and fallback behavior are recorded and deterministic policy remains final.
- A timeout/restart cannot create a duplicate financial result; request status can be rediscovered.
- Logs/evidence contain correlation references but not secrets or prohibited sensitive payment data.

Common Failure Modes

- Premature multi-channel/multi-agent complexity.
- Capacity claims without measurement.
- Disconnected-spend mode without strict risk caps.
- Migration without rollback rehearsal.

Expert Review Questions

- What happens if this component succeeds but the next component fails?
- How is the authorizing identity/device/key revoked or recovered?
- Can the operation be safely retried after the client loses the response?
- What happens if AI is unavailable, slow, or wrong?
- Can an auditor reconstruct debit, credit, actor, risk decision, approval, and Fabric txId from retained evidence?
- Which compensating transaction resolves a downstream operational error without rewriting history?

Chapter 287 - Business Continuity Without AI

Prove the payment platform can continue safely if every AI component is intentionally disabled. Scale only after measuring the bottleneck. Fabric ordering, peer endorsement, CouchDB access, PostgreSQL projections, AI inference, local LLM context, and UI concurrency all have different capacity characteristics. Add complexity only for a measured availability, governance, security, or performance need.

Design rule: Hard rules and human review remain

Implementation Objective

The required output is: AI-free continuity test. It must be version-controlled, reviewable, testable with the WAN disabled, and linked to the architecture/source-of-truth model. For a payment platform, every implementation decision should make it possible to explain who authorized an action, which facts were trusted, which controls ran, what financial journal committed, and how the result is recovered after failure.

Key Engineering Decisions

- Hard rules and human review remain
- core ledger/API available
- support assistant loss is noncritical
- document throughput impact

Deep Implementation Guidance

Prefer the simplest topology that satisfies the trust and recovery model. Every new site, asset, peer, channel, HSM, model, queue, and HA layer adds operational failure modes. Require measured benefit and a tested rollback plan.

Data and Control Questions

- What is the authoritative source for every input used by this component?
- Which user, device, service, MSP, or administrator identity is allowed to initiate or approve it?
- Does the operation move value, reserve value, create a side effect, or only provide advisory information?
- What idempotency/replay key proves that a retry is the same business intent?
- What evidence is retained off-chain and what stable reference/hash belongs on the ledger?

Chapter 287 Implementation - Business Continuity Without AI

Implementation Procedure

1431. Baseline end-to-end latency and throughput before tuning.
1432. Measure each component separately and identify the dominant bottleneck.
1433. Use HA/redundancy only where the business RTO/availability requires it.
1434. Test scale changes under realistic concurrency and failure.
1435. Keep migration/rollback evidence for topology, Fabric, model, and database upgrades.

Implementation Example

```
# Business Continuity Without AI
INPUT  -> authenticate, validate schema, bind requestId/nonce
FACTS  -> read authoritative wallet/merchant/state data
AI     -> optional bounded recommendation with decisionId/version
POLICY -> enforce ownership, hard limits, approvals, risk fallback
LEDGER -> submit one idempotent deterministic transaction when value changes
EVENT  -> update projection/case/monitoring asynchronously
VERIFY -> discover committed result and retain correlationId + txId evidence
```

Verification Checklist

- The ai-free continuity test exists and is version-controlled.
- The feature works with all public Internet/WAN connectivity disabled.
- Authorization, malformed input, duplicate retry, and failure behavior have explicit tests.
- If financial value changes, journal debits and credits balance exactly and historical entries remain immutable.
- If AI is involved, model/prompt/feature version and fallback behavior are recorded and deterministic policy remains final.
- A timeout/restart cannot create a duplicate financial result; request status can be rediscovered.
- Logs/evidence contain correlation references but not secrets or prohibited sensitive payment data.

Common Failure Modes

- Premature multi-channel/multi-agent complexity.
- Capacity claims without measurement.
- Disconnected-spend mode without strict risk caps.
- Migration without rollback rehearsal.

Expert Review Questions

- What happens if this component succeeds but the next component fails?
- How is the authorizing identity/device/key revoked or recovered?
- Can the operation be safely retried after the client loses the response?
- What happens if AI is unavailable, slow, or wrong?
- Can an auditor reconstruct debit, credit, actor, risk decision, approval, and Fabric txId from retained evidence?
- Which compensating transaction resolves a downstream operational error without rewriting history?

Chapter 288 - Migration from Legacy Payment Database

Import opening balances safely into the new double-entry ledger. Scale only after measuring the bottleneck. Fabric ordering, peer endorsement, CouchDB access, PostgreSQL projections, AI inference, local LLM context, and UI concurrency all have different capacity characteristics. Add complexity only for a measured availability, governance, security, or performance need.

Design rule: Freeze/cutoff source

Implementation Objective

The required output is: Migration runbook. It must be version-controlled, reviewable, testable with the WAN disabled, and linked to the architecture/source-of-truth model. For a payment platform, every implementation decision should make it possible to explain who authorized an action, which facts were trusted, which controls ran, what financial journal committed, and how the result is recovered after failure.

Key Engineering Decisions

- Freeze/cutoff source
- reconcile account totals
- create controlled opening journal
- hash migration report
- dual sign-off

Deep Implementation Guidance

Prefer the simplest topology that satisfies the trust and recovery model. Every new site, asset, peer, channel, HSM, model, queue, and HA layer adds operational failure modes. Require measured benefit and a tested rollback plan.

Data and Control Questions

- What is the authoritative source for every input used by this component?
- Which user, device, service, MSP, or administrator identity is allowed to initiate or approve it?
- Does the operation move value, reserve value, create a side effect, or only provide advisory information?
- What idempotency/replay key proves that a retry is the same business intent?
- What evidence is retained off-chain and what stable reference/hash belongs on the ledger?

Chapter 288 Implementation - Migration from Legacy Payment Database

Implementation Procedure

1436. Baseline end-to-end latency and throughput before tuning.
1437. Measure each component separately and identify the dominant bottleneck.
1438. Use HA/redundancy only where the business RTO/availability requires it.
1439. Test scale changes under realistic concurrency and failure.
1440. Keep migration/rollback evidence for topology, Fabric, model, and database upgrades.

Implementation Example

```
# Migration from Legacy Payment Database
INPUT  -> authenticate, validate schema, bind requestId/nonce
FACTS  -> read authoritative wallet/merchant/state data
AI     -> optional bounded recommendation with decisionId/version
POLICY -> enforce ownership, hard limits, approvals, risk fallback
LEDGER -> submit one idempotent deterministic transaction when value changes
EVENT  -> update projection/case/monitoring asynchronously
VERIFY -> discover committed result and retain correlationId + txId evidence
```

Verification Checklist

- The migration runbook exists and is version-controlled.
- The feature works with all public Internet/WAN connectivity disabled.
- Authorization, malformed input, duplicate retry, and failure behavior have explicit tests.
- If financial value changes, journal debits and credits balance exactly and historical entries remain immutable.
- If AI is involved, model/prompt/feature version and fallback behavior are recorded and deterministic policy remains final.
- A timeout/restart cannot create a duplicate financial result; request status can be rediscovered.
- Logs/evidence contain correlation references but not secrets or prohibited sensitive payment data.

Common Failure Modes

- Premature multi-channel/multi-agent complexity.
- Capacity claims without measurement.
- Disconnected-spend mode without strict risk caps.
- Migration without rollback rehearsal.

Expert Review Questions

- What happens if this component succeeds but the next component fails?
- How is the authorizing identity/device/key revoked or recovered?
- Can the operation be safely retried after the client loses the response?
- What happens if AI is unavailable, slow, or wrong?
- Can an auditor reconstruct debit, credit, actor, risk decision, approval, and Fabric txId from retained evidence?
- Which compensating transaction resolves a downstream operational error without rewriting history?

Chapter 289 - External Integration Gateway Option

If future bank/card/core integration is added, isolate it behind a dedicated boundary. Scale only after measuring the bottleneck. Fabric ordering, peer endorsement, CouchDB access, PostgreSQL projections, AI inference, local LLM context, and UI concurrency all have different capacity characteristics. Add complexity only for a measured availability, governance, security, or performance need.

Design rule: No direct Internet from chaincode

Implementation Objective

The required output is: Integration boundary. It must be version-controlled, reviewable, testable with the WAN disabled, and linked to the architecture/source-of-truth model. For a payment platform, every implementation decision should make it possible to explain who authorized an action, which facts were trusted, which controls ran, what financial journal committed, and how the result is recovered after failure.

Key Engineering Decisions

- No direct Internet from chaincode
- adapter translates external state into signed/local business events
- timeouts/reconciliation/PCI/legal scope handled separately

Deep Implementation Guidance

Prefer the simplest topology that satisfies the trust and recovery model. Every new site, asset, peer, channel, HSM, model, queue, and HA layer adds operational failure modes. Require measured benefit and a tested rollback plan.

Data and Control Questions

- What is the authoritative source for every input used by this component?
- Which user, device, service, MSP, or administrator identity is allowed to initiate or approve it?
- Does the operation move value, reserve value, create a side effect, or only provide advisory information?
- What idempotency/replay key proves that a retry is the same business intent?
- What evidence is retained off-chain and what stable reference/hash belongs on the ledger?

Chapter 289 Implementation - External Integration Gateway Option

Implementation Procedure

1441. Baseline end-to-end latency and throughput before tuning.
1442. Measure each component separately and identify the dominant bottleneck.
1443. Use HA/redundancy only where the business RTO/availability requires it.
1444. Test scale changes under realistic concurrency and failure.
1445. Keep migration/rollback evidence for topology, Fabric, model, and database upgrades.

Implementation Example

```
# External Integration Gateway Option
INPUT  -> authenticate, validate schema, bind requestId/nonce
FACTS  -> read authoritative wallet/merchant/state data
AI     -> optional bounded recommendation with decisionId/version
POLICY -> enforce ownership, hard limits, approvals, risk fallback
LEDGER -> submit one idempotent deterministic transaction when value changes
EVENT  -> update projection/case/monitoring asynchronously
VERIFY -> discover committed result and retain correlationId + txId evidence
```

Verification Checklist

- The integration boundary exists and is version-controlled.
- The feature works with all public Internet/WAN connectivity disabled.
- Authorization, malformed input, duplicate retry, and failure behavior have explicit tests.
- If financial value changes, journal debits and credits balance exactly and historical entries remain immutable.
- If AI is involved, model/prompt/feature version and fallback behavior are recorded and deterministic policy remains final.
- A timeout/restart cannot create a duplicate financial result; request status can be rediscovered.
- Logs/evidence contain correlation references but not secrets or prohibited sensitive payment data.

Common Failure Modes

- Premature multi-channel/multi-agent complexity.
- Capacity claims without measurement.
- Disconnected-spend mode without strict risk caps.
- Migration without rollback rehearsal.

Expert Review Questions

- What happens if this component succeeds but the next component fails?
- How is the authorizing identity/device/key revoked or recovered?
- Can the operation be safely retried after the client loses the response?
- What happens if AI is unavailable, slow, or wrong?
- Can an auditor reconstruct debit, credit, actor, risk decision, approval, and Fabric txId from retained evidence?
- Which compensating transaction resolves a downstream operational error without rewriting history?

Part XVI - Reproducible Full Implementation Labs

Each lab is reproducible evidence. Complete it from a clean environment with the WAN disconnected, retain commands/config hashes/test outputs, and prove both successful behavior and at least one failure/recovery path.

Part focus: This part contains 15 implementation chapters. Each chapter includes design rules, implementation procedure, a practical artifact, verification, failure analysis, and expert review questions.

Chapter 290 - Lab 1 - Build the Offline Artifact Bundle

Create a complete package/model/container repository for disconnected installation. Each lab is reproducible evidence. Complete it from a clean environment with the WAN disconnected, retain commands/config hashes/test outputs, and prove both successful behavior and at least one failure/recovery path.

Design rule: Download only on controlled staging station

Implementation Objective

The required output is: Lab evidence bundle. It must be version-controlled, reviewable, testable with the WAN disabled, and linked to the architecture/source-of-truth model. For a payment platform, every implementation decision should make it possible to explain who authorized an action, which facts were trusted, which controls ran, what financial journal committed, and how the result is recovered after failure.

Key Engineering Decisions

- Download only on controlled staging station
- verify hashes
- export container images
- cache Node/Python packages
- include model and license manifest

Deep Implementation Guidance

Store lab evidence in a versioned folder: terminal transcript, config/artifact hashes, test output, transaction references, screenshots where useful, and a short result/gap report. A lab is not complete if it cannot be rerun offline from a clean snapshot.

Data and Control Questions

- What is the authoritative source for every input used by this component?
- Which user, device, service, MSP, or administrator identity is allowed to initiate or approve it?
- Does the operation move value, reserve value, create a side effect, or only provide advisory information?
- What idempotency/replay key proves that a retry is the same business intent?
- What evidence is retained off-chain and what stable reference/hash belongs on the ledger?

Chapter 290 Implementation - Lab 1 - Build the Offline Artifact Bundle

Implementation Procedure

1446. Start from a clean snapshot and verify the WAN is disabled.
1447. Apply only the documented offline artifacts/configuration.
1448. Run the implementation procedure and capture evidence.
1449. Execute success, failure, and retry/recovery tests.
1450. Write a short result report including versions, hashes, gaps, and next actions.

Implementation Example

```
# Lab 1 - Build the Offline Artifact Bundle
INPUT  -> authenticate, validate schema, bind requestId/nonce
FACTS  -> read authoritative wallet/merchant/state data
AI     -> optional bounded recommendation with decisionId/version
POLICY -> enforce ownership, hard limits, approvals, risk fallback
LEDGER -> submit one idempotent deterministic transaction when value changes
EVENT  -> update projection/case/monitoring asynchronously
VERIFY -> discover committed result and retain correlationId + txId evidence
```

Verification Checklist

- The lab evidence bundle exists and is version-controlled.
- The feature works with all public Internet/WAN connectivity disabled.
- Authorization, malformed input, duplicate retry, and failure behavior have explicit tests.
- If financial value changes, journal debits and credits balance exactly and historical entries remain immutable.
- If AI is involved, model/prompt/feature version and fallback behavior are recorded and deterministic policy remains final.
- A timeout/restart cannot create a duplicate financial result; request status can be rediscovered.
- Logs/evidence contain correlation references but not secrets or prohibited sensitive payment data.

Common Failure Modes

- Lab requires Internet despite offline claim.
- Commands cannot be repeated from clean state.
- No retained evidence.
- Failure case skipped.

Expert Review Questions

- What happens if this component succeeds but the next component fails?
- How is the authorizing identity/device/key revoked or recovered?
- Can the operation be safely retried after the client loses the response?
- What happens if AI is unavailable, slow, or wrong?
- Can an auditor reconstruct debit, credit, actor, risk decision, approval, and Fabric txId from retained evidence?
- Which compensating transaction resolves a downstream operational error without rewriting history?

Chapter 291 - Lab 2 - Build the Four-Domain Fabric Network

Deploy WalletOrg MerchantOrg AuditOrg and OrdererOrg with local CAs and three Raft orderers. Each lab is reproducible evidence. Complete it from a clean environment with the WAN disconnected, retain commands/config hashes/test outputs, and prove both successful behavior and at least one failure/recovery path.

Design rule: Use static internal DNS/TLS

Implementation Objective

The required output is: Working Fabric network. It must be version-controlled, reviewable, testable with the WAN disabled, and linked to the architecture/source-of-truth model. For a payment platform, every implementation decision should make it possible to explain who authorized an action, which facts were trusted, which controls ran, what financial journal committed, and how the result is recovered after failure.

Key Engineering Decisions

- Use static internal DNS/TLS
- join peers to payments channel
- verify quorum
- record organization/MSP inventory

Deep Implementation Guidance

Store lab evidence in a versioned folder: terminal transcript, config/artifact hashes, test output, transaction references, screenshots where useful, and a short result/gap report. A lab is not complete if it cannot be rerun offline from a clean snapshot.

Data and Control Questions

- What is the authoritative source for every input used by this component?
- Which user, device, service, MSP, or administrator identity is allowed to initiate or approve it?
- Does the operation move value, reserve value, create a side effect, or only provide advisory information?
- What idempotency/replay key proves that a retry is the same business intent?
- What evidence is retained off-chain and what stable reference/hash belongs on the ledger?

Chapter 291 Implementation - Lab 2 - Build the Four-Domain Fabric Network

Implementation Procedure

1451. Start from a clean snapshot and verify the WAN is disabled.
1452. Apply only the documented offline artifacts/configuration.
1453. Run the implementation procedure and capture evidence.
1454. Execute success, failure, and retry/recovery tests.
1455. Write a short result report including versions, hashes, gaps, and next actions.

Implementation Example

```
# Lab 2 - Build the Four-Domain Fabric Network
INPUT  -> authenticate, validate schema, bind requestId/nonce
FACTS  -> read authoritative wallet/merchant/state data
AI     -> optional bounded recommendation with decisionId/version
POLICY -> enforce ownership, hard limits, approvals, risk fallback
LEDGER -> submit one idempotent deterministic transaction when value changes
EVENT  -> update projection/case/monitoring asynchronously
VERIFY -> discover committed result and retain correlationId + txId evidence
```

Verification Checklist

- The working fabric network exists and is version-controlled.
- The feature works with all public Internet/WAN connectivity disabled.
- Authorization, malformed input, duplicate retry, and failure behavior have explicit tests.
- If financial value changes, journal debits and credits balance exactly and historical entries remain immutable.
- If AI is involved, model/prompt/feature version and fallback behavior are recorded and deterministic policy remains final.
- A timeout/restart cannot create a duplicate financial result; request status can be rediscovered.
- Logs/evidence contain correlation references but not secrets or prohibited sensitive payment data.

Common Failure Modes

- Lab requires Internet despite offline claim.
- Commands cannot be repeated from clean state.
- No retained evidence.
- Failure case skipped.

Expert Review Questions

- What happens if this component succeeds but the next component fails?
- How is the authorizing identity/device/key revoked or recovered?
- Can the operation be safely retried after the client loses the response?
- What happens if AI is unavailable, slow, or wrong?
- Can an auditor reconstruct debit, credit, actor, risk decision, approval, and Fabric txId from retained evidence?
- Which compensating transaction resolves a downstream operational error without rewriting history?

Chapter 292 - Lab 3 - Deploy Wallet Chaincode Skeleton

Package and deploy TypeScript chaincode with basic wallet/query methods. Each lab is reproducible evidence. Complete it from a clean environment with the WAN disconnected, retain commands/config hashes/test outputs, and prove both successful behavior and at least one failure/recovery path.

Design rule: Install on required peers

Implementation Objective

The required output is: Committed chaincode definition. It must be version-controlled, reviewable, testable with the WAN disabled, and linked to the architecture/source-of-truth model. For a payment platform, every implementation decision should make it possible to explain who authorized an action, which facts were trusted, which controls ran, what financial journal committed, and how the result is recovered after failure.

Key Engineering Decisions

- Install on required peers
- approve per org
- commit lifecycle definition
- evaluate a read transaction

Deep Implementation Guidance

Store lab evidence in a versioned folder: terminal transcript, config/artifact hashes, test output, transaction references, screenshots where useful, and a short result/gap report. A lab is not complete if it cannot be rerun offline from a clean snapshot.

Data and Control Questions

- What is the authoritative source for every input used by this component?
- Which user, device, service, MSP, or administrator identity is allowed to initiate or approve it?
- Does the operation move value, reserve value, create a side effect, or only provide advisory information?
- What idempotency/replay key proves that a retry is the same business intent?
- What evidence is retained off-chain and what stable reference/hash belongs on the ledger?

Chapter 292 Implementation - Lab 3 - Deploy Wallet Chaincode Skeleton

Implementation Procedure

1456. Start from a clean snapshot and verify the WAN is disabled.
1457. Apply only the documented offline artifacts/configuration.
1458. Run the implementation procedure and capture evidence.
1459. Execute success, failure, and retry/recovery tests.
1460. Write a short result report including versions, hashes, gaps, and next actions.

Implementation Example

```
# Lab 3 - Deploy Wallet Chaincode Skeleton
INPUT  -> authenticate, validate schema, bind requestId/nonce
FACTS  -> read authoritative wallet/merchant/state data
AI     -> optional bounded recommendation with decisionId/version
POLICY -> enforce ownership, hard limits, approvals, risk fallback
LEDGER -> submit one idempotent deterministic transaction when value changes
EVENT  -> update projection/case/monitoring asynchronously
VERIFY -> discover committed result and retain correlationId + txId evidence
```

Verification Checklist

- The committed chaincode definition exists and is version-controlled.
- The feature works with all public Internet/WAN connectivity disabled.
- Authorization, malformed input, duplicate retry, and failure behavior have explicit tests.
- If financial value changes, journal debits and credits balance exactly and historical entries remain immutable.
- If AI is involved, model/prompt/feature version and fallback behavior are recorded and deterministic policy remains final.
- A timeout/restart cannot create a duplicate financial result; request status can be rediscovered.
- Logs/evidence contain correlation references but not secrets or prohibited sensitive payment data.

Common Failure Modes

- Lab requires Internet despite offline claim.
- Commands cannot be repeated from clean state.
- No retained evidence.
- Failure case skipped.

Expert Review Questions

- What happens if this component succeeds but the next component fails?
- How is the authorizing identity/device/key revoked or recovered?
- Can the operation be safely retried after the client loses the response?
- What happens if AI is unavailable, slow, or wrong?
- Can an auditor reconstruct debit, credit, actor, risk decision, approval, and Fabric txId from retained evidence?
- Which compensating transaction resolves a downstream operational error without rewriting history?

Chapter 293 - Lab 4 - Implement Double-Entry Transfer

Implement wallet balances, journal lines, and atomic P2P transfer. Each lab is reproducible evidence. Complete it from a clean environment with the WAN disconnected, retain commands/config hashes/test outputs, and prove both successful behavior and at least one failure/recovery path.

Design rule: Use integer minor units

Implementation Objective

The required output is: Transfer implementation. It must be version-controlled, reviewable, testable with the WAN disabled, and linked to the architecture/source-of-truth model. For a payment platform, every implementation decision should make it possible to explain who authorized an action, which facts were trusted, which controls ran, what financial journal committed, and how the result is recovered after failure.

Key Engineering Decisions

- Use integer minor units
- assert balanced journal
- enforce balance/status
- store request idempotency
- test duplicate

Deep Implementation Guidance

Store lab evidence in a versioned folder: terminal transcript, config/artifact hashes, test output, transaction references, screenshots where useful, and a short result/gap report. A lab is not complete if it cannot be rerun offline from a clean snapshot.

Data and Control Questions

- What is the authoritative source for every input used by this component?
- Which user, device, service, MSP, or administrator identity is allowed to initiate or approve it?
- Does the operation move value, reserve value, create a side effect, or only provide advisory information?
- What idempotency/replay key proves that a retry is the same business intent?
- What evidence is retained off-chain and what stable reference/hash belongs on the ledger?

Chapter 293 Implementation - Lab 4 - Implement Double-Entry Transfer

Implementation Procedure

1461. Start from a clean snapshot and verify the WAN is disabled.
1462. Apply only the documented offline artifacts/configuration.
1463. Run the implementation procedure and capture evidence.
1464. Execute success, failure, and retry/recovery tests.
1465. Write a short result report including versions, hashes, gaps, and next actions.

Implementation Example

```
# Lab 4 - Implement Double-Entry Transfer
INPUT  -> authenticate, validate schema, bind requestId/nonce
FACTS  -> read authoritative wallet/merchant/state data
AI     -> optional bounded recommendation with decisionId/version
POLICY -> enforce ownership, hard limits, approvals, risk fallback
LEDGER -> submit one idempotent deterministic transaction when value changes
EVENT  -> update projection/case/monitoring asynchronously
VERIFY -> discover committed result and retain correlationId + txId evidence
```

Verification Checklist

- The transfer implementation exists and is version-controlled.
- The feature works with all public Internet/WAN connectivity disabled.
- Authorization, malformed input, duplicate retry, and failure behavior have explicit tests.
- If financial value changes, journal debits and credits balance exactly and historical entries remain immutable.
- If AI is involved, model/prompt/feature version and fallback behavior are recorded and deterministic policy remains final.
- A timeout/restart cannot create a duplicate financial result; request status can be rediscovered.
- Logs/evidence contain correlation references but not secrets or prohibited sensitive payment data.

Common Failure Modes

- Lab requires Internet despite offline claim.
- Commands cannot be repeated from clean state.
- No retained evidence.
- Failure case skipped.

Expert Review Questions

- What happens if this component succeeds but the next component fails?
- How is the authorizing identity/device/key revoked or recovered?
- Can the operation be safely retried after the client loses the response?
- What happens if AI is unavailable, slow, or wrong?
- Can an auditor reconstruct debit, credit, actor, risk decision, approval, and Fabric txId from retained evidence?
- Which compensating transaction resolves a downstream operational error without rewriting history?

Chapter 294 - Lab 5 - Implement Holds Capture Release and Refund

Add merchant-style authorization and refund lifecycle. Each lab is reproducible evidence. Complete it from a clean environment with the WAN disconnected, retain commands/config hashes/test outputs, and prove both successful behavior and at least one failure/recovery path.

Design rule: Create hold

Implementation Objective

The required output is: Payment lifecycle implementation. It must be version-controlled, reviewable, testable with the WAN disabled, and linked to the architecture/source-of-truth model. For a payment platform, every implementation decision should make it possible to explain who authorized an action, which facts were trusted, which controls ran, what financial journal committed, and how the result is recovered after failure.

Key Engineering Decisions

- Create hold
- partial/full capture
- release
- refund
- reject illegal transitions
- verify journal balance

Deep Implementation Guidance

Store lab evidence in a versioned folder: terminal transcript, config/artifact hashes, test output, transaction references, screenshots where useful, and a short result/gap report. A lab is not complete if it cannot be rerun offline from a clean snapshot.

Data and Control Questions

- What is the authoritative source for every input used by this component?
- Which user, device, service, MSP, or administrator identity is allowed to initiate or approve it?
- Does the operation move value, reserve value, create a side effect, or only provide advisory information?
- What idempotency/replay key proves that a retry is the same business intent?
- What evidence is retained off-chain and what stable reference/hash belongs on the ledger?

Chapter 294 Implementation - Lab 5 - Implement Holds Capture Release and Refund

Implementation Procedure

1466. Start from a clean snapshot and verify the WAN is disabled.
1467. Apply only the documented offline artifacts/configuration.
1468. Run the implementation procedure and capture evidence.
1469. Execute success, failure, and retry/recovery tests.
1470. Write a short result report including versions, hashes, gaps, and next actions.

Implementation Example

```
# Lab 5 - Implement Holds Capture Release and Refund
INPUT  -> authenticate, validate schema, bind requestId/nonce
FACTS  -> read authoritative wallet/merchant/state data
AI     -> optional bounded recommendation with decisionId/version
POLICY -> enforce ownership, hard limits, approvals, risk fallback
LEDGER -> submit one idempotent deterministic transaction when value changes
EVENT  -> update projection/case/monitoring asynchronously
VERIFY -> discover committed result and retain correlationId + txId evidence
```

Verification Checklist

- The payment lifecycle implementation exists and is version-controlled.
- The feature works with all public Internet/WAN connectivity disabled.
- Authorization, malformed input, duplicate retry, and failure behavior have explicit tests.
- If financial value changes, journal debits and credits balance exactly and historical entries remain immutable.
- If AI is involved, model/prompt/feature version and fallback behavior are recorded and deterministic policy remains final.
- A timeout/restart cannot create a duplicate financial result; request status can be rediscovered.
- Logs/evidence contain correlation references but not secrets or prohibited sensitive payment data.

Common Failure Modes

- Lab requires Internet despite offline claim.
- Commands cannot be repeated from clean state.
- No retained evidence.
- Failure case skipped.

Expert Review Questions

- What happens if this component succeeds but the next component fails?
- How is the authorizing identity/device/key revoked or recovered?
- Can the operation be safely retried after the client loses the response?
- What happens if AI is unavailable, slow, or wrong?
- Can an auditor reconstruct debit, credit, actor, risk decision, approval, and Fabric txId from retained evidence?
- Which compensating transaction resolves a downstream operational error without rewriting history?

Chapter 295 - Lab 6 - Build the Fabric Gateway REST API

Connect Node.js/TypeScript API to the local peer Gateway. Each lab is reproducible evidence. Complete it from a clean environment with the WAN disconnected, retain commands/config hashes/test outputs, and prove both successful behavior and at least one failure/recovery path.

Design rule: TLS client connection

Implementation Objective

The required output is: Local REST service. It must be version-controlled, reviewable, testable with the WAN disabled, and linked to the architecture/source-of-truth model. For a payment platform, every implementation decision should make it possible to explain who authorized an action, which facts were trusted, which controls ran, what financial journal committed, and how the result is recovered after failure.

Key Engineering Decisions

- TLS client connection
- auth middleware
- schema validation
- submit/evaluate wrappers
- commit status and stable errors

Deep Implementation Guidance

Store lab evidence in a versioned folder: terminal transcript, config/artifact hashes, test output, transaction references, screenshots where useful, and a short result/gap report. A lab is not complete if it cannot be rerun offline from a clean snapshot.

Data and Control Questions

- What is the authoritative source for every input used by this component?
- Which user, device, service, MSP, or administrator identity is allowed to initiate or approve it?
- Does the operation move value, reserve value, create a side effect, or only provide advisory information?
- What idempotency/replay key proves that a retry is the same business intent?
- What evidence is retained off-chain and what stable reference/hash belongs on the ledger?

Chapter 295 Implementation - Lab 6 - Build the Fabric Gateway REST API

Implementation Procedure

1471. Start from a clean snapshot and verify the WAN is disabled.
1472. Apply only the documented offline artifacts/configuration.
1473. Run the implementation procedure and capture evidence.
1474. Execute success, failure, and retry/recovery tests.
1475. Write a short result report including versions, hashes, gaps, and next actions.

Implementation Example

```
# Lab 6 - Build the Fabric Gateway REST API
INPUT  -> authenticate, validate schema, bind requestId/nonce
FACTS  -> read authoritative wallet/merchant/state data
AI     -> optional bounded recommendation with decisionId/version
POLICY -> enforce ownership, hard limits, approvals, risk fallback
LEDGER -> submit one idempotent deterministic transaction when value changes
EVENT  -> update projection/case/monitoring asynchronously
VERIFY -> discover committed result and retain correlationId + txId evidence
```

Verification Checklist

- The local rest service exists and is version-controlled.
- The feature works with all public Internet/WAN connectivity disabled.
- Authorization, malformed input, duplicate retry, and failure behavior have explicit tests.
- If financial value changes, journal debits and credits balance exactly and historical entries remain immutable.
- If AI is involved, model/prompt/feature version and fallback behavior are recorded and deterministic policy remains final.
- A timeout/restart cannot create a duplicate financial result; request status can be rediscovered.
- Logs/evidence contain correlation references but not secrets or prohibited sensitive payment data.

Common Failure Modes

- Lab requires Internet despite offline claim.
- Commands cannot be repeated from clean state.
- No retained evidence.
- Failure case skipped.

Expert Review Questions

- What happens if this component succeeds but the next component fails?
- How is the authorizing identity/device/key revoked or recovered?
- Can the operation be safely retried after the client loses the response?
- What happens if AI is unavailable, slow, or wrong?
- Can an auditor reconstruct debit, credit, actor, risk decision, approval, and Fabric txId from retained evidence?
- Which compensating transaction resolves a downstream operational error without rewriting history?

Chapter 296 - Lab 7 - Build PostgreSQL Projection and Reconciliation

Consume events into searchable tables and compare with ledger. Each lab is reproducible evidence. Complete it from a clean environment with the WAN disconnected, retain commands/config hashes/test outputs, and prove both successful behavior and at least one failure/recovery path.

Design rule: Checkpoint events

Implementation Objective

The required output is: Projection worker. It must be version-controlled, reviewable, testable with the WAN disabled, and linked to the architecture/source-of-truth model. For a payment platform, every implementation decision should make it possible to explain who authorized an action, which facts were trusted, which controls ran, what financial journal committed, and how the result is recovered after failure.

Key Engineering Decisions

- Checkpoint events
- idempotent writes
- rebuild from ledger/event export
- run reconciliation report

Deep Implementation Guidance

Store lab evidence in a versioned folder: terminal transcript, config/artifact hashes, test output, transaction references, screenshots where useful, and a short result/gap report. A lab is not complete if it cannot be rerun offline from a clean snapshot.

Data and Control Questions

- What is the authoritative source for every input used by this component?
- Which user, device, service, MSP, or administrator identity is allowed to initiate or approve it?
- Does the operation move value, reserve value, create a side effect, or only provide advisory information?
- What idempotency/replay key proves that a retry is the same business intent?
- What evidence is retained off-chain and what stable reference/hash belongs on the ledger?

Chapter 296 Implementation - Lab 7 - Build PostgreSQL Projection and Reconciliation

Implementation Procedure

1476. Start from a clean snapshot and verify the WAN is disabled.
1477. Apply only the documented offline artifacts/configuration.
1478. Run the implementation procedure and capture evidence.
1479. Execute success, failure, and retry/recovery tests.
1480. Write a short result report including versions, hashes, gaps, and next actions.

Implementation Example

```
# Lab 7 - Build PostgreSQL Projection and Reconciliation
INPUT  -> authenticate, validate schema, bind requestId/nonce
FACTS  -> read authoritative wallet/merchant/state data
AI     -> optional bounded recommendation with decisionId/version
POLICY -> enforce ownership, hard limits, approvals, risk fallback
LEDGER -> submit one idempotent deterministic transaction when value changes
EVENT  -> update projection/case/monitoring asynchronously
VERIFY -> discover committed result and retain correlationId + txId evidence
```

Verification Checklist

- The projection worker exists and is version-controlled.
- The feature works with all public Internet/WAN connectivity disabled.
- Authorization, malformed input, duplicate retry, and failure behavior have explicit tests.
- If financial value changes, journal debits and credits balance exactly and historical entries remain immutable.
- If AI is involved, model/prompt/feature version and fallback behavior are recorded and deterministic policy remains final.
- A timeout/restart cannot create a duplicate financial result; request status can be rediscovered.
- Logs/evidence contain correlation references but not secrets or prohibited sensitive payment data.

Common Failure Modes

- Lab requires Internet despite offline claim.
- Commands cannot be repeated from clean state.
- No retained evidence.
- Failure case skipped.

Expert Review Questions

- What happens if this component succeeds but the next component fails?
- How is the authorizing identity/device/key revoked or recovered?
- Can the operation be safely retried after the client loses the response?
- What happens if AI is unavailable, slow, or wrong?
- Can an auditor reconstruct debit, credit, actor, risk decision, approval, and Fabric txId from retained evidence?
- Which compensating transaction resolves a downstream operational error without rewriting history?

Chapter 297 - Lab 8 - Build Customer Wallet and Merchant POS

Implement local browser/POS flows with safe pending/retry states. Each lab is reproducible evidence. Complete it from a clean environment with the WAN disconnected, retain commands/config hashes/test outputs, and prove both successful behavior and at least one failure/recovery path.

Design rule: No CDN

Implementation Objective

The required output is: Wallet and POS UI. It must be version-controlled, reviewable, testable with the WAN disabled, and linked to the architecture/source-of-truth model. For a payment platform, every implementation decision should make it possible to explain who authorized an action, which facts were trusted, which controls ran, what financial journal committed, and how the result is recovered after failure.

Key Engineering Decisions

- No CDN
- P2P transfer
- QR merchant payment
- status recovery
- receipt reference
- role-aware navigation

Deep Implementation Guidance

Store lab evidence in a versioned folder: terminal transcript, config/artifact hashes, test output, transaction references, screenshots where useful, and a short result/gap report. A lab is not complete if it cannot be rerun offline from a clean snapshot.

Data and Control Questions

- What is the authoritative source for every input used by this component?
- Which user, device, service, MSP, or administrator identity is allowed to initiate or approve it?
- Does the operation move value, reserve value, create a side effect, or only provide advisory information?
- What idempotency/replay key proves that a retry is the same business intent?
- What evidence is retained off-chain and what stable reference/hash belongs on the ledger?

Chapter 297 Implementation - Lab 8 - Build Customer Wallet and Merchant POS

Implementation Procedure

1481. Start from a clean snapshot and verify the WAN is disabled.
1482. Apply only the documented offline artifacts/configuration.
1483. Run the implementation procedure and capture evidence.
1484. Execute success, failure, and retry/recovery tests.
1485. Write a short result report including versions, hashes, gaps, and next actions.

Implementation Example

```
# Lab 8 - Build Customer Wallet and Merchant POS
INPUT  -> authenticate, validate schema, bind requestId/nonce
FACTS  -> read authoritative wallet/merchant/state data
AI     -> optional bounded recommendation with decisionId/version
POLICY -> enforce ownership, hard limits, approvals, risk fallback
LEDGER -> submit one idempotent deterministic transaction when value changes
EVENT  -> update projection/case/monitoring asynchronously
VERIFY -> discover committed result and retain correlationId + txId evidence
```

Verification Checklist

- The wallet and pos ui exists and is version-controlled.
- The feature works with all public Internet/WAN connectivity disabled.
- Authorization, malformed input, duplicate retry, and failure behavior have explicit tests.
- If financial value changes, journal debits and credits balance exactly and historical entries remain immutable.
- If AI is involved, model/prompt/feature version and fallback behavior are recorded and deterministic policy remains final.
- A timeout/restart cannot create a duplicate financial result; request status can be rediscovered.
- Logs/evidence contain correlation references but not secrets or prohibited sensitive payment data.

Common Failure Modes

- Lab requires Internet despite offline claim.
- Commands cannot be repeated from clean state.
- No retained evidence.
- Failure case skipped.

Expert Review Questions

- What happens if this component succeeds but the next component fails?
- How is the authorizing identity/device/key revoked or recovered?
- Can the operation be safely retried after the client loses the response?
- What happens if AI is unavailable, slow, or wrong?
- Can an auditor reconstruct debit, credit, actor, risk decision, approval, and Fabric txId from retained evidence?
- Which compensating transaction resolves a downstream operational error without rewriting history?

Chapter 298 - Lab 9 - Build Fraud Rules and Feature Pipeline

Create deterministic baseline rules and versioned features. Each lab is reproducible evidence. Complete it from a clean environment with the WAN disconnected, retain commands/config hashes/test outputs, and prove both successful behavior and at least one failure/recovery path.

Design rule: Velocity/new-device/high-value/new-recipient rules

Implementation Objective

The required output is: Risk feature service. It must be version-controlled, reviewable, testable with the WAN disabled, and linked to the architecture/source-of-truth model. For a payment platform, every implementation decision should make it possible to explain who authorized an action, which facts were trusted, which controls ran, what financial journal committed, and how the result is recovered after failure.

Key Engineering Decisions

- Velocity/new-device/high-value/new-recipient rules
- feature schema
- local dataset export
- decision audit records

Deep Implementation Guidance

Store lab evidence in a versioned folder: terminal transcript, config/artifact hashes, test output, transaction references, screenshots where useful, and a short result/gap report. A lab is not complete if it cannot be rerun offline from a clean snapshot.

Data and Control Questions

- What is the authoritative source for every input used by this component?
- Which user, device, service, MSP, or administrator identity is allowed to initiate or approve it?
- Does the operation move value, reserve value, create a side effect, or only provide advisory information?
- What idempotency/replay key proves that a retry is the same business intent?
- What evidence is retained off-chain and what stable reference/hash belongs on the ledger?

Chapter 298 Implementation - Lab 9 - Build Fraud Rules and Feature Pipeline

Implementation Procedure

1486. Start from a clean snapshot and verify the WAN is disabled.
1487. Apply only the documented offline artifacts/configuration.
1488. Run the implementation procedure and capture evidence.
1489. Execute success, failure, and retry/recovery tests.
1490. Write a short result report including versions, hashes, gaps, and next actions.

Implementation Example

```
# Lab 9 - Build Fraud Rules and Feature Pipeline
INPUT  -> authenticate, validate schema, bind requestId/nonce
FACTS  -> read authoritative wallet/merchant/state data
AI     -> optional bounded recommendation with decisionId/version
POLICY -> enforce ownership, hard limits, approvals, risk fallback
LEDGER -> submit one idempotent deterministic transaction when value changes
EVENT  -> update projection/case/monitoring asynchronously
VERIFY -> discover committed result and retain correlationId + txId evidence
```

Verification Checklist

- The risk feature service exists and is version-controlled.
- The feature works with all public Internet/WAN connectivity disabled.
- Authorization, malformed input, duplicate retry, and failure behavior have explicit tests.
- If financial value changes, journal debits and credits balance exactly and historical entries remain immutable.
- If AI is involved, model/prompt/feature version and fallback behavior are recorded and deterministic policy remains final.
- A timeout/restart cannot create a duplicate financial result; request status can be rediscovered.
- Logs/evidence contain correlation references but not secrets or prohibited sensitive payment data.

Common Failure Modes

- Lab requires Internet despite offline claim.
- Commands cannot be repeated from clean state.
- No retained evidence.
- Failure case skipped.

Expert Review Questions

- What happens if this component succeeds but the next component fails?
- How is the authorizing identity/device/key revoked or recovered?
- Can the operation be safely retried after the client loses the response?
- What happens if AI is unavailable, slow, or wrong?
- Can an auditor reconstruct debit, credit, actor, risk decision, approval, and Fabric txId from retained evidence?
- Which compensating transaction resolves a downstream operational error without rewriting history?

Chapter 299 - Lab 10 - Train and Serve an ONNX Risk Model

Train a simple model offline, export to ONNX, and serve through FastAPI. Each lab is reproducible evidence. Complete it from a clean environment with the WAN disconnected, retain commands/config hashes/test outputs, and prove both successful behavior and at least one failure/recovery path.

Design rule: Time-aware split

Implementation Objective

The required output is: ONNX risk service. It must be version-controlled, reviewable, testable with the WAN disabled, and linked to the architecture/source-of-truth model. For a payment platform, every implementation decision should make it possible to explain who authorized an action, which facts were trusted, which controls ran, what financial journal committed, and how the result is recovered after failure.

Key Engineering Decisions

- Time-aware split
- evaluate precision/recall/calibration
- hash model
- load with ONNX Runtime
- strict response schema

Deep Implementation Guidance

Store lab evidence in a versioned folder: terminal transcript, config/artifact hashes, test output, transaction references, screenshots where useful, and a short result/gap report. A lab is not complete if it cannot be rerun offline from a clean snapshot.

Data and Control Questions

- What is the authoritative source for every input used by this component?
- Which user, device, service, MSP, or administrator identity is allowed to initiate or approve it?
- Does the operation move value, reserve value, create a side effect, or only provide advisory information?
- What idempotency/replay key proves that a retry is the same business intent?
- What evidence is retained off-chain and what stable reference/hash belongs on the ledger?

Chapter 299 Implementation - Lab 10 - Train and Serve an ONNX Risk Model

Implementation Procedure

1491. Start from a clean snapshot and verify the WAN is disabled.
1492. Apply only the documented offline artifacts/configuration.
1493. Run the implementation procedure and capture evidence.
1494. Execute success, failure, and retry/recovery tests.
1495. Write a short result report including versions, hashes, gaps, and next actions.

Implementation Example

```
# Lab 10 - Train and Serve an ONNX Risk Model
INPUT  -> authenticate, validate schema, bind requestId/nonce
FACTS  -> read authoritative wallet/merchant/state data
AI     -> optional bounded recommendation with decisionId/version
POLICY -> enforce ownership, hard limits, approvals, risk fallback
LEDGER -> submit one idempotent deterministic transaction when value changes
EVENT  -> update projection/case/monitoring asynchronously
VERIFY -> discover committed result and retain correlationId + txId evidence
```

Verification Checklist

- The onnx risk service exists and is version-controlled.
- The feature works with all public Internet/WAN connectivity disabled.
- Authorization, malformed input, duplicate retry, and failure behavior have explicit tests.
- If financial value changes, journal debits and credits balance exactly and historical entries remain immutable.
- If AI is involved, model/prompt/feature version and fallback behavior are recorded and deterministic policy remains final.
- A timeout/restart cannot create a duplicate financial result; request status can be rediscovered.
- Logs/evidence contain correlation references but not secrets or prohibited sensitive payment data.

Common Failure Modes

- Lab requires Internet despite offline claim.
- Commands cannot be repeated from clean state.
- No retained evidence.
- Failure case skipped.

Expert Review Questions

- What happens if this component succeeds but the next component fails?
- How is the authorizing identity/device/key revoked or recovered?
- Can the operation be safely retried after the client loses the response?
- What happens if AI is unavailable, slow, or wrong?
- Can an auditor reconstruct debit, credit, actor, risk decision, approval, and Fabric txId from retained evidence?
- Which compensating transaction resolves a downstream operational error without rewriting history?

Chapter 300 - Lab 11 - Build Local LLM and RAG

Run llama.cpp, local embeddings, pgvector, and a policy assistant. Each lab is reproducible evidence. Complete it from a clean environment with the WAN disconnected, retain commands/config hashes/test outputs, and prove both successful behavior and at least one failure/recovery path.

Design rule: Import approved model

Implementation Objective

The required output is: Local RAG assistant. It must be version-controlled, reviewable, testable with the WAN disabled, and linked to the architecture/source-of-truth model. For a payment platform, every implementation decision should make it possible to explain who authorized an action, which facts were trusted, which controls ran, what financial journal committed, and how the result is recovered after failure.

Key Engineering Decisions

- Import approved model
- index policy docs
- permission filters
- schema output
- citations/no-answer
- injection tests

Deep Implementation Guidance

Store lab evidence in a versioned folder: terminal transcript, config/artifact hashes, test output, transaction references, screenshots where useful, and a short result/gap report. A lab is not complete if it cannot be rerun offline from a clean snapshot.

Data and Control Questions

- What is the authoritative source for every input used by this component?
- Which user, device, service, MSP, or administrator identity is allowed to initiate or approve it?
- Does the operation move value, reserve value, create a side effect, or only provide advisory information?
- What idempotency/replay key proves that a retry is the same business intent?
- What evidence is retained off-chain and what stable reference/hash belongs on the ledger?

Chapter 300 Implementation - Lab 11 - Build Local LLM and RAG

Implementation Procedure

1496. Start from a clean snapshot and verify the WAN is disabled.
1497. Apply only the documented offline artifacts/configuration.
1498. Run the implementation procedure and capture evidence.
1499. Execute success, failure, and retry/recovery tests.
1500. Write a short result report including versions, hashes, gaps, and next actions.

Implementation Example

```
# Lab 11 - Build Local LLM and RAG
INPUT  -> authenticate, validate schema, bind requestId/nonce
FACTS  -> read authoritative wallet/merchant/state data
AI     -> optional bounded recommendation with decisionId/version
POLICY -> enforce ownership, hard limits, approvals, risk fallback
LEDGER -> submit one idempotent deterministic transaction when value changes
EVENT  -> update projection/case/monitoring asynchronously
VERIFY -> discover committed result and retain correlationId + txId evidence
```

Verification Checklist

- The local rag assistant exists and is version-controlled.
- The feature works with all public Internet/WAN connectivity disabled.
- Authorization, malformed input, duplicate retry, and failure behavior have explicit tests.
- If financial value changes, journal debits and credits balance exactly and historical entries remain immutable.
- If AI is involved, model/prompt/feature version and fallback behavior are recorded and deterministic policy remains final.
- A timeout/restart cannot create a duplicate financial result; request status can be rediscovered.
- Logs/evidence contain correlation references but not secrets or prohibited sensitive payment data.

Common Failure Modes

- Lab requires Internet despite offline claim.
- Commands cannot be repeated from clean state.
- No retained evidence.
- Failure case skipped.

Expert Review Questions

- What happens if this component succeeds but the next component fails?
- How is the authorizing identity/device/key revoked or recovered?
- Can the operation be safely retried after the client loses the response?
- What happens if AI is unavailable, slow, or wrong?
- Can an auditor reconstruct debit, credit, actor, risk decision, approval, and Fabric txId from retained evidence?
- Which compensating transaction resolves a downstream operational error without rewriting history?

Chapter 301 - Lab 12 - Integrate AI Risk with Deterministic Payment Gate

Call local risk service before Fabric submission without making AI authoritative. Each lab is reproducible evidence. Complete it from a clean environment with the WAN disconnected, retain commands/config hashes/test outputs, and prove both successful behavior and at least one failure/recovery path.

Design rule: Timeout policy

Implementation Objective

The required output is: Integrated risk gate. It must be version-controlled, reviewable, testable with the WAN disabled, and linked to the architecture/source-of-truth model. For a payment platform, every implementation decision should make it possible to explain who authorized an action, which facts were trusted, which controls ran, what financial journal committed, and how the result is recovered after failure.

Key Engineering Decisions

- Timeout policy
- hard limits
- ALLOW REVIEW BLOCK mapping
- record decisionId/modelVersion
- submit only after deterministic checks

Deep Implementation Guidance

Store lab evidence in a versioned folder: terminal transcript, config/artifact hashes, test output, transaction references, screenshots where useful, and a short result/gap report. A lab is not complete if it cannot be rerun offline from a clean snapshot.

Data and Control Questions

- What is the authoritative source for every input used by this component?
- Which user, device, service, MSP, or administrator identity is allowed to initiate or approve it?
- Does the operation move value, reserve value, create a side effect, or only provide advisory information?
- What idempotency/replay key proves that a retry is the same business intent?
- What evidence is retained off-chain and what stable reference/hash belongs on the ledger?

Chapter 301 Implementation - Lab 12 - Integrate AI Risk with Deterministic Payment Gate

Implementation Procedure

1501. Start from a clean snapshot and verify the WAN is disabled.
1502. Apply only the documented offline artifacts/configuration.
1503. Run the implementation procedure and capture evidence.
1504. Execute success, failure, and retry/recovery tests.
1505. Write a short result report including versions, hashes, gaps, and next actions.

Implementation Example

```
# Lab 12 - Integrate AI Risk with Deterministic Payment Gate
INPUT  -> authenticate, validate schema, bind requestId/nonce
FACTS  -> read authoritative wallet/merchant/state data
AI      -> optional bounded recommendation with decisionId/version
POLICY  -> enforce ownership, hard limits, approvals, risk fallback
LEDGER  -> submit one idempotent deterministic transaction when value changes
EVENT   -> update projection/case/monitoring asynchronously
VERIFY  -> discover committed result and retain correlationId + txId evidence
```

Verification Checklist

- The integrated risk gate exists and is version-controlled.
- The feature works with all public Internet/WAN connectivity disabled.
- Authorization, malformed input, duplicate retry, and failure behavior have explicit tests.
- If financial value changes, journal debits and credits balance exactly and historical entries remain immutable.
- If AI is involved, model/prompt/feature version and fallback behavior are recorded and deterministic policy remains final.
- A timeout/restart cannot create a duplicate financial result; request status can be rediscovered.
- Logs/evidence contain correlation references but not secrets or prohibited sensitive payment data.

Common Failure Modes

- Lab requires Internet despite offline claim.
- Commands cannot be repeated from clean state.
- No retained evidence.
- Failure case skipped.

Expert Review Questions

- What happens if this component succeeds but the next component fails?
- How is the authorizing identity/device/key revoked or recovered?
- Can the operation be safely retried after the client loses the response?
- What happens if AI is unavailable, slow, or wrong?
- Can an auditor reconstruct debit, credit, actor, risk decision, approval, and Fabric txId from retained evidence?
- Which compensating transaction resolves a downstream operational error without rewriting history?

Chapter 302 - Lab 13 - Build Human Review and Dual Approval

Pause selected payments/issuance for human decision. Each lab is reproducible evidence. Complete it from a clean environment with the WAN disconnected, retain commands/config hashes/test outputs, and prove both successful behavior and at least one failure/recovery path.

Design rule: Queue immutable request snapshot

Implementation Objective

The required output is: Review/approval workflow. It must be version-controlled, reviewable, testable with the WAN disabled, and linked to the architecture/source-of-truth model. For a payment platform, every implementation decision should make it possible to explain who authorized an action, which facts were trusted, which controls ran, what financial journal committed, and how the result is recovered after failure.

Key Engineering Decisions

- Queue immutable request snapshot
- separate requester/approver
- expiry
- record reason
- link approval to ledger transaction

Deep Implementation Guidance

Store lab evidence in a versioned folder: terminal transcript, config/artifact hashes, test output, transaction references, screenshots where useful, and a short result/gap report. A lab is not complete if it cannot be rerun offline from a clean snapshot.

Data and Control Questions

- What is the authoritative source for every input used by this component?
- Which user, device, service, MSP, or administrator identity is allowed to initiate or approve it?
- Does the operation move value, reserve value, create a side effect, or only provide advisory information?
- What idempotency/replay key proves that a retry is the same business intent?
- What evidence is retained off-chain and what stable reference/hash belongs on the ledger?

Chapter 302 Implementation - Lab 13 - Build Human Review and Dual Approval

Implementation Procedure

1506. Start from a clean snapshot and verify the WAN is disabled.
1507. Apply only the documented offline artifacts/configuration.
1508. Run the implementation procedure and capture evidence.
1509. Execute success, failure, and retry/recovery tests.
1510. Write a short result report including versions, hashes, gaps, and next actions.

Implementation Example

```
# Lab 13 - Build Human Review and Dual Approval
INPUT  -> authenticate, validate schema, bind requestId/nonce
FACTS  -> read authoritative wallet/merchant/state data
AI      -> optional bounded recommendation with decisionId/version
POLICY  -> enforce ownership, hard limits, approvals, risk fallback
LEDGER  -> submit one idempotent deterministic transaction when value changes
EVENT   -> update projection/case/monitoring asynchronously
VERIFY  -> discover committed result and retain correlationId + txId evidence
```

Verification Checklist

- The review/approval workflow exists and is version-controlled.
- The feature works with all public Internet/WAN connectivity disabled.
- Authorization, malformed input, duplicate retry, and failure behavior have explicit tests.
- If financial value changes, journal debits and credits balance exactly and historical entries remain immutable.
- If AI is involved, model/prompt/feature version and fallback behavior are recorded and deterministic policy remains final.
- A timeout/restart cannot create a duplicate financial result; request status can be rediscovered.
- Logs/evidence contain correlation references but not secrets or prohibited sensitive payment data.

Common Failure Modes

- Lab requires Internet despite offline claim.
- Commands cannot be repeated from clean state.
- No retained evidence.
- Failure case skipped.

Expert Review Questions

- What happens if this component succeeds but the next component fails?
- How is the authorizing identity/device/key revoked or recovered?
- Can the operation be safely retried after the client loses the response?
- What happens if AI is unavailable, slow, or wrong?
- Can an auditor reconstruct debit, credit, actor, risk decision, approval, and Fabric txId from retained evidence?
- Which compensating transaction resolves a downstream operational error without rewriting history?

Chapter 303 - Lab 14 - Run Security and Failure Drills

Attack replay auth bypass quorum loss DB outage AI outage injection and model tampering. Each lab is reproducible evidence. Complete it from a clean environment with the WAN disconnected, retain commands/config hashes/test outputs, and prove both successful behavior and at least one failure/recovery path.

Design rule: Prove controls/fallbacks

Implementation Objective

The required output is: Failure/security evidence pack. It must be version-controlled, reviewable, testable with the WAN disabled, and linked to the architecture/source-of-truth model. For a payment platform, every implementation decision should make it possible to explain who authorized an action, which facts were trusted, which controls ran, what financial journal committed, and how the result is recovered after failure.

Key Engineering Decisions

- Prove controls/fallbacks
- record tx IDs and logs
- exercise kill switch
- verify no duplicate/unauthorized value movement

Deep Implementation Guidance

Store lab evidence in a versioned folder: terminal transcript, config/artifact hashes, test output, transaction references, screenshots where useful, and a short result/gap report. A lab is not complete if it cannot be rerun offline from a clean snapshot.

Data and Control Questions

- What is the authoritative source for every input used by this component?
- Which user, device, service, MSP, or administrator identity is allowed to initiate or approve it?
- Does the operation move value, reserve value, create a side effect, or only provide advisory information?
- What idempotency/replay key proves that a retry is the same business intent?
- What evidence is retained off-chain and what stable reference/hash belongs on the ledger?

Chapter 303 Implementation - Lab 14 - Run Security and Failure Drills

Implementation Procedure

1511. Start from a clean snapshot and verify the WAN is disabled.
1512. Apply only the documented offline artifacts/configuration.
1513. Run the implementation procedure and capture evidence.
1514. Execute success, failure, and retry/recovery tests.
1515. Write a short result report including versions, hashes, gaps, and next actions.

Implementation Example

```
# Lab 14 - Run Security and Failure Drills
INPUT  -> authenticate, validate schema, bind requestId/nonce
FACTS  -> read authoritative wallet/merchant/state data
AI     -> optional bounded recommendation with decisionId/version
POLICY -> enforce ownership, hard limits, approvals, risk fallback
LEDGER -> submit one idempotent deterministic transaction when value changes
EVENT  -> update projection/case/monitoring asynchronously
VERIFY -> discover committed result and retain correlationId + txId evidence
```

Verification Checklist

- The failure/security evidence pack exists and is version-controlled.
- The feature works with all public Internet/WAN connectivity disabled.
- Authorization, malformed input, duplicate retry, and failure behavior have explicit tests.
- If financial value changes, journal debits and credits balance exactly and historical entries remain immutable.
- If AI is involved, model/prompt/feature version and fallback behavior are recorded and deterministic policy remains final.
- A timeout/restart cannot create a duplicate financial result; request status can be rediscovered.
- Logs/evidence contain correlation references but not secrets or prohibited sensitive payment data.

Common Failure Modes

- Lab requires Internet despite offline claim.
- Commands cannot be repeated from clean state.
- No retained evidence.
- Failure case skipped.

Expert Review Questions

- What happens if this component succeeds but the next component fails?
- How is the authorizing identity/device/key revoked or recovered?
- Can the operation be safely retried after the client loses the response?
- What happens if AI is unavailable, slow, or wrong?
- Can an auditor reconstruct debit, credit, actor, risk decision, approval, and Fabric txId from retained evidence?
- Which compensating transaction resolves a downstream operational error without rewriting history?

Chapter 304 - Lab 15 - Backup Restore Reconcile and Go-Live Rehearsal

Recover the full system on isolated hardware and execute a controlled launch rehearsal. Each lab is reproducible evidence. Complete it from a clean environment with the WAN disconnected, retain commands/config hashes/test outputs, and prove both successful behavior and at least one failure/recovery path.

Design rule: Restore Fabric/DB/config/models

Implementation Objective

The required output is: Go-live evidence pack. It must be version-controlled, reviewable, testable with the WAN disabled, and linked to the architecture/source-of-truth model. For a payment platform, every implementation decision should make it possible to explain who authorized an action, which facts were trusted, which controls ran, what financial journal committed, and how the result is recovered after failure.

Key Engineering Decisions

- Restore Fabric/DB/config/models
- rotate a cert
- run payments/refunds
- reconcile
- measure RTO
- execute rollback/incident runbook

Deep Implementation Guidance

Store lab evidence in a versioned folder: terminal transcript, config/artifact hashes, test output, transaction references, screenshots where useful, and a short result/gap report. A lab is not complete if it cannot be rerun offline from a clean snapshot.

Data and Control Questions

- What is the authoritative source for every input used by this component?
- Which user, device, service, MSP, or administrator identity is allowed to initiate or approve it?
- Does the operation move value, reserve value, create a side effect, or only provide advisory information?
- What idempotency/replay key proves that a retry is the same business intent?
- What evidence is retained off-chain and what stable reference/hash belongs on the ledger?

Chapter 304 Implementation - Lab 15 - Backup Restore Reconcile and Go-Live Rehearsal

Implementation Procedure

1516. Start from a clean snapshot and verify the WAN is disabled.
1517. Apply only the documented offline artifacts/configuration.
1518. Run the implementation procedure and capture evidence.
1519. Execute success, failure, and retry/recovery tests.
1520. Write a short result report including versions, hashes, gaps, and next actions.

Implementation Example

```
# Lab 15 - Backup Restore Reconcile and Go-Live Rehearsal
INPUT  -> authenticate, validate schema, bind requestId/nonce
FACTS  -> read authoritative wallet/merchant/state data
AI     -> optional bounded recommendation with decisionId/version
POLICY -> enforce ownership, hard limits, approvals, risk fallback
LEDGER -> submit one idempotent deterministic transaction when value changes
EVENT  -> update projection/case/monitoring asynchronously
VERIFY -> discover committed result and retain correlationId + txId evidence
```

Verification Checklist

- The go-live evidence pack exists and is version-controlled.
- The feature works with all public Internet/WAN connectivity disabled.
- Authorization, malformed input, duplicate retry, and failure behavior have explicit tests.
- If financial value changes, journal debits and credits balance exactly and historical entries remain immutable.
- If AI is involved, model/prompt/feature version and fallback behavior are recorded and deterministic policy remains final.
- A timeout/restart cannot create a duplicate financial result; request status can be rediscovered.
- Logs/evidence contain correlation references but not secrets or prohibited sensitive payment data.

Common Failure Modes

- Lab requires Internet despite offline claim.
- Commands cannot be repeated from clean state.
- No retained evidence.
- Failure case skipped.

Expert Review Questions

- What happens if this component succeeds but the next component fails?
- How is the authorizing identity/device/key revoked or recovered?
- Can the operation be safely retried after the client loses the response?
- What happens if AI is unavailable, slow, or wrong?
- Can an auditor reconstruct debit, credit, actor, risk decision, approval, and Fabric txId from retained evidence?
- Which compensating transaction resolves a downstream operational error without rewriting history?

Appendix A - Recommended Repository Structure

Reference directory structure for infrastructure, chaincode, backend, AI, frontend, database, tests, runbooks, and release evidence.

```
wallet-platform/
infra/
  fabric/{compose,configtx,crypto,scripts}
  network/{dns,ntp,firewall,tls}
  monitoring/
chaincode/
  src/{contracts,models,accounting,auth,validation}
  test/
backend/
  src/{fabric,auth,risk,policy,routes,projection,jobs,audit}
  migrations/
ai/
  risk/{training,models,service,evals}
  llm/{models,prompts,schemas,rag,evals}
frontend/
  wallet/
  merchant-pos/
  treasury/
  review/
  audit/
data/
  postgres/
  knowledge/
operations/
  backup/ restore/ cert-rotation/ incident/ releases/
tests/
  integration/ concurrency/ security/ ai/ recovery/ uat/
docs/
  architecture/ adr/ api/ runbooks/ evidence/
```

Implementation Notes

Adapt all example hostnames, ports, roles, schemas, limits, certificate paths, model dimensions, and version pins to the controlled environment. Replace placeholders with values from the production inventory. Security-sensitive configuration should be independently reviewed and tested on an offline staging network before production.

Validation Checklist

- Examples match the real organization/role names and source-of-truth matrix.
- No placeholder/test credentials or private keys remain.
- Artifact versions and hashes match the approved offline release manifest.
- Failure/retry/recovery behavior is documented and tested.
- A second engineer/operator reviewed the final production adaptation.

Appendix B - Core TypeScript Wallet Chaincode

Consolidated reference patterns for wallets, balanced journals, transfer, holds, refunds, authorization evidence, and idempotency.

```
@Transaction()
public async Transfer(ctx: Context, requestId: string, inputJson: string): Promise<string> {
  const prior = await ctx.stub.getState('REQUEST~${requestId}');
  if (prior.length) return prior.toString('utf8');

  const input = JSON.parse(inputJson);
  assertPositiveInt(input.amountMinor);
  const from = await this.mustGetWallet(ctx, input.fromWalletId);
  const to = await this.mustGetWallet(ctx, input.toWalletId);
  this.requireDebitAuthority(ctx, from);
  if (from.asset !== to.asset || from.asset !== input.asset) throw new Error('ASSET_MISMATCH');
  if (from.status !== 'ACTIVE' || to.status !== 'ACTIVE') throw new Error('WALLET_NOT_ACTIVE');
  if (from.availableMinor < input.amountMinor) throw new Error('INSUFFICIENT_FUNDS');

  from.availableMinor -= input.amountMinor;
  to.availableMinor += input.amountMinor;
  const journal = balancedJournal(requestId, input.asset, [
    debit(from.walletId, input.amountMinor),
    credit(to.walletId, input.amountMinor),
  ]);
  await this.putWallet(ctx, from); await this.putWallet(ctx, to);
  await ctx.stub.putState('JOURNAL~${journal.journalId}', Buffer.from(JSON.stringify(journal)));
  const result = JSON.stringify({requestId, journalId: journal.journalId});
  await ctx.stub.putState('REQUEST~${requestId}', Buffer.from(result));
  ctx.stub.setEvent('TransferCommitted', Buffer.from(result));
  return result;
}

// Balanced-journal helper pattern
function assertBalanced(lines: JournalLine[]) {
  let d = 0n, c = 0n;
  for (const line of lines) {
    d += BigInt(line.debitMinor || 0);
    c += BigInt(line.creditMinor || 0);
  }
  if (d !== c) throw new Error('UNBALANCED_JOURNAL');
}

// Idempotency MUST be checked before writes.
const reqKey = 'REQUEST~${requestId}';
const prior = await ctx.stub.getState(reqKey);
if (prior.length) return prior.toString('utf8');
```

Implementation Notes

Adapt all example hostnames, ports, roles, schemas, limits, certificate paths, model dimensions, and version pins to the controlled environment. Replace placeholders with values from the production inventory. Security-sensitive configuration should be independently reviewed and tested on an offline staging network before production.

Validation Checklist

- Examples match the real organization/role names and source-of-truth matrix.
- No placeholder/test credentials or private keys remain.
- Artifact versions and hashes match the approved offline release manifest.

- Failure/retry/recovery behavior is documented and tested.
- A second engineer/operator reviewed the final production adaptation.

Appendix C - Node.js Fabric Gateway Backend

Gateway connection, submit/evaluate wrapper, authorization digest, risk decision, and transfer endpoint patterns.

```
import { createHash } from 'node:crypto';

export function authorizationDigest(canonicalAuthorization: Buffer): string {
  return createHash('sha256').update(canonicalAuthorization).digest('hex');
}

export async function submitJson(fn: string, ...args: string[]) {
  const bytes = await contract.submitTransaction(fn, ...args);
  return JSON.parse(Buffer.from(bytes).toString('utf8') || '{}');
}

app.post('/api/v1/transfers', requireUser, async (req, res) => {
  const requestId = req.header('Idempotency-Key');
  if (!requestId) return res.status(400).json({code:'IDEMPOTENCY_REQUIRED'});
  const input = transferSchema.parse(req.body);
  const auth = await verifyUserAndDeviceIntent(req.user, requestId, input);
  enforceHardTransferPolicy(req.user, input);
  const risk = await riskService.evaluate(buildFeatures(req.user, input));
  if (risk.recommendation === 'BLOCK') return res.status(403).json({code:'RISK_BLOCK', decisionId:risk.decisionId});
  if (risk.recommendation === 'REVIEW') return res.status(202).json(await createReviewCase(requestId,input,risk));
  const result = await submitJson("Transfer", requestId, JSON.stringify({...input, authDigest: auth.digest, riskDecisionId:risk.decisionId}));
  res.status(201).json(result);
});
```

Implementation Notes

Adapt all example hostnames, ports, roles, schemas, limits, certificate paths, model dimensions, and version pins to the controlled environment. Replace placeholders with values from the production inventory. Security-sensitive configuration should be independently reviewed and tested on an offline staging network before production.

Validation Checklist

- Examples match the real organization/role names and source-of-truth matrix.
- No placeholder/test credentials or private keys remain.
- Artifact versions and hashes match the approved offline release manifest.
- Failure/retry/recovery behavior is documented and tested.
- A second engineer/operator reviewed the final production adaptation.

Appendix D - Python ONNX Risk Service

A minimal local FastAPI/ONNX Runtime service pattern with structured risk output.

```
from fastapi import FastAPI
from pydantic import BaseModel, Field
import numpy as np
import onnxruntime as ort
import uuid

app = FastAPI()
session = ort.InferenceSession('/opt/models/risk-v7.onnx', providers=['CPUExecutionProvider'])

class RiskRequest(BaseModel):
    requestId: str
    amountMinor: int = Field(gt=0)
    features: list[float]

@app.post('/risk')
def risk(req: RiskRequest):
    x = np.asarray([req.features], dtype=np.float32)
    score = float(session.run(None, {session.get_inputs()[0].name: x})[0].reshape(-1)[0])
    recommendation = 'BLOCK' if score >= 0.95 else ('REVIEW' if score >= 0.70 else 'ALLOW')
    return {
        'decisionId': str(uuid.uuid4()),
        'modelVersion': 'risk-v7',
        'featureVersion': 'fv3',
        'riskScore': score,
        'recommendation': recommendation,
        'reasonCodes': []
    }
```

Implementation Notes

Adapt all example hostnames, ports, roles, schemas, limits, certificate paths, model dimensions, and version pins to the controlled environment. Replace placeholders with values from the production inventory. Security-sensitive configuration should be independently reviewed and tested on an offline staging network before production.

Validation Checklist

- Examples match the real organization/role names and source-of-truth matrix.
- No placeholder/test credentials or private keys remain.
- Artifact versions and hashes match the approved offline release manifest.
- Failure/retry/recovery behavior is documented and tested.
- A second engineer/operator reviewed the final production adaptation.

Appendix E - llama.cpp and Local RAG

Local server commands, OpenAI-compatible client example, PostgreSQL/pgvector schema, and policy retrieval pattern.

```
# Start local model server
./llama-server -m /opt/models/approved-assistant.gguf --host 10.20.30.40 --port 8080

# Python client pointed at LAN endpoint
from openai import OpenAI
client = OpenAI(base_url='http://10.20.30.40:8080/v1', api_key='local-placeholder')
resp = client.chat.completions.create(
    model='local-model',
    messages=[
        {'role':'system','content':'Answer only from supplied policy evidence. If evidence is insufficient, say so.'},
        {'role':'user','content':'What happens to a pending hold after expiry?'}
    ]
)
print(resp.choices[0].message.content)

# RAG flow
question -> authenticated role filters -> pgvector similarity search -> approved chunks
        -> local LLM -> answer + evidenceRefs -> application validation
```

Implementation Notes

Adapt all example hostnames, ports, roles, schemas, limits, certificate paths, model dimensions, and version pins to the controlled environment. Replace placeholders with values from the production inventory. Security-sensitive configuration should be independently reviewed and tested on an offline staging network before production.

Validation Checklist

- Examples match the real organization/role names and source-of-truth matrix.
- No placeholder/test credentials or private keys remain.
- Artifact versions and hashes match the approved offline release manifest.
- Failure/retry/recovery behavior is documented and tested.
- A second engineer/operator reviewed the final production adaptation.

Appendix F - Docker Compose and LAN Configuration

Representative Fabric/API/database/AI service configuration and offline deployment notes.

```
services:
  orderer1:
    image: hyperledger/fabric-orderer:2.5.16
    pull_policy: never
    environment:
      - ORDERER_GENERAL_LOCALMSPID=OrdererMSP
      - ORDERER_GENERAL_TLS_ENABLED=true
    volumes:
      - orderer1_data:/var/hyperledger/production/orderer

  peer0-wallet:
    image: hyperledger/fabric-peer:2.5.16
    pull_policy: never
    environment:
      - CORE_PEER_LOCALMSPID=WalletMSP
      - CORE_PEER_TLS_ENABLED=true
      - CORE_LEDGER_STATE_STATEDATABASE=CouchDB
    volumes:
      - peer_wallet_data:/var/hyperledger/production

  postgres:
    image: postgres:17
    pull_policy: never
    volumes:
      - postgres_data:/var/lib/postgresql/data

  risk-service:
    image: local/risk-service:1.0.0
    pull_policy: never
    environment:
      - MODEL_PATH=/models/risk-v7.onnx
    volumes:
      - ./approved-models:/models:ro

# In production, pin image digests in the offline release manifest.
```

Implementation Notes

Adapt all example hostnames, ports, roles, schemas, limits, certificate paths, model dimensions, and version pins to the controlled environment. Replace placeholders with values from the production inventory. Security-sensitive configuration should be independently reviewed and tested on an offline staging network before production.

Validation Checklist

- Examples match the real organization/role names and source-of-truth matrix.
- No placeholder/test credentials or private keys remain.
- Artifact versions and hashes match the approved offline release manifest.
- Failure/retry/recovery behavior is documented and tested.
- A second engineer/operator reviewed the final production adaptation.

Appendix G - REST API Specification

Endpoint catalog, idempotency headers, authentication expectations, and response/error conventions.

```
GET /api/v1/health
POST /api/v1/auth/login
GET /api/v1/wallets/:walletId
GET /api/v1/wallets/:walletId/statement
POST /api/v1/transfers      Idempotency-Key: <uuid>
POST /api/v1/merchant-payments  Idempotency-Key: <uuid>
POST /api/v1/holds          Idempotency-Key: <uuid>
POST /api/v1/holds/:holdId/capture  Idempotency-Key: <uuid>
POST /api/v1/holds/:holdId/release  Idempotency-Key: <uuid>
POST /api/v1/refunds        Idempotency-Key: <uuid>
POST /api/v1/treasury/issue  Idempotency-Key: <uuid> + approval
POST /api/v1/treasury/redeem Idempotency-Key: <uuid> + approval
GET /api/v1/transactions/by-request/:requestId
GET /api/v1/review/cases
POST /api/v1/review/cases/:caseId/decision
GET /api/v1/audit/transactions/:txId
```

Implementation Notes

Adapt all example hostnames, ports, roles, schemas, limits, certificate paths, model dimensions, and version pins to the controlled environment. Replace placeholders with values from the production inventory. Security-sensitive configuration should be independently reviewed and tested on an offline staging network before production.

Validation Checklist

- Examples match the real organization/role names and source-of-truth matrix.
- No placeholder/test credentials or private keys remain.
- Artifact versions and hashes match the approved offline release manifest.
- Failure/retry/recovery behavior is documented and tested.
- A second engineer/operator reviewed the final production adaptation.

Appendix H - PostgreSQL and pgvector Schemas

Operational projection, AI decision, case, approval, model registry, and vector knowledge tables.

```
CREATE TABLE wallet_projection (
  wallet_id text PRIMARY KEY,
  owner_ref text NOT NULL,
  asset text NOT NULL,
  status text NOT NULL,
  available_minor bigint NOT NULL,
  reserved_minor bigint NOT NULL,
  ledger_tx_id text NOT NULL,
  updated_at timestamptz NOT NULL DEFAULT now()
);

CREATE TABLE ai_decision (
  decision_id uuid PRIMARY KEY,
  request_id text NOT NULL,
  model_version text NOT NULL,
  feature_version text NOT NULL,
  risk_score double precision,
  recommendation text NOT NULL,
  reason_codes jsonb NOT NULL,
  created_at timestamptz NOT NULL DEFAULT now()
);

CREATE TABLE approval (
  approval_id uuid PRIMARY KEY,
  request_id text NOT NULL,
  action text NOT NULL,
  approver_user_id text NOT NULL,
  status text NOT NULL,
  expires_at timestamptz,
  evidence_hash text
);

CREATE EXTENSION IF NOT EXISTS vector;
CREATE TABLE knowledge_chunk (
  chunk_id text PRIMARY KEY,
  source_id text NOT NULL,
  access_role text NOT NULL,
  content text NOT NULL,
  embedding vector(768) NOT NULL
);
```

Implementation Notes

Adapt all example hostnames, ports, roles, schemas, limits, certificate paths, model dimensions, and version pins to the controlled environment. Replace placeholders with values from the production inventory. Security-sensitive configuration should be independently reviewed and tested on an offline staging network before production.

Validation Checklist

- Examples match the real organization/role names and source-of-truth matrix.
- No placeholder/test credentials or private keys remain.
- Artifact versions and hashes match the approved offline release manifest.
- Failure/retry/recovery behavior is documented and tested.
- A second engineer/operator reviewed the final production adaptation.

Appendix I - Fraud Feature and Evaluation Template

Feature dictionary, dataset lineage, threshold worksheet, and release-gate metrics.

Feature dictionary columns:

feature_name | type | source | window | null_policy | pii_class | version | notes

Candidate release report:

dataset_hash
label_definition
training_code_commit
model_sha256
feature_version
time_split_definition
precision_at_review_threshold
recall_at_block_threshold
false_positive_rate
calibration_error
p95_inference_ms
shadow_override_rate
approval_owner
rollback_model_version

Implementation Notes

Adapt all example hostnames, ports, roles, schemas, limits, certificate paths, model dimensions, and version pins to the controlled environment. Replace placeholders with values from the production inventory. Security-sensitive configuration should be independently reviewed and tested on an offline staging network before production.

Validation Checklist

- Examples match the real organization/role names and source-of-truth matrix.
- No placeholder/test credentials or private keys remain.
- Artifact versions and hashes match the approved offline release manifest.
- Failure/retry/recovery behavior is documented and tested.
- A second engineer/operator reviewed the final production adaptation.

Appendix J - Security and Production Readiness Checklist

Go-live controls covering network, keys, authorization, AI, backups, operations, and evidence.

- WAN disconnected during final acceptance test; all payment/wallet/AI fallback workflows still behave as designed.
- WalletOrg, MerchantOrg, AuditOrg, and OrdererOrg keys/admin responsibilities are documented and separated.
- All financial chaincode paths enforce balanced journals, integer money, state validation, authorization, and idempotency.
- Treasury issuance/redemption and high-risk overrides use MFA/dual approval according to policy.
- No PAN, CVV, passwords, private keys, or unnecessary PII are stored on-chain.
- Risk model/prompt/feature versions are pinned, hashed, evaluated, and rollback-tested.
- AI has no unrestricted financial write/signing tool and core payment operation has a tested AI-unavailable fallback.
- Three-node Raft quorum failure behavior has been tested.
- Projection database rebuild/reconciliation has been tested from authoritative ledger evidence.
- Backups were restored on isolated hardware and RTO/RPO evidence was recorded.
- Certificate/key revocation and rotation were successfully rehearsed.
- Incident kill switches, write-disable routes, wallet freeze procedures, and evidence preservation are documented.
- Release bundle contains exact binary/container/package/model hashes and rollback artifacts.

Implementation Notes

Adapt all example hostnames, ports, roles, schemas, limits, certificate paths, model dimensions, and version pins to the controlled environment. Replace placeholders with values from the production inventory. Security-sensitive configuration should be independently reviewed and tested on an offline staging network before production.

Validation Checklist

- Examples match the real organization/role names and source-of-truth matrix.
- No placeholder/test credentials or private keys remain.
- Artifact versions and hashes match the approved offline release manifest.
- Failure/retry/recovery behavior is documented and tested.
- A second engineer/operator reviewed the final production adaptation.

Appendix K - Comprehensive Test and Failure Matrix

Functional, concurrency, quorum, database, AI, security, recovery, and UAT test catalog.

Item	Implementation Guidance
Accounting	Issue, redeem, transfer, merchant pay, hold, capture, release, refund, reversal; verify balanced journals.
Authorization	Wrong wallet, wrong merchant, wrong role, revoked device, expired approval, wrong MSP.
Idempotency	Repeat same request after success, timeout, API restart, projection outage, delayed commit.
Concurrency	Parallel spend of same balance, simultaneous capture/refund, competing settlement/adjustment.
Fabric faults	One orderer down, two orderers down, peer down, CouchDB restart, cert revocation.
Database faults	PostgreSQL unavailable, projection worker crash, corrupt/stale projection, rebuild.
AI faults	Risk timeout, invalid schema, unknown model hash, LLM unavailable, pgvector unavailable.
AI attacks	Prompt injection, poisoned RAG document, secret request, financial-tool request, malformed features.
Security	Replay, auth bypass, oversized body, forged merchant/amount, stolen device simulation, break-glass use.
Recovery	Full backup restore, key/cert rotation, model rollback, chaincode upgrade, DR rehearsal.
Performance	Target concurrency, p95 commit latency, risk p95 inference, LLM support load, storage growth.
UAT	Customer/merchant/treasury/reviewer/auditor workflows and pending/recovery UX.

Implementation Notes

Adapt all example hostnames, ports, roles, schemas, limits, certificate paths, model dimensions, and version pins to the controlled environment. Replace placeholders with values from the production inventory. Security-sensitive configuration should be independently reviewed and tested on an offline staging network before production.

Validation Checklist

- Examples match the real organization/role names and source-of-truth matrix.
- No placeholder/test credentials or private keys remain.
- Artifact versions and hashes match the approved offline release manifest.
- Failure/retry/recovery behavior is documented and tested.
- A second engineer/operator reviewed the final production adaptation.

Appendix L - Glossary and Official References

Core terms and official Fabric, llama.cpp, pgvector, ONNX Runtime, NIST, OWASP, and PCI references.

Core Glossary

Item	Implementation Guidance
Double-entry journal	A financial record where total debits and credits balance for an asset.
Wallet	A logical account holding available/reserved value for an owner and asset.
Peer	Fabric node that maintains ledger/state and participates in endorsement/validation.
Orderer	Fabric ordering-service node that establishes transaction/block order.
MSP	Fabric membership configuration defining trusted organization identities.
Chaincode	Deterministic Fabric smart-contract program implementing agreed financial rules.
Fabric Gateway	Peer service/client API that simplifies endorsement, submission, commit status, and events.
Raft	Crash-fault-tolerant consensus/orderer protocol used by Fabric ordering service.
ONNX Runtime	Local cross-platform inference runtime for approved machine-learning models.
pgvector	PostgreSQL extension for vector similarity search used here for local RAG.
RAG	Retrieval-augmented generation: retrieve approved evidence then ask an LLM to answer from it.
Idempotency	Repeating the same business request does not create a second financial effect.
Compensating entry	A new journal entry that corrects/reverses earlier economic effect without editing history.

Official Technical References

- Hyperledger Fabric documentation - <https://hyperledger-fabric.readthedocs.io/en/latest/>
- Fabric installation - <https://hyperledger-fabric.readthedocs.io/en/latest/install.html>
- Fabric Gateway - <https://hyperledger-fabric.readthedocs.io/en/latest/gateway.html>
- Fabric ordering service / Raft - https://hyperledger-fabric.readthedocs.io/en/latest/orderer/ordering_service.html
- Fabric HSM / PKCS11 - <https://hyperledger-fabric.readthedocs.io/en/latest/hsm.html>
- Fabric CA - <https://hyperledger-fabric-ca.readthedocs.io/>
- llama.cpp server - <https://github.com/ggml-org/llama.cpp/tree/master/tools/server>

- pgvector - <https://github.com/pgvector/pgvector>
- ONNX Runtime - <https://onnxruntime.ai/docs/>
- NIST SP 800-57 Part 1 Rev. 5 - Recommendation for Key Management
- NIST AI Risk Management Framework and Generative AI Profile - <https://www.nist.gov/itl/ai-risk-management-framework>
- OWASP Application Security Verification Standard - <https://owasp.org/www-project-application-security-verification-standard/>
- OWASP LLM Security Verification Standard - <https://owasp.org/www-project-llm-verification-standard/>
- PCI Security Standards Council - <https://www.pcisecuritystandards.org/> (only if cardholder data enters system scope)

Implementation Notes

Adapt all example hostnames, ports, roles, schemas, limits, certificate paths, model dimensions, and version pins to the controlled environment. Replace placeholders with values from the production inventory. Security-sensitive configuration should be independently reviewed and tested on an offline staging network before production.

Validation Checklist

- Examples match the real organization/role names and source-of-truth matrix.
- No placeholder/test credentials or private keys remain.
- Artifact versions and hashes match the approved offline release manifest.
- Failure/retry/recovery behavior is documented and tested.
- A second engineer/operator reviewed the final production adaptation.

Final Implementation Roadmap - 16 Weeks

Item	Implementation Guidance
Week 1	Business boundary, closed-loop value definition, double-entry accounting, trust domains, ADRs.
Week 2	LAN topology, VLAN/firewall, local DNS/time/TLS, offline artifact/model bundle.
Week 3	Fabric CAs/MSPs, Wallet/Merchant/Audit/Orderer identities, three-node Raft.
Week 4	Peers/CouchDB, payments channel, endorsement policies, Fabric Gateway, failure tests.
Week 5	Wallet/journal/issuance/redemption/transfer chaincode and accounting invariant tests.
Week 6	Merchant payment, hold/capture/release/refund/freeze/idempotency/concurrency chaincode.
Week 7	Local authentication, device authorization, key/approval model, Gateway REST API.
Week 8	PostgreSQL projections, event worker, reconciliation, wallet/POS UI, QR payments.
Week 9	Fraud rules, feature definitions, dataset/evaluation baseline, shadow decision logging.
Week 10	Train/export approved risk model, ONNX Runtime service, threshold/fallback integration.
Week 11	llama.cpp, embeddings, pgvector, policy RAG, support/dispute/ops assistants.
Week 12	AI security, prompt injection tests, human review, model/prompt registry and rollback.
Week 13	Network/app/key/HSM/security hardening, access review, incident kill switches.
Week 14	Full fault/concurrency/security/AI/RAG/performance/UAAT test campaign.
Week 15	Backup/restore, certificate rotation, model rollback, chaincode/app release rehearsal.
Week 16	Production rehearsal, go-live controls, reconciliation baseline, operator training, controlled launch.

Definition of done: The platform is complete only when the Internet/WAN can be disconnected, authorized users can perform supported wallet/payment workflows, unauthorized or duplicate actions cannot move value, all financial journals balance, AI can fail without corrupting accounting, human-review paths work, ledger and projections reconcile, backups restore, certificates/models can roll back

or rotate, and trained operators can recover the system from written runbooks.

Final Production Evidence Package

- Architecture diagrams, ADRs, source-of-truth and trust-domain matrices.
- Approved Fabric configuration, channel policies, chaincode package/source hashes and lifecycle records.
- Accounting invariant specification and reconciliation reports.
- API schemas, authorization/approval policy, device/key inventory, and secret/key lifecycle records.
- Risk feature/model registry, evaluation reports, shadow/enforcement decisions, and rollback artifacts.
- LLM/RAG model/prompt/corpus/schema hashes plus groundedness/security evaluation results.
- Functional/concurrency/failure/security/recovery/UAT test evidence with request/tx references.
- Backup/restore, certificate rotation, model rollback, incident, startup/shutdown, and offline update runbooks.
- Go-live checklist, baseline metrics, operator training sign-off, and business owner acceptance.